Dinesh Priyankara, Robert C. Cain

SQL Server 2016 Reporting Services Cookbook

Create interactive cross-platform reports and dashboards using SQL Server 2016 Reporting Services



Packt>

SQL Server 2016 Reporting Services Cookbook

Create interactive cross-platform reports and dashboards using SQL Server 2016 Reporting Services

Dinesh Priyankara

Robert C. Cain



BIRMINGHAM - MUMBAI

SQL Server 2016 Reporting Services Cookbook

Copyright © 2016 Packt Publishing

All rights reserved. No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, without the prior written permission of the publisher, except in the case of brief quotations embedded in critical articles or reviews.

Every effort has been made in the preparation of this book to ensure the accuracy of the information presented. However, the information contained in this book is sold without warranty, either express or implied. Neither the authors, nor Packt Publishing, and its dealers and distributors will be held liable for any damages caused or alleged to be caused directly or indirectly by this book.

Packt Publishing has endeavored to provide trademark information about all of the companies and products mentioned in this book by the appropriate use of capitals. However, Packt Publishing cannot guarantee the accuracy of this information.

First published: November 2016

Production reference: 1211116

Published by Packt Publishing Ltd.
Livery Place
35 Livery Street
Birmingham
B3 2PB, UK.
ISBN 978-1-78646-181-0

www.packtpub.com

Credits

Authors

Dinesh Priyankara

Robert C. Cain

Reviewers

Gogula Aryalingam

Dave Wentzel

Commissioning Editor

Amey Varangaonkar

Acquisition Editor

Vinay Argekar

Content Development Editor

Amrita Noronha

Technical Editor

Akash Patel

Copy Editor

Vikrant Phadkay

Project Coordinator

Shweta H Birwatkar

Proofreader

Safis Editing

Indexer

Mariammal Chettiyar

Graphics

Disha Haria

Production Coordinator

Arvindkumar Gupta

About the Authors

Dinesh Priyankara is an experienced professional and database enthusiast with skills in database management systems and business intelligence, especially on the Microsoft SQL Server product suite. Possessing over 16 years of experience on data related technologies, he does training, consulting, and is a top contributor to the local SQL Server community. In recognition of his contributions and expertise he has demonstrated, he was awarded the Microsoft Most Valuable Professional (MVP) award for SQL Server for 11 consecutive years since 2006.

Dinesh is the Founder/Principal Architect of dinesQL (Pvt) Ltd (`http://dinesql.com`), a company that provides services on database, business intelligence, and training. In his spare time, he runs a blog at `http://dinesql.blogspot.com` with a follower base of over 100,000 from around the world and is a co-organizer of SQL Server Sri Lanka User Group (`http://www.meetup.com/ssslug/`).

Even though he has written many articles on the database area, this is his first official book as a co-author.

I would like to especially thank my daughter, Minupama Dulnethmi, who always encouraged me to complete my book with her fullest support. And I would also like to thank all of my family members and friends who helped on this in many ways.

My sincere appreciation to the entire team at Packt Publishing, with special thanks to Amrita Noronha and Vinay Argekar.

Robert C. Cain (`http://arcanecode.com`) is a Microsoft MVP, MCTS Certified in BI, and is the owner of Arcane Training and Consulting, LLC. He is also a course author for Pluralsight, with over 11 courses to his credit. Robert is a team mate at Linchpin People, and co-author of four books.

A popular speaker, Robert has presented at events such as the SQL PASS Summit, IT/Dev Connections, TechEd, CodeStock, and numerous SQL Saturdays. Robert has over 25 years' experience in the IT industry, working in a variety of fields including manufacturing, insurance, telecommunications and nuclear power.

I would like to thank my wife and daughters for their patience as I worked on this book. I also wish to thank my coauthor, Dinesh. Without his help, this book would not have been possible. My editor, Amrita, needs special thanks; this book would not be what it is without her patience and guidance.

I would also like to thank two long-time mentors. The first is Adam Curry. From his humble beginnings as a radio personality and MTV VJ, Adam became a pioneer in the early days of the Internet. In addition, his implementation of the podcasting platform opened a wealth of knowledge to both myself and many others.

I would also be remiss not to acknowledge John C. Dvorak. From his early columns in PC Magazine to his various TV shows and podcasts, John provided a wealth of information that guided my career.

Finally, I would like to thank the pair of them for their work on The No Agenda Show (`http://noagendashow.com`). I spent many late nights working on the demos for this volume listening to their entertaining and informative podcast.

About the Reviewers

Gogula Aryalingam is a database and business intelligence professional with more than 11 years of experience in the field. He is a Microsoft MVP for 9 years on the data platform and SQL Server, and PASS Regional Mentor for South Asia. His passion is in building business intelligence solutions, writing about the things he works with and talking about them. He is actively involved with the technical community; he co-runs the SQL Server user group in Colombo, Sri Lanka, speaks at local and international technical events and blogs his experiences. Gogula possesses an MSc and is a Microsoft Certified Solutions Expert, while also having played a part in developing exams for Microsoft certifications.

Dave Wentzel is a Data Solutions Architect at Microsoft with the Azure Enablement Team. He helps customers with their Digital Transformation, focused on all things data science, big data, and SQL Server. He knows how to converge big, fast, and unstructured data with traditional relational data to create actionable intelligence. He takes his customer experiences and feedback back to the product groups at Microsoft to make better solutions. Dave has been working with SQL Server for, let's just say, a long time, and with SSRS since its infancy. Dave shares his experiences at `http://davewentzel.com/`. He's always looking for new customers. Would you like to engage?

www.Packtpub.com

For support files and downloads related to your book, please visit `www.PacktPub.com`.

Did you know that Packt offers eBook versions of every book published, with PDF and ePub files available? You can upgrade to the eBook version at `www.PacktPub.com` and as a print book customer, you are entitled to a discount on the eBook copy. Get in touch with us at `service@packtpub.com` for more details.

At `www.PacktPub.com`, you can also read a collection of free technical articles, sign up for a range of free newsletters and receive exclusive discounts and offers on Packt books and eBooks.



`https://www.packtpub.com/mapt`

Get the most in-demand software skills with Mapt. Mapt gives you full access to all Packt books and video courses, as well as industry-leading tools to help you plan your personal development and advance your career.

Why subscribe?

- Fully searchable across every book published by Packt
- Copy and paste, print, and bookmark content
- On demand and accessible via a web browser

Table of Contents

Preface 1

Chapter 1: Getting It Ready – Configuring Reporting Services 9

- **Introduction** 9
- **Selecting the edition** 11
  - Getting ready 11
  - How to do it… 12
  - How it works… 12
  - See also 13
- **Installing Reporting Services with default configurations** 13
  - Getting ready 13
  - How to do it… 14
  - How it works… 18
  - There's more… 19
  - See also 19
- **Installing Reporting Services-related tools** 20
  - Getting ready 20
  - How to do it… 21
  - How it works… 22
  - There's more… 22
  - See also 23
- **Accessing Reporting Services Configuration Manager** 23
  - Getting ready 24
  - How to do it… 25
  - How it works… 27
  - There's more… 27
  - See also 27
- **Configuring the service account of Reporting Services** 28
  - Getting ready 28
  - How to do it… 31
  - How it works… 33
  - There's more… 34
- **Configuring Reporting Services URLs** 34
  - Getting ready 35
  - How to do it… 37

How it works… 43
There's more… 44
Configuring the Reporting Services database 44
Getting ready 45
How to do it… 46
How it works… 50
There's more… 52
See also 53
Executing post-installation tasks 53
Getting ready 53
How to do it… 53
How it works… 54
There's more… 54
Chapter 2: Authoring Reports with SQL Server Data Tools 55
Introduction 55
The designer tool – exploring SSDT 56
Getting ready 56
How to do it… 57
How it works… 62
Visual Studio 2015 support 62
Getting ready 63
How to do it… 63
How it works… 64
Creating reports with a table data region 64
Getting ready… 64
How to do it… 65
How it works… 87
There's more… 90
Creating reports with a matrix data region 94
Getting ready 94
How to do it… 95
How it works… 99
Creating reports with a chart data region 99
Getting ready 100
How to do it… 100
How it works… 116
There's more…. 117
Managing and configuring parameters 118
Getting ready 118

How to do it… 119
How it works… 126
Chapter 3: Advanced Report Authoring with SQL Server Data Tools 128
Introduction 128
Creating reports with multiple axis charts 129
Getting ready 129
How to do it… 130
How it works… 135
Creating reports with multiple data regions 136
Getting ready 136
How to do it… 137
How it works… 144
Creating reports with nested data regions 145
Getting ready 145
How to do it… 146
How it works… 150
Creating reports with a map data region 151
Getting ready 151
How to do it… 152
How it works… 160
Creating reports with cascading parameters 162
Getting ready 162
How to do it…. 163
How it works… 164
Creating reports with dynamic datasets 164
Getting ready 164
How to do it… 165
How it works… 167
How to publish reports 170
Getting ready 170
How to do it… 173
How it works… 178
How to view the XML code related to reports 178
Getting ready 178
How to do it… 179
How it works… 181
Chapter 4: Authoring Reports with Report Builder 182
Introduction 182

Is this for a developer or business user? 183
Getting ready… 183
How to do it… 183
How it works… 184
Enhancements to Report Builder 185
Getting ready… 185
How to do it… 185
How it works… 186
Connecting Report Builder to Reporting Services 187
Getting ready 187
How to do it… 187
How it works… 188
How to design reports with the wizard 188
Getting ready 189
How to do it… 189
How it works… 192
There's more… 194
How to design reports manually 194
Getting ready… 194
How to do it… 195
How it works… 197
Working with published data sources 197
Getting ready 197
How to do it… 198
How it works… 200
Working with published datasets 201
Getting ready 201
How to do it… 201
How it works… 205
How to publish reports to Reporting Services 205
Getting ready 205
How to do it… 205
How it works… 207
Working with published report parts 208
Getting ready 208
How to do it… 209
How it works… 213
Changing reports published in Reporting Services 213
Getting ready 213

How to do it… 214
How it works… 215
Chapter 5: Improving User Experience – New Designing and Visualization Enhancements 216
Introduction 216
Positioning parameters 218
Getting ready 219
How to do it… 220
How it works… 222
Visualizing data with modern styles 222
Getting ready 223
How to do it… 224
How it works… 225
There's more… 225
Visualizing data with the TreeMap data region 225
Getting ready 226
How to do it… 227
How it works… 236
Visualizing data with the Sunburst data region 236
Getting ready 238
How to do it… 239
How it works… 244
There's more… 245
Integrating R with Reporting Services 246
Getting ready 247
How to do it… 248
How it works… 255
There's more… 255
See also 255
Chapter 6: Authoring Reports with the Mobile Report Publisher 256
Introduction 257
Understanding the new Mobile Report Publisher 258
Getting ready 258
How to do it… 259
How it works… 261
How to create a mobile report 262
Getting ready 262
How to do it… 262

How it works… 264
Using simulated data 264
Getting ready 264
How to do it… 265
How it works… 267
Setting up data sources 267
Getting ready 268
How to do it…. 268
How it works… 270
How to use datasets for mobile reporting 271
Getting ready 271
How to do it… 271
How it works… 280
Understanding mobile report graphical elements 280
Getting ready 280
How to do it… 281
How it works… 284
Understanding the mobile report layout 285
Getting ready 285
How to do it… 285
How it works… 290
Configuring navigators 290
Getting ready 290
How to do it… 291
How it works… 294
Configuring parameters for drill-through reports 295
Getting ready 296
How to do it… 296
How it works… 301
Chapter 7: Consuming Reports – Report Access Enhancement 302
Introduction 303
Exploring the new Web Portal 304
Getting ready 305
How to do it… 306
How it works… 311
There's more… 312
Managing reports with the Web Portal 312
Getting ready 312
How to do it… 313

How it works… 317
There's more… 317
Viewing reports on any web browser 318
Getting ready 318
How to do it… 318
How it works… 321
There's more… 322
Caching the report 322
Getting ready 323
How to do it… 324
How it works… 327
There's more… 329
Refreshing the cache 330
Getting ready 330
How to do it… 331
How it works… 336
There's more… 337
Creating a KPI with predefined values 338
Getting ready 338
How to do it… 339
How it works… 341
There's more… 342
Creating a KPI with shared datasets 342
Getting ready 342
How to do it… 343
How it works… 348
There's more… 349
Customizing the look-and-feel of the Web Portal 350
Getting ready 351
How to do it… 351
How it works… 355
There's more… 356
Printing reports with the PDF rendering extension 356
Getting ready 357
How to do it… 357
How it works… 362
There's more… 362
Embedding reports in other applications 362
Getting ready 363

How to do it… 364
How it works… 369
There's more… 372
See also 373
Chapter 8: Reporting Solutions for BI – Integration 374
Introduction 375
Making Reporting Services as a part of BI 376
Getting ready 376
How to do it… 377
How it works… 377
There's more… 378
See also 378
Creating a report using Multidimensional models 378
Getting ready 379
How to do it… 379
How it works… 385
There's more… 385
Getting formatting from the Multidimensional model 385
Getting ready 386
How to do it… 387
How it works… 391
There's more… 392
Getting aggregated values from the Multidimensional model 392
Getting ready 392
How to do it… 395
How it works… 396
There's more… 396
Creating a report using Tabular models 396
Getting ready 397
How to do it… 397
How it works… 401
Connecting to the HDInsight cluster and getting unstructured data 401
Getting ready 402
How to do it… 409
How it works… 417
There's more… 418
See also 418
Chapter 9: SharePoint Integration 419

Introduction 419
Configuring SharePoint for Reporting Services 420
Getting ready 420
How to do it… 420
How it works… 425
Handling security with SharePoint 426
Getting ready 427
How to do it… 427
Adding a user to your SharePoint site 427
Altering permissions to a document library 428
Altering permissions to a specific report item 430
Removing a user's permissions from a report 432
How it works… 432
There's more… 434
Creating dashboards using Reporting Services 434
Getting ready 434
How to do it… 435
How it works… 443
Configuring e-mail with SharePoint 444
Getting ready 444
How to do it… 445
How it works… 446
There's more… 446
Configuring subscriptions 446
Getting ready 447
How to do it… 447
How it works… 451
Configuring data alerts 451
Getting ready 451
How to do it… 452
How it works… 454
Chapter 10: Administering and Managing Reporting Services 455
Introduction 456
Managing data sources 456
Getting ready 457
How to do it… 458
How it works… 462
There's more… 462
Assigning configured data sources to published reports 462

Getting ready 462
How to do it… 465
How it works… 470
There's more… 470
Caching reports 470
Getting ready 471
How to do it… 475
How it works… 479
There's more… 480
Creating snapshots of reports 481
Getting ready 481
How to do it… 481
How it works… 486
Backing up Reporting Services 487
Getting ready 487
How to do it… 488
How it works… 494
There's more… 495
Monitoring Reporting Services 495
Getting ready 495
How to do it… 498
How it works… 501
There's more… 501
Chapter 11: Securing Reports in Reporting Services 502
Introduction 502
Authenticating users for Reporting Services 503
Getting ready 503
How to do it… 503
How it works… 504
Becoming a Report Portal System Administrator 505
Getting ready 505
How to do it… 505
How it works… 507
Authorizing users for accessing reports via SSRS roles 508
Getting ready 508
How to do it… 508
How it works… 510
There's more… 511
Creating new roles in Reporting Services 511

Getting ready 511
How to do it… 512
How it works… 513
Editing existing roles in Reporting Services 513
Getting ready 513
How to do it… 513
How it works… 514
Filtering data for the current user 515
Getting ready 515
How to do it… 516
How it works… 518
There's more… 519
Handling security on shared data sources 520
Getting ready 520
How to do it… 520
How it works… 522
There's more… 523
Securing the channel using certificates 523
Getting ready 524
How to do it… 527
How it works… 530
Chapter 12: Custom Programming and Integration to .NET Applications 531
Introduction 532
Adding embedded codes to reports 532
Getting ready 533
How to do it… 535
How it works… 541
There's more… 542
Referencing external .NET assemblies 542
Getting ready 542
How to do it… 546
How it works… 552
Opening reports using URL access 552
Getting ready 552
How to do it… 555
How it works… 559
There's more… 559
Using ReportViewer control for accessing reports 559
Getting ready 560

How to do it… 560
How it works… 566
There's more… 566
Index 567

Preface

SQL Server 2016 Reporting Services is truly revolutionary. SQL Server Reporting Services saw few enhancements since the SQL Server 2008R2 release. With SQL Server 2016, all of that has changed.

Many longstanding complaints of users have been addressed. Reports now render in HTML 5, allowing compatibility across a multitude of platforms. The old Report Manager has received a major overhaul, and with that a new name, the Report Portal.

The Report Portal provides many new capabilities. It will, of course, host the traditional SSRS reports (now renamed Paginated Reports). Now, though, it can host a variety of other items, such as KPIs and PowerBI Desktop reports.

An exciting new addition is the Mobile Report Publisher, allowing you to create reports that can easily be viewed across computers, tablets, and phones.

The installation and configuration has been enhanced and streamlined to make setting up a new instance of Reporting Services even easier. Management tools have been updated as well, providing an easier to use experience for administrators.

In this book, we'll explore all the new, exciting enhancements that come with SQL Server 2016 Reporting Services. The recipes have been designed to get you up to speed quickly, allowing you to jump right in to all the great features included in SSRS 2016.

What this book covers

`Chapter 1`, *Getting It Ready – Configuring Reporting Services*, will show you how to plan for and install Reporting Services. We'll see how to configure SSRS for Native mode, as well as gain an understanding of backend databases.

`Chapter 2`, *Authoring Reports with SQL Server Data Tools,* will begin your authoring of reports by working with SQL Server Data Tools (SSDT). You'll see how to create a variety of reports using tables, matrixes, as well as charts. Finally, you'll see how to publish reports to the SSRS server from within SSDT.

`Chapter 3`, *Advanced Report Authoring with SQL Server Data Tools,* will continue your education on SSDT by moving into some advanced report topics. You'll learn how to use multiple datasets in the same report, and create charts and tables with multiple axis. You'll also get a view of SSRS behind the scenes, looking at the XML code used within an SSRS report file.

`Chapter 4`, *Authoring Reports with Report Builder,* will focus on the newly revamped Report Builder tool. You'll see how an end user could leverage this tool to create a variety of objects. Not just reports, but data sources and data sets.

`Chapter 5`, *Improving User Experience – New Designing and Visualization Element*s, will focus on the new visualization and design elements introduced with SSRS. You'll see the new Sunburst and Treemap charts, as well as learning how to integrate the R language into your reports.

`Chapter 6`, *Authoring Reports with the Mobile Report Publisher*, explains that Microsoft addresses the need for reporting on mobile devices, such as phones and tablets, through the addition of the Mobile Report Publisher. In this chapter, you'll learn how to use this brand-new tool to create reports customized for a variety of platforms.

`Chapter 7`, *Consuming Reports – Report Access Enhancement*, will explore the new Report Portal, the redesigned Report Manager. You'll see how to manage reports and data caching. You'll also see how to create the new KPIs, as well how to customize the portal for your environment.

`Chapter 8`, *Reporting Solutions for BI – Integratio*n, shows that Reporting Services is not the only tool in a BI developer's tool box. In this chapter, you'll see how to integrate other tools, such as PowerBI and SQL Server Analysis Services, into your SSRS reports.

`Chapter 9`, *SharePoint Integration,* explains that, in addition to the Report Portal, SharePoint can also act as a host for your SSRS reports. In this chapter, you'll see how to configure SharePoint and SSRS to work together.

`Chapter 10`, *Administering and Managing Reporting Services,* reveals that as good as Reporting Services is just out of the box, understanding how to manage key components will make the experience even better. In this chapter, you'll learn how to manage data sources, history, caching, scheduling, and other key topics that will create a better experience for your users.

`Chapter 11`, *Securing Reports in Reporting Services,* highlights that it is critical to protect your data from unauthorized eyes. In this chapter, you'll learn how to secure your server and its reports. In addition, you'll find recipes explaining how to secure the data within your reports.

`Chapter 12`, *Custom Programming and Integration to .NET Applications*, explains that .NET developers often have need to integrate reporting into their application. In this chapter, we'll see the various techniques developers can use leverage the power of SSRS within their programs.

What you need for this book

To work through the recipes in this book, you will, of course, need a copy of SQL Server 2016. If you don't have one, Microsoft has now released the SQL Server 2016 Developer Edition as a free download, and it will work just fine for working through the recipes.

You can download the developer edition at:

`https://www.microsoft.com/en-us/sql-server/sql-server-editions-developers`

or use the shortcut:

`http://bit.ly/sql2016dev`

In addition, you will want a copy of the brand-new SQL Server 2016 sample database, WideWorldImporters. Specifically, the data warehouse version of the sample, WideWorldImportersDW, was used for the examples in this book. You can find the samples on GitHub, at:

`https://github.com/Microsoft/sql-server-samples/tree/master/samples/databases/wide-world-importers`

or use the shortcut:

`http://bit.ly/wwisample`

You also need AdventureWorks 2014 multi-dimensional and tabular databases since they were used with some examples. You can find the samples database at: `https://msftdbprodsamples.codeplex.com/releases/view/125550`

Who this book is for

A wide variety of users will find this book valuable. Naturally, IT Professionals whose chief duties are the development of reports will find these recipes useful.

In today's world, many end users are also developing their own reports, using tools such as Report Builder and the Mobile Report Publisher. They will find the recipes quite useful as they continue to develop their own reports.

Additionally, system administrators who are responsible for installation, security, and maintaining Reporting Services will find valuable information herein to guide them as the manage SSRS.

Finally, we've included a whole chapter aimed at .NET developers, showing how to integrate SSRS directly into their applications.

Sections

In this book, you will find several headings that appear frequently (Getting ready, How to do it, How it works, There's more, and See also).

To give clear instructions on how to complete a recipe, we use these sections as follows:

Getting ready

This section tells you what to expect in the recipe, and describes how to set up any software or any preliminary settings required for the recipe.

How to do it…

This section contains the steps required to follow the recipe.

How it works…

This section usually consists of a detailed explanation of what happened in the previous section.

There's more…

This section consists of additional information about the recipe in order to make the reader more knowledgeable about the recipe.

See also

This section provides helpful links to other useful information for the recipe.

Conventions

In this book, you will find a number of styles of text that distinguish between different kinds of information. Here are some examples of these styles, and an explanation of their meaning.

Code words in text, database table names, folder names, filenames, file extensions, path names, dummy URLs, user input, and Twitter handles are shown as follows: "We can include other contexts through the use of the `include` directive. "

A block of code is set as follows:

```
[default]
exten => s,1,Dial(Zap/1|30)
exten => s,2,Voicemail(u100)
exten => s,102,Voicemail(b100)
exten => i,1,Voicemail(s0)
```

When we wish to draw your attention to a particular part of a code block, the relevant lines or items are set in bold:

```
[default]
exten => s,1,Dial(Zap/1|30)
exten => s,2,Voicemail(u100)
exten => s,102,Voicemail(b100)
exten => i,1,Voicemail(s0)
```

Any command-line input or output is written as follows:

```
# cp /usr/src/asterisk-addons/configs/cdr_mysql.conf.sample
     /etc/asterisk/cdr_mysql.conf
```

New terms and **important words** are shown in bold. Words that you see on the screen, in menus or dialog boxes for example, appear in the text like this: "clicking the **Next** button moves you to the next screen".



Warnings or important notes appear in a box like this.



Tips and tricks appear like this.

Reader feedback

Feedback from our readers is always welcome. Let us know what you think about this book—what you liked or may have disliked. Reader feedback is important for us to develop titles that you really get the most out of.

To send us general feedback, simply send an e-mail to `feedback@packtpub.com`, and mention the book title via the subject of your message.

If there is a topic that you have expertise in and you are interested in either writing or contributing to a book, see our author guide on `https://www.packtpub.com/books/info/packt/authors`.

Customer support

Now that you are the proud owner of a Packt book, we have a number of things to help you to get the most from your purchase.

Downloading the example code

You can download the example code files for all Packt books you have purchased from your account at `http://www.packtpub.com`. If you purchased this book elsewhere, you can visit `http://www.packtpub.com/support`and register to have the files e-mailed directly to you.

You can download the code files by following these steps:

1. Log in or register to our website using your e-mail address and password.
2. Hover the mouse pointer on the **SUPPORT** tab at the top.
3. Click on **Code Downloads & Errata**.
4. Enter the name of the book in the **Search** box.
5. Select the book for which you're looking to download the code files.
6. Choose from the drop-down menu where you purchased this book from.
7. Click on **Code Download**.

Once the file is downloaded, please make sure that you unzip or extract the folder using the latest version of:

- WinRAR / 7-Zip for Windows
- Zipeg / iZip / UnRarX for Mac
- 7-Zip / PeaZip for Linux

The code bundle for the book is also hosted on GitHub at `https://github.com/PacktPublishing/SQL-Server-2016-Reporting-Services-Cookbook`. We also have other code bundles from our rich catalog of books and videos available at `https://github.com/PacktPublishing/`. Check them out!

If you are using **Tableau Public**, you'll need to locate the workbooks that have been published to Tableau Public. These may be found at the following link: `http://goo.gl/wJzfDO`.

Downloading the color images of this book

We also provide you a PDF file that has color images of the screenshots/diagrams used in this book. The color images will help you better understand the changes in the output. You can download this file from: `https://www.packtpub.com/sites/default/files/downloads/SQLServer2016ReportingServicesCookbook_ColorImages.pdf`.

Errata

Although we have taken every care to ensure the accuracy of our content, mistakes do happen. If you find a mistake in one of our books—maybe a mistake in the text or the code—we would be grateful if you would report this to us. By doing so, you can save other readers from frustration and help us improve subsequent versions of this book. If you find any errata, please report them by visiting `http://www.packtpub.com/submit-errata`, selecting your book, clicking on the errata submission form link, and entering the details of your errata. Once your errata are verified, your submission will be accepted and the errata will be uploaded on our website, or added to any list of existing errata, under the Errata section of that title. Any existing errata can be viewed by selecting your title from `http://www.packtpub.com/support`.

Piracy

Piracy of copyright material on the Internet is an ongoing problem across all media. At Packt, we take the protection of our copyright and licenses very seriously. If you come across any illegal copies of our works, in any form, on the Internet, please provide us with the location address or website name immediately so that we can pursue a remedy.

Please contact us at `copyright@packtpub.com` with a link to the suspected pirated material.

We appreciate your help in protecting our authors, and our ability to bring you valuable content.

Questions

You can contact us at `questions@packtpub.com` if you are having a problem with any aspect of the book, and we will do our best to address it.

1
Getting It Ready – Configuring Reporting Services

In this chapter, we will cover the following recipes related to installation and configuration of Microsoft SQL Server Reporting Services:

- Selecting the edition
- Installing Reporting Services with default configurations
- Installing Reporting Services-related tools
- Accessing Reporting Services Configuration Manager
- Configuring the service account of Reporting Services
- Configuring Reporting Services URLs
- Configuring the Reporting Services database
- Executing post-installation tasks

Introduction

Stepping back to the year 2004, the release of Reporting Services as a post-release add-on to Microsoft SQL Server 2000 was a big in with the market as there were not many user-friendly reporting tools out there. Microsoft makes it easy and user-friendly; this worldwide-accepted catchphrase was expected by developers for the same and Microsoft kept the word again. Although it initially focused more on IT-driven reports, it became the most wanted and in-demand reporting tool for report designers and consumers.

Rosetta, the code name used for Reporting Services, was a perfect match. The Rosetta Stone found in 1799 explained the mystery of Egyptian hieroglyphics because it provided information needed to understand their history. Reporting Services does the same; it allows you to have the mystery or reasons for the success of your business revealed.

The first release of Reporting Services did not provide a wide range of data regions for addressing all Enterprise requirements, but it offered enough. However, the key was its architecture. It was designed to run under Internet Information Services as a middle-tier service, making it a server-based and web-based reporting platform. Reports designed with Reporting Services were able to be sourced by any type that has .NET managed provider, OLE DB provider, or ODBC provider. Since most Reporting Services processors and services were distributed, obtaining and processing data was efficient and extensible. The rich architecture used was supported with modern rendering extensions and delivered reports in HTML, PDF, and Excel. In addition to that, it allowed us to have our own extensions, making sure that reports were delivered as per the requirements and printing devices used.

Along with other Reporting Services releases, it was significantly improved and enhanced, making it the best reporting platform in the world. Before Microsoft SQL Server 2016, there were four more releases, with SQL Server 2005, 2008 R2, 2012, and 2014. Among all of these fantastic releases, 2005, 2008 R2, and 2016 are considered major releases of Reporting Services.

The latest release, Reporting Services 2016, is becoming the foremost reporting platform because of the features added, enhancements done on existing features, and the completeness in terms of modern reporting and BI platform. Since there were no major enhancements after 2008 R2, we thought that Reporting Services was about to retire, but the 2016 release showed us that it is still a part of the Microsoft BI ecosystem.

This chapter focuses on the installation and configuration of Reporting Services 2016. The recipes in this chapter explain typical installation and configurations as well as further and infrequent configurations that go beyond the traditional installation and configurations. You will be guided through the installation using recipes, and we will explain which sections are important, why they are important, and how they should be configured.

Selecting the edition

The edition of SQL Server 2016 should be selected based on your needs. SQL Server 2016 offers five downloadable editions, targeting specific business use cases for different prices and levels of capabilities. These five editions are Enterprise, Standard, Web, Developer, and Express. Generally, for an organization, you select one of the premium editions (which is Enterprise) or one of the core editions (which is Standard). There is another premium edition called **Analytics Platform System** (**APS**), formerly **Parallel Data Warehouse** (**PDW**); that will not be discussed in this book as it is beyond the scope. For an individual, it is always the Developer edition that has all functionality of the Enterprise edition. It can be downloaded and installed without paying Microsoft. Yes, it is free for developers and is licensed for use as a development or test system, not as a production server.

SQL Server 2012 and 2014 had an edition called **Business Intelligence**, which is no longer available with SQL Server 2016. You can try out Reporting Services with Enterprise Edition without purchasing, as it is available for a 180-day trial period. If you need to use the cheapest edition, which is Express, you need to have Express with Advanced Services.

Getting ready

Features supported by editions are different. For understanding, without discussing all features, we will see the features that are supported only by Enterprise edition.

The following features are supported only by Enterprise edition:

- Mobile reports and KPIs
- Data-driven report subscription
- Scale out deployment
- Alerting
- Power view

Reporting Services maintain two databases: ReportServer database and ReportServerTempDB. It does not necessarily require the same edition for databases as you have used for Reporting Services. The following list shows you which editions of the Database engine can be used for specific editions of Reporting Services:

- **Enterprise edition of SSRS**: Enterprise or Standard edition database engine
- **Standard edition of SSRS**: Enterprise or Standard edition database engine
- **Web edition of SSRS**: Web edition database engine (local only)
- **Express edition of SSRS**: Express edition database engine (local only)
- **Developer edition of SSRS**: Enterprise, Standard, or Developer edition database engine

How to do it…

1. Select the edition based on the capabilities required for your application. If you need the aforementioned items that are supported only by the Enterprise edition, then you have no option but select the Enterprise edition. If they are not required, then the Standard edition is enough for your implementation. Cost and limited budget are some other factors for selecting the Standard edition.
2. Select the Developer edition for development or testing, not as a production server.
3. Try using Web or Express, which have very limited features. These two editions are good for small-scale implementations.

How it works…

You will be able to create, deploy, and consume reports with functionalities as per the features offered by the selected edition. Not only that, but the usage of processors of the system and memory consumption are based on the edition you have selected. Processors are more relevant to database engine. The Enterprise edition will use operating system maximum, but Standard and Web can utilize only four sockets or 24 cores. If you have selected, Express edition, it is limited to one socket or four cores.

Reporting Services consume memory for various operations. If the edition is Enterprise, it can consume operating system maximum. However, if the edition is Standard or Web, it consumes only 64 GB and, Express edition consumes only 1 GB.

See also

For more information about feature comparison between editions, visit the following URL: `https://msdn.microsoft.com/en-us/library/cc645993.aspx`

Installing Reporting Services with default configurations

Installing Reporting Services is not a complex process. It is a part of SQL Server installation; just like selecting any other component, Reporting Services can be selected as a required component. Reporting Services 2016 offers two modes: Native and SharePoint integrated. SharePoint integrated mode will be discussed in detail in `Chapter 9`, *SharePoint Integration*. Hence, this recipe focuses on Native mode.

Native mode is the default Reporting Services mode. You can install Reporting Services by either launching the wizard or using the command line.

Getting ready

It is always better to check and see whether your system is ready for Reporting Services before installing it. This check-up can be done using a tool given with the installation, which is called **System Configuration Checker**.

You need to make sure that you have admin rights on the system that you intend to run the installation on. For that, you need to be a member of a local administrator group, or you need to have credentials of an account that has admin rights.

During the installation, you can decide whether the installation wizard should install files and configure Reporting Services with default configurations, or only install files. If you select Install only, then you need to do the configuration later using Reporting Services Configuration Manager. Selecting the Install and configure option allows you to use Reporting Services immediately after setup is finished. However, this option will not be available if Database Engine Services is not selected as part of the instance features or the database engine is not installed on the same instance.

How to do it…

Let's go through the specific steps and options you select during SQL Server installation when you do the installation for Reporting Services. Note that we will not go through every page in the installation wizard, but only the pages related to this recipe. The steps are as follows:

1. Once the installation wizard is launched by executing `setup.exe`, it opens **SQL Server Installation Center**, which has multiple tabs. The default tab is opened, which is **Planning**, and the detail page has the link for **System Configuration Checker**. The following screenshot shows this:

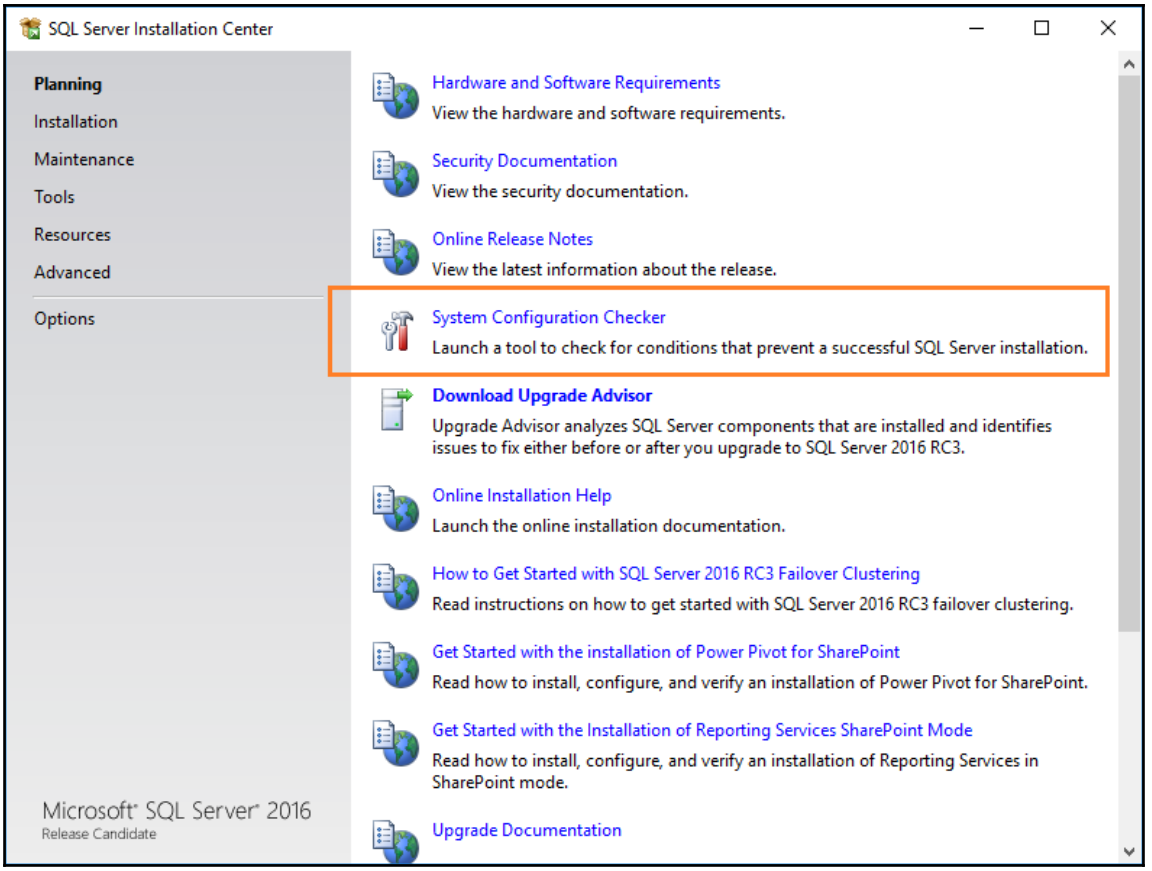


Figure 1.01

2. When you click on **System Configuration Checker**, the tool goes through a set of predefined rules and tells you whether the system is ready for installation of SQL Server or not. The following screenshot shows the result of it:

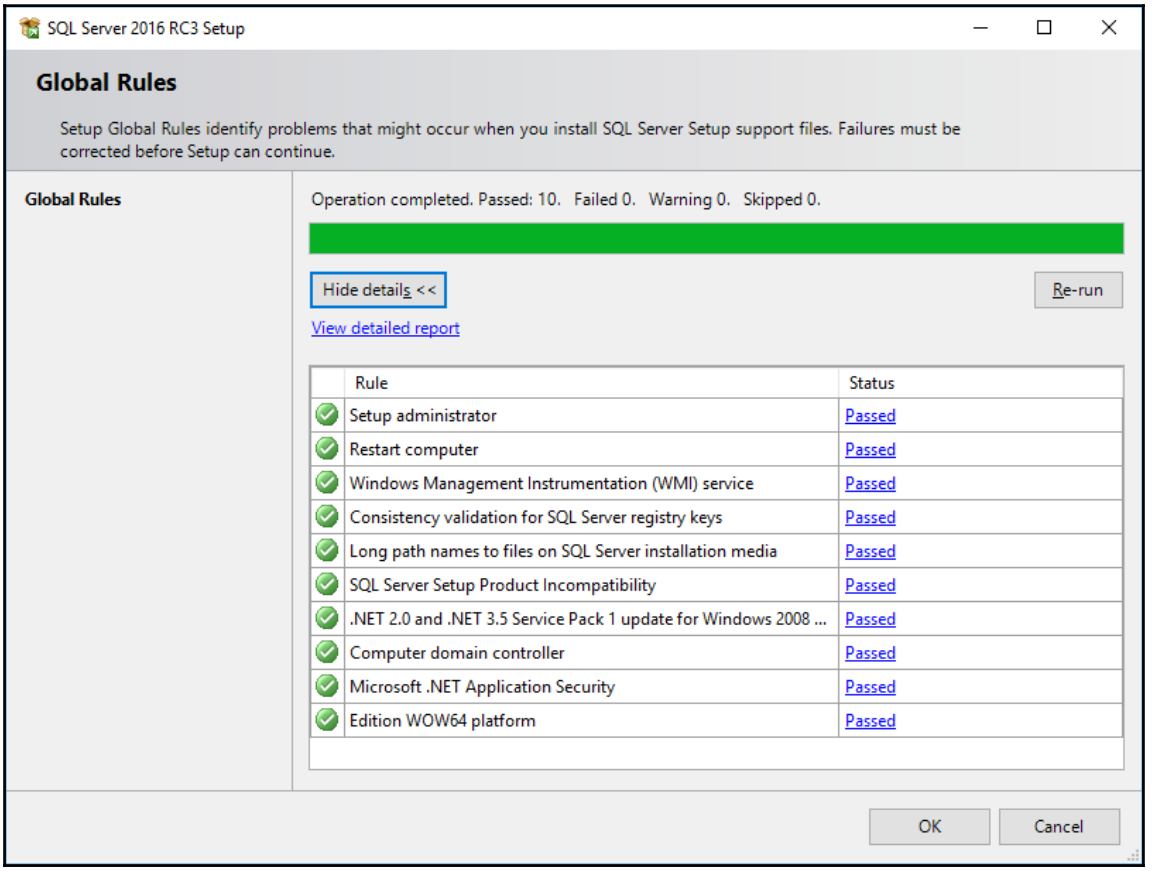


Figure 1.02

3. If everything is okay, continue with the installation. When you see the **Feature Selection** page, make sure **Database Engine Services** and **Reporting Services – Native** are selected:

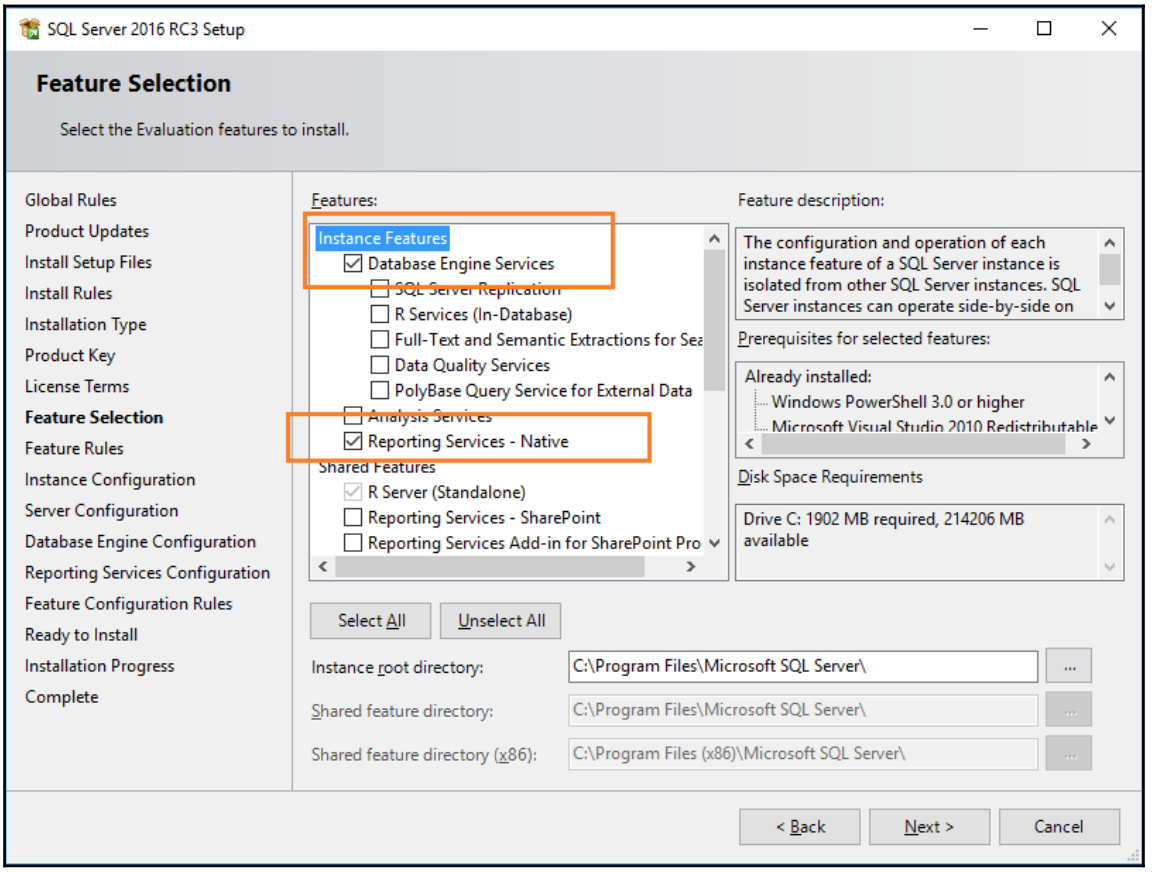


Figure 1.03

4. When you see **Reporting Services Configuration**, select the **Install and configure** option to install Reporting Services with the default configurations:

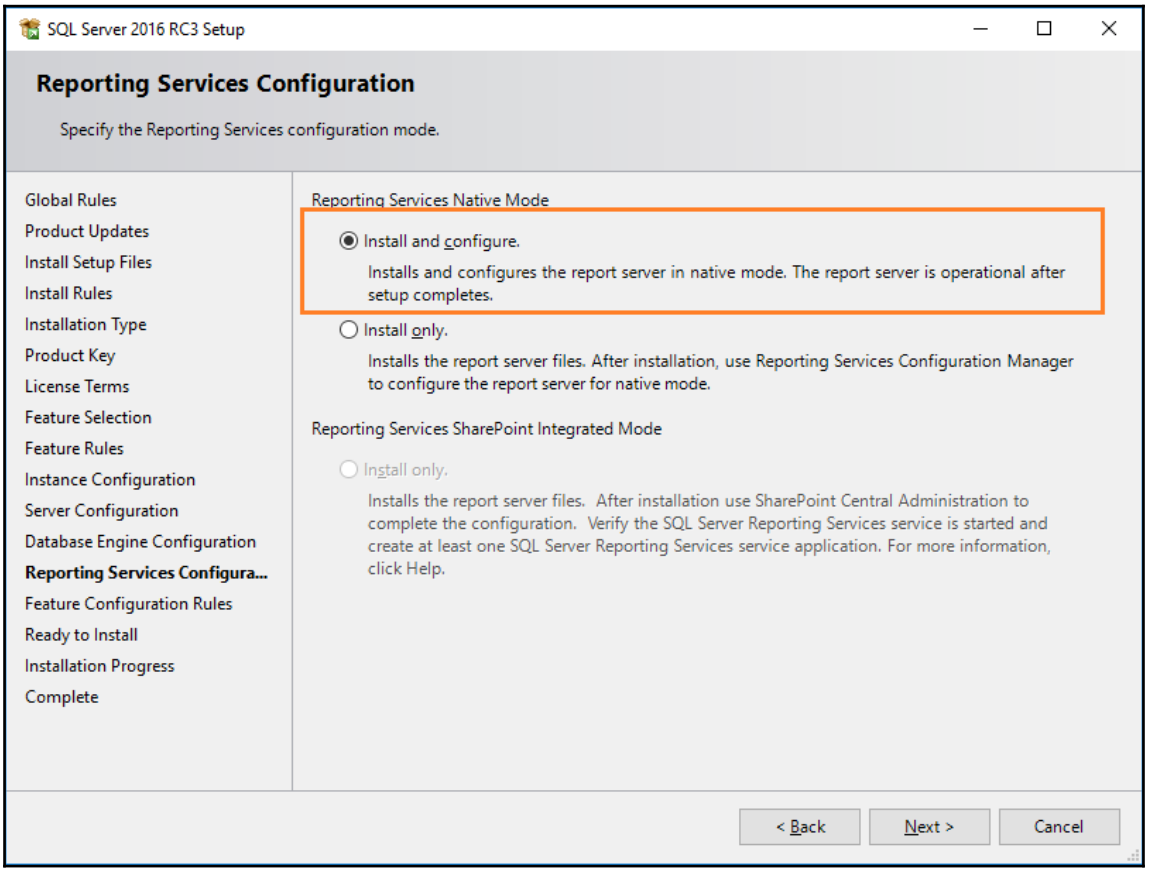


Figure 1.04

Now you can continue with the other pages and complete the installation.

How it works…

Since the installation is done with the **Install and configure** option, it installs the following items/features for us:

- Report Server service:
    - Report Server Web Service
    - Report Server Web Portal
    - Other background processing applications
- Reporting Services Configuration Manager
- Reporting Services command-line utilities

In addition to that, it configures following items too:

- Service account for the Report Server service
- Report Server Web Service URL
- The Web Portal URL
- Report Server databases
- Service accounts for accessing Report Server databases

To verify the installation, you can access the Web Portal and see whether it works or not. The URL for the Web Portal is different based on the installation:

- With a default instance: `http://MachineName:80/Reports`
- With a named instance: `http://MachineName:80/Reports_InstanceName`

This image shows the Web Portal configured with a named instance:

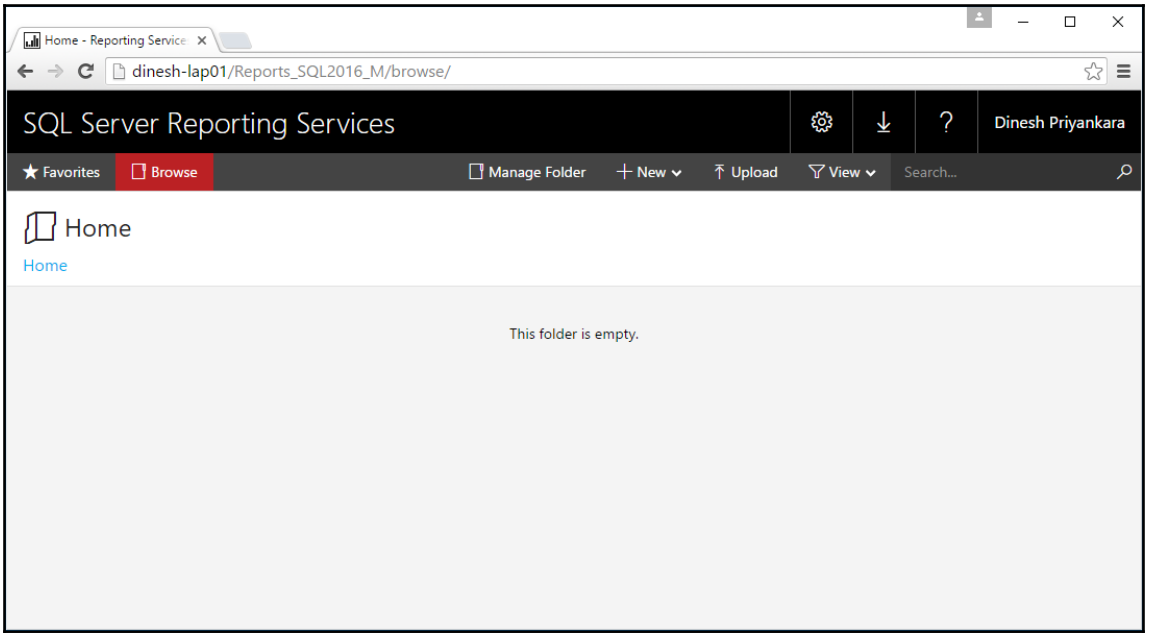


Figure 1.05

There's more…

If you open Reporting Services Configuration Manager (this will be discussed in detail in the *Accessing Reporting Services Configuration Manager* recipe), you will see that some of the services related to Reporting Services are not configured. Unattended Execution Account, Email Setting, Encryption Keys, Subscription Settings, Scale-out Deployment, and Power BI integration are not configured as part of the installation, and they need to be configured manually using Reporting Services Configuration Manager.

See also

The next recipe, *Installing Reporting Services-related tools*, discusses the other required installations for completing a Reporting Services installation.

Installing Reporting Services-related tools

Installing only Reporting Services does not make your environment ready for working with Reporting Services. In order to author reports, you need Report Designer, which provides an interface for designing reports with various data regions. Report Designer ships with **SQL Server Data Tools** (**SSDT**), which was previously called **Business Intelligence Development Studio** (**BIDS**) with SQL Server 2005, **SQL Server Data Tools** (**SSDT**) with SQL Server 2012, and **Microsoft SQL Server Data Tools – Business Intelligence for Visual Studio** (**SSDTBI**) with SQL Server 2014. SSDT is based on Visual Studio and it provides not only Report Designer for building reporting solutions, but also many other interfaces for working with relational databases, Azure SQL databases, SSIS, and SSAS solutions.

Until SQL Server 2014, this tool was automatically installed with SQL Server, but now you need to download it separately and install it.

The other most important tool that is required is **SQL Server Management Studio** (**SSMS**). This is not just for Reporting Services; it's an integrated environment for accessing, configuring, managing, administering, and developing all components of SQL Server. This was part of the default installation with previous versions of SQL Server, but now it needs to be downloaded separately and installed as it is a standalone tool.

Getting ready

You can find links to download SSDT and SSMS using any search engine. But the Microsoft SQL Server team has given us an easier way of finding the relevant download page. These links are given with the SQL Server installation wizard, and this makes sure that you download the latest versions of SSDT and SSMS.

How to do it…

Note that the SQL Server installation page has a number of installations related to SQL Server, but we focus only on the tools related to this book. Here is the way of installing the related tools:

1. Execute the SQL Server setup.exe to open **SQL Server Installation Center**. Click on the **Installation** tab; this shows the various types of installation that can be done with it:

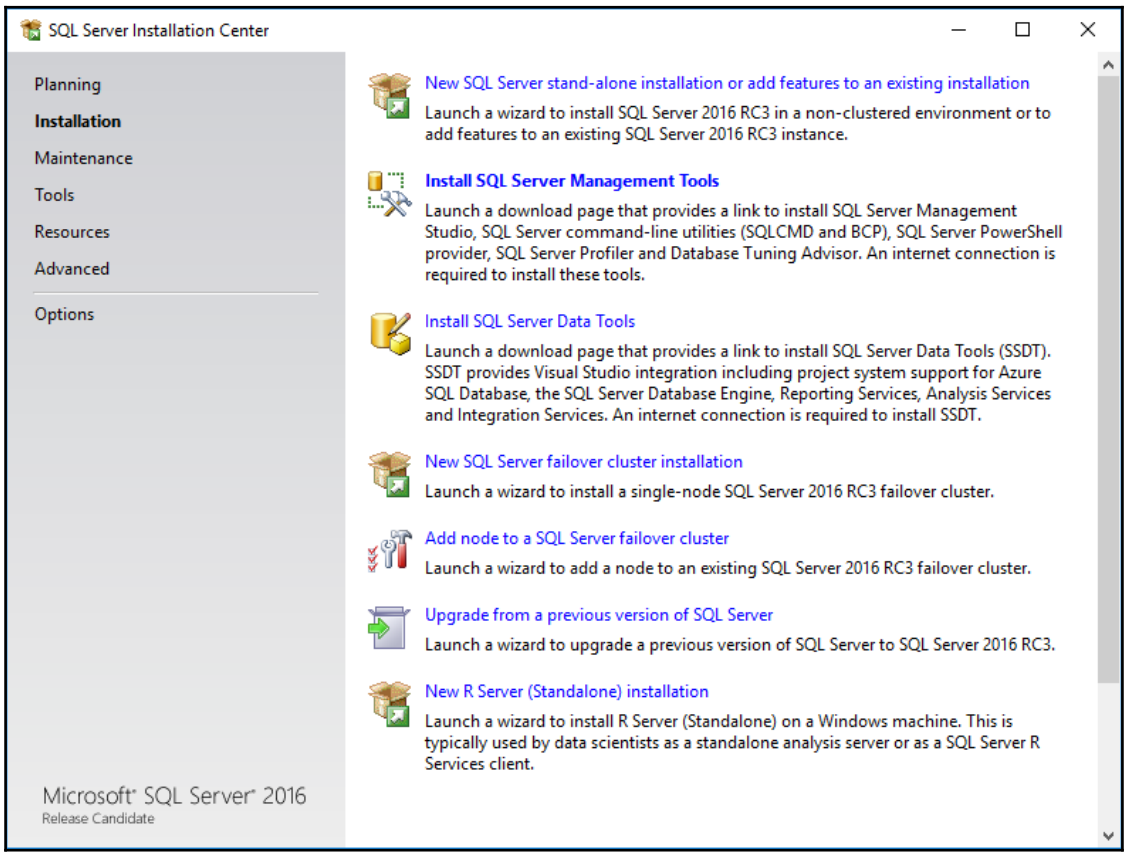


Figure 1.06

2. Click on **Install SQL Server Management Tools** to open the web page that provides links for downloading Management Studio. This page shows you the link for downloading the latest SSMS.



Note that at the time of writing this book, the version of SSMS is 13.0.15000.23, and it is June 2016 generally available release. This version of SSMS is free and does not require any license to install and use.

3. You need to click on **Install SQL Server Data Tools** to install SSDT.



Note that at the time of writing this book, the version of SSDT is 14.0.60525.0. Microsoft recommends that you install Visual Studio 2015 prior to this, and this update will replace SSDT RTM in Visual Studio 2015. If Visual Studio 2015 is not installed, SSDT will install Visual Studio Integrated Shell.

How it works…

Once both tools are installed, SSMS can be used to connect and manage Reporting Services instances and SSDT can be used to design and deploy reporting solutions. You can launch SSMS and connect with Reporting Services. We will talk more about this tool in `Chapter 10`, *Administering and Managing Reporting Services*.

You can create Reporting Services projects using SSDT. It allows you to create a new project, and select the Reporting Services template and Report Server Project type. You will see more on this in `Chapter 2`, *Authoring Reports with SQL Server Data Tools*.

There's more…

There is another useful tool called Report Builder, which is specifically available for power users. This does not mean that it cannot be used by developers, but as it is more of a Microsoft Office-like tool, it is more relevant to power users and business users. This will be discussed in detail in `Chapter 4`, *Authoring Reports with Report Builder*.

See also

A newly added tool called SQL Server Mobile Report Publisher is available with Reporting Services 2016. See `Chapter 6`, *Authoring Mobile Reports with the Mobile Report Publisher*, for more on it.

Accessing Reporting Services Configuration Manager

Reporting Services Configuration Manager is a tool that you can use to configure Reporting Services settings and related various components. It allows you to access the settings configured during the installation and modify them if required. In addition to that, you can use it to configure settings that are not available during installation.



Note that you cannot use Reporting Services Configuration Manager to manage SharePoint integrated report servers. Prior to SQL Server 2012, SharePoint integrated reporting services instances were able to manage with it, but starting with SQL Server 2012, SharePoint integrated mode is managed and configured by using SharePoint Central Administration.

Unlike SQL Server Management Studio, Reporting Services Configuration Manager is version-specific. You cannot use a Configuration Manager installed for a specific version in your machine to manage another instance of Reporting Services that is not the same version, even if it is a lower version. As an example, if you have installed Reporting Services 2016 Configuration Manager, you cannot use it to manage Reporting Services 2014.

Most of the settings configured through Configuration Manager are stored in the `rsreportserver.config` file. Even though it is possible to make changes to the file directly without using Configuration Manager, you should not do it unless it is really required. If you have to, make sure you have copy of the file before making any changes.

Getting ready

Reporting Services Configuration Manager is installed with the default installation. It allows you to manage the following items:

- **Report Server service account**: You can update the account that is already added or you can add a new account for the service
- **Web Service URL**: You can modify or create new URLs for accessing Report Server Web Service
- **Database**: You can create or configure the report server database for the Report Server. It also allows you to connect with a database that is already created and set with the required content
- **Web Portal URL**: You can modify or create new URLs for accessing the Reporting Services Web Portal
- **Email settings**: You can configure an SMTP server or gateway that Report Server can use to deliver reports to e-mail addresses
- **Execution account**: You can configure an unattended account that report server can use for accessing various sources on scheduled operations, or when user credentials are not given for operations
- **Encryption keys**: You can back up, restore, or change the symmetric key used to encrypt stored connections and credentials
- **Subscription settings**: You can configure an account for subscriptions that need to access file shares
- **Scale-out Deployment**: You can add multiple report servers to scale out your reporting solution
- **Power BI integration**: You can use this section to register a Power BI account if you plan to integrate your reports with Power BI

In order to access either a local report server or remote report server using Configuration Manager, you need local system administrator permission of the server.

How to do it…

Let's see how we can open Configuration Manager and see the configuration settings. The steps are as follows:

1. When you open the Configuration Manager, it prompts you with a dialog box for **Server Name** and **Report Server Instance**:

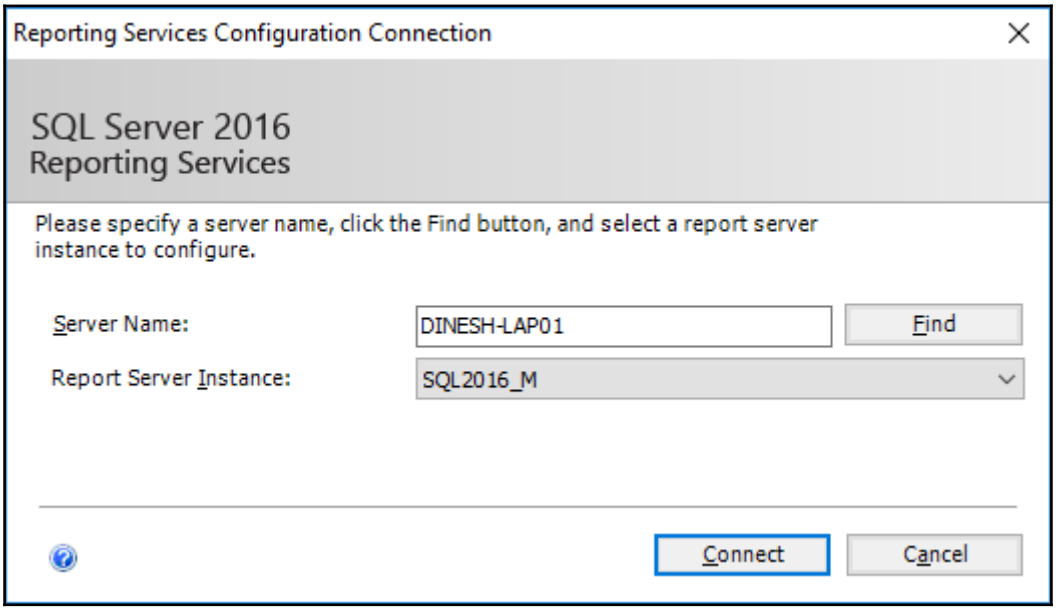


Figure 1.07

2. Enter the server name and select the **Report Server Instance** for connecting with the required report server. If you have provided valid values and you have local administrator permission, it will open Configuration Manager for you:

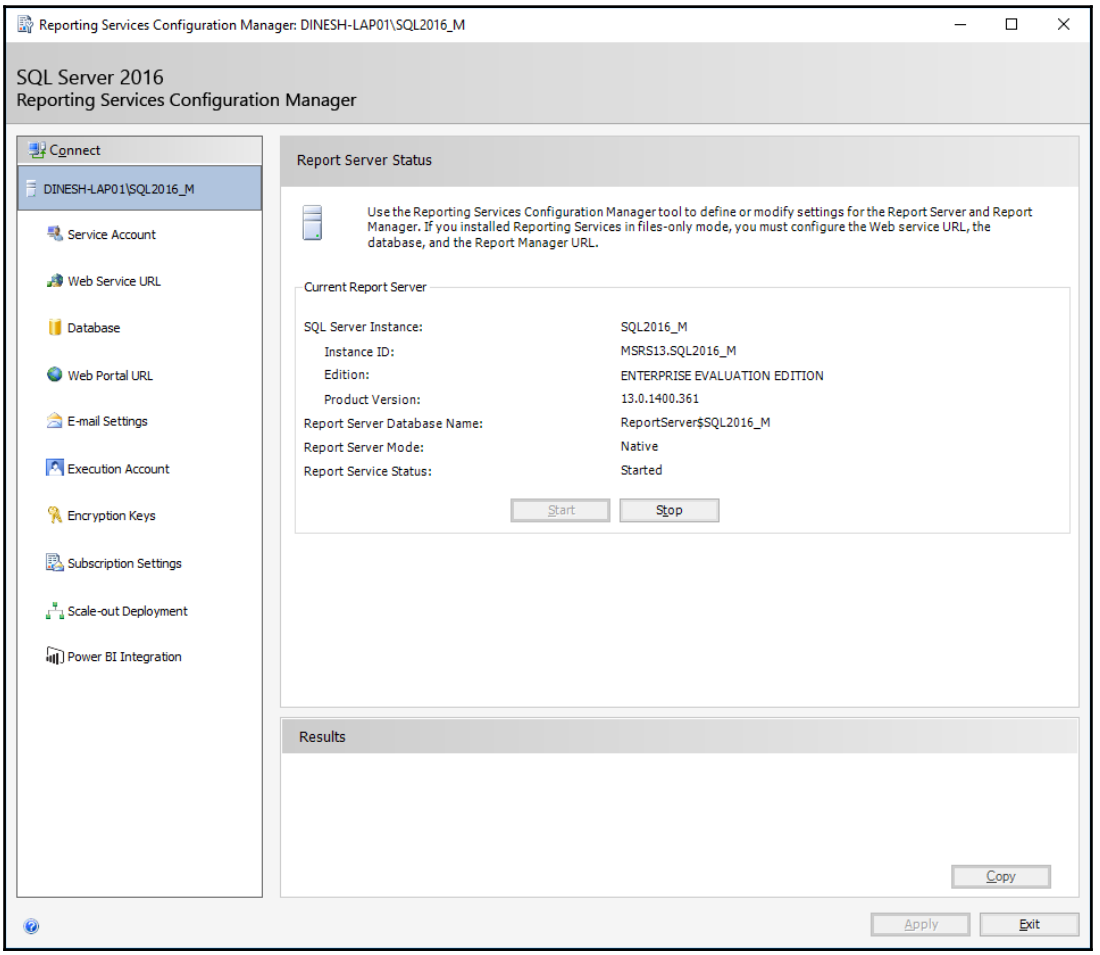


Figure 1.08

3. You can navigate through the tabs given in the left pane. Note that the **Apply** button is grayed out until you make a change. Once the changes are made, you need to click on the **Apply** button to save and apply your changes.

How it works…

The `rsreportserver.config` XML file holds most of the settings related to Configuration Manager. When you change any settings with Configuration Manager, the changes are applied to this file. You can see this file in `the <installationpath>eport\Microsoft SQL Server\<instance name>\Reporting Services\ReportServer` folder. You can open it and edit using any text editor or XML editor, but it is not recommended because your changes may prevent Report Server from functioning properly.

There's more…

There are some settings that are not available through Configuration Manager. For example, if you need to add another Authentication Type, it has to be manually added to the `rsreportserver.config` file, as Configuration Manager does not provide an interface for adding Authentication Types.

Some of the settings should only change with Configuration Manager even though it is possible to change them through `rsreportserver.config`. For example, encrypted values that are related to some nodes should not be changed through the `rsreportserver.config` file manually.

See also

In addition to the `rsreportserver.config` file, there are two more configuration files: `rsreportserverservices.exe.config` and `rsreportdesigner.config`. You can read more about them at `https://msdn.microsoft.com/en-us/library/ms157273.aspx` and `https://msdn.microsoft.com/en-us/library/bb630448.aspx`.

Configuring the service account of Reporting Services

The service account for Reporting Services is assigned during the installation, it is configured to run the Reporting Services service within the context of a specific Windows account. Installation allows you to select a local account, domain user account, or built-in account for the service. Once set, the installation makes sure that the account has the required permissions to access resources and run processes related to Reporting Services by adding the account to relevant security groups.

If you need to view or reconfigure the assigned account, Microsoft recommends that you use Reporting Services Configuration Manager without using Services Desktop Application. You might change the assigned account to a new account, or you might change the password set for the account if it has been changed at the Windows level. If you use Services Desktop Application to manage the service account, the additional settings that are linked with the assigned account will not be automatically changed. Specifically, it will not be added to relevant security groups. But Reporting Services Configuration Manager makes sure that all required settings are updated accordingly when a change is applied to the assigned account.

Getting ready

You can see the current assigned account using **Reporting Services Configuration Manager**. Once connected to the instance, you can click on the **Service Account** page to see the current account set or to set a new account for the service:

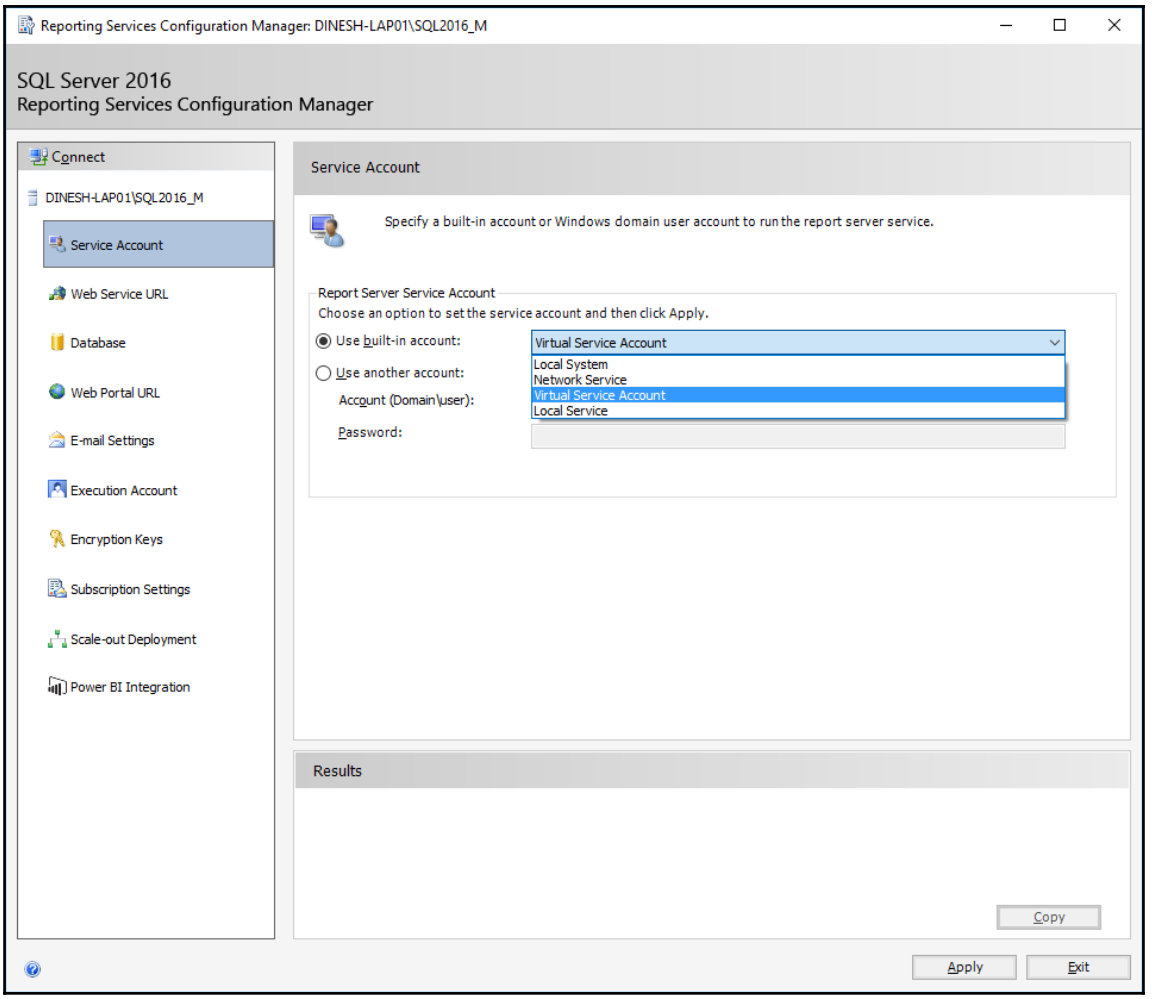


Figure 1.09

As you can see, there are two options for setting it. You can either select a built-in account or a user-defined (local or domain) account. If you have accepted the default during account selection, you should see the selected account as **Virtual Service Account**.

When selecting an account, either during installation or later using Configuration Manager, it is important to remember that the account you select has no excessive privileges. Not only that, but also you must make sure that it is a dedicated account for Reporting Services.

There are different types of account that you can select for Reporting Services:

- **Local system account**: This account is a highly privileged account that should not be selected (or required) for Reporting Services. Generally, we should avoid highly privileged accounts for SQL Server services.
- **Network service account**: This account has fewer privileges compared to the Local System account but has network log-on permissions. Therefore, it is not a bad practice to select this account as the Reporting Services service account. However, since this account is used by many applications, it might introduce a security threat, so it is better to avoid this account type as well.
- **Virtual service account**: Virtual Service account was introduced with Windows Server 2008 R2, and SQL Server started supporting it from SQL Server 2012. It is simply a virtualized account based on a Network Service account but with its unique identifier. It is a local account, and you do not need to create or configure it. For SQL Server services, virtual accounts are automatically created during installation. Since it is a managed account, you do not need to worry about policies or password expiry issues, and this account type is recommended if you do not have a domain user account configured.
- **Local service account**: This is a local account that is similar to an authenticated Windows user account in a local users group. This account accesses resources as a null session or without credentials. Therefore, this will not work as expected when report server needs to access external resources such as a remote database.
- **Local user account**: There is a local, least-privileged Windows account configured for Reporting Services. This is recommended when Reporting Services is installed in a non-domain environment.
- **Domain user account**: A least-privileged Windows domain account configured for Reporting Services. This is recommended when Reporting Services is installed in a domain environment.
- **Managed service account**: This account is similar to a Virtual Service account, but this is a type of domain account. It is more secure than a domain user account and, unlike the Virtual service account, it needs to be created and configured by a domain administrator before it can be used.

How to do it…

In order to manage the assigned account, follow these steps:

1. Start the **Reporting Services Configuration Manager** as shown in the preceding image.
2. Make the necessary changes to the Report Server service account and click on **Apply**. You may change the account from **Virtual service** to **Network service** account for testing.
3. The **Apply** button opens the **SQL Server Connection Dialog** window. It requires an account that has administrative privileges for applying grant rights script to a newly selected service account. If you have administrative permissions, select **Current User – Integrated Security**, or supply SQL Server credentials by selecting **SQL Server Account**:

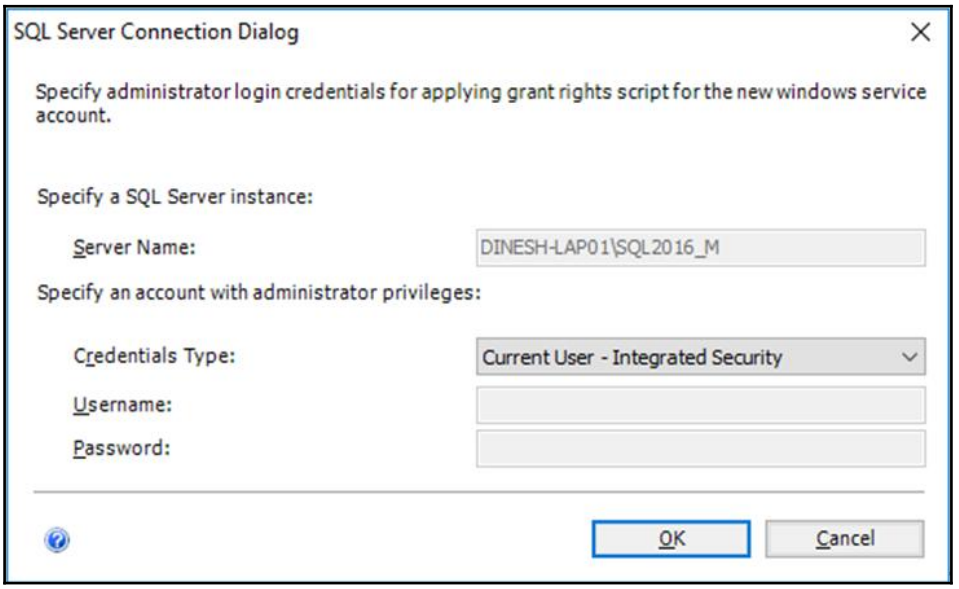


Figure 1.10

4. Click on **OK** to save the changes. Once completed, verify the messages in the results shown in the next image and see whether everything is successfully completed:

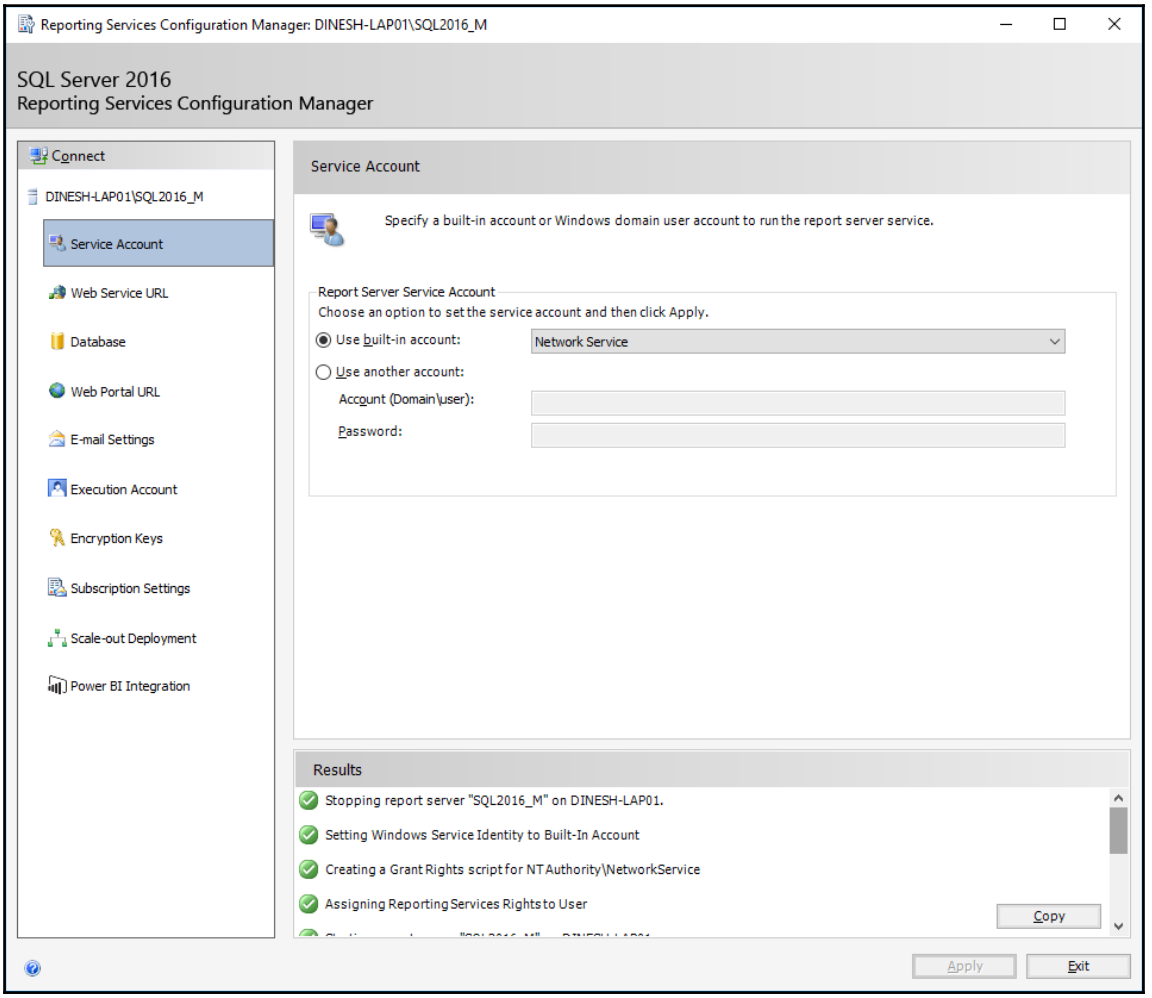


Figure 1.11

How it works…

Reporting Services Configuration Manager performs a few operations to apply your changes successfully:

- A newly configured account is added to the report server group created on the local server.
- The account is added to the necessary security roles in the SQL Server database engine instance that hosts the report server database, only if the connection configured to the report server database is set to service account. This account is not just a part of report server database; it gets added to the Master, Msdb, ReportServer, and ReportServerTempDB databases under different roles:
    - **Master database**: This account is added to RSExecRole
    - **Msdb database**: This account is added to RSExecRole, SQLAgentOperatorRole, SQLAgentReaderRole, and SQLAgentUserRole
    - **ReportServer database**: This account is added to db_owner and RSExecRole
    - **ReportServerTempDB database**: This account is added to db_owner and RSExecRole
- The encryption key is updated, including the profile information of the account.

There's more…

Note that if you add a new service account to Reporting Services, the old account will not be removed from the database instance. See the next screenshot. It shows both accounts under the **Users** node; **NT AUTHORITY\NETWORK SERVICE** is the new account and **NT SERVICE\ReportServer$SQL2016_M** is the old account. Therefore, it should be manually removed if it is not required by the database engine:

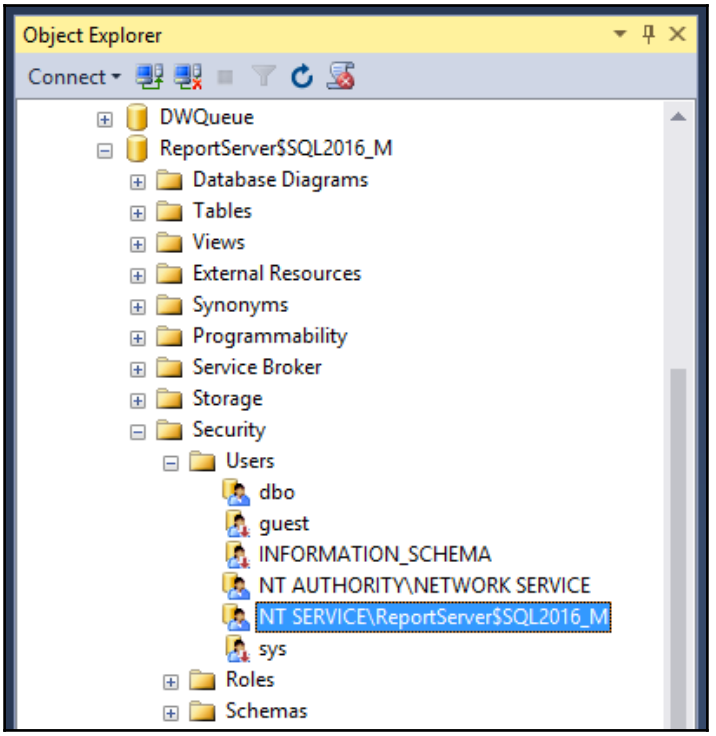


Figure 1.12

Configuring Reporting Services URLs

As you saw in the *Installing Reporting Services with default configurations* recipe in this chapter, once the installation is done, one way of verifying that the installation is successfully completed is by accessing one of the URLs that are configured automatically during the setup. If the installation is done with **install only** mode, then the URLs have to be configured using Reporting Services Configuration Manager, because only the **install and configure** option creates URLs for Reporting Services during the installation.

Installation with the default configuration reserves two URLs for Reporting Services, one for the **Web Service** application, and another for the **Web Portal** application. These URLs are defined as URL reservations in `HTTP.SYS`, which is an operating system component that listens to network requests and handles them. Even though it is possible to create and configure these URLs using different tools, it is recommended to use only Reporting Services Configuration Manager for creating and configuration. URL reservations are automatically created when you configure URLs using Reporting Services Configuration Manager.

URL reservations in HTTP.SYS define which URLs are configured for accessing Reporting Services applications. You can configure one or more URLs for accessing the Report Server Web Service and one or more URLs for accessing the report Web Portal. Although multiple URLs can be configured for web service and web portal, all URLs have to be connected to the web service virtual directory and the web portal virtual directory respectively. It is not possible to have multiple virtual directories.



Note that **Internet Information Services** (**IIS**) is no longer used by Reporting Services to manage applications, and it is not required. All requests are completely handled by `HTTP.SYS`.

Getting ready

If you need to create or configure URLs using Reporting Services Configuration Manager, you need to understand how default URLs are created during the installation. This table explains how URLs are set by default:

Application	**Instance type**	**Default URL**	**Reservation in** `HTTP.SYS`
Web service	Default instance	`http://<servername} reportserver<="" p="">`	`http://<servername}:80 reportserver<="" p="">`
Web service	Named instance	`http://<servername} reportserver_<instancename}<="" p="">`	`http://<servername}:80 reportserver_<instancename}<="" p="">`
Web Portal	Default instance	`http://<servername} reports<="" p="">`	`http://<servername}:80 reports<="" p="">`
Web Portal	Named instance	`http://<servername} reports_<instancename}<="" p="">`	`http://<servername}:80 reports_<instancename}<="" p="">`

You can see this in the `rsreportserver.config` file located in the `<installationpath}\Microsoft SQL Server\<instancename}\Reporting Services\ReportServer` folder:

rsreportserver.config

```
<Add Key="DisableSecureFormsAuthenticationCookie" Value="false"/>
<Add Key="CleanupCycleMinutes" Value="10"/>
<Add Key="MaxActiveReqForOneUser" Value="20"/>
<Add Key="DatabaseQueryTimeout" Value="120"/>
<Add Key="RunningRequestsScavengerCycle" Value="60"/>
<Add Key="RunningRequestsDbCycle" Value="60"/>
<Add Key="RunningRequestsAge" Value="30"/>
<Add Key="MaxScheduleWait" Value="5"/>
<Add Key="DisplayErrorLink" Value="true"/>
<Add Key="WebServiceUseFileShareStorage" Value="false"/>
<!--  <Add Key="ProcessTimeout" Value="150" /> -->
<!--  <Add Key="ProcessTimeoutGcExtension" Value="30" /> -->
<!--  <Add Key="WatsonFlags" Value="0x0430" /> full dump-->
<!--  <Add Key="WatsonFlags" Value="0x0428" /> minidump -->
<!--  <Add Key="WatsonFlags" Value="0x0002" /> no dump-->
<Add Key="WatsonFlags" Value="0x0428"/>
<Add Key="WatsonDumpOnExceptions" Value="Microsoft.ReportingServices.Diagnostics.Utilities.InternalCatal
<Add Key="WatsonDumpExcludeIfContainsExceptions" Value="System.Threading.ThreadAbortException,System.Web
<URLReservations>
  <Application>
    <Name>ReportServerWebService</Name>
    <VirtualDirectory>ReportServer_SQL2016_M</VirtualDirectory>
    <URLs>
      <URL>
        <UrlString>http://+:80</UrlString>
        <AccountSid>S-1-5-80-3946330831-378249602-484118862-83638884-4098085075</AccountSid>
        <AccountName>NT SERVICE\ReportServer$SQL2016_M</AccountName>
      </URL>
    </URLs>
  </Application>
  <Application>
    <Name>ReportServerWebApp</Name>
    <VirtualDirectory>Reports_SQL2016_M</VirtualDirectory>
    <URLs>
      <URL>
        <UrlString>http://+:80</UrlString>
        <AccountSid>S-1-5-80-3946330831-378249602-484118862-83638884-4098085075</AccountSid>
        <AccountName>NT SERVICE\ReportServer$SQL2016_M</AccountName>
      </URL>
    </URLs>
  </Application>
</URLReservations>
```

Figure 1.13

As mentioned before, you should not use any other tool to create or configure URLs, including the `rsreportserver.config` file.

In order to create or configure them, you need to make sure that you are a member of local administrator group of the server. In addition to that, if IIS is running in the same server, make sure that there are no virtual directories already configured with the names you have planned for Reporting Services virtual directories for applications.

How to do it…

Reporting Services Configuration Manager has two pages for managing URLs, one for web service applications, and another for web portal applications. It is shown in the following screenshot:

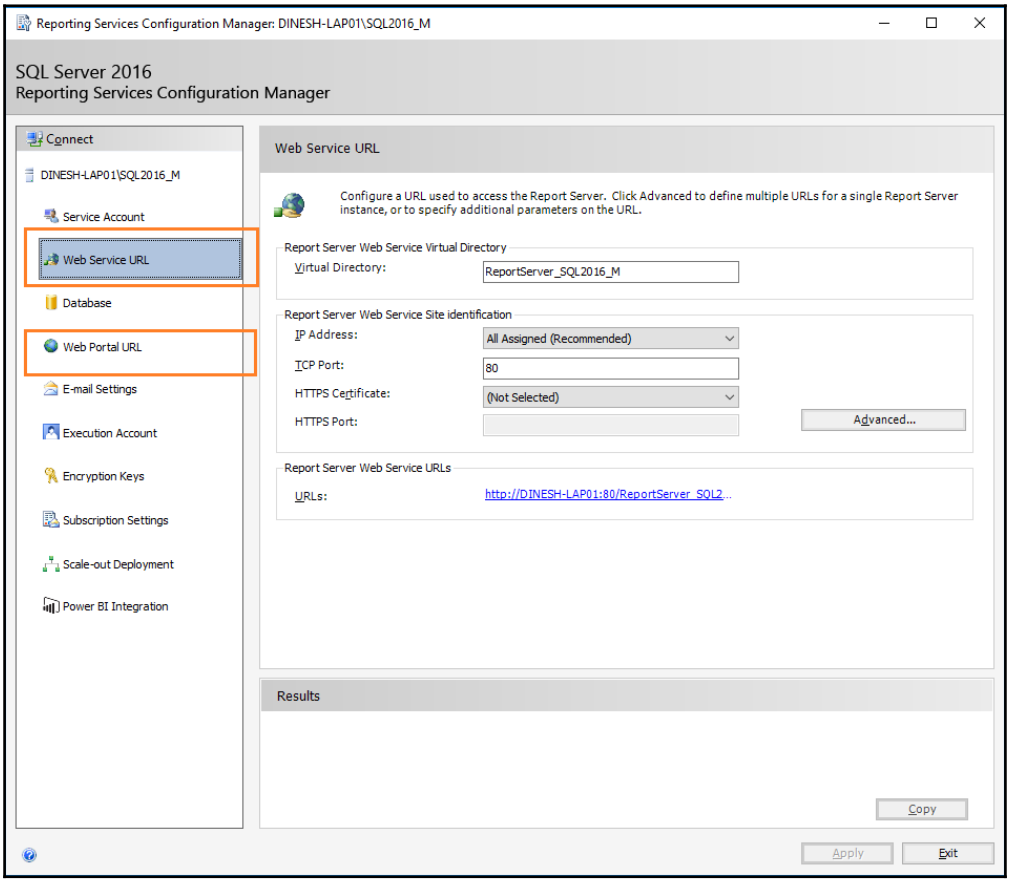


Figure 1.14

If you need to configure the **Web Service URL**, select the page and make the necessary changes:

1. If you need to configure the **Web Portal URL**, you can select it to view and configure it. The preceding screenshot shows the settings related to the Web Service URL. As you see, the URL is formed by combining the machine name and virtual directory name.
2. If you need to change the name of the virtual directory, you can simply change the text in the **Virtual Directory** input box. Remember that the name of the virtual directory is case sensitive, and only one virtual directory is allowed for the web service application.
3. If you want to configure multiple URLs for the web service application, you can click on **Advanced…** and add more URLs.
4. When you click on **Advanced…**, it opens a window that can be used to add both HTTP and HTTPS URLs. For testing purposes, let's allow users to connect with the web service using another port. Follow these steps to configure a URL with another port:
    1. Click on the **Advanced…** button to open the **Advanced Multiple Web Site Configuration** window.
    2. Click on **Add** in the HTTP section to add a new report server HTTP URL.
    3. Set the **TCP Port** as you want and click on **OK** to save it. Click on **OK** again to save the settings:

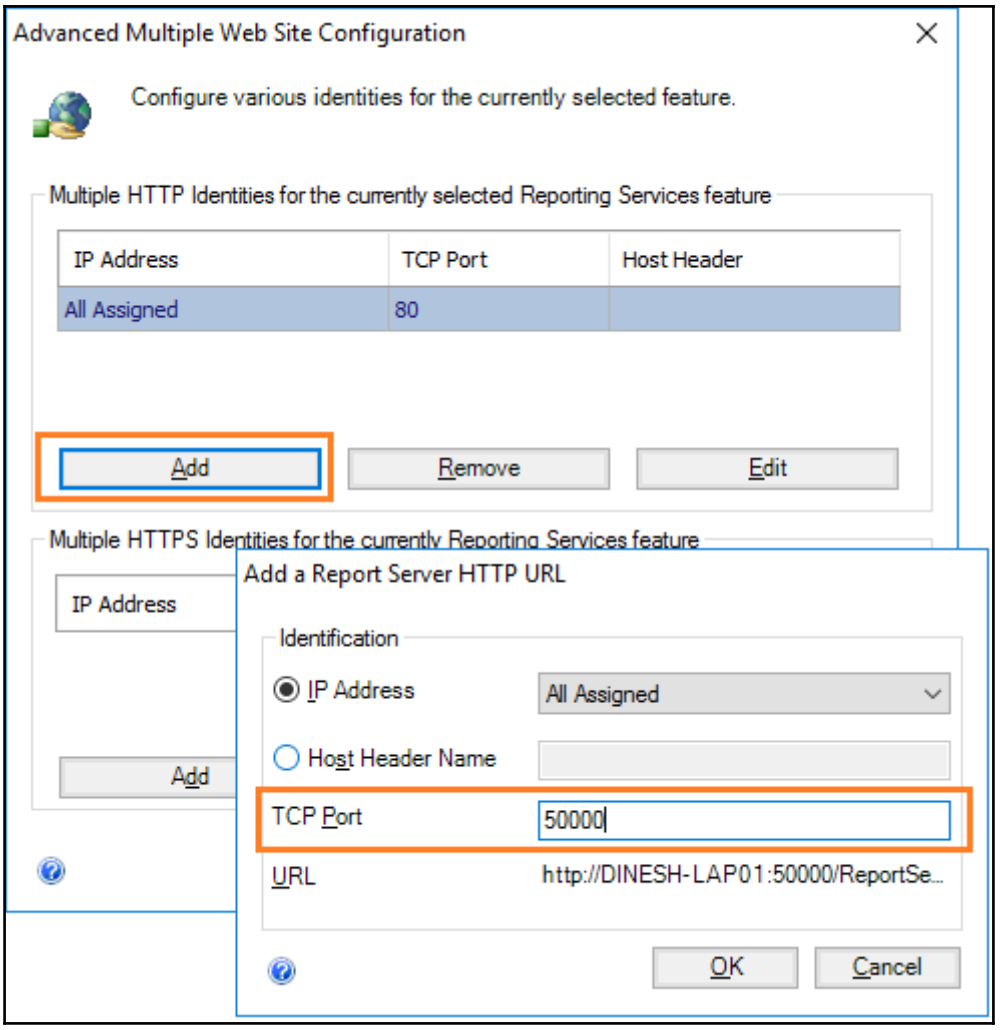

Advanced Multiple Web Site Configuration
Configure various identities for the currently selected feature.
Multiple HTTP Identities for the currently selected Reporting Services feature
IP Address
TCP Port
Host Header
All Assigned
80
Add
Remove
Edit
Multiple HTTPS Identities for the currently Reporting Services feature
IP Address
Add
Add a Report Server HTTP URL
Identification
IP Address
All Assigned
Host Header Name
TCP Port
50000
URL
http://DINESH-LAP01:50000/ReportSe...
OK
Cancel

Figure 1.15

5. Once the settings are saved, you should see two URLs appearing: one with port 80, which is the default, and another with port `50000`. When you set a port, make sure that it is not reserved by another application:

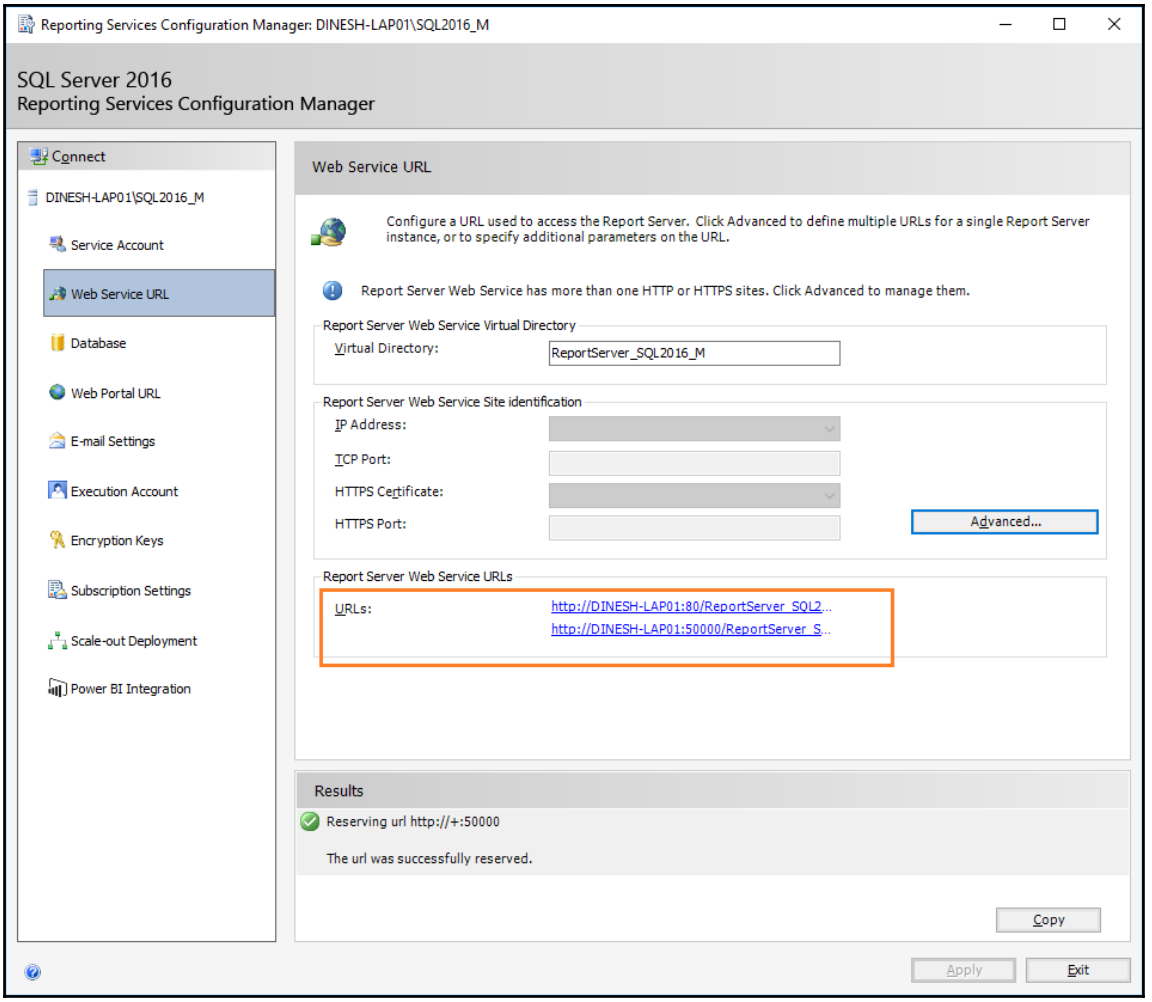


Figure 1.16

6. You can click on either URL to open the web service application. The configuration of **Web Portal URL** is the same as the **Web Service URL**. The **Web portal URL** page also has the **Advanced…** button for configuring additional URLs. Let's see how we can change the name of the virtual directory for web portal.

7. For the Web Portal URL, the default virtual directory name is formed as `Reports_<instancename>`. As you can see in the following image, in our testing machine, it has been set as `Reports_SQL2016_M` and the URL has been formed as `http://DINESH-LAP01:80/Reports_SQL2016_M`:

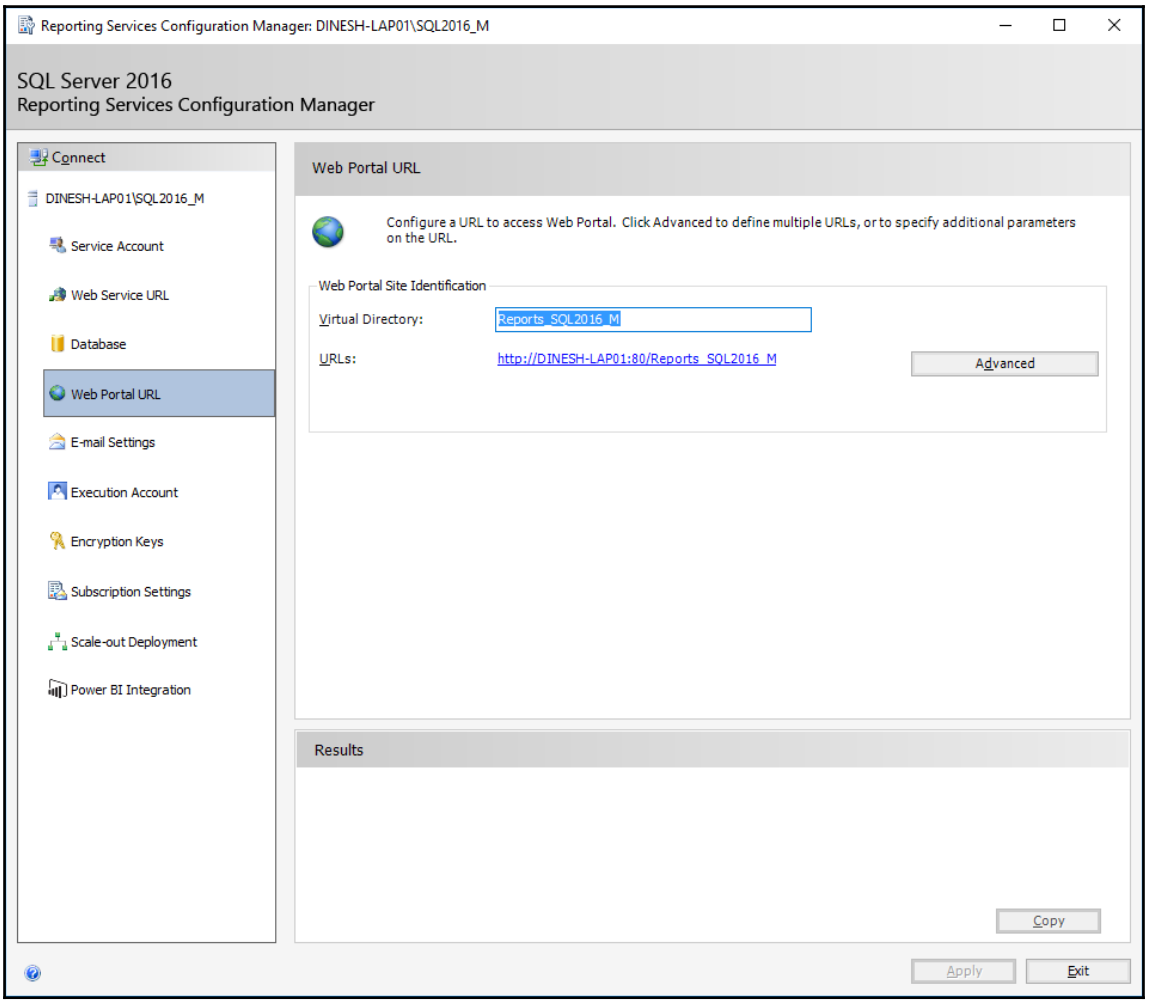


Figure 1.17

8. Let's change the name of the virtual directory to `ProductionReports`, as shown next. Once changed, the **Apply** button must be clicked on to save the changes. Note that you need to make sure that there is no virtual directory in IIS with the same name, if IIS is installed in the same server:

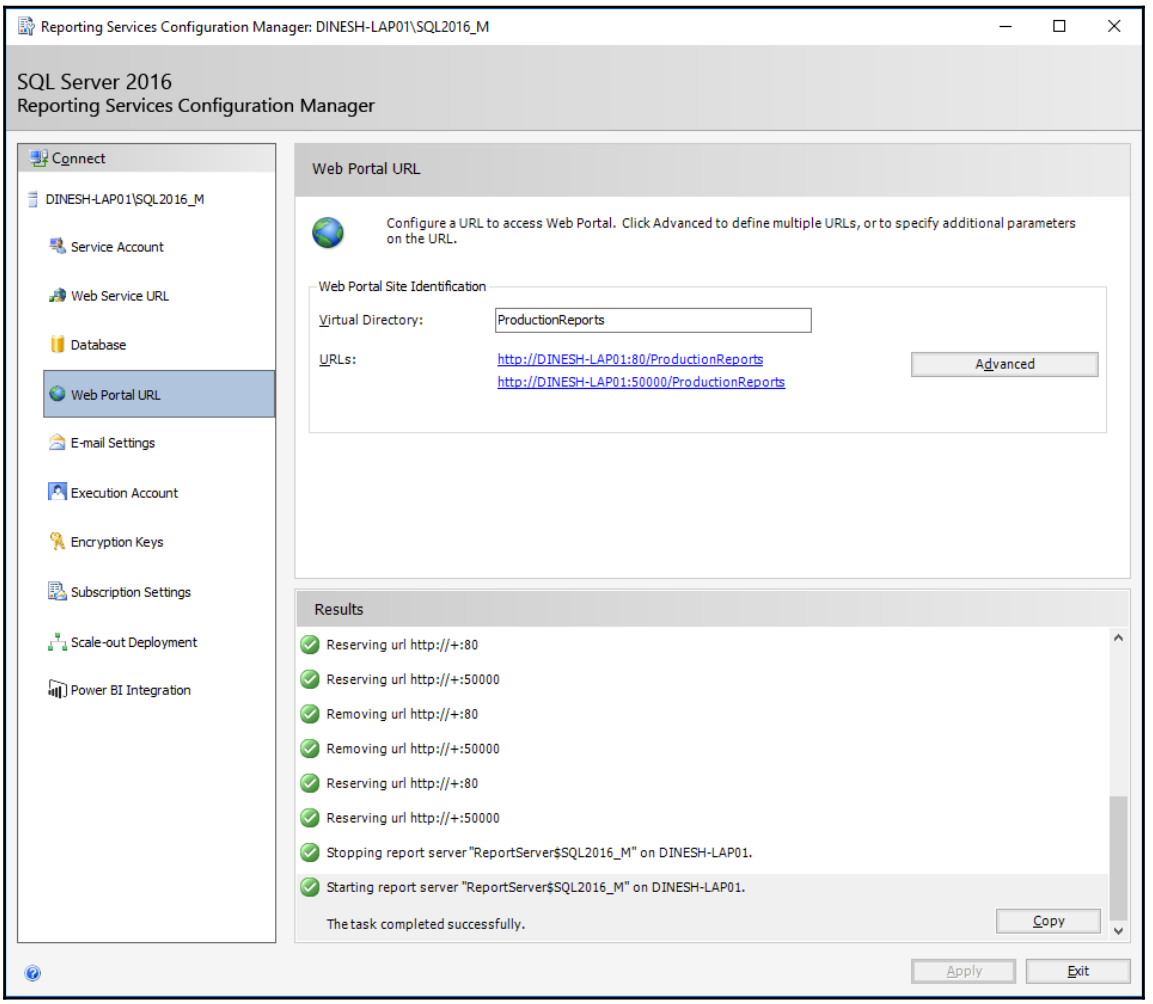


Figure 1.18

You can click on the new URL and see whether it opens web portal or not.

How it works…

As we discussed before, Reporting Services Configuration Manager creates URL reservations in `HTTP.SYS` when changes are applied. If you open the `rsreportserver.config` file, you will see how changes are applied to the file.

As we set an additional port to the web service application, we can set additional IP addresses or hostnames for forming new URLs. One advantage with additional URLs is that you can provide different URLs for different audiences, as shown here:

- **For internal (or Intranet) users**: `http://server/reports`
- **For external (or Internet) users**: `http://www.company.com/reports`

```
<URLReservations>
  <Application>
    <Name>ReportServerWebService</Name>
    <VirtualDirectory>ReportServer_SQL2016_M</VirtualDirectory>
    <URLs>
      <URL>
        <UrlString>http://+:80</UrlString>
        <AccountSid>S-1-5-80-3946330831-378249602-484118862-83638884-4098085075</AccountSid>
        <AccountName>NT SERVICE\ReportServer$SQL2016_M</AccountName>
      </URL>
      <URL>
        <UrlString>http://+:50000</UrlString>
        <AccountSid>S-1-5-80-3946330831-378249602-484118862-83638884-4098085075</AccountSid>
        <AccountName>NT SERVICE\ReportServer$SQL2016_M</AccountName>
      </URL>
    </URLs>
  </Application>
  <Application>
    <Name>ReportServerWebApp</Name>
    <VirtualDirectory>ProductionReports</VirtualDirectory>
    <URLs>
      <URL>
        <UrlString>http://+:80</UrlString>
        <AccountSid>S-1-5-80-3946330831-378249602-484118862-83638884-4098085075</AccountSid>
        <AccountName>NT SERVICE\ReportServer$SQL2016_M</AccountName>
      </URL>
      <URL>
        <UrlString>http://+:50000</UrlString>
        <AccountSid>S-1-5-80-3946330831-378249602-484118862-83638884-4098085075</AccountSid>
        <AccountName>NT SERVICE\ReportServer$SQL2016_M</AccountName>
      </URL>
    </URLs>
  </Application>
</URLReservations>
```

Figure 1.19



If you have multiple instances installed on the same machine, you need to have a proper naming convention to avoid conflicts with URL reservations.

There's more…

When accessing either web service or web portal in the local server, you can use localhost instead of an IP address or server name. You can simply call applications using `http://localhost/reportserver` and `http://localhost/reports`. The `http://localhost` URL is interpreted as `http://127.0.0.1`. However, if you have configured URLs to the computer name or a single IP address, this will not work unless you have added an additional reservation for 127.0.0.1.

Configuring the Reporting Services database

Reporting Services stores and maintains different types of information such as settings, metadata, and objects related to its applications. Some of this information is stored in configuration files and some of it, specifically metadata and objects, is stored in a SQL Server database. Reporting Services native mode uses two databases named **ReportServer** and **ReportServerTempDB** by default. The ReportServer database is the primary storage for holding all metadata and objects. ReportServerTempDB is for maintaining temporary information.



Reporting Services uses three databases if the mode is SharePoint integrated mode. In addition to the ReportServer and ReportServerTempDB databases, it uses a third one to hold altering metadata.

If the Reporting Services instance is the default instance, the names of these databases are set as Report Server and `ReportServerTempDB`. If the instance is a named instance, then the default names become `ReportServer$<instancename>` and `ReportServerTempDB$<instancename>`.

The Reporting Services database is used to maintain report definitions, report models, shared data sources, schedules, security information, and snapshots. You should treat this database as a normal database; hence, it should be regularly backed up and should be a part of **High Availability and Disaster Recovery** strategies.

ReportServerTempdDB stores temporary data, such as session and cache data. Since it maintains temporary data, you do not need to have a backup strategy for this database. However, if this database gets corrupted or accidently deleted, you can create it using the CatalogTempDB.sql query located in the `<installationpath>\Microsoft SQL Server\<instancename>\Reporting Services\ReportServer` folder.

Getting ready

Similar to Reporting Services URLs discussed in the previous recipe, if you selected I**nstall and Configure** during the installation, these two databases are created automatically as a part of the installation. If you selected **Install only**, then you need to use Reporting Services Configuration Manager to create databases. You can host these databases either in a local database engine instance or in a remote database engine instance. Reporting Services supports the following database engine versions for hosting its databases:

- SQL Server 2016
- SQL Server 2014
- SQL Server 2012
- SQL Server 2008 R2
- SQL Server 2008

However, you need to make sure that the Reporting Services database is hosted in the correct edition:

- **Enterprise edition**: The database can be hosted in Enterprise or Standard edition
- **Standard edition**: It can be hosted in Enterprise or Standard edition
- **Web edition**: The database can be hosted in Web edition (local only)
- **Express edition**: The database can be hosted in Express edition (local only)
- **Developer edition**: It can be hosted in Enterprise, Standard, or developer edition

How to do it…

You can use Reporting Services Configuration Manager to create and configure the report server database. Or you can use it to view or modify settings related to the database that is already configured. Use the following steps to create or modify the report server database:

1. Open **Reporting Services Configuration Manager** and go to the **Database** page.
2. You should see two sections: **Current Report Server Database** and **Current Report Server Database Credential**. Click on the **Change Database** button:

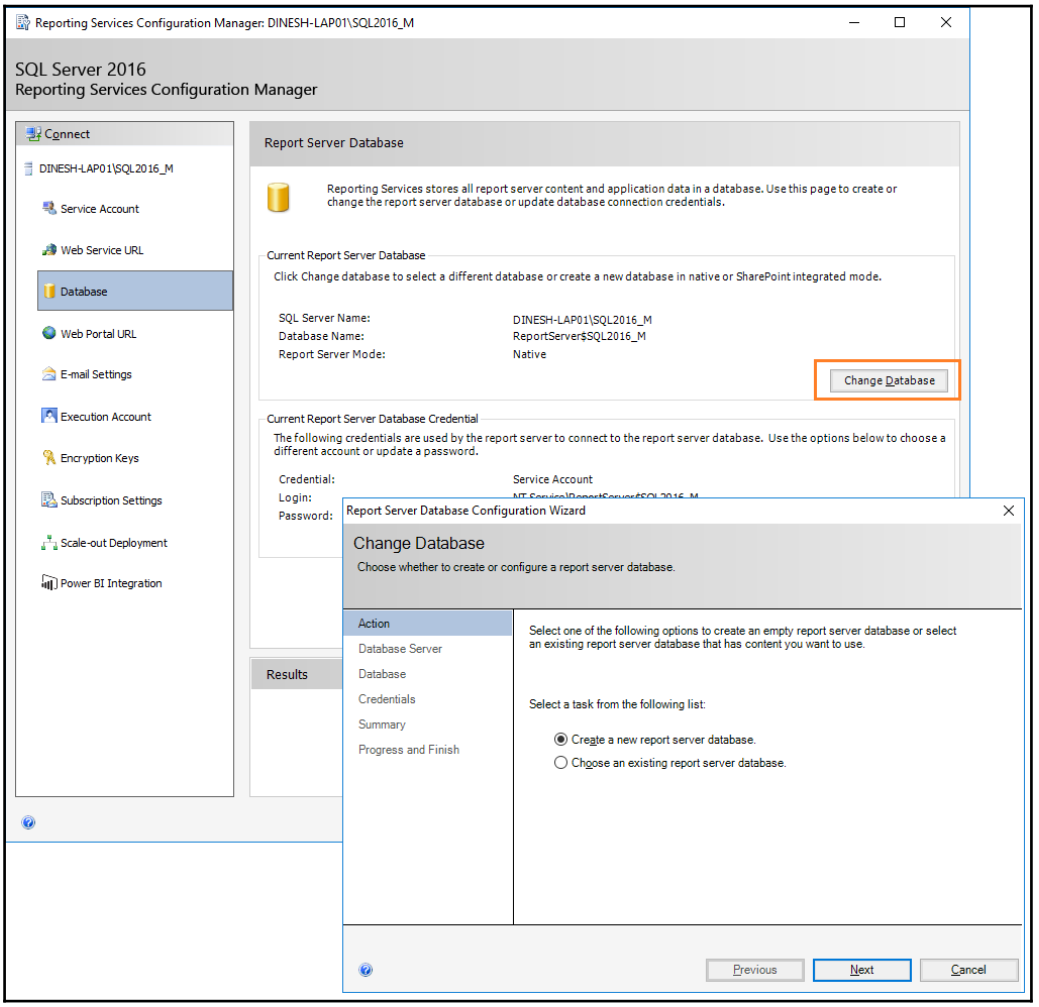


Figure 1.20

3. **Change Database** starts a wizard. You need to select a task from first page; select **Create a new report server database** to create a new one or select **Choose an existing report server database** to add an existing one. Select the first option for standalone report server.
4. The next page allows you to select the hosting server and credentials to connect with it. Note that the credential required in this page is for creating the database only. It will not be used by Reporting Services later to connect with the database:

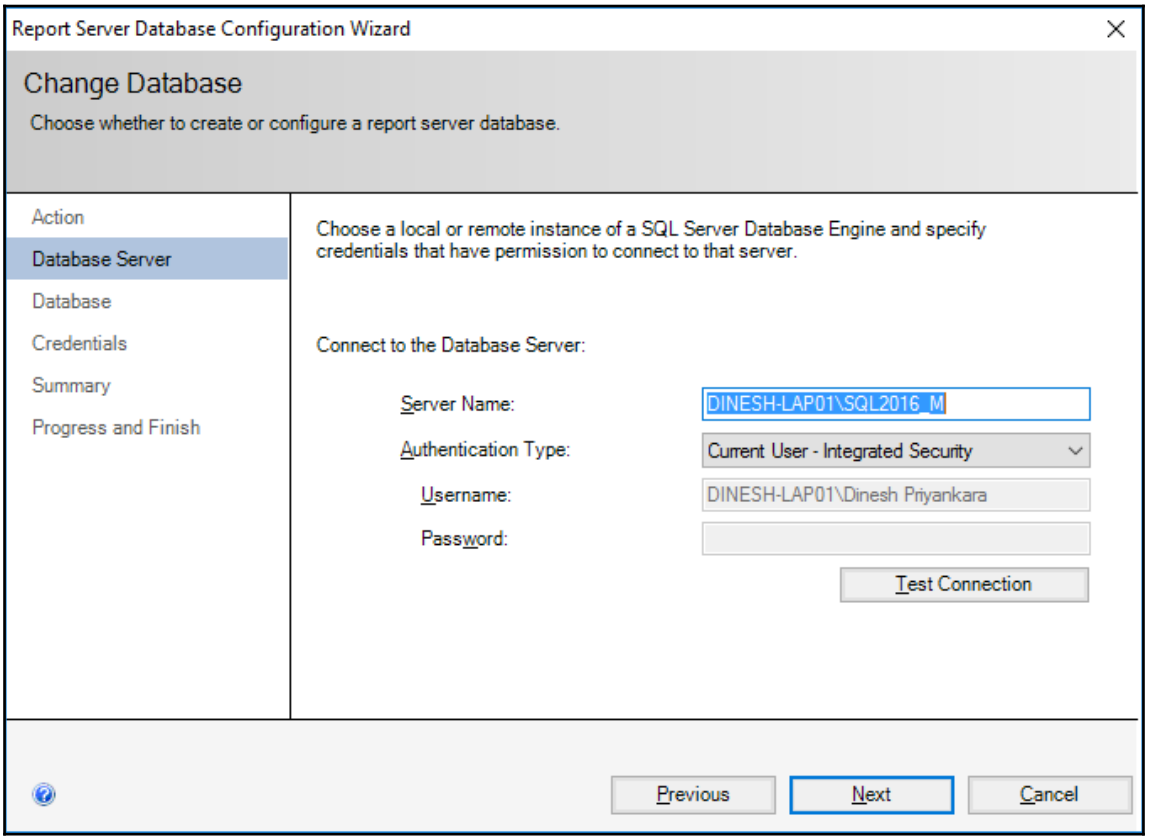


Figure 1.21

5. Select the hosting server and click on **Next** after setting the credentials. The next page allows you to set **Database Name** and **Language**.

6. There is no restriction on the database name. You can enter the name of the database as you want. However, it is always better to have a proper naming convention and name it accordingly. As you can see, the name of the temp database is automatically set using the database name. Click on **Next** to continue:

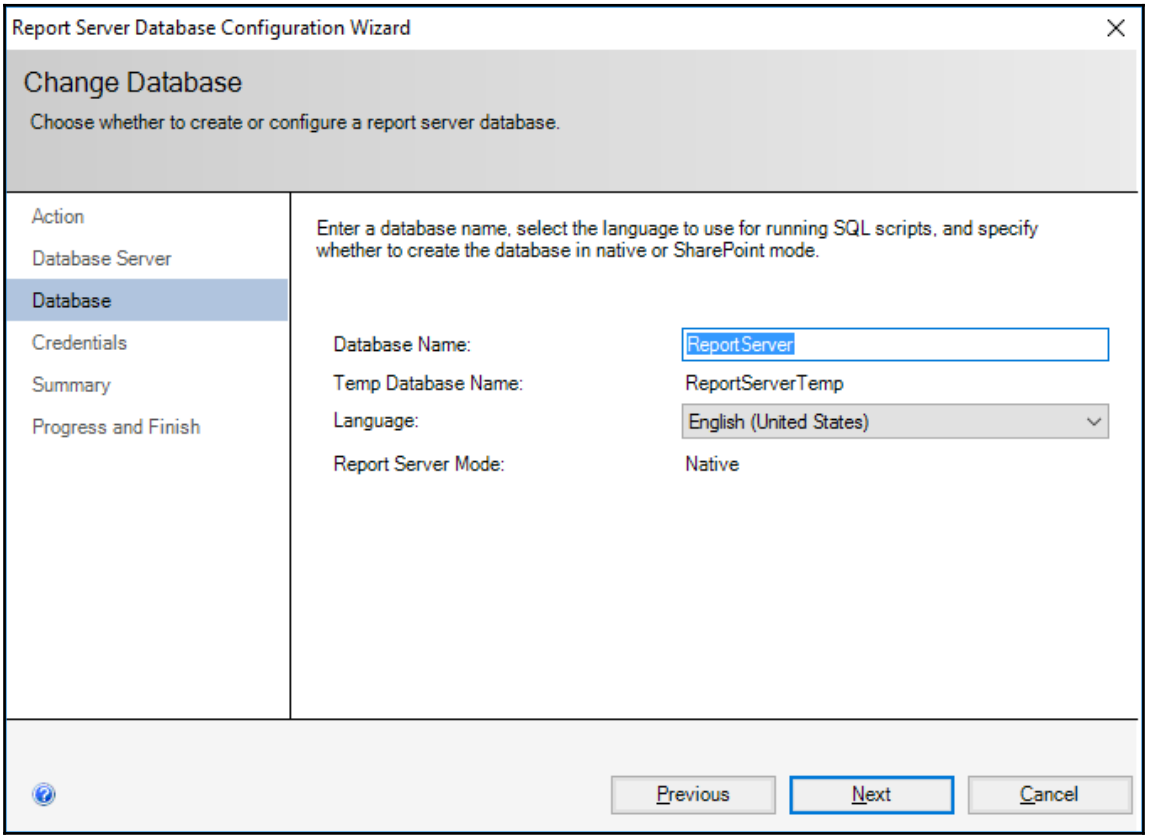


Figure 1.22

7. The next page is for setting credentials to connect to the database by Reporting Services. You have three options: Windows credentials, SQL Server credentials, or Service credentials. If the database hosting server is a remote server, you should select either Windows or SQL Server credentials; otherwise, you can select one of these as per your requirements:

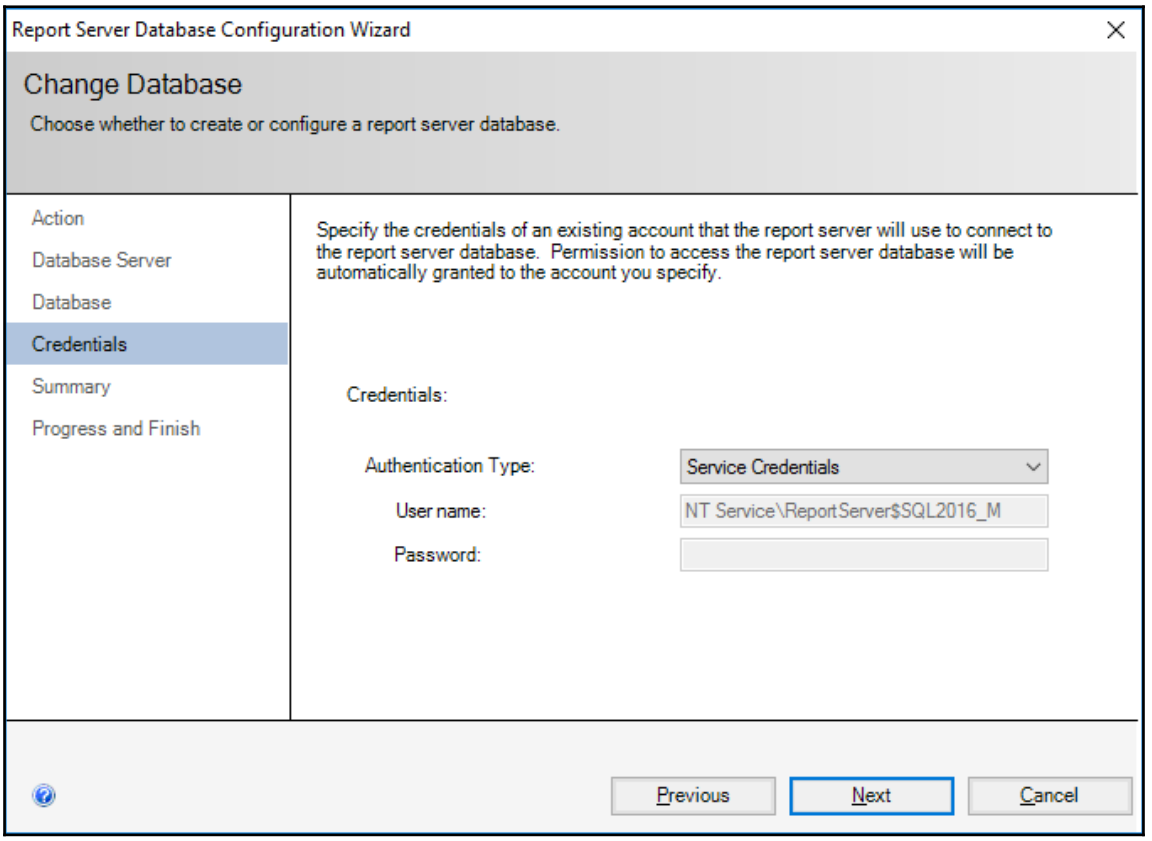


Figure 1.23

8. Everything required is set. Click on **Next** to continue. You should see a summary page with all your input. Continue with the wizard to complete the database configuration.
9. If you go back to the second section of the Database page in Reporting Services Configuration Manager, you will see that there is a button named **Change Credentials**. This is for changing the credentials configured for Reporting Services to connect with the report server database. Although this gets configured either during the installation or while configuring the report database, you may use this option for changing credentials in the following situation:
    - The password set to the local or domain account is changed
    - The authentication type needs to be changed

How it works…

The wizard creates both ReportServer and `ReportServerTempDB` in the selected SQL Server instance. It uses codes stored in the `<installationpath>\Microsoft SQL Server\<instance name>\Reporting Services\ReportServer` folder to create these databases. Once the report server databases are created, report server uses the credentials stored for accessing report databases. During the configuration, it creates the encryption keys required and adds the necessary permissions for accessing the database.



Users who access reports do not require permissions for report database or login for report database server. Report server uses credentials stored for accessing the database when required.

Reporting Services stores the connection for report database in the `rsreportserver.config` file in an encrypted format. Depending on the authentication type selected, some of the nodes related to the connection may be empty. As an example, if the authentication type selected is **Service Credentials**, then **LogonDomain**, and **LogonCred** are empty:

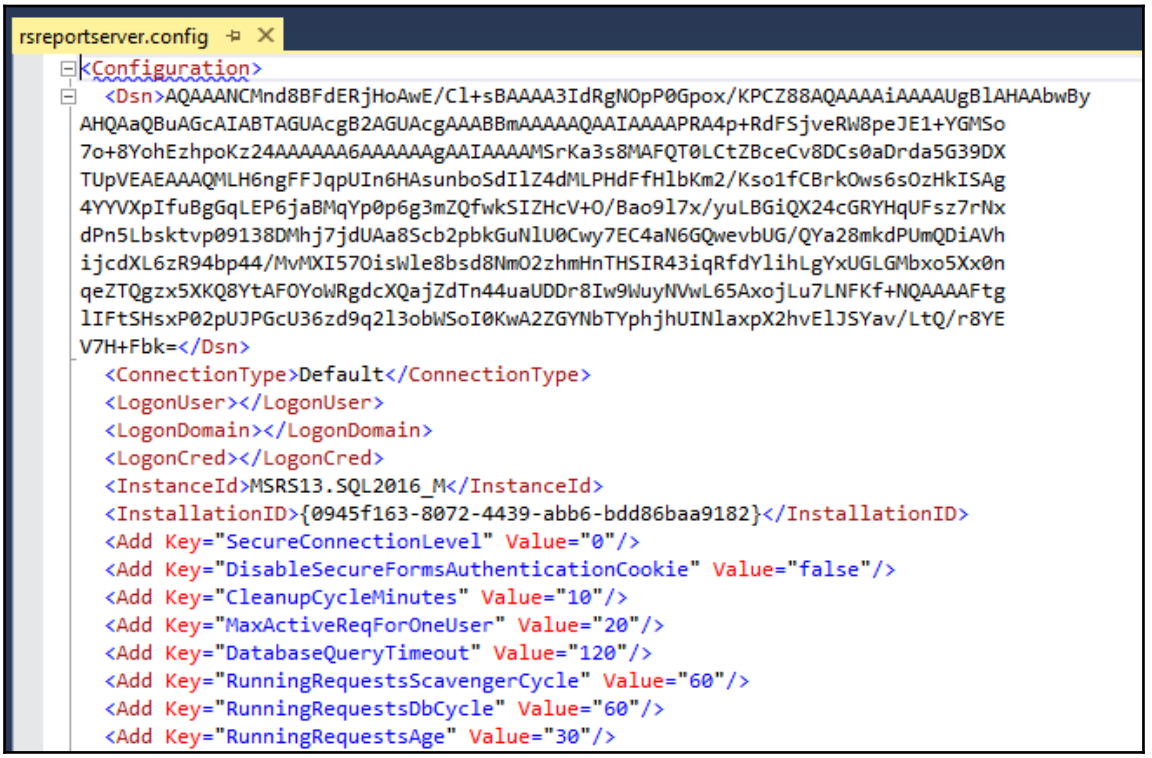


Figure 1.24

In order to verify the report database configuration, log in to the SQL Server instance that hosts the databases. You should see two databases configured with the names you used:

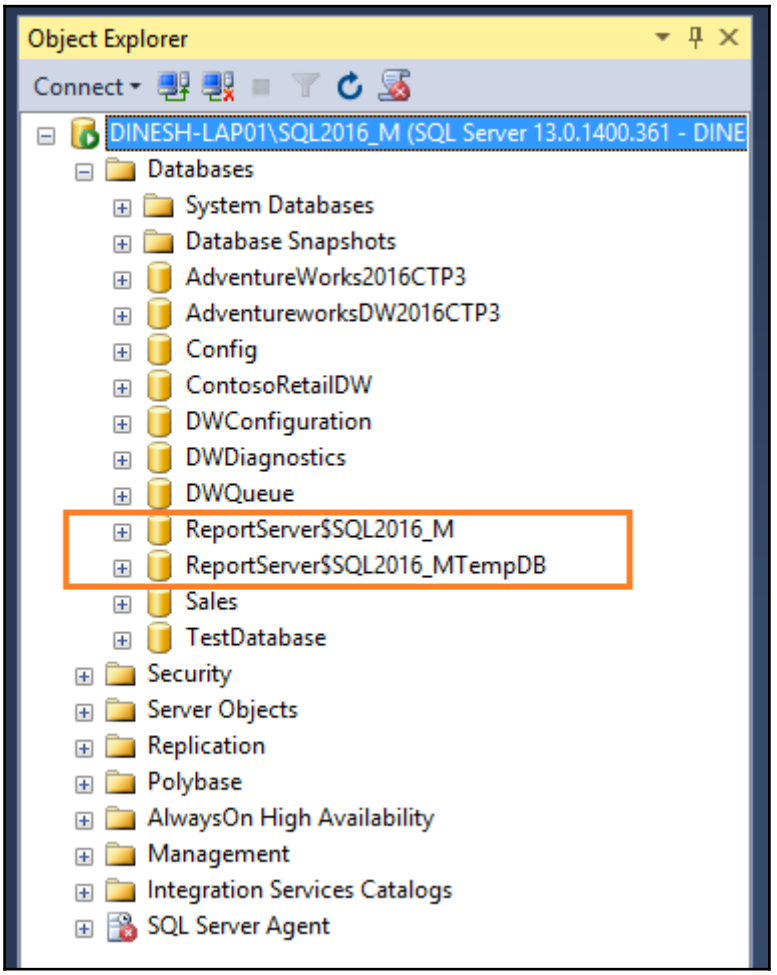


Figure 1.25

There's more…

Remember, Microsoft does not recommend using these databases directly to query or modify data. It is an internal database for report server and should not be accessed and modified.

See also

As mentioned before, Reporting Services uses three databases when the mode is SharePoint integrated mode. Read more about SharePoint integrated mode in `Chapter 9`, *SharePoint Integration*.

Executing post-installation tasks

Although the installation and configurations are done and everything works fine as you expected, it does not mean that the process of installation and configuration is completed. You have to do a few more tasks to complete it. These tasks make sure that your Reporting Services is protected and maintained properly.

Getting ready

There are a few key tasks you should perform after installation and configuration:

- Make sure that report database is a part of your backup strategy
- Back up the `rsreportserver.config` file
- Make sure you have backed up Encryption Keys
- The startup type of the services related to Reporting Services and database engine are set as Automatic

How to do it…

The required tasks can be performed using standard techniques and available tools:

1. Taking a backup of the ReportServer database is easy with Management Studio. Or it can be a part of your scheduled backup job. The main thing to consider is that it should be treated as a normal database and backed up regularly. You may implement a strategy that has one backup type, such as **Full Database backup**, or based on the complexity in terms of number of reports it holds, frequency of changes, or heavy user interaction. There can also be a strategy that has a combination of various backup types, such as **Full**, **Differential,** and **Log**.

2. To back up the `rsreportserver.config` file, you can take a copy of it and keep it in a safe place. Since most settings related to report server are held with this, you need to make sure that you have a backup copy of it.
3. You can use Reporting Services Configuration Manager to back up Encryption Keys. This should be done because this requires many instances when recovering after failure. We will discuss Encryption keys in more depth in `Chapter 10`, *Administering and Managing Reporting Services*.
4. In order to make the startup type of related services automatic, use Services Desktop Application. Note that you should not use this application to change any other settings.

How it works…

The key thing to understand is how quickly you respond to an issue, troubleshoot an issue, or how you can recover your reporting environment in a disaster. There are a few standard recovery strategies for reporting environments, but these post installation tasks will definitely help you to recover in most situations. As an example, if you make a mistake while changing the `rsreportserver.config` file manually, you can simply restore it to the previous stage if you have a backup copy of it. If, for some reason, your ReportServer database becomes corrupted, you can get it recovered using the database backup. Therefore, post installation tasks should not be ignored and must be a part of the installation and configuration process.

There's more…

You can secure your reporting server more by configuring a **High Availability and Disaster Recovery** strategy. Considering the demand, requirements, and the budget, a proper HA and DR should be implemented.

2
Authoring Reports with SQL Server Data Tools

In this chapter, we will introduce report development using **SQL Server Data Tools (SSDT),** integrated with Visual Studio 2015, exploring new features and enhancements related to report authoring. We will cover the following recipes:

- The designer tool – exploring SSDT
- Visual Studio 2015 support
- Creating reports with a table data region
- Creating reports with a matrix data region
- Creating reports with a chart data region
- Managing and configuring parameters

Introduction

In this chapter, we'll roll up our sleeves and start producing reports. But before we do so, we'll need to get familiar with the **SQL Server Data Tools** environment. Then we'll begin authoring the most common types of reports, including table reports, matrix reports, and list and chart reports.

We'll also see how to work with our reports behind the scenes, looking at configuration options as well as the underlying XML code for a report.

The designer tool – exploring SSDT

SQL Server Data Tools (**SSDT**) is the main interface for working with SQL Server Business Intelligence projects, including Reporting Services. It offers a full suite of utilities to not only author your reports but also assist you in doing so. It is worth taking time to explore SSDT.

Getting ready

In `Chapter 1`, *Getting It Ready – Configuring Reporting Services*, *Installing Reporting Services-related tools* recipe, describes how to install SQL Server Data Tools. If these instructions were completed successfully, you should be able to launch SSDT. On Windows 10, simply click on the **Start** menu, pick **All apps**, and scroll to **SQL Server Data Tools 2015**, as shown in figure 2.01:

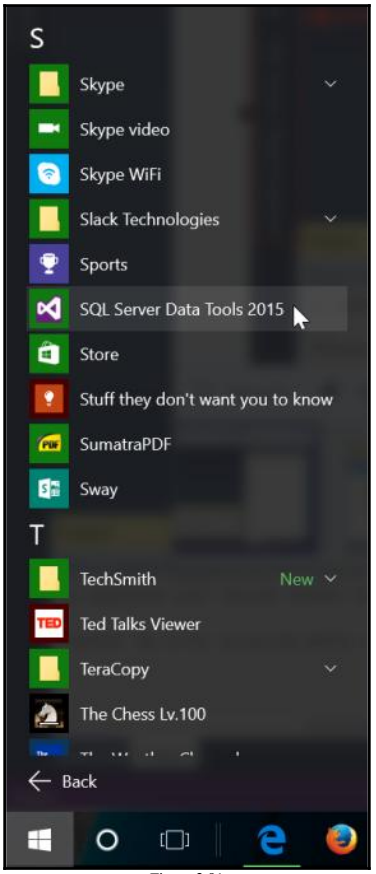


Figure 2.01

Don't confuse the version number of SSDT with the version of SQL Server. The version for SSDT is based on Visual Studio (more on that later). By default, SQL Server Data Tools 2015 comes already installed as part of Visual Studio 2015. If you don't have Visual Studio 2015 already, you can obtain SSDT as a free download from `https://msdn.microsoft.com/en-us/library/mt204009.aspx`.

How to do it…

1. Begin by launching SSDT. Then, from the menu, go to **File** | **New** | **Project**. You will see a dialog similar to what is shown in figure 2.02:

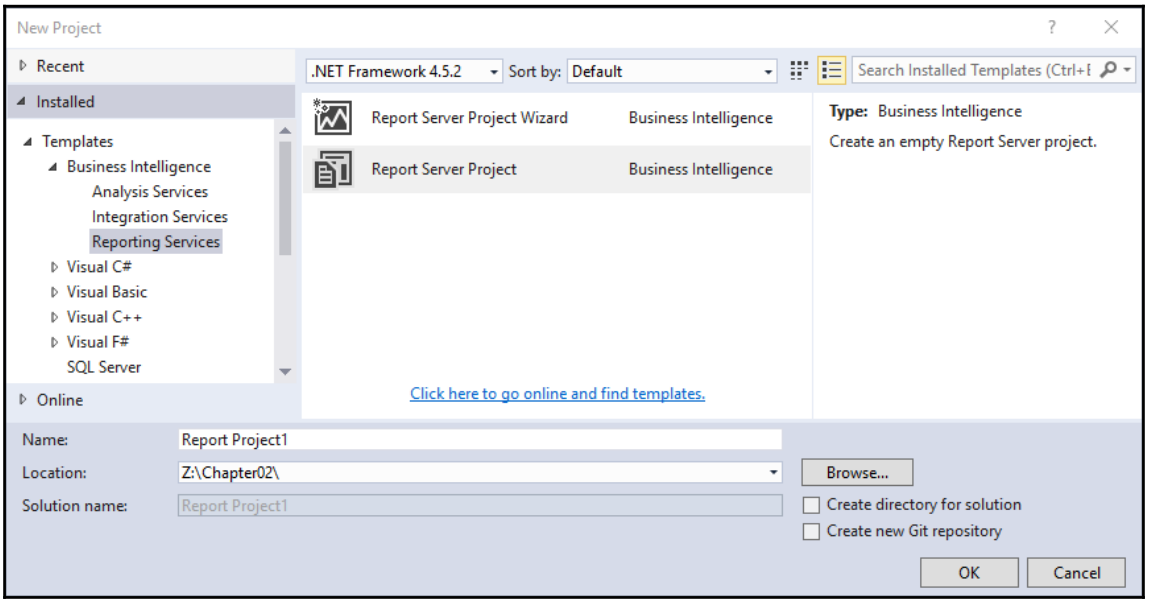


Figure 2.02

2. To the left, you will see a list of **Installed** templates. Drill down into **Business Intelligence** and then **Reporting Services**. Once done, your center area should show two options, **Report Server Project Wizard**, and **Report Server Project**.

3. The wizard will create the project, and then launch the wizard to begin creating your first report. As it is possible to launch the wizard later, for now we can just pick the **Report Server Project** option. This will open up SSDT, so we can look around.

4. Once open, your SSDT environment will resemble the one shown in Figure 2.03:



Note that your SSDT won't exactly match with the following figure. However, with a few minor tweaks, you can access everything you need. We'll talk more about the individual panes later; for now this will give you a general overview.

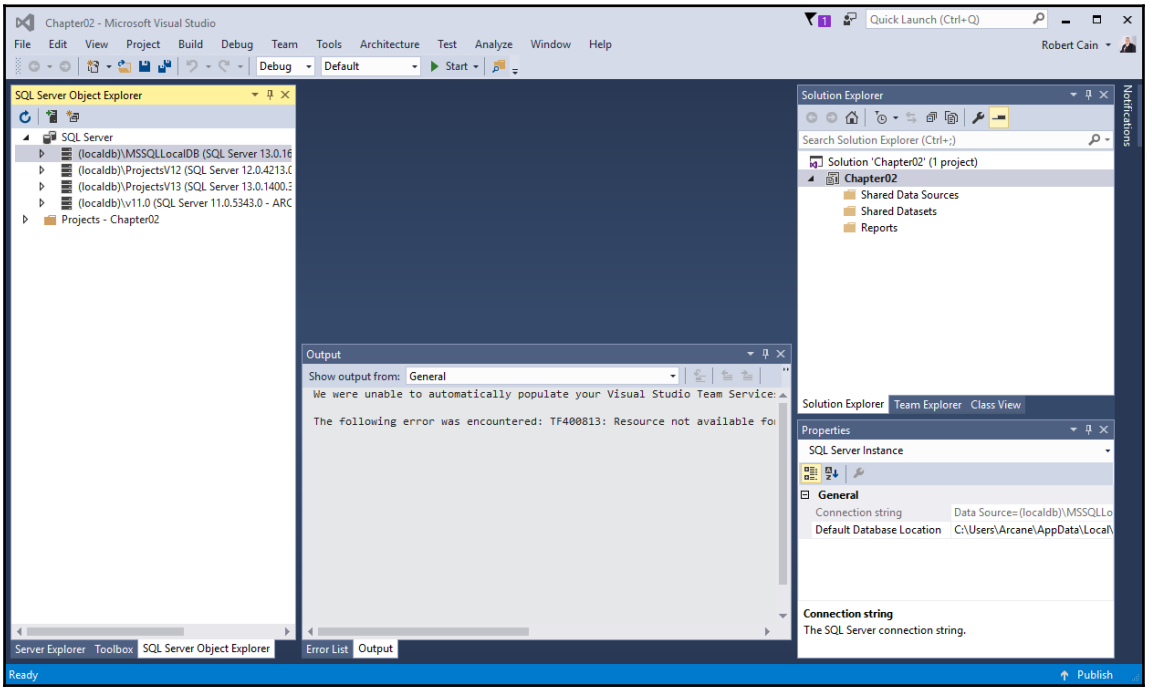


Figure 2.03

5. Across the top is the set of menus, traditional to most applications. Depending on which components are installed on your system, and what project you are working on, these menus will alter to reflect your project. Below this is the toolbar, making access to commonly used commands easy.

6. Taking up the left-hand side is the tools area. If you are in a position where you do not see a specific set of tools, simply click on the **View** menu to see the wide array of tools available to you.
7. Below is the area that holds informational messages about your project. Errors, output (from deployments and the like), and more can be found in this messages area.

8. To the right, at the top, is information about your solution, held in the **Solution Explorer**.
9. Below the **Solution Explorer** is the **Properties** window. This will display the properties of a specific item in your solution.
10. SSDT is completely customizable. If you are on a small monitor, you may wish to employ the Auto Hide feature. In the header of every window, to the right is a small icon that resembles a push pin, as shown in figure 2.04:

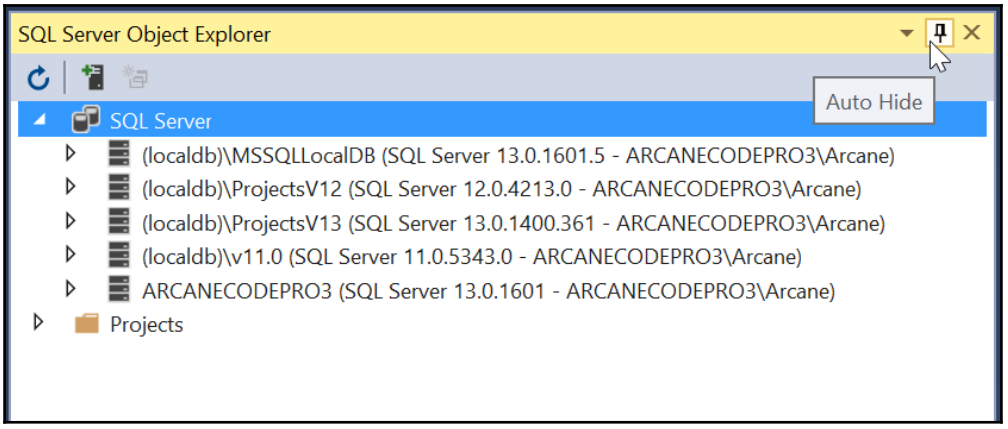


Figure 2.04

11. When your mouse hovers over the icon, it will display the help text **Auto Hide**. Clicking on it will cause all the tool windows in that area to collapse, as seen in figure 2.05:

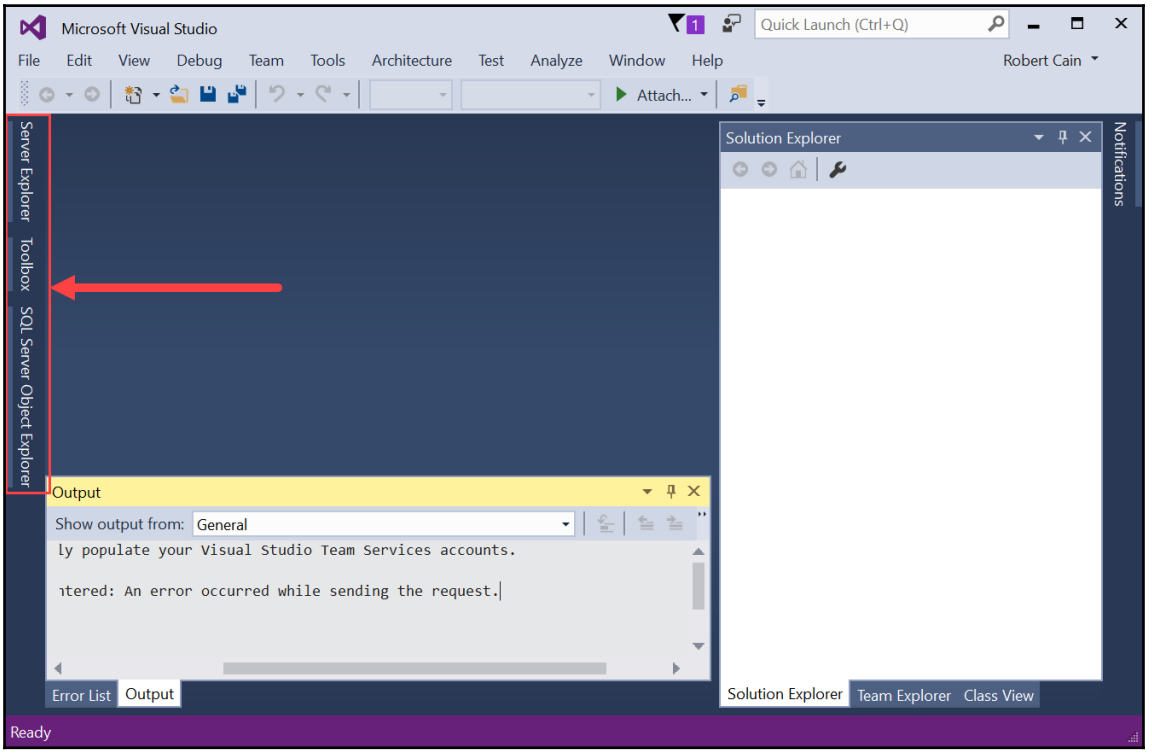


Figure 2.05

12. Click on the title bar and hold down the mouse to move the window. If you have multiple monitors, you can drag these to the second monitor maximizing your report designer area. You can also combine windows. When hovering over another pane, a set of positioning handles will appear, as shown in figure 2.06:

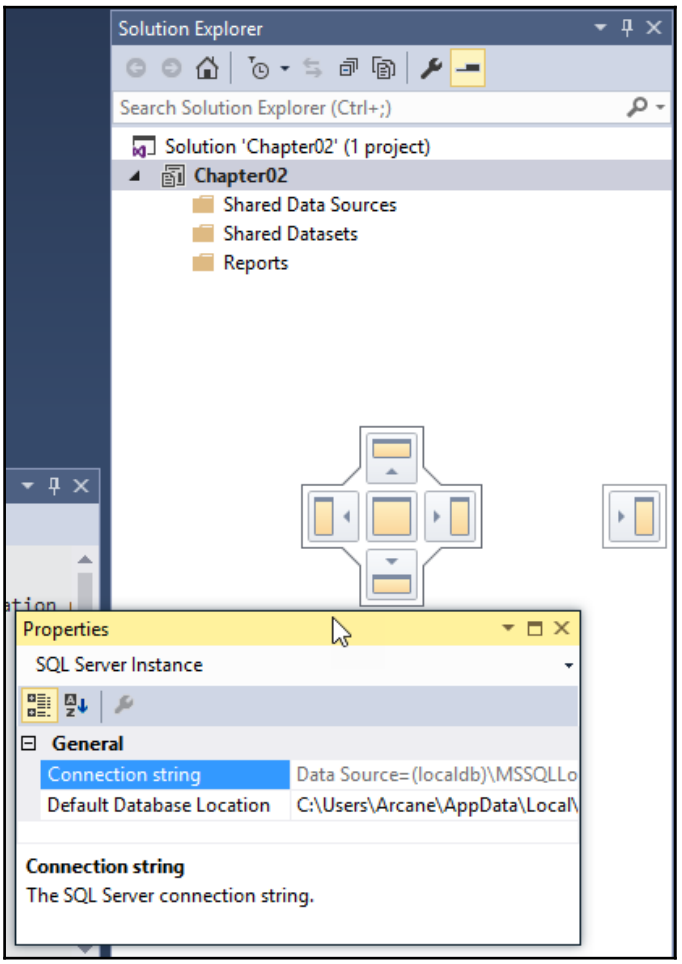


Figure 2.06

How it works…

SSDT includes a rich set of tools that can assist you with developing your reports. Of course there's the toolbox, which has all of the controls that can be placed on a report. There's also a SQL Server Object Explorer, from which you can view the structure of a database without ever having to leave SSDT.

Sharp-eyed readers may have noticed that in the title bar, the caption does not read SQL Server Data Tools, but Visual Studio. SSDT is a set of templates that are added into Visual Studio. If you did not have Visual Studio prior to installing SQL Server, then it will install the Visual Studio shell, and the title bar will read **SQL Server Data Tools**. Otherwise, it will display Visual Studio as shown in the aforementioned figure. Because SSDT is part of the very powerful Visual Studio **Integrated Development Environment** (**IDE**), it has all of the power of a fully developed IDE. What we have seen is the tip of the iceberg; future recipes will further explore the Visual Studio environment.

Visual Studio uses the concept of templates to manage the types of projects it can create. When you created your Reporting Services project, you may have noticed in Figure 2.02 that it read **Installed Templates**, and under it was the branch for **Business Intelligence**; under that is the template for **Reporting Services**.

Looking over the list, you'll note many types of templates installed. Within the Business Intelligence list, there were templates allowing you to create Analysis Services or Integration Services projects. Additionally, there were templates for Visual C#, Visual Basic, F#, and more. Of course, your system may not have all of these, depending on the options you selected when Visual Studio was installed or whether all you installed was SQL Server Data Tools. It is through the use of templates that Visual Studio can support a wide range of project types.

Visual Studio 2015 support

As mentioned in the previous recipe, SSDT is actually a set of templates that exist within the Visual Studio shell. Because of this, Visual Studio is able to support SSDT in a variety of ways. The powerful IDE was explored in the previous section; another key though is keeping up with updates to the SSDT templates.

Getting ready

Prior to checking for updates, you must have an Internet connection. For most users working on development computers or their home workstations, this won't be an issue. If you are updating SSDT on a Windows server, the administrators may have firewall rules in place, which may interfere with the update process. Be sure to check with your system administrators should you have any issues.

How to do it…

1. Within Visual Studio, select the **Tools** menu and then **Extensions and Updates**. When the **Extensions and Updates** window appears, check the **Updates** link on the left. You will then see a list of available updates, as shown in figure 2.07:

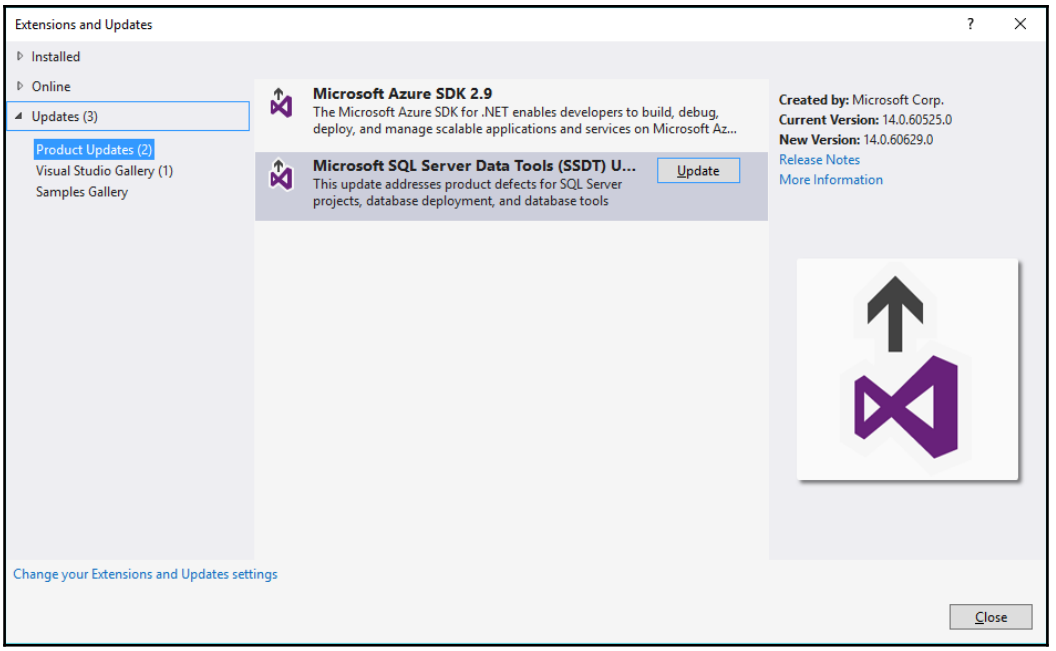


Figure 2.07

2. If an update has been released, you will see it within **Extensions and Updates**. You can update your SSDT templates by simply clicking on the entry for SSDT and then clicking the **Update** button.

How it works…

Microsoft maintains a server with which it can notify users of updates to the components installed in Visual Studio, and allow users to get updated to the most current version of the product. Hence, there is a need for an Internet connection when following this recipe.

Creating reports with a table data region

It's time to create your first report. The most basic of all reports is the table report. This is a row-by-row listing of information. In the process, we'll learn much about the SSDT report interface, including how to use many of the windows within SSDT.

Getting ready…

Before writing any reports, we first need a data source. With SQL Server 2016, Microsoft introduced an entirely new set of sample databases named Wide World Importers. You can find the announcement, along with download links, on the Microsoft website at `https://blogs.technet.microsoft.com/dataplatforminsider/2016/06/09/wideworldimporters-the-new-sql-server-sample-database/`. If you find that a lot to type, you can use the shortcut, `http://bit.ly/packtwwi`. Note that with `bit.ly`, the URL is case sensitive, so be sure to enter it precisely as shown.

The Wide World Importers, samples actually contain two databases, `WideWorldImporters` and `WideWorldImportersDW`. The first is the source database. It is designed to facilitate transactional processing, in other words working as the backend to an application that users access to do their day-to-day jobs. The `WideWorldImportersDW` version is the data warehouse. The data warehouse has a design made to facilitate reporting. They are also used as the basis for building cubes in SQL **Server Analysis Services** or other analytic systems. Data warehouses are a fundamental concept in the world of Business Intelligence, or BI for short.

Because Reporting Services is most commonly thought of as a BI tool, for this chapter, we will be using `WideWorldImportersDW` as the data source. Know though that the techniques shown herein will work with any of the many data sources supported by Reporting Services. After downloading and installing the Wide World Importers, for example, you need to create a new report project to hold your reports. Follow the instructions from the *The designer tool – exploring SSDT* recipe, found previously in this chapter to create your new report project. Each chapter in this book will have its own project: `Chapter01`, `Chapter02`, and so on.

How to do it…

1. Create an empty project. To start the process, look at the **Solution Explorer** located to the upper right if you are using the default layout. Right-click on the **Reports** branch and select **Add New Report** as shown in the next figure. This will launch the **Report Wizard**. Click on the **Next** button:

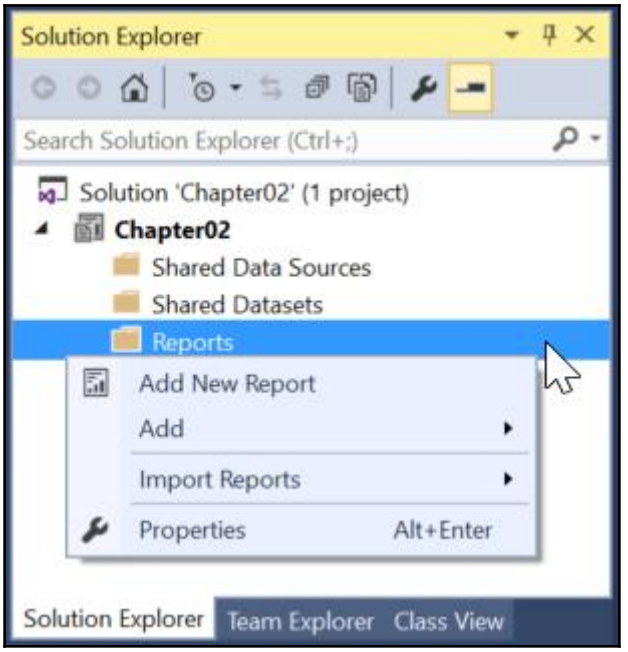


Figure 2.08

2. After clicking on **Next**, select the data source in order to provide the connection information. We have named it `WideWorldImporters`, as shown in the following figure.
3. Click on the drop-down to see the wide variety of data sources that Reporting Services can connect with. We'll be selecting Microsoft SQL Server.

4. To build the connection string, simply click on the **Edit** button. It will bring up a common dialog, where you can supply the server name and database. Figure 2.09 shows what a filled-out form looks like:

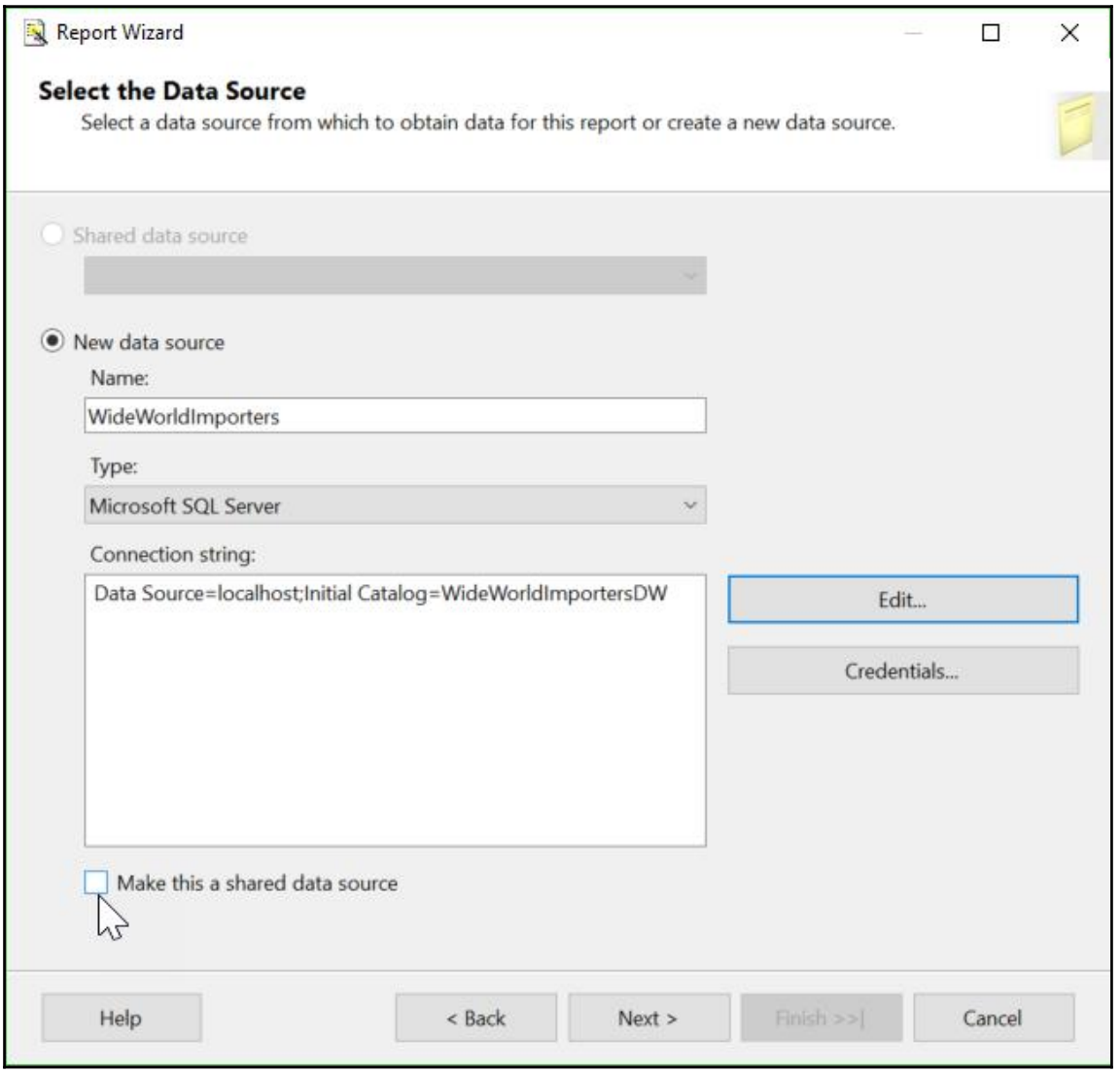


Figure 2.09

5. Leave the checkbox (**Make this a shared data source**) unchecked and click on **Next**.
6. On the next screen, you will either build or enter the query that supplies data to your report. If you are not skilled in the language of SQL, you may find the **Query Builder** option useful. Click on the big **Query Builder** button to bring up the dialog, as shown in figure 2.10:

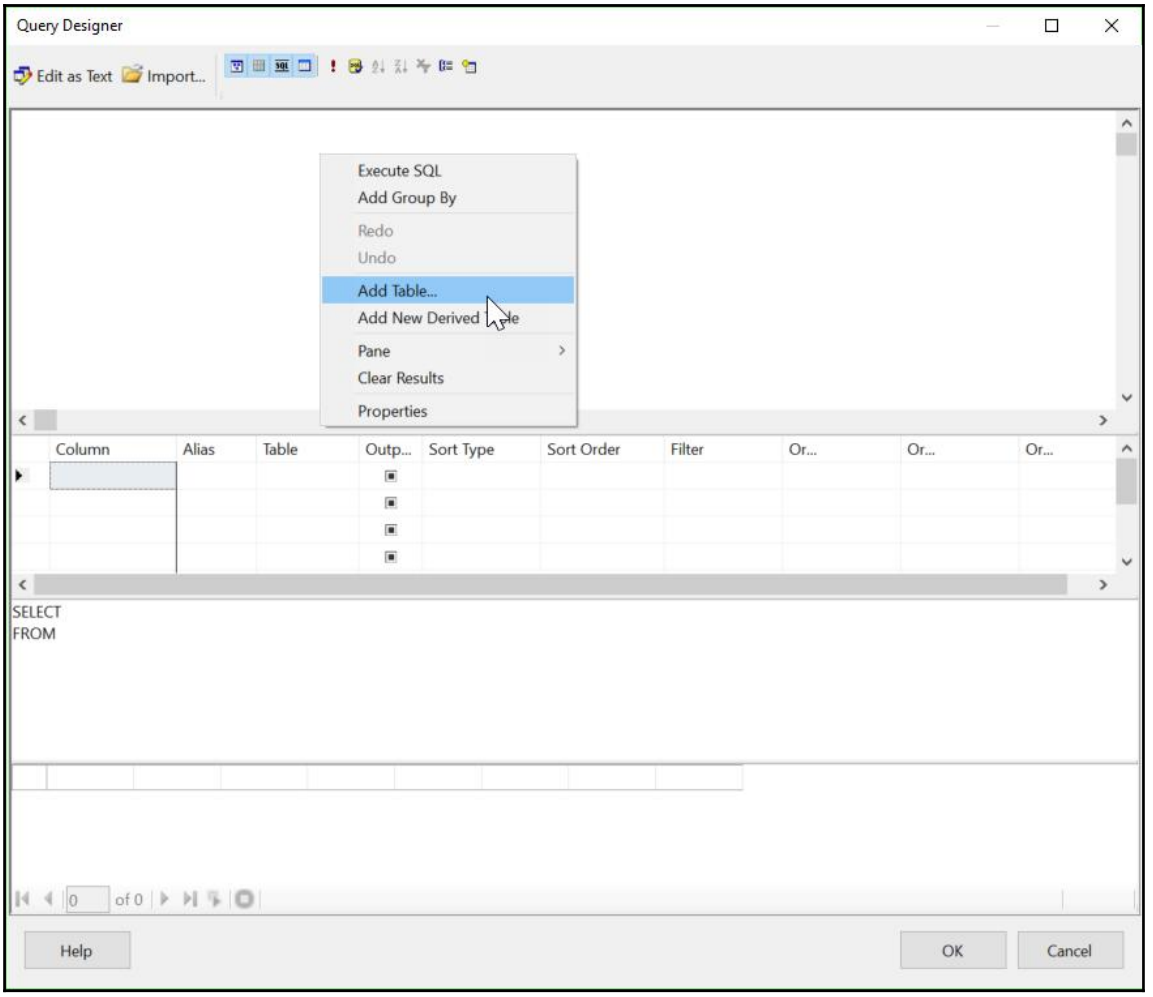


Figure 2.10

7. Start by right-clicking on the big white area, which brings up the menu you see. Click on the **Add Table…** option.
8. In the menu, first scroll down to **Sale (Fact)**, and click on it to select it.
9. Now scroll up to the **Customer (Dimension)** table, and press *Ctrl* and click on it. (hold down the *Ctrl* key while clicking). You could repeat this to add as many tables as you want, and later you could bring up this dialog again to add even more tables, but this will suffice for now.
10. Click on **Add** to place these tables in the dialog, and then click on **Close**.
11. In the **Customer** list of columns on the left, place a check mark next to **Customer**.
12. In the **Sale** table on the right, put check marks beside the **Description**, **Invoice Date Key**, and **Total Excluding Tax** columns.



Note in the pane below how it populates with the information we selected. This also gives a place where we can supply filters. On the row with Invoice Date Key, go over to the column labeled Filter and enter `>= '2016-05-31'`.

13. You will see the SQL Query as the designer built it from your selections. You can run the query by clicking on the tiny red **exclamation mark** in the toolbar. When you do, the resulting dialog should look like figure 2.11:

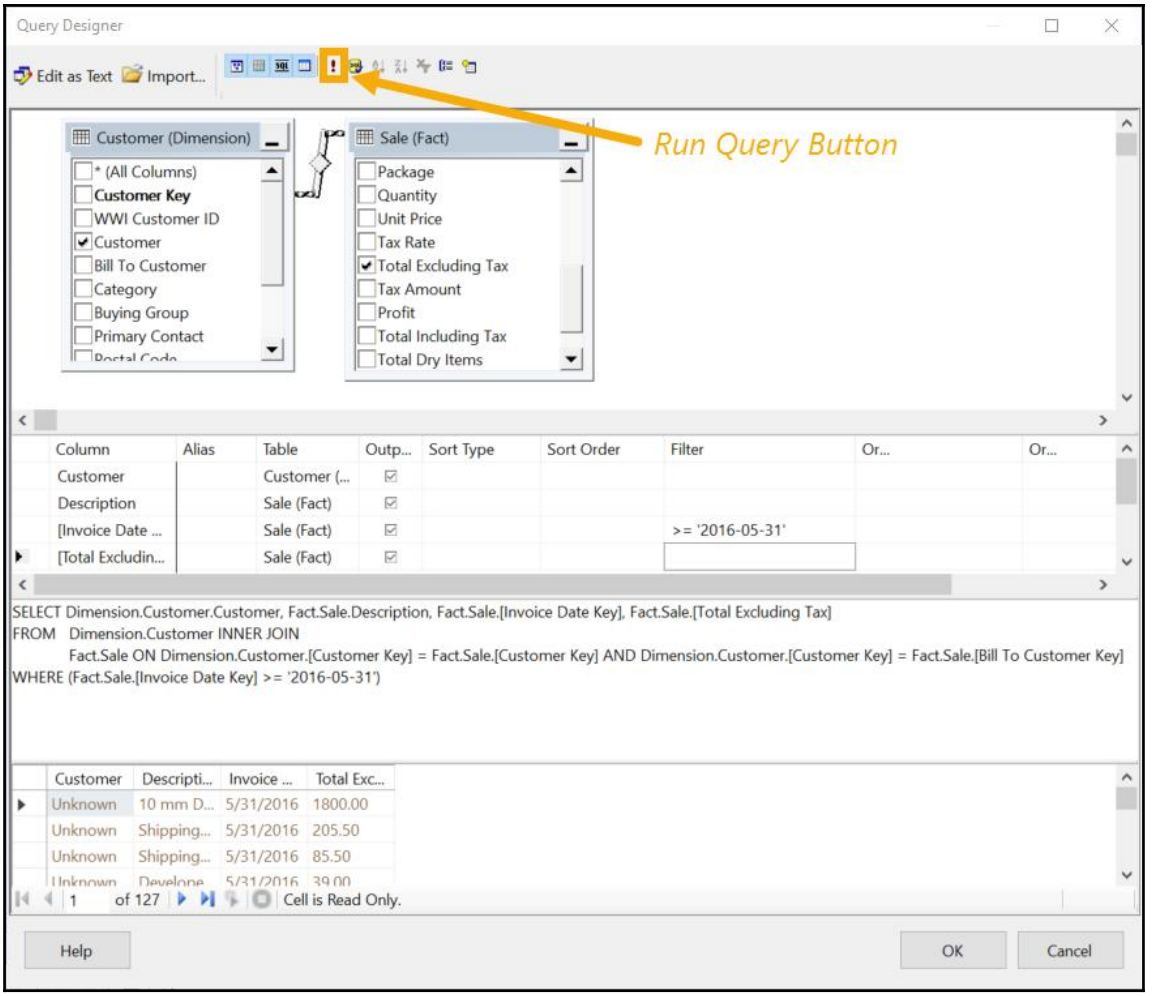


Figure 2.11

14. While this method is good for the novice SQL coder, most people prefer to use **SQL Server Management Studio** (**SSMS**) to develop their queries. Click on **Cancel** to close the query designer.
15. Open up SQL Server Management Studio from your **Windows** menu, or use the Windows search feature to find it.

16. Within SSMS, you can develop your query. Here is a screen capture of SSMS with our query in it:

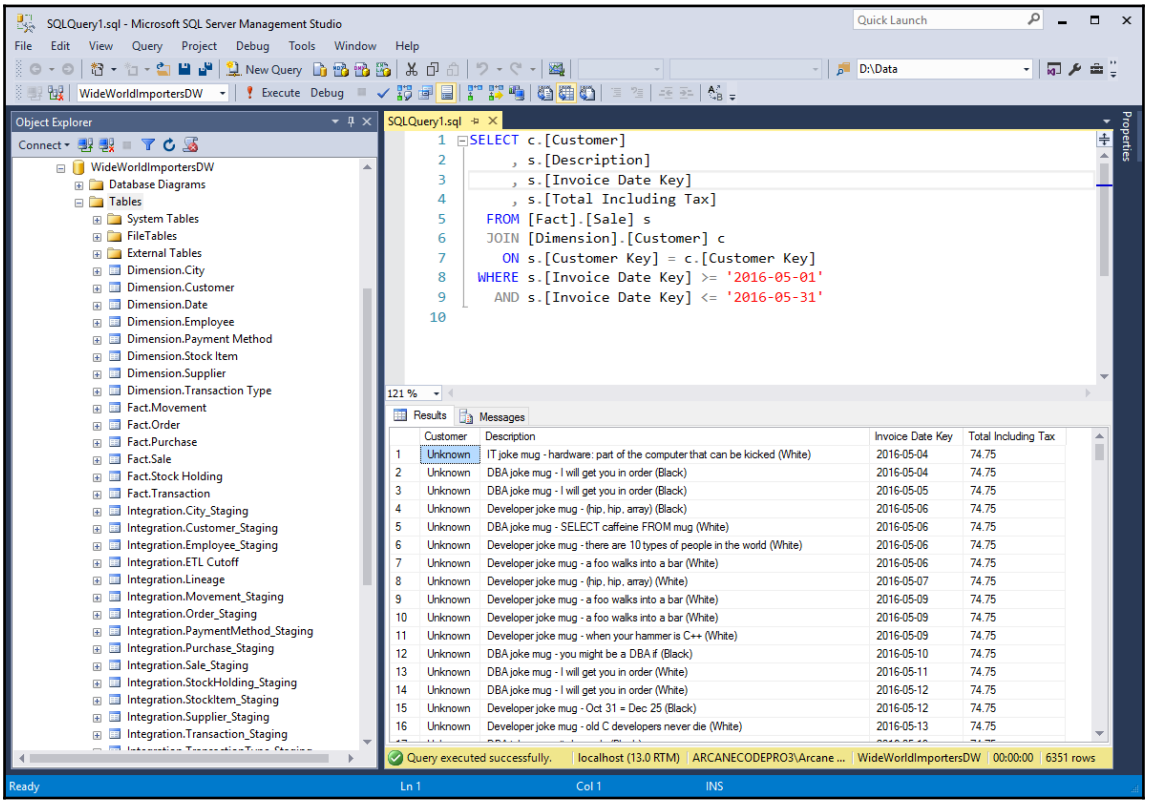


Figure 2.12

For readability purposes, here is the query we'll use for the first report:

```
SELECT c.[Customer]
     , s.[Description]
     , s.[Invoice Date Key]
     , s.[Total Including Tax]
  FROM [Fact].[Sale] s
  JOIN [Dimension].[Customer] c
    ON s.[Customer Key] = c.[Customer Key]
 WHERE s.[Invoice Date Key] >= '2016-05-01'
   AND s.[Invoice Date Key] <= '2016-05-31'
```

17. As you can see, we've also added a filter to restrict the date range. Copy this query to the clipboard and paste it in the Query string area of the query designer, like so:

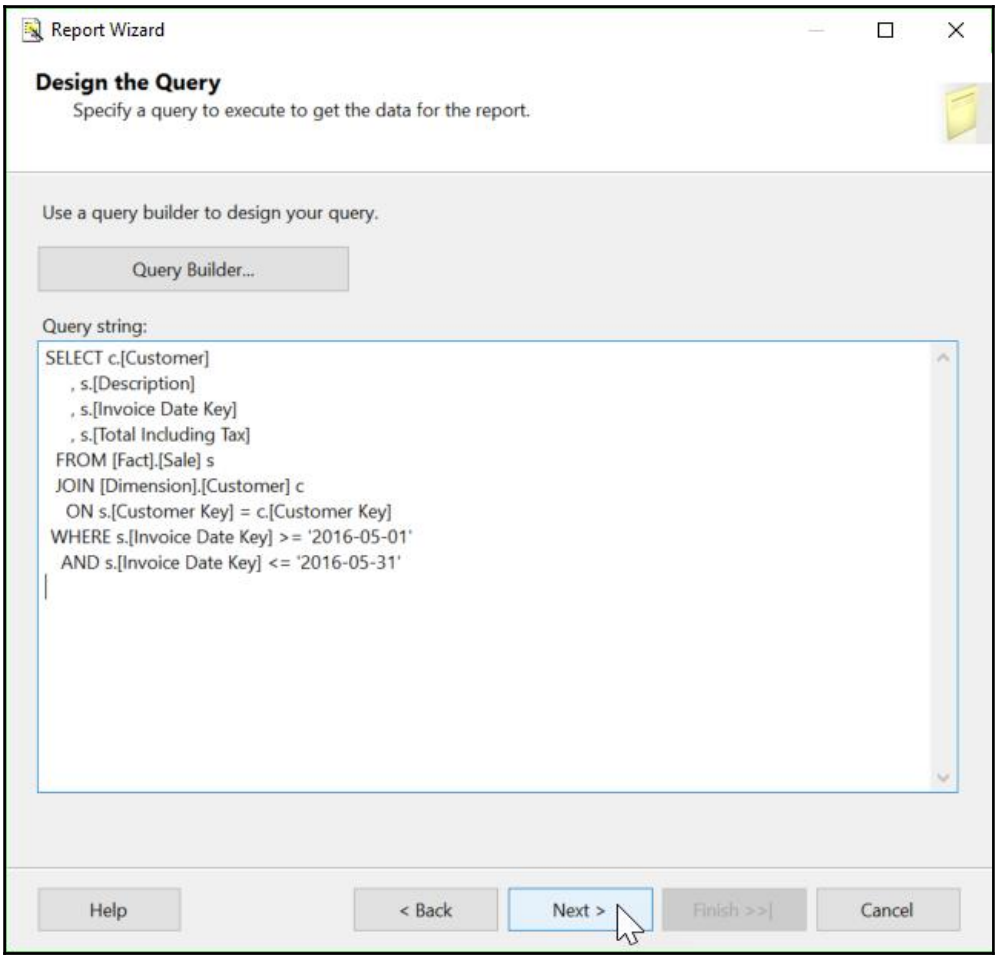


Figure 2.13

18. Click on **Next** to proceed, where you will select the report type. This screen is pretty simple, with only two options: **Tabular** and **Matrix**. We'll look more at Matrix reports in the next recipe; for now leave the option as Tabular and click on **Next**.

19. In order to group the data in the report, click on **Customer** from the **Available fields** area on the left. Then click on the button labelled **Group**. Next, click on **Description**; and now click on **Details**. You should now get a dialog box appearing like figure 2.14:



Note that you can also drag and drop. Try clicking on the two remaining fields and dropping them into the details area.

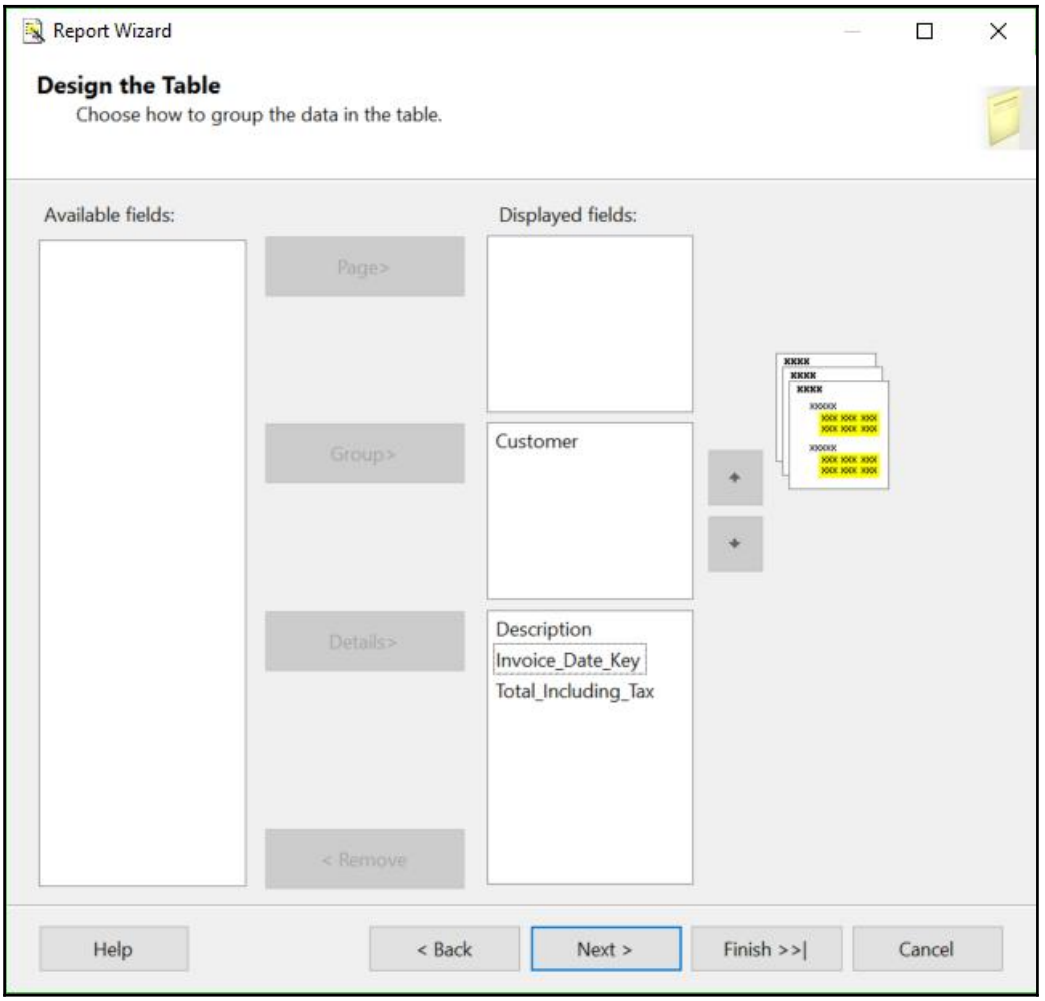


Figure 2.14

20. Note that the **Finish >>|** button is now enabled. At this point, SSRS has enough information to generate the report. However, we can save ourselves a little effort if we click on **Next**, as it will allow us to set additional properties of the report and avoid a lot of manual work later. After clicking on **Next**, you are presented with the following dialog:

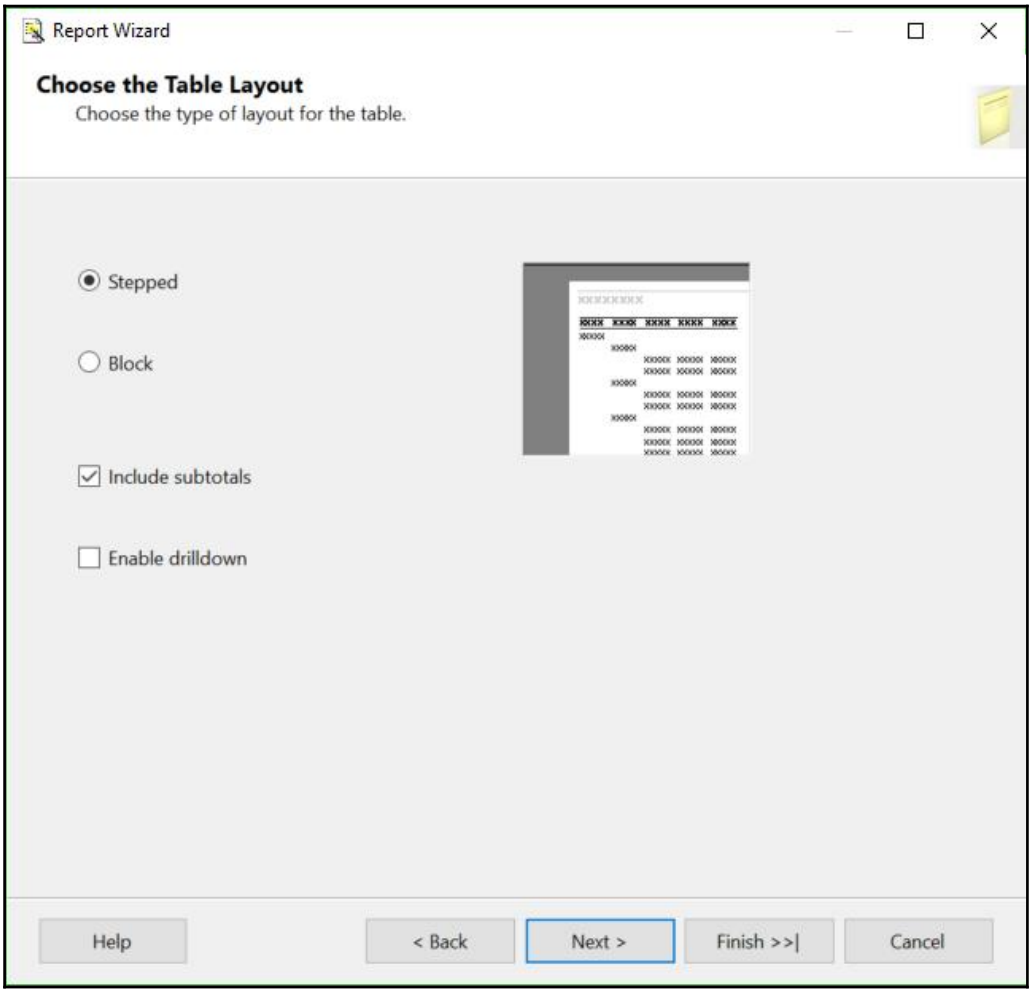


Figure 2.15

21. For the layout, we'll accept the default option of **Stepped**.
22. Check **Include subtotals**, which will automatically insert calculated values and add more rows to handle subtotaling of your groups for you.As the final step, enter a good name for your report, then click on Finish to have the **Report Wizard** generate your report and add it to your project:

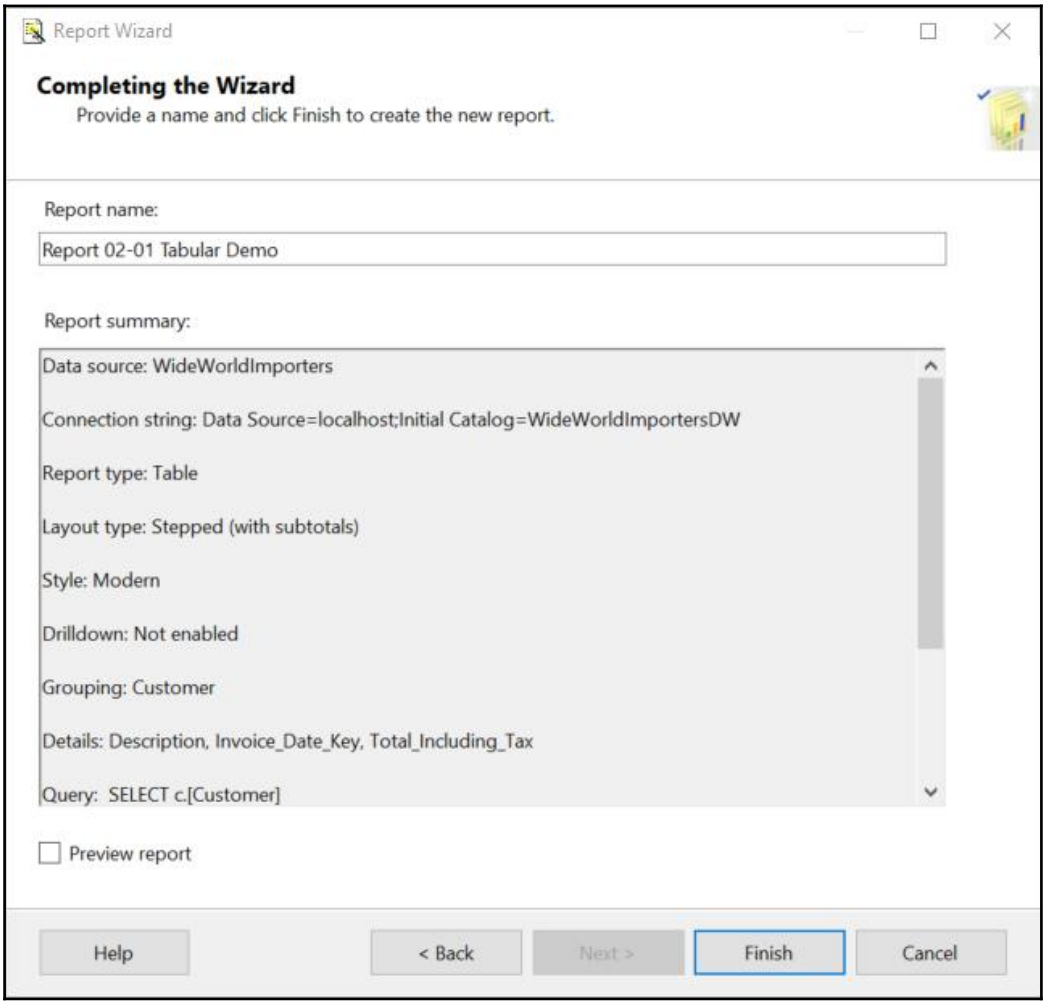


Figure 2.16

23. Review Figure 2.17, which shows the report designer. There are three sections. At the top are the parameters, below that is the main designer area, and at the bottom is the grouping area:

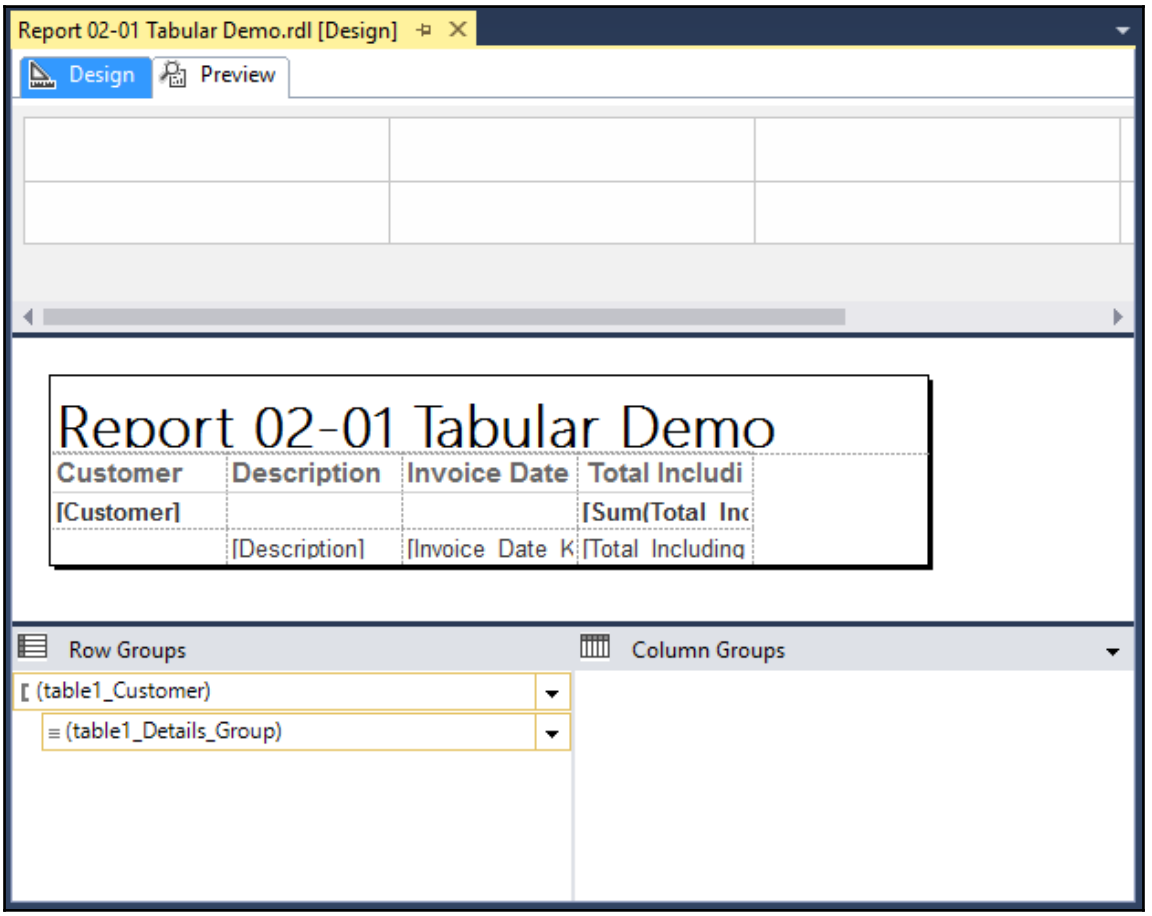


Figure 2.17

24. Next, we'll set properties about the report itself. Use the **Report** menu in SSDT, and select the **Report Properties** menu option. When you do, SSDT displays an easy-to-understand dialog:

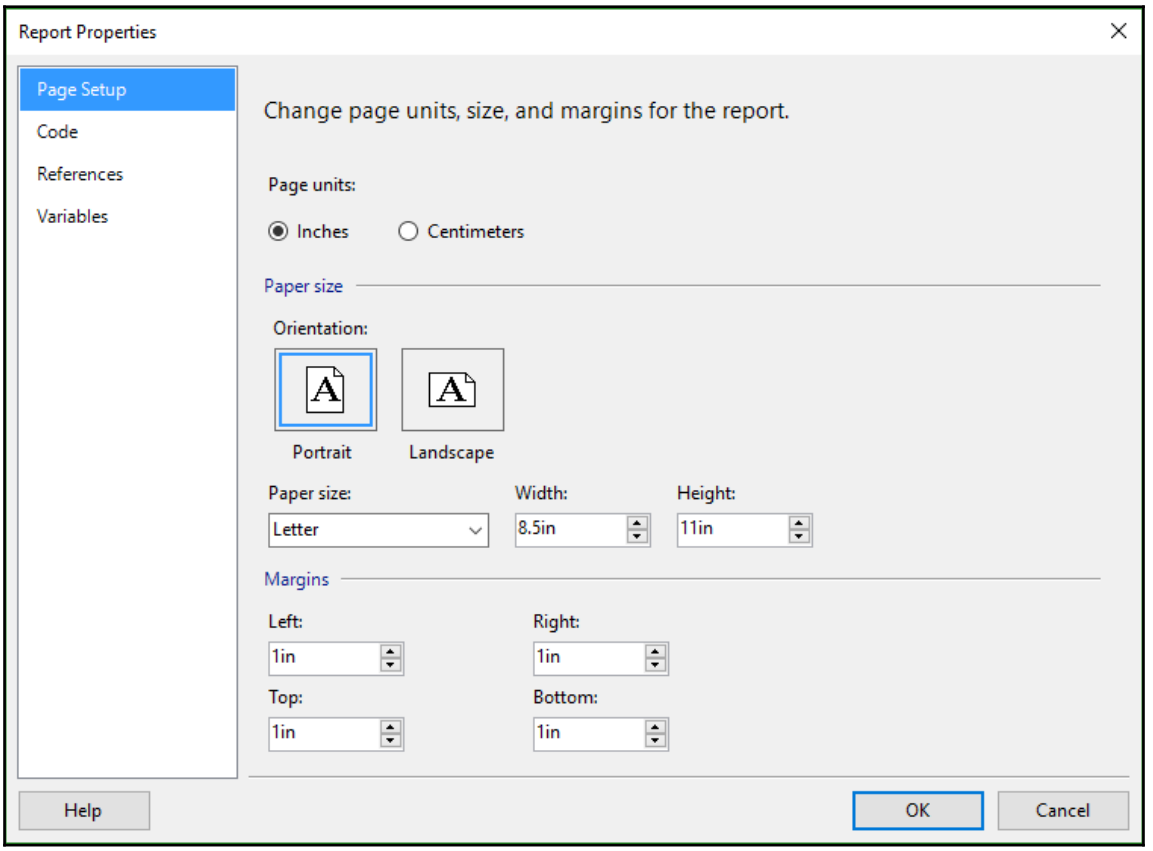


Figure 2.18

25. Alternatively, you could set these properties individually through the properties pane on the lower right. Click on **Cancel** to close this dialog.

26. In the properties pane (by default in the lower-right side of SSDT), click on the dropdown at the top, as shown in the following figure:

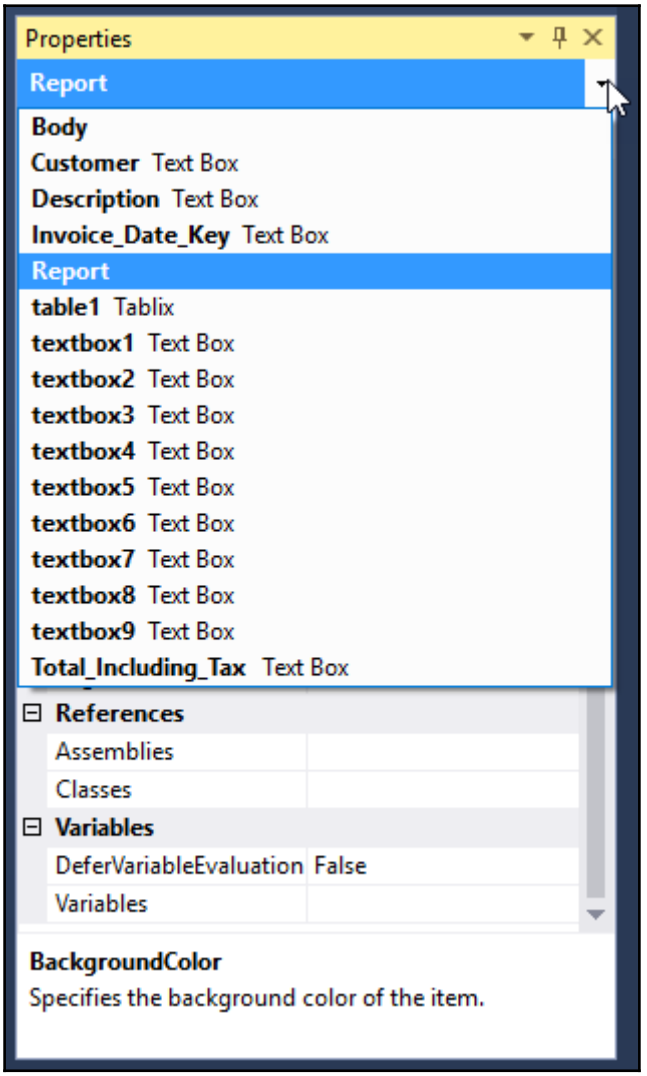


Figure 2.19

27. Select the **Report** item and note all the properties for the report itself, such as page size and orientation. Note that any of the report items or report sections on the report can be selected in this fashion.

28. In addition to using the dropdown, you can click on a component in the report designer area and the properties dialog will update to reflect the properties for the component you clicked on. To select the report itself, click on the white area outside of the main report body. The body is the component with the drop shadow:

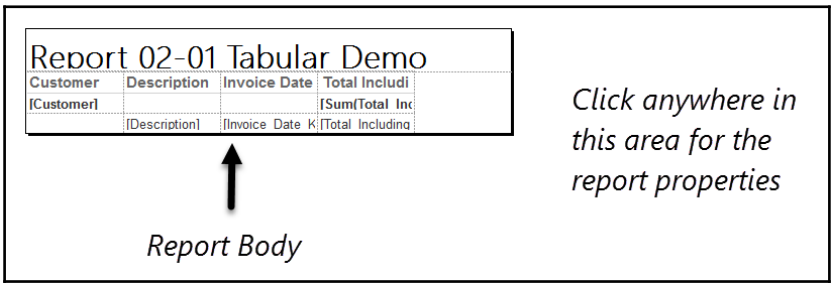


Figure 2.20

29. When you click on the **Preview** tab, SSDT automatically saves your report:

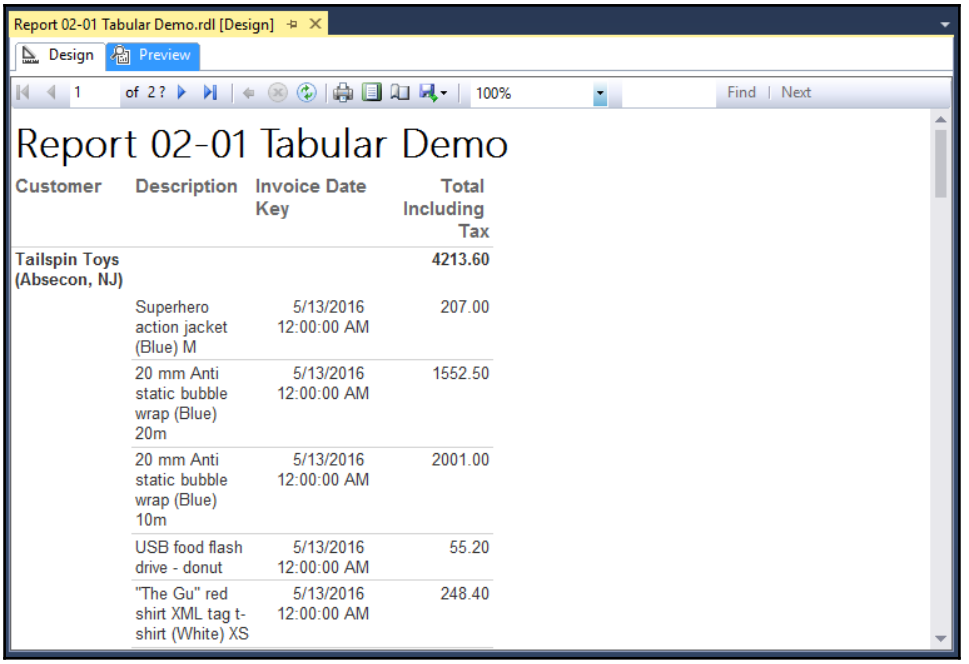


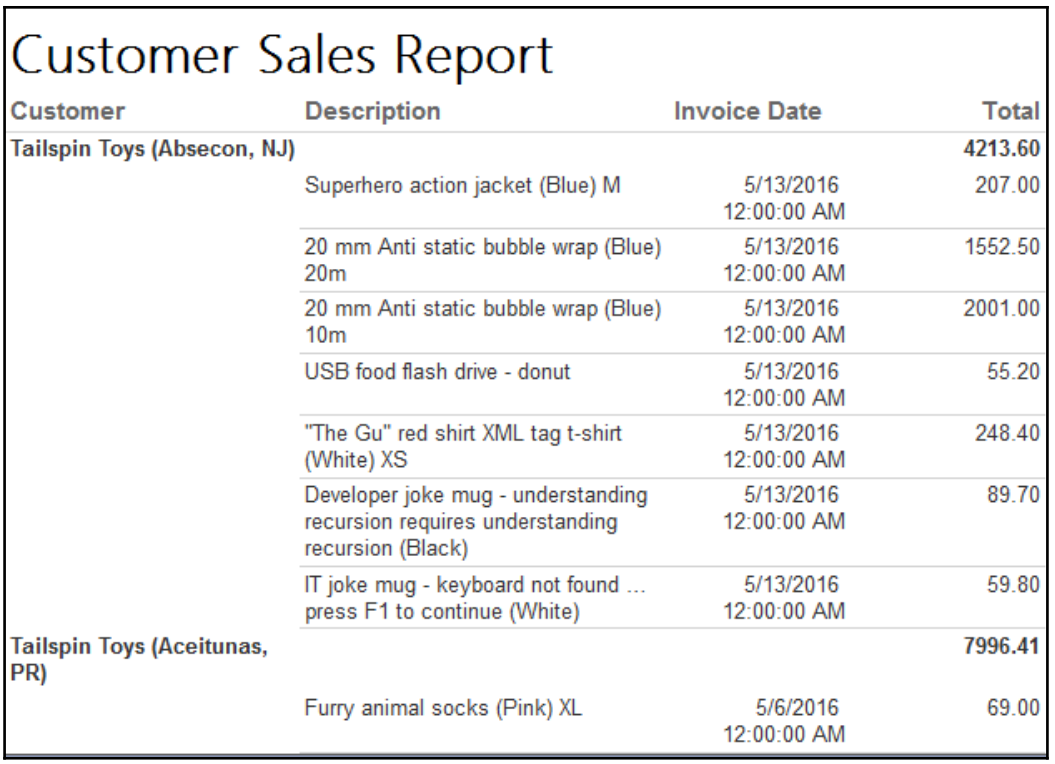
Figure 2.21



Note the toolbar at the top of the report. It lets you move to each page, refresh the report data, print it, export it, and more. Hover the mouse over each icon to see the tool tip for that function. As these are straightforward, we won't take detailed steps; however, take a moment to explore these various tools before continuing.

30. Next, we'll set the properties of the **Report Body**. Click on the **Designer** tab to return to design mode.
31. Click on the **Report Body** (or pick it from the dropdown in the properties area).
32. Click inside the grid (anywhere will do). When you do, a set of grey bars appears to the left and above the grid. Click on the right one and drag to expand the width. We don't have to worry about height, as Reporting Services will automatically expand the height to take up what it can on a page.
33. In the grey bar, hover the mouse over a divider between two columns until the cursor changes to resize mode. Resize each column to a reasonable size, adjusting until you've taken up the width of the body.
34. Click on the report title, and change `Report 02-01 Tabular Demo` to read `Customer Sales Report`. Note that in the SSDT toolbar, the text editor toolbar now activates and allows you to change things such as the font type, size, and more. These can also be set via the properties window.
35. Now we'll ensure that the column headers won't wrap as they did in the original preview. Click in the **Invoice Date Key** cell, and change the text to just `Invoice Date`. Repeat this for **Total Including Tax**, changing it to just `Total`.

36. Click on the Preview tab to preview the report:



Customer Sales Report

Customer	Description	Invoice Date	Total
Tailspin Toys (Absecon, NJ)			**4213.60**
	Superhero action jacket (Blue) M	5/13/2016 12:00:00 AM	207.00
	20 mm Anti static bubble wrap (Blue) 20m	5/13/2016 12:00:00 AM	1552.50
	20 mm Anti static bubble wrap (Blue) 10m	5/13/2016 12:00:00 AM	2001.00
	USB food flash drive - donut	5/13/2016 12:00:00 AM	55.20
	"The Gu" red shirt XML tag t-shirt (White) XS	5/13/2016 12:00:00 AM	248.40
	Developer joke mug - understanding recursion requires understanding recursion (Black)	5/13/2016 12:00:00 AM	89.70
	IT joke mug - keyboard not found ... press F1 to continue (White)	5/13/2016 12:00:00 AM	59.80
Tailspin Toys (Aceitunas, PR)			**7996.41**
	Furry animal socks (Pink) XL	5/6/2016 12:00:00 AM	69.00

Figure 2.22

37. Return to the Design mode, click in the cell that reads [**Customer**], and drag it so that the selection includes the two columns to the right, the empty areas under **Description**, and **Invoice Date**. Right-click, and from the pop-up menu, pick **Merge Cells**. The new layout should resemble the following figure:



Customer Sales Report

Customer	Description	Invoice Date	Total
[Customer]			**[Sum(Total_Including_Tax)]**
	[Description]	[Invoice_Date_Key]	[Total_Including_Tax]

Figure 2.23

38. Before previewing, let's fix the date. Right-click in the cell with **[Invoice Date Key]**, and from the pop-up menu, select **Text Box Properties**.
39. Click on the **Number** option on the left.
40. In the middle under **Category**, select **Date**. To the right, a list appears with the many predefined date formats Reporting Services supports, as shown in the following figure:

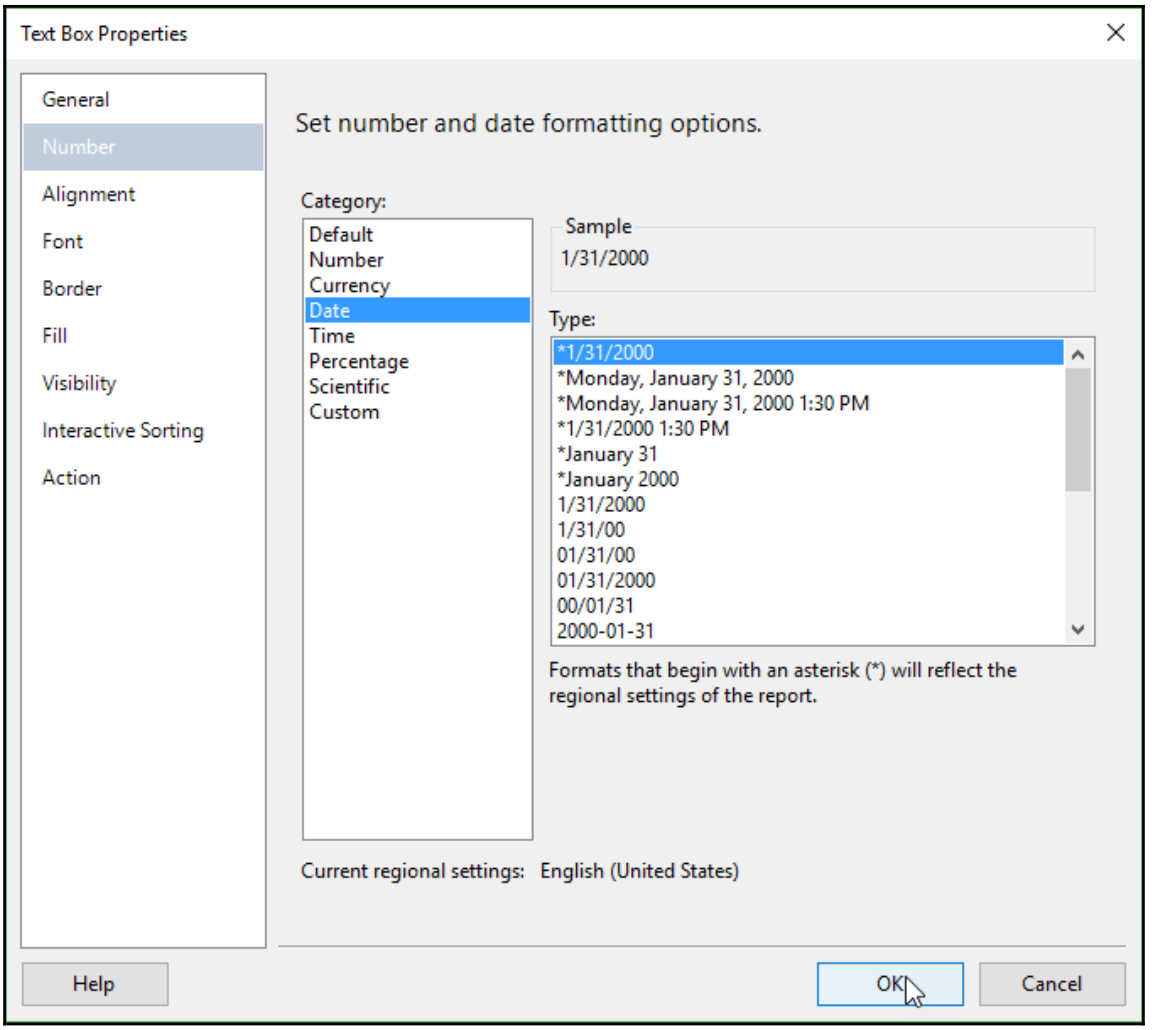


Figure 2.24

41. Select a date option appropriate for your need; here we are taking the top option. Note that the top entries in the list begin with an asterisk (*). These denote choices that will be set based on the regional settings of the host machine. For example, the next figure reflects the settings of a computer set to a US region, and shows the date in month/day/year format. Had this computer been set to the UK region, it would show day/month/year.
42. Click on **OK**. Then preview your report again, as shown in the following figure:

Customer Sales Report

Customer	Description	Invoice Date	Total
Tailspin Toys (Absecon, NJ)			**4213.60**
	Superhero action jacket (Blue) M	5/13/2016	207.00
	20 mm Anti static bubble wrap (Blue) 20m	5/13/2016	1552.50
	20 mm Anti static bubble wrap (Blue) 10m	5/13/2016	2001.00
	USB food flash drive - donut	5/13/2016	55.20
	"The Gu" red shirt XML tag t-shirt (White) XS	5/13/2016	248.40
	Developer joke mug - understanding recursion requires understanding recursion (Black)	5/13/2016	89.70
	IT joke mug - keyboard not found ... press F1 to continue (White)	5/13/2016	59.80
Tailspin Toys (Aceitunas, PR)			**7996.41**
	Furry animal socks (Pink) XL	5/6/2016	69.00
	Superhero action jacket (Blue) 3XL	5/24/2016	78.20
	10 mm Double sided bubble wrap 20m	5/6/2016	2070.00
	Shipping carton (Brown) 457x457x457mm	5/6/2016	603.75
	Office cube periscope (Black)	5/6/2016	1063.75
	32 mm Double sided bubble wrap 10m	5/5/2016	1012.00
	USB food flash drive - shrimp cocktail	5/5/2016	44.16

Figure 2.25

43. Next, we will move the totals area to the bottom. Click in the grid, and note the look of the grey row bars on the left. The detail line is denoted by three horizontal bars. The customer is considered a row group. These have a relationship with the row group's panel at the bottom, as shown in Figure 2.26:

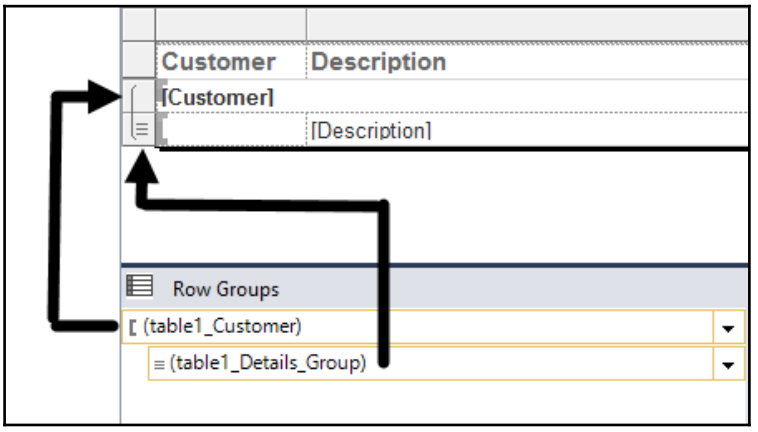


Figure 2.26

44. To add another row, right-click on the three bars that indicate the details group (the same row as **[Description]** before); then, from the pop-up menu, go to **Insert Row | Outside Group – Below**:

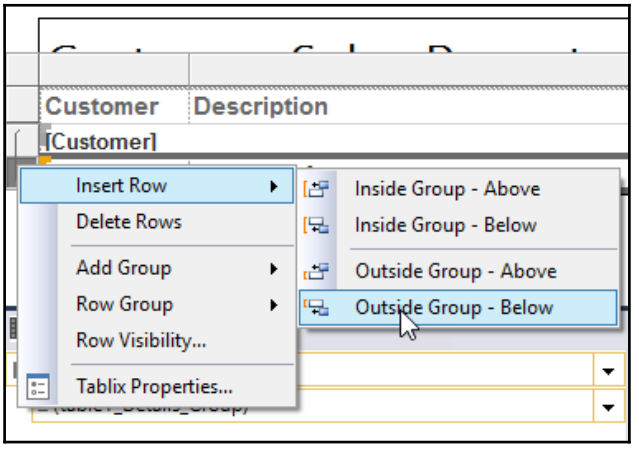


Figure 2.27

45. Now we'll copy the total field. Right-click in the **[Sum(Total_Including_Tax)]** field, pick **Cut**, and then go to the cell at the bottom. Right-click and **Paste**:
46. Now we'll ensure that the borders for this area are set to match the existing borders. Right-click on the cell and pick **Text Box Properties**.
47. Click the **Border** page.
48. Click on the left and right borders to unselect them, and ensure that the top and bottom borders are selected, as shown in the next figure:

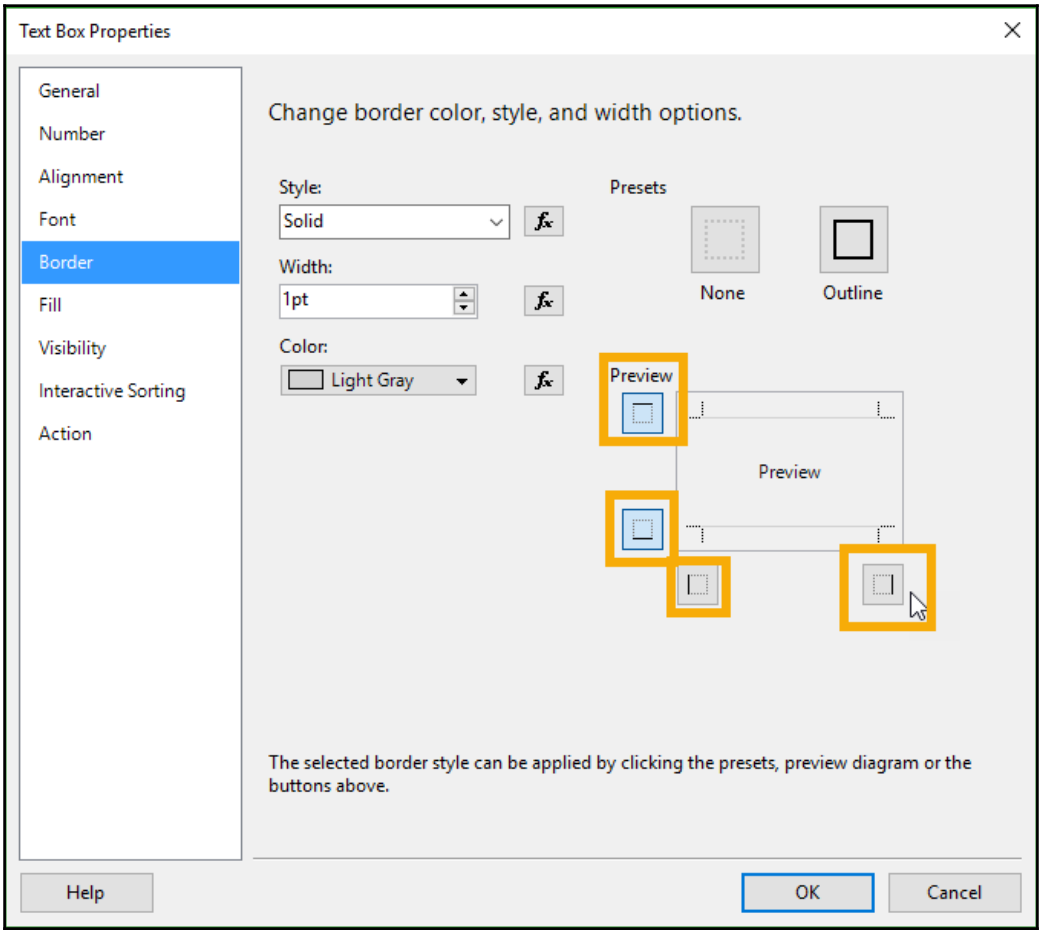


Figure 2.28

49. As you can see, the left and right borders have been deselected while the top and bottom remain selected. Click **OK** to apply the changes to the border.
50. On the new row, click in the leftmost cell and, while holding the mouse, drag to highlight three cells (exclude the one holding the total). Release the mouse, right-click, and pick **Merge Cells**.
51. Right-click on your new large cell and pick **Text Box Properties**.
52. We want to display a phrase like `Total Sales`. However, if the report group were to span many pages, you might forget which group's total sales you were looking at. Hence, we are going to use our first formula.
53. On the **General** page, to the right of the **Value** field is a small *fx* button, as seen here:

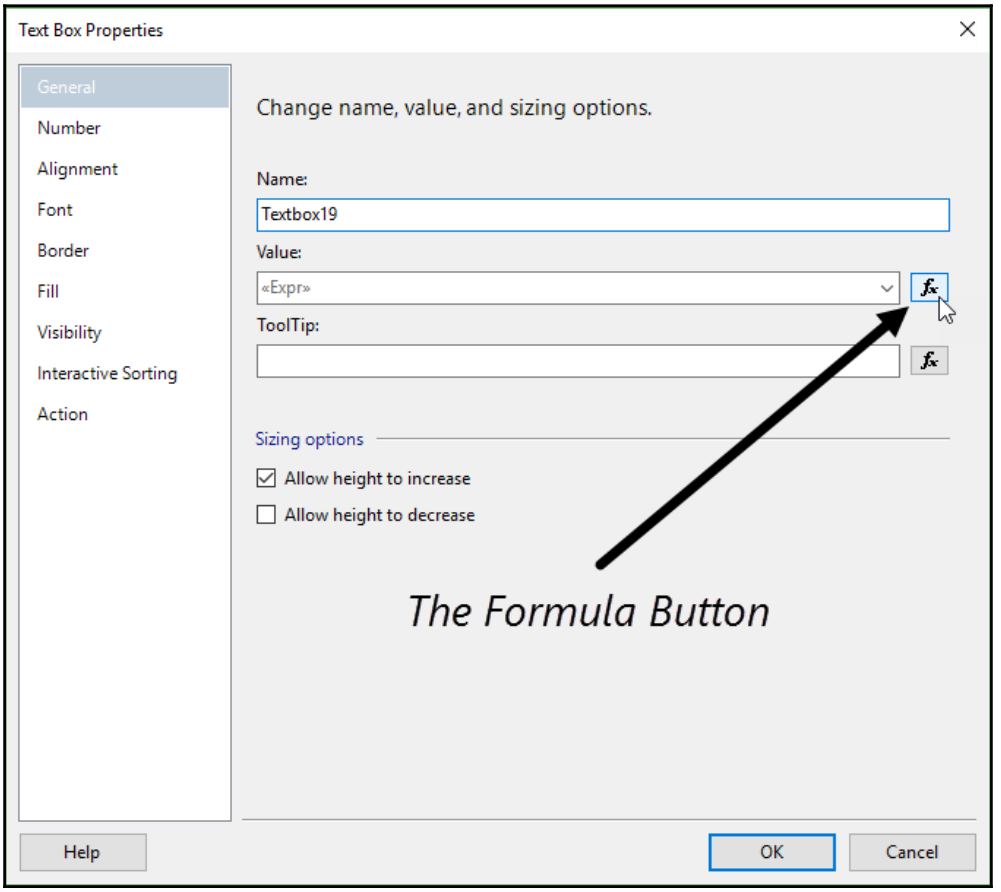


Figure 2.29

54. This opens up a new dialog, called the **expression editor**. The area at the top of the expression editor is for your expression. At the bottom are three panes that will assist you in writing your formula. Begin by typing in:

```
= " Total for customer " +
```

55. That will be an equal to sign, quotation mark, five spaces, the words **Total for customer,** followed by another single space, closing quotation mark, and a plus sign. SSRS has a rule that all formulas must begin with an equal to sign. The five spaces are just to indent the text a little when it is displayed; this is optional based upon user preference. The static text comes next, followed by the plus sign.
56. Now we want to add the field to be displayed. In the **Category** section on the lower left, click on **Fields (DataSet1)**. In the **Values** area on the right, double-click on the **Customer** field. It should now be added to the formula in the panel at the top, as seen in the following figure:

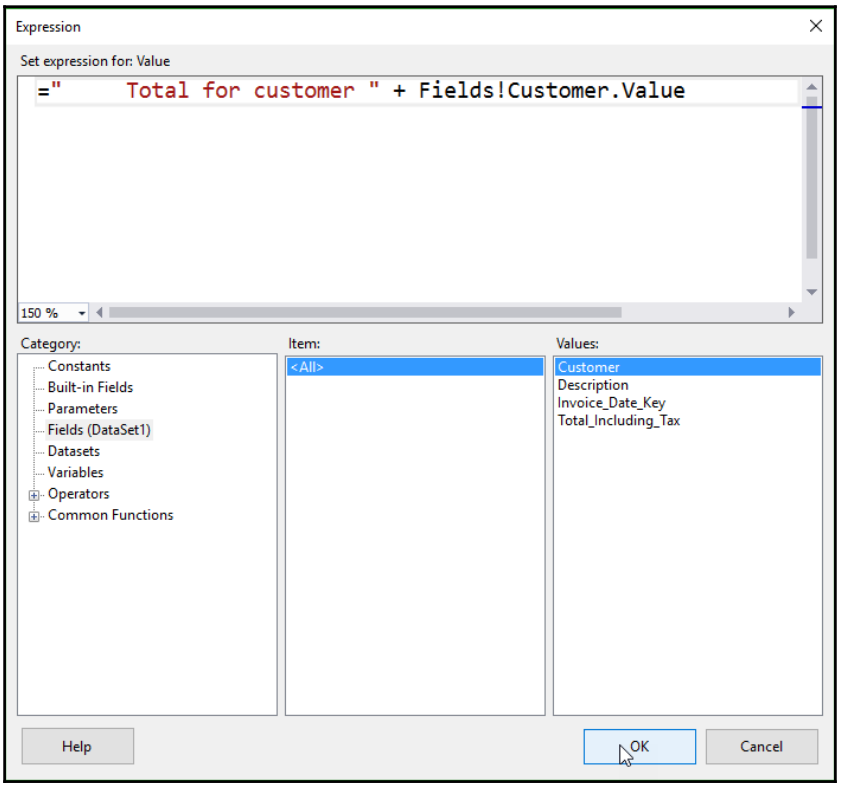
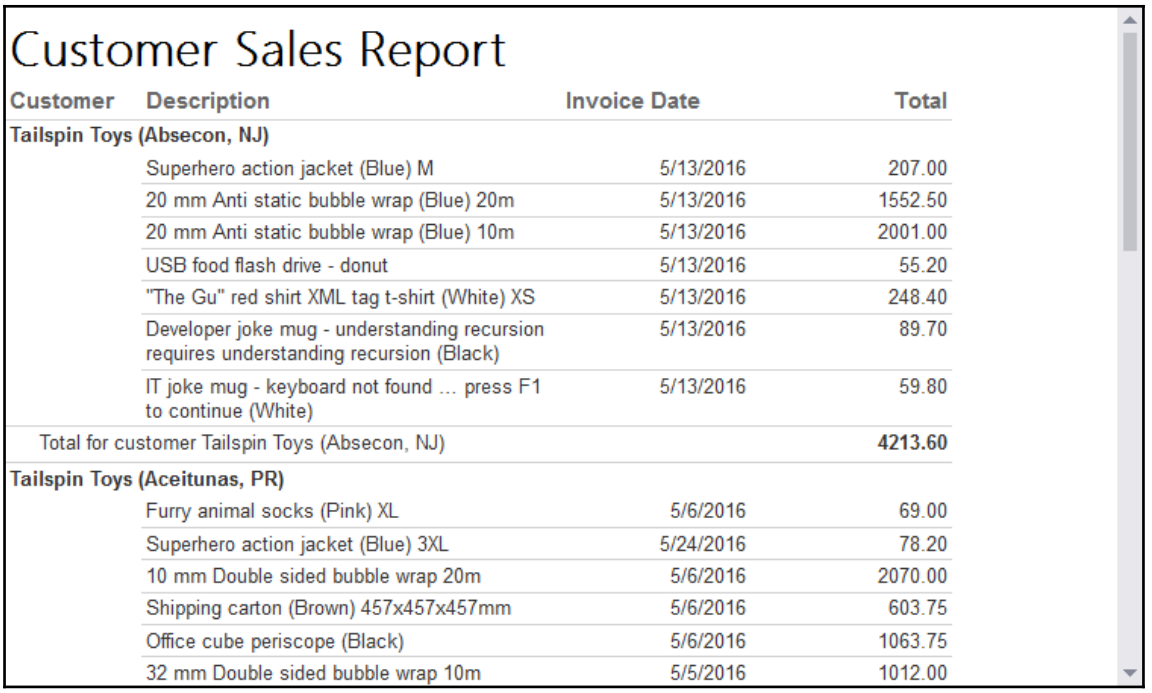


Figure 2.30

57. Click on **OK** to save the formula.
58. Check the borders again before you close the **Text Box Properties** dialog. When you merge cells, SSRS throws away any formatting, so you'll want to make sure the top and bottom border are set using the technique you read a moment ago.
59. Once you're happy with the borders, click on **OK** to close the **Text Box Properties** and then on **Preview** to run the report:



Customer Sales Report

Customer	Description	Invoice Date	Total
Tailspin Toys (Absecon, NJ)			
	Superhero action jacket (Blue) M	5/13/2016	207.00
	20 mm Anti static bubble wrap (Blue) 20m	5/13/2016	1552.50
	20 mm Anti static bubble wrap (Blue) 10m	5/13/2016	2001.00
	USB food flash drive - donut	5/13/2016	55.20
	"The Gu" red shirt XML tag t-shirt (White) XS	5/13/2016	248.40
	Developer joke mug - understanding recursion requires understanding recursion (Black)	5/13/2016	89.70
	IT joke mug - keyboard not found ... press F1 to continue (White)	5/13/2016	59.80
Total for customer Tailspin Toys (Absecon, NJ)			**4213.60**
Tailspin Toys (Aceitunas, PR)			
	Furry animal socks (Pink) XL	5/6/2016	69.00
	Superhero action jacket (Blue) 3XL	5/24/2016	78.20
	10 mm Double sided bubble wrap 20m	5/6/2016	2070.00
	Shipping carton (Brown) 457x457x457mm	5/6/2016	603.75
	Office cube periscope (Black)	5/6/2016	1063.75
	32 mm Double sided bubble wrap 10m	5/5/2016	1012.00

Figure 2.31

How it works…

This recipe covered a lot; as your first report, there were many steps. It started by adding a new report, which launched the **Report Wizard**. The wizard is a great place to start as it automates the creation of datasets and the placing of various report items on the report such as textboxes and tables.

As the first step in the wizard, we had to indicate where we were pulling data from. In this book we'll be primarily using SQL Server; however, SSRS supports a wide variety of platforms to pull data from. This process builds the connection string SSRS will use to communicate with the source system.

An option during this process is to make this a *shared data source*. A shared data source exists as an object within your project. All the reports in the project could then refer to the shared data source to obtain the connection string. This is a great option as when the reports are deployed to an environment, such as the testing server, the shared data source can be set to work with the testing database. Later, when the reports are deployed to a production environment, the shared data source can be pointed to the production database and all the reports using the shared data source will automatically now pull from production.

Leaving the shared data source option unchecked, as we did here, means the data source is embedded in this report. As the report is deployed to different environments, you will have to open up that report and update its data source information.

The majority of the time you will want to use a shared data source, as we'll do throughout the book. On occasion, you will have a report that is pulling from a special source, and it will be the only report you plan to write that will do so. In that scenario, you would likely choose to embed the source in the report, which we've done for this first report to illustrate the technique.

After establishing *where* to get the data from, we then need to tell SSRS *what* data to retrieve. While we showed the query builder, this is an option best suited only to a novice in the SQL language. If you are going to be working with SSRS on a regular basis, you are encouraged to become familiar with SQL.

In this example, the report author used SQL Server Management Studio to develop the query and pasted the finished query into the query area in Figure 2.12.

The report wizard is capable of generating two types of reports, Tabular and Matrix. The Tabular style is the most common, and is simply a line item listing of data. When designing a table, as shown in Figure 2.14, you can arrange the placement of data into the table in any order you wish. It also has the option to group data, allowing you to break the report into smaller chunks and include subtotals.

In Figure 2.15, you have several choices. First is the format, Stepped or Block. This is strictly a visual style, determining whether the report should use gridlines (block) or not. Stepped is the most common, however; use your own company guidelines to affect your choice. The next option is pretty straightforward, whether or not to have SSRS generate subtotals for the groups. The final option, **enable drilldown**, is important as it has an effect on the report when it is generated. When enabled, SSRS will collapse each group to show only the group name and subtotal. It will have a small plus sign to the left of the group, allowing the user to expand it in order to see the details. If your users will primarily be using the report in interactive mode and not printing it, then this is a good option to select.

If the report will primarily be printed, or exported to a static format such as a PDF or Microsoft Word document, then leave this unchecked in order for the user to see all the details.

We've finally reached the last stage of the wizard, in which we give the report a name. This is the name SSRS will use as the filename to save the report as. In addition, it will also use it as a title in the report. It is rare that the filename and the report title will be exactly the same, so use the name you want for the file, as it is easy to update the report title later. Note that there is also an option to go ahead and preview the report, allowing you to see the output of the wizard immediately.

After the report is generated, you are shown the report designer. It is broken into three areas, as shown in Figure 2.17. The top section, with the empty grid of boxes, is for the placement of parameters, which will be covered later in this chapter, in the *Managing and configuring parameters* recipe.

The middle area was the main designer, where we determined what the report should actually look like. Below the report, you will notice two areas, **Row Groups** and **Column Groups**. In this area, you can specify how report areas will be grouped and, within the groups, produce totals. The Column Groups area is empty because this is a tabular report, and hence we have no columns.

Referring back to Figure 2.17, you will note the report is to be grouped on Customer, and for each customer it includes which properties and subgroups may be present. The downward arrow at the end of each entry brings up a menu, through which you can add or remove sections to the group as well as access the group properties. It is a bit early in your reporting services career to dive into these advanced options, but you may wish to take a quick look as many are self-explanatory.

Just because the wizard completed doesn't mean your work is done, especially if you plan to print the report. You'll want to set the page size, margins, page orientation, and other properties. You'll also need to resize the columns in order to avoid the wrapping of report items. As part of resizing the columns, we want to format the data within those columns, as we did in this recipe.

The report wizard likes to place subtotals at the top of the groups. This is a good choice when you plan to enable drill down. For printed reports, however, it tends to throw off many users, who expect totals to appear at the bottom. Hence we covered the steps needed to create a nice subtotals section at the bottom, complete with text and moving the subtotals from the group header to the group footer.

As part of creating the group footer, we created a *Totals for* expression using the **fx** button to launch the SSRS Expression editor. You may have noticed these **fx** buttons in other places. Reporting Services is incredibly powerful in that it will let you use formulas for a large majority of its values. Its Expression language is incredibly rich and allows you to control many aspects of your report.

As you can see from the amount of work, the report wizard can only take you this far. There will always be additional cleanup needed in order to bring the report to professional standards.



Before writing your first report, your team should agree upon and document a set of standards. What font should be used in each area? Should the report title be left, center, or right aligned? Should subtotals go at the top or bottom of a group? By creating a standard up front, you ensure your reports will have a consistent look and feel, something that will be greatly appreciated by your management.

There's more…

When you run through the SSRS Report Wizard, it is automating a lot of things for you. Under the hood, it is taking tools from the *Toolbox* and placing them on the report surface. The Toolbox (by default) is shown on the left side of SSDT; if you don't see it, just go to **View** | **Toolbox** from the menu:

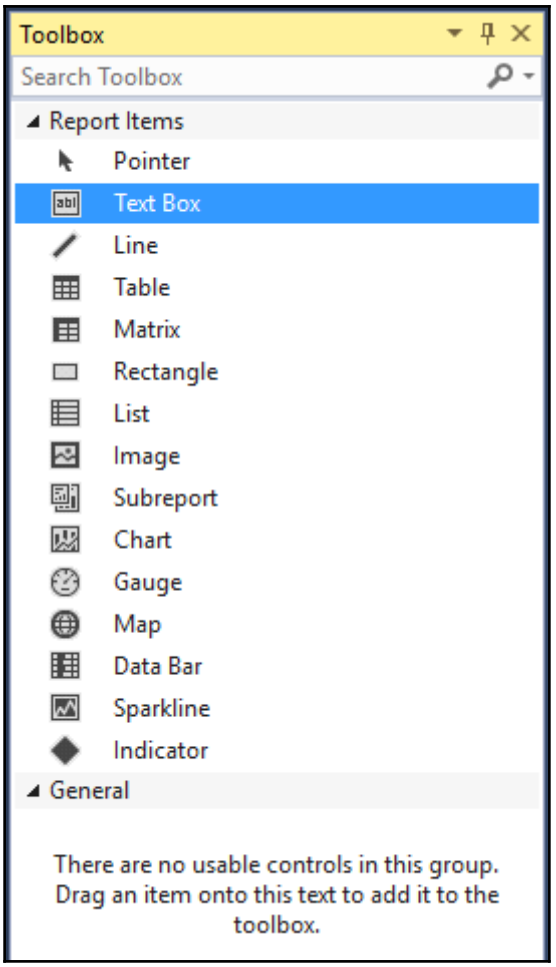


Figure 2.32

The **Report Wizard** first took a **Text Box** control and placed it at the top of the report with the report title in it. It then inserted a **Table** control, placing Text Box controls in each cell. As you can see though, there are many other types of controls we can use on our reports, and we'll be looking at these in `Chapter 3`, *Advanced Report Authoring with SQL Server Data Tools*.

How does it know which data to put in which area? First, you'll want to look at the Report Data tab, also in the toolbox area on the left. If you don't see the tab for Report data, go to **View** | **Report Data** from the menu. The menu is context sensitive to what you are doing in SSDT. If you don't see a **Report Data** option, click inside the report designer; then check again, and it should be there:

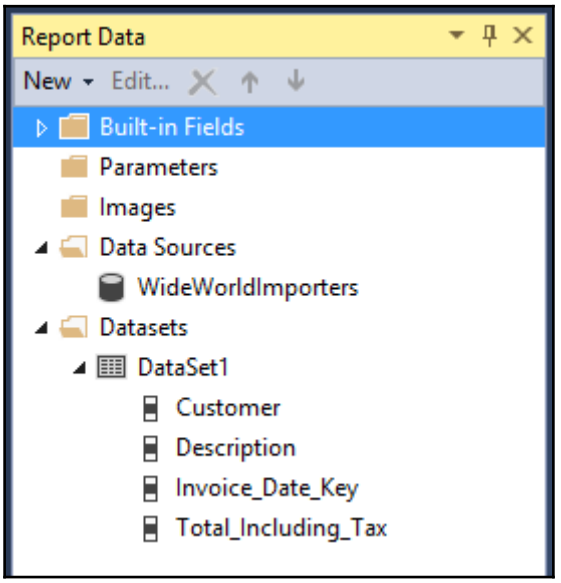


Figure 2.33

The report data pane, shown in Figure 2.33, represents the data being supplied to the report. You can view the properties of any of these objects shown in Figure 2.33 by right-clicking and picking properties. If you wanted to come back and put an additional field, you could, for example, right-click on **DataSet1**, pick properties, and update the query to add the field.

The connection between the controls and the report data is made in the **Text Box Properties** dialog. Return to the grid, right-click on the [**Description**], and choose **Text Box Properties**:

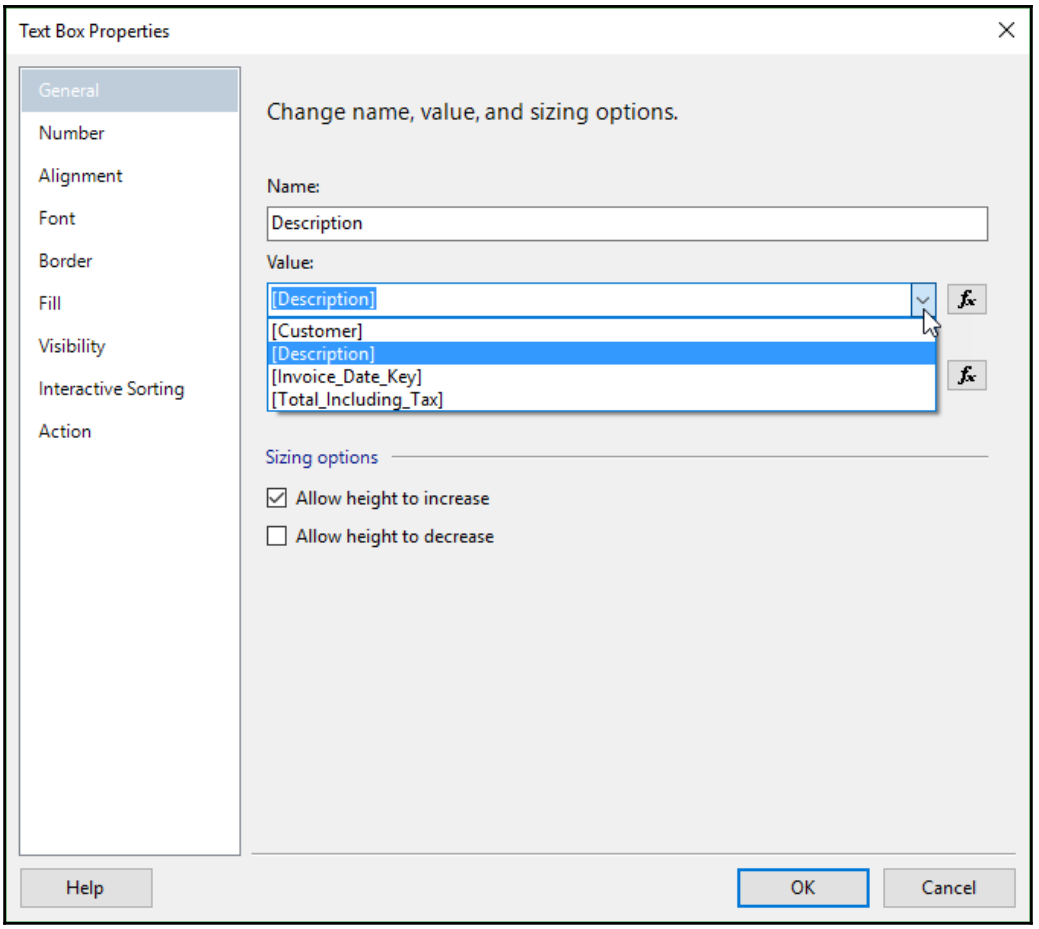


Figure 2.34

Right in the middle is a field labelled **Value**. In the dropdown, you could select any field from the report data shown in the previous figure 2.34. It is through the Value field that SSRS knows which field from the Report Data to place into this tool.

Note too that the *fx* formula option can be used should you need to combine fields from the data source, perhaps a *Last Name, First Name* scenario.

While you could have done all of the aforementioned steps manually, I think you'll agree that having the Report Wizard is a great benefit to get up and going with your report.

Creating reports with a matrix data region

In the previous recipe, you created a report using a Table. A table is great when an entire row is considered a single entity; all of the data on that row together represents what a user needs to know.

Consider a situation where a user is looking at data in aggregate. They are looking for a single data point, such as the grand total sales of product X for the current year. These reports are typically done as a *pivot table*, a style of report in which both the rows and columns have information and it is the intersection of the row and column that contains the valuable piece of data.

Excel is famous for creating these types of reports, and we can achieve the same result in SSRS by using a *Matrix*.

Getting ready

We will once again use the `WideWorldImportersDW` database for our demo. Rather than using the Query Designer in SSRS, we'll use SQL Server Management Studio to develop our query.

In addition to being a bit easier for an experienced developer, it can also be a useful tool with which to communicate to the end user prior to even starting the report. The output of the query can be sent to the end user, who can confirm that it is indeed the data they wanted. This can save time creating a report built from a faulty assumption.

For this report, we will create a matrix where we list the years across the top and the customers down the left on the rows. The intersection will be Total Sales Including Tax:

```
SELECT c.[Customer]
     , YEAR(s.[Invoice Date Key])   AS InvoiceYear
     , SUM(s.[Total Including Tax]) AS TotalSalesWithTax
  FROM [Fact].[Sale] s
  JOIN [Dimension].[Customer] c
    ON s.[Customer Key] = c.[Customer Key]
 GROUP BY c.[Customer], YEAR(s.[Invoice Date Key])
```

To get our totals, we'll leverage the power of SQL Server to SUM our values. We're also using the `YEAR` function to extract just the year of the `Invoice Date Key field`. Note the inclusion of the `Group By` in order for `SUM` to work.

Finally-and this is *very important*-the two columns with the `SUM` and `YEAR` functions applied have had *aliases* applied. They both have been given `AS` clauses with new column names. While SSMS will happily let you run a query without the aliases, simply displaying *(No column name)* in the output, SSRS will give an error if you omit the name of any of your columns.

How to do it…

1. Create a new report using the technique as described in *The designer tool – exploring SSDT* recipe earlier in the chapter.
2. On the next page of the wizard, design the query, paste in the query from the Getting ready section, and then click on **Next**:
3. Select the **Report Type** screen, where we'll change it to **Matrix**. As shown in figure 2.35, the image at the right updates to show a Matrix style report:



Figure 2.35

4. Next we come to the most important screen of the wizard, **Design the Matrix.** It is here that we let SSRS know which field is the column, which is the row, and which goes into the details. For this, we'll click on **InvoiceYear** and put it in the columns. Likewise, **Customer** will go in the Rows area, and finally **TotalSalesWithTax** into the Details section, as the next figure illustrates:

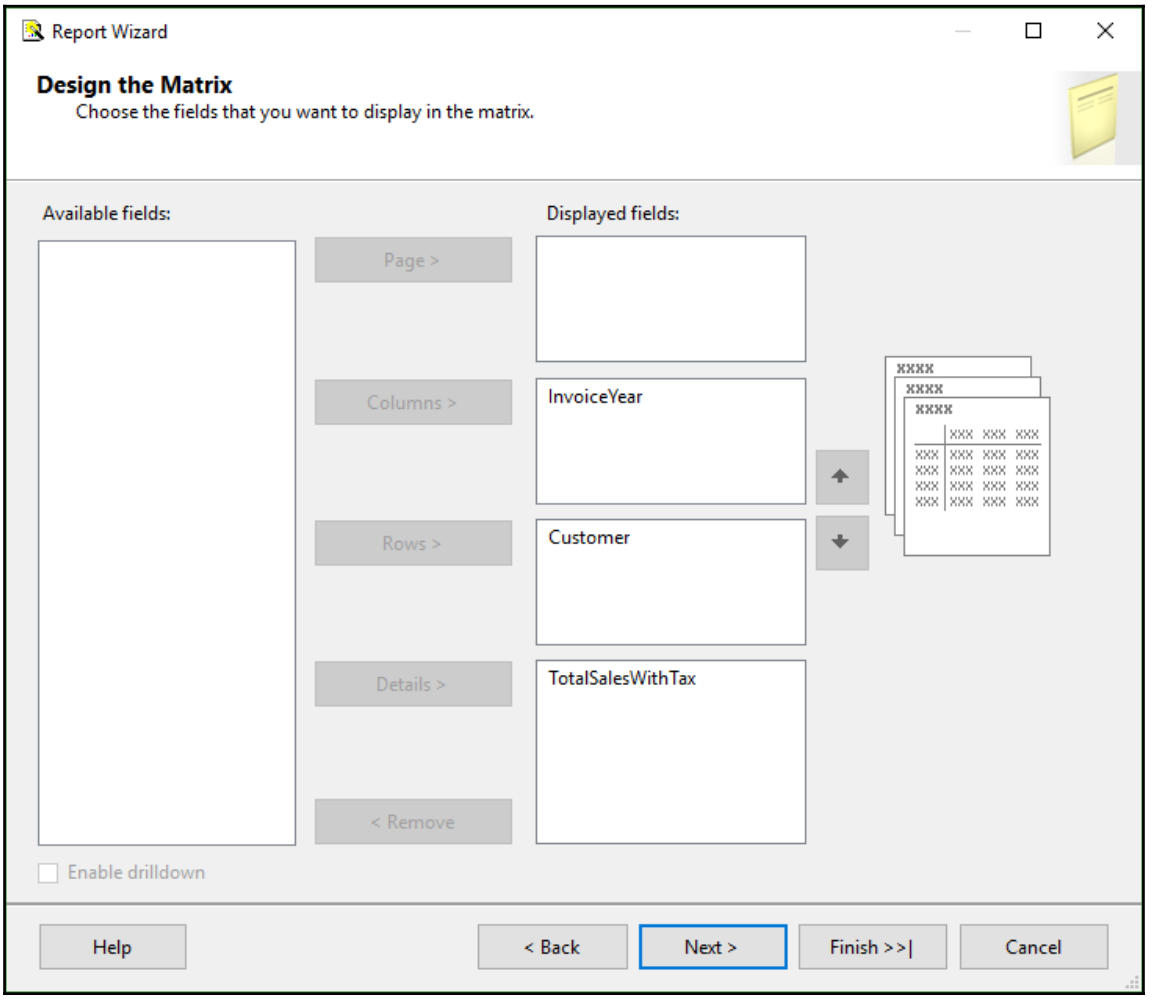
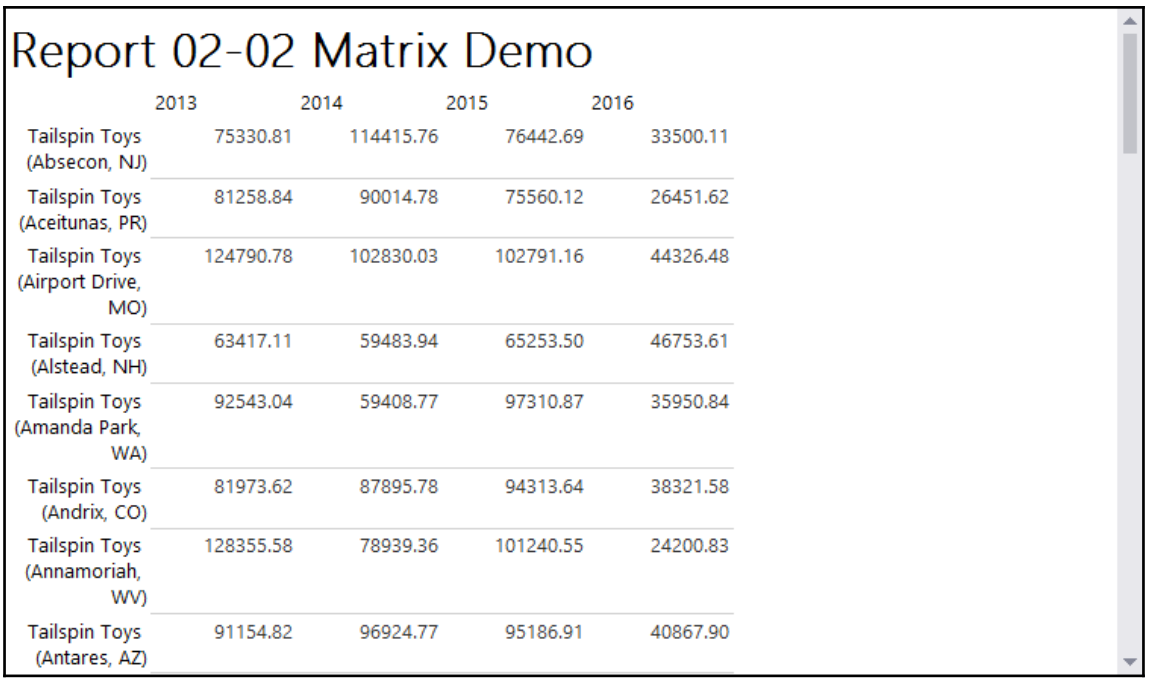


Figure 2.36

5. Click on next. Name the report `Report 02-02 Matrix Demo`. Let's go ahead and put a checkbox in the Preview report option so that we can see what the report created for us:



Report 02-02 Matrix Demo

	2013	2014	2015	2016
Tailspin Toys (Absecon, NJ)	75330.81	114415.76	76442.69	33500.11
Tailspin Toys (Aceitunas, PR)	81258.84	90014.78	75560.12	26451.62
Tailspin Toys (Airport Drive, MO)	124790.78	102830.03	102791.16	44326.48
Tailspin Toys (Alstead, NH)	63417.11	59483.94	65253.50	46753.61
Tailspin Toys (Amanda Park, WA)	92543.04	59408.77	97310.87	35950.84
Tailspin Toys (Andrix, CO)	81973.62	87895.78	94313.64	38321.58
Tailspin Toys (Annamoriah, WV)	128355.58	78939.36	101240.55	24200.83
Tailspin Toys (Antares, AZ)	91154.82	96924.77	95186.91	40867.90

Figure 2.37

6. While this actually looks pretty good, there are just three little things we'll fix. Return to the **Design** tab.
7. Make the customer column wider by clicking in a cell to make the row/column handles appear, and drag it to make it bigger.

8. Right-click on **[Sum(TotalSalesWithTax)]** and select **Text Box Properties**. Since this is a sales report, pick **Number**, and then in the **Category** list, **Currency**. When reports deal with large sums such as this one, the cents are usually left off, so set the **Decimal** places to 0. Then put a check mark in the **Use 1000 separator** option:

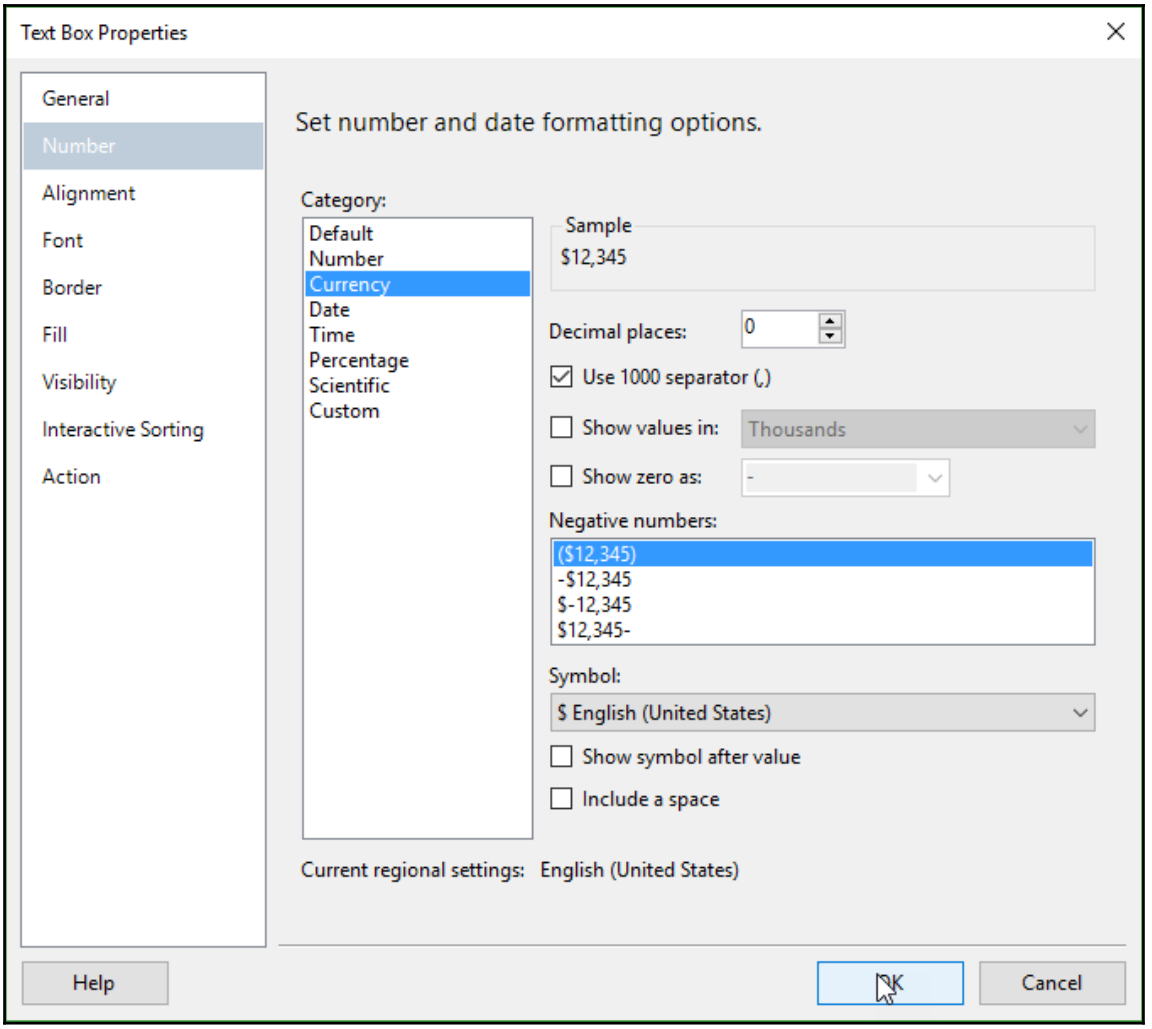


Figure 2.38

9. Since the sales amounts will be right aligned, we'll align the [InvoiceYear] header column to use center alignment. You could of course click on the cell, bring up Text Box Properties, go to the Alignment tab, and set the Horizontal alignment to Center. But a faster way is to simply click on the cell and in the SSDT toolbar:



Figure 2.39

10. If this toolbar isn't present in SSDT, simply go to **View** | **Toolbars** | **Report Formatting** from the SSDT menu.

How it works…

When using a Matrix, SSRS creates a pivot on the data, similar to the way the `PIVOT` function works in SQL. It understands how to correctly organize the rows and columns to provide the user the desired feedback.

The key driver occurred in Figure 2.36, where we instructed the wizard as to which data fields were in rows and columns; and in the details area, we placed the actual values. The values must be a numeric field that can be summed up.

Creating reports with a chart data region

Charts and graphs are an increasingly important component of reporting. They convey information in just a glance, with a simple picture. It is likely you will be asked to create many reports with graphs, combining them to create dashboards.

In this section, we'll start with a simple graph to convey sales month by month based on the invoice date.

Getting ready

In the `WideWorldImportersDW` database, the data will be coming from the sales table. The report will display the total, including tax, summed up by invoice month/year and then charted. We'll start with the following query:

```
SELECT s.[Invoice Date Key]
     , SUM(s.[Total Including Tax]) AS TotalSalesWithTax
  FROM [Fact].[Sale] s
 GROUP BY s.[Invoice Date Key]
```

How to do it…

1. Right-click on the Reports branch in the solution explorer.
2. Do *NOT* click on the **Add New Report** option in the pop-up menu. This launches the wizard, which does not support charts. Instead, select **Add** | **New Item**:

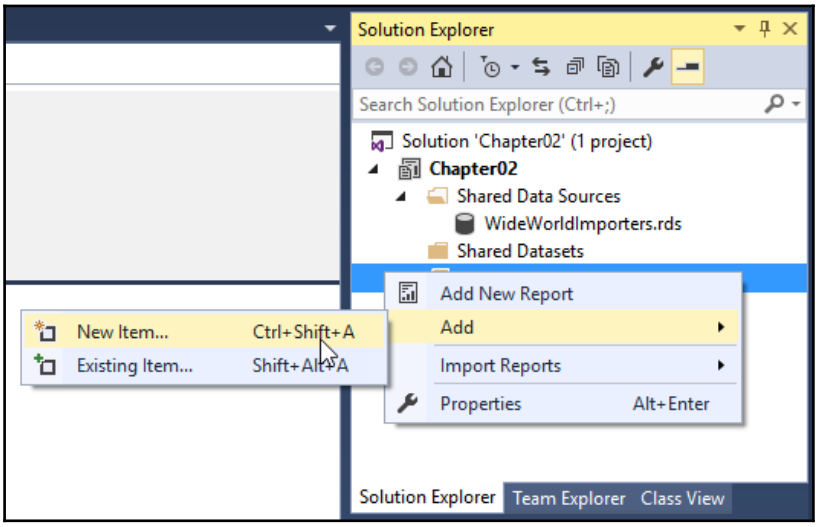


Figure 2.40

3. In the **Add New Item** dialog, click on **Report**, use a report name of `Report 02-03 Chart Demo.rdl`, and click on **Add**.
4. On the left side of SSDT, click on the **Report Data** toolbox. If you don't see it, use the **View** | **Report Data** menu option.

5. In the **Report Data** tool, right-click on the branch that reads **Data Sources**. In the menu, select the **Add Data Source…** option.
6. In the **Data Source Properties** dialog, give the data source a good name that represents where the data will come from. In this example, name it after the database holding the data, `WideWorldImportersDW`.
7. Pick the **Use shared data source reference** option, and in the dropdown, pick `WideWorldImporters`. Then click on **OK**:

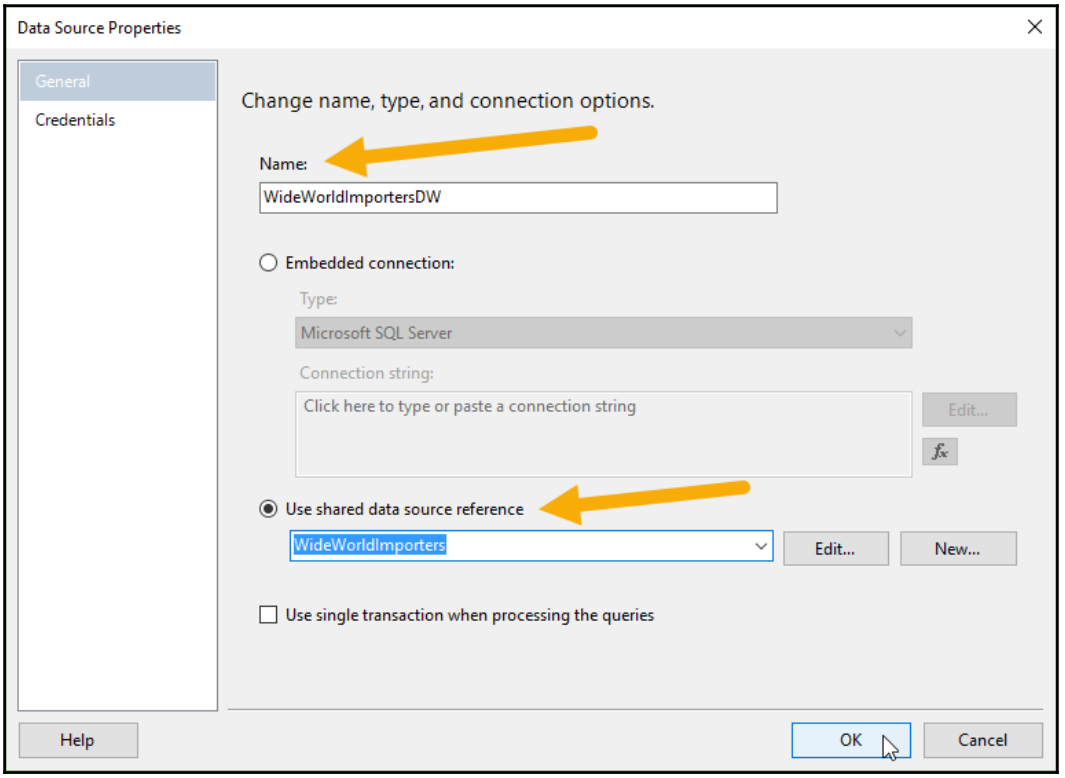


Figure 2.41

8. Right-click on the **Datasets** branch and click on **Add Dataset…**.
9. Give the data source a name; in Figure 2.42, we used `SalesByInvoiceMonth`.



Note that spaces are not allowed in the name.

10. Select the **Use a dataset embedded in my report** option.
11. For the data source, select **WideWorldImportersDW** in the dropdown.
12. For the query type, leave the default at **Text**. Then paste in the query from the *Getting ready* section of this recipe. Don't click on **OK** yet, as we still have more to do!

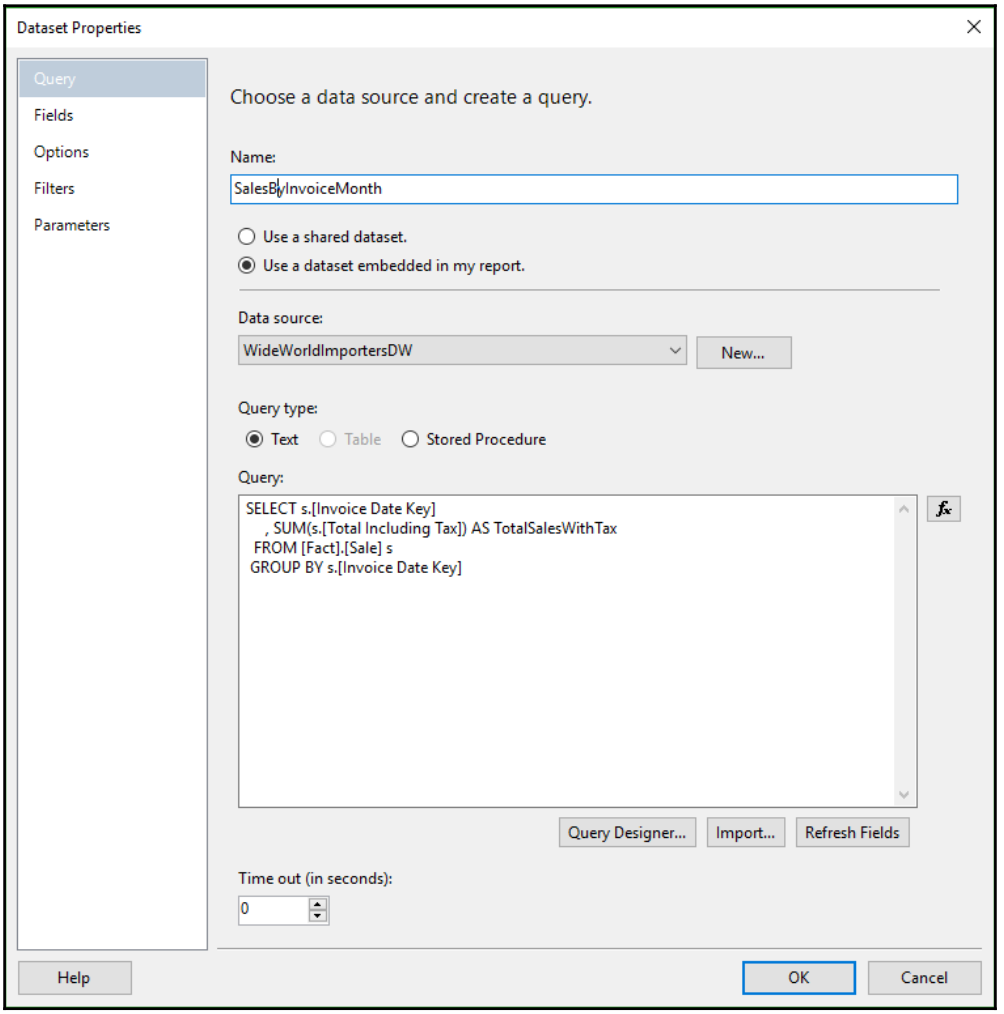


Figure 2.42

13. In order to sum up each month's sales, we need to add two fields. First, go to the **Fields** page of the **Dataset Properties** dialog.

14. The first field we will add will take the invoice date and create a nicely formatted Month/Year to use in the display of the chart, for example, *Jul 2016*. Click on the **Add** button at the top and then on **Calculated Field** from the menu.
15. A new line will appear. Under field name, enter `InvoiceDateMY`.
16. Click on the **fx** button on the right. In the Expression dialog, use the following expression. Be sure to type it all in one line:

```
= MonthName(Month(Fields!Invoice_Date_Key.Value),
True) + " " +
CSTR(YEAR(Fields!Invoice_Date_Key.Value))
```

17. This will extract the month name from the **Invoice_Date_Key** field. `True` tells the `MonthName` function to abbreviate the month name. The remaining text extracts the year from the same field. Note that there should be a single space between the quotation marks. Click on **OK** to close the **Expression** editor:

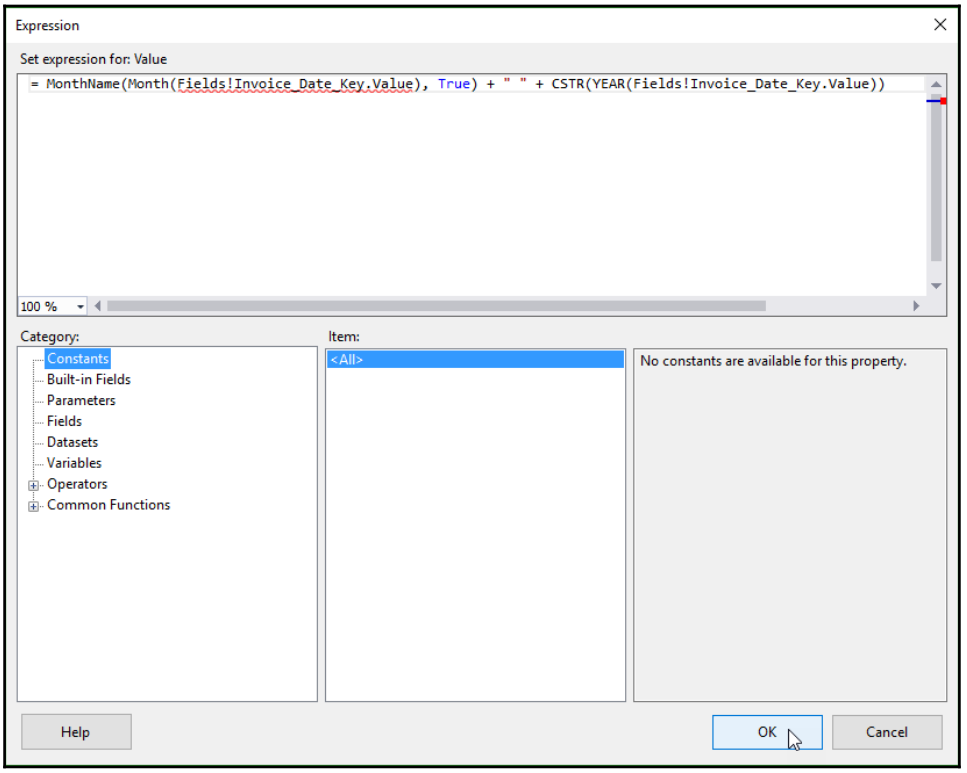


Figure 2.43

18. We now have a field that we can display. Unfortunately, if we attempt to sort by this field, we would get alphabetic sorting, that is Apr 2013, Apr 2014, Apr 2015 and so on. Thus we need to create another field we can use for sorting. Back at the **Dataset** Properties, again go to **Add | Calculated Field**.
19. In the name, use `InvoiceDateYM`; then click on the *fx* button.
20. Use the following formula in the Expression editor:

```
=CSTR(YEAR(Fields!Invoice_Date_Key.Value))
 + "-" + Right
("00" + CSTR(Month(Fields!Invoice_Date_Key.Value)), 2)
```

21. This will extract the year, converting it to a string. We then add a dash into the string. Finally, the month will be converted to a number, then to a string. It will then have 00 added to the front, so we can gather the right most two characters. This will ensure that the month numbers are in the format 01, 02, 03… 10, 11, 12- necessary to use in sorting. Click on **OK** to close the expression editor.
22. With your dataset properties looking like the next figure, click on **OK** to close the **Dataset Properties** window:

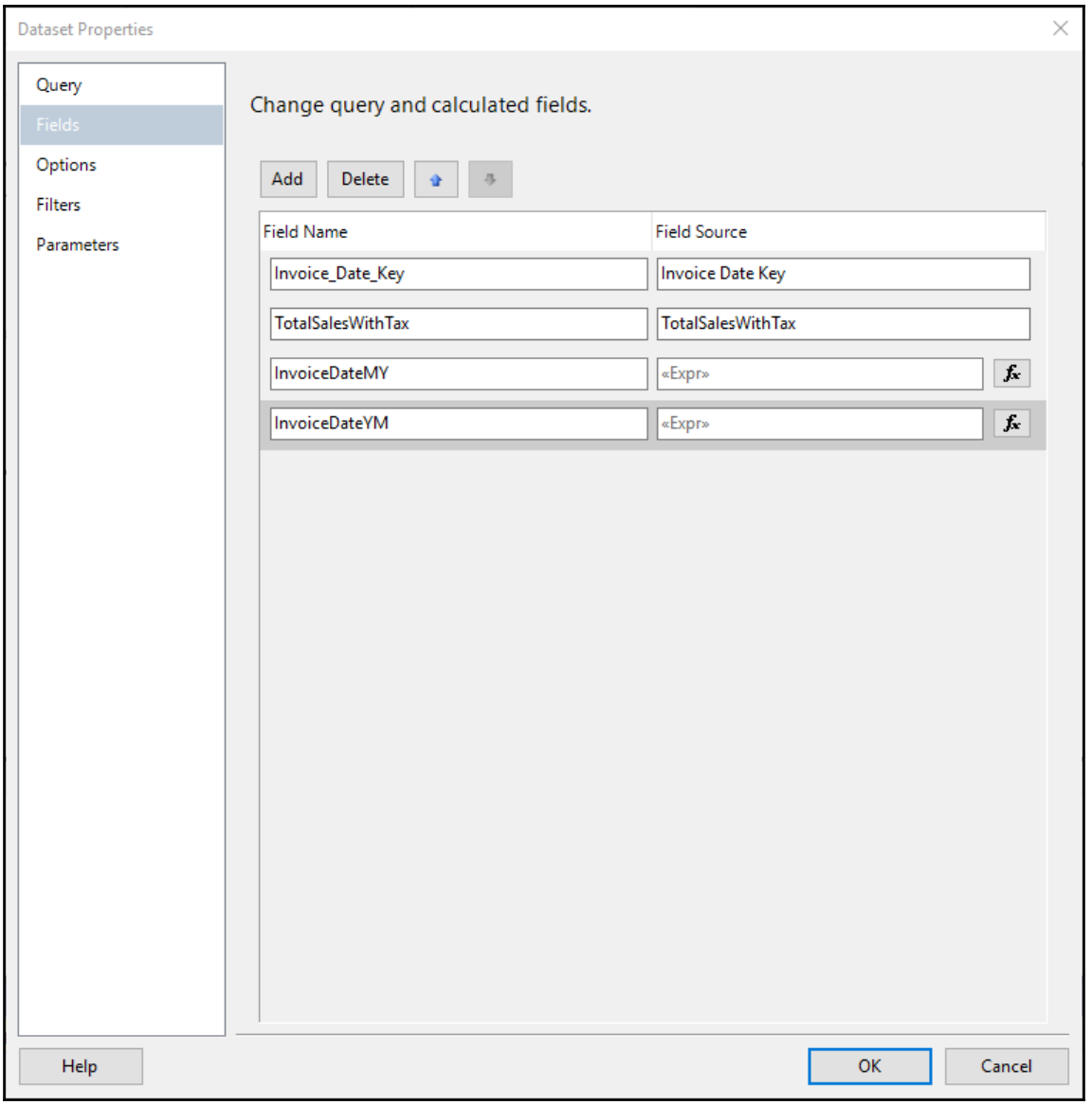


Figure 2.44

23. In the tools area to the left, select the tab with **Toolbox**. If you don't see it, use **View** | **Toolbox** in the SSDT menus.

24. Find the **Chart** control. You can add it to the report by one of two methods. Double-click on it, or drag and drop it onto the report:

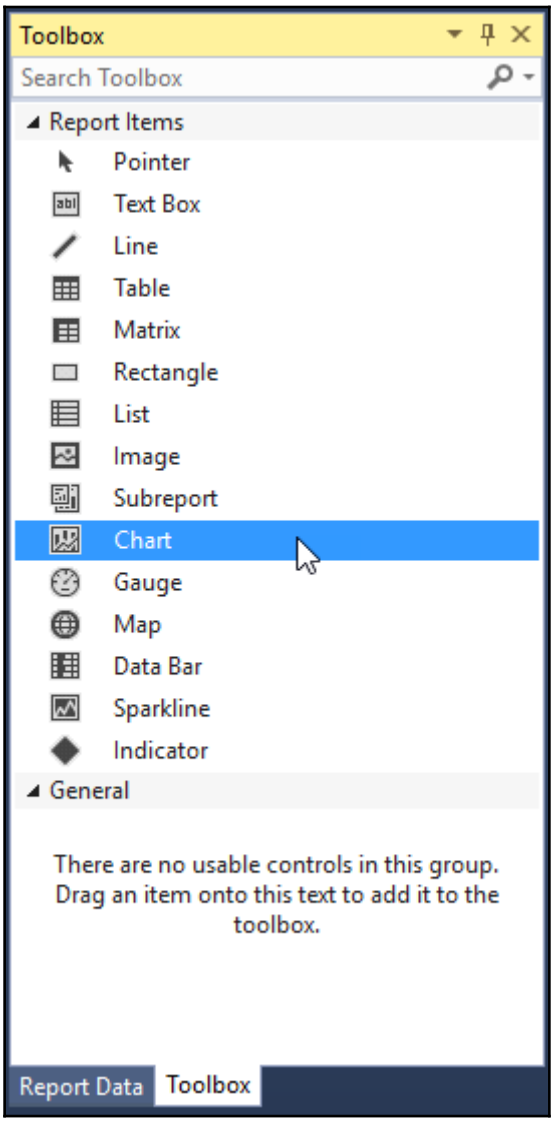


Figure 2.45

25. After performing the preceding step, you are presented with the **Select Chart Type** dialog. This provides a dizzying array of different chart types. Select the leftmost item under the **Line** area, as shown in Figure 2.46, and then click on **OK**:

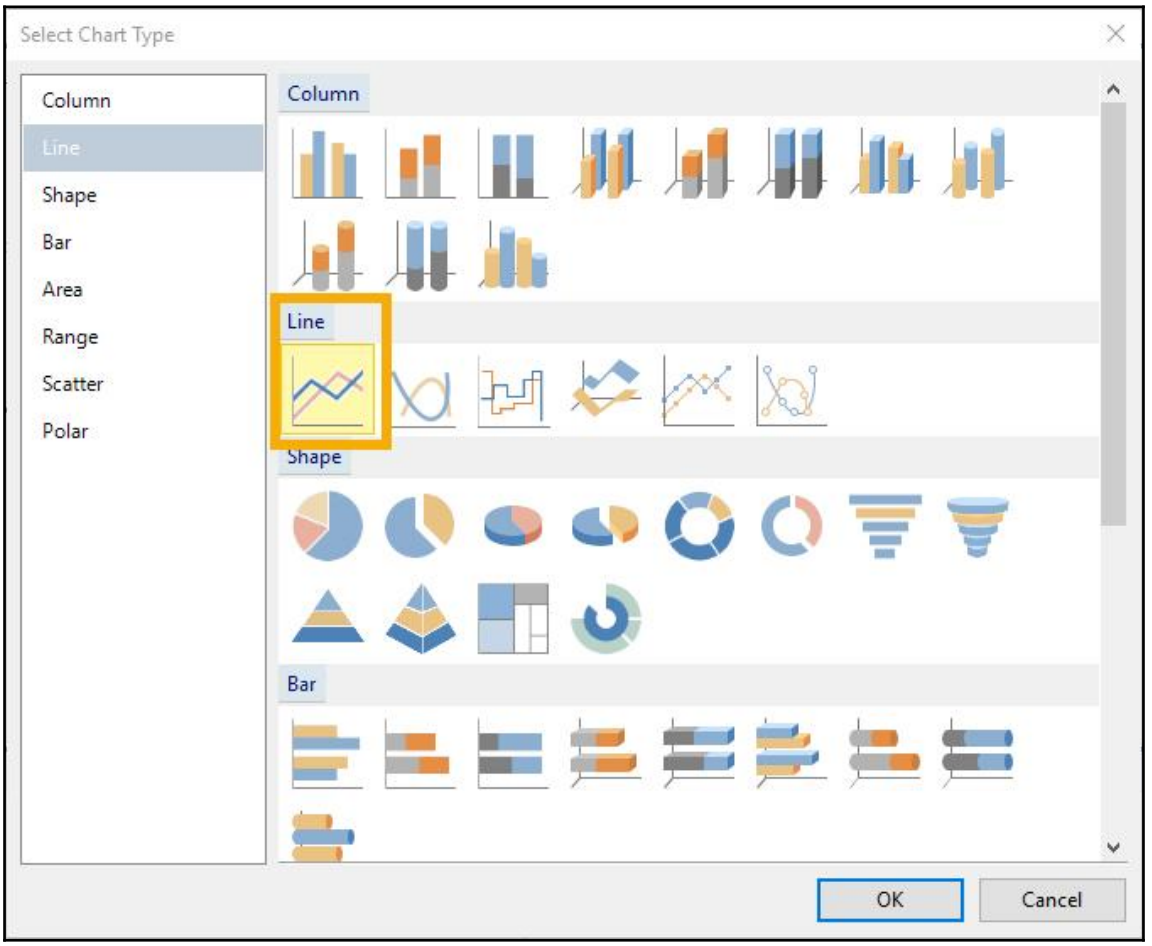


Figure 2.46

26. Use the sizing handles at the corners of the chart tool to resize the chart to take up more area, as this chart will hold a lot of data. As this is just a demo, the exact size is not too important; just do something that looks good on your screen.
27. Click inside the chart area, where the line graph appears, to make the **Chart Data** dialog appear.

28. Click on the plus sign beside the **Values** area, and select the **TotalSalesWithTax** field:

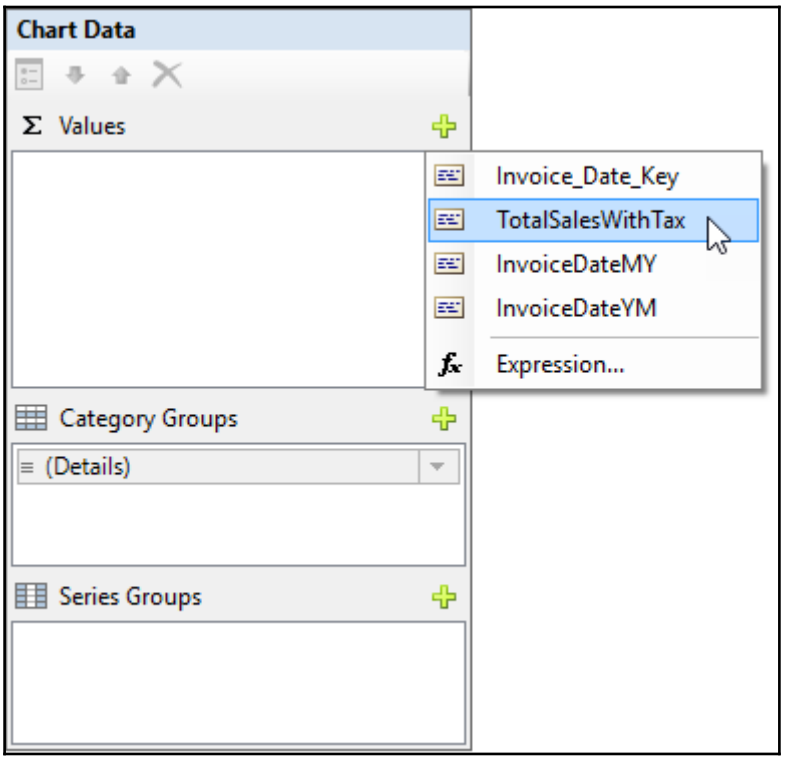


Figure 2.47

29. Next click the plus sign beside **Category Groups**, and pick **InvoiceDateMY**.

30. When we added the calculated field, we mentioned that while this field would be used to display, we would need to sort using a different field. We'll fix that now. Click on the drop-down arrow beside the **InvoiceDateMY** field, and pick **Category Group Properties** in the menu:

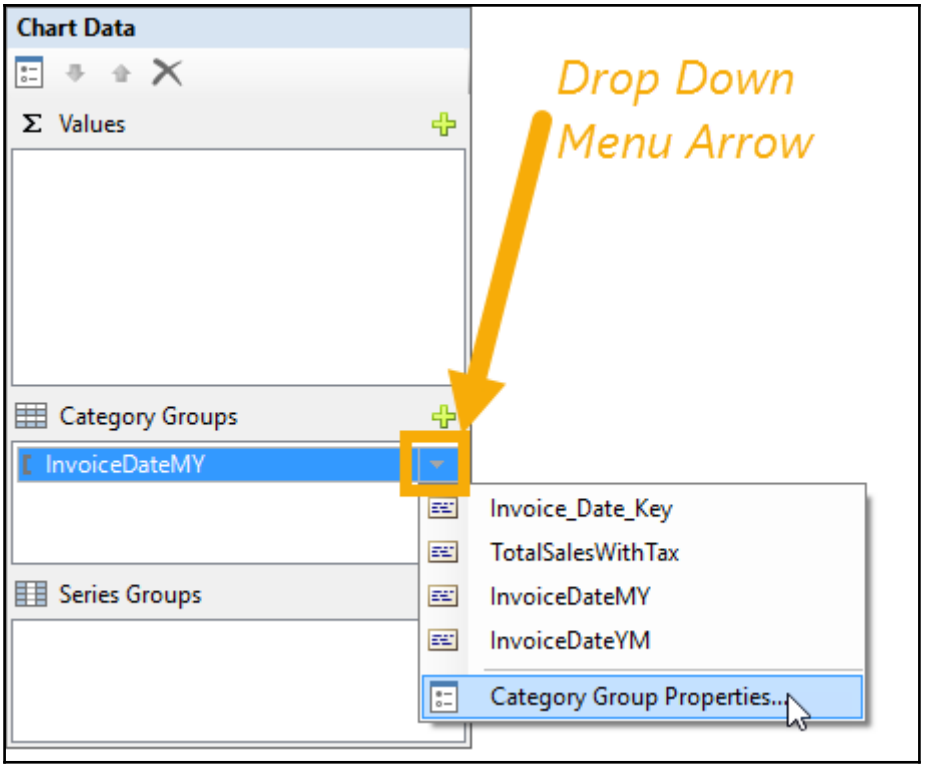


Figure 2.48

31. When the **Category Group** Properties dialog appears, click on the **Sorting** page on the left and change **Sort by** to `InvoiceDateYM`; then click on **OK**:

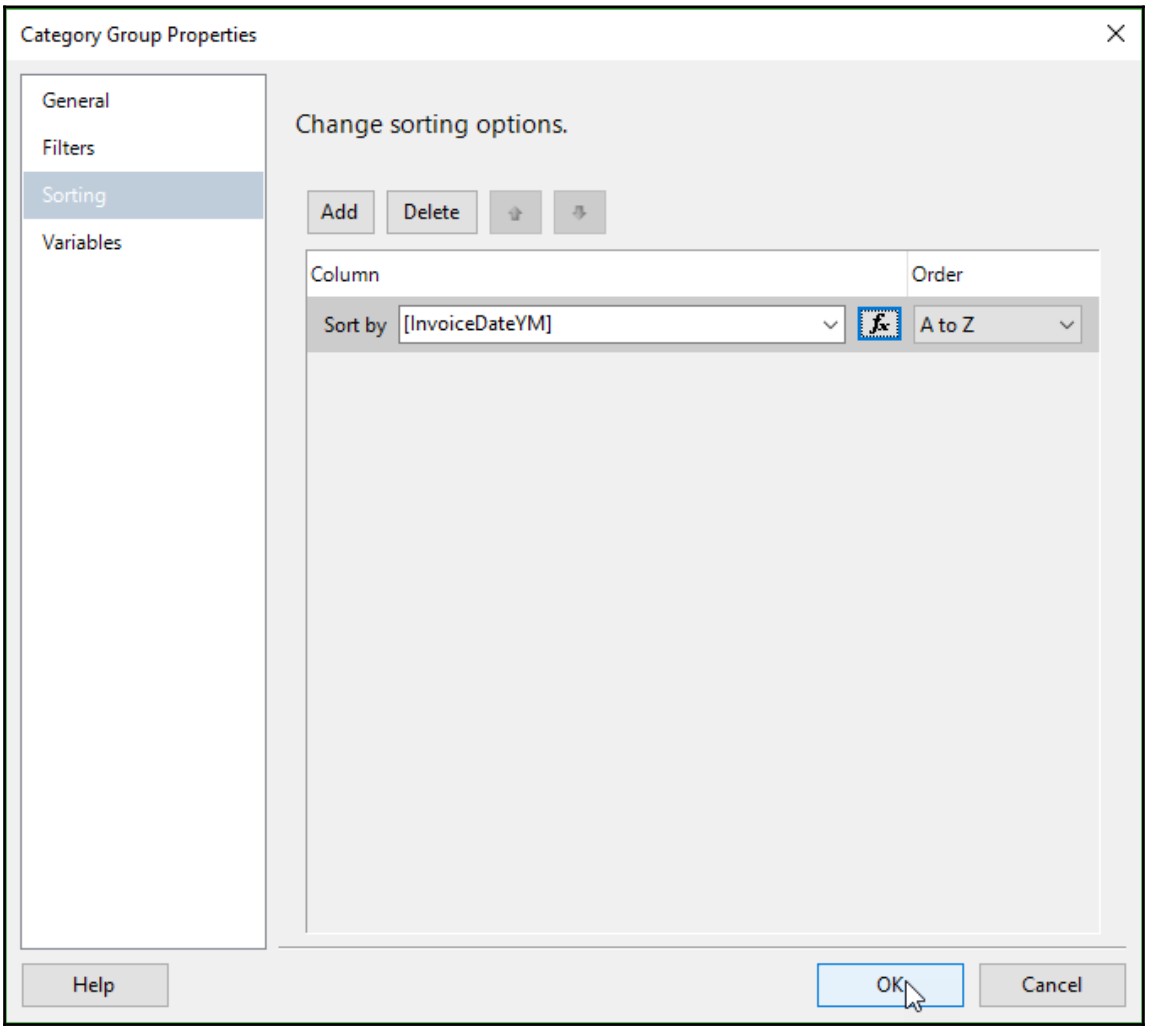


Figure 2.49

32. Your **Chart Data** area should now resemble Figure 2.50. If so, good news! You're almost done:

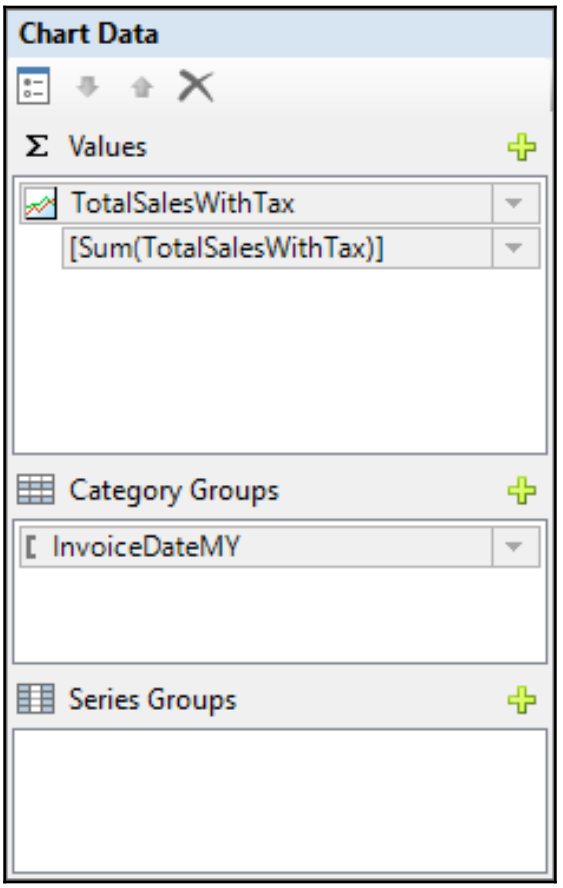


Figure 2.50

33. In the upper left of the chart, you should notice the words **Chart Title**. This is where you can rename the chart, and you should do so. Just double-click on the **Chart Title** and rename it to **Sales by Invoice Month/Year**:



Figure 2.51

34. Click on **Preview** to see the work thus far. As Figure 2.52 shows, it looks pretty good, but we can improve it with two simple tweaks:

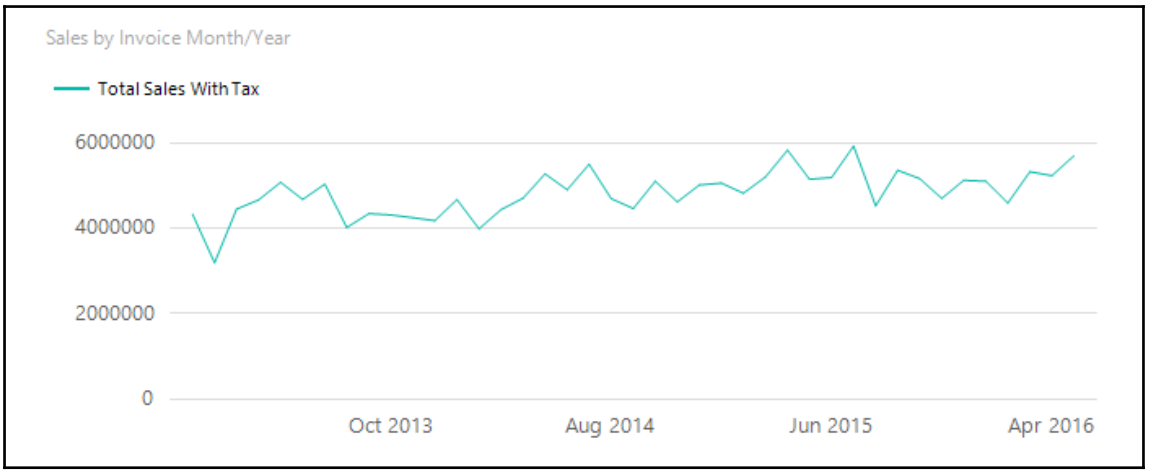


Figure 2.52

35. There is a lot of empty space from the lowest value to the zero that marks the vertical axis. In addition, the numbers themselves are rather difficult to read. Return to the **Design** tab and right-click on any of the numbers in the vertical axis tab. In the menu, select **Vertical Axis Properites**.

36. On the main **Axis Options** tab, uncheck **Always include zero**. This allows reporting services to set the scale automatically instead of always using zero as the lowest value:

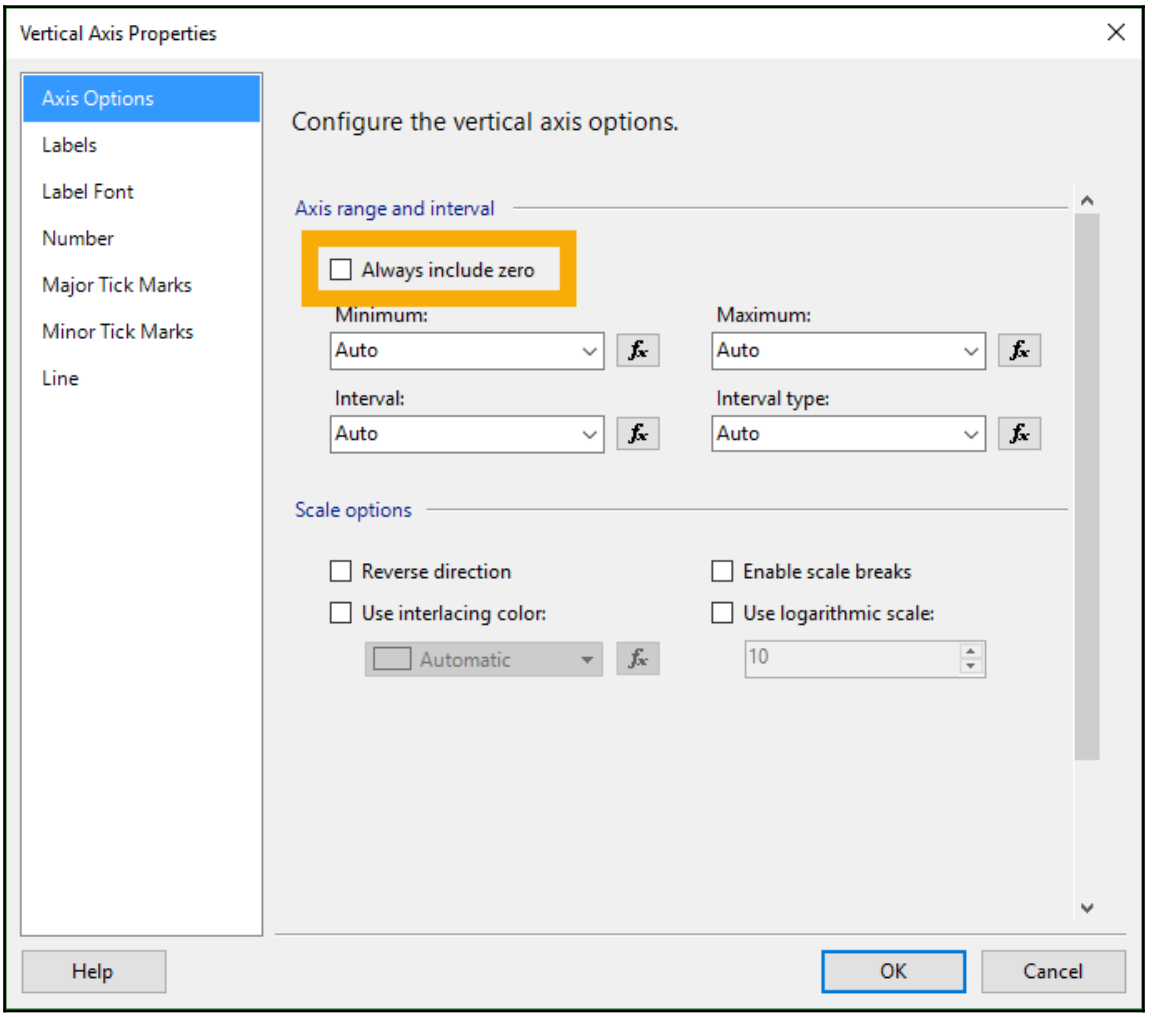


Figure 2.53

37. Next, click on the **Number** page on the left.
38. Under **Category**, select **Currency** and change the **Decimal places** to zero.
39. Check the box beside **Use 1000 separator**.

40. Once your **Propreties** dialog looks like figure 2.54, you can click on **OK**:

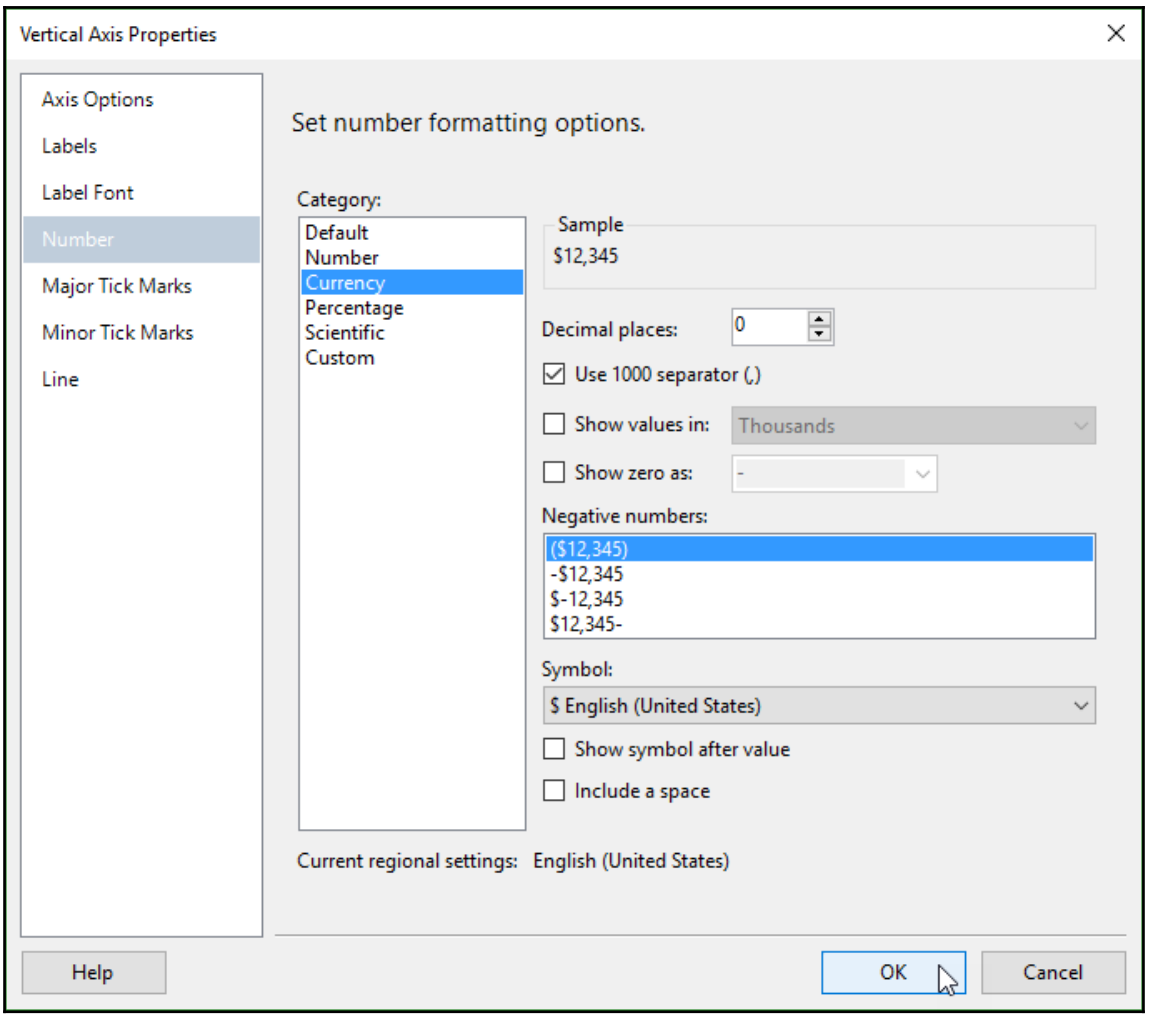


Figure 2.54

41. On the horizontal axis, it only displayed a few of the months. We'd like it to display at least every other month name to give the users a better idea of the data they are seeing. Right-click anywhere in the horizontal axis area and select **Horizontal Axis Properties** from the menu.

42. When the dialog appears, click on the **Interval** field, delete the word **Auto**, and type in the number **2**. This will tell SSRS to display every other value on the horizontal axis (3 would mean every third value, and so forth).

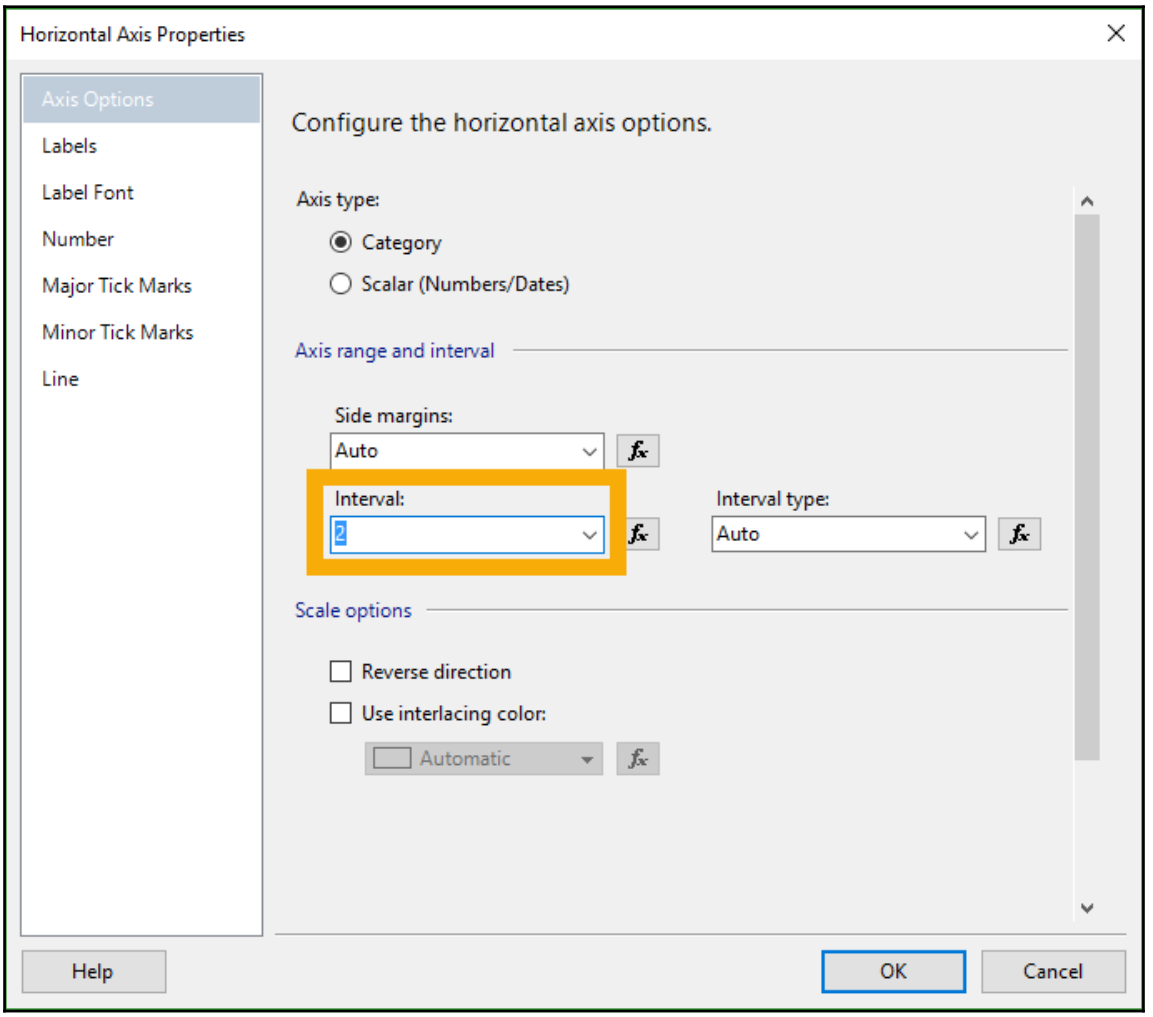


Figure 2.55

43. Click on **OK**, and then preview your report. You should have an attractive-looking chart, as shown in figure 2.56:

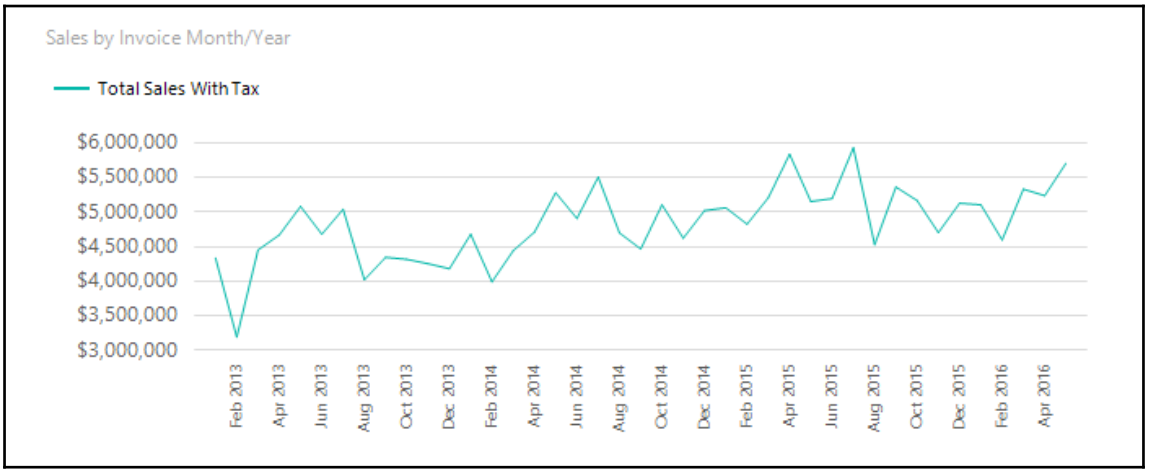


Figure 2.56

How it works…

The reporting services wizard is great for generating either a tabular or a matrix report. Unfortunately, it doesn't provide a method for generating a report with charts and graphs. Thus there is a need to create an empty report.

We then added the data, starting with the data source, and then the dataset based on a SQL query. Because the query did not provide all the fields we wanted, we added two new calculated fields using Reporting Services' powerful expression language. This language is very similar to Visual Basic in its form and structure.

SSRS is very customizable; wherever you see the *fx* symbol beside a field, you can use an expression to determine the value of the field at runtime. We used this to create two new fields, one to hold the date in Month-Year format, such as *Jul 2016*. As this would not sort correctly, we added a second field to be used in sorting, which created the date in Year-Month format, such as *2016-07*. We next needed to tell the chart what data to display. Using the Chart Data dialog, we first selected a field to put in the **Values** area. This is the value that gets plotted against the vertical axis.

Category Groups then used the `InvoiceDateMY` field to populate the horizontal axis. Because we knew the text in this field would sort in alphabetical order instead of chronological, we went into the properties for that group and set the property to the other calculated field we had added in order to sort the horizontal axis chronologically.

We then performed three tasks to make the chart look good. We set the chart title to a meaningful value. Next we altered the vertical axis properties to format the values to a more readable currency value. Because the number range was so high, having the chart zero-based produced a big gap at the bottom and compressed the line at the top, making it hard to read. We removed the zero-based setting, which allowed SSRS to scale the value appropriately to our data.

Finally, the horizontal axis was only displaying a handful of Month/Year values, making it difficult to pinpoint which values on the chart were associated with which dates. Changing the horizontal axis interval property allowed us to choose a pattern more appropriate for our data.

There's more….

In this recipe, we manually added two calculated fields to use in the chart. As an alternative, we could have pushed those calculations *upstream*. That is, we could have had the SQL Server engine calculate the values in the SQL query:

```
SELECT CAST(YEAR( s.[Invoice Date Key] ) AS NVARCHAR)
          + '-'
          + RIGHT('00' + CAST(MONTH( s.[Invoice Date Key]) AS NVARCHAR), 2
)
        AS InvoiceDateYM
      , LEFT( DATENAME(MONTH, s.[Invoice Date Key] ), 3)
          + ' ' + CAST(YEAR( s.[Invoice Date Key] ) AS NVARCHAR)
        AS InvoiceDateMY
     , SUM(s.[Total Including Tax]) AS TotalSalesWithTax
  FROM [Fact].[Sale] s
 GROUP BY CAST(YEAR( s.[Invoice Date Key] ) AS NVARCHAR)
          + '-'
          + RIGHT('00' + CAST(MONTH( s.[Invoice Date Key]) AS NVARCHAR), 2
)
      , LEFT( DATENAME(MONTH, s.[Invoice Date Key] ), 3)
          + ' ' + CAST(YEAR( s.[Invoice Date Key] ) AS NVARCHAR)
```

While the SQL is more involved, the SQL Server engine handles the logic instead of making SSRS handle it. Note that we made **InvoiceDateYM** a character field to match the example in the report; it could just as easily have been numeric (201607 for example) and still be used in sorting. Where possible, it is generally best to push calculated values as close to the data source as possible, as we did in the preceding SQL.

Unfortunately, you don't always have the ability to construct queries in this manner, or perhaps the query may get complex very quickly. Knowing how to do calculated fields within the SSRS Dataset can be a valuable skill to have.

Managing and configuring parameters

In the construction of the tabular report earlier in this chapter (see *Creating reports with a table data region* recipe), in the query, the date range was hardcoded. It would be beneficial to the user of the report if they could select the date range to run the report with.

SQL Server Reporting Services uses the concept of *parameters* to accomplish this. In this section, we'll see how to implement parameters in the tabular report.

Getting ready

We will want to create a copy of the original tabular report and add it to our project so that we can work with it. This is actually a pretty common task; often times you will have a report that is almost, but not quite, what is needed for a new report. Copying that existing report can be a shortcut to getting up and going with the new one.

Navigate to the folder where you placed the demos. Then, using Windows file explorer, copy and paste the `Report 02-01 Tabular Demo.rdl` file, renaming it to `Report 02-04 Parameters Demo.rdl`.

Return to SQL Server Data Tools. In Solution Explorer, right-click on the **Reports** branch. Select **Add** | **Existing Item** in the menu, and pick the newly copied report. Once added, open up the report to get started with the recipe.

How to do it…

1. In the Report **Data** tool, right-click on Parameters and select **Add Parameter…**:

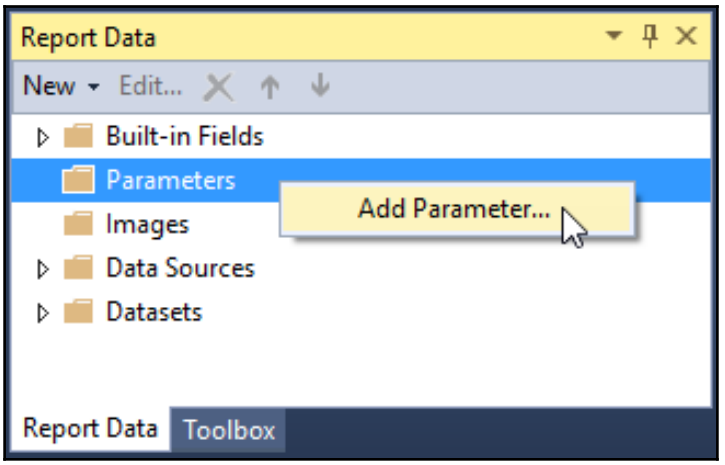


Figure 2.57

2. Change the name of the parameter to `FromDate` (no spaces). This will be the name used internally by the report.
3. Change the prompt to read `From Date`. This is what the user will see in the report.

4. In the `Data type` dropdown, change it to **Date/Time**. This will cause SSRS to display a date picker, allowing users to easily choose the **From Date**. In addition, it restricts the allowed input, reducing the chance of errors during report generation:

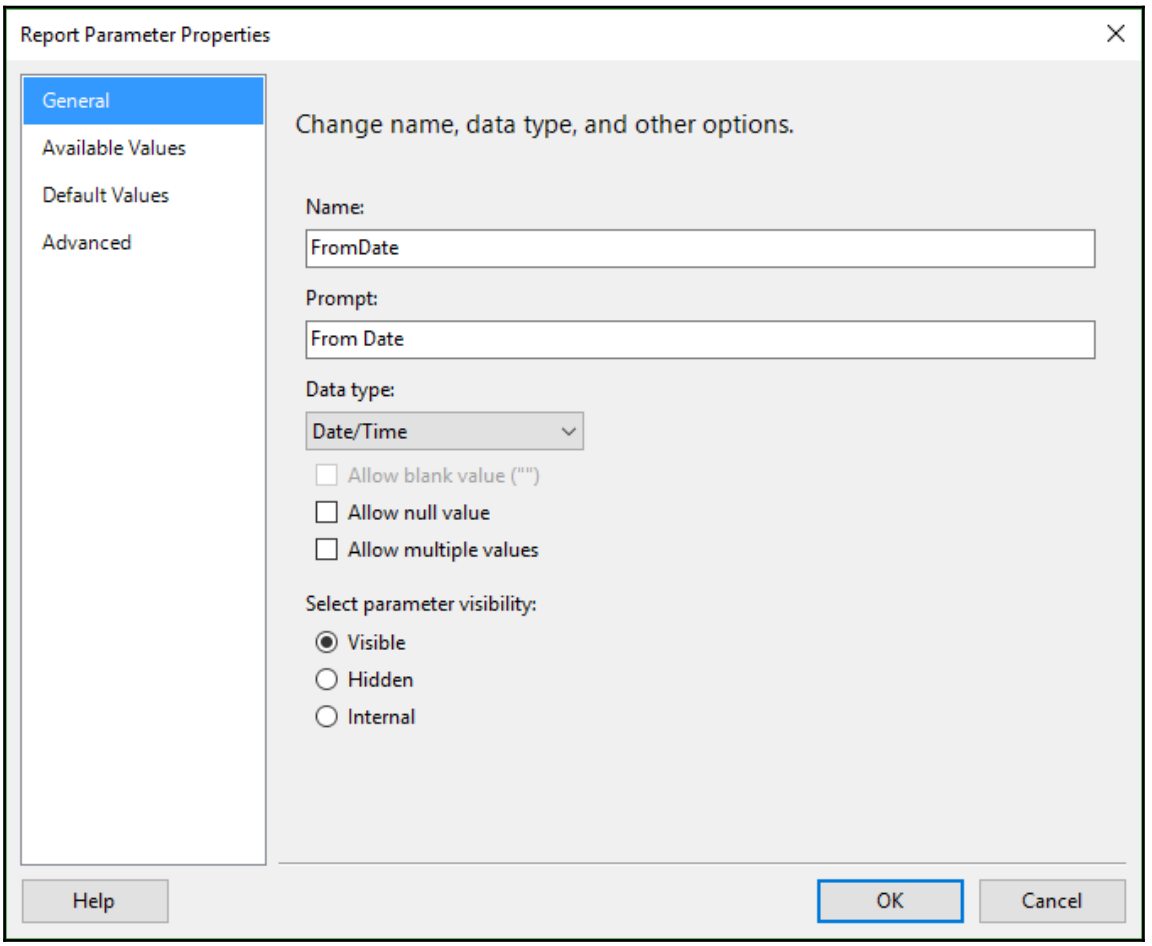


Figure 2.58

5. Click on the **Default Values** page to the left.
6. In the options, choose **Specify values**. The lower half of the page will update; click on the **Add** button and also the *fx* button.

7. We want to default to the first day of the current month. To do so, we'll use the following expression:

```
=DateSerial(Year(Today), Month(Today), 1)
```

8. The `DateSerial` method will take the year number, month number, and day number passed in and convert it to a date. Using 1 as the day will result in the first day of the current month. Once entered, click on **OK** to close the expression editor and return to the **Report Parameter Properties** dialog:

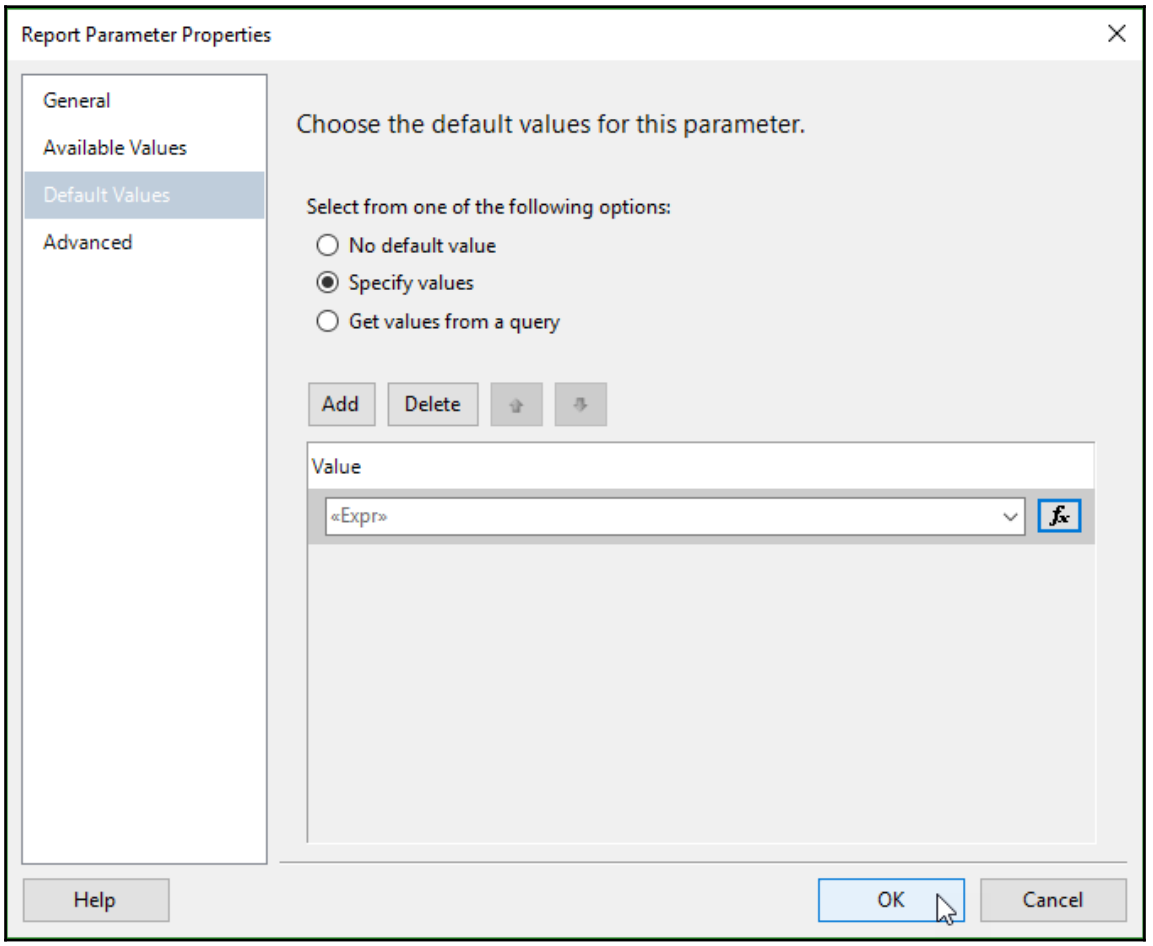


Figure 2.59

9. We'll now add the `ThruDate` parameter. This is almost a repeat of the preceding steps. Start by going to the **Report Data** pane, right-clicking on **Parameters**, and selecting **Add Parameter**.
10. In the **Name**, enter `ThruDate`.
11. In the **Prompt**, enter **Through Date**.
12. Change the Data type to **Date/Time**.
13. Click on the **Default Values** page.
14. Select the **Specify values** option.
15. Click on the **Add** button and *fx* button.
16. In the expression editor, simply enter `=Today` to return today's date, and click **OK** to close the editor and also to close **Report Parameter Properties**.
17. Now we need to connect the parameters to the query. In the **Report Data** panel, expand the **Datasets** branch, right-click on **DataSet1**, and select **Dataset Properties** from the menu.
18. Click on the **Parameters** page and the **Add** button.
19. In the **Parameter Name** field, enter `@FromDate`. This is the name that will be used inside the query. As best practice, you'll want to use the same name as the parameter, although technically it's not required.
20. In the **Parameter Value** dropdown, select **[@FromDate]**. This name comes from the parameters you added in previous steps.

21. Repeat these steps, using `@ThruDate` for the **Parameter Name** and select `[@ThruDate]` for the **Parameter Value**:

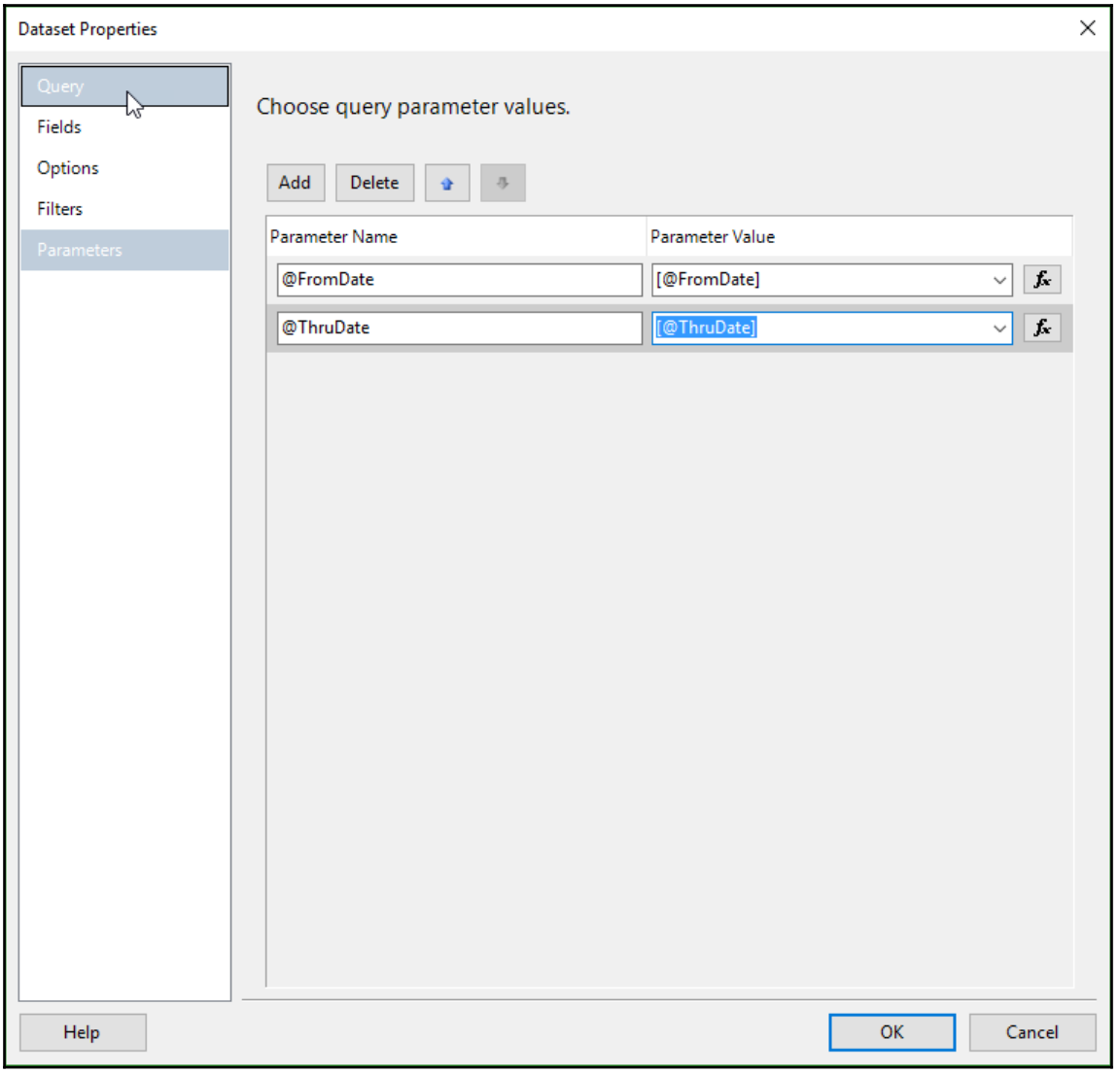


Figure 2.60

22. Now we need to modify the query to use the parameters. Click on the **Query** page.
23. In the query area, change the `WHERE` clause to read:

```
WHERE s.[Invoice Date Key] BETWEEN @FromDate AND @ThruDate
```

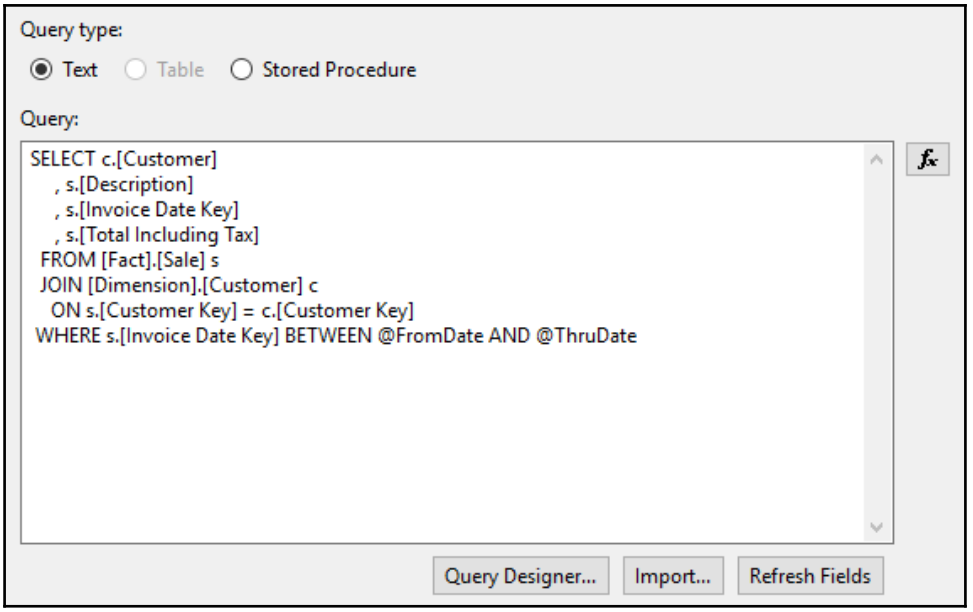


Figure 2.61

24. Click on **OK** to close the **Dataset Properties** dialog.
25. At the top of the report designer, you may have noticed our new parameters appearing. You can rearrange them by simply clicking and dragging to move the parameter into a new position. To illustrate, click on **Through Date** and drag it under **From Date**:



Figure 2.62

26. Click on **Preview** to run the report. If you are using the default `WideWorldImportersDW` sample database, then the report is going to be empty. This is because the sample database ends on May 31, 2016. To test, simply alter the **From Date** to `5/01/2016` and the **Through Date** to `5/31/2016`. Then click on the **View Report** button on the right to regenerate the report using the new parameters. Feel free to experiment with other dates:

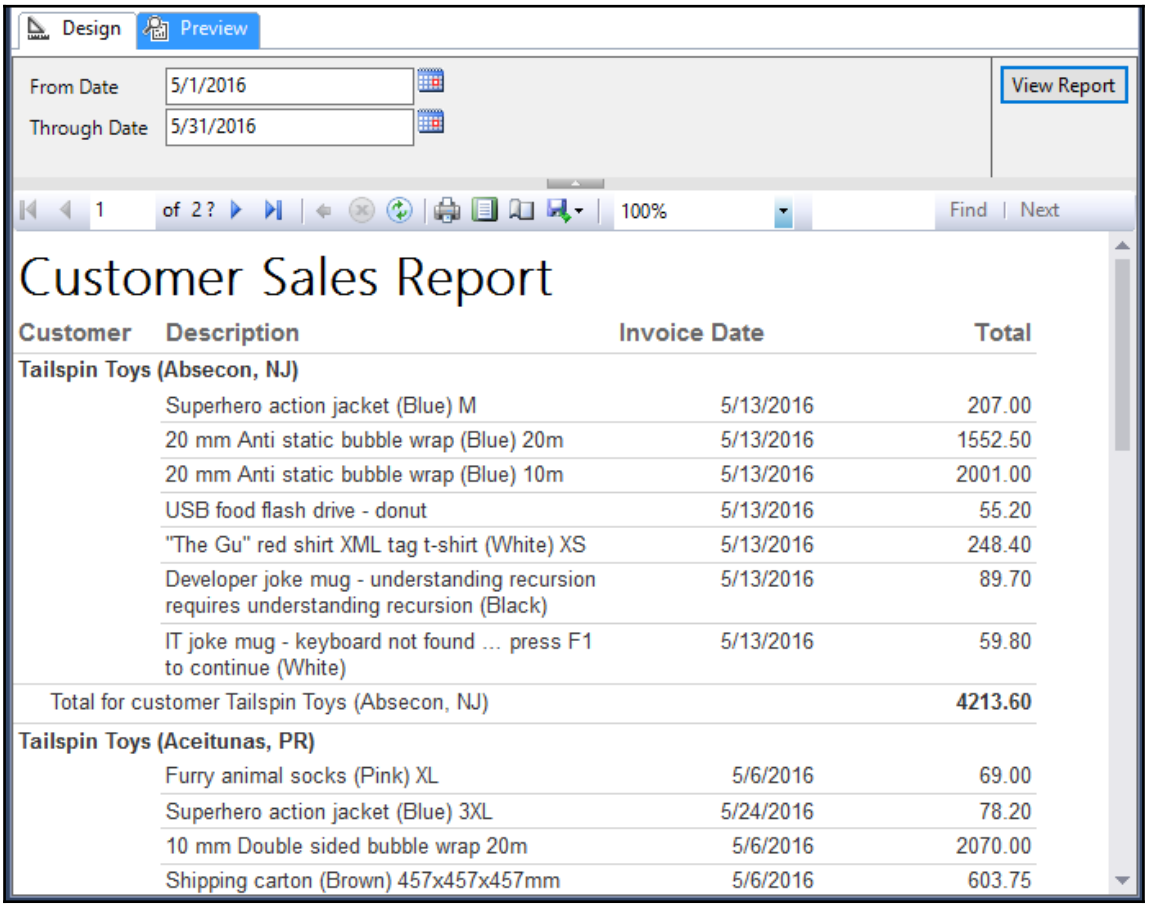


Figure 2.63

How it works…

Creating parameters for a report is not difficult, but it does require several steps, which must occur in the correct order. The first step is adding the parameters themselves to the Parameters area in the **Report Data** tab. When you add a parameter here, it tells SSRS to add a new data entry field into the parameter grid at the top of the report.

The parameter is configured in the **Report Parameter Properties** dialog (refer back to Figure 2.58 as a reminder of what is on this dialog). Using the Data type dropdown allows you to limit the input a user can give, helping prevent errors:

- **Text**: Allows the user to enter anything they wish
- **Boolean**: The user is limited to either a `True` or `False` option
- **Date/Time**: Displays a date picker (a calendar) the user can select from
- **Integer**: Only allows the entry of integer values
- **Float**: Only allows the entry of numeric values with a decimal place

The other area that is not intuitive is the **Select parameter visibility** area. Here are three values that you need to understand:

- **Visible**: This parameter will be displayed to the end user. In addition, it can be specified when the report is called via a URL to the Report Portal, or when embedded in a .NET application.
- **Hidden**: This parameter is NOT displayed to the end user. However, it can be specified when the report is called via a URL to the Report Portal, or when embedded in a .NET application.
- **Internal**: This parameter is not displayed to the end user, nor is it available through the URL or callable from .NET. Instead, it acts like a variable inside the report, allowing you to create a calculated value and use it throughout the report-not just in the query either, but wherever an *fx* button is displayed.

Users will be aided if the report can be populated with default values. On the default values page, there were three options:

- **No default value**: When the report is run, it will not show any data. The user will be forced to select data and then click on **View Report** before the report is generated. This is a good choice when you cannot reasonably predict what values a report should run with.

- **Specify values**: This is a good option if you can reasonably guess what values the majority of users will use to run your report. For example, the current month might be the default used the majority of the time. In this example, we used a calculation to specify a date. We could have also hardcoded multiple values. For example, if the parameter were *Who is your favorite podcaster?* you could have clicked on **Add**, typed `Adam Curry` in the Value field, clicked on **Add** again, and typed in `John C. Dvorak`, and so forth until you entered all the values.
- **Get values from a query**: This will let you hook up the values to be displayed in the parameter dropdown to a dataset in your report. In addition to your main query, you may also have a query to populate a list of countries from your database. The users can then select from the predefined list of countries, avoiding misspellings or entering a non-existent country. In addition, by coming from a query, if a new country were added to the database, it would automatically show up in the parameter list the next time the report is run.

Once the parameter is created, it needs to be added to the dataset in order for the dataset to use it. This was the purpose of the **Parameters** page on Figure 2.60. In essence, it tells the dataset: *hey, here are two parameters you can use*.

The **Parameter Value** area is a dropdown that is populated from the list of parameters added earlier. **Parameter Name** is what you wish to call it in the query. It is generally considered best practice to use the same name as the parameter value, but it is not required.

So far, we've created parameters and then made those parameters available to the dataset to use. Now we need to actually use those parameters. That's when we modified the query to replace the hardcoded dates with the parameter names (Figure 2.61).

And that's it! Three simple steps, and your report will become dynamic and more flexible to the end user.

3

Advanced Report Authoring with SQL Server Data Tools

We will continue our journey into report authoring by moving into advanced topics. We will now cover the following recipes:

- Creating reports with multiple-axis charts
- Creating reports with multiple data regions
- Creating reports with nested data regions
- Creating reports with a map data region
- Creating reports with cascading parameters
- Creating reports with dynamic datasets
- How to publish reports
- How to view the XML code related to reports

Introduction

This chapter will focus primarily on the use of multiple datasets within the same report. In addition, we'll see how to create a chart with multiple axes, as well as using maps in our reports. We will take a look at publishing reports to the new SSRS Report Portal, and will see what an RDL file looks like behind the scenes.

Please note that this chapter will build upon the recipes in `Chapter 2`, *Authoring Reports with SQL Server Data Tools* such as creating a report or project, which will be in summary format. If you need a step by step guide for these items, please refer back to the previous chapter.

Creating reports with multiple axis charts

Often you will want to have multiple items plotted on a chart. In the previous recipe on charts in `Chapter 2`, *Authoring Reports with SQL Server Data Tools*, we plotted a single value over time.

For this recipe, we will plot two values over time, in this case, the Total Sales Amount (Excluding Tax) and the Total Tax Amount. As you might expect though, the tax amounts are going to be a small percentage of the sales amounts. By default, this would create a chart with a huge gap in the middle and a Y Axis that is quite large and difficult to pinpoint values on.

To prevent this, Reporting Services allows us to place a second Y Axis on our charts. With this recipe, we'll explore both adding a second line to our chart as well as having it plotted on a second Y-Axis.

Getting ready

First, since this is a new chapter, we'll create a new Reporting Services project to contain it. Name this new project `Chapter03`.

Within the new project, create a **Shared Data Source** that will connect to the `WideWorldImportersDW` database. Name the new data source after the database, `WideWorldImportersDW`.

Next, we'll need data. Our data will come from the sales table, and we will want to sum our totals by year, so we can plot our years across the X-Axis. For the Y-Axis, we'll use the totals of two fields: `TotalExcludingTax` and `TaxAmount`. Here is the query by which we will accomplish this:

```
SELECT YEAR([Invoice Date Key]) AS InvoiceYear
     ,SUM([Total Excluding Tax]) AS TotalExcludingTax
     ,SUM([Tax Amount]) AS TotalTaxAmount
FROM [Fact].[Sale]
GROUP BY YEAR([Invoice Date Key])
```

How to do it…

1. Right-click on the **Reports** branch in the **Solution Explorer**.
2. Go to **Add** | **New Item**… from the pop-up menu.
3. On the **Add New Item** dialog, select **Report** from the choice of templates in the middle (do not select **Report** wizard).
4. At the bottom, name the report `Report 03-01 Multi Axis Charts.rdl` and click on **Add**.
5. Go to the Report Data tool pane. Right-click on **Data Sources** and then click **Add Data Source…** from the menu.
6. In the **Name:** area, enter `WideWorldImportersDW`.
7. Change the data source option to the **Use shared dataset** source reference.
8. In the dropdown, select `WideWorldImportersDW`.
9. Click on **OK** to close the **DataSet Properties** window.
10. Right-click on the Datasets branch and select **Add Dataset…**.
11. Name the dataset `SalesTotalsOverTime`.
12. Select the **Use a dataset embedded in my report** option.
13. Select `WideWorldImportersDW` in the **Data source** dropdown.

14. Paste in the query from the *Getting ready* area of this recipe:

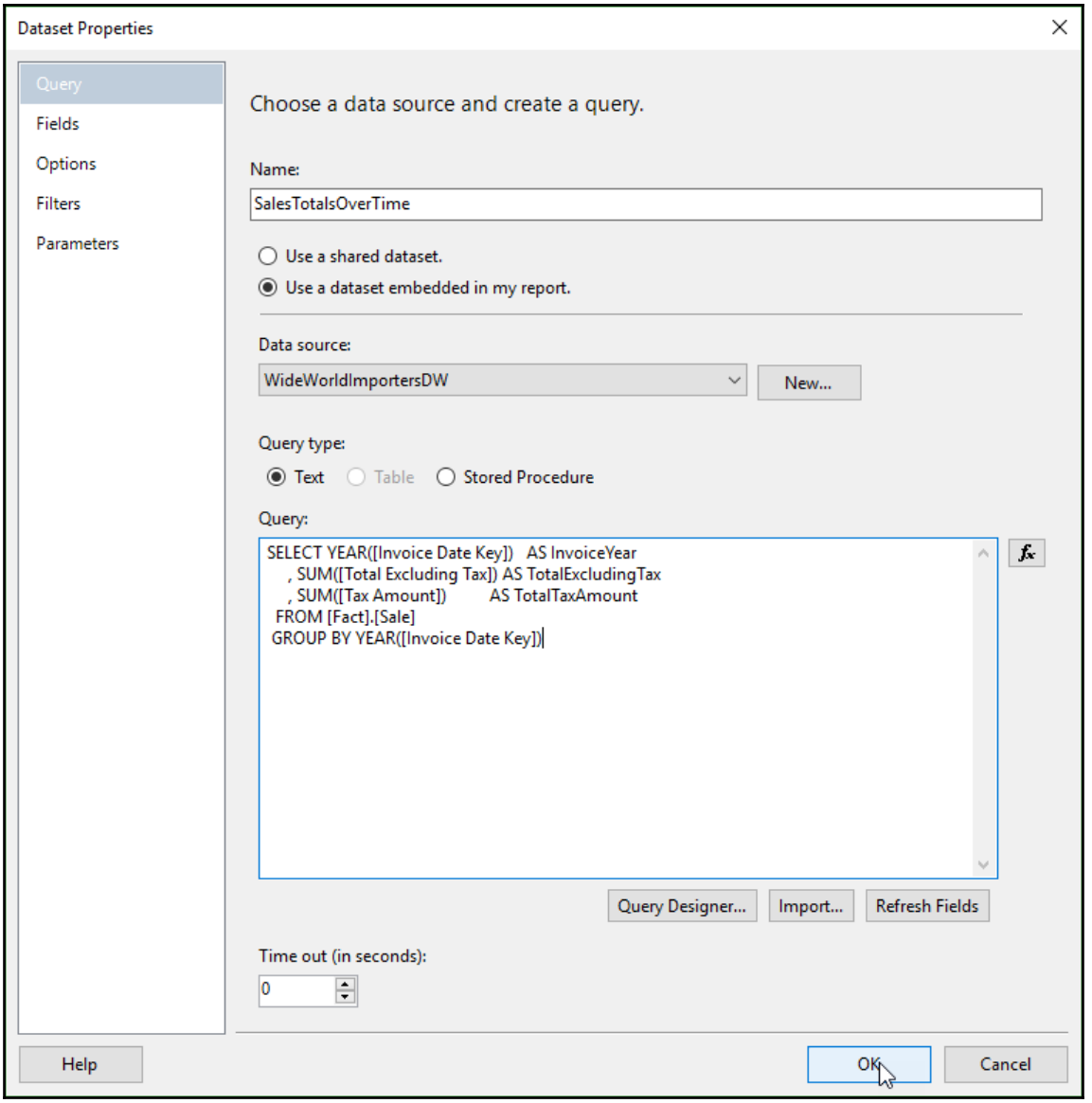


Figure 3.01

15. When your window resembles that of the preceding figure, click on **OK**.
16. Next, go to the Toolbox pane.

17. Drag and drop a **Chart** tool onto the report.
18. Select the leftmost **Line chart** from the **Select Chart Type** window, and click on **OK**.
19. Resize the chart to a larger size. (For this demo, the exact size is not important. For your production reports, you can resize as needed using the Properties window, as seen previously.)
20. Click inside the main chart area to make the **Chart Data** dialog appear to the right of the chart.
21. Click on the **+** (plus button) to the right of the **Values**.
22. Select **TotalExcludingTax**.
23. Click on the plus button again, and now pick **TotalTaxAmount**.
24. Click on the **+** (plus button) beside **Category Groups**, and pick **InvoiceYear**.
25. Click on **Preview**. You will note the large gap between the two graphed lines. In addition, the values for the Total Tax Amount are almost impossible to guess, as shown in the following figure:

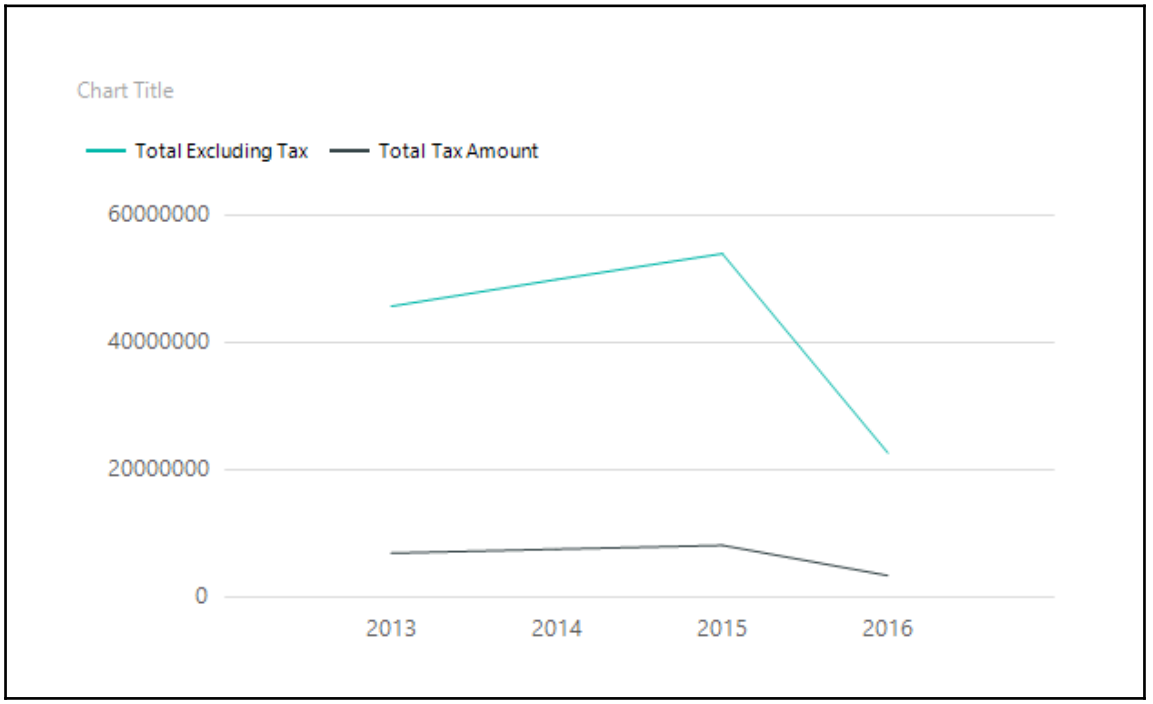


Figure 3.02

26. Return to the designer, and again click in the chart area to make the **Chart Data** dialog appear.
27. In the **Chart Data** dialog, click on the dropdown beside **TotalTaxAmount** (see Figure 3.03):

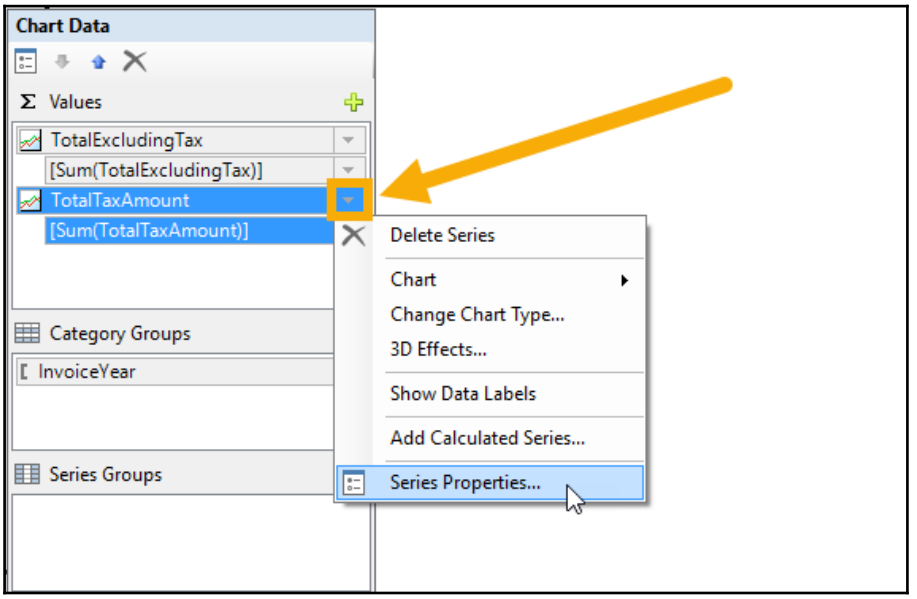


Figure 3.03

28. Select **Series Properties…**.

29. Click on the **Axes and Chart Area** page, and for **Vertical axis**, select **Secondary**:

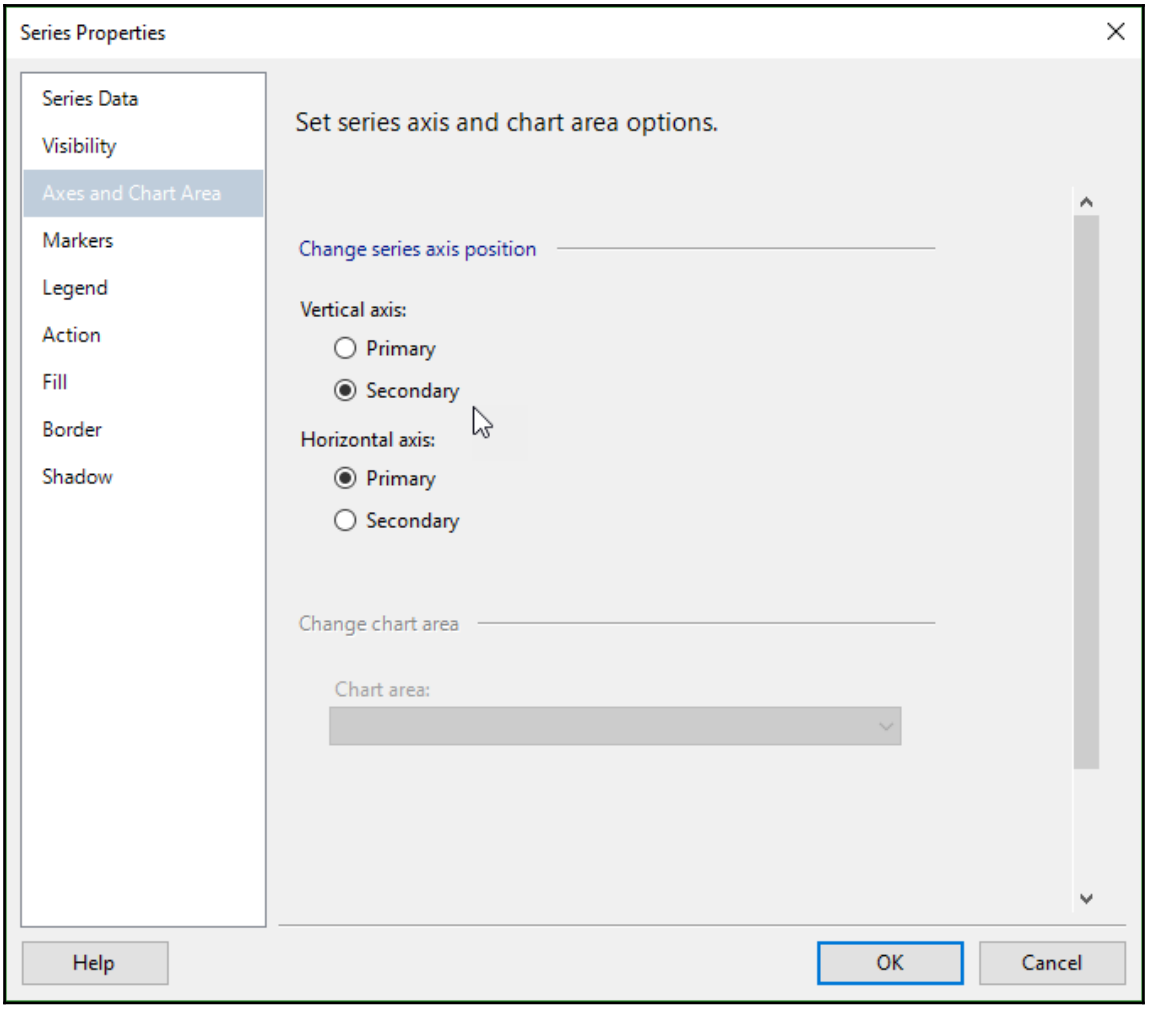


Figure 3.04

30. Click on **OK** to close the **Series Properties** window.
31. Right-click on the numbers now appearing on the right in the vertical axis area, and select **Secondary Vertical Axis Properties** in the menu.
32. In the **Axis Options**, uncheck **Always include zero**.

33. Click on the **Number** page.
34. Under Category, select **Currency**. Change the **Decimal places** to 0, and place a check in **Use 1000 separator** (for details and images, refer to figures 2.53 and 2.54 in `Chapter 2`, *Authoring Reports with SQL Server Data Tools*).
35. Click on **OK** to close this window.
36. Now move to the vertical axis on the left-hand side of the chart, right-click, and pick **Vertical Axis Properties**.
37. Uncheck **Always include zero**.
38. On the **Number** page, pick **Currency**, set **Decimal places** to 0, and check **Use 1000 separator**. Click on **OK** to close.
39. Click on the **Preview** tab to see the results:

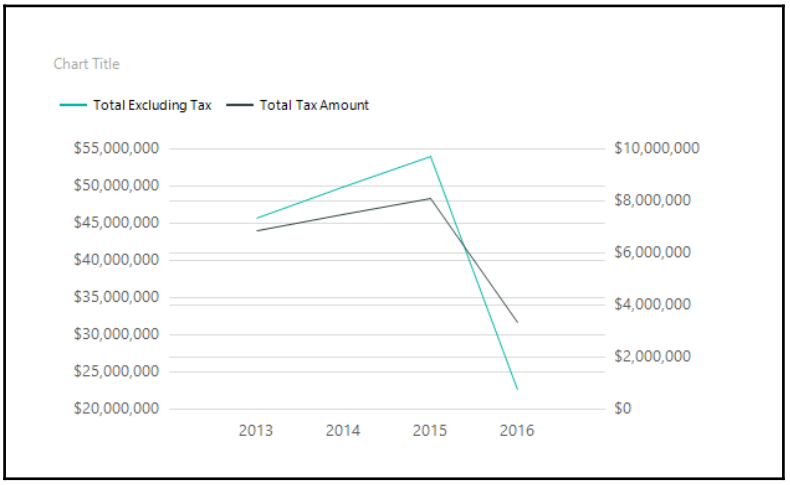


Figure 3.05

40. You can now see a chart with a second axis. The monetary amounts are much easier to read. Further, the plotted lines have a similar rise and fall, indicating the taxes collected matched the sales totals in terms of trending.

How it works…

SSRS is capable of plotting multiple lines on a chart. Here we've just placed two fields, but you can add as many as you need. But do realize that the more lines included, the harder the chart can become to read.

All that is needed is to put the additional fields into the **Values** area of the **Chart Data** window. When these values are of similar scale, for example, sales broken up by state, this works fine.

There are times though when the scale between plotted values is so great that it distorts the entire chart, leaving one value in a slender line at the top and another at the bottom, with a huge gap in the middle. To fix this, SSRS allows a second Y-Axis to be included. This will create a scale for the field (or fields) assigned to that axis in the Series Properties window.

Creating reports with multiple data regions

There is an old saying, *A picture is worth a thousand words*. Unfortunately, all too often, a picture needs a little helping text to explain it. In the reporting world, it is common to want to display multiple objects, most commonly a chart and a table or matrix, on the same report, both sourced from the same data. This technique is known as having *multiple data regions* on the same report.

Getting ready

For this report, we want to plot the total sales by year for each of the sales territories in our sales fact table. This will naturally fit well into a chart, in order to quickly perform a comparison between territories. However, the management also needs to know the exact values for each territory by year.

To accomplish this, we'll place two objects on the report, both a chart and a matrix. For the query, we'll use the following:

```
SELECT c.[Sales Territory]
     , YEAR(s.[Invoice Date Key])   AS [Invoice Year]
     , SUM(s.[Total Excluding Tax]) AS [Total Sales]
  FROM [Fact].[Sale] s
  JOIN [Dimension].[City] c
    ON s.[City Key] = c.[City Key]
 WHERE c.[Sales Territory] IN (@Territory)
 GROUP BY c.[Sales Territory], YEAR(s.[Invoice Date Key])
 ORDER BY c.[Sales Territory], YEAR(s.[Invoice Date Key])
```

In addition, we want to allow the user to select only certain sales territories to appear on the report. To handle this, we'll create a report parameter with the list of territories supplied by the database, using the following query. We'll tie the parameter to the query so that when the user changes the parameter, both the chart and matrix will update:

```
SELECT DISTINCT [Sales Territory]
   FROM [Dimension].[City]
  ORDER BY [Sales Territory]
```

How to do it…

1. Right-click on the **Reports** branch in the **Solution Explorer**.
2. Select **Add** | **New Item**… from the pop-up menu.
3. On the **Add New Item** dialog, select **Report** from the choice of templates in the middle (do not select **Report** wizard).
4. At the bottom, name the report`Report 03-02 Multi Data Regions.rdl` and click on **Add**.
5. Go to the Report Data tool pane. Right click on **Data Sources** and then click **Add Data Source…** from the menu.
6. In the **Name:** area, enter `WideWorldImportersDW`.
7. Change the data source option to the **Use shared dataset** reference.
8. In the dropdown, select `WideWorldImportersDW`.
9. Click on **OK** to close the **DataSet Properties** window.
10. Now we'll add the dataset to supply the list of sales territories to our parameter. Right-click on the **Datasets** branch and select **Add Dataset…**.
11. Name the dataset `Territory`.
12. Select the **Use a dataset embedded in my report** option.
13. In the Data source dropdown, select `WideWorldImportersDW`.
14. Place the second query found in the *Getting ready* section into the Query area. Click on **OK** to close.
15. Next we'll add the parameter. Right-click on the **Parameters** branch, and click **Add Parameter**.
16. In the **Report Parameter Properties** window, enter `Territory` for both **Name** and **Prompt**.

17. Check **Allow multiple values**, as we want to allow the user to select multiple (or all) territories:

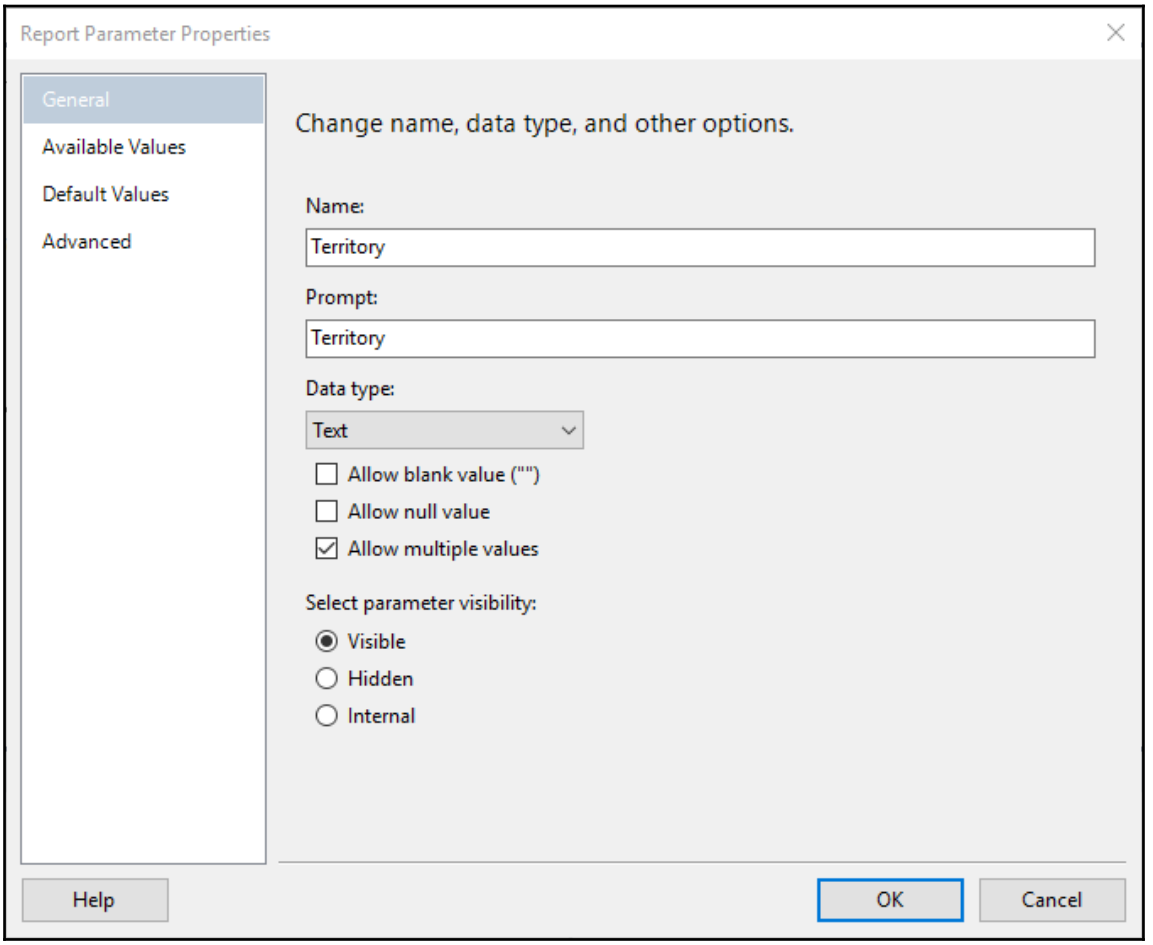


Figure 3.06

18. Once your screen looks like the previous figure, click on the **Available Values** page. It is on this page that we tell SSRS where to get the values to place in the parameter's drop-down box.
19. Select the **Get values from a query** option.
20. For the **dataset** dropdown, select **Territory**.

21. From both the **value** and**label** field dropdowns, select **Sales_Territory:**

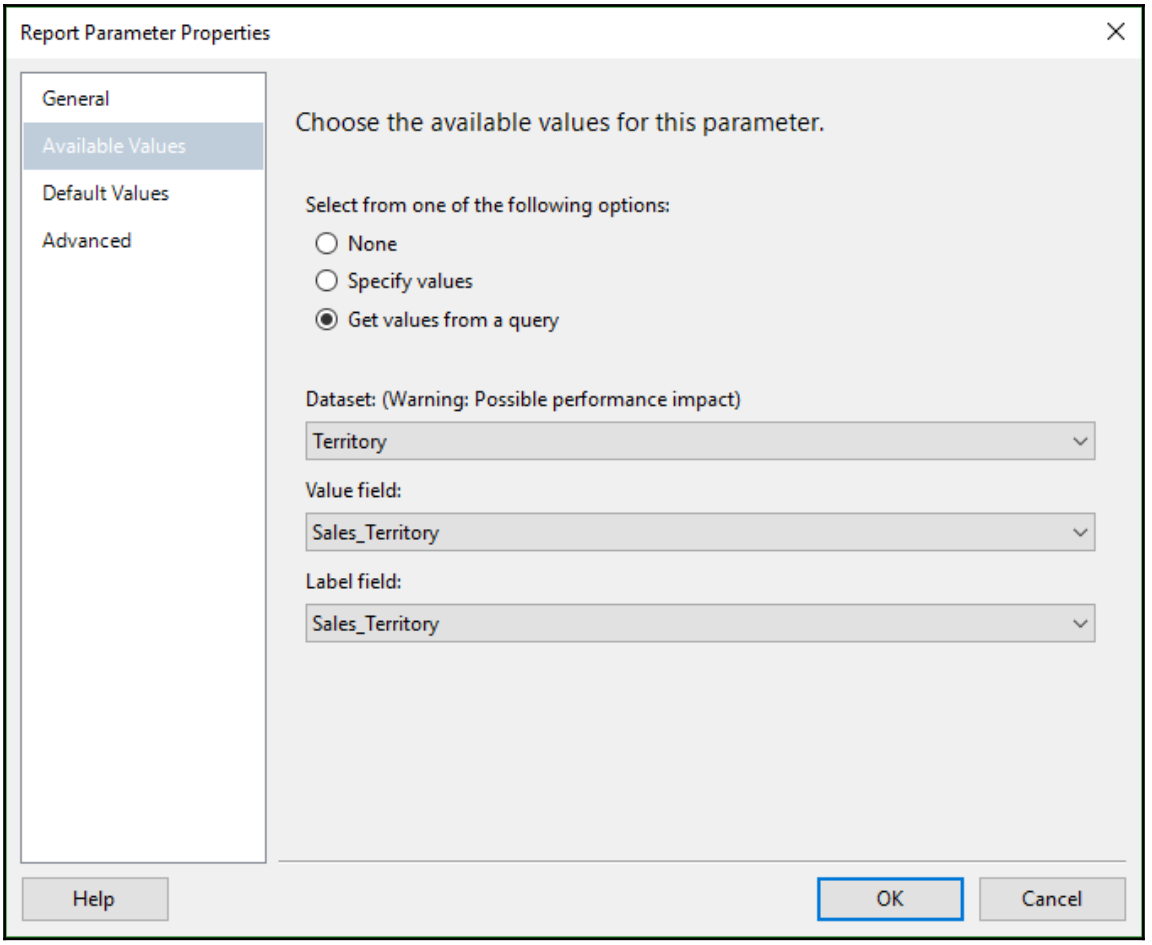


Figure 3.07

22. Now move to the **Default Values** page. If we had no default values, the user would be forced to select some values and click on view report before seeing any data. Here though, we want the report to run immediately when it is opened, using all the territories. That's what we'll do here.
23. Select the **Get values from a query** option.
24. For **Dataset**, pick **Territory** from the drop-down.

25. In the`Value` field dropdown, pick **Sales_Territory**:

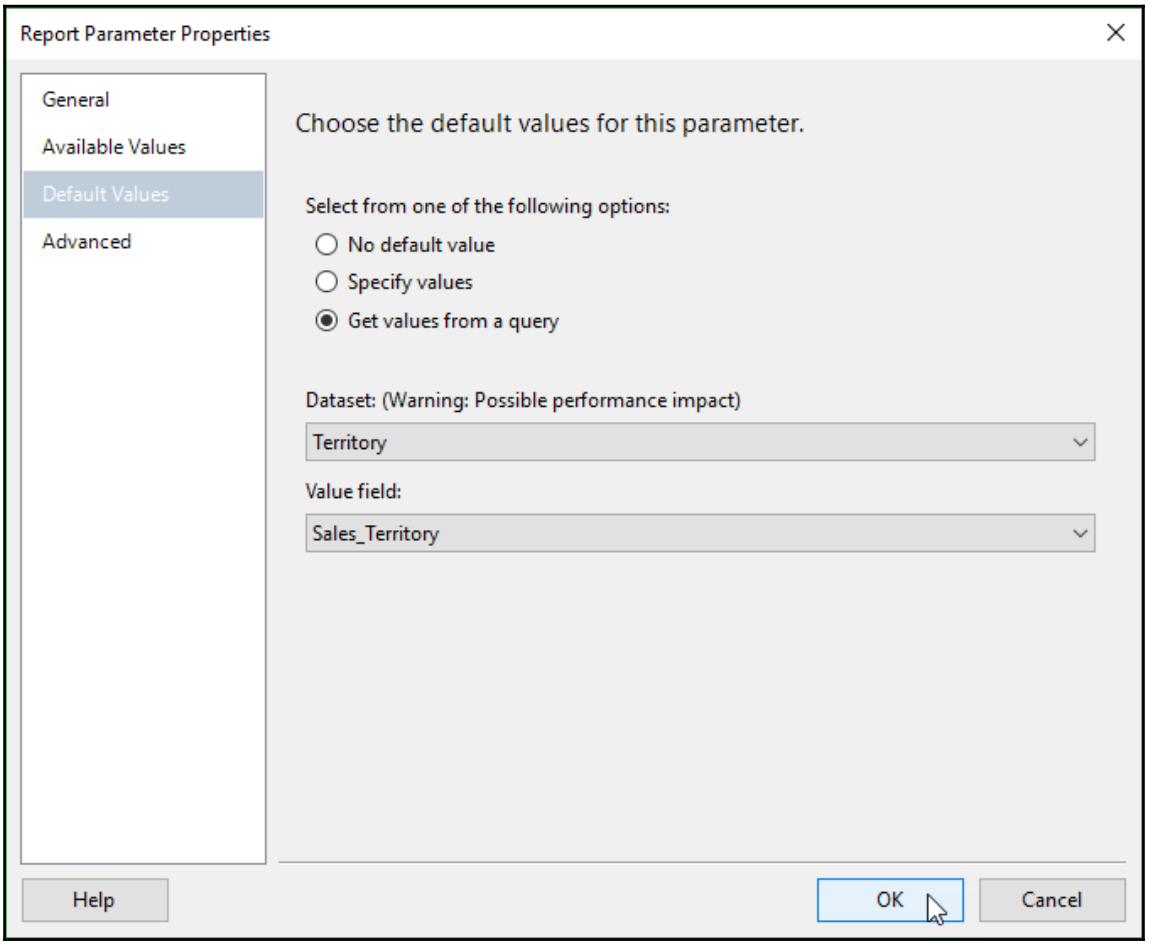


Figure 3.08

26. Click on **OK** to close the **Report Parameter Properties** window.
27. Now that we have the parameters, we can add the main dataset that will supply data to our report, and have it use our parameter. Right click on the **Datasets** branch and select **Add Dataset…**.
28. Name the dataset `TerritorySalesByYear`.
29. Select the **Use a dataset embedded in my report** option.

30. In the **Data** source drop-down, select `WideWorldImportersDW`.
31. Place the query found in the *Getting ready* section in the Query area.
32. Click on the **Parameters** page.
33. Because SSRS has found `@Territory` in the query, it knows we will be using a parameter. Thus you'll find a line has already been added to the page, with `@Territory` already filled in under Parameter Name. All you need to do is select `[@Territory]` from the drop-down under **Parameter Value**.
34. Click on **OK** to close the **Dataset Properties** window.
35. Now we can start building the report. Return to the Toolbox and drag and drop a Chart tool onto the report design surface. Enlarge it to make it easy to read.
36. For **Chart Type**, select the **Column chart** in the very upper left (this is the default chart type). Click on **OK**.
37. Click inside the chart to bring up the **Chart Data** window. Click on the plus sign next to **Values**. You'll note that this will now behave a bit differently than before; this is due to having multiple datasets. The top menu item will be the data source, then comes the name of the dataset, and finally we have the list of fields in that dataset. For this chart, select **WideWorldImporters** | **TerritorySalesByYear** | **Total_Sales**:

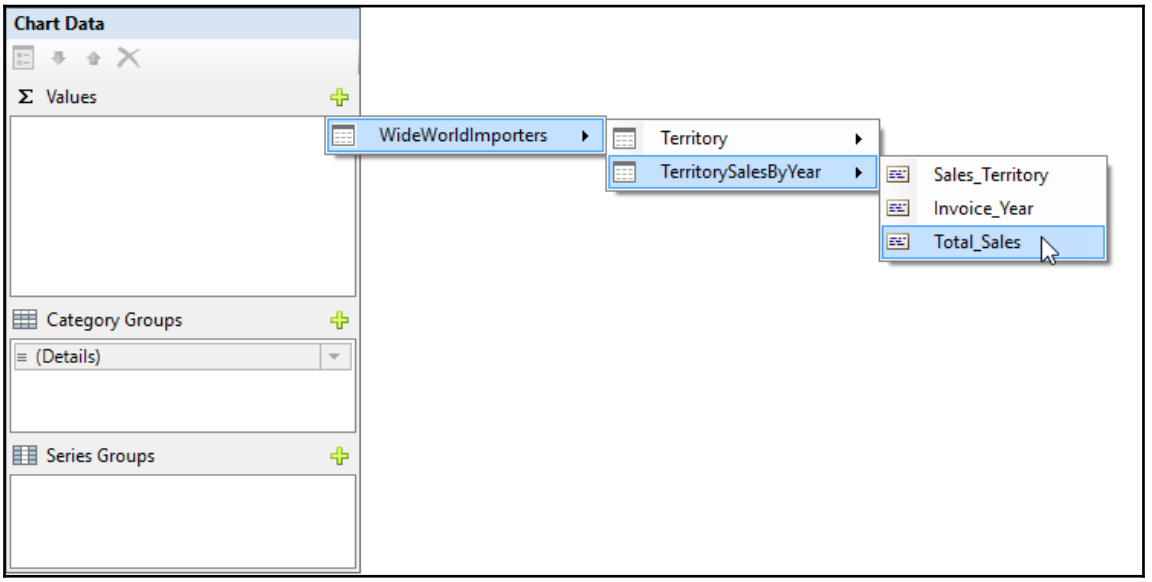


Figure 3.09

38. Click on the plus sign by **Category Groups**, and select **Invoice_Year** in the menu.
39. Now for another new step, click on the green plus beside **Series Groups** and select **Sales_Territory**. Series groups tell SSRS to expect multiple data points (the sales territories) to plot within a single category (that is, the invoice year).
40. Right-click on the Y-Axis scale to the left of the chart, and select **Vertical Axis Properties**.
41. Uncheck **Always include zero on the Axis Options** page.
42. On the **Number** page, set the category to **Currency**, 0 decimal places, and check the **Use 1000 separator**. Click on **OK** to close the window.
43. Click on the **Chart Title** and enter `Sales by Territory`.
44. Start expanding the report body by clicking and dragging the lower border of the body to give some space under the chart.
45. Into this blank space, drag and drop a Matrix control from the toolbox.
46. Click in the **Columns** cell. In the upper left, a little icon appears (see Figure 3.10):

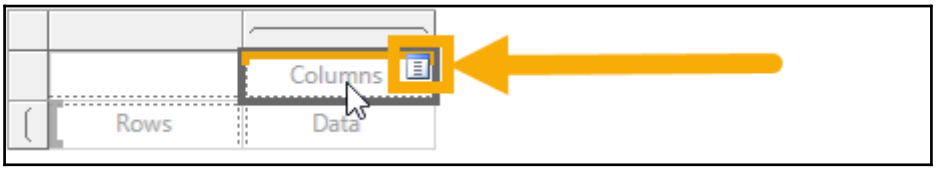
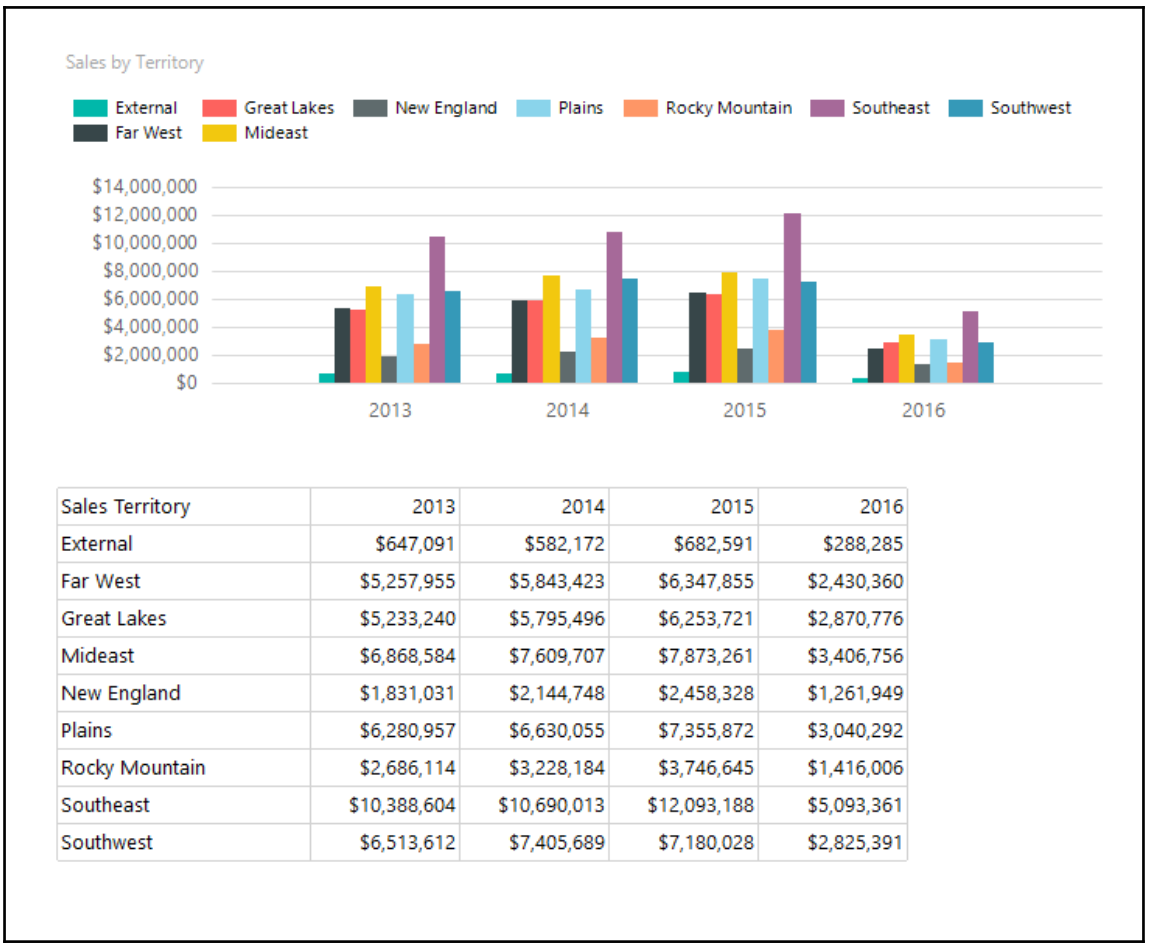


Figure 3.10

47. Click on it, and in the menu, navigate to **WideWorldImporters** | **TerritorySalesByYear** | **Invoice_Year**.
48. Note that SSRS may attempt to perform a **SUM** on **Invoice_Year**. If the cell reads `SUM(Invoice_Year)` instead of just `Invoice_Year`, right-click on the cell and pick **Text Box Properties**. In the dropdown for **Value**, change it to just `[Invoice_Year]`.
49. Click on the menu icon in the **Rows** cell, and select **Sales_Territory**.
50. Finally, in the **Data** cell, from the menu icon, select the **Total_Sales**.
51. Right-click on the cell with **Total Sales** and select **Text Box Properties** in the menu.
52. On the **Number** page, set the category to **Currency**, 0 decimal places, and check the **Use 1000 separator**. Click on **OK** to close the window.
53. Expand the first column, **Sales Territory**, by clicking on the divider in the gray bar at the top and dragging to the right to give the text in **Sales Territory** a bit more room.

54. Click on the **Preview** tab to see your work:

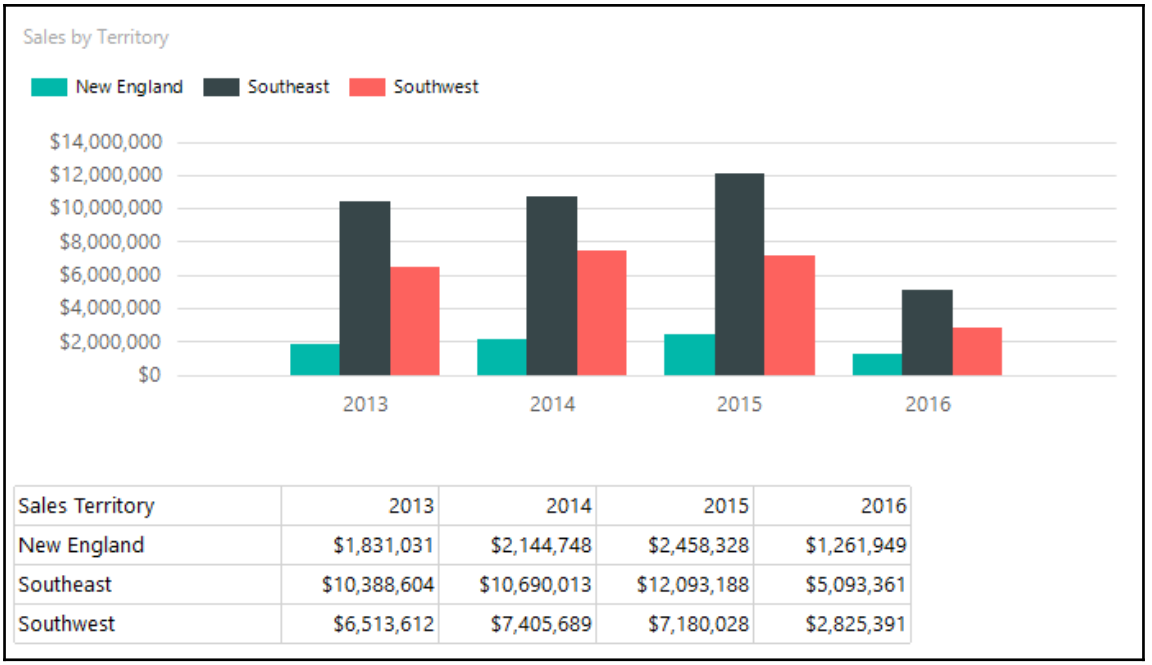


Sales Territory	2013	2014	2015	2016
External	$647,091	$582,172	$682,591	$288,285
Far West	$5,257,955	$5,843,423	$6,347,855	$2,430,360
Great Lakes	$5,233,240	$5,795,496	$6,253,721	$2,870,776
Mideast	$6,868,584	$7,609,707	$7,873,261	$3,406,756
New England	$1,831,031	$2,144,748	$2,458,328	$1,261,949
Plains	$6,280,957	$6,630,055	$7,355,872	$3,040,292
Rocky Mountain	$2,686,114	$3,228,184	$3,746,645	$1,416,006
Southeast	$10,388,604	$10,690,013	$12,093,188	$5,093,361
Southwest	$6,513,612	$7,405,689	$7,180,028	$2,825,391

Figure 3.11

55. Your chart should resemble the one just shown. Note how each territory is displayed within a year in the chart. In addition, the specific values are found in the following chart.

56. In the **Territory** parameter, click on **Select All** to deselect all the choices. Now select several of them for the report to focus on. For the next figure, I'll be picking **New England**, **Southeast**, and **Southwest**. Click on **View Report** to refresh the report, as shown in the following figure:



Sales Territory	2013	2014	2015	2016
New England	$1,831,031	$2,144,748	$2,458,328	$1,261,949
Southeast	$10,388,604	$10,690,013	$12,093,188	$5,093,361
Southwest	$6,513,612	$7,405,689	$7,180,028	$2,825,391

Figure 3.12

57. Note how both the chart and the matrix update to reflect our new choices.

How it works…

Creating reports with multiple data regions turns out to be pretty simple, conceptually. Each tool placed on a report is sourced from a dataset. As such, multiple tools can all use the same dataset as their source. When that dataset is refreshed, all tools that were connected to that dataset are refreshed.

In this case, we used the most common scenario -having both a chart and a matrix on the report. This allows the user of the report to get a quick overview of their data, but at the same time to be able to go into the details if desired.

You may have noticed that for the first field we added to the matrix, we had to specify the full path; the data source, dataset, and field name. After that we only had to select the field. Once you make the first selection, SSRS binds the matrix to that particular dataset. Since it now knows the dataset to use, it can easily provide the list of fields for the remaining columns in the matrix.

In addition, we added a parameter to the mix. Parameters can source from their own dataset, using them to supply a list of choices to the user. When possible, try to restrict users to a valid set of values, such as we did here with sales territories, rather than letting them type into a textbox. This prevents users from having to guess at correct names, as well as ensuring that only reports with valid queries are executed.

Creating reports with nested data regions

In the previous recipe, we used the concept of multiple data regions to display both a graph and a detailed text report in the same report body. Sometimes you would like to be able to provide the users with the *at a glance* information provided by a chart, but not have a chart tool because it takes up much of the report area. In other words, you'd like to embed a chart within the main body of your text.

SSRS allows you to accomplish this through the concept of *nested data regions*. In this recipe, we'll see how to create a matrix report and then add a sparkline graph to it in order to show the trending of sales totals for a specific sales territory over time.

Getting ready

For this report, we'll use the report wizard to quickly generate a report with a matrix. We'll then modify the report to add a new column, in which we'll embed our sparkline. A sparkline is a tiny chart that is embedded within tables and matrixes.

Naturally we'll need data. We'll use a similar query to those we've used before:

```
SELECT c.[Sales Territory]
     , YEAR(s.[Invoice Date Key]) AS [Invoice Year]
     , SUM(s.[Total Excluding Tax]) AS [Total Sales]
  FROM [Fact].[Sale] s
  JOIN [Dimension].[City] c
    ON s.[City Key] = c.[City Key]
 GROUP BY c.[Sales Territory]
     , YEAR(s.[Invoice Date Key])
 ORDER BY c.[Sales Territory]
     , YEAR(s.[Invoice Date Key])
```

How to do it…

1. Right-click on the **Reports** branch of **Solution Explorer**, and select **Add New Report…** on the menu.
2. When the **Welcome to the Report Wizard** screen appears, just click on **Next** (note that if you wish to skip this screen in the future, you can check on the **Don't show this page again** option).
3. Use the **Shared data source** option, and select `WideWorldImportersDW` in the dropdown. Click on **Next**.
4. Paste or type in the query from the *Getting ready* section into the Query string area of the **Design the Query** page and click on **Next**.
5. Select **Matrix** for the **Report Type** and click on **Next**.
6. On the **Design the Matrix** page of the wizard, place the `Total_Sales` in the Details area, `Sales_Territory` in **Rows**, and `Invoice_Year` in Columns. Then click on **Next**.
7. Name the report `Report 03-03 Nested Data Region` and click on **Finish**.
8. We'll take a moment to fix the display of the numbers. Right-click in the **[Sum(**Total_Sales**)]** cell of the matrix and pick **Text Box Properties** from the menu.
9. Click on the **Number** page. Under **Category**, select **Currency**. Set the decimal places to 0 and check **Use 1000 separator**. Click on **OK** to close the window.

10. Right-click again in the **[Sum(Total_Sales)]** cell of the matrix. In the menu, select **Insert Column | Outside Group – Right**:

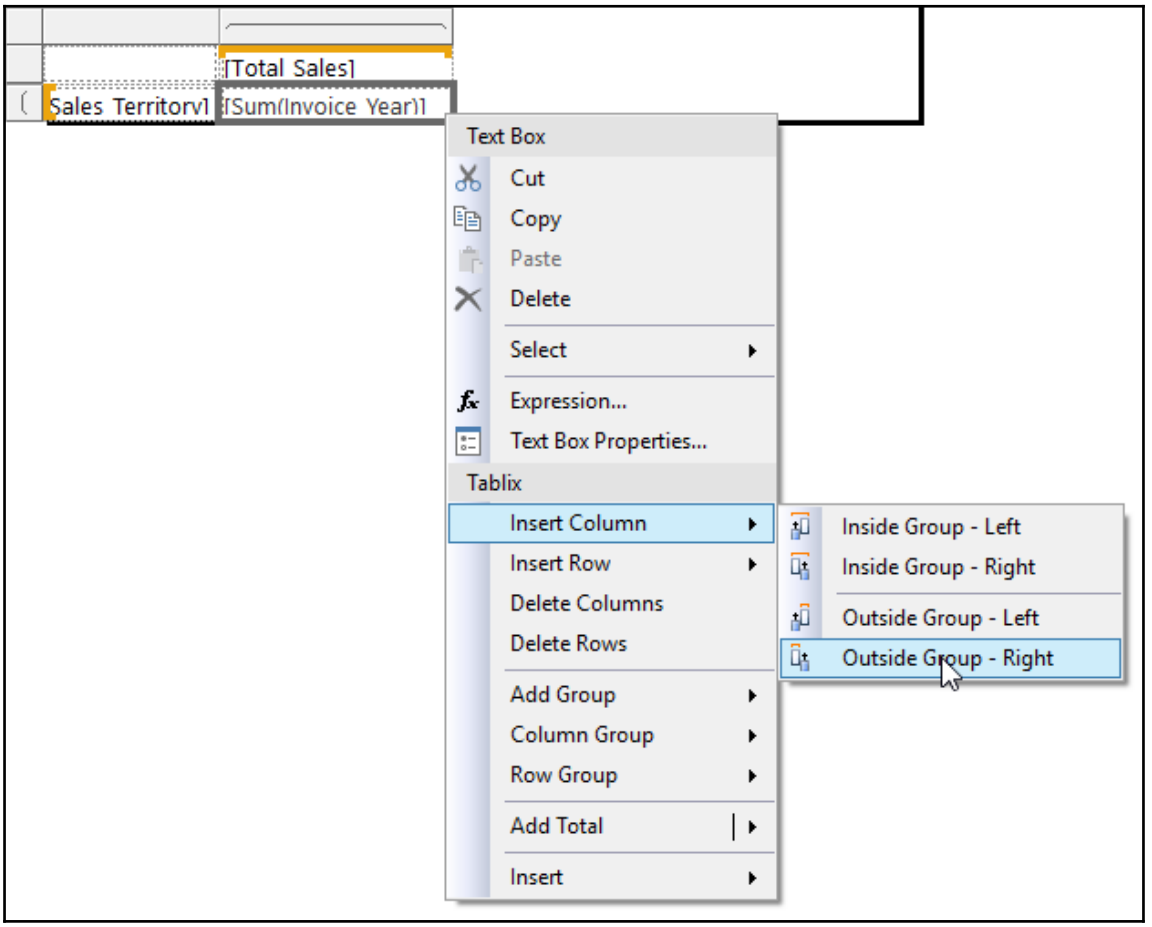


Figure 3.13

11. Right click in the new column in the cell directly to the right of **[Sum(Total_Sales)].** In the menu, select **Insert** | **Sparkline**:

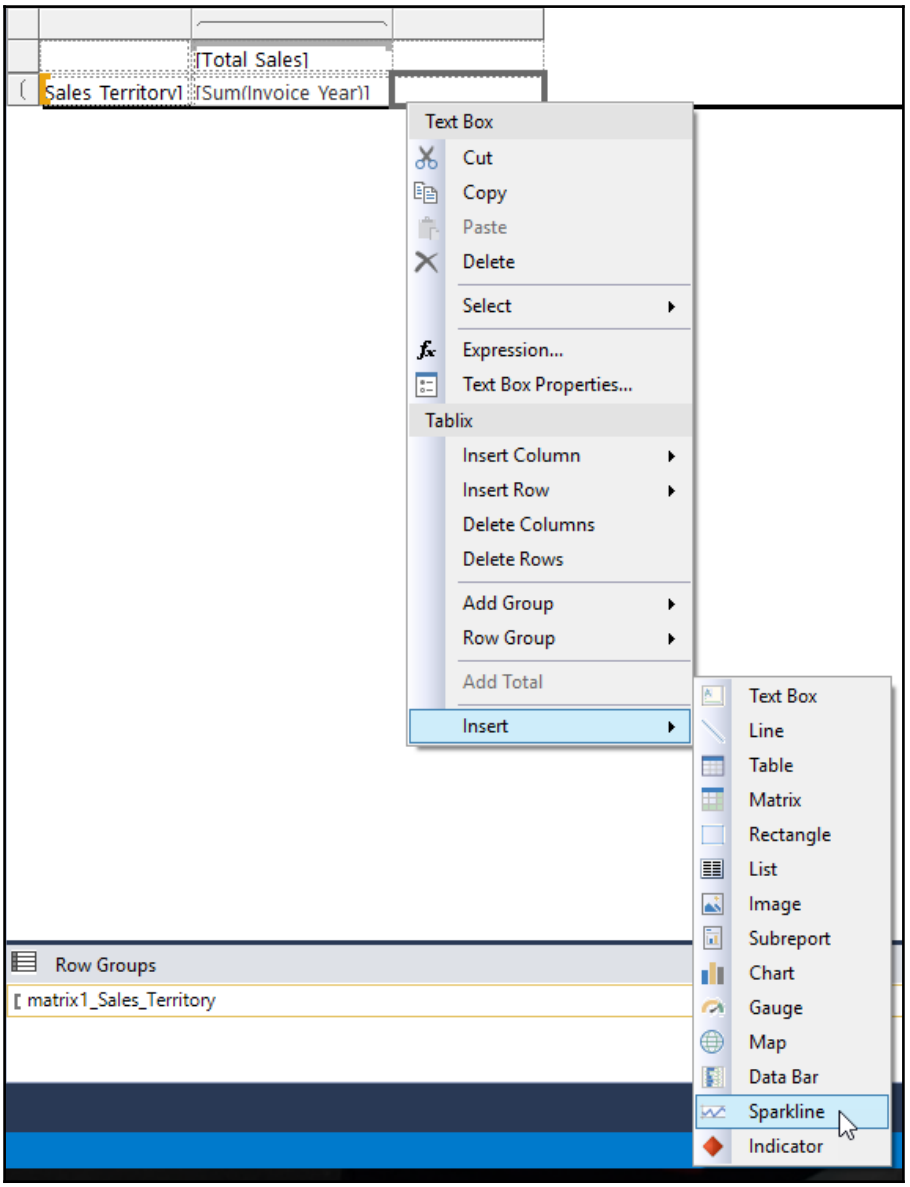


Figure 3.14

12. When the **Sparkline Type** window appears, you will see a wide variety of options to select from. For this example, let's pick the leftmost item under Area:

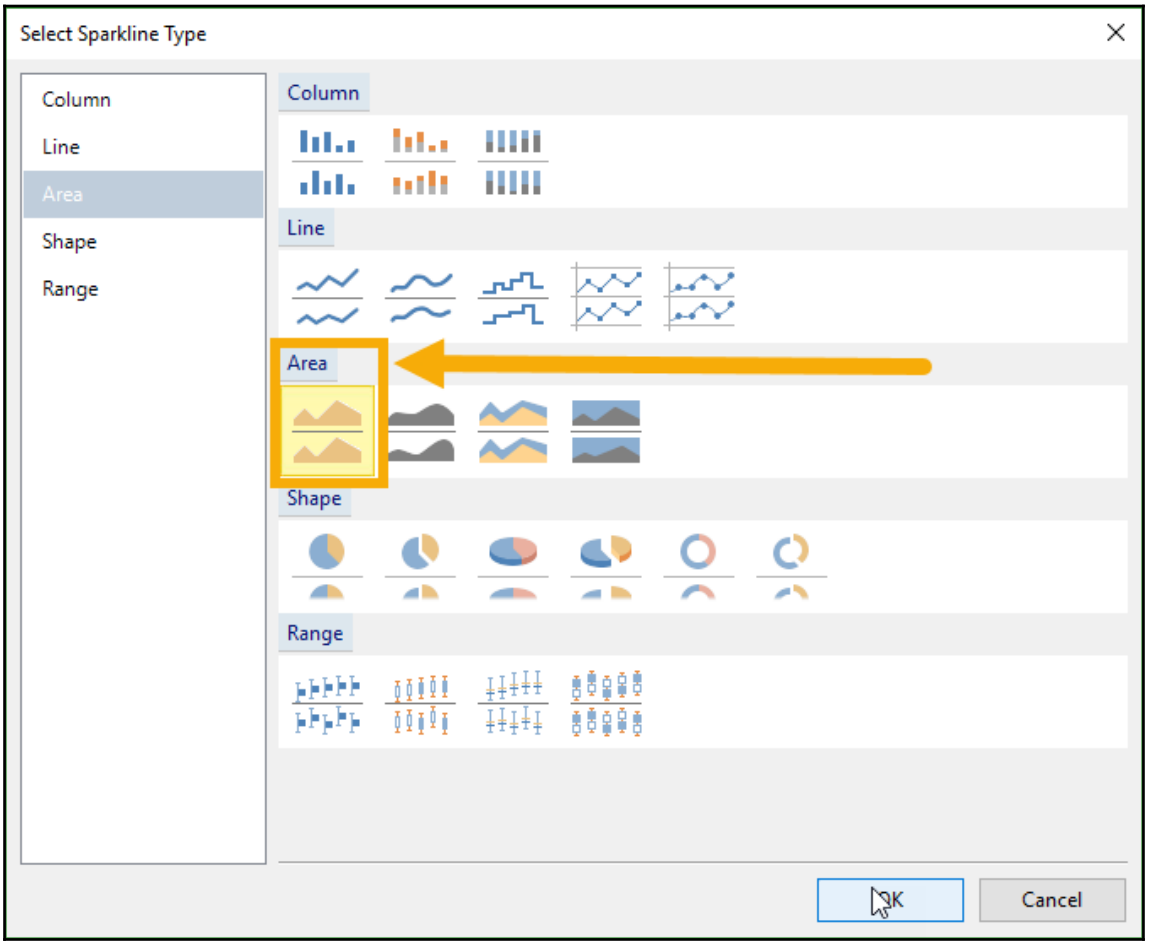


Figure 3.15

13. Click on **OK** to close the **Select Sparkline Type** window.
14. Click in the cell with the new sparkline, and the **Chart Data** window should appear. Click on the plus sign beside **Values** and select **Total_Sales**.

15. Go to the **Preview** tab. You should now see a sparkline that reflects the change in values of the individual total sales appearing to the left, as shown in the following figure:

	2013	2014	2015	2016	
External	$647,091	$582,172	$682,591	$288,285	
Far West	$5,257,955	$5,843,423	$6,347,855	$2,430,360	
Great Lakes	$5,233,240	$5,795,496	$6,253,721	$2,870,776	
Mideast	$6,868,584	$7,609,707	$7,873,261	$3,406,756	
New England	$1,831,031	$2,144,748	$2,458,328	$1,261,949	
Plains	$6,280,957	$6,630,055	$7,355,872	$3,040,292	
Rocky Mountain	$2,686,114	$3,228,184	$3,746,645	$1,416,006	
Southeast	$10,388,604	$10,690,013	$12,093,188	$5,093,361	
Southwest	$6,513,612	$7,405,689	$7,180,028	$2,825,391	

Figure 3.16

How it works…

Sparklines are a very useful tool for conveying information on a small scale. If you refer back to Figure 3.14, you'll note that sparklines aren't the only thing that can be nested. Indicators and Data bars are also useful tools that can be nested in your tables and matrixes.

The first step to making this work was the addition of another column. It was very important to select the *Outside Group* option. By placing it outside the group, it tells SSRS to first create a column for each year. Only after it has generated all the yearly totals columns does it add the new column.

In the new column, we inserted our sparkline control. Sparklines consist of many formats. While the line chart type is the most common, it can be difficult to read at times on a small report. Area sparklines are a good alternative for readability.

After adding the sparkline, we must tell it the data value to use. Like other charts, all we have to do is click inside to make the chart data window appear, and then add the value to graph (in this case, **Total Sales**) to the **Values** area.

Creating reports with a map data region

Data visualization is becoming increasingly important to the world of reporting. This is especially true in terms of sales being able to determine sales information based on geographical data at a glance.

SSRS provides an easy-to-use mapping tool, providing the ability to take your data and place it on a map. This gives quick, easy-to-understand visuals for your sales team to determine hot spots for sales or see what geographic areas need attention.

Of course, providing reports for the sales team, while the most common scenario, is not the only application for mapping. Imagine being able to plot the outbreak of flu on a map, or map the location of new drinking water wells, including their capacities, in desert locations.

Getting ready

In order to use a map, there must be some data that a map can recognize as an entity within a map. State, Country, City, and Zip Code are examples of fields that a map control can understand and thus be able to place data onto.

`WideWorldImporters` has map data contained in the `Dimension.City` table. For this recipe, we'll use the following query, which will return sales data on a state-by-state basis:

```
SELECT c.[State Province]
     , SUM(s.[Total Excluding Tax]) AS [Total Sales]
  FROM [Fact].[Sale] s
  JOIN [Dimension].[City] c
    ON s.[City Key] = c.[City Key]
 GROUP BY c.[State Province]
 ORDER BY c.[State Province]
```

How to do it…

1. Right-click on the **Reports** branch in the **Solution Explorer**.
2. Select **Add** | **New Item**… from the pop-up menu.
3. On the **Add New Item** dialog, select **Report** from the choice of templates in the middle (do not select **Report Wizard**).
4. At the bottom, name the report `Report 03-04 Maps.rdl` and click on **Add**.
5. Go to the **Report Data** tool pane. Right-click on **Data Sources** and then on **Add Data Source…** from the menu.
6. In the **Name:** area, enter `WideWorldImportersDW`.
7. Change the data source option to **Use shared dataset** reference.
8. In the dropdown, select `WideWorldImportersDW`.
9. Click on **OK** to close the **Data Source Properties** window.
10. Right-click on the **Datasets** branch and select **Add Dataset…**.
11. Name the dataset `SalesByState`.
12. Select the option **Use a dataset embedded in my report**.
13. Select `WideWorldImportersDW` in the **Data** source dropdown.
14. Paste in the query from the *Getting ready* area of this recipe.
15. Click on **OK** to close the window.
16. Go to the **Toolbox** pane.
17. Place a map control onto the report. When you do, the **New Map Layer** window opens.
18. The default is **Map gallery**, which will be used for this recipe.

19. In the map gallery section at the bottom, select **USA by State Exploded**. When you do, a preview of what the map looks like will appear to the right:

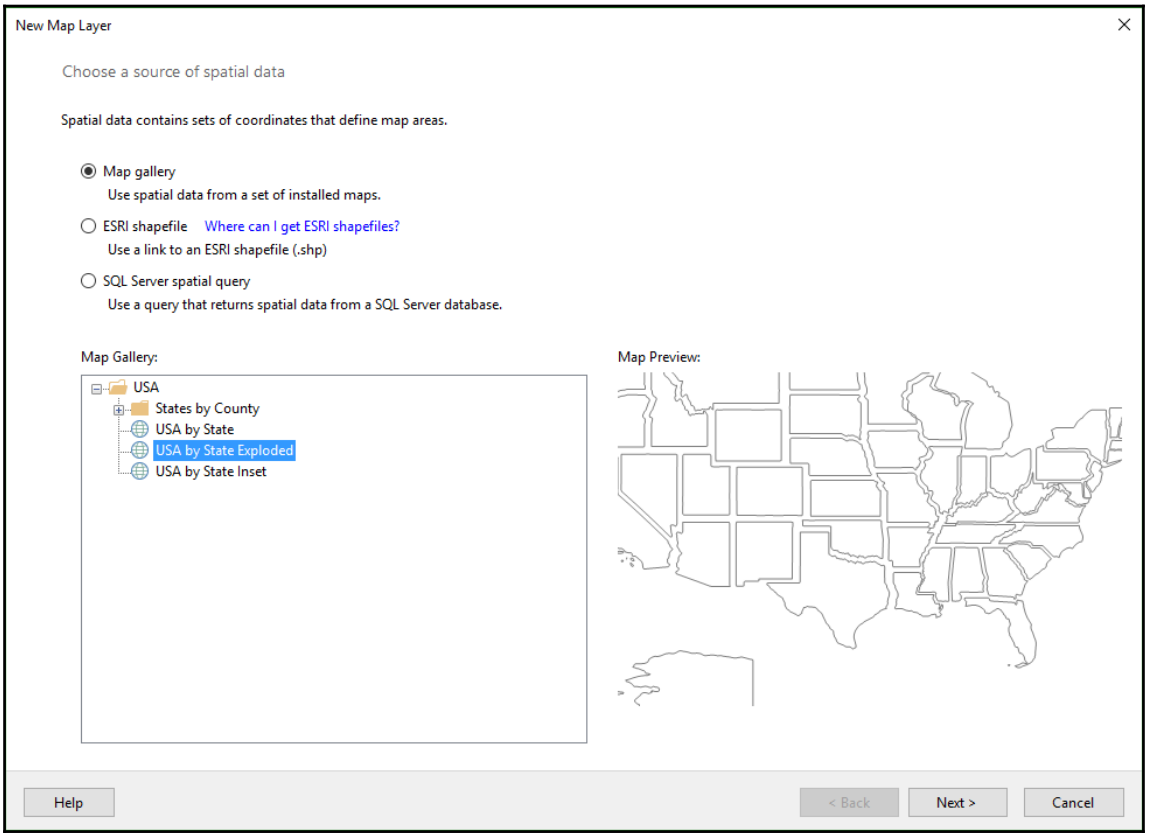


Figure 3.17

20. Click on **Next** to proceed to the next page in the wizard.

21. Depending on your system, the map may appear to take up more than the space in the frame. If so, click on the map and use the scroll wheel of your mouse to resize it. Alternatively, use the slider bar below the four arrow buttons on the left to resize, as shown in the following figure:

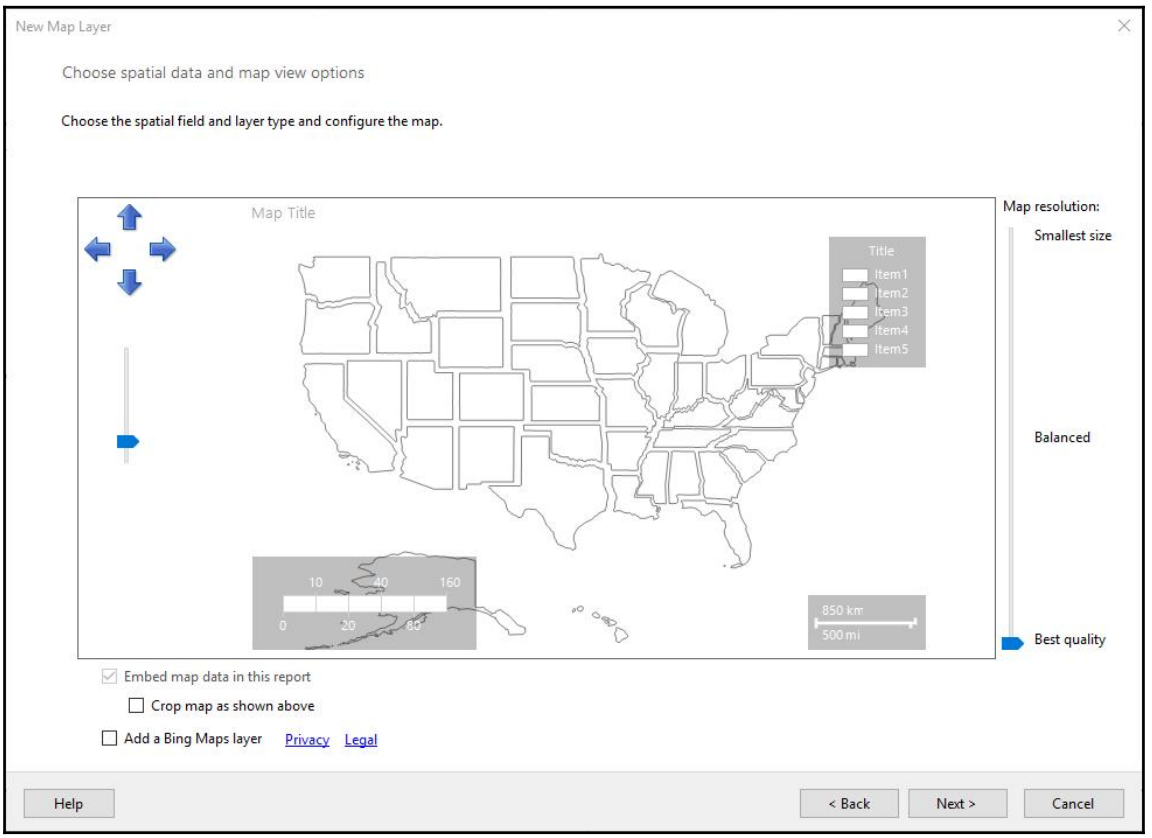


Figure 3.18

22. Click on **Next** to move to the next step.

23. You are now asked what type of visualization should be used with the map. Select **Color Analytical Map** and click on **Next**:

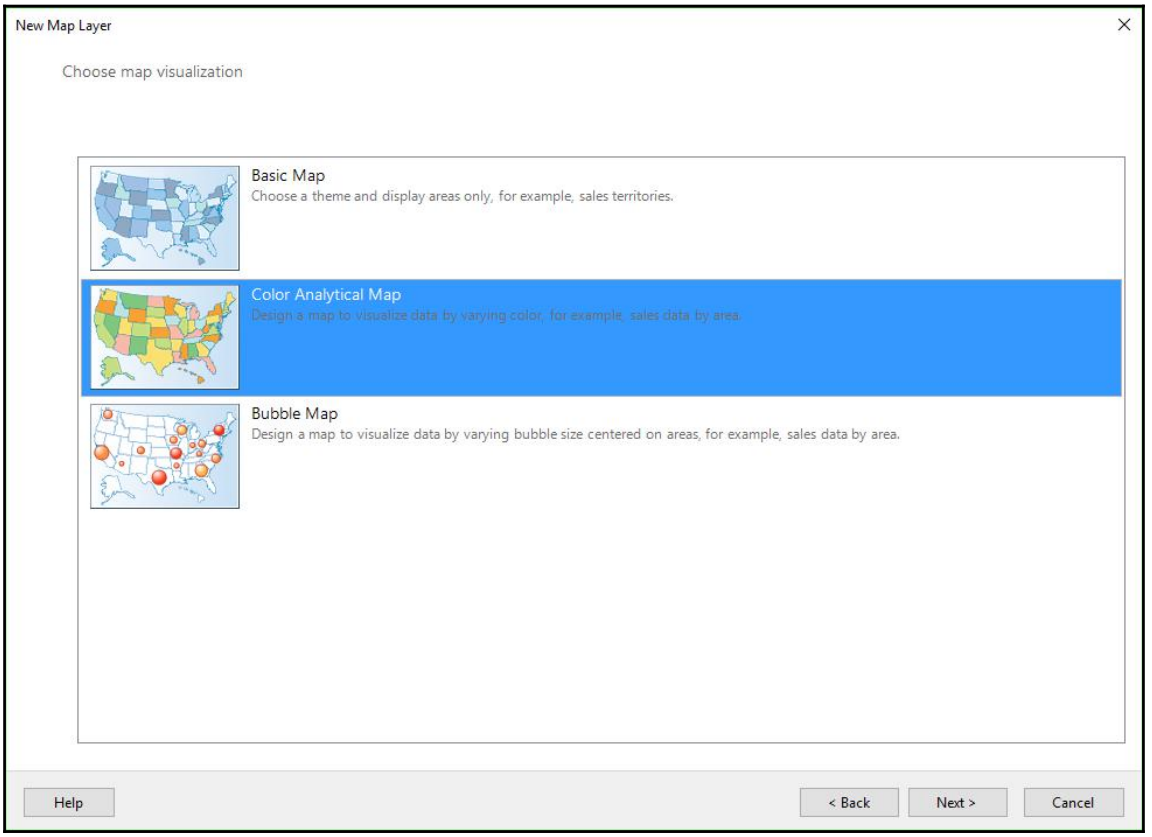


Figure 3.19

24. The map now needs to know what dataset to use. Since we only have one, **SalesByState**, we click on it and then click on **Next**.
25. The next screen is where the data is linked to the map, consisting of **Analytical Data** and **Spatial data**.

26. Check **STATENAME**. The analytical dataset field is **State_Province**:

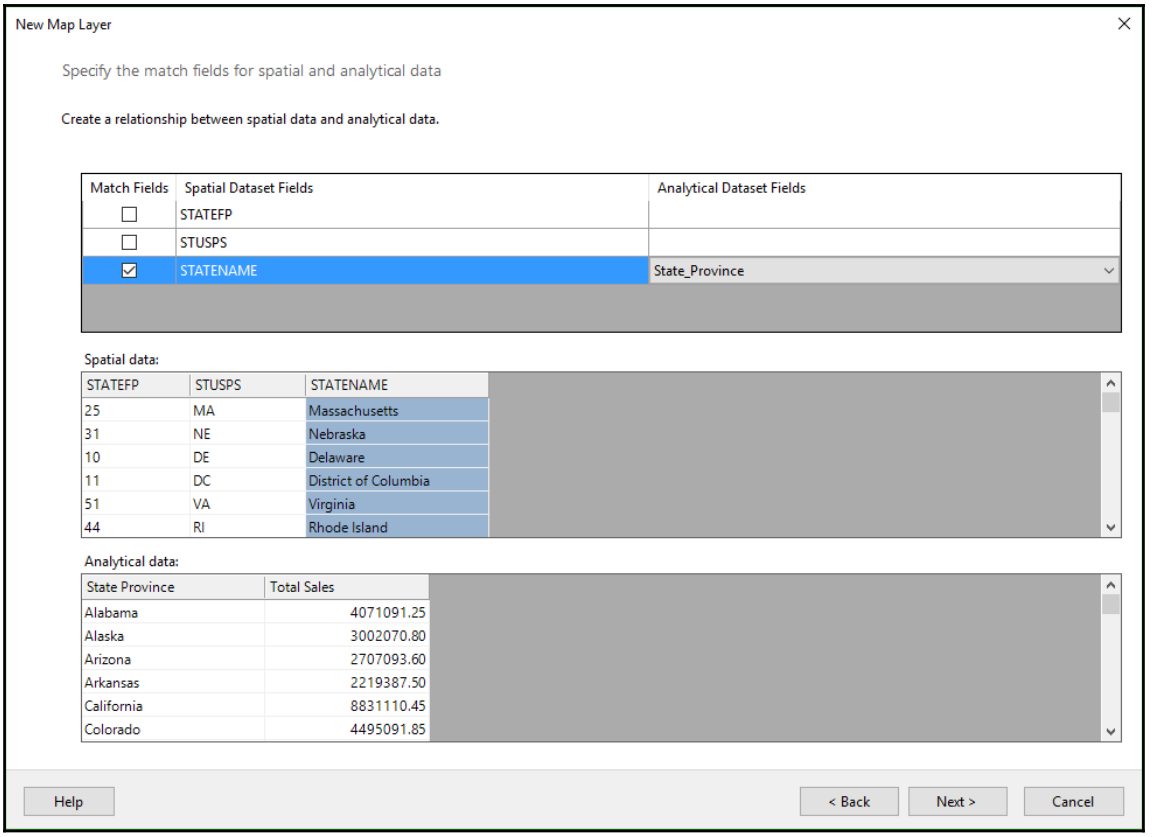


Figure 3.20

27. Click on **Next** to proceed.
28. Now we need to let the map know what field it should be visualizing. Under **Field to visualize**, use the dropdown to select **[Sum(Total_Sales)]**.

29. The **Color rule** tells the map how to associate colors with values. The default, Green-Yellow-Red, indicates that smaller numbers are good, and larger numbers are bad. For sales, this is the opposite of what we want, so use the dropdown to change it to **Red-Yellow-Green**:

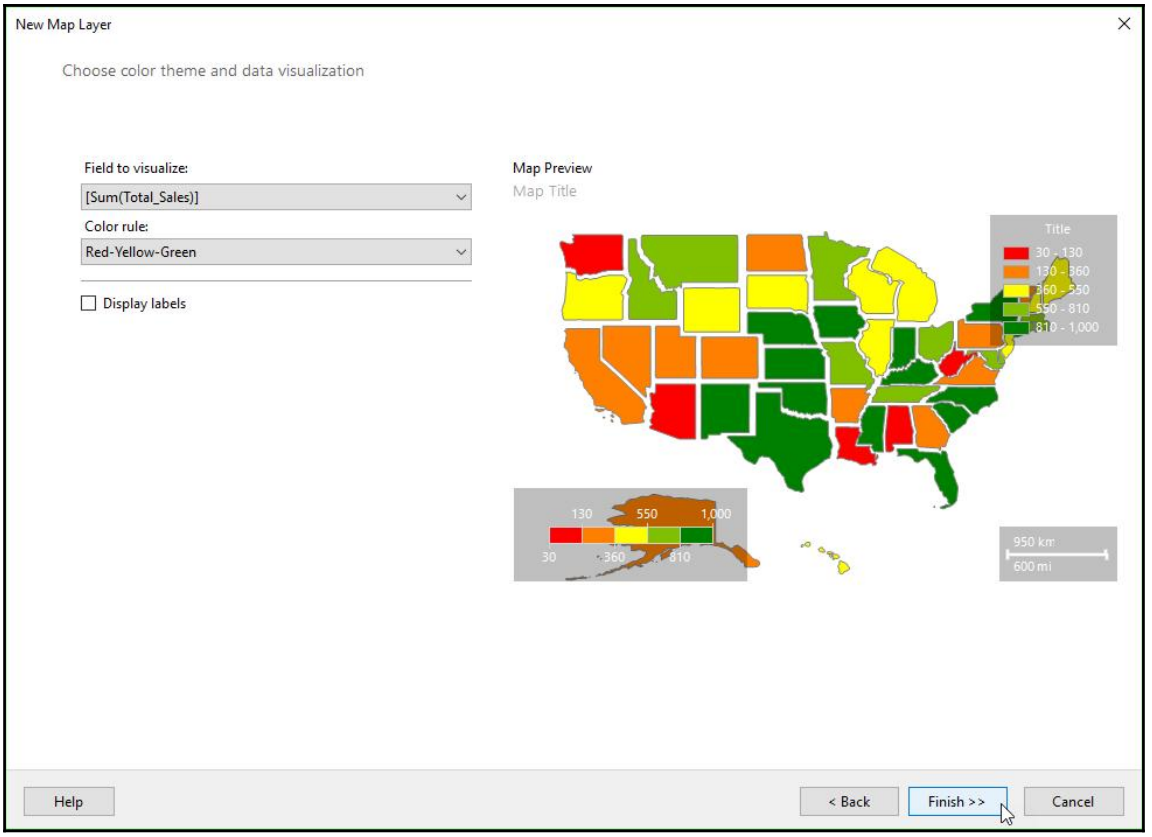


Figure 3.21

30. Click on **Finish** to complete the map.

31. With this report, both the color scale in the lower left and the ranges in the upper right, are redundant. Right-click on the color scale and click on **Show Color Scale** to uncheck it:

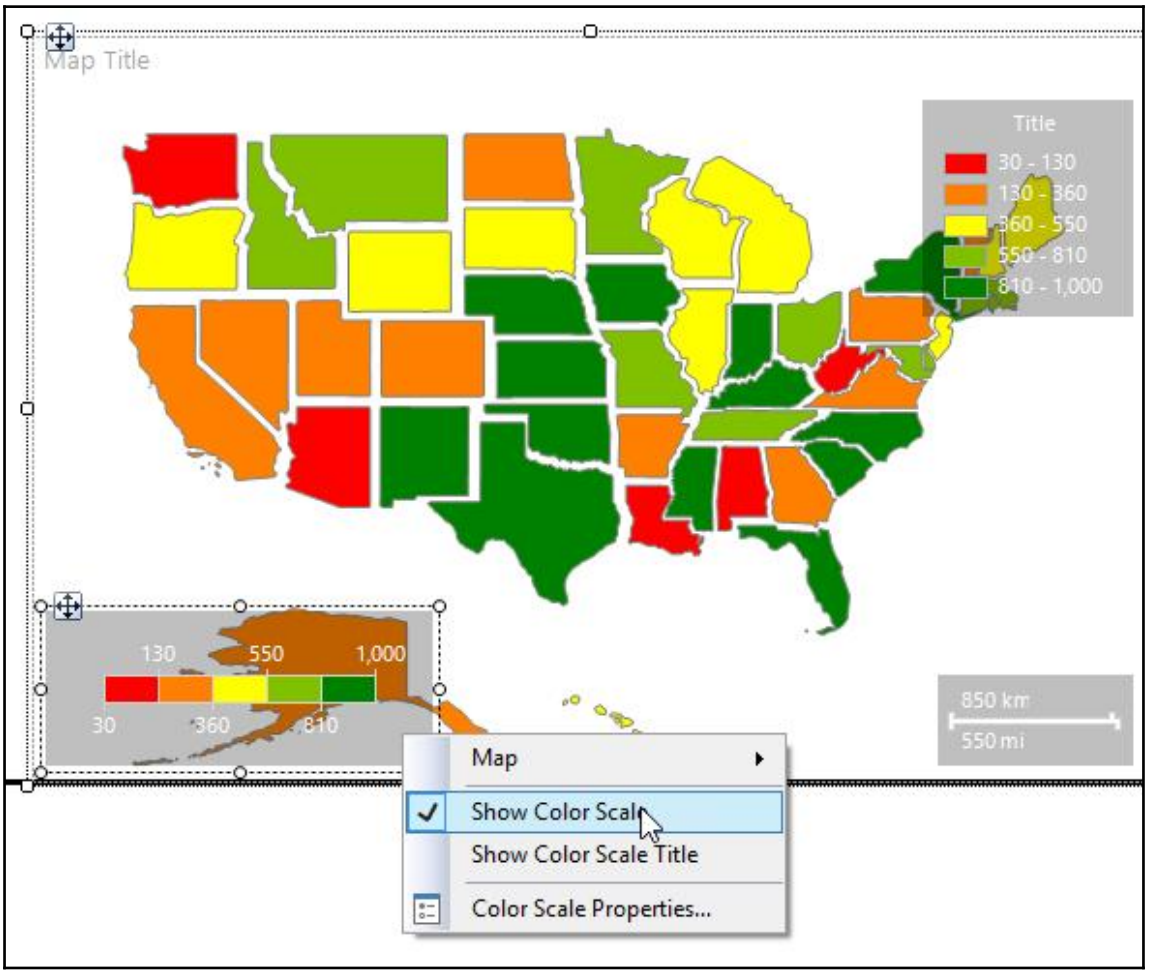


Figure 3.22

32. Likewise, the distance scale on the lower right isn't overly helpful. Right-click on it and select S**how Distance Scale** to hide it.

33. Move the legend off of **New England** by simply clicking and dragging it down to the lower-right corner.
34. With the legend in place, right-click and select**Show Legend Title** to hide it.
35. Click on **Map Title**, and change it to **Sales Total By State**. In designer mode, your map should now look like Figure 3.23:

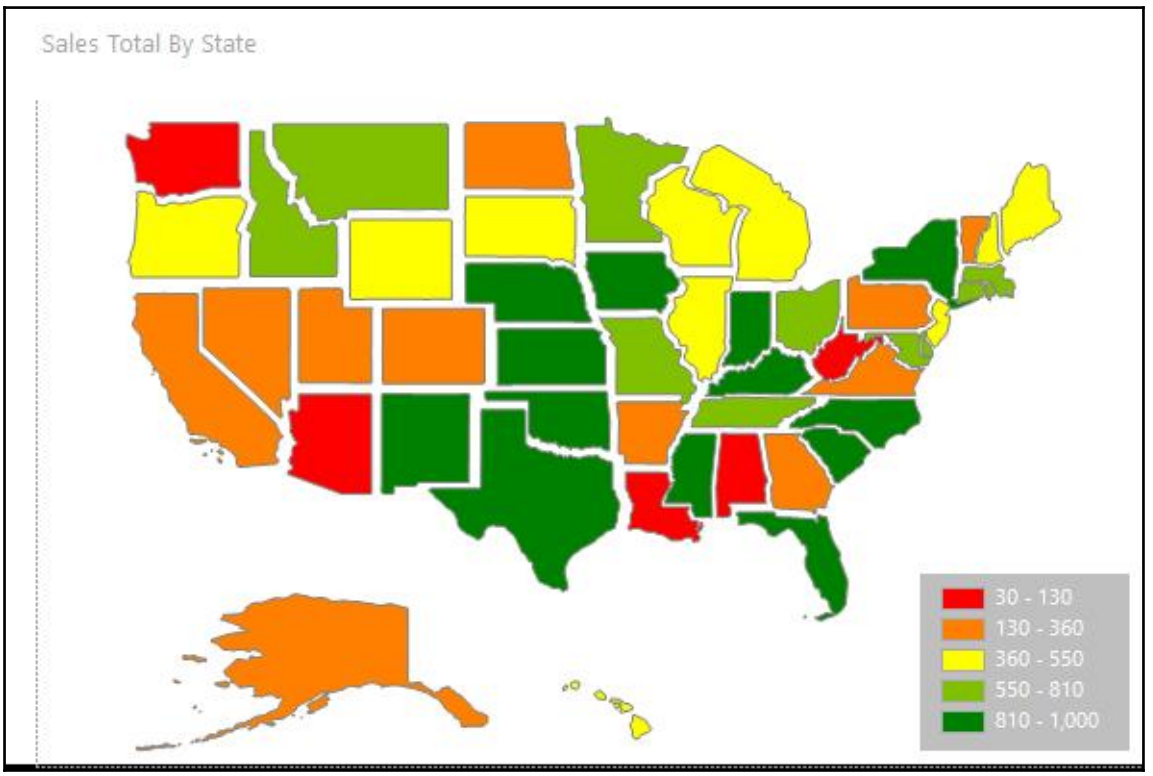


Figure 3.23

36. With all the elements in place, go to the **Preview** tab to see the results:

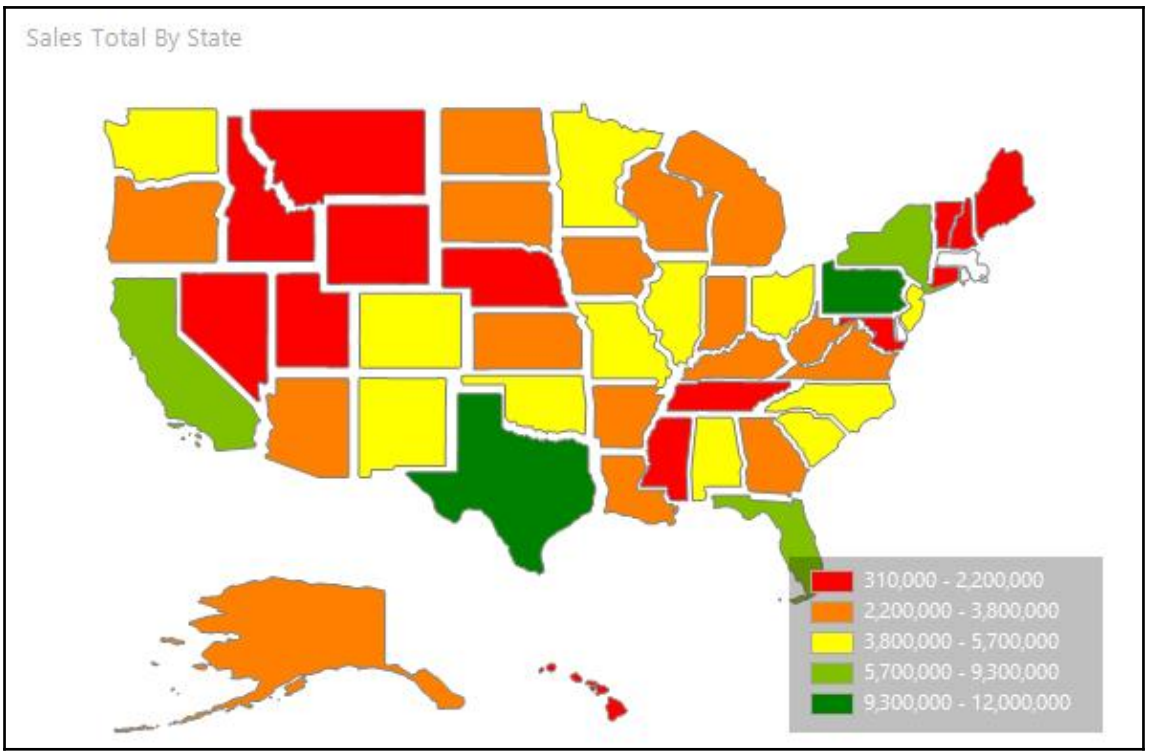


Figure 3.24

37. The map reflects the data in our table; the lower sales numbers appear in red, while the higher numbers are in green.

How it works…

The support that SQL Server Reporting Services offers for mapping is pretty amazing. Of course, it has to start with your dataset. In it, you must have values recognized by your map. In our recipe, it was the name of the state.

In Figure 3.17, the first page of the map wizard, you were asked to select the map to use. SSRS includes built-in maps for the United States within the map gallery. You can easily select one of these views within your report.

What if you need a map other than the US? That's where ESRI shapefiles come in. In Figure 3.17, to the right of the **ESRI shapefile** option is a link labeled **Where can I get ESRI shapefiles?**. Through this link, you can find and download ESRI maps for other countries, as well as for the entire planet earth. Once downloaded, you simply select the ESRI option and point to the map file.

The other option is to use the SQL Server spatial data type. A spatial data type contains information for drawing maps within itself.

The next page of the wizard (Figure 3.18) provides a way to resize the map to your needs. For example, if you are only concerned with continental USA, you can use it to crop out Hawaii and Alaska.

There is also an option to add a Bing map layer. This will require an Internet connection when your report is run, but it will provide the ability to overlay a map layer with roads, geography, and the like.

The third page, shown in Figure 3.19, gives the ability to choose the type of map. The Basic Map option just displays a map and is not overly useful. The Color Analytical Map is what we used in the recipe, it color-codes each geographical region (in this recipe, states) according to the values you choose to map.

The third option, Bubble Map, places a bubble over the map coordinate. The bubble is scaled according to the value being charted. A value of 10 might produce a small bubble, a value of 33 a slightly larger bubble.

The next page of the wizard simply displays a list of available datasets, and allows you to pick one. In our case we only had one, but as you've seen in other recipes, it is quite possible for a report to contain multiple datasets.

The next page of the wizard, shown in Figure 3.20, is probably the most important. It is where you map the fields in your report to the data types the map understands. Each map contains built-in data, which it understands. Those map fields and examples of data are held in the center piece of the page, under **Spatial Data**.

At the bottom is **Analytical Data**, the data in your dataset. It allows you to see the field names from your dataset as well as samples of the data contained in it. This makes matching up your data with the data in the map easy.

That mapping is established in the top grid. It lists each field from the map in the left column. To match one, simply place a check mark next to the map field and it will activate the cell next to it, under the **Analytical Dataset Fields** column. Simply use the dropdown to pick the corresponding field in your dataset.

You must choose at least one field in order for the map to work. However, if you have multiple fields, the more you can match, the more accurately your map will be drawn.

The last page of the wizard allows you to select the color scale. Maps use colors to relate to a range of values. The color selection is set within this page. By default, the map tool uses Green-Yellow-Red. Lower values always occur first and higher values last. Thus, by default, it would have shown our low sales in green and high sales in red.

For our sales data, this is the opposite of what we want. The higher the sales, the better; so we used the dropdown to pick the opposite value of Red-Yellow-Green. Now high sales will show in green and low sales in red.

Speaking of our total sales, we had to tell SSRS that that was the field we wanted to place on the map. That's what we did in the upper dropdown, under **Field to visualize**.

After the map was generated, we did a little cleanup. For our report, the distance scale on the lower right was of little use for our end users, so we removed it as we did with the color scale.

We then moved the legend from the upper right to the lower left so that we could see the New England area.

What we've shown here is the tip of the iceberg when it comes to mapping. There are many customizations that can occur. With this recipe though, you can quickly get up and running, adding attractive-looking maps to your reports.

Creating reports with cascading parameters

In reporting services, it can be useful to have parameters in which the choice of one parameter is dependent upon another parameter. In this recipe we'll show how to first load up a list of states. Then, once the user selects a state as a second parameter, City will populate with a list of cities for that state.

This recipe will also build on to the next recipe, *Creating reports with dynamic datasets.*

Getting ready

Create a new report in your `Chapter 3` solution, and name it `Report 03-05 Cascading Params and Dynamic Datasets.rdl`. Use the **Add | New Item…** menu option, and just pick **Report**, don't use the **Report Wizard**.

How to do it….

1. Add a shared data source, pointing to `WideWorldImportersDW`.
2. Add a new Dataset. Name it `State` and use the `WideWorldImportersDW` data source. Select the embedded dataset option, use the SQL as the query, and then click on **OK** to close the window:

```
SELECT DISTINCT
       [State Province]
 FROM [Dimension].[City]
ORDER BY [State Province]
```

3. Add a new parameter (see the *Managing and configuring parameters* recipe in `Chapter 2`, *Authoring Reports with SQL Server Data Tools*).
4. Enter `State` for both **Name** and **Prompt**.
5. Click on **Available Values**. Select **Get Values** from a query.
6. Select **State** for the Dataset.
7. For both the **Value** field and the **Label** field, select **State_Province**. Click on **OK** to close the parameter dialog.
8. Add another dataset. Use **City** for the name and `WideWorldImportersDW` for the data source. For the Query, use the following SQL statement:

```
SELECT DISTINCT [City]
  FROM [Dimension].[City]
 WHERE [State Province] = @State
 ORDER BY [City]
```

9. Click on the **Parameters** page. The `@State` parameter should already be populated. In the dropdown beside it, select `@State` and then click on **OK**.
10. Add another parameter. For both **Name** and **Prompt**, use **City**.
11. On the **Available Values** page, select **Get Values** from a query.
12. For **Dataset**, **Value Field**, and **Label field**, select **City**, and then click **OK** to close the window.

13. Click on **Preview** to see the parameters. Note that while **State** presents a list of drop-down values, **City** is disabled, nothing appears.
14. Select a state in the dropdown, such as **Alabama**. Note that doing so now refreshes the parameters area. City is now populated, and you can select a value from the list, such as **Tuscaloosa**:

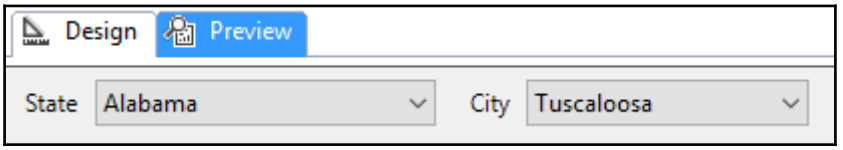


Figure 3.25

How it works…

The first parameter, **State**, works like a normal parameter. For the second parameter though, SSRS detected that its data source included a reference to the State parameter. When SSRS sees this, it waits to populate the **City** parameter until the user has made a selection in the first parameter.

Cascading parameters are fairly common in SSRS. Any time your data has a hierarchy, users will often need to drill down to the lowest level in order to execute a report. Here State and City were used. Other examples might be product categories, subcategories, and products, or various levels in your company's department tree.

Creating reports with dynamic datasets

As powerful as the SQL language is, sometimes it is not enough. You need to create complex logic in order to generate the SQL used in your dataset. For example, you have two tables with similar columns. You want to let the user select which table to use as the data source at runtime. We'll do just that with this recipe.

Getting ready

This recipe builds on top of the previous one, *Creating reports with cascading parameters*. If you've not done so, please create the report following that recipe. If it has been created, open it to begin working.

How to do it…

1. Add a new parameter. Use `ReportSource` (no spaces) as the name and `Report Source` (with a space) as the prompt.
2. On the **Available Values** page, select the **Specify Values** option.
3. Click on the **Add** button, and then enter `Order` for both **Label** and **Value**.
4. Click on **Add** again, and enter `Sale` as both **Label** and **Value**.
5. Click on **OK** to close the properties dialog.
6. In the report parameters area at the top of the report, click and drag **Report Source** under **State:**

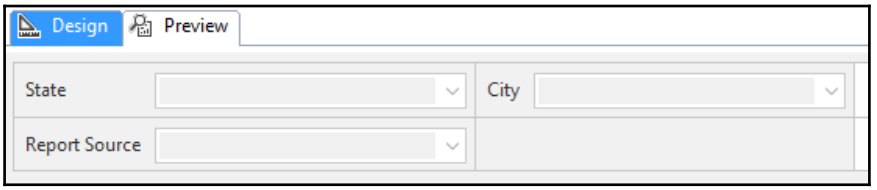


Figure 3.26

7. Add a new dataset. For the name, use `ReportData`. Use `WideWorldImportersDW` as the data source. Finally, use the following `select` statement for the query and click on **OK** to close the window:

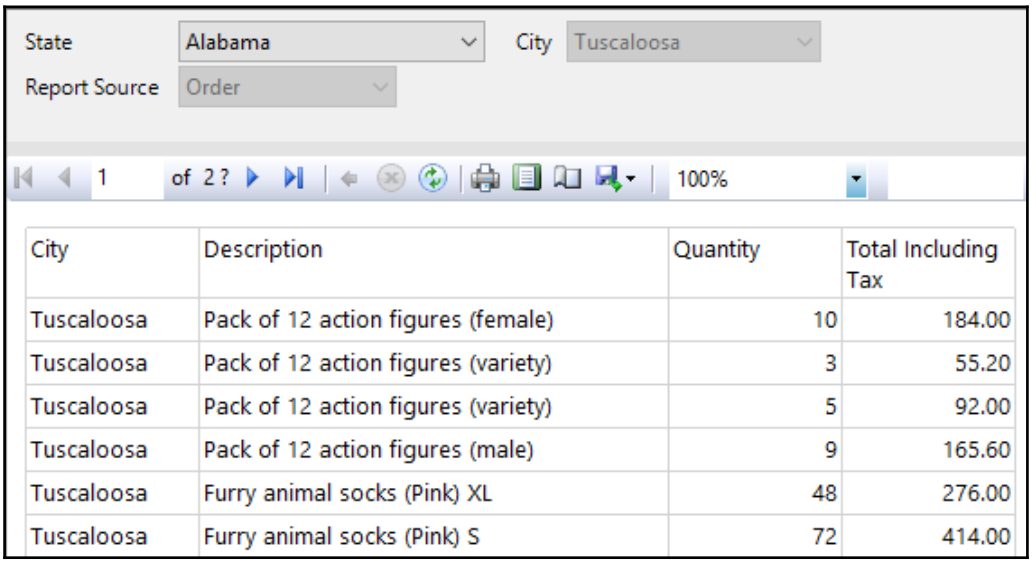
```
SELECT c.[State Province]
     , c.[City]
     , f.[City Key]
     , f.[Customer Key]
     , f.[Stock Item Key]
     , f.[Salesperson Key]
     , f.[Description]
     , f.[Package]
     , f.[Quantity]
     , f.[Unit Price]
     , f.[Tax Rate]
     , f.[Total Excluding Tax]
     , f.[Tax Amount]
     , f.[Total Including Tax]
  FROM [Fact].[Order] f
  JOIN [Dimension].[City] c
    ON f.[City Key] = c.[City Key]
```

8. Right-click on the new **ReportData** dataset, and select **Dataset Properties** on the menu.
9. Click on the *fx* button to the right of the Query area.
10. In the **Expression** window, enter the following expression:

```
= Switch(Parameters!ReportSource.Value = "Order",",
  "SELECT c.[State Province], c.[City], f.[City Key],
f.[Customer    Key]" &
     ",", f.[Stock Item Key], f.[Salesperson Key],
f. [Description], f.[Package]" &
     ",", f.[Quantity], f.[Unit Price], f.[Tax Rate],
 f.[Total Excluding Tax]" &
     ",", f.[Tax Amount], f.[Total Including Tax]" &
  "  FROM [Fact].[Order] f" &
  "  JOIN [Dimension].[City] c" &
  "    ON f.[City Key] = c.[City Key]" &
  " WHERE c.[City] = "'" & Parameters!City.Value & ' ""' " &
  "   AND c.[State Province] = "'" & Parameters
!State.Value & ' ""' "  , Parameters!
ReportSource.Value = "Sale",",
  "SELECT c.[State Province], c.[City], f.[City Key],
f.[Customer Key]" &
     ",", f.[Stock Item Key], f.[Salesperson Key],
 f.[Description], f.[Package]" &
     ",", f.[Quantity], f.[Unit Price], f.[Tax Rate],
f.[Total Excluding Tax]" &
     ",", f.[Tax Amount], f.[Total Including Tax]" &
  "  FROM [Fact].[Sale] f" &
  "  JOIN [Dimension].[City] c" &
  "    ON f.[City Key] = c.[City Key]" &
  " WHERE c.[City] = "'" & Parameters!City.Value & ' ""' " &
  "   AND c.[State Province] = "'" & Parameters!State.
Value & ' ""' "
  )
```

11. Click on **OK** to close the window.
12. Drag and drop a Table control from the Toolbox onto the report.
13. In the properties pane for the table, scroll down to the **General** area. Click on the **DataSetName** property, and in the dropdown, pick **ReportData**.
14. In the table, select several fields to display. In this example, we just used four: `City`, `Description`, `Quantity`, and `Total_Including_Tax`.

15. Click on the **Preview** tab, and then select a **State** such as the great state of **Alabama**. When the city populates, select a city such as **Tuscaloosa**. Select a report source of **Order**, and then click on **View Report**:



City	Description	Quantity	Total Including Tax
Tuscaloosa	Pack of 12 action figures (female)	10	184.00
Tuscaloosa	Pack of 12 action figures (variety)	3	55.20
Tuscaloosa	Pack of 12 action figures (variety)	5	92.00
Tuscaloosa	Pack of 12 action figures (male)	9	165.60
Tuscaloosa	Furry animal socks (Pink) XL	48	276.00
Tuscaloosa	Furry animal socks (Pink) S	72	414.00

Figure 3.27

16. Now change the report source parameter to **Sale**, and again click on **ViewReport**. You should now see a new report with different values.

How it works…

The Order and Sale tables in the Wide World Importers data warehouse have similar fields. It is easy to imagine then that a user might want to run a report first with data from the order table and again from the sale table. Using a dynamic data source means the user doesn't have to jump between various reports. It is also less work for the report developer, as they can generate a single report rather than two reports that would be identical, except for the table the data comes from.

We started by adding a third parameter, Report Source. In it we hardcoded the options for the user to select as the data source for the report. While we only included two choices here, with this report design, we could have added more.

Next came the creation of the dataset. You may have wondered why we entered a SQL statement, closed the dataset properties window, and then immediately reopened it.

In the query, we want to use an expression for the query. Unfortunately, SSRS is unable to extract column names from a query. Thus we needed to create a query SSRS could use to read from one of the tables and get the column metadata information such as column data type, size, and so on.

Once we made reporting services populate a list of columns, we could then update the query to use an expression. The expression takes just a bit of explaining, so here it is again for easy reference:

```
= Switch(Parameters!ReportSource.Value = "Order",",
  "SELECT c.[State Province], c.[City], f.[City Key], f.[Customer Key]" &
     ",", f.[Stock Item Key], f.[Salesperson Key], f.[Description],
f.[Package]" &
     ",", f.[Quantity], f.[Unit Price], f.[Tax Rate], f.[Total Excluding
Tax]" &
     ",", f.[Tax Amount], f.[Total Including Tax]" &
  "  FROM [Fact].[Order] f" &
  "  JOIN [Dimension].[City] c" &
  "    ON f.[City Key] = c.[City Key]" &
  " WHERE c.[City] = ""'" & Parameters!City.Value & ' ""' " &
  "   AND c.[State Province] = ""'" & Parameters!State.Value & ' ""' "
  , Parameters!ReportSource.Value = "Sale",",
  "SELECT c.[State Province], c.[City], f.[City Key], f.[Customer Key]" &
     ",", f.[Stock Item Key], f.[Salesperson Key], f.[Description],
f.[Package]" &
     ",", f.[Quantity], f.[Unit Price], f.[Tax Rate], f.[Total Excluding
Tax]" &
     ",", f.[Tax Amount], f.[Total Including Tax]" &
  "  FROM [Fact].[Sale] f" &
  "  JOIN [Dimension].[City] c" &
  "    ON f.[City Key] = c.[City Key]" &
  " WHERE c.[City] = ""'" & Parameters!City.Value & ' ""' " &
  "   AND c.[State Province] = ""'" & Parameters!State.Value & ' ""' "
  )
```

The expression begins with the `Switch` statement. `Switch` is a command that lets you choose a list of items and return a value based on the input. The basic syntax of a Switch statement is:

```
Switch(Input1, ReturnValue1,
            Input2, ReturnValue2,
            ...
            InputN, ReturnValueN)
```



Note that the SSRS expression language also has an `IIF` command, which is basically an if-then-else statement. Since we only had two options, we could have used that. However, it is best to leave your reports flexible. Using a switch allows us to add more report sources in the future with ease.

As the first input to the Switch, we used the `Parameters!ReportSource.Value = "Order"."` statement. `Parameters` indicates where we want to pull this value from (obviously from our list of parameters). Next is the parameter name. The value is a property of the parameter indicating what the user selected. We then compare it to the hardcoded value of `Order`. Note that hardcoded (also called static) strings in an expression are surrounded by double quotation marks.

We then begin to build our select statement by concatenating strings using the ampersand (`&`). Inside the `Where` clause, we reference two other parameters, city and state, to retrieve their values and place them in the string.

The process is then repeated for the next item in the `switch`, `Sale`.



Be careful when concatenating your SQL text to leave spaces in the appropriate places. It's far too easy to enter:
`"FROM Fact.Sale f & "" & "JOIN Dimension.City c"`
Which would evaluate to:
`FROM Fact.Sale JOIN Dimension.City`
an obviously invalid SQL command!

One final note: in previous recipes, where we used a parameter within a dataset, we had to go to the parameters page of the dataset properties and map the parameters. That is unnecessary here, as we directly reference the parameter from within the expression.

How to publish reports

In this recipe, we will provide a brief overview of how to publish reports to the new SSRS 2016 Report Portal. Formerly called Report Manager in previous versions of SQL Server, the Report Portal has been completely redesigned.

Getting ready

For this recipe, we won't be creating new reports; instead we will be using the reports we've created in this chapter.

You will need to know the two URLs for your report portal. Yes, two. One is the user interface with which users can browse your reports. The other is the backend you will send the reports to when you publish them. The tool to find both of these is the **Reporting Services Configuration Manager**.

Go to the **Start** menu, locate Microsoft SQL Server 2016, and open the **Reporting Services Configuration Manager**. On versions of Windows without a desktop start menu, such as Windows 8 or Windows Server 2012 R2, you can use the search option.

When it opens, you'll need to connect to your server. For SQL Server 2016 Developer Edition, you can use either your computer name or `localhost`. Once open, you'll need two key pieces of information.

Click on the tab marked **Web Service URL**. At the bottom is an item, **Report Server Web Service URLs**. This is the URL you will use in Visual Studio to tell it where to send the reports when you publish them. By default, it is `http://localhost:80/ReportServer`:

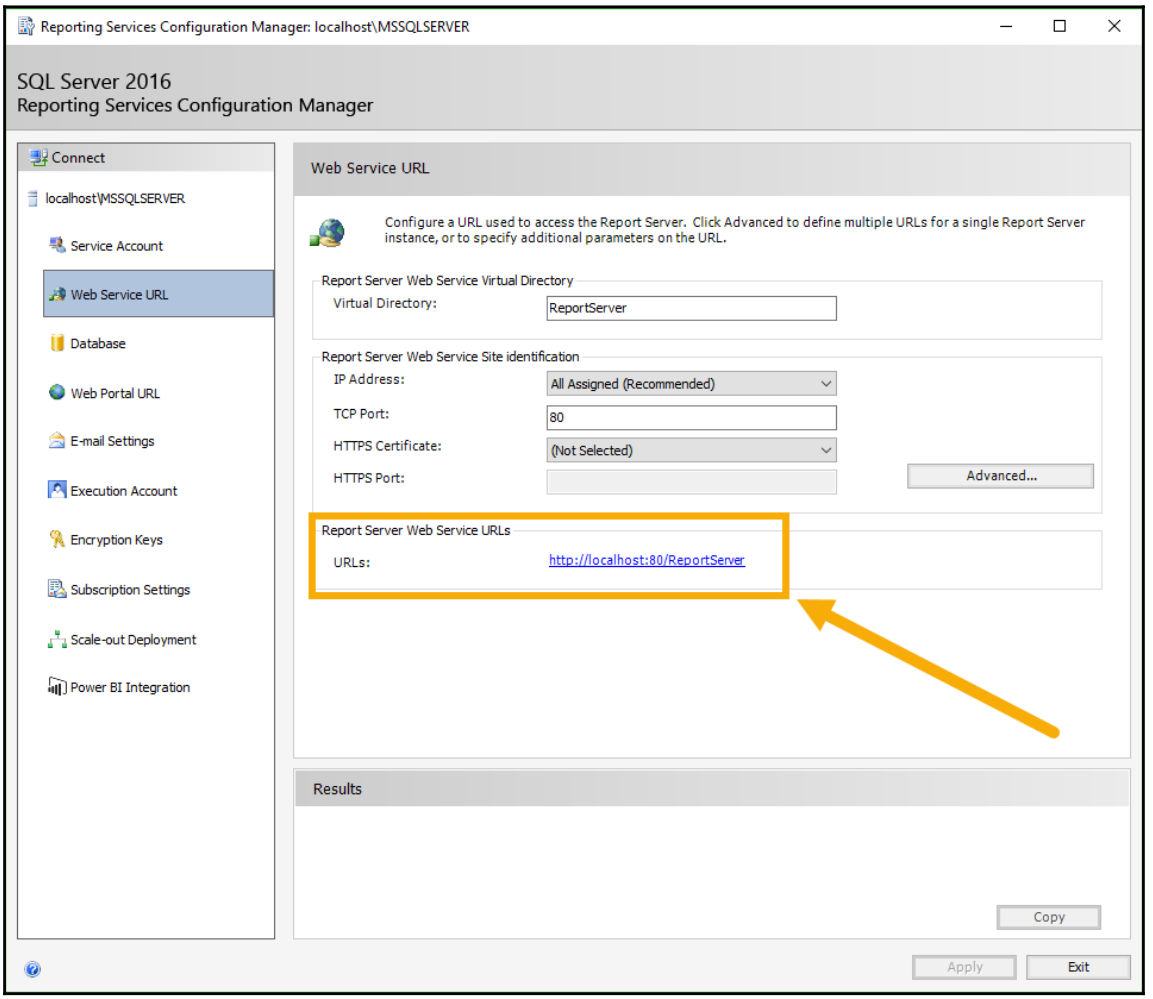


Figure 3.28

Now navigate to the **Web Portal URL** page. This has the web address for the portal that users would normally use to view reports. It is a simple screen; the URL can be found in the center of the screen:

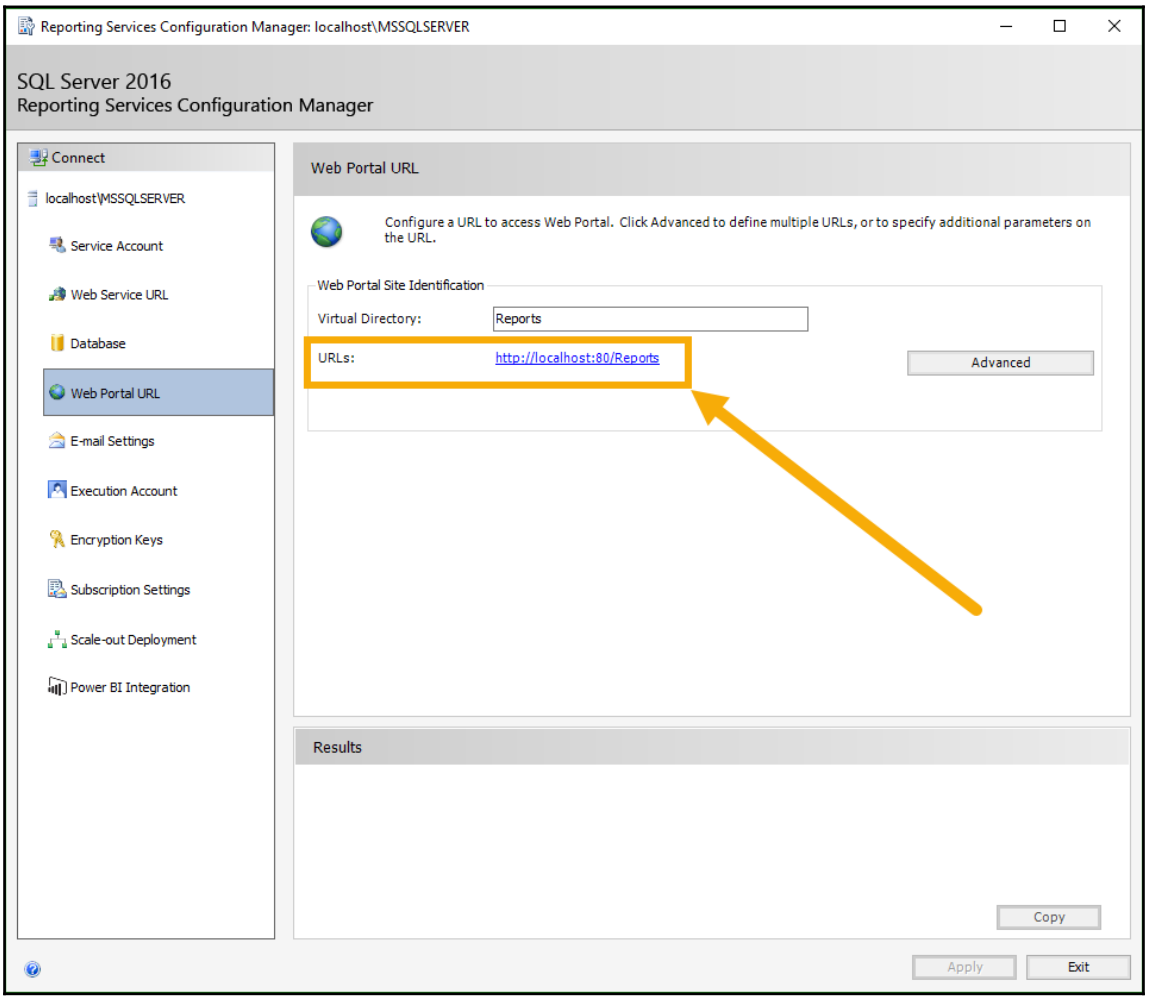


Figure 3.29

Note these two URLs as you'll need them shortly.

How to do it…

1. In **Solution Explorer**, right-click on the `Chapter03` project name (not the solution, but the project name under it).
2. In the menu, select **Properties**.
3. In the properties window, you can take the defaults for the most part. Just ensure that **TargetServerURL** is set to the URL listed under **Web Service URL** in the **Reporting Services Configuration Manager**:

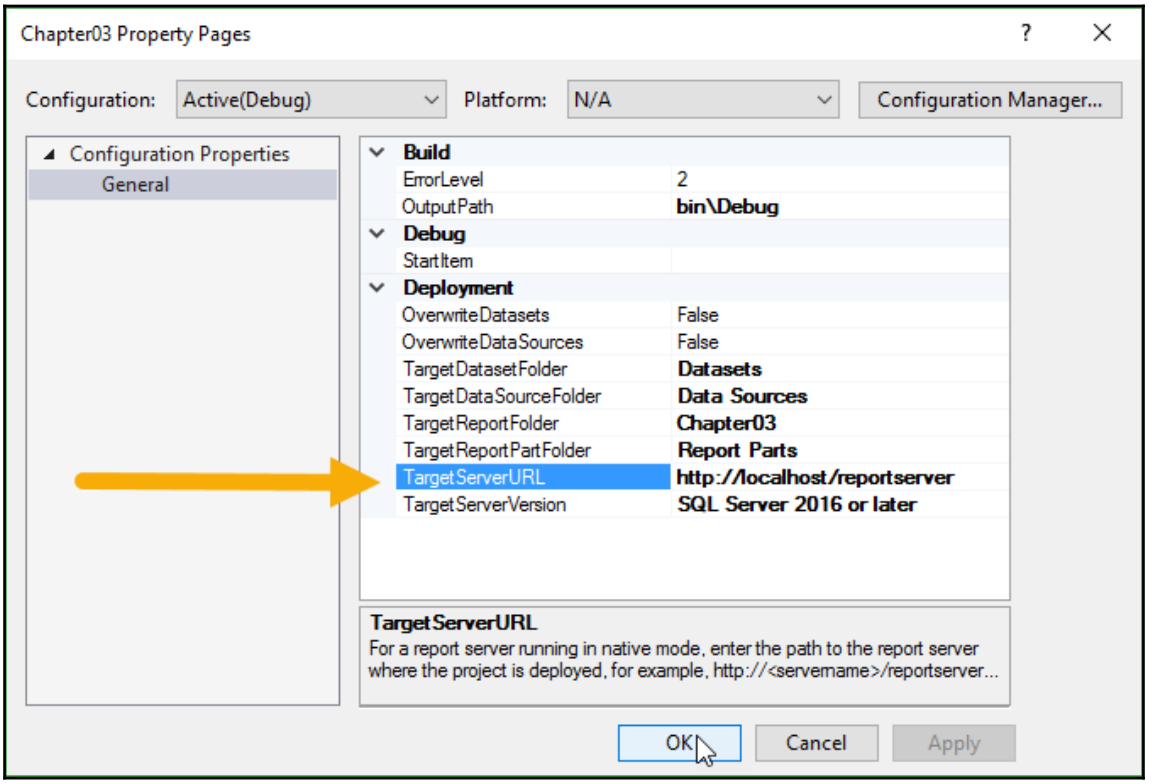


Figure 3.30

4. Click on **OK** to close the **Properties** window.
5. Right-click on the `Chapter03` project again, and pick **Deploy** on the menu.
6. The **Output** window should appear at the bottom of Visual Studio, with the progress of deployment. The following figure shows a sample of the deploy progress:

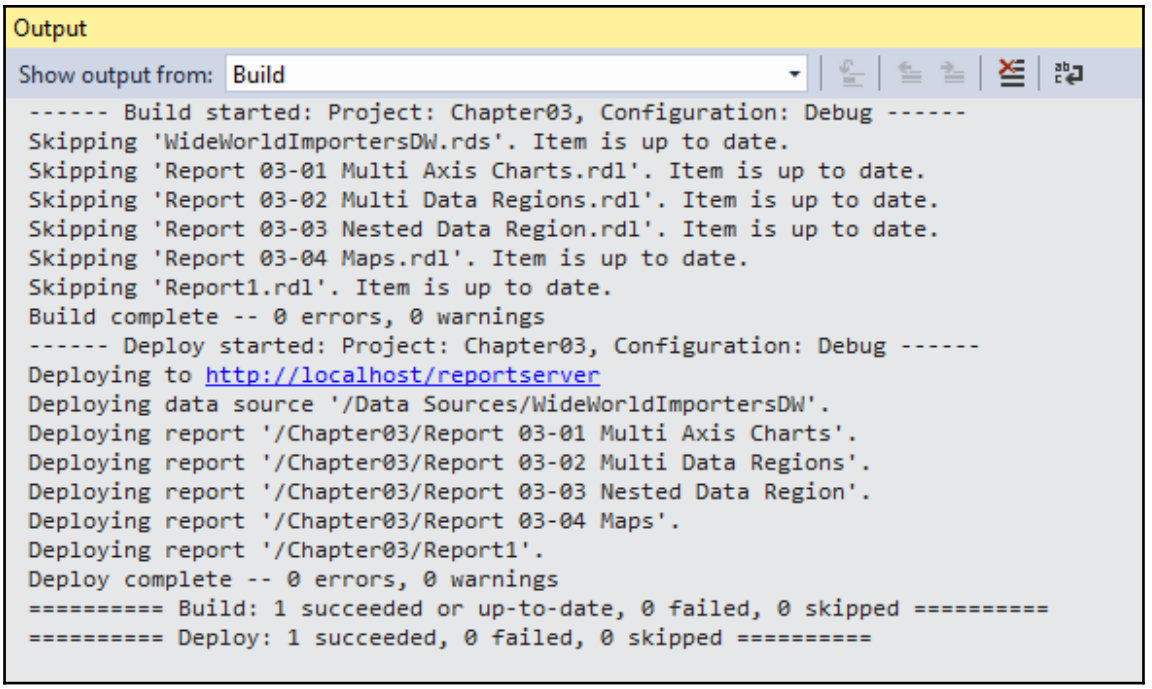


Figure 3.31

7. Open your web browser and navigate to the URL `http://localhost:80/Reports` from the Web Portal URL shown in the **Reporting Services Configuration Manager**:

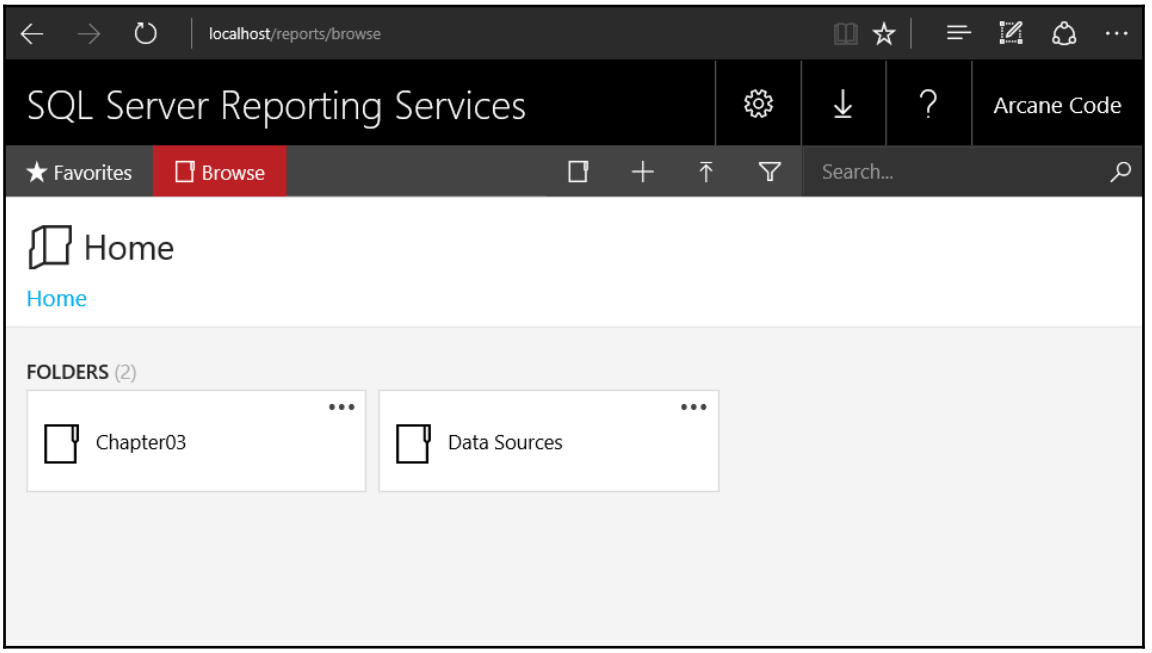


Figure 3.32

8. Here you'll see two folders. Click on the one labeled **Chapter03**. When it opens, you will see each report from our project:

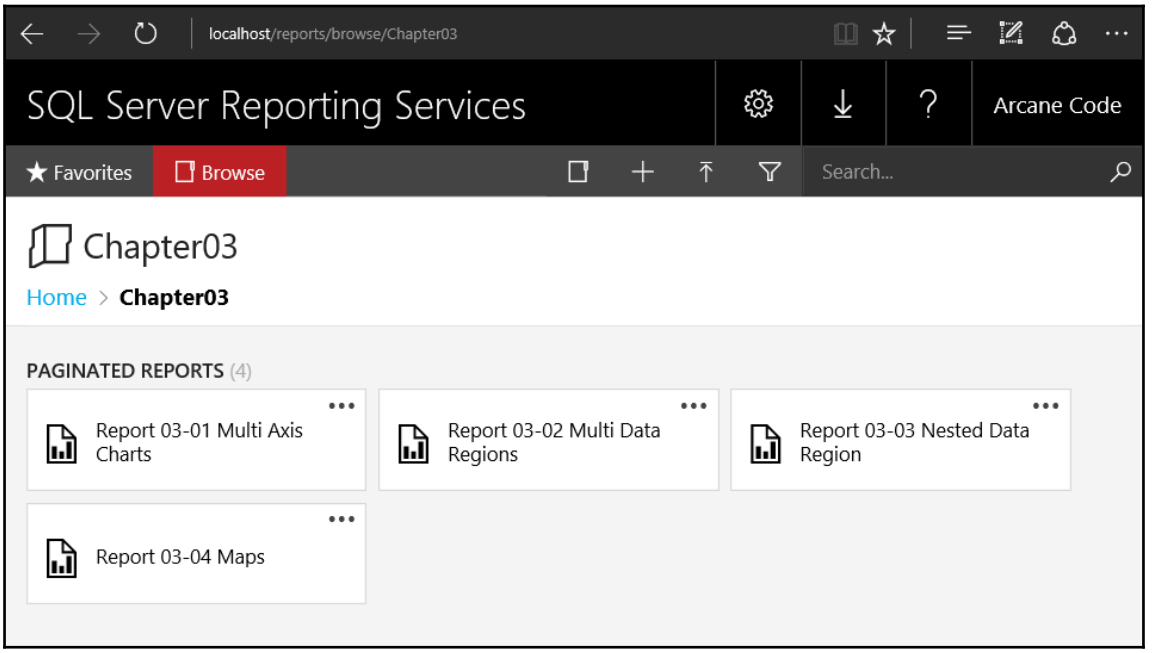


Figure 3.33

9. Click on the first report, **Report 03-01 Multi Axis Charts**. It should now appear in your browser:

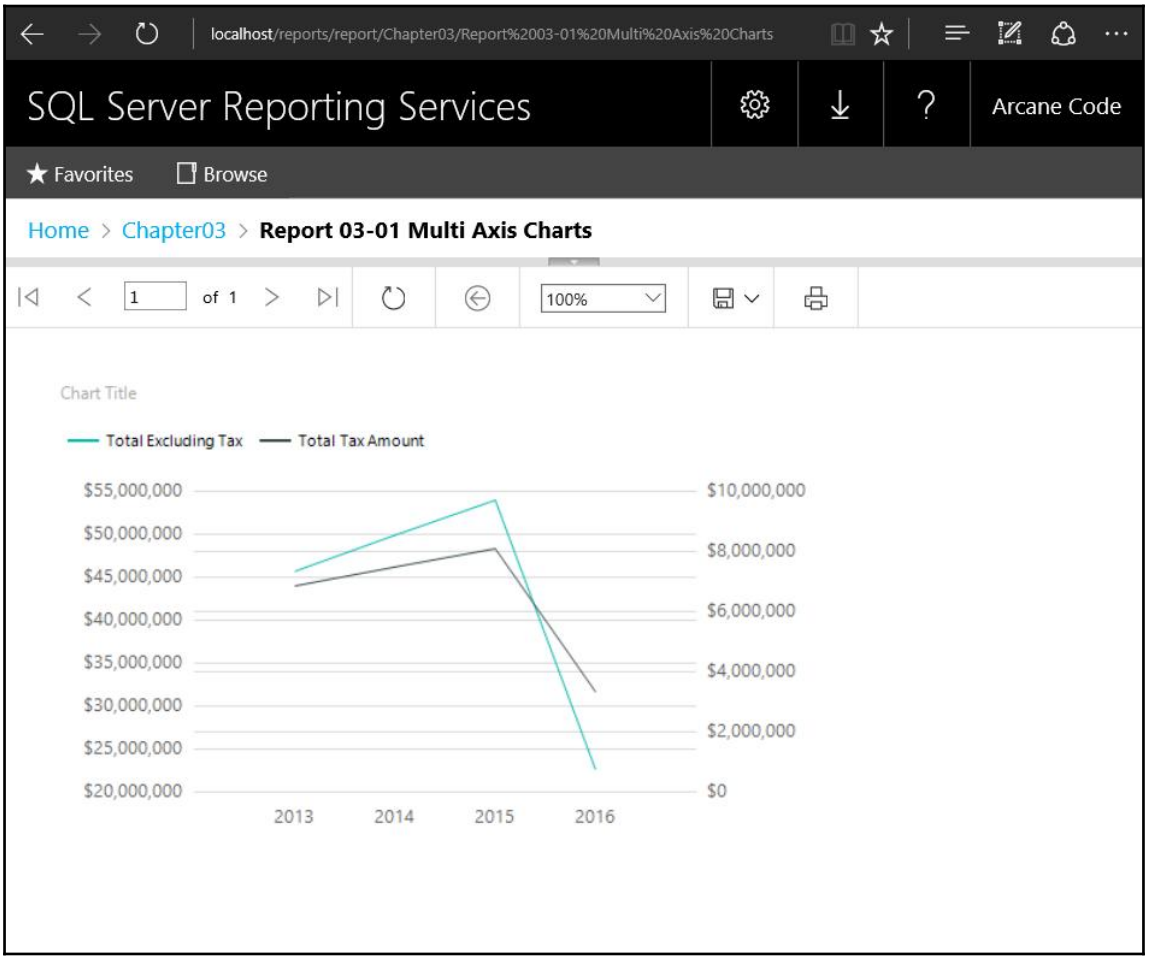


Figure 3.34

How it works…

To deploy a report, you simply need to tell Visual Studio, also known as SQL Server Data Tools, where to send the reports and data sources. That is done in the Web Service URL. If you know how to use the Reporting Services Configuration Manager, as we did in the *Getting ready* section of this recipe, it is pretty easy to locate.

Once you supply the information, it is simply a matter of right-clicking and selecting the **Deploy** menu option. There is one thing to note though about shared data sources (as well as shared datasets, which we haven't covered yet).

If you refer back to Figure 3.30, you'll note the first two lines under the Deployment section: **OverwriteDatasets** and **OverwriteDataSources**. Both are set to **False** by default.

This means once our shared data source, `WideWorldImportersDW`, is deployed, it won't be overwritten. This is a safety feature in that it allows an administrator to edit the data source on the Report Portal and set its connection information. SSDT then won't be overwriting it.

In a development environment, however, using your local host and SQL Server Developer Edition, you may wish to work with the shared data source by using a variety of credentials for testing purposes. You may find the need to be able to overwrite the data source, and can do so via this setting.

How to view the XML code related to reports

Behind the scenes, the report files (the files with the RDL extension) are really just specialized XML. In this recipe, we'll see how to view them.

Getting ready

In order to make demonstrating the recipe easier, we'll make a minor change to one of the reports. First, open the `Report 03-03 Nested Data Regions` report.

Click on the report title at the top of the report, which bears the same name as the report.

Go to the **Properties** window, scroll down, and find the **Name** property. Change it to **ReportTitle**:

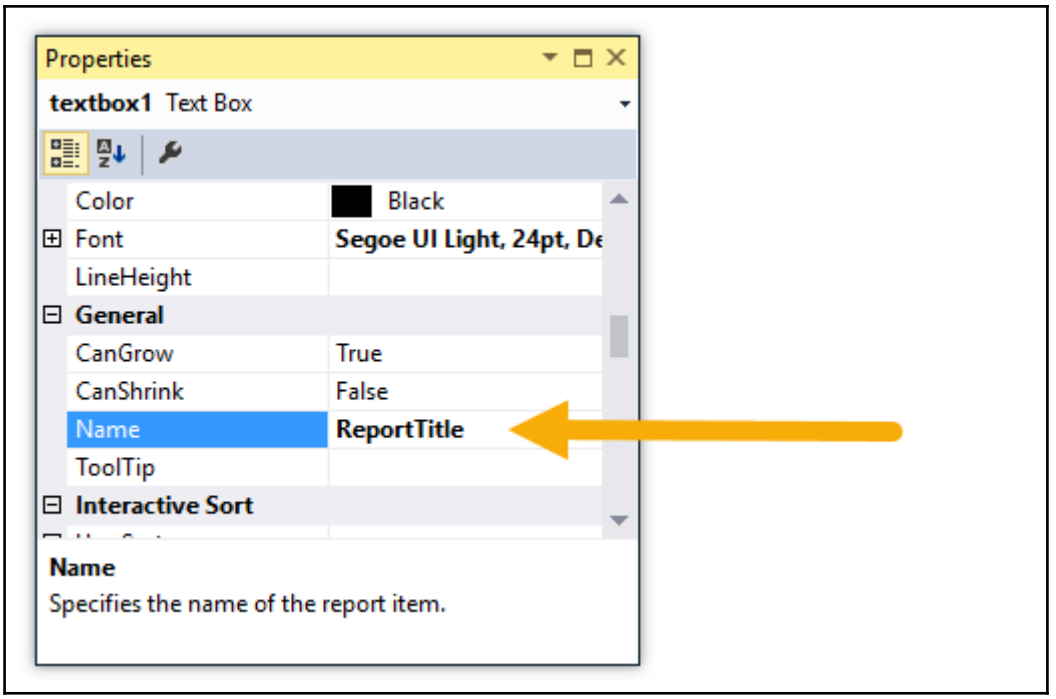


Figure 3.35

Save the report and close it.

How to do it…

1. Right-click on the `Report 03-03 Nested Data Regions` file.
2. In the menu, select **View Code**.

3. You will now see the XML behind the report. At over 650 lines, the listing is too much to reproduce here, but let's look at a small piece. Scroll down to the section that begins with **<ReportSections>** (in the figure below, it is on line 42, but yours may vary):



Figure 3.36

4. Note line 46, `Textbox`, which now has the name `ReportTitle`. Looking further down, at line 53 is `Value`, which has the text that was in the textbox, namely the title of the report.
5. Change the value to `Nested Data Region Report`. The line should now look like the following figure:

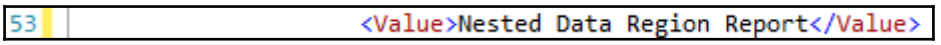


Figure 3.37

6. Save the report and close the XML window.

7. Open the report normally. You should now see that the title has been updated:

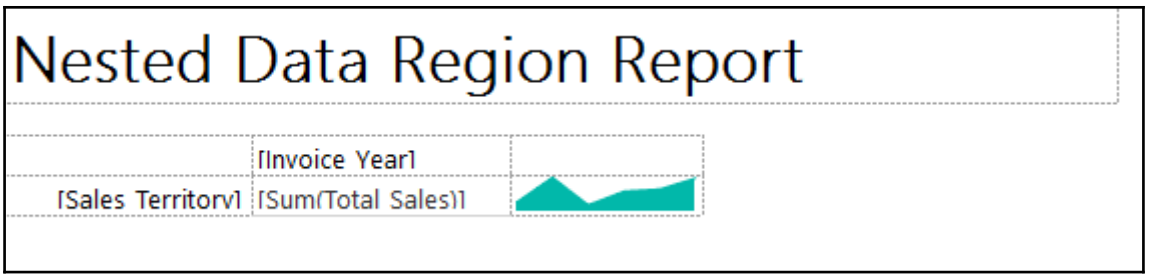


Figure 3.38

How it works…

Behind the scenes, Microsoft stores the report in basic XML. As this is just text, you can open the report and edit it with any text editor. Knowing how to do so can allow you to make quick changes, or even use a code generator to create reports.

If you plan to edit the XML, using good names for your report tool properties can be a good practice. It makes it easy to find and modify objects within your code.

Be aware that this can also be a dangerous proposition. It is very easy to make a mistake, and then your report will no longer load correctly or be understood by SSDT. In addition, direct editing of XML is not supported by Microsoft, so if you do make an error, you are *on your own*.



Always back up your RDL file prior to editing the XML! That way, should you make a mistake, you can recover your original file.

4

Authoring Reports with Report Builder

We will continue our journey into report authoring by moving into advanced topics. We will cover the following recipes:

- Is this for a developer or a business user?
- Enhancements to Report Builder
- Connecting Report Builder to Reporting Services
- Designing reports with the wizard
- Designing reports manually
- Working with published data sources
- Working with published datasets
- Publishing reports to Reporting Services
- Working with published report parts
- Changing reports published in Reporting Services

Introduction

So far, the book has focused on using SQL Server Data Tools as the main method for creating reports. SQL Server Reporting Services actually contains many different tools for generating reports. In `Chapter 6`, *Authoring Reports with the Mobile Report Publisher*, we'll see how to use the new mobile reporting tools to create reports. `Chapter 8`, *Reporting Solutions for BI – Integration*, will include a section integrating Power BI reports into SSRS.

In this chapter, we'll look at *Report Builder*. Report Builder is a simplified interface for generating SSRS reports geared toward the business user. While it has been around for several versions of SSRS, it has received some upgrades and enhancements for SQL Server 2016.

Is this for a developer or business user?

Microsoft originally designed Report Builder as a tool for advanced business users to create their reports. As you'll see, its interface resembles that of a Microsoft Office application, with the tab controls and side panels.

Many developers though, appreciate its simplicity and ease of use. Additionally, the wizards in SSDT are limited compared to those included with Report Builder. Best of all, both SSDT and Report Builder use the same file format on the backend-RDL files. Thus, the same report can be edited in either tool without issues.

By the end of this chapter, you'll realize that the true answer to the question, *Is this for a developer or business users?* is Both!

Getting ready…

In order to use Report Builder, you'll need to install it. Fortunately, this is made easy via the Report Portal. As a starting place, open up your Report Portal. If you need a refresher on where to find it, refer back to the recipe in `Chapter 3`, *Advanced Report Authoring with SQL Server Data Tools*.

How to do it…

1. In the tool bar, there is an arrow pointing downwards. Click on it to open the menu.

2. Select **Report Builder** from the menu as shown in Figure 4.01:

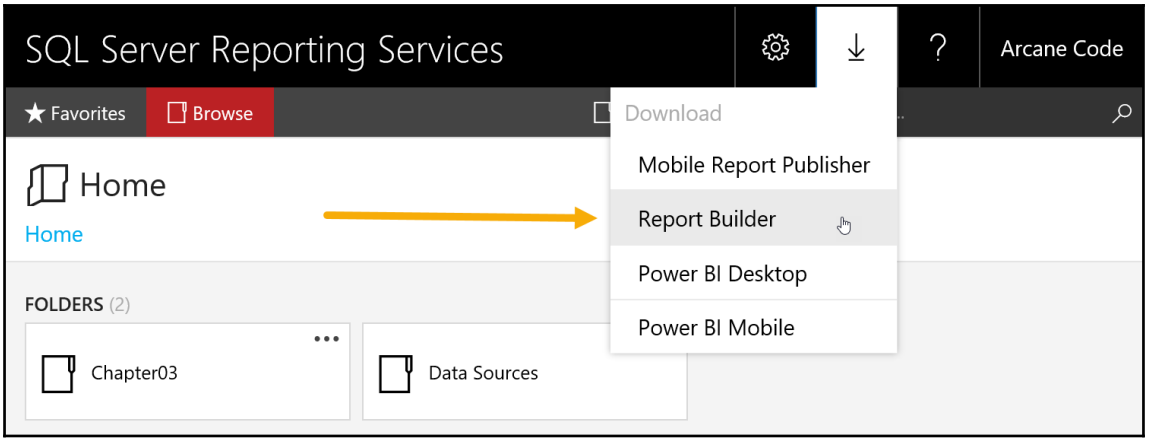


Figure 4.01

3. This will open up a Microsoft website in your default browser where you can download and install Report Builder. Simply follow the instructions in the installation wizard to get Report Builder installed on your system.

How it works…

In previous versions of SQL Server, the Report Builder install was bundled with SQL Server itself, in addition to being a standalone download from the Microsoft website. With SQL Server 2016, Microsoft has no longer included Report Builder with the SQL Server installation, making Report Builder available only as a separate install from the Web. This allows them to make regular updates to Report Builder without being tied to version releases of SQL Server.

During the installation of Report Builder, there is one question that might be considered tricky. It asks for the location of your reporting service. This is not the URL to the report portal, which typically has a format of `http://localhost/reports`. Instead, it is to the services, which takes the format of `http://localhost/reportserver` by default.

Enhancements to Report Builder

There were some enhancements to Report Builder with the 2016 release. In this section, we'll look at two of them and then discuss some behind-the-scenes improvements.

Getting ready…

To begin this exercise, simply open Report Builder. In Windows 10, navigate to **Start** | **All apps** | **Microsoft SQL Server 2016 Report Builder** | **Report Builder**. In other versions of Windows, simply use the search feature to locate **Report Builder**.

How to do it…

1. When Report Builder opens, select **Blank Report** on the **Getting Started** dialog.
2. Note the clean, office-style tab bar across the top. Click on **Insert** to see the many toolbox items available to insert into the report, as shown in the following figure:

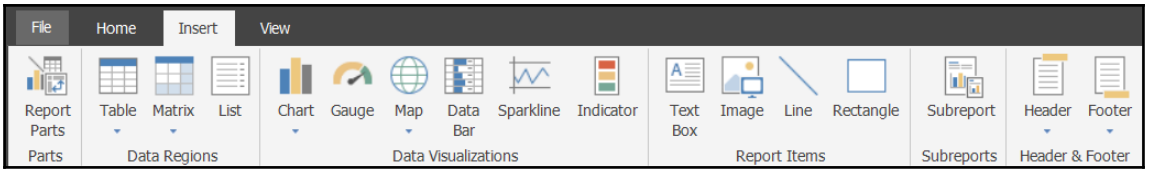


Figure 4.02

3. Next we'll see the addition of PowerPoint to the list of export formats. Click on the **Home** tab, and then click on the **Run** button.
4. On the Run tab, click on the **Export** drop-down.

5. Note the addition of **PowerPoint** to the list of valid formats:

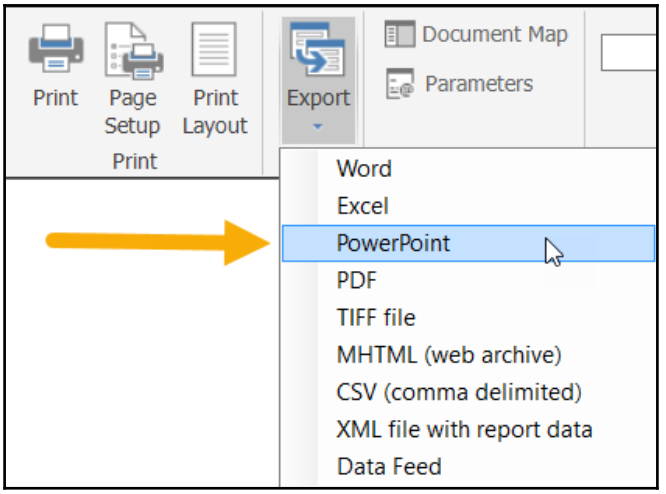


Figure 4.03

How it works…

The updating of Report Builder's look and feel to match the most recent versions of Microsoft Office was a nice but mostly cosmetic update. The widespread use of PowerPoint as a presentation tool makes the exporting of SSRS directly to PowerPoint a welcome addition.

Behind the scenes were a few other updates. Naturally Report Builder supports the new SSRS features such as customizable parameter positioning and HTML 5 rendering. Finally, it now supports the High DPI scaling that is available in Windows 8.1 and Windows 10.

Connecting Report Builder to Reporting Services

During the installation of Report Builder, you were asked the address of your SSRS Web Service URL. This was an optional step, and you might have skipped over it.

If you plan to run all of your reports in Report Builder, and never deploy them, then entering this information is not important. However, it is much more likely users will wish to generate reports, and then share them with coworkers via SSRS Report Portal. Thus, they will need to enter this into Report Builder.

In addition, they may need to update this address depending on what server the reports will be deployed to. It is quite common to have multiple SSRS servers set up within a larger enterprise, serving various departments or locations.

Getting ready

Make sure you have the Web Service URL for SSRS (to look up the web service URL, see the recipe in `Chapter 3`, *Advanced Report Authoring with SQL Server Data Tools*). Note that the web service URL is different from the Report Portal URL.

How to do it…

1. Open Report Builder, close the new report dialog by clicking on the *X* in the upper-right corner, or just select **Blank Report**.
2. Click on **File** and then on **Options**.
3. The very first item reads Use this report server or SharePoint site by default. This is where you place the full URL to your web service URL (again, as defined in `Chapter 3`, *Advanced Report Authoring with SQL Server Data Tools*).
4. Click on **OK** to save the new settings.

How it works…

It is possible to create reports and run them on a local workstation entirely within Report Builder and without the aid of a Report Services server. But this has some definite drawbacks.

The primary drawback is the location where the report is executed. When running a report in Report Builder, the report is running on the local workstation. Data is brought across the network, the report is compiled within Report Builder, and then it is displayed. Typically, local workstations have far less resources compared to a server.

When the user launches a report from the Report Portal, that report is generated on the SSRS server. Only the output (typically in the form of a web page) is returned to the user's workstation.

Additionally, shared items such as data sources and datasets stored on the Report Portal are available to Report Builder when it is connected to the SSRS web services. This can be a great time saver and provide more reliability as components that have already been tested are being reused.

For the rest of this chapter, the recipes will assume you have created at least some of the reports from `Chapter 3`, *Advanced Report Authoring with SQL Server Data Tools*, and deployed them as shown in the same chapter. If you've not done so, you should return to create at least one report from that chapter and deploy it.



In the Report Builder Options window was a second page, Resources. Here you'll find links to online SSRS resources as well as have the ability to send feedback to the Report Builder team.

How to design reports with the wizard

In this section, we'll create a report using the wizard built into Report Builder. We will recreate the same report from `Chapter 2`, *Advanced Reports with SQL Server Data Tools*, in order to compare the way a report is created in Report Builder with the way you create reports in SSDT.

Getting ready

To get to the wizard, simply open Report Builder. If you already have Report Builder open, simply select the **File** tab and click on **New**.

How to do it…

1. On the **Getting Started** dialog, click on **Table or Matrix Wizard**:

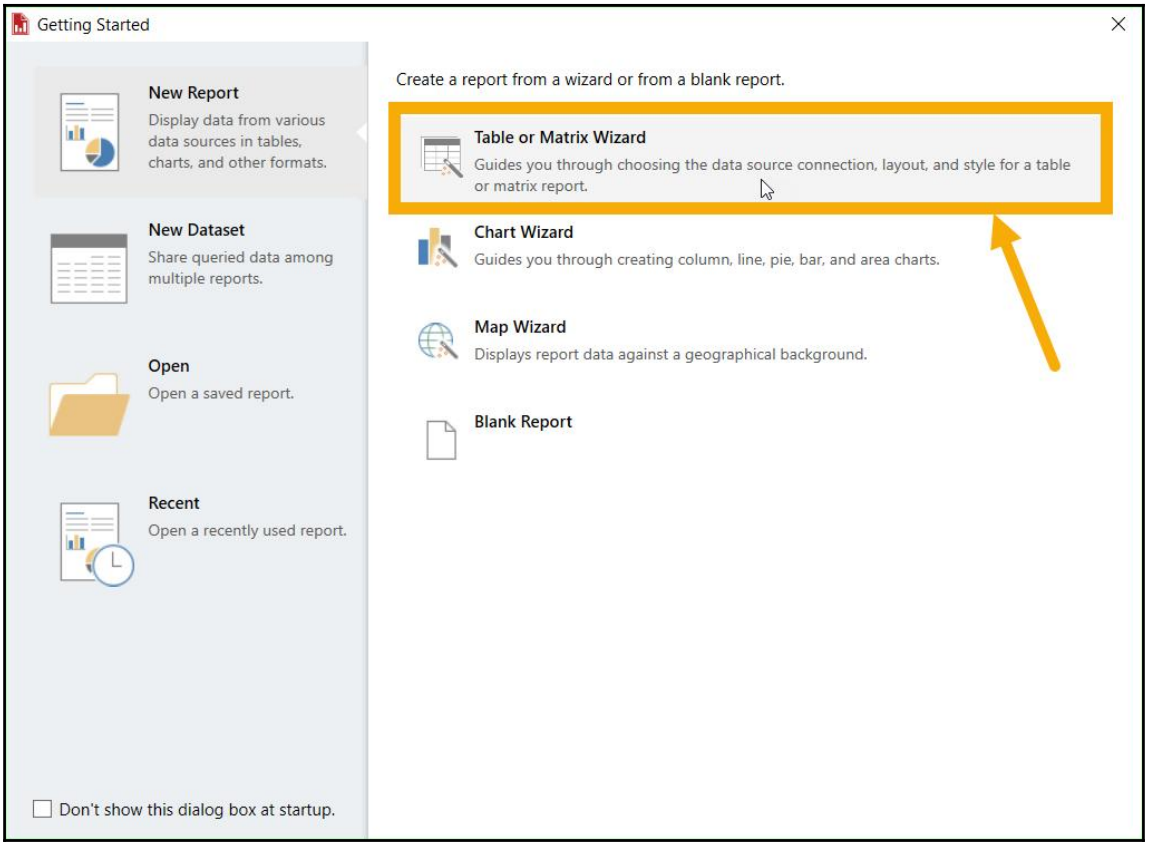


Figure 4.04

2. On the next page of the wizard, ensure that the **Create a dataset** option is selected and then click on **Next**.
3. If you deployed the reports in `Chapter 3`, *Advanced Report Authoring with SQL Server Data Tools*, *Installing Reporting Services Related Tools* and *How to publish reports* recipe, you should see an existing data source connection for **WideWorldImportersDW**. If it is there, simply click on it and pick **Next**. If not, you can use the **New** button to create a new connection to the **WideWorldImportersDW** database. Once it is created, click on **Next**.
4. In the Database view area on the left, navigate to **Dimension** | **Tables** | **Customer**; then place a check beside the **Customer** field.
5. Expand the **Fact** branch, then **Tables**, and finally**Sale**. Place checks beside **Invoice Date Key**, **Description**, and **Total Including Tax**.
6. On the lower right is the **Applied filters** area. Near it is a funnel-like object that represents filters. (See Figure 4.05). Click on it to add a new filter:

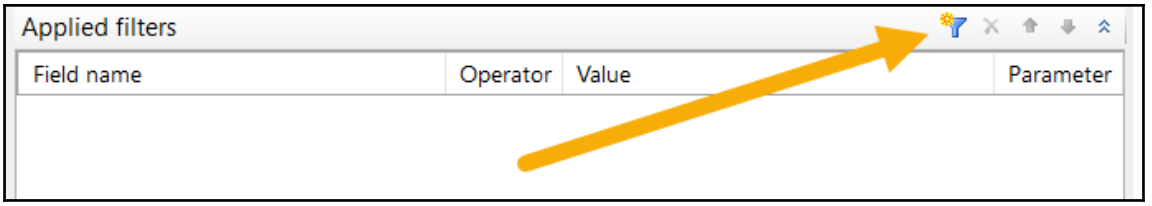


Figure 4.05

7. Click on the **Field name** text box in the newly appearing filter. Navigate to **Fact** | **Sale** | **Invoice Date Key** and click on it.
8. Change the operator to **is more than or equal to**.
9. Click on in the **Value** area. Enter **2016-05-01** and press *Enter*.
10. Click on the filter icon again. In **Field name**, again select **Invoice Date Key**.

11. Change the operator to **is less than or equal to.** In **Value**, enter `2016-05-31`. Your query window should now look like Figure 4.06:

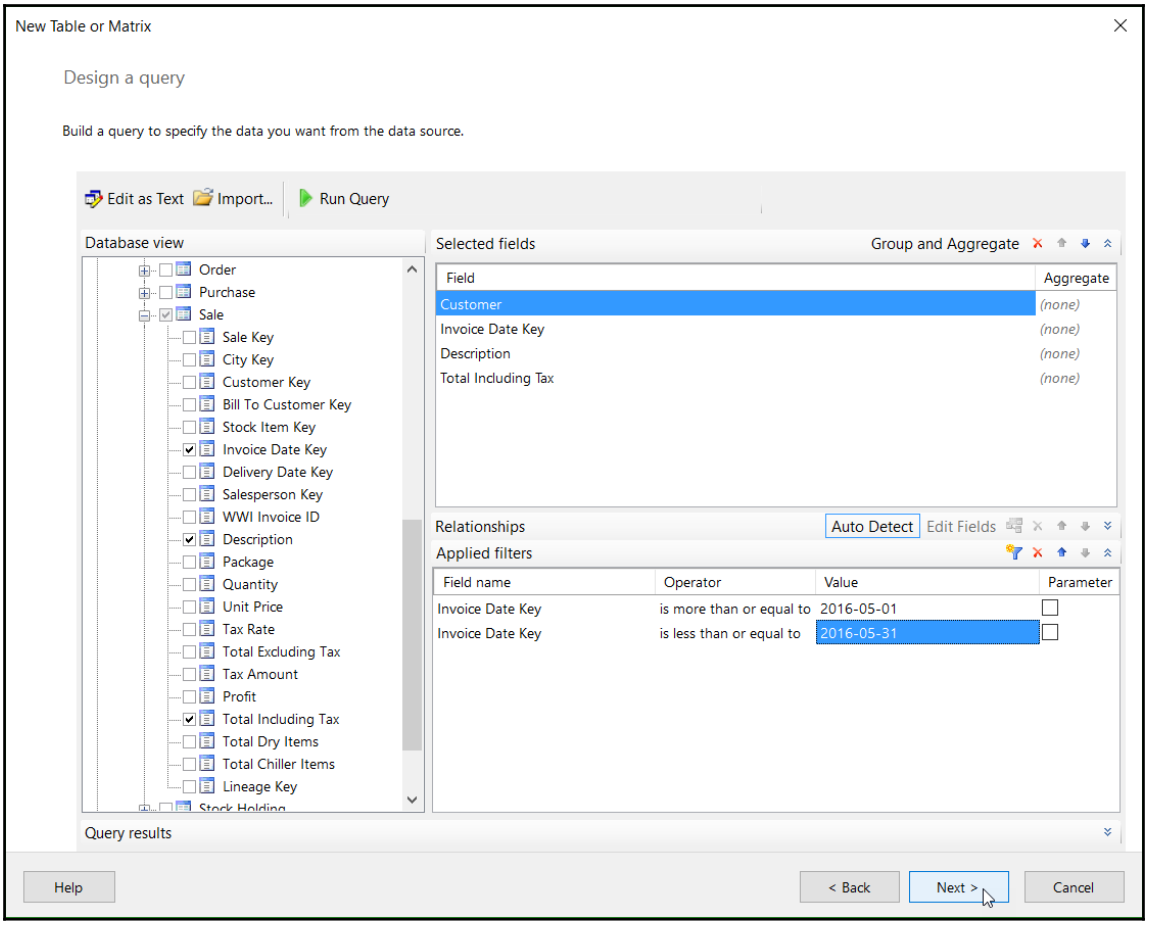


Figure 4.06

12. Click on **Next** to proceed.
13. On the **Arrange fields** screen, drag and drop the **Customer** field into the **Row group** area.

14. Drag the remaining fields into the **Values** area, and then click on **Next**.
15. On the **Choose the layout** screen, uncheck **Expand/collapse groups**, and click on **Next**.
16. The last screen simply shows a preview; just click on **Finish** to generate the report.
17. Goto **File** | **Save** from the menu.
18. In the **Save As Report** window, select **My Computer** on the left and navigate to a folder on your local drive where you wish to store your report.
19. Save your report as `Report 04-01.rdl`.
20. On the **Home** tab, click on **Run** to see the report with data.
21. When done viewing, click on the **Design** button to return to the report designer.

How it works…

The wizard works similarly to the one in SSDT. Filling out the wizard gives Report Builder sufficient information to generate the report.

On the design the query page, we used the wizard to build the SQL needed to run the report. If you refer back to Figure 4.06, you will note the **Edit as Text** button. This will update the query window to let you enter a SQL statement. As a report developer, this might be an option you'd prefer. For the average business user, they'd likely be more comfortable building the query as shown in this recipe.

You may have wondered how Report Builder knew how to link the Customer table to the Sales table. This was due to the existing foreign key relationship between the two tables. Report Builder detected it and knew how to link the two tables.

What if there was no foreign key? In that case, you'd have to create the relationship within the report yourself. In the dialog that builds the query, in the middle, there is an area called **Relationships**. On the very right is what appears to be two arrows pointing down (see the following figure):

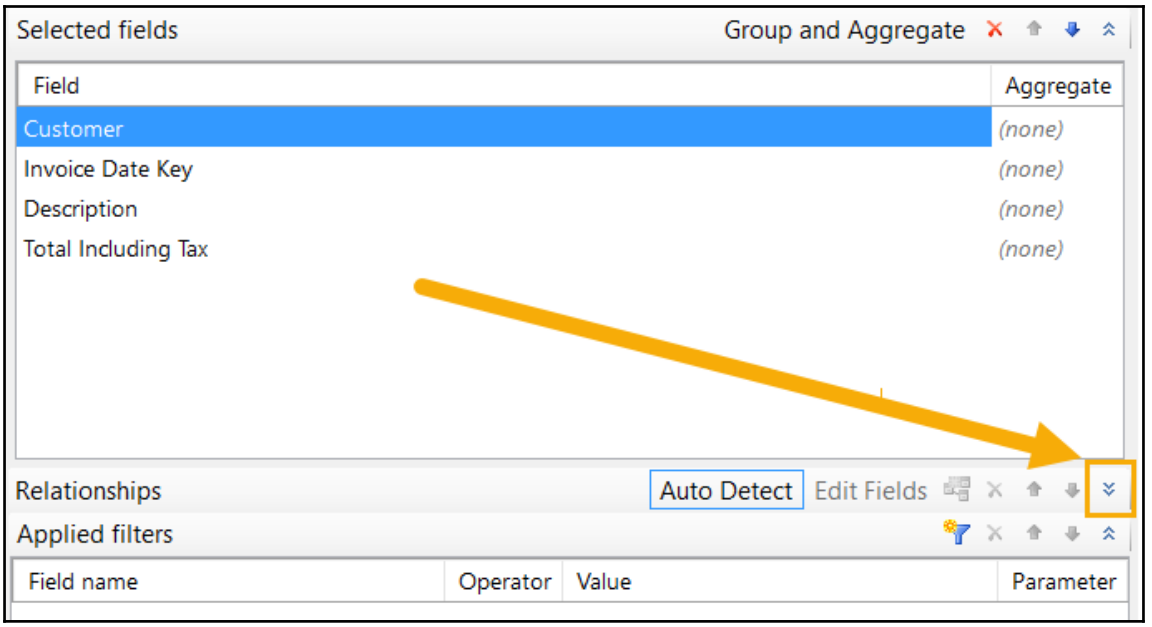


Figure 4.07

Clicking on this expands the relationships area, as seen in Figure 4.08. Once expanded, you can see the relationships auto-detected by Report Builder:

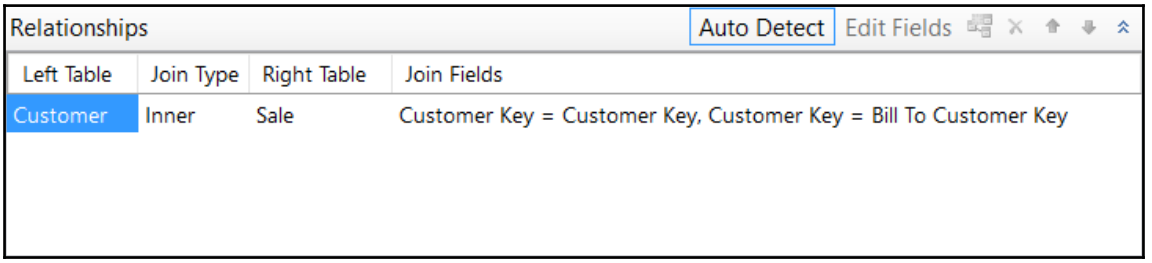


Figure 4.08

To manually edit these, click on the **Auto Detect** button, which will deselect it and enable the **Edit Fields** button. From here, you can add, alter, and remove relationships as needed in order for your report to work.

In the Arrange fields page, you organized the fields into the layout desired for the report. The Column group was ignored, which made the generated report a table. If any fields had been placed in the column area, the Report Builder would have placed a matrix on the report instead of a table.

There's more…

From here you can customize the report just as you would in SSDT. Right-clicking on the date, for example, would allow you to access the same pop-up menu as in the SSDT designer. You can then access the text box properties and format the date.

How to design reports manually

The wizard in Report Builder is much more powerful than the one included in SSDT. For example, it can generate charts, something the SSDT wizard cannot do.

It can be useful though to know how to generate a report within Report Builder manually. This will give you exposure to some of the behind-the-scenes functionality of Report Builder, which you'll find isn't much different from SSDT.

Getting ready…

Open Report Builder, if it is not open, to bring up the **Getting Started** window, or go to **File | New** in order to bring up the New Report or Dataset window. Both windows will achieve the same goal letting us start a new report.

How to do it…

1. Select **Blank Report**.
2. In the Report Data tool area on the left, right-click on Data Sources and select **Add Data Source…** from the menu.
3. Rename the data source from the default of `DataSource1` to `WideWorldImportersDW`.
4. Select the shared data source of **WideWorldImportersDW**, and click on **OK**.
5. Right-click on **Datasets** and select **Add Dataset…**.
6. Change the name of the dataset to **TotalSalesByYear**.
7. Select the **Use a dataset embedded in my report** option.
8. Select **WideWorldImportersDW** as the data source.
9. Enter this query in the query window:

```
SELECT YEAR(s.[Invoice Date Key]) AS InvoiceYear
     , SUM(s.[Total Including Tax]) AS TotalSalesWithTax
  FROM [Fact].[Sale] s
 GROUP BY YEAR(s.[Invoice Date Key])
 ORDER BY YEAR(s.[Invoice Date Key])
```

10. Click on **OK** to close the dataset properties window.
11. Click on the **Insert** tab in the toolbar at the top of Report Builder.
12. Click on **Chart** and then on **Chart Wizard**.
13. Select the **TotalSalesByYear** dataset, and then click on **Next**.
14. For the chart type, select **Pie chart** and click on **Next**.

15. On the **arrange chart fields** page, drag and drop **InvoiceYear** into the **Categories** area and **TotalSalesWithTax** into the **Values** area. Click on **Next**:

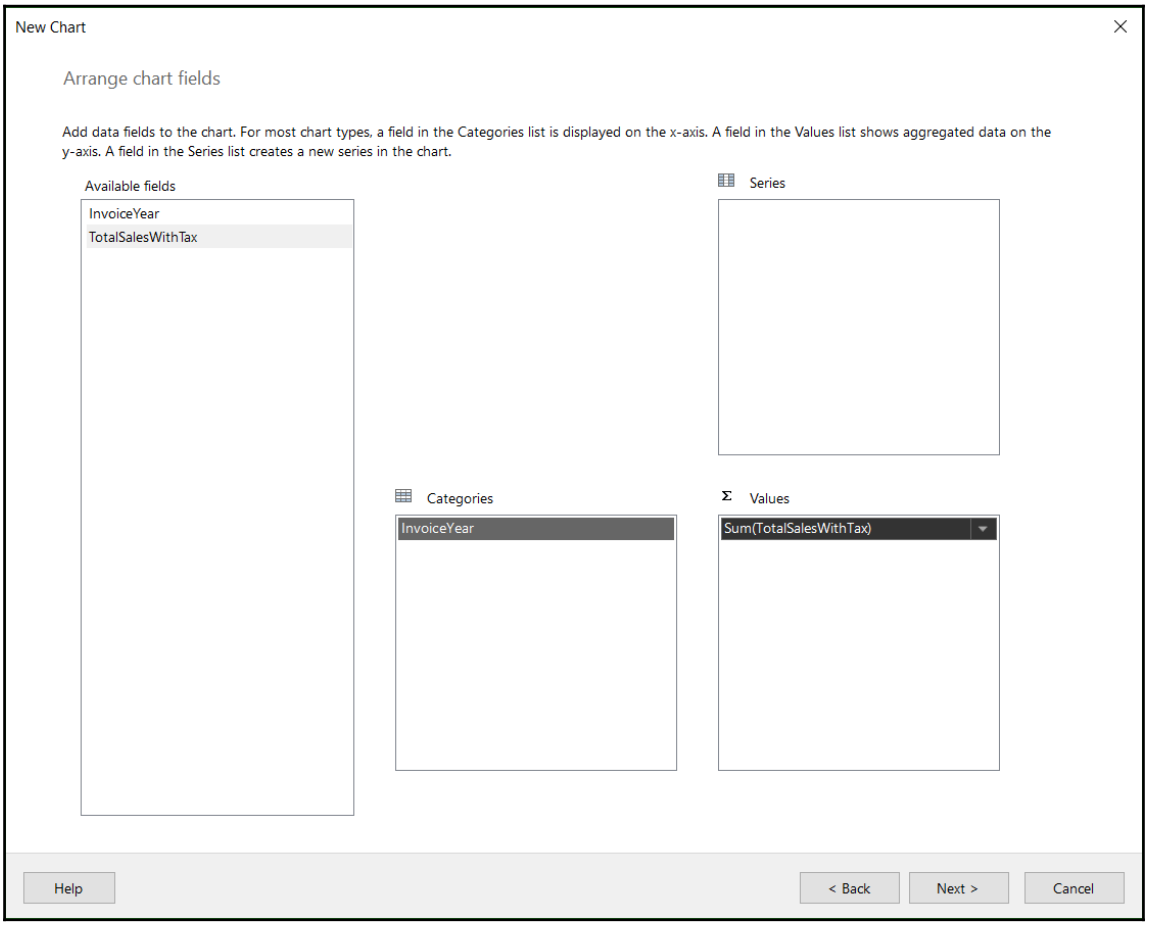


Figure 4.09

16. Finally, you are shown a preview. Just click on **Finish** to generate the pie chart on the report.
17. Save the report as `Report 04-02.rdl`.
18. Run the report by clicking on **Run** in the **Home** tab.

How it works…

If you have a feeling that this is familiar, it is. Under the hood, SSDT and Report Builder share a lot of similar code. Thus tasks such as adding a dataset, data source, or formatting text boxes are almost identical.

In this recipe we used the chart wizard to demonstrate adding the report. We could have also added the chart manually. In that case, the steps would have been similar to what you read in *Creating reports with a chart data region* recipe in `Chapter 2`, *Advanced Reports with SQL Server Data Tools*.

Working with published data sources

So far, you've already been working with data sources that were published back in `Chapter 3`, *Advanced Report Authoring with SQL Server Data Tools.* What if you want to create a new shared data source?

Unfortunately, there is no direct method to create a shared data source within Report Builder. However, we can create it within the Report Portal, and then use it in Report Builder.

Let's see how to do that with this recipe.

Getting ready

For this example, we're going to use the `WideWorldImporters` database, not the **Data Warehouse** (**DW**) version.

We'll use report builder later in this recipe, so go ahead and open report builder if it is not open to bring up the Getting Started window, or go to **File** | **New** in order to bring up the New Report or Dataset window. Both windows will achieve the same goal, letting us start a new report.

How to do it…

1. Open your web browser, and navigate to your Report Portal.
2. You should see a folder, **Data Sources**. Click on it to open it.
3. In the toolbar of the Report Portal is a plus button (depending on the size of your display, it may also have the word **New** beside it). Clicking on it opens a menu, from which we will select **Data Source**:

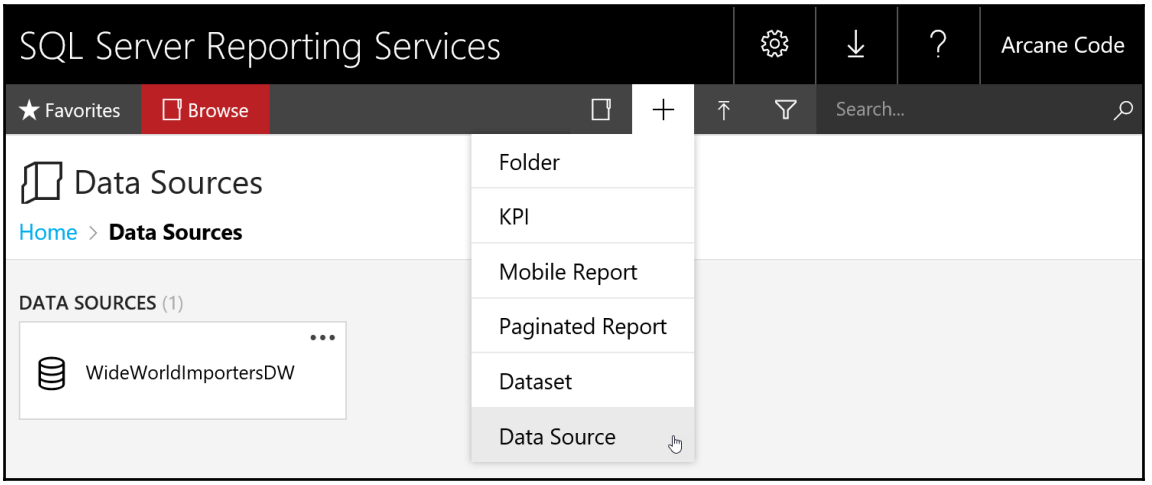


Figure 4.10

4. In the **Properties** area, enter `WideWorldImporters`.
5. For the description, enter `The Wide World Importers transactional database`.
6. Ensure that the connection type is Microsoft SQL Server.
7. In Connectionstring, enter `Data Source=localhost;Initial Catalog=WideWorldImporters`.
8. The information for credentials will depend upon the environment you are in:
    - If you are operating in an enterprise with Active Directory credentials and those credentials have access to the **WideWorldImporters** database, select the **As the user viewing the report** option.
    - If you are running on your local computer and are referencing localhost as the location of Report Portal, select the **Use the following credentials** option. Enter your username in the format *computer-name\login-name* and give it your password.

9. Validate that you have entered everything correctly by clicking on **Test connection**.
10. Assuming the test has passed, click on **Create**.
11. If you've not done so, launch Report Builder (or if already open, go to **File | New**).
12. In the new report area, click on the **Table or Matrix Wizard**.
13. On the **Choose a dataset** page, ensure that **Create a dataset** is selected and click on **Next**.
14. If you've worked with the **WideWorldImporters** connection before, it would have already appeared on the list. You can simply click on it and click on **Next** to proceed, as we've done in the past with **WideWorldImportersDW**. If it does not appear, click on the **Browse** button at the bottom.
15. Double-click on **Data Sources** to open that folder.
16. Click on **WideWorldImporters**; then click on **Open**:

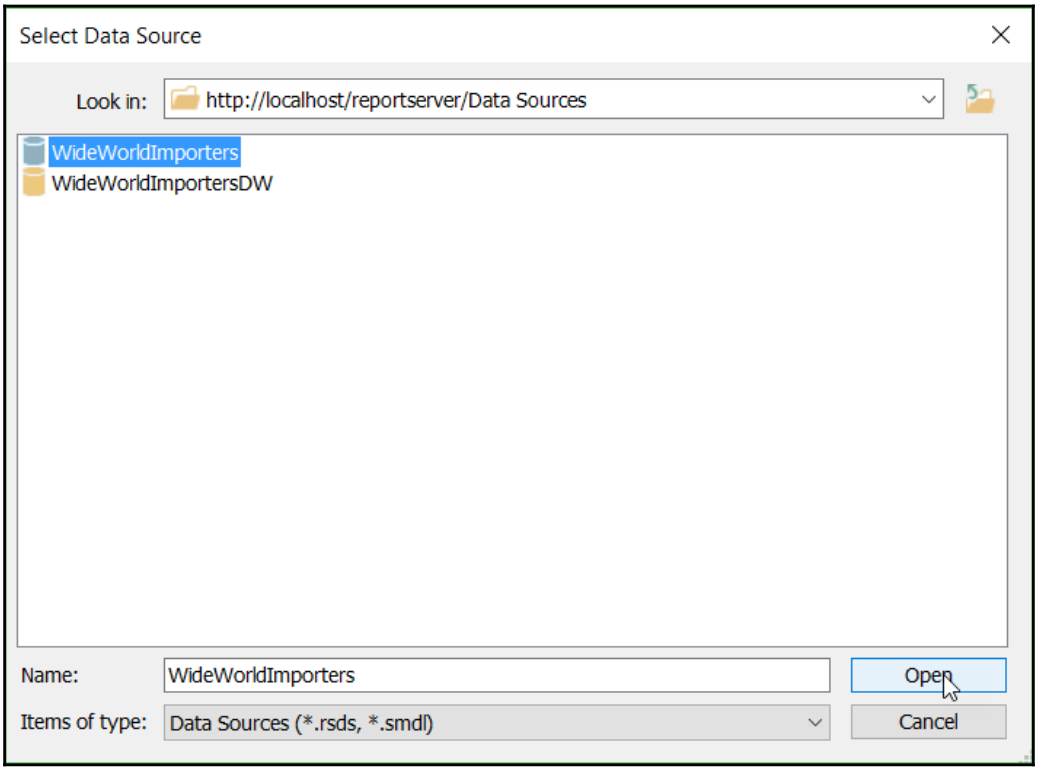


Figure 4.11

17. When **WideWorldImporters** appears on the list, select it and click on **Next**:

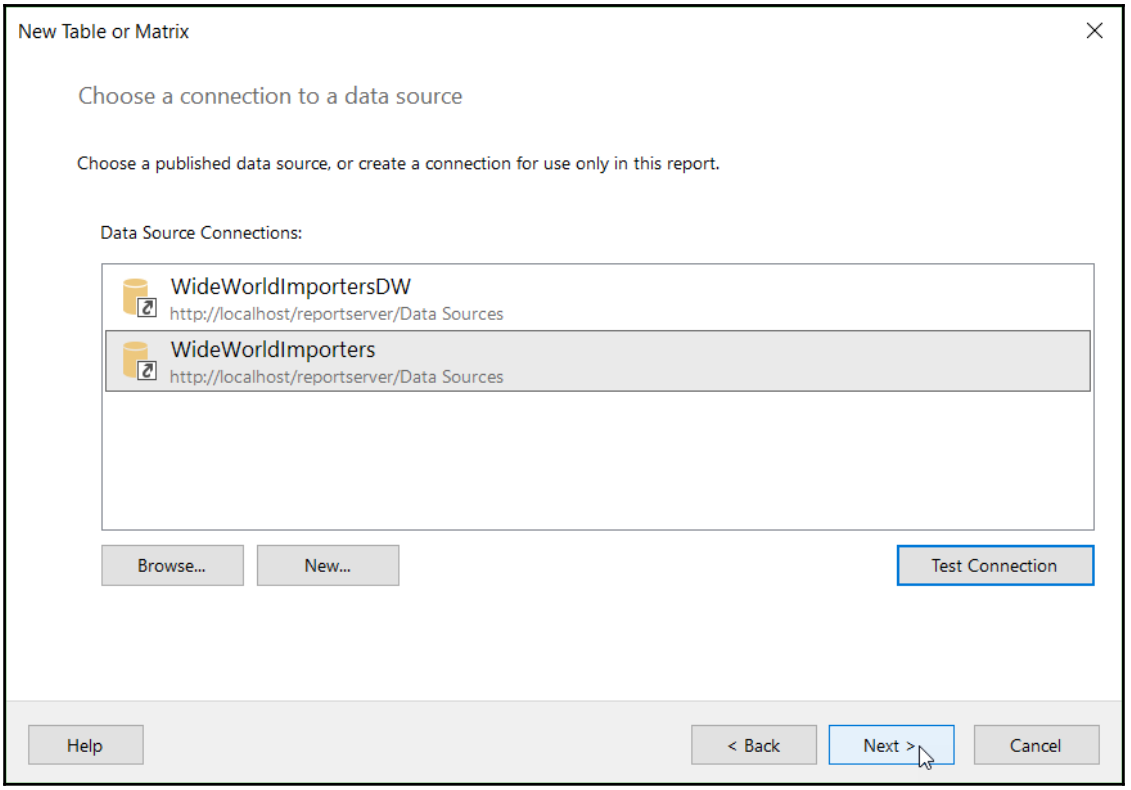


Figure 4.12

18. Since we've already built reports using these data sources, we don't need to go further. This is because the purpose of this recipe was to show how to create a new data source, not generate a report, which by now should be familiar to you. Just click **Cancel** to close the dialog.

How it works…

The Report Portal understands how to store a wide variety of objects, including Data Sources. Data Sources are essentially connection strings in a nice wrapper. By placing these in the Report Portal, you make it easy for business users, who might not be technically proficient, to gain access to a variety of databases from which they can create their reports.

Working with published datasets

In simple terms, a published dataset is nothing more than a SQL statement that has been stored in the Report Portal. This can be extremely valuable as business users often lack the ability to write even simple SQL queries.

In addition, this allows experienced SQL developers to write complex SQL queries and make them available to business users so that they can write reports from them. It also allows queries that are used over and over to be stored, saving users from having to rewrite them every time they wish to write a report.

Getting ready

Unlike published data sources, it is possible to create a published dataset from Report Builder.

Prior to creating the dataset, we will need a place to store it in the Report Portal. Go to the Report Portal, and click on the **+** (**New**) button. In the menu, click on **Folder** and name the new folder `Datasets`.

Now open report builder if it is not open to bring up the **Getting Started** window, or go to **File** | **New** in order to bring up the **New Report** or **Dataset window**.

How to do it…

1. Click on **New Dataset**.
2. Select the **WideWorldImportersDW** connection.
3. Click on **Create**.
4. Then click on **Edit as Text**.
5. Paste in the following query:

```
SELECT c.[Customer]
     , s.[Description]
     , s.[Invoice Date Key]
     , s.[Total Including Tax]
  FROM [Fact].[Sale] s
  JOIN [Dimension].[Customer] c
    ON s.[Customer Key] = c.[Customer Key]
 WHERE s.[Invoice Date Key] >= '2016-05-01'
   AND s.[Invoice Date Key] <= '2016-05-31'
```

6. In order to validate your SQL, click on the exclamation mark to run the query, as shown in figure 4.13:

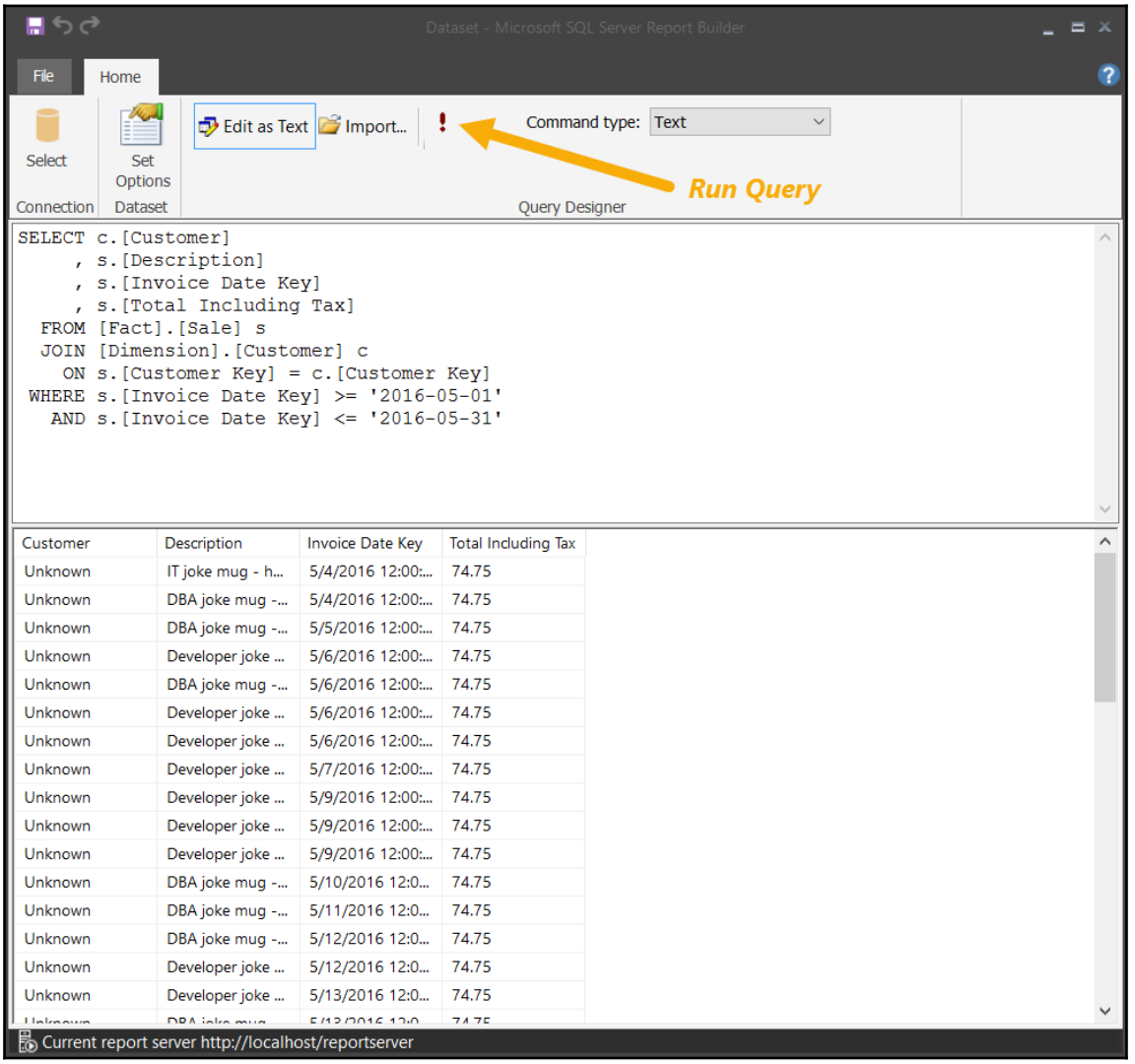


Figure 4.13

7. Click on the **File** menu, and then on **Save**.
8. Double-click on the **Datasets** folder you created earlier.
9. Name the dataset `SalesByDate.rsd` and click on **OK**, as shown here:

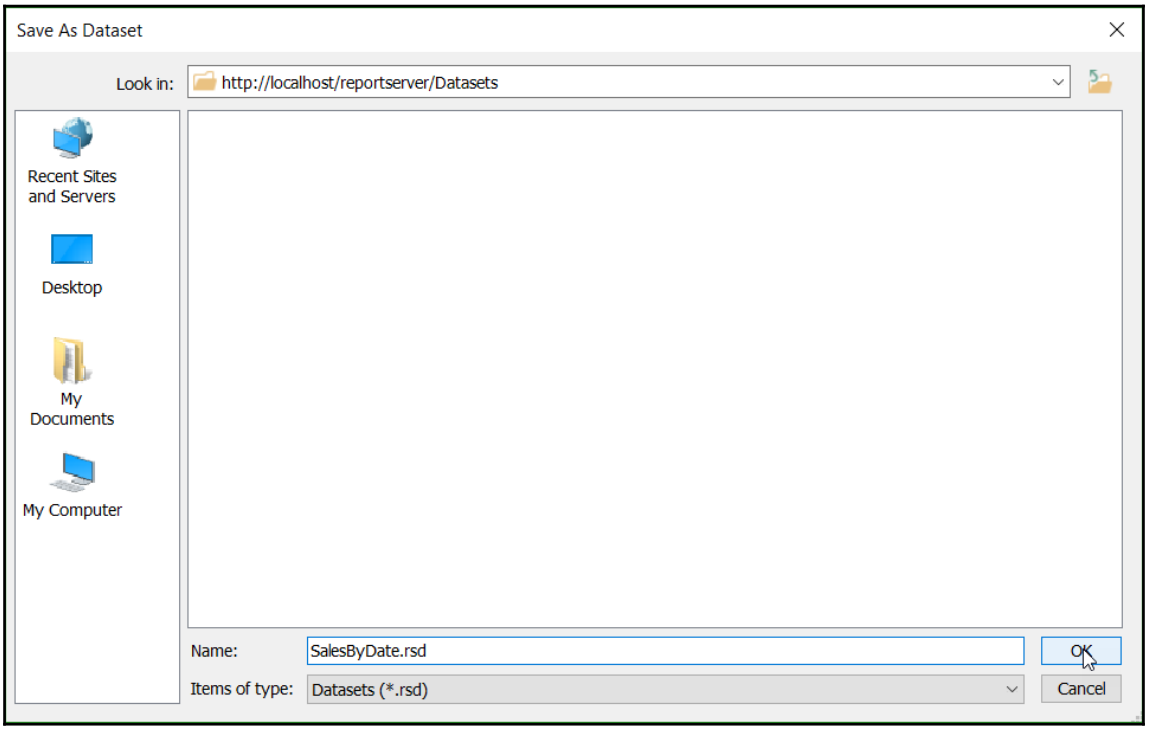


Figure 4.14

10. Begin a new report by going to **File** | **New** on the menu.
11. Select **Table or Matrix** wizard.
12. Select the **Choose an existing dataset in this report or a shared dataset** option and click on **Browse**.

13. Navigate to the **Datasets** folder, and select the new **SalesByDate** dataset; then click on **Open**:

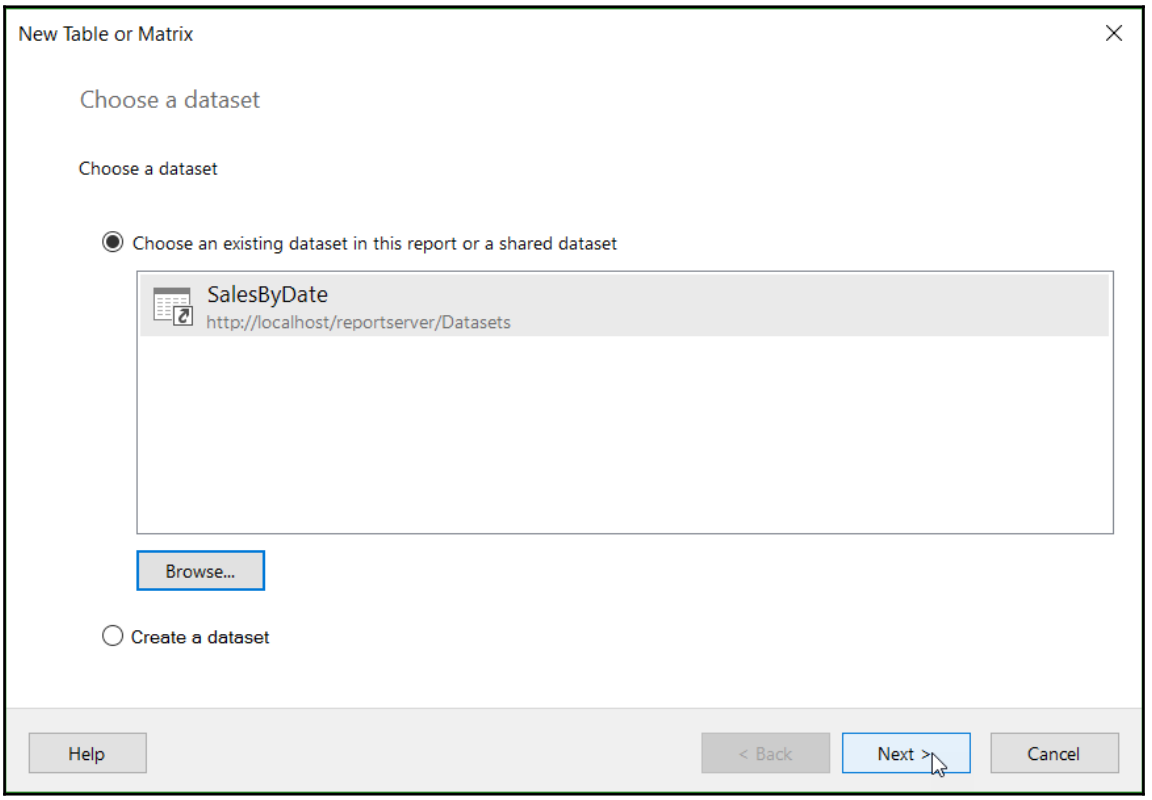


Figure 4.15

14. Click on **Next**.
15. Place **Customer** in the **Row** groups and the remaining fields in the **Values** area, and then click on **Next**.
16. Uncheck **Expand/collapse** groups, and click on **Next**.
17. Click on **Finish** to generate the report.
18. Navigate to **File** | **Save** to save the report to your local drive. Use `Report 04-03.rdl` as the name.
19. Click on **Run** to view the report.

How it works…

SSRS allows users to store SQL queries in the form of datasets in the Report Portal. This makes them reusable even for users unfamiliar with the creation of SQL statements.

In this recipe, we used the **Edit as Text** option to paste in a SQL query (Figure 4.13). In most cases, developers are the ones who generate datasets, and thus using Edit as Text would be a common option, pasting in the query developed in SQL Server Management Studio. However, a user can use the tool in query builder mode (as seen in Figure 4.06).

How to publish reports to Reporting Services

Publishing reports to the SSRS Report Portal is very simple. It's basically a matter of saving to a location, the Report Portal.

Getting ready

We'll be using `Report 04-01`, created in this chapter's *How to design reports with the wizard* recipe, for this example. Be sure you have created it and know where it is located.

In addition, we will want a folder to save the report into. Just as you did when creating the folder for datasets, go to the home of your Report Portal. Select the + (**New**) button and pick **Folder**. Name it `Chapter04`.

How to do it…

1. If Report Builder is closed, open Report Builder. On the **Getting Started** screen, select **Open**. If you already have Report Builder open, simply select **File** | **Open** on the menu.
2. Select **My Computer**; find and open `Report 04-01.rdl`.
3. On the menu, select **File** | **Save As**.
4. In the **Save As Report** window, click on **Recent Sites and Servers**.

5. In the center area will be all the servers you've previously accessed. In this recipe, we've used `http://localhost/reportserver`. Double-click on it.
6. At the top of the window is a text box labelled **Look in.** The URL to the server should now appear. The main window area should now have all the folders on the server, including the new `Chapter04`. Double click on it.
7. The **Look in** area should now have the full path to the server. Just click on the **Save** button:

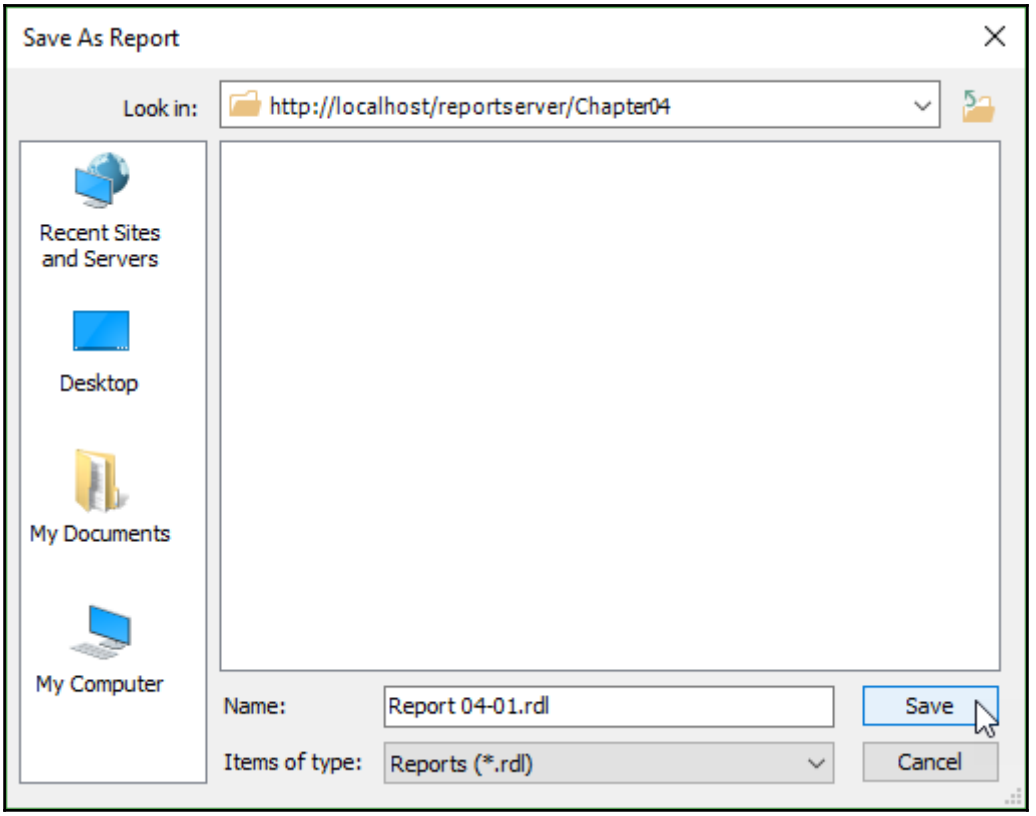


Figure 4.16

8. Go back to your web browser and return to Report Portal.
9. Open the `Chapter04` folder.
10. You should see `Report 04-01` in the list. Click on it to open.

11. The report should run and display the data as shown in Figure 4.17:

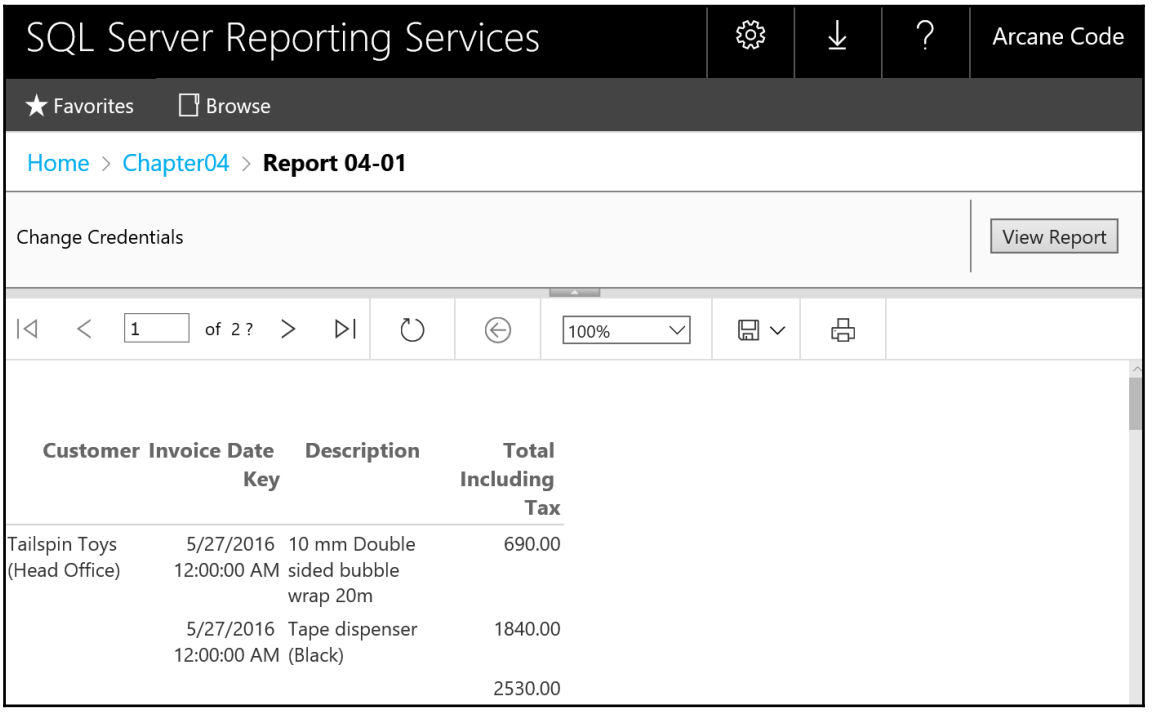


Figure 4.17

How it works…

Report Builder allows users to easily save reports directly to the SSRS server. This makes it easy to create new reports and share them with other users. Microsoft made this process very simple, leveraging the standard File / Save paradigm everyone is familiar with.

Working with published report parts

In addition to entire reports, Report Builder allows you to publish parts of a report. Components that are eligible to become a report part include:

- Charts
- Gauges
- Images (and embedded images)
- Maps
- Parameters
- Rectangles
- Tables
- Matrices
- Lists

By creating report parts, users can integrate these into their own reports to create SSRS reports that have the design of a dashboard.

Getting ready

We will reuse the report created in this chapter *How to design reports manually* recipe. It included a chart that we will publish as a part and then import into a second report. Be sure you have created this report and have it available to use in this recipe.

How to do it…

1. In Report Builder, open `Report 04-02.rdl`.
2. In the **View** tab, place a check beside **Properties**.
3. Click on **Chart** within the report to highlight the **Chart1** control.
4. In the properties on the right, find the **Name** property.
5. Rename it from `Chart1` to `TotalSalesByYearPieChart`.
6. On the **File** menu, click on **Publish Report Parts** as shown here:

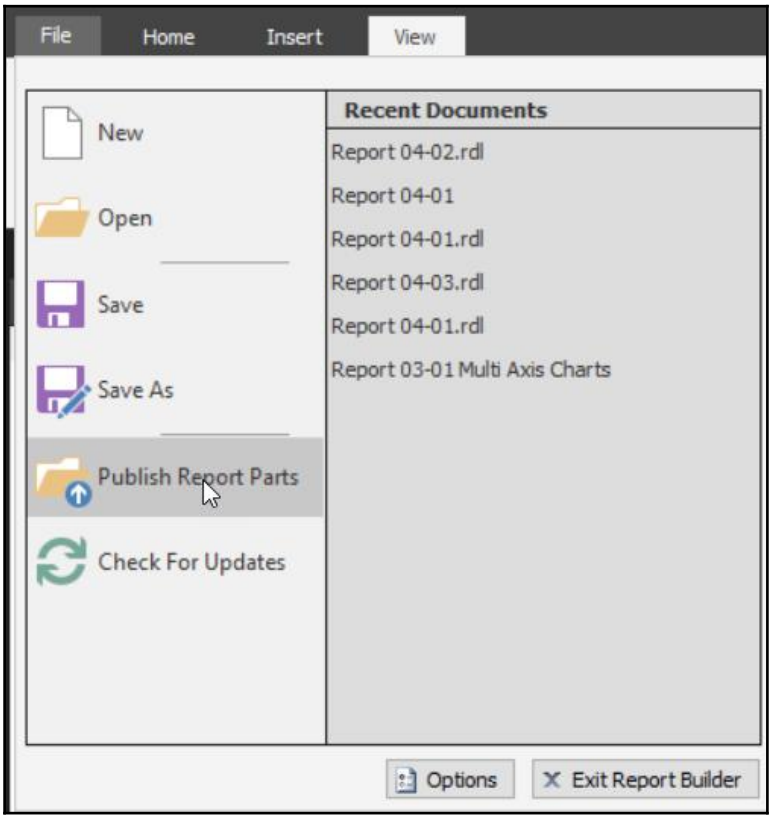


Figure 4.18

7. In the **Publish Report Parts** window shown next, click on **Publish all report parts with default settings**:

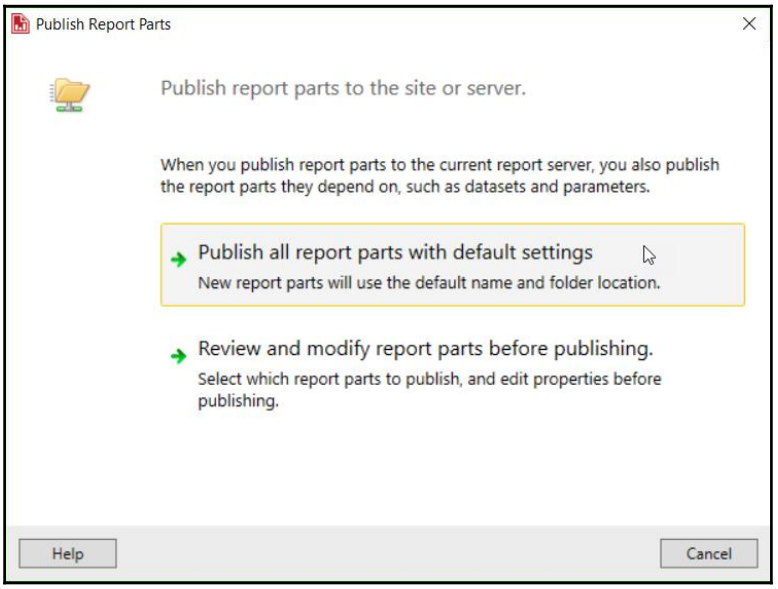


Figure 4.19

8. Return to the Report Portal. As shown in the next figure, you will now see a new folder, **Report Parts**. Click on it to open the **Report Parts** folder:

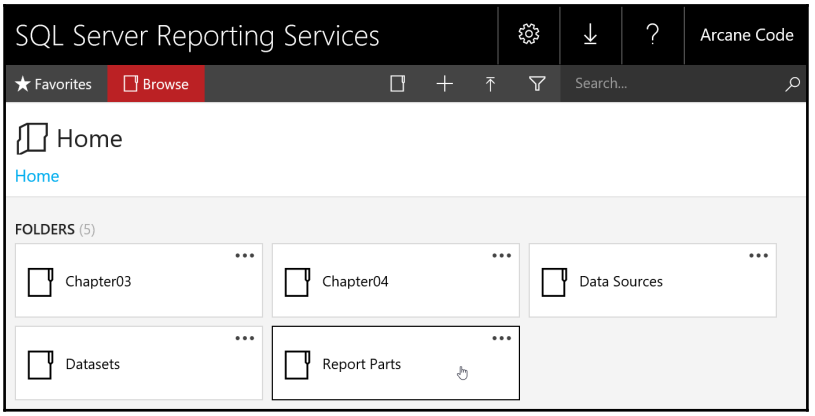


Figure 4.20

9. Your new report part should now appear.
10. Return to Report Builder, and begin a new report (**File** | **New**).
11. Select **Blank Report**.
12. Click on the **Insert** tab.
13. Click on **Report Parts**.
14. The **Report Part Gallery** will now appear on the right.
15. In the search area at the top, type **Total**; then click on the magnifying glass on the right. You should then see a list of parts that contain the word **Total**:

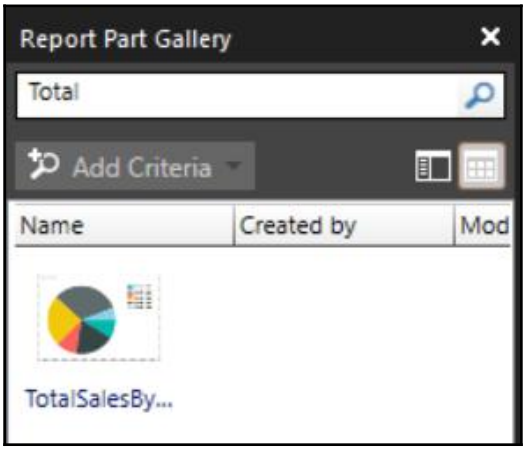


Figure 4.21

16. Drag and drop the report part from the gallery onto the report.
17. Save the report to your local drive as `Report 04-04.rdl`.

18. On the **Home** tab, click on **Run** to execute the report and check the displayed results:

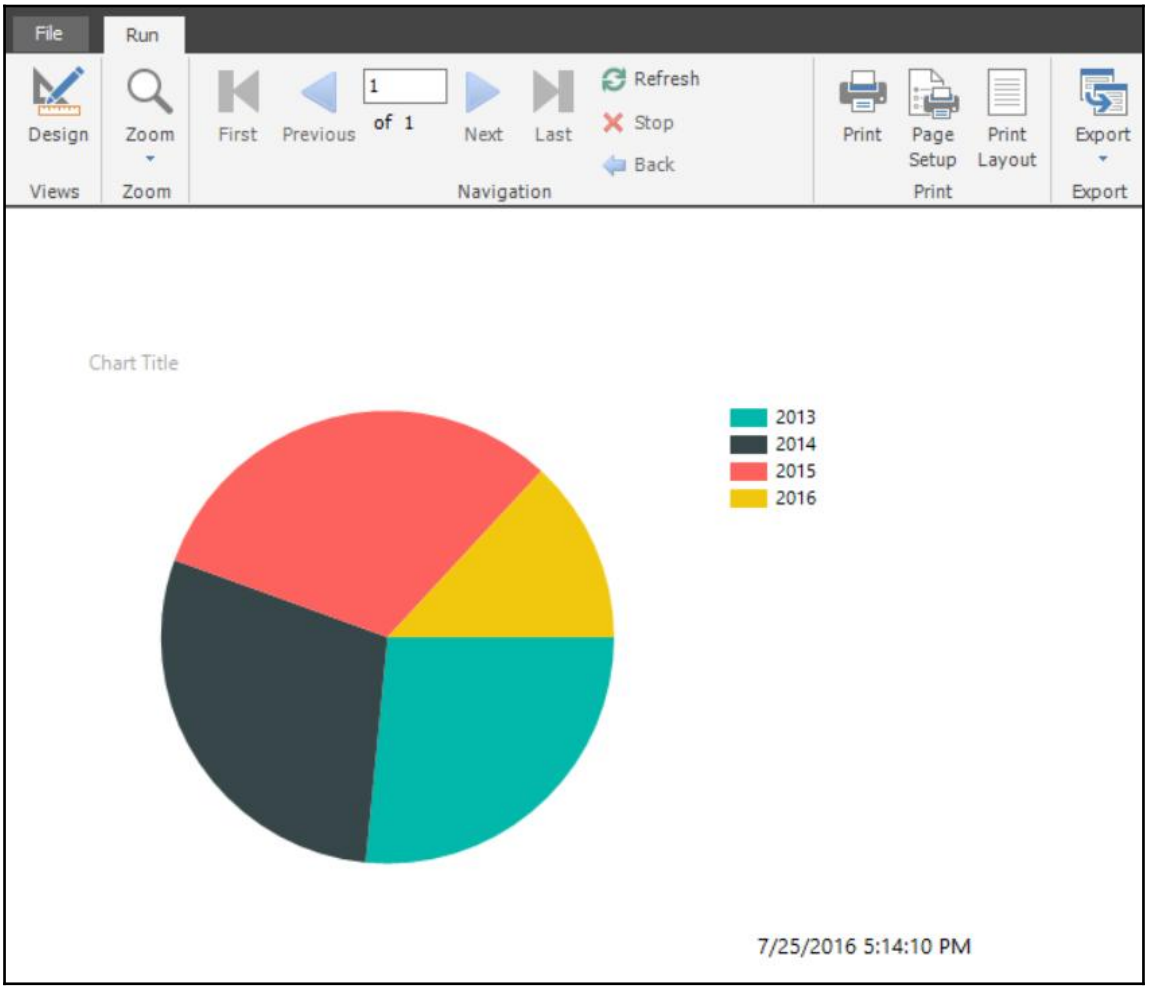


Figure 4.22

19. Click on the **Design** button to return to the report designer.

How it works…

In this example, we published and used a single report part. This functionality allows users to create many report parts, from which they can assemble reports and dashboards for their use.

In the course of the recipe, we accessed the **Properties** pane. This properties pane works similarly to the one in SSDT, letting you update and alter properties of the various items on the report.

Also note in Figure 4.19 that we chose to publish all items in the report. This is a common scenario; typically you would create a report with a single component that you would want to convert to a part.

If you had an existing report with multiple components and only wished to publish some of them as parts, you could have selected the second option, **Review and modify report parts before publishing**. This would have brought up a dialog with a list of all the components in the report eligible to be published. That lets you select which ones to convert to report parts.

Changing reports published in Reporting Services

There is an old saying: *the only thing constant is change*. It is inevitable that once a report is published, changes will be required. Fortunately, Report Builder makes updating a report quite easy.

In this recipe, we'll make some simple changes; remember though that you can make any type of change to a report.

Getting ready

Earlier in this chapter we published the `Report 04-01` report to the Report Portal. We will make an update to the report in this recipe, so if you've not created and published it, take a moment to do so now. The report was created in the *How to design reports with the wizard* recipe and published in the *How to publish reports to Reporting Services* recipe.

How to do it…

1. If Report Builder is closed, launch it. Select Open to access an existing report. If you already have Report Builder running, you can simply select **File** | **Open**.
2. Click on **Recent Sites and Servers** in the Open Report dialog.
3. Navigate to your SSRS server and then to the `Chapter04` folder.
4. Click on `Report 04-01`, and then click on **Open** (you can also opt to double-click on the report name to open it).
5. Click on the **Click on to add title** text box and enter `Customer Sales`.
6. Click in the table and expand the columns to give each one more room.
7. Right-click on the **Invoice Date** field (not the column header), and select **Text Box Properties**.
8. Click on the **Number** page, under **Category** select **Date**. We'll take the default date, so just click on **OK**.
9. In the menu, select **File** | **Save**.
10. Return to the Report Portal, navigate to the `Chapter04` folder, and run the report. The report should now display the updated title, as shown here:

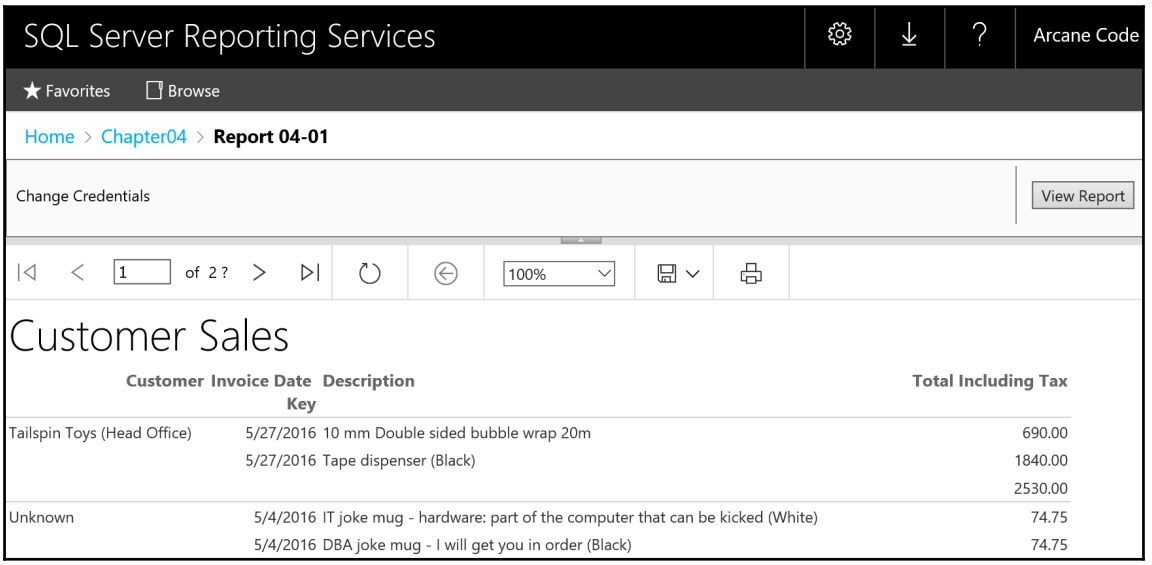


Figure 4.23

How it works…

When you open a report from the server, Report Builder remembers where that report came from. Thus, when you save it, Report Builder saves it on the server it came from.

After clicking on the Recent Sites and Servers button, you may not see your server. If not, you can simply type in the address in the filename area at the bottom. The dialog will then navigate to that server and allow you to select a report from it.

5

Improving User Experience – New Designing and Visualization Enhancements

Along with previous chapters, you are now well informed on basic to advanced report authoring, including ad-hoc report building. Since we spoke about paginated reports (traditional tabular and graphical reports) as well as modern graphical reports, you have clearly seen the value of Reporting Services, how user-friendly the environment is, and how you can address enterprise reporting requirements with Reporting Services 2016.

SQL Server 2016 comes with enhancements for user experience. In order to provide a modern reporting platform, new visualizations along with additional designing features have been added to Reporting Services 2016. It is time to have a look at these new enhancements.

In this chapter, we will cover the following recipes:

- Positioning parameters
- Visualizing data with modern styles
- Visualizing data with the TreeMap data region
- Visualizing data with the Sunburst data region
- Integrating R with Reporting Services

Introduction

Being a report developer, what we really need is an environment that helps us to make great reporting solutions using modern techniques and well-equipped tools. If the tool makes report design complex and hard-to-manage, then output of the effort will not be as good as everyone expects. It will not only discourage us from continuing to use the tool, but also force us to go for another tool. Knowing this, Microsoft has been maintaining its standard and it has done it again, giving us more enhancements on the toolset that we use for creating reports.

End users always prefer to see a pleasant output on required information. They consider the appearance of the output, whether it is properly structured, whether the right visual element has been used, and whether it supports the analysis they need to do on it. Microsoft has not forgotten the end-user-experience; considering the importance of it, just like the improvement on development, many enhancements have been done in various areas addressing end-user-experience.

This chapter explains some of these with five recipes. Just like the previous chapters, a Reporting Services solution is available with this chapter called `Chapter05.sln`. It has six ready-made reports that are related to recipes discussed in this chapter:

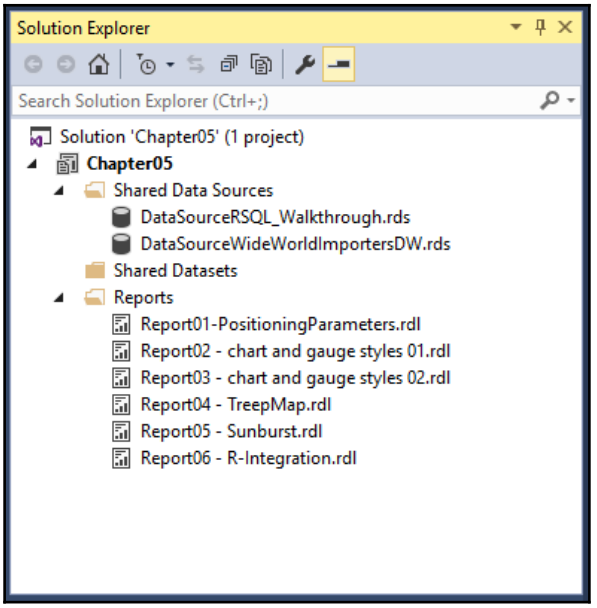


Figure 5.01



Note that you need to configure the shared data source `DataSourceWideWorldImportersDW` to `WideWorldImportersDW` database. The other shared data source `RSQL_Walkthrough` is discussed in one of the recipes, including the configuration.

Positioning parameters

It is true that most of the formal reports (or production report) are parameterized. We add parameters to the report for making it look dynamic and flexible without making it a static report. Therefore, report parameterization is one of the key features in designing, hence it needs flexibility.

Since Reporting Services inception, we can add parameters to reports. Parameters are shown in a layout that is fixed for two columns. We have no way of changing the position we need, but it allows us to change the order. A parameter pane in a report with one or two parameters does not look unorganized, but a pane with more than two parameters may look disorganized and untidy. Assume that you have a report that shows City, Start Date, and End Date as parameters. If the order of them is set as City, Start Date, and End Date, then City and Start Date will be displayed in the first row and End Date will be displayed in the second row. That makes the report parameter pane look disorganized, as it is always better to display Start Date and End Date together, showing that it expects a range.

Reporting Services 2016 solves this by allowing us to use position parameters as per our needs, giving more control over them.

Getting ready

First of all, let's have a look at how prior versions of SQL Server Reporting Services display parameters. The report `Report01-PositioningParameters` in the solution has five parameters: **Stock Item**, **Employee**, **City**, **Start Date**, **End Date**. If you design this report using a version prior to SQL Server 2016 Reporting Services and set the order of them as mentioned above, the following report will look like figure 5.02:

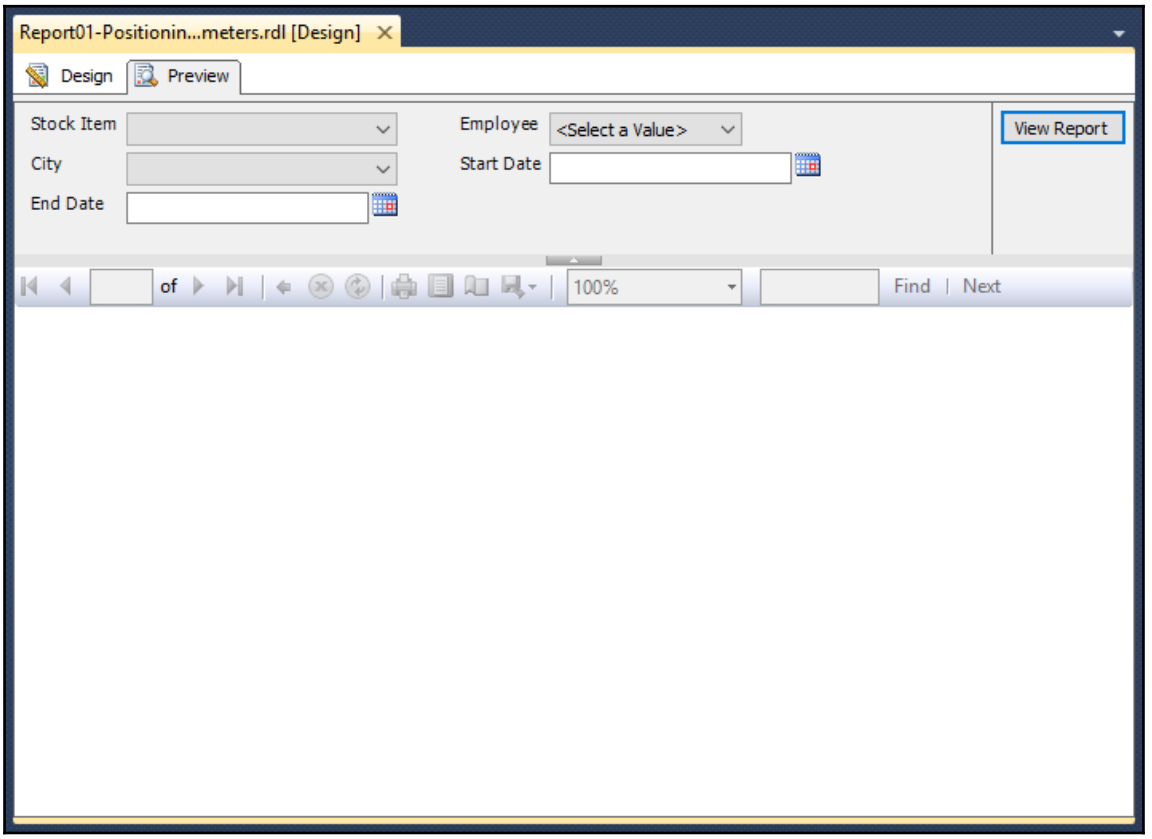


Figure 5.02

As you can see, it shows parameters as the order is set. It will look more organized if **Start Date** and **End Data** are set with one line and rest in another line.

Now, open the `Report01-PositioningParameters` report given with the solution and click on **Preview**. You will see how parameters are organized in the parameter pane. The **Start Date** and **End Date** parameters are in one line and one section, and the other parameters are in a different section, making the parameter layout clearer and organized:

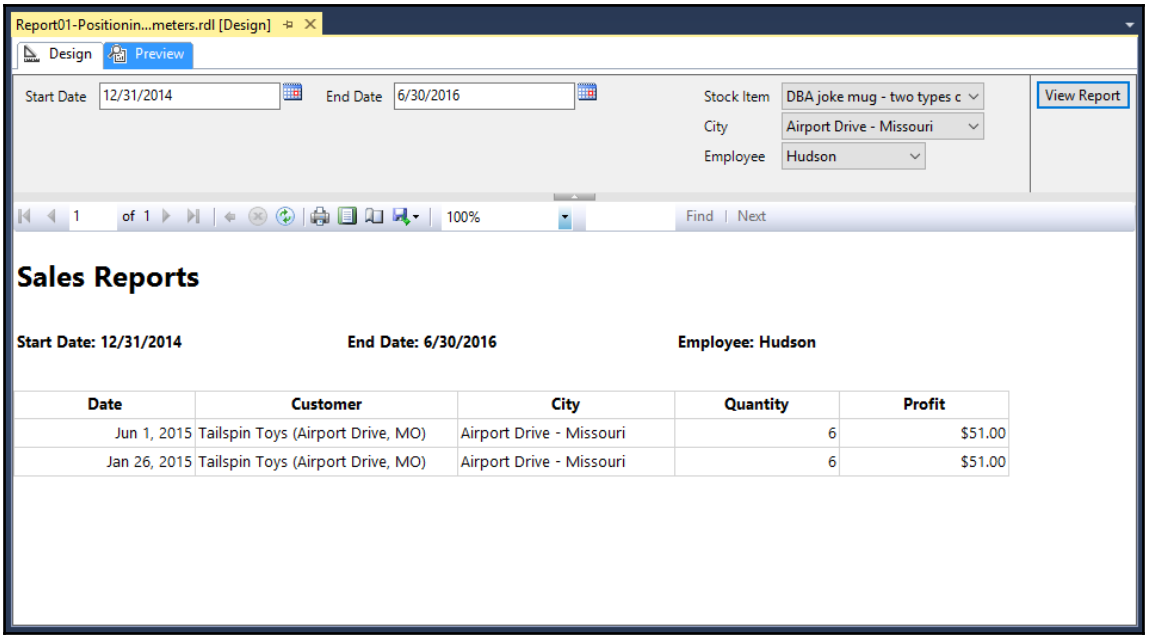


Figure 5.03

Let's see how they are organized as shown previously.

How to do it…

1. Open the **Design** of the report, you will see **Parameter Pane** in **Design View**. This pane was not available with Reporting Services 2014 and this can be used to change the position of parameters as needed.
2. Drag a parameter from Parameters in Report Data and drop it into any cell in the table. You can get additional options by right clicking on the table for adding more columns or more rows. You can have empty cells for keeping gaps between parameters, forming sections for related parameters.



Note that the default pane has four columns and two rows. The new pane distinguishes visible parameters and hidden parameters well. As you see with the following image, parameters with a white background indicate that cell contains either a hidden or internal parameter.

3. If your parameter accepts NULL as one of the possible values and you have enabled it by selecting **Allow null value** checkbox, the parameter pane in the **Design View** shows it as well. As you can see, there is an indicator for an **Employee** parameter because this parameter accepts a null value.



Note that selecting Allow null value does not add a null to available values. If you want to allow a null value for the parameter, you must include it in your dataset.

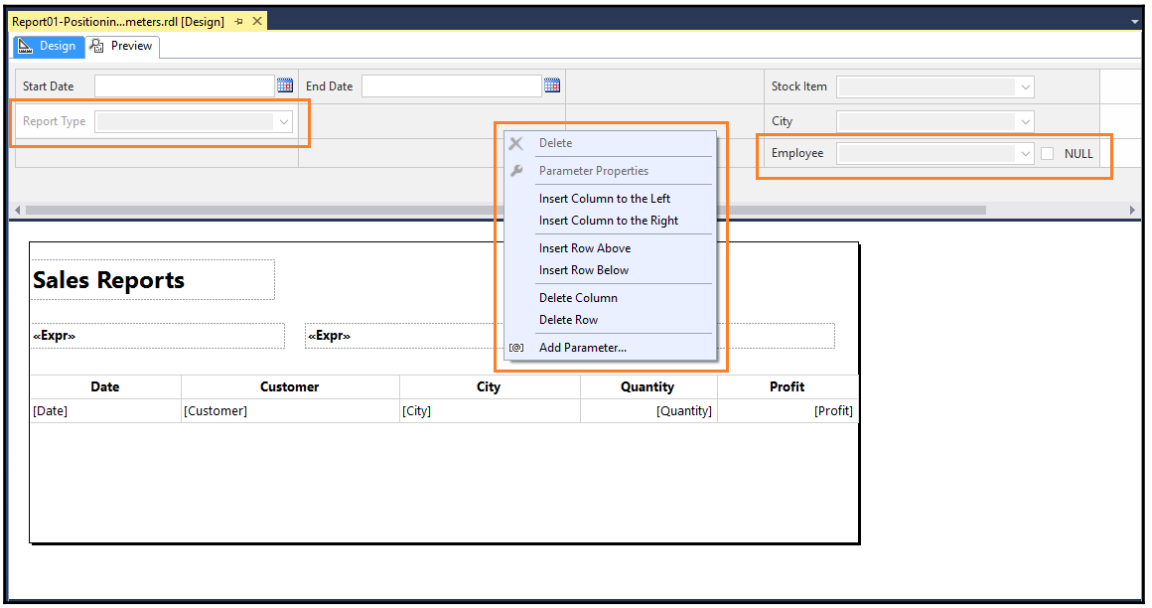


Figure 5.04

How it works…

Once the Parameter Pane is configured, it will be displayed based on the positions set. Just like the previous version, you will see the **Show or Hide Parameter Area** icon just below the parameter pane in preview. In addition to that, you will see a horizontal scrollbar if you have a lot of columns in your parameter pane. You do not get a vertical scrollbar if you have a lot of rows, but it increases the height of the parameter pane based on rows you have. This is how you see it in the preview:

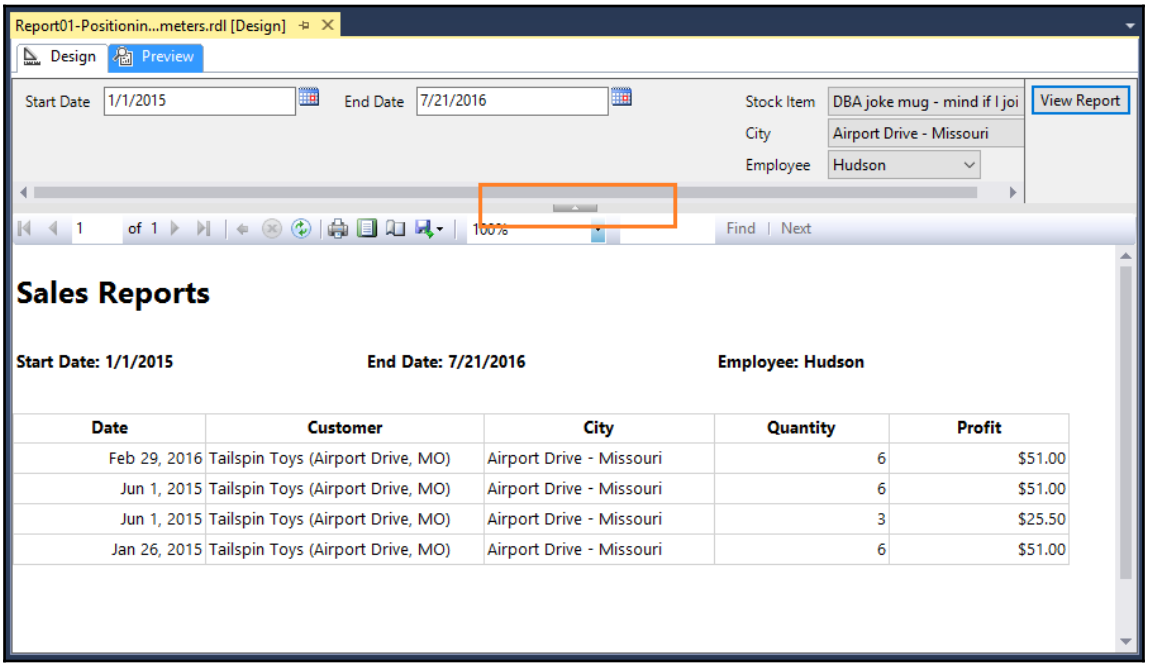


Figure 5.05

Visualizing data with modern styles

Let's see the modern look of data regions. A significant enhancement has been done on it by Microsoft, making sure that data regions appear with beautiful visualizations empowered by modern styles.

Getting ready

Let's compare some modern styles with old Reporting Services 2014 styles. The following image shows a chart created with 3-D Stacked Cylinder chart type using sales data:

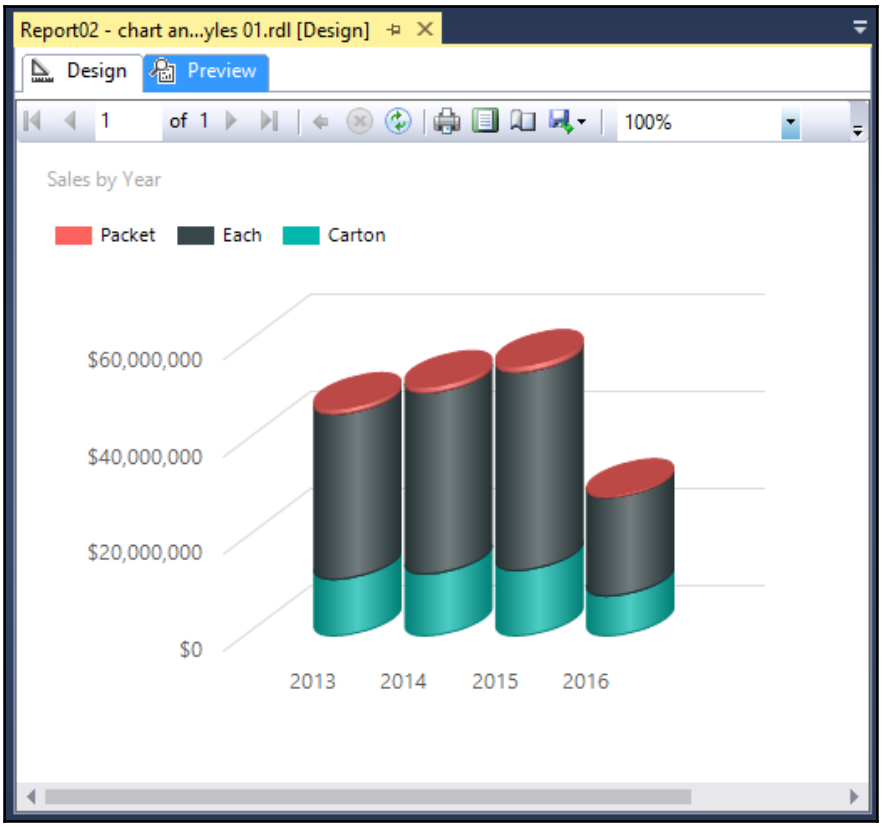


Figure 5.06

The same chart using Reporting Services 2014 looks like figure 5.07:

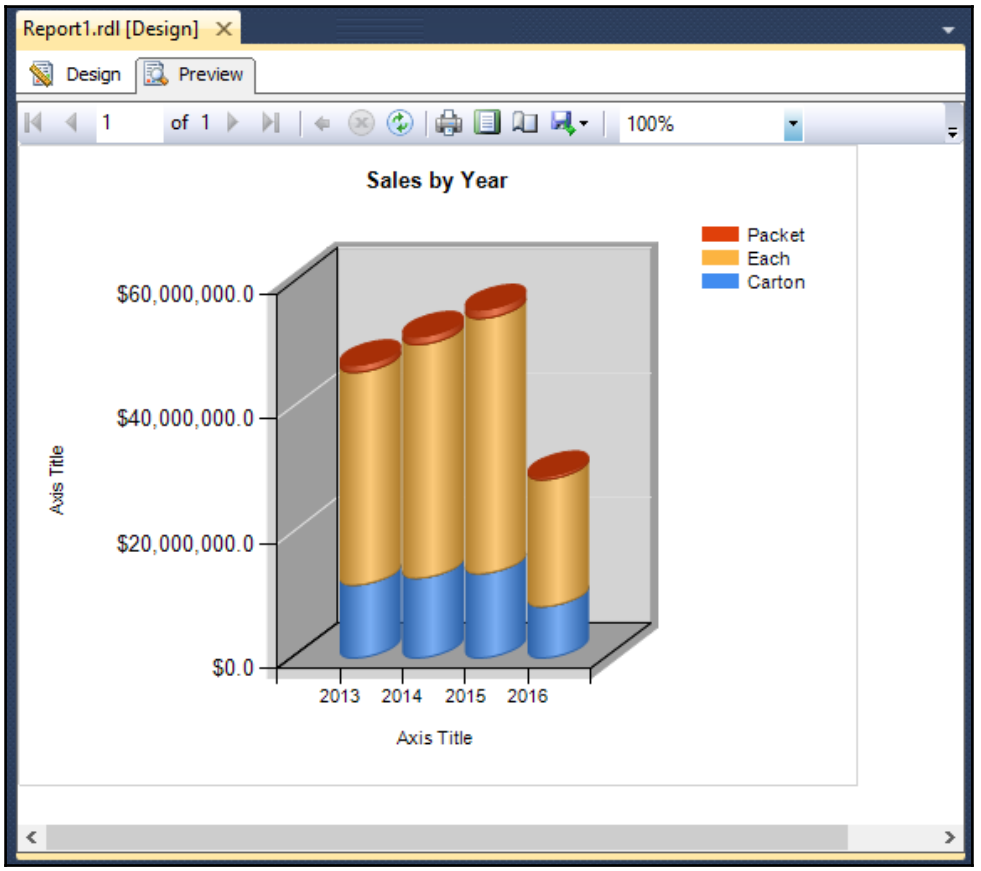


Figure 5.07

How to do it…



Note that creating basic charts and steps to follow are covered in `Chapter 2`, *Authoring Reports with SQL Server Data Tools*. Please refer to it for understanding how to create a basic chart.

How it works…

As you can see, there is a significant difference between the two charts in terms of look and feel. It is all because of the modern styles applied to Reporting Services 2016. In order to get this look and feel, you do not need to do anything specifically, it comes by default.

Not only the smoothness of the chart, even the default colors that are automatically set for chart components such as bars or cylinders, the background colors, and the way components are displayed, have been changed, making the chart look smooth and modern.

There's more…

Along with the new, modern visualization, Reporting Services introduces two new chart types: TreeMap and Sunburst, which are ideally suited for visualizing summarized values with hierarchical data. The next two recipes are dedicated to them, where you will learn more about TreeMap and Sunburst chart types.

Visualizing data with the TreeMap data region

Reporting Services 2016 introduces two new chart data regions: TreeMap and Sunburst. Let's discuss the TreeMap data region with this recipe and Sunburst with next recipe.

Visualization plays the key role of presenting information. Yes, it is true that most reports are formal reports and based on tabular format. But when presenting summarized values, specifically with hierarchical data, graphical data regions do the best and deliver the required information in a user-friendly and meaningful manner.

The TreeMap helps you to divide loaded data into rectangles representing different levels in the hierarchy. The sizes of rectangles are based on numeric values related to items in the level. Although it does not look like a hierarchical structure at a glance, just like other hierarchical visual data regions such as decomposition tree, it shows the hierarchy with arranged and structured rectangles.

The visualization of TreeMap forms with a larger rectangle that represents the highest level and multiple smaller rectangles, breaking the largest one, representing either the next level of the hierarchy or items in the same hierarchy. It further breaks inner rectangles into multiple rectangles if more levels are available. The rectangles are arranged such a way that shows largest rectangle in upper left corner and smaller one in lower right corner based on the numeric values.

Getting ready

Before we start with creating a report, let's see how a TreeMap appears with a hierarchical dataset. Figure 5.08 shows a preview of the `Report02 - TreeMap` report in the `Chaper05 solution`:

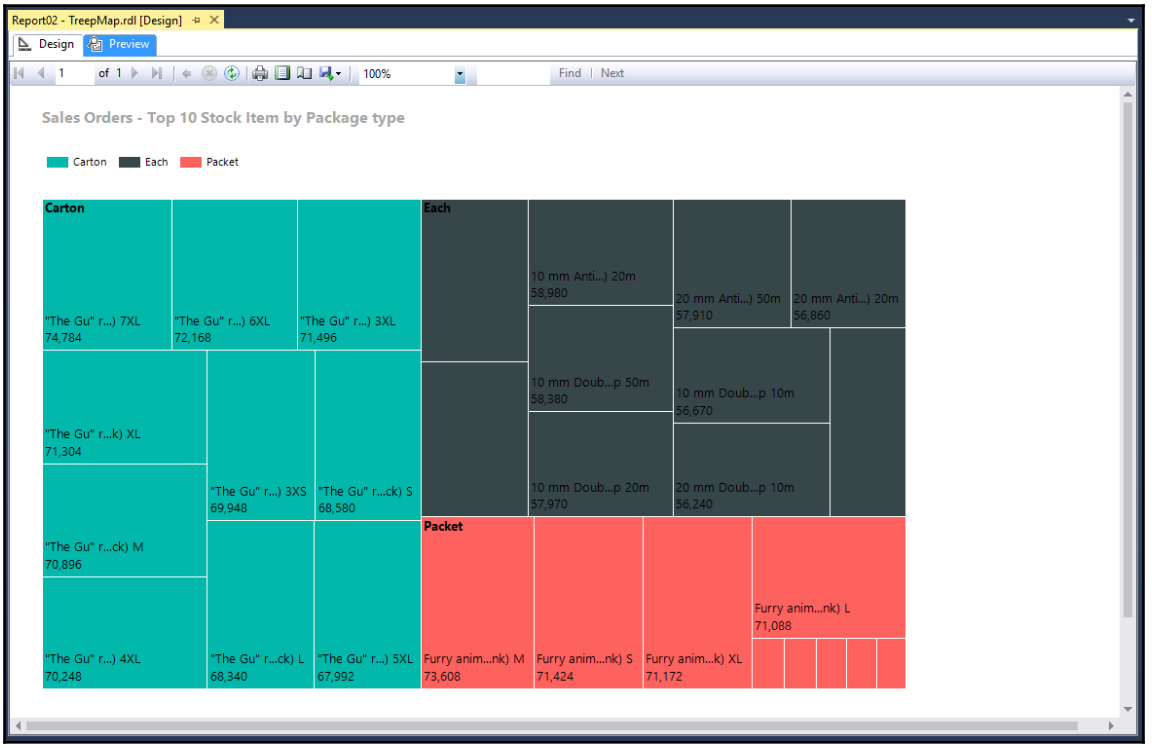


Figure 5.08

This report shows Order Quantity based on orders placed by customers. The Order Quantity has been summarized by using a hierarchy of starts from Package Type and setting the next level as Stock Item. As you can see, the package type **Each** is shown with the largest rectangle in the upper left corner and the package type **Packet** is shown with the smallest rectangle in the lower right corner, because the highest order quantity is for **Each** and the smallest order quantity is for **Packet**. Other rectangles represent Stock Items and their sizes are based on order quantities.

The query used against the `WideWorldImportersDW` database is as follows:

```
SELECT t2.RowNumber, t2.[Buying Package], t2.[Stock Item], t2.Quantity
FROM (
  SELECT ROW_NUMBER() OVER (PARTITION BY [Buying Package] ORDER BY
t1.[Stock Item]) RowNumber
    , t1.[Buying Package], t1.[Stock Item], t1.Quantity
  FROM (
      SELECT  SI.[Buying Package], si.[Stock Item], SUM(o.Quantity)
Quantity
      FROM Fact.[Order] o
        INNER JOIN Dimension.[Date] d
          ON d.Date = o.[Order Date Key]
        INNER JOIN Dimension.[Stock Item] si
          ON si.[Stock Item Key] = o.[Stock Item Key]
      GROUP BY SI.[Buying Package], si.[Stock Item]
    ) t1
  ) t2
WHERE t2.RowNumber < 11
```

Let's see how we can create a report like this using the query given.

How to do it…

Add a new report to your Reporting Services solution and name it `Sales Orders - Top 10 Stock Item by Package type`. If you use the same solution, which is `Chapter05.sln` given with the book, you have to use the `DataSourceWideWorldImportersDW` shared data source for this report. If you use your own solution, create a connection to the `WideWorldImportersDW` database.

Let's follow the steps for creating the report:

1. Open the **Designer** and add a **DataSet** called **SalesOrders** connecting with the `WideWorldImportersDW` database. Set the query given above and click **OK** to create the dataset as shown in the following figure:

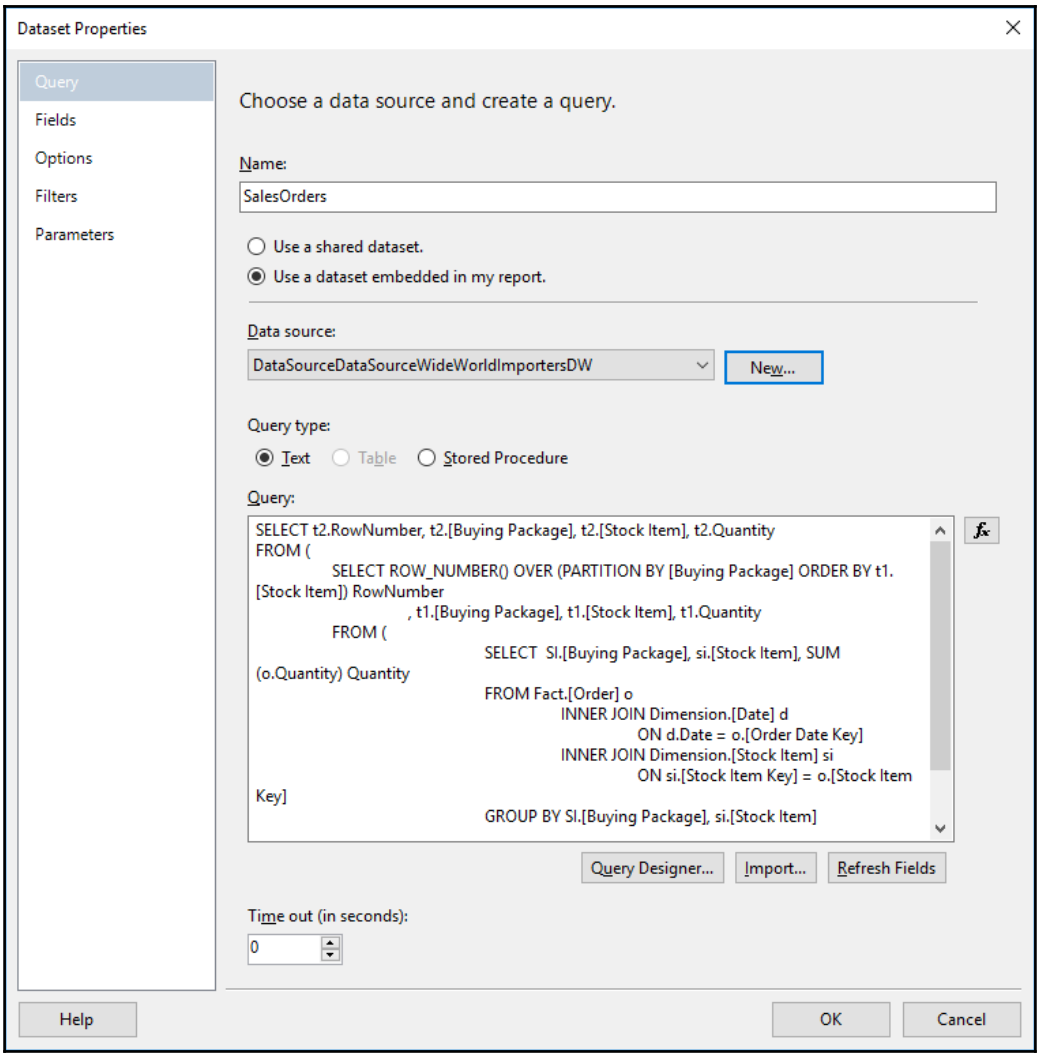


Figure 5.09

2. Add the TreeMap data region to the report. The TreeMap is listed under **Shape** category. Select it and click **OK**:

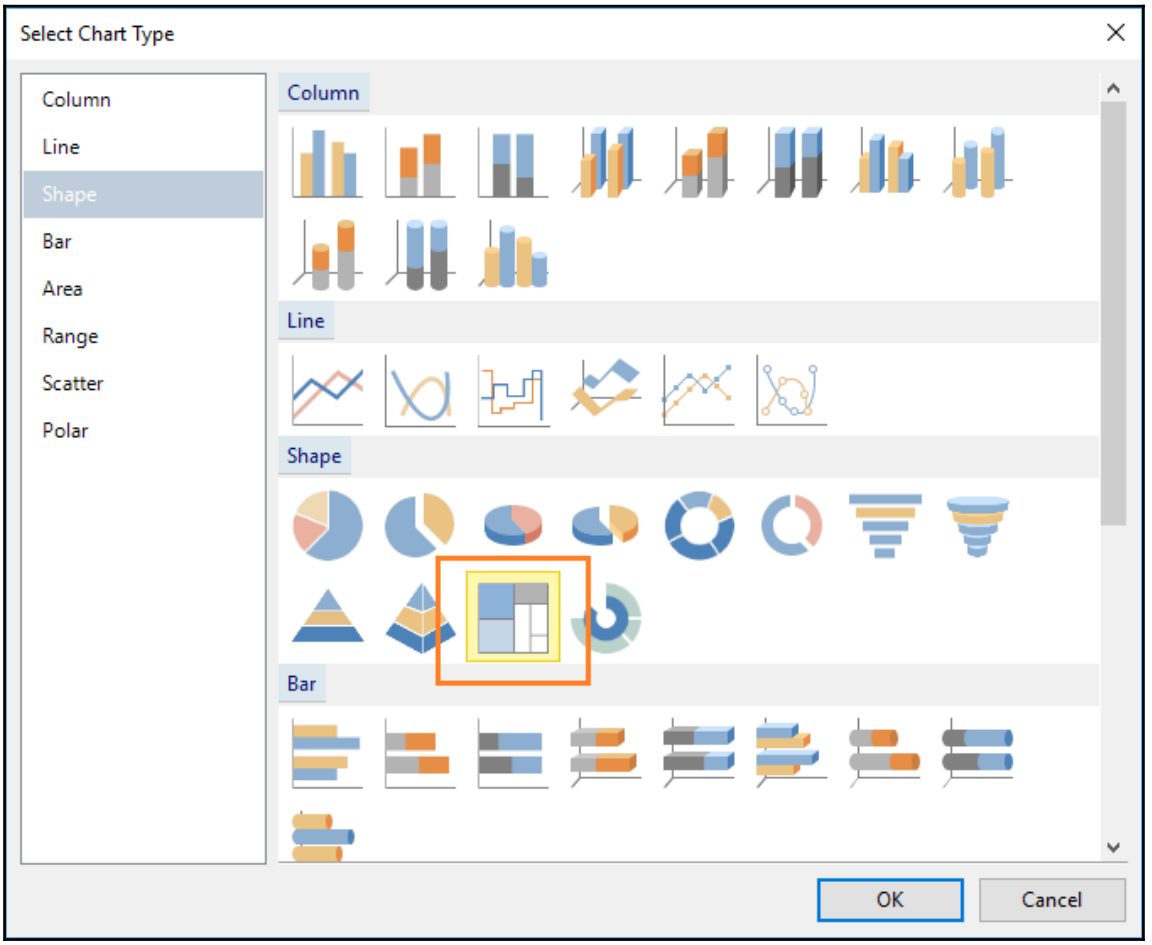


Figure 5.10

3. Now you have **Tree Map** data region in your report design view. Set the Chart Title as `Sales Orders - Top 10 Stock Item by Package type`.
4. Click on **Chart Area** for opening the **Chart Data** pane which can be used for setting **Values**, **Category Groups** and **Series Groups**. Note that we have only two levels with our data set. If you have more levels, they can be set with **Category Groups** in order. Set values for these as follows:
    - **Values:** Sum(Quantity)
    - **Category Groups:** Stock_Item
    - **Series Groups:** Buying_Package

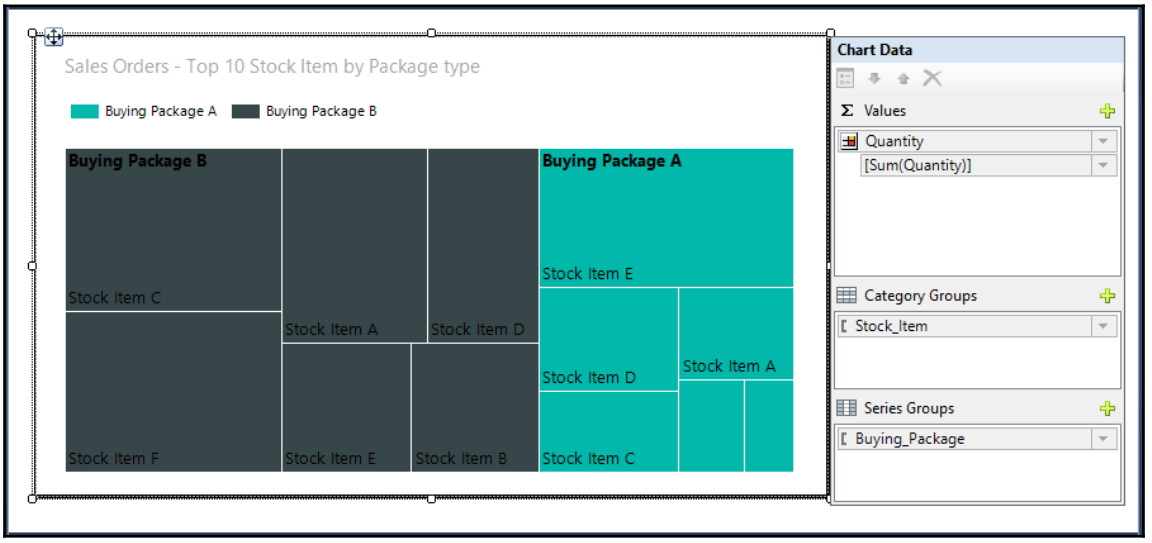


Figure 5.11

5. Click on the Preview tab and see how it appears:

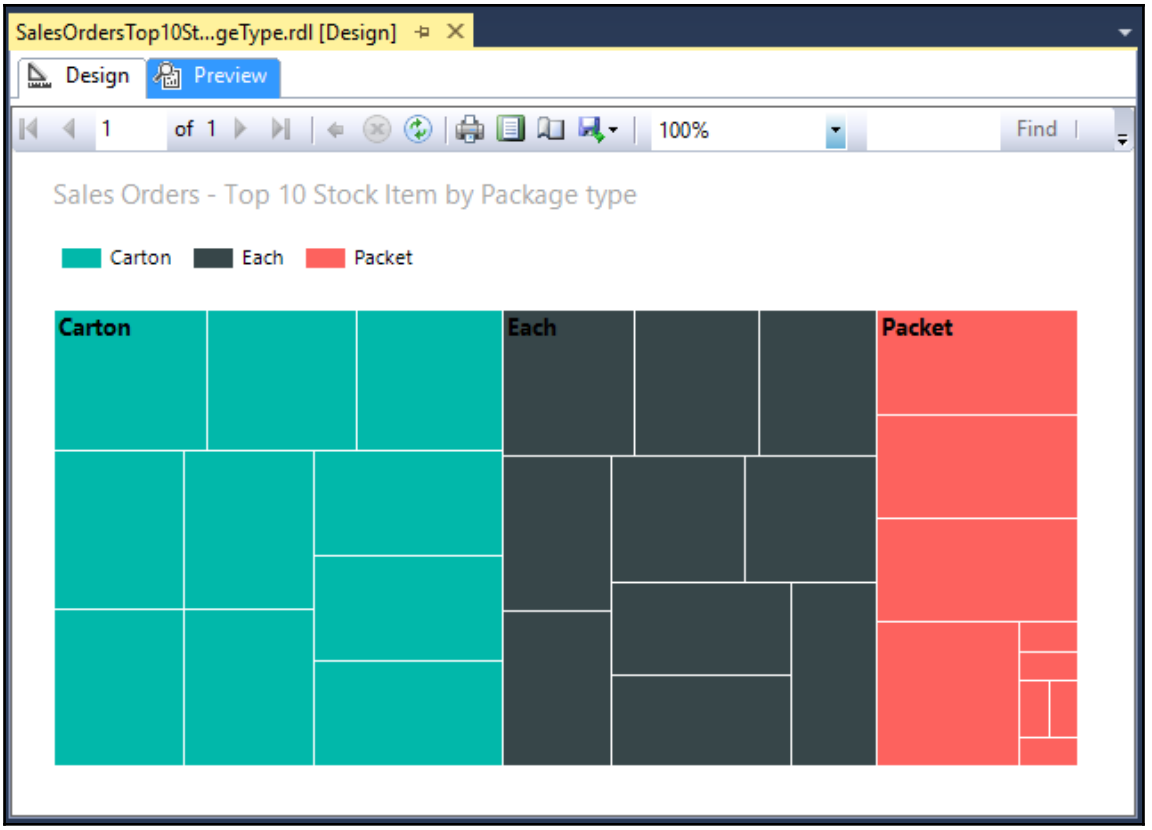


Figure 5.12

6. If it does not show labels in rectangles, we can shorten the label and add the quantity as well. Go back to **DesignView**. You will notice that you cannot edit the label related to **Stock Item**. In order to edit, right click on **Chart Series** and click **Show Data Labels** once:

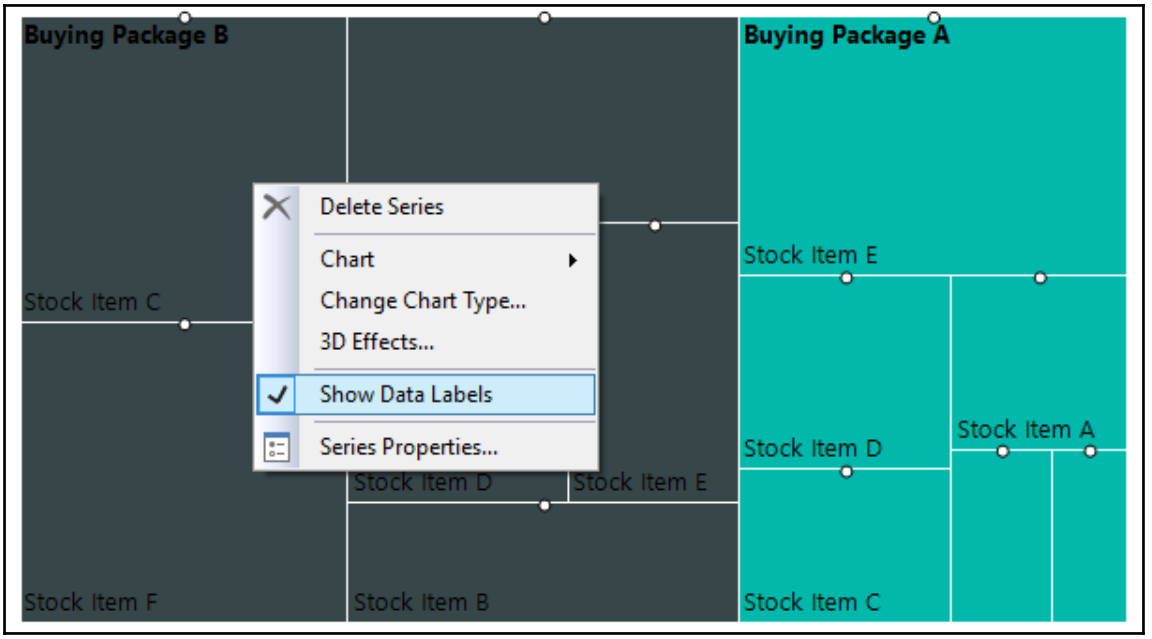


Figure 5.13

7. Once clicked, you should see updatable labels as shown here:

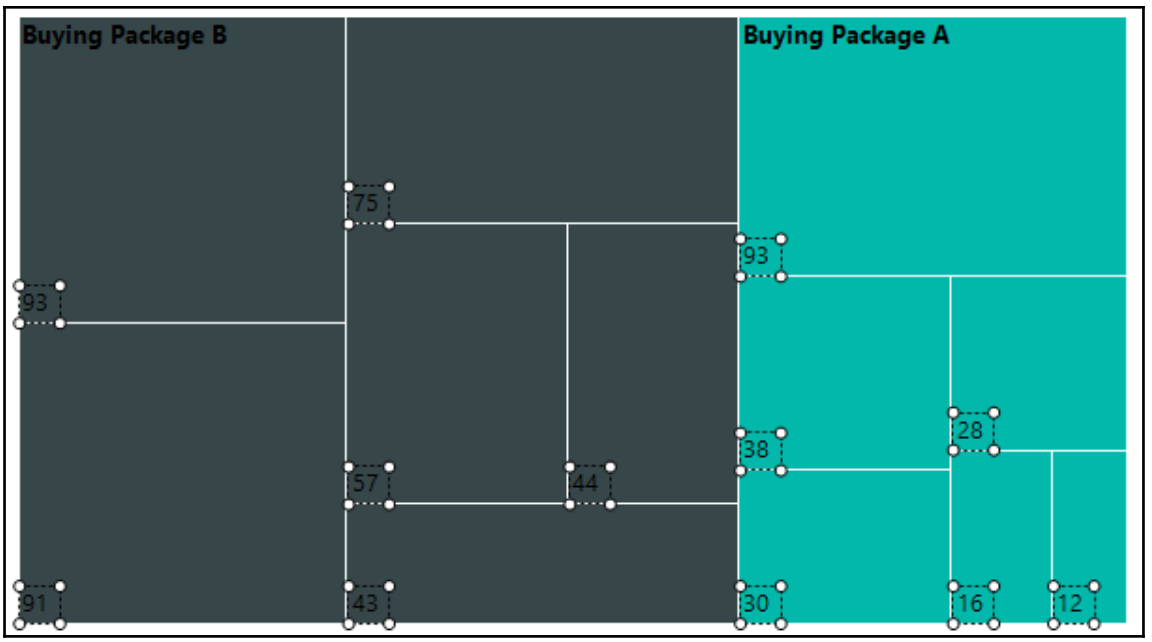


Figure 5.14

8. Right click on one of the labels for opening the related menu. Select **Series Label Properties** for configuring the label:

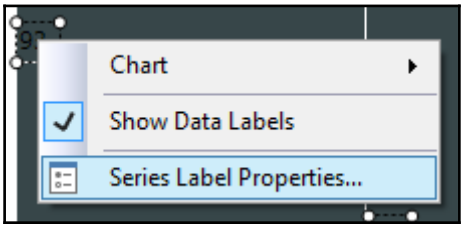


Figure 5.15

9. This opens the **Series Label Properties** window. Click on **Expression Icon** for **Label Data.** Enter the following expression for **Label Data**:

```
=LEFT(Fields!Stock_Item.Value, 10)
& "..." & RIGHT(Fields!Stock_Item.Value, 5)
& "\n" & Format(Fields!Quantity.Value, "N0")
```

10. You will get a **Confirm Action** dialog box. Select **Yes** and continue:

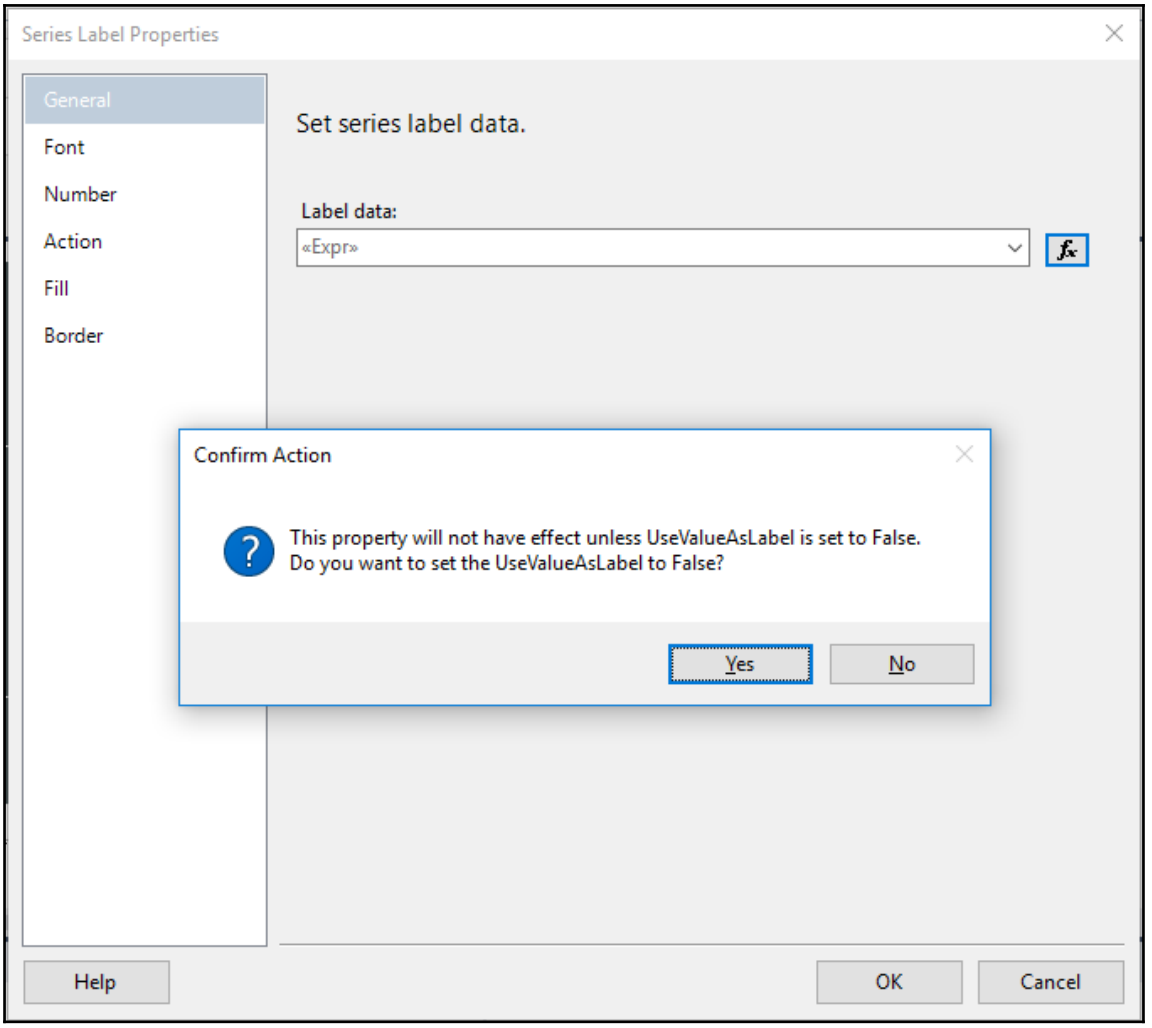


Figure 5.16

11. You may increase the width of *Chart* to make more space for labels. Preview and see how it is displayed. You should see the added labels clearly now.

The appearance of the chart is based on the width of the chart you set and the number of levels set with it. If you set more levels, there will be more rectangles and not all labels will be displayed. Therefore, to make it more readable, you can add tooltips for each rectangle. To add the tooltip, follow these steps:

1. Right-click on **Chart Series** and click **Series Properties** to open the **Series Properties** window:

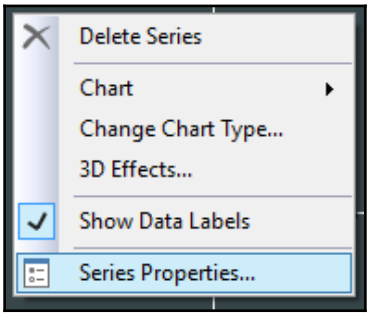


Figure 5.17

2. Once it is opened, set the same expression you set for **Data Label** to the tooltip under **Series Data**:

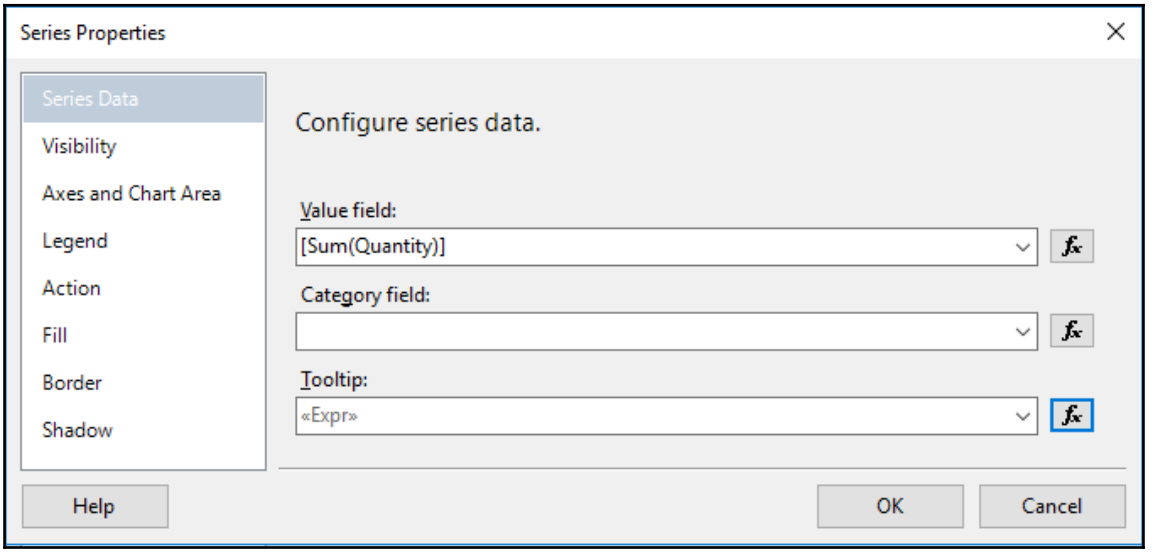


Figure 5.18

3. Click **OK** to save the setting and preview. You should see the tooltips when hovering over the rectangles.

How it works…

This data region consumes a data set based on levels and shows them with different sizes of rectangles. As mentioned above, it is always better to set tooltips as it is not possible to show all labels.

Visualizing data with the Sunburst data region

The Sunburst data region functions similarly to TreeMap. It is a great visualization for hierarchical data set with summarized values and it uses circles instead of rectangles.

The Sunburst forms a series of circles to represent levels of the hierarchy. The highest level of the hierarchy is set with the inner circle and lower levels of the hierarchy are formed outside the inner circle as rings. Multiple rings represent multiple levels.

The following chart shows a hierarchical data set with the Sunburst data region:

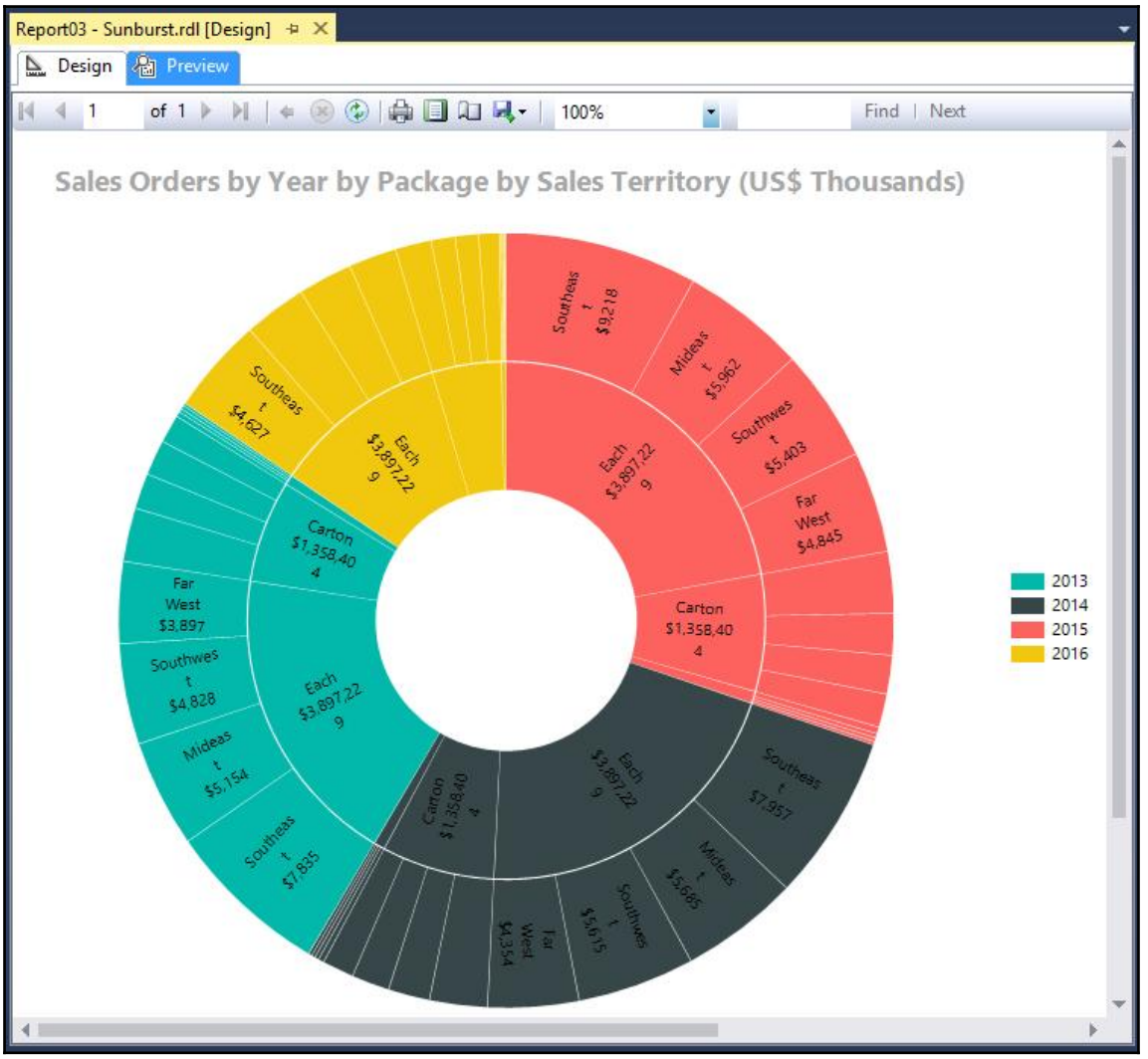


Figure 5.19

You can find this report in `Chapter05 solution` as **Report03 – Sunburst**.

Getting ready

Let's make a report with the Sunburst data region. Follow these steps for preparing the project for a new report:

1. Add a new report to your Reporting Services solution and name it `Sales Orders by Year by Package by Sales Territory (US$ Thousands)`.
2. Make a data set using the following query. Make sure the data set is connected to the `WideWorldImportersDW` database:

```
SELECT d.[Calendar Year], SI.[Buying Package],
c.[Sales Territory], SUM(o.[Total Excluding Tax]) TotalAmount
FROM Fact.[Order] o
  INNER JOIN Dimension.[Date] d
    ON d.Date = o.[Order Date Key]
  INNER JOIN Dimension.[Stock Item] si
    ON si.[Stock Item Key] = o.[Stock Item Key]
  INNER JOIN Dimension.City c
    ON c.[City Key] = o.[City Key]
WHERE c.[Sales Territory] IN ('Southeast',
 'Mideast', 'Southwest', 'Far West')
GROUP BY d.[Calendar Year], SI.[Buying Package],
c.[Sales Territory]
ORDER BY d.[Calendar Year], SI.[Buying Package],
 c.[Sales Territory]
```

The query we added returns **Total Sales Amount for Year**, **Package Type** and **Sales Territory**. Note that it is only for four sales territories.

Let's use the Sunburst data region for showing the resultset. Since the result is hierarchical, Sunburst visualizes the information in an appropriate manner, allowing us to see how the company performs in terms of **Package Type** and allowing us to see the areas to be monitored and improved.

How to do it…

Follow the steps for adding the Sunburst data region and configuring it.

1. Click on the **Chart** item in the **Toolbox** and select the Sunburst data region:

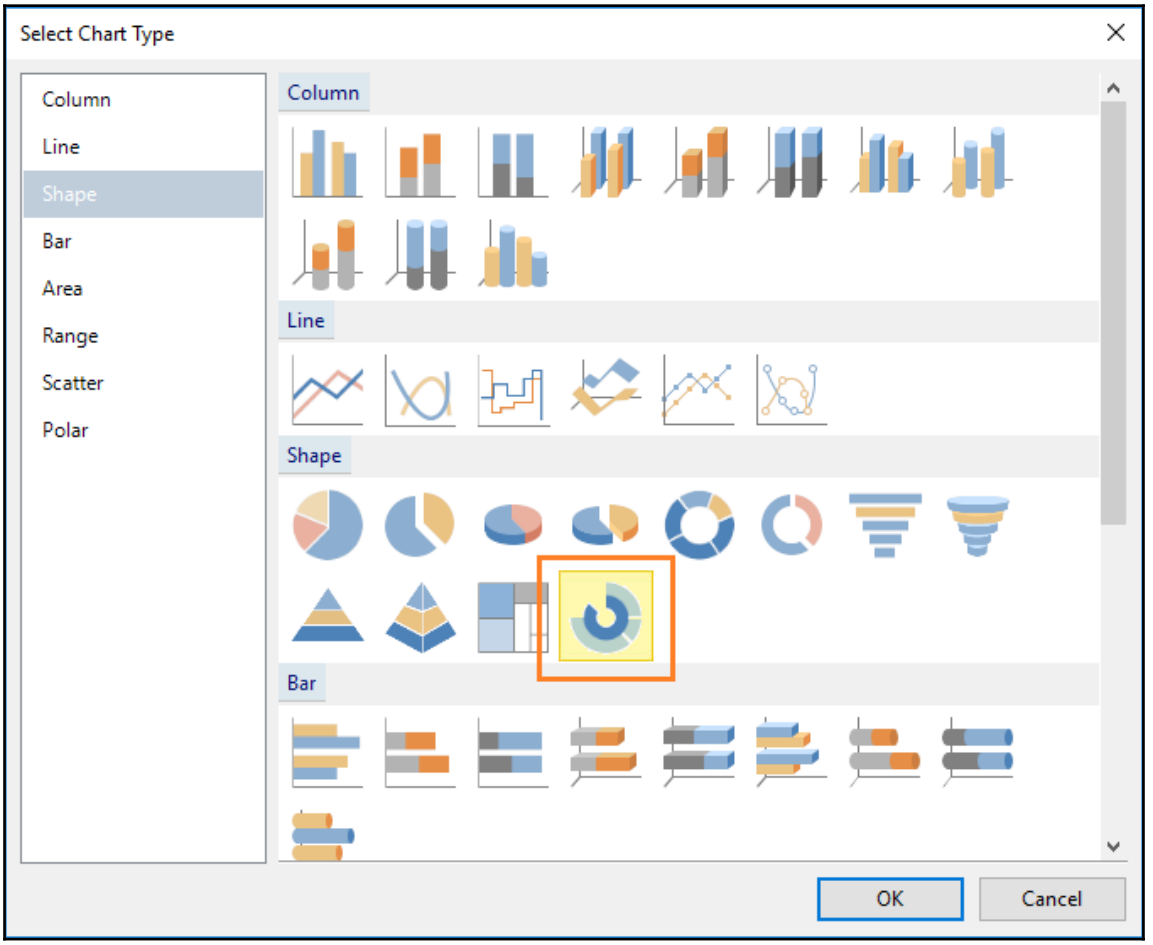


Figure 5.20

2. Change the `title as Sales Orders by Year by Package by Sales Territory (US$ Thousands):`

3. Click on the **Chart Area** and open the **Chart Data** pane. Add the following fields to each section:
    - **Values**: **SUM(Total_Amount)**
    - **Category Groups**: **Buying_Package**, **Sales_Territory**
    - **Series Groups**: **Calendar_Year**

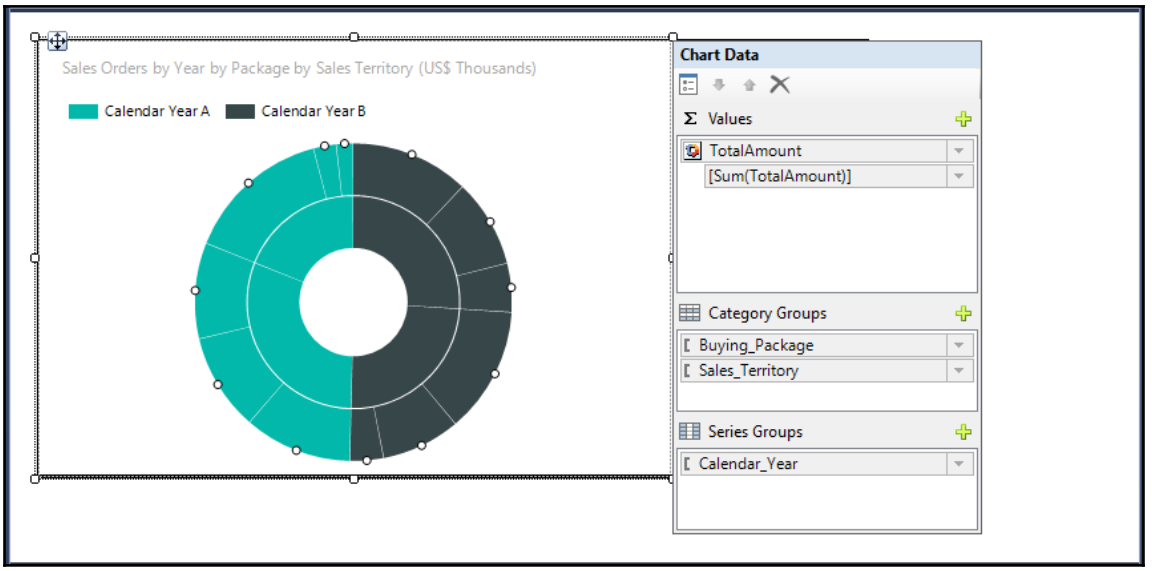


Figure 5.21

4. In the **Category Groups** section, click on the arrow next to the **Buying_Package** category group and click on **Category Group Properties…:**

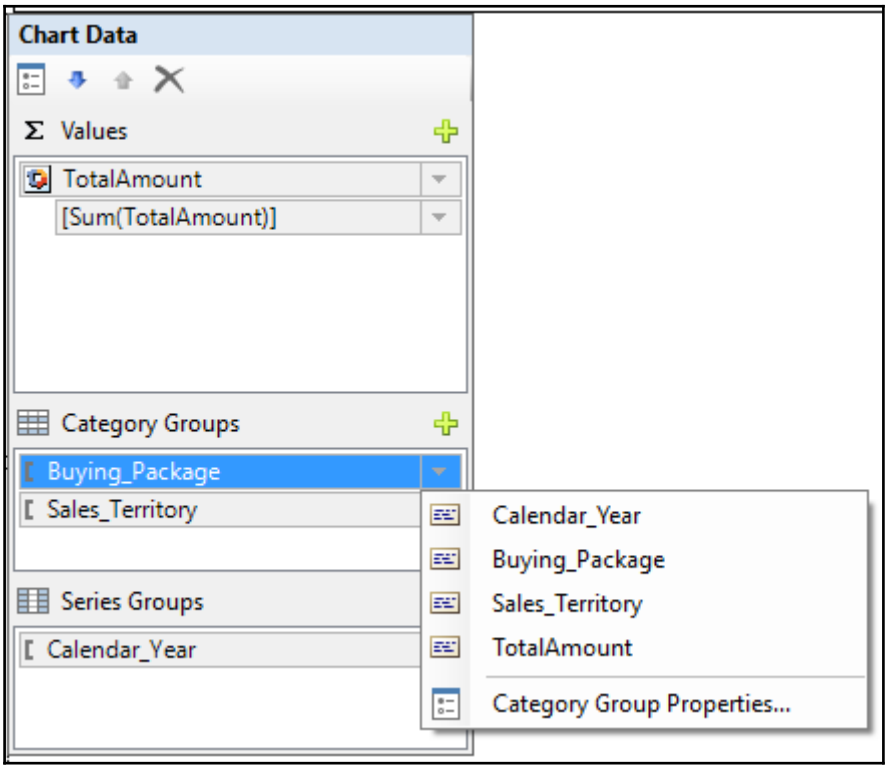


Figure 5.22

5. Set the following expression to **Label** property under **General** section:

```
Fields!Buying_Package.Value &VBCRLF &
Cstr(Format(Fields!TotalAmount.Value, "C0"))
```

6. Right-click onSeries in **Chart Area** and select **Show Data Labels**:

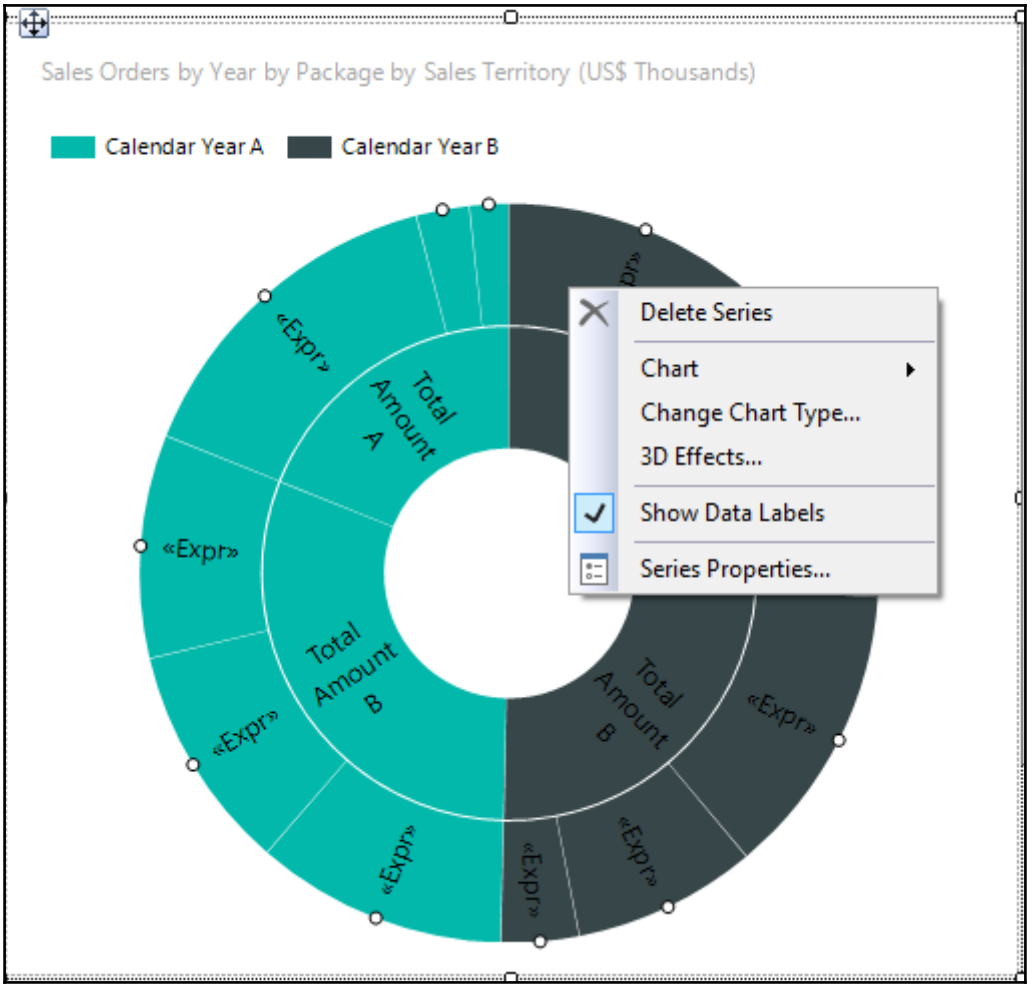


Figure 5.23

7. Right-click on **<<Expr>>** in Chart Area and select **Series Label Properties…**:

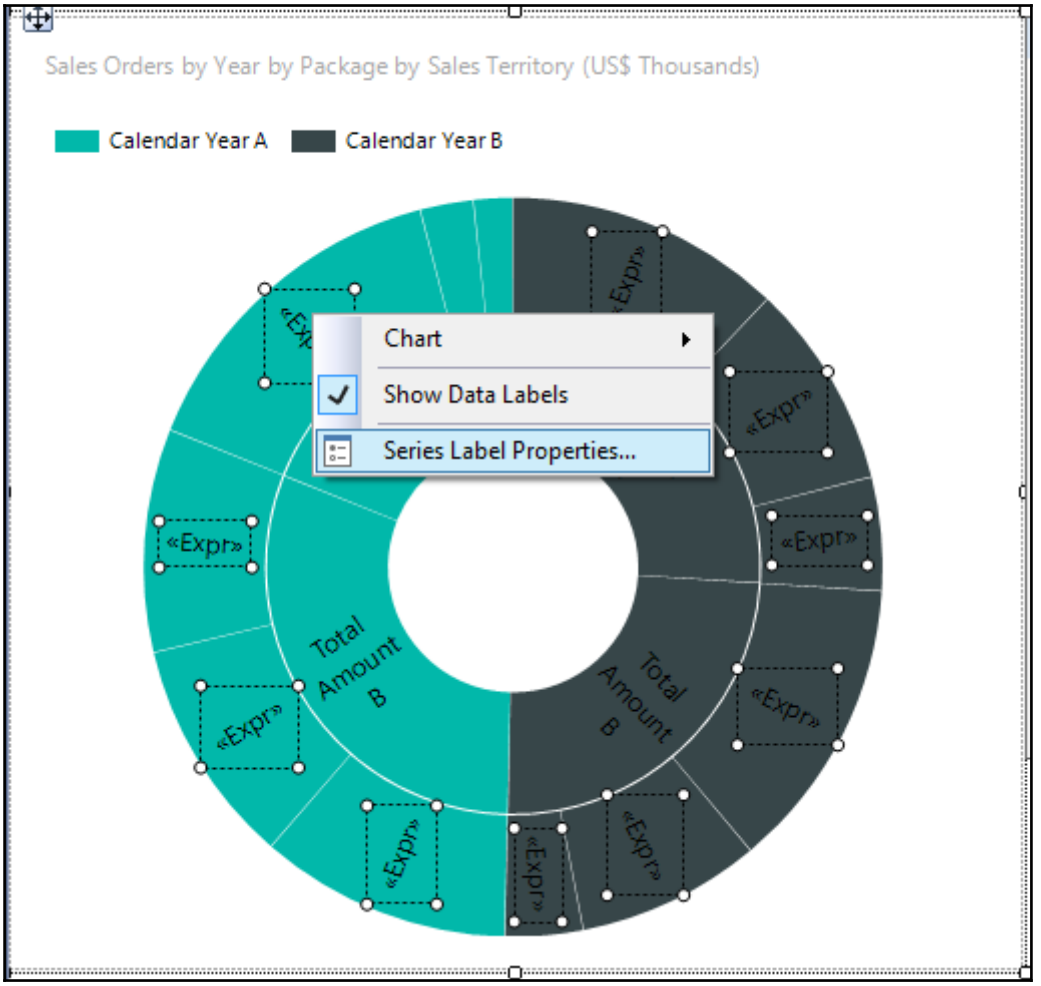


Figure 5.24

8. Set the following expression to the **Label data** property under **General** section:

```
Cstr(Fields!Sales_Territory.Value) &VBCRLF &
Cstr(Format(Fields!TotalAmount.Value/1000, "C0"))
```

9. Once the expression is set, you will get a message as shown here:

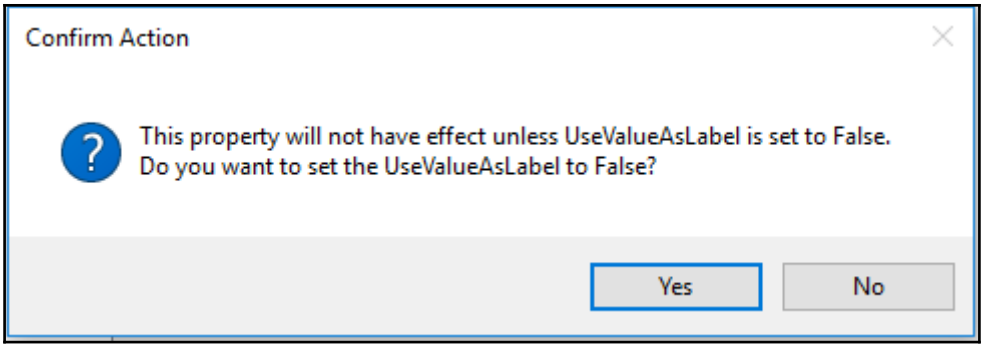


Figure 5.25

10. Click on **Yes** and continue. This is required as we set values through expressions.
11. Everything is set. Preview and see how it shows rings for hierarchy levels. If not all the labels are visible, increase the size of the chart.

How it works…

As you can see, **Calendar Year** has been taken as the **Series** for the chart. The hierarchy has been built on **Package Type** and **Stock Item**, and first ring is set to **Package Type** and second the outer one is set for **Stock Item**.

Although you can set many levels with the hierarchy and show the data set with the Sunburst data region, it is better to avoid many levels as it adds many rings and makes the chart unreadable.

There's more…

The hierarchy need not always be a balanced hierarchy. It supports a ragged hierarchy as well. If you add a data set with a ragged hierarchy, then the chart will be displayed as figure 5.26:

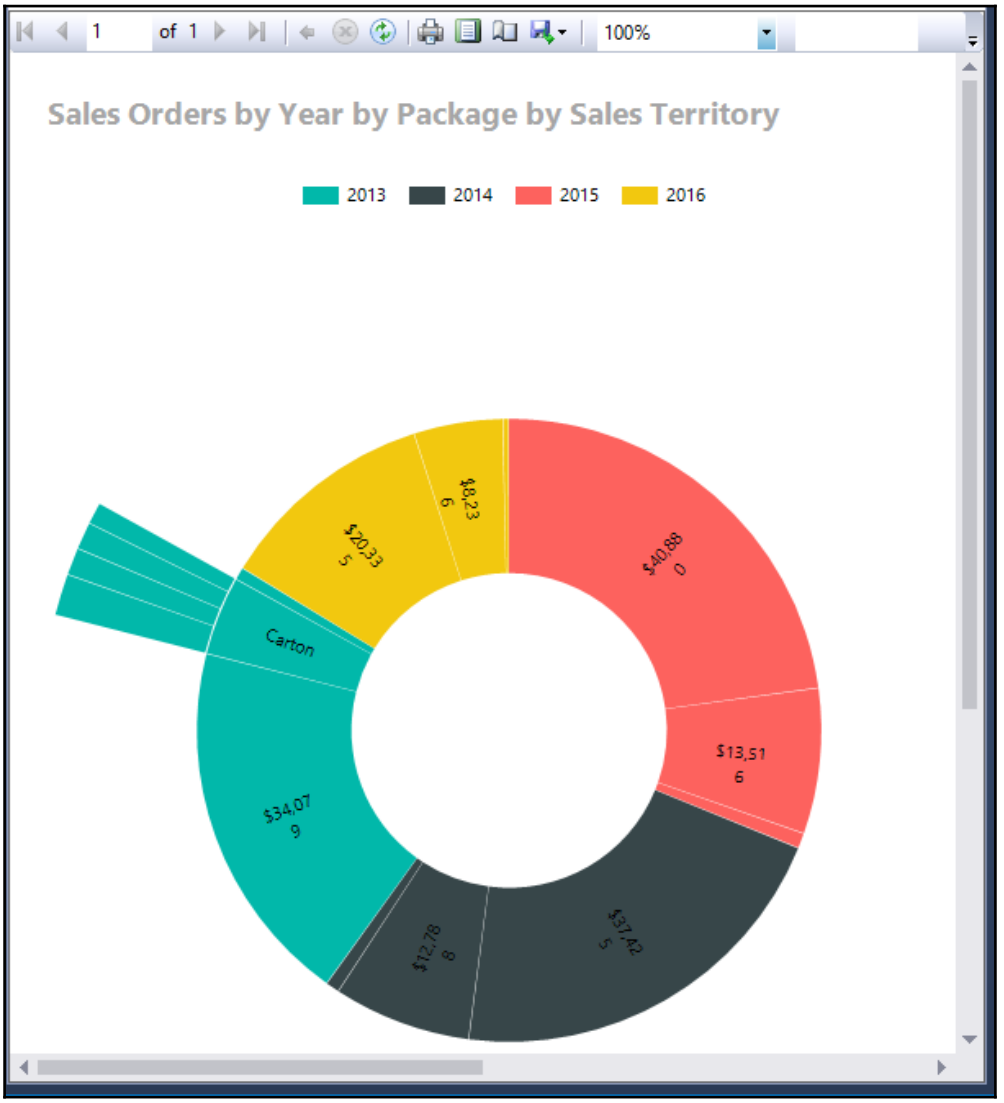


Figure 5.26

Integrating R with Reporting Services

Without a doubt, R is the most popular language among data statisticians and data miners for statistical computing and graphics. It is an open-source language and it provides highly extensible statistical and graphical techniques such as time series analysis, classifications and clustering for data analysis. R is not a new language; its history goes back to 1992. However, the beta version was released in 2000 but R became more popular with a high demand in recent years.

Realizing the importance of R, Microsoft introduced SQL Server R Services with SQL Server 2016. It extends the SQL Server platform, integrating R language with SQL Server, allowing us to analyse data in the database using richest and powerful R language without taking data outside the database. This integration makes it possible to use TSQL and call the R runtime using packages created by the community for data analysis, prediction and for making visuals.

We are talking about Reporting Services and report authoring with this book. You might be thinking the reason for having a recipe on R with this book. The main reason is data analysis and visualization. Visualization is the richest way of presenting data for understanding the distribution of the data and outliers. Since R provides many packages for visualizing data using visuals such as histograms, scatter plots and data exploration graphs, we should see how such visuals can be rendered with Reporting Services reports. Although there are no data regions given yet for these visuals, we can use SQL Server R services for getting R queries executed and send the output to Reporting Services reports.

Let's prepare the environment for R before integrating R language with Reporting Services.

Getting ready

The installation of SQL Server R Services comes with the standard SQL Server 2016 setup. During the SQL Server installation, you can see **R Services (In-Database)** in the **Feature Selection** for installing R Services. This selection makes sure that you have **Advanced Analytics Extensions** and extensions that support external scripts and processes installed:

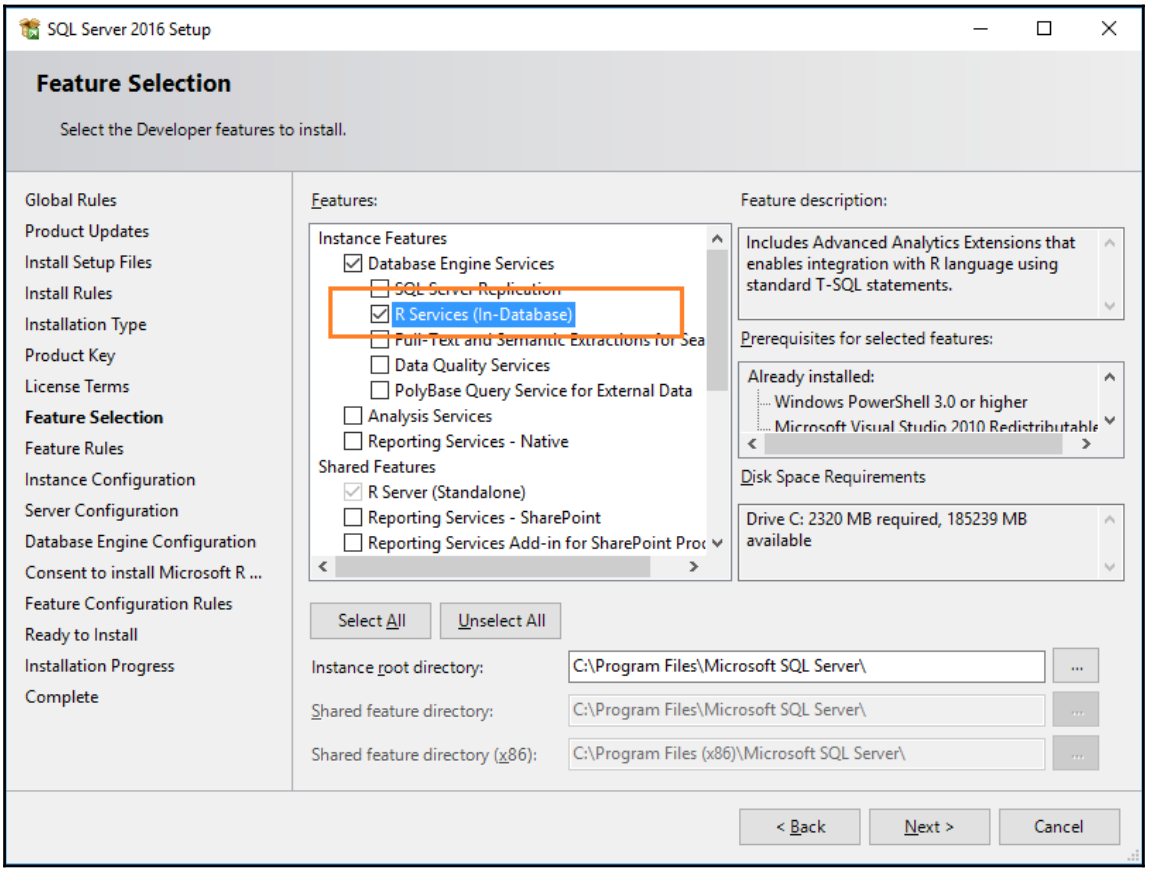


Figure 5.27

Selecting this option prompts an agreement with the page **Consent to install Microsoft R Open.** This needs to be accepted in order to continue with the installation.

Once the installation is done, you need to enable `external scripts` using `sp_configure`. You will not be able to execute R codes without enabling it. Use the following code for enabling `external scripts`.

```
Exec sp_configure  'external scripts enabled', 1
Reconfigure  with  override
```

For more information on this, refer to the *See also* section in this recipe.

How to do it…

We need a good data set for testing data analysis using R and visualizing them, hence we will be using the data set published by Microsoft, specifically on SQL Server R Services. There is a good tutorial for creating a sample database and adding required data and procedures, which we will be using the same for testing R integration with Reporting Services. Since the tutorial is lengthy, let's take what we want only and use it with a report.

Here are steps for creating the database and adding procedures:

1. Go to `https://msdn.microsoft.com/en-us/library/mt683480.aspx`for opening *In-Database Advanced Analytics for SQL Developers (Tutorial)* page.
2. Click on *Step 1: Download the sample data* to open the next page.
3. You will see a PowerShell code given with the page. You may encounter some security issues if you try to execute the given code without enabling script execution, hence it needs a slight modification.

4. Open PowerShell Command Console with the option, Run as Administrator and run the modified script given here:

```
Set-ExecutionPolicy RemoteSigned
$source = 'https://raw.githubusercontent.com/Azure/Azure
MachineLearning-DataScience/master/Misc/
RSQL/Download_Scripts_SQL_Walkthrough.ps1'
$ps1_dest = "$pwd\Download_Scripts_SQL_Walkthrough.ps1"
$wc = New-Object System.Net.WebClient
$wc.DownloadFile($source, $ps1_dest)
.\Download_Scripts_SQL_Walkthrough.ps1 -DestDir 'C:\tempRSQL'
```

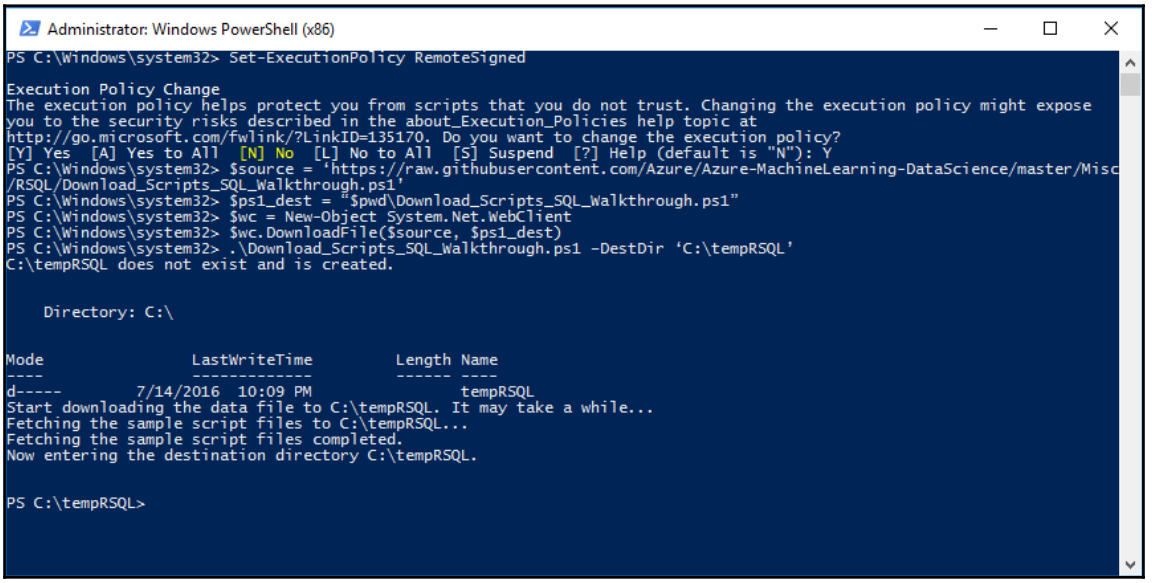


Figure 5.28

5. Once the files are downloaded, you need to execute the PowerShell file given for adding the database and procedures.

6. Execute the following code in PowerShell Command Console:

```
.\RunSQL_SQL_Walkthrough.ps1
```

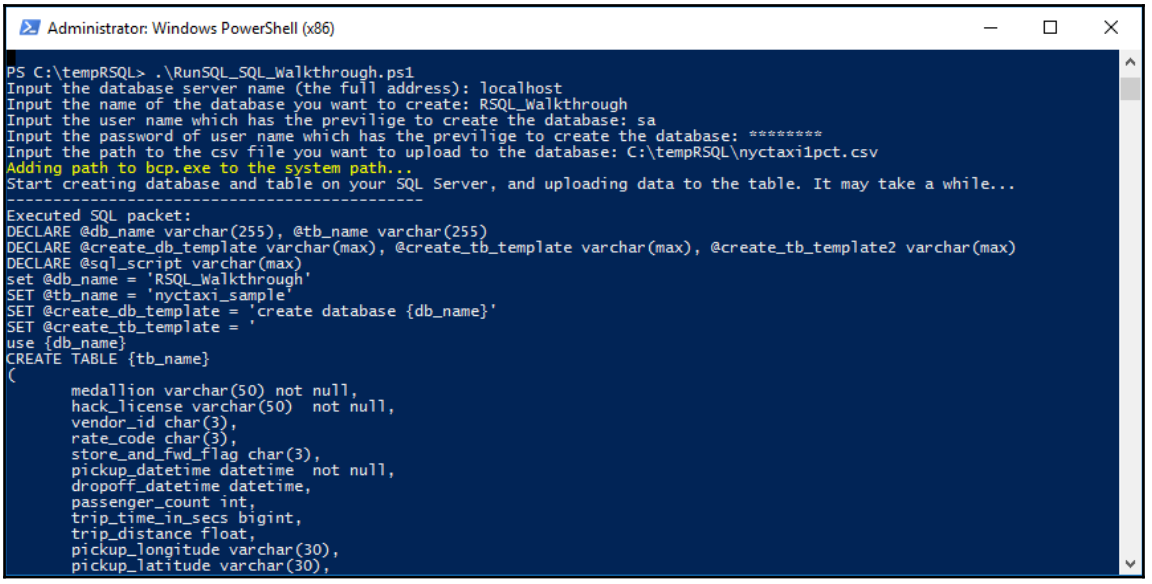


Figure 5.29

7. You will be prompted for **SQL Server Instance Name**, **Database Name** to be created, **User Id** and **Password** and the data file. Set the database name as `RSQL_Walkthrough` and use `nyctaxi1pct.csv` as the data file name.
8. Once the execution is complete, navigate through **Object Explorer** and see whether the database is added. We will be using data in the `nyctaxi_sample` table and the `PlotHistogram` stored procedure.

Let's try to understand the data in the table. This `nyctaxi_sample` table contains a public NYC Taxi data set. Each record includes pickup and drop-off location, time, the trip distance, payment type, payment, tipped, and tip amount. We can do various analysis such as regression analysis or classifications on these columns and visualize the result using R. The procedure `PlotHistogram` contains R code wrapped with TSQL for plotting a histogram on the tipped column, and performing classification on tipped column, and it returns the output as an image. We can call this stored procedure for making a data set for the report and consume the output of the procedure for showing the histogram. The report `Report04 - R-Integration` given with `Chapter05 solution` executes this stored procedure and shows the histogram. This is how it displays:

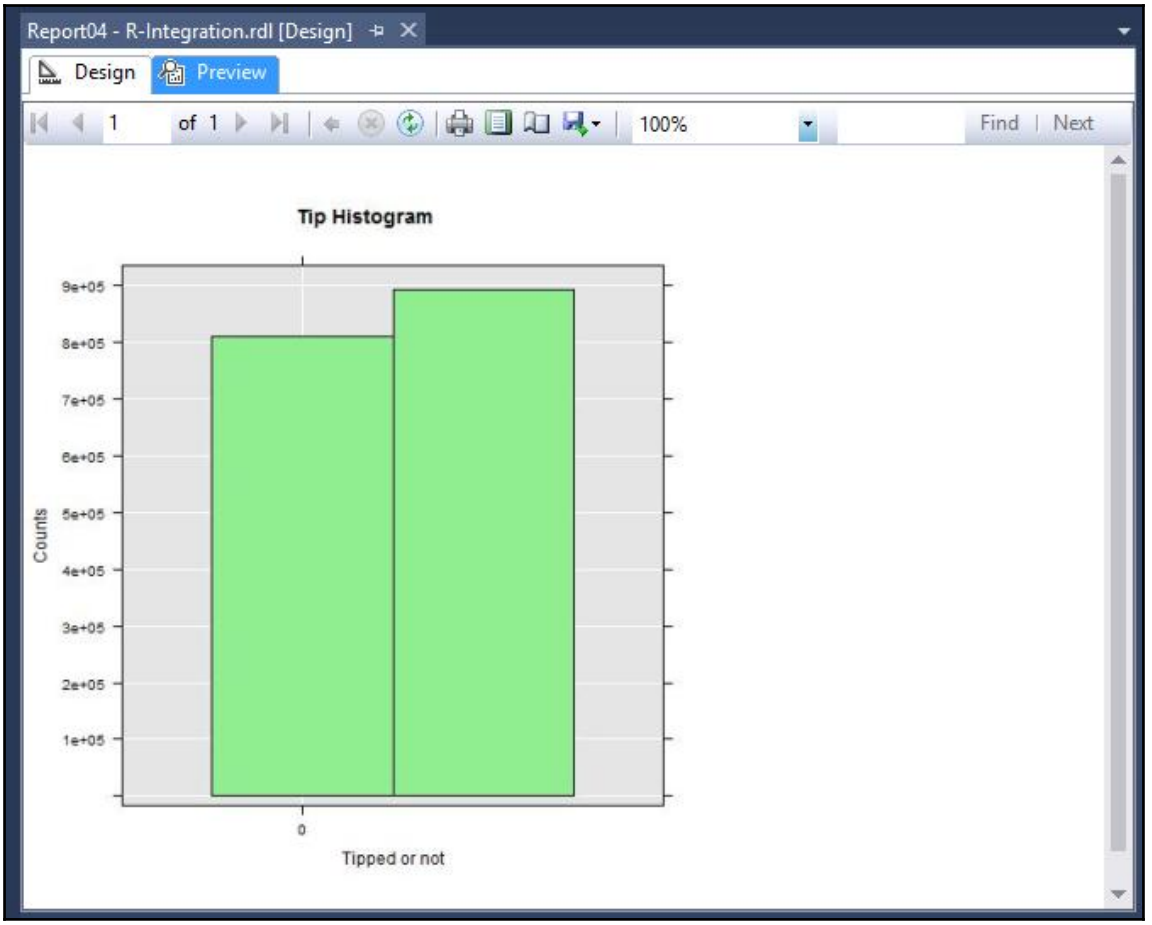


Figure 5.30

Let's see the steps for making a new report and showing the output of R with it.

1. Open your Reporting Services solution and create a new report called Tip Histogram.
2. Create a Data Source named `DataSourceRSQL_Walkthrough` that connects with the `RSQL_Walkthrough` database.
3. Create a Dataset named `DataSetTip`. Connect it to `DataSourceRSQL_Walkthrough` data source and select **Query type** as **Stored Procedure** (see the following image).
4. Select PlotHistogram in the **Select or enter stored procedure name** dropdown (see the following image):

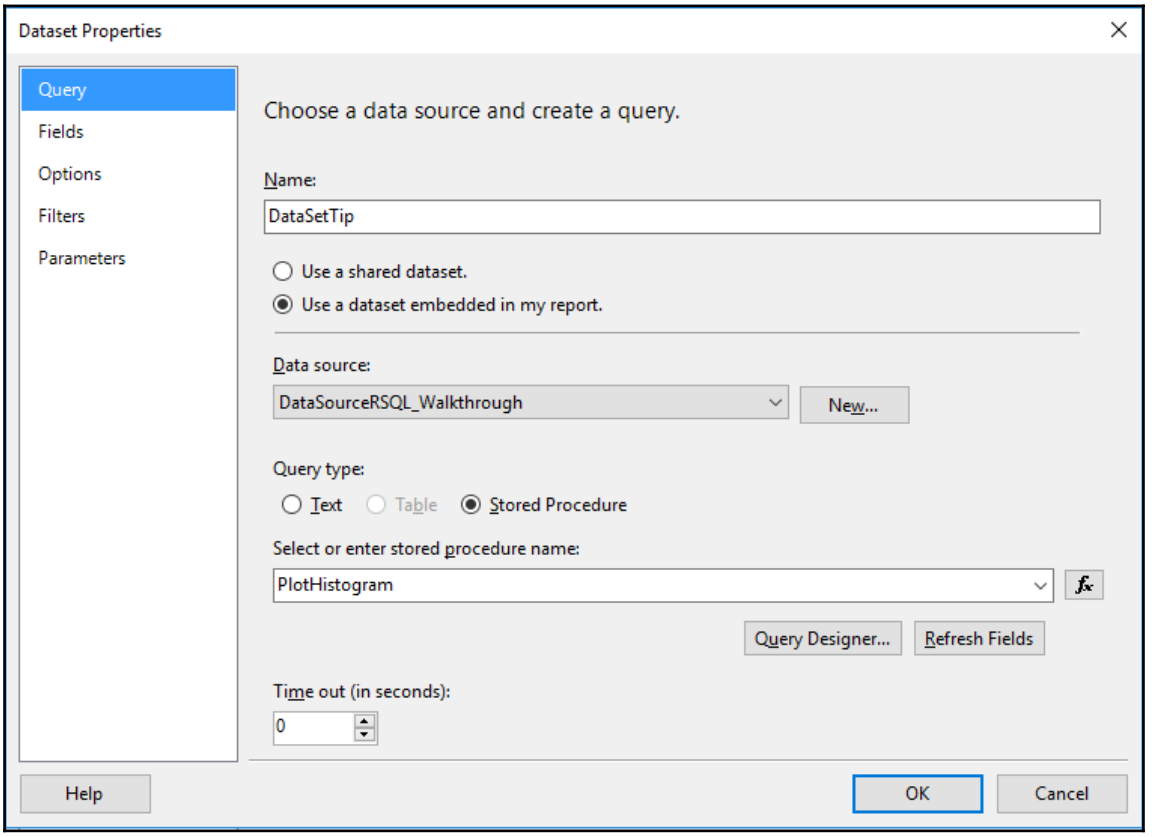


Figure 5.31

5. Go to the **Fields** page and make sure that `plot` **Field Name** is taken (see Figure 5.32):

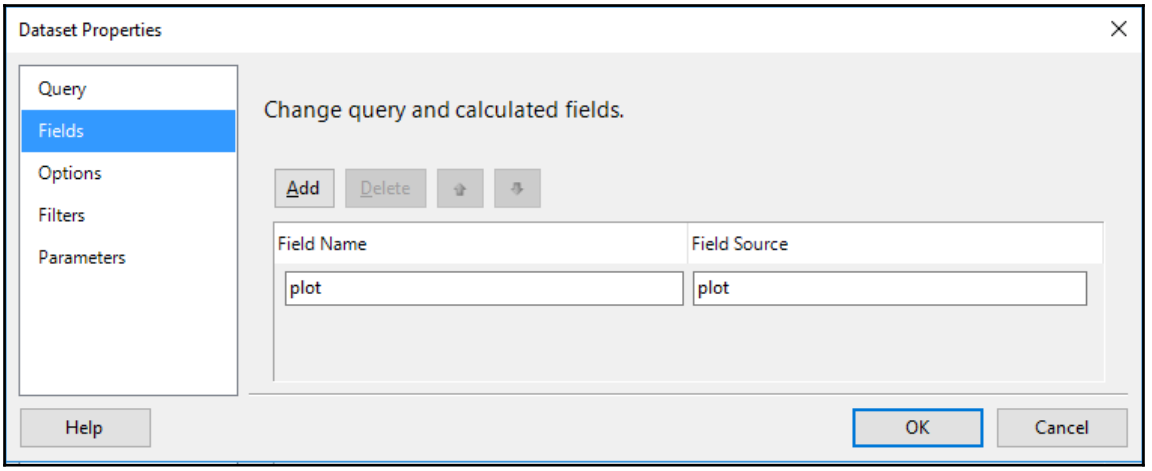


Figure 5.32

6. Open the toolbox and double-click on **Image** data region for opening **Image Properties** and adding an image to the report.
7. Select **Database** in **Select the image source** dropdown (see Figure 5.33).
8. Select **Plot** data field in **Use this field** drop down (see Figure 5.33).

9. Set **Use this MIME type** as **image/jpeg** as shown in the following figure:

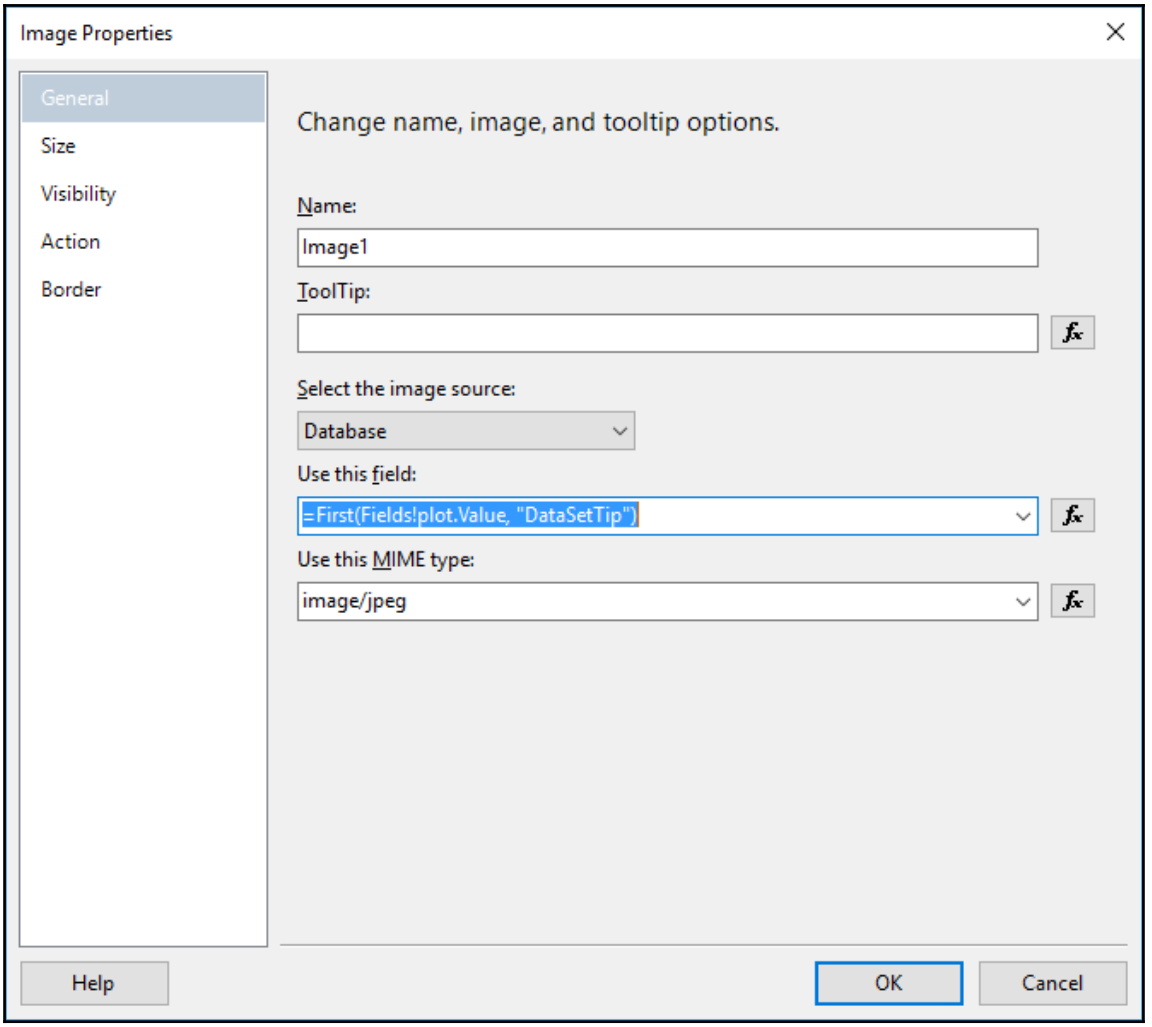


Figure 5.33

10. Click **OK** for saving everything and preview the report. You will see a similar preview just like the preview of `Report04 - R-Integration` report, and a histogram which is the output of R code wrapped with TSQL stored procedure.

How it works…

Although Reporting Services does not execute any R code, SQL Server R Services makes it possible to use implemented R codes. Put simply, code gets executed with the database engine and the report picks and displays the image generated by R.

There's more…

What if we can make R visualizations available as a data region in reporting solutions without using SQL Server R Services? There is a project called **R Graphics Device for Microsoft Reporting Services** (**SSRS**) in CodePlex that makes it possible.

This project has been created by Timo Klimmer, Microsoft. However, this is not an official product of Microsoft, hence there is no support on this. This project creates a Custom Report Item for Reporting Services that renders the graphical output of R code in Reporting Services. This has been created and tested for all SQL Server versions starting from 2005, except 2016 (at the time of writing this book, there is no indication on extending this for 2016). Check the following URL; new versions might be available for supporting Reporting Services 2016.

Here is the URL for **R Graphics Device for Microsoft Reporting Services** (**SSRS**): `https://rgraphicsdevice.codeplex.com/`.

See also

For more information on R Services (In-Database), refer to `https://msdn.microsoft.com/en-us/library/mt696069.aspx`.

6
Authoring Reports with the Mobile Report Publisher

In this chapter, we'll dive into the Mobile Report Publisher, a brand new addition to SSRS introduced in SQL Server 2016. In this chapter, we'll cover the following recipes:

- Understanding the new Mobile Report Publisher
- How to create a mobile report
- Using simulated data
- Setting up data sources
- How to use datasets for mobile reporting
- Understanding mobile report graphical elements
- Understanding the mobile report layout
- Configuring navigators
- Configuring parameters for drill-through reports

Introduction

Over the past few years, Microsoft has been making commitments to multiple platforms, including those not created by them. They have continued that commitment with SQL Server 2016 Reporting Services and the Mobile Report Publisher.

In the past, Reporting Services has largely been useful in creating paginated reports, that is, reports that are designed for the printed page. While it could be done, it was difficult to create reports that had what is known as a **dashboard** look and feel.

Dashboards are typically pages with summary information, with charts that show trends such as sales year by year, gauges that give a current value contrasted with a total, and maps that show sales by area. These are common in dashboards.

Because dashboards are often referenced by *on the go* professionals, they are frequently viewed on mobile devices such as phones and tablets, often with non-Microsoft operating systems.

With the Mobile Report Publisher, Microsoft solved many challenges when it came to mobile reporting. From the perspective of the report designer, the Mobile Report Publisher is an easy-to-use interface that allows the designer to lay out reports for a variety of screen sizes.

For the end user, Microsoft makes reports available through its PowerBI app, available on multiple platforms, such as Apple's iOS and Google's Android OS. Of course, these reports are also viewable through any browser that supports HTML version 5.

In this chapter, we'll look at creating mobile reports and connecting them to data.

Understanding the new Mobile Report Publisher

In this section, we'll take a brief look at the new Mobile Report Publisher. We'll identify the various components of the tool and what each is used for.

Getting ready

Before you can use the Mobile Report Publisher, you'll need to install it. You can do so from the Report Portal. In the Report Portal header, simply click on the down arrow on the upper right. From the menu, select **Mobile Report Publisher**; it will open up a web page from which you can download and install the **Mobile Report Publisher**:

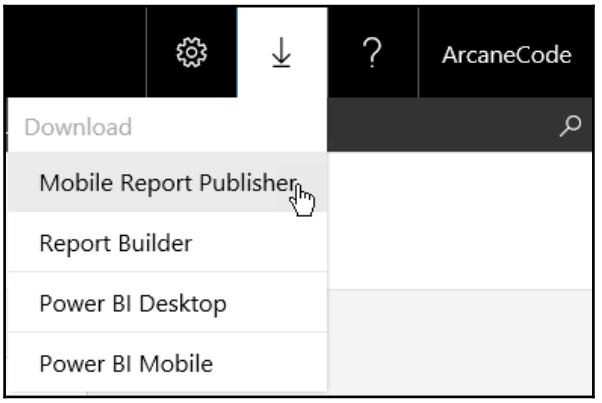


Figure 6.01

How to do it…

1. After installation, launch the **Mobile Report Publisher**. When it launches, you will see this image:

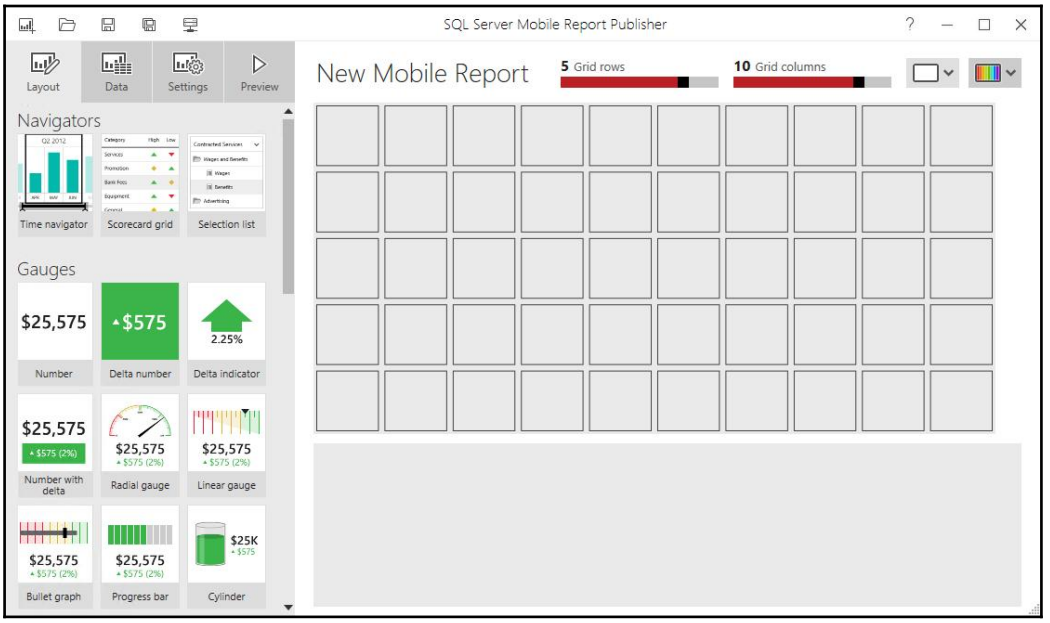


Figure 6.02

2. On the upper left are four tabs: **Layout**, **Data**, **Settings**, and **Preview**. Clicking on each one will populate the tool panel on the left, under the tabs, with the appropriate tools for the area:



Figure 6.03

3. While on the Layout tab, scroll down. You will see other visual tools available, such as **Charts**, **Maps**, and **Data grids**:

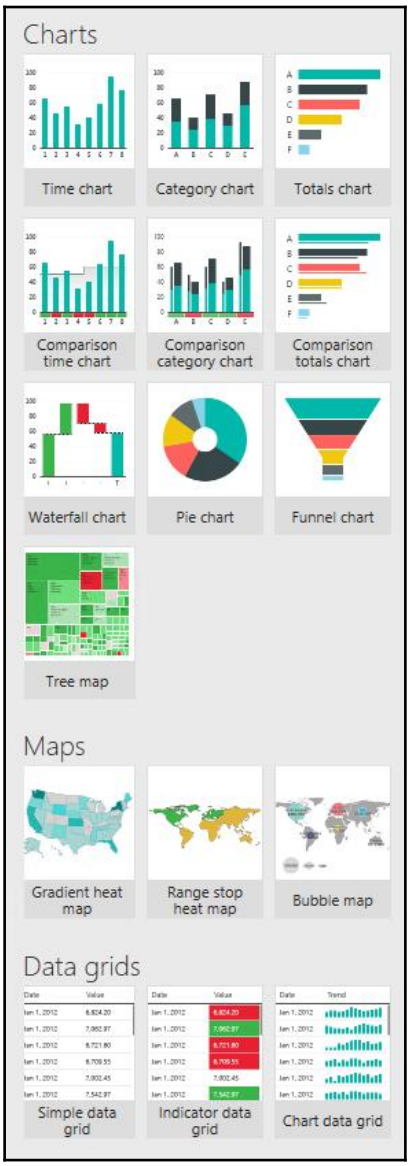


Figure 6.04

4. The center area, with the rows and columns of boxes, is the *design surface*. You'll see how to interact with it in the next recipe.
5. Above the design area are controls that allow you to control the look and feel of the report. The title of the report comes first, and is editable by simply clicking in it. Next are controls to set the number of rows and columns in the grid. The white button controls the layout, and the colored button lets you select a color palette for the report:



Figure 6.05

6. Finally, look at the small set of icons at the very top left of the interface. These are the traditional file commands:



Figure 6.06

7. Hovering your mouse over each one, you should see popups for each command. From left to right, you should see **New**, **Open**, **Save**, **Save As**, and **Server connections**.

How it works…

The new Mobile Report Publisher was designed to make setting up dashboards on multiple platforms as simple and painless as possible. From the toolbox in the Layout area are a wide variety of visual components you can simply drag and drop onto the design surface. Once there, you can resize them to take up the number of boxes appropriate for the content.

In a later recipe, you'll see how to use the Data tab to associate your data sources with each visual component.

How to create a mobile report

In this section, we'll begin the process of creating a mobile report by placing the visual elements we want onto the designer. This is a great feature of the Mobile Report Publisher, as it lets you work with the end user to design the report before taking the more involved step of accessing the required data.

Getting ready

Simply open the **Mobile Report Publisher**. If you already have it open, ensure that it is ready by using the New button in the toolbar in the very upper left.

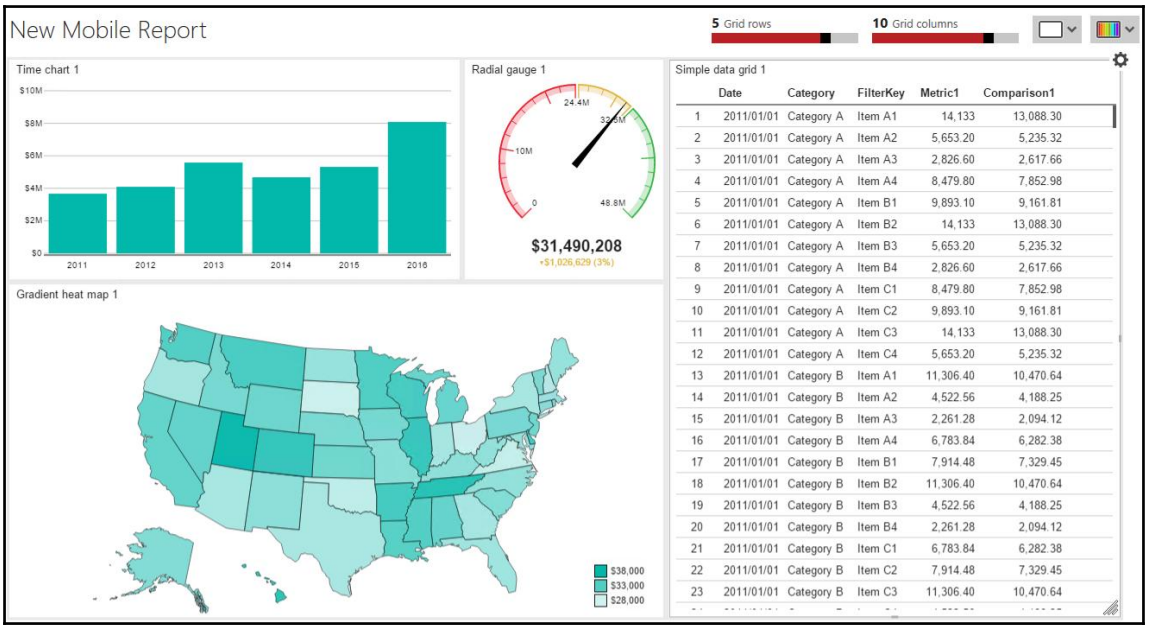
How to do it…

1. On the Layout tab, drag and drop the **Time chart** control into the very upper left block in the designer. It will be auto-renamed to **Time chart 1**.
2. In the bottom-right corner, you will see a sizing handle:



Figure 6.07

3. Drag the handle so that the chart is four blocks wide and two blocks high.
4. In the Gauges area, find **Radial gauge** and drop it beside the time chart. Expand so that it takes up an area two blocks wide and two blocks high.
5. Scroll down to the Maps area and place **Gradient heat map** directly underneath the time chart. Expand the map so that it is six blocks wide by three blocks tall.
6. For the last component, scroll down to the Data grids in the Layout pane and place a **Simple data grid** component beside the radial gauge. Expand it to take up the remaining space, four blocks wide by five blocks high.
7. When done, your designer should resemble this image:



	Date	Category	FilterKey	Metric1	Comparison1
1	2011/01/01	Category A	Item A1	14,133	13,088.30
2	2011/01/01	Category A	Item A2	5,653.20	5,235.32
3	2011/01/01	Category A	Item A3	2,826.60	2,617.66
4	2011/01/01	Category A	Item A4	8,479.80	7,852.98
5	2011/01/01	Category A	Item B1	9,893.10	9,161.81
6	2011/01/01	Category A	Item B2	14,133	13,088.30
7	2011/01/01	Category A	Item B3	5,653.20	5,235.32
8	2011/01/01	Category A	Item B4	2,826.60	2,617.66
9	2011/01/01	Category A	Item C1	8,479.80	7,852.98
10	2011/01/01	Category A	Item C2	9,893.10	9,161.81
11	2011/01/01	Category A	Item C3	14,133	13,088.30
12	2011/01/01	Category A	Item C4	5,653.20	5,235.32
13	2011/01/01	Category B	Item A1	11,306.40	10,470.64
14	2011/01/01	Category B	Item A2	4,522.56	4,188.25
15	2011/01/01	Category B	Item A3	2,261.28	2,094.12
16	2011/01/01	Category B	Item A4	6,783.84	6,282.38
17	2011/01/01	Category B	Item B1	7,914.48	7,329.45
18	2011/01/01	Category B	Item B2	11,306.40	10,470.64
19	2011/01/01	Category B	Item B3	4,522.56	4,188.25
20	2011/01/01	Category B	Item B4	2,261.28	2,094.12
21	2011/01/01	Category B	Item C1	6,783.84	6,282.38
22	2011/01/01	Category B	Item C2	7,914.48	7,329.45
23	2011/01/01	Category B	Item C3	11,306.40	10,470.64

Figure 6.08

How it works…

The visual designer in Mobile Report Builder makes it very easy to prototype your dashboards. You can sit with your end users and arrange components on the screen until the report meets the end users' requirements.

Once on the designer, components can be moved by simply dragging to a new location, or resized using the sizing handles in the bottom-right corner.

You may have noticed that each component comes with data prepopulated in order to make visualizing the end result easier. This is known as *simulated data* and is the subject of the next recipe.

Using simulated data

When designing a dashboard, it can often be difficult to visualize the end result. Placing an empty gauge or report makes it hard for some people to get an idea of the final version of the report.

To assist with this issue, each component includes a set of *simulated data*. This is a set of fictional data the component can use when rendering the report before real data is connected to your dashboard.

Getting ready

This recipe will pick up right where the previous one ended, so be sure you have completed the steps in *How to create a mobile report* recipe first.

How to do it…

1. Click on the **Preview** button in the tab area. You'll see that the report is rendered, complete with data:

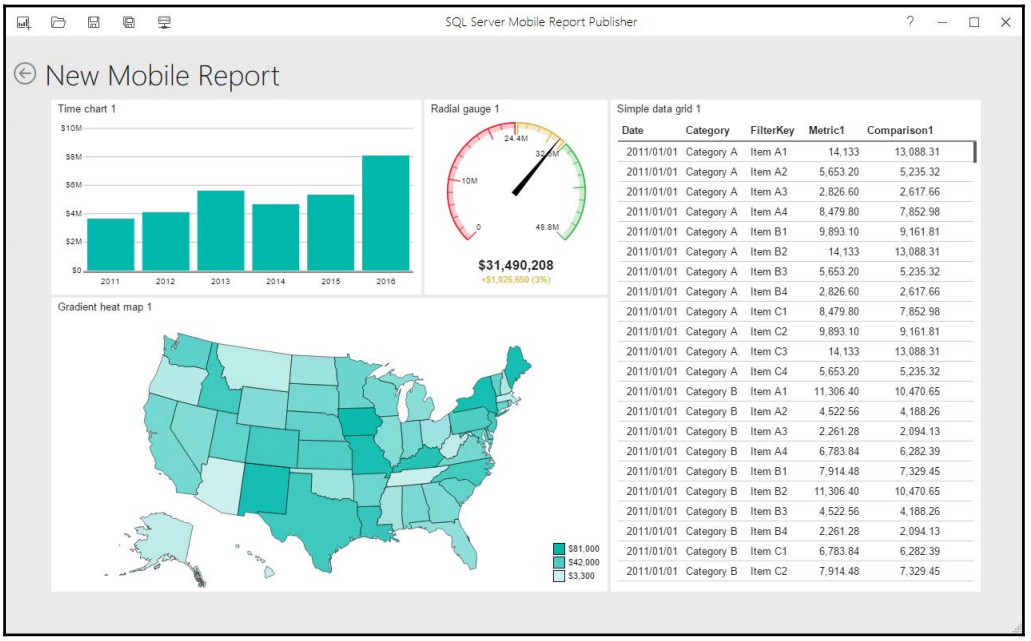


Figure 6.09

2. To see where this data comes from, first click on the left arrow directly next to the report title, in this example, **New Mobile Report**.
3. Click on the **Data** tab.

4. In the data area, you see two tabs; the first is for **SimulatedTable:**

	Date	Category	FilterKey	Metric1	Metric2	Metric3
1	1/1/2011 12:00:00 AM	Category A	Item A1	14,133.00	6,377.16	12,804.47
2	1/1/2011 12:00:00 AM	Category A	Item A2	5,653.20	2,329.46	4,894.45
3	1/1/2011 12:00:00 AM	Category A	Item A3	2,826.60	1,223.03	1,940.91
4	1/1/2011 12:00:00 AM	Category A	Item A4	8,479.80	3,135.37	6,477.25
5	1/1/2011 12:00:00 AM	Category A	Item B1	9,893.10	3,852.96	7,715.18
6	1/1/2011 12:00:00 AM	Category A	Item B2	14,133.00	5,886.83	12,370.40
7	1/1/2011 12:00:00 AM	Category A	Item B3	5,653.20	2,042.10	3,907.87
8	1/1/2011 12:00:00 AM	Category A	Item B4	2,826.60	1,299.39	2,584.27
9	1/1/2011 12:00:00 AM	Category A	Item C1	8,479.80	3,183.46	6,305.93
10	1/1/2011 12:00:00 AM	Category A	Item C2	9,893.10	3,829.38	9,080.73
11	1/1/2011 12:00:00 AM	Category A	Item C3	14,133.00	6,201.41	9,725.81
12	1/1/2011 12:00:00 AM	Category A	Item C4	5,653.20	2,056.76	4,936.95
13	1/1/2011 12:00:00 AM	Category B	Item A1	11,306.40	4,505.74	9,868.95
14	1/1/2011 12:00:00 AM	Category B	Item A2	4,522.56	1,888.65	3,384.10
15	1/1/2011 12:00:00 AM	Category B	Item A3	2,261.28	788.02	2,047.88

SimulatedTable SimulatedMapUSA

Figure 6.10

5. Click on the second tab, **SimulatedMapUSA**:

	Date	MapKey	Metric	Comparison
1	1/1/2011 12:00:00 AM	Alabama	614.41	831.29
2	1/1/2011 12:00:00 AM	Alaska	180.62	228.36
3	1/1/2011 12:00:00 AM	Arizona	33.73	27.37
4	1/1/2011 12:00:00 AM	Arkansas	565.04	654.54
5	1/1/2011 12:00:00 AM	California	607.23	346.75
6	1/1/2011 12:00:00 AM	Colorado	621.49	328.12
7	1/1/2011 12:00:00 AM	Connecticut	836.49	920.55
8	1/1/2011 12:00:00 AM	Delaware	834.64	1,194.48
9	1/1/2011 12:00:00 AM	District of Columbia	903.25	562.69
10	1/1/2011 12:00:00 AM	Florida	447.77	235.96
11	1/1/2011 12:00:00 AM	Georgia	426.28	316.76
12	1/1/2011 12:00:00 AM	Hawaii	161.13	141.11
13	1/1/2011 12:00:00 AM	Idaho	553.59	601.60
14	1/1/2011 12:00:00 AM	Illinois	387.46	440.50
15	1/1/2011 12:00:00 AM	Indiana	208.87	286.95

SimulatedTable SimulatedMapUSA

Figure 6.11

6. Underneath the data, you will see the report element and what data it uses:



Figure 6.12

7. On the left side of the **Mobile Report Publisher** are the four report elements used in this dashboard. Click on each one to see the data it uses.

How it works…

Each component has its own set of data; however, multiple elements can share the same simulated data source. In this example, the time chart, radial gauge, and simple data grid all use **SimulatedTable** as their data source. The heat map used a different source, **SimualtedMapUSA**.

If desired, you can select different columns from the sample data to display, but there is no functionality for updating the sample data.

Setting up data sources

While simulated data is nice, you'll obviously want to use your real data in your reports. The Mobile Report Publisher uses datasets stored in the Report Portal to supply data to the reports.

Getting ready

The easiest way to create a dataset is with Report Builder. For a detailed explanation, see the *Working with published datasets* recipe in `Chapter 4`, *Authoring Reports with Report Builder*, to learn how to create them. In this section, we'll supply just the basic steps.

How to do it….

1. Open **Report Builder**.
2. Select **New Dataset**.
3. Select the **WideWorldImportersDW** data source, or browse to the one stored on your report server's data sources you saved previously.
4. Click on the **Create** button.
5. Click on **Edit as Text**.
6. Paste in the following query:

```
SELECT fs.[Sale Key]
     , fs.[Invoice Date Key]
     , fs.[Delivery Date Key]
     , dc.[City]
     , dc.[State Province]
     , ds.[Stock Item]
     , fs.[Description]
     , fs.[Package]
     , fs.[Quantity]
     , fs.[Unit Price]
     , fs.[Tax Rate]
     , fs.[Total Excluding Tax]
     , fs.[Tax Amount]
     , fs.[Profit]
     , fs.[Total Including Tax]
  FROM [Fact].[Sale] fs
  JOIN [Dimension].[City] dc
    ON fs.[City Key] = dc.[City Key]
  JOIN [Dimension].[Stock Item] ds
    ON fs.[Stock Item Key] = ds.[Stock Item Key]
```



This query will bring back nearly 300,000 rows of data. Be aware that to handle a query of this size, the Mobile Report Publisher must have a LOT of memory and CPU space.
If you are running on a lower end computer, you may receive errors such as `System.OutOfMemoryException` or the cryptic `An Error Has Occurred, Please Close The Application And Try Later` (this author was running 16 gig of RAM, four CPUs on a core i7, and still had this error from time to time).
Should you encounter this, simply modify the query to limit the amount of data returned. You could use `a SELECT TOP 50000` at the beginning of the query. Alternately, you can add a where clause, such as `WHERE ds.[Stock Item Key] IN (194, 204, 168, 180, 174, 171, 199).`

7. Click on the red exclamation mark to run the query, to ensure that it works.
8. Use the save icon to save the dataset to the Report Portals, datasets folder, where you saved your other datasets from `Chapter 4`, *Authoring Reports with Report Builder*.
9. Name the dataset `DashboardData`.
10. Create a second dataset, repeating the preceding steps. Use the following query and name it `YearlySalesByState`:

```
SELECT  dc.[State Province]
      , YEAR(fs.[Invoice Date Key]) AS InvoiceYear
      , SUM(fs.[Total Excluding Tax]) AS  SumTotalExcludingTax
  FROM [Fact].[Sale] fs
  JOIN [Dimension].[City] dc
    ON fs.[City Key] = dc.[City Key]
 GROUP BY dc.[State Province], YEAR(fs.[Invoice Date Key])
 ORDER BY dc.[State Province], YEAR(fs.[Invoice Date Key])
```

11. Create a third and final dataset. This one will be named `YTDSales`, and use the following query:

```
SELECT
      (SELECT SUM([Total Excluding Tax])
        FROM [Fact].[Sale]
       WHERE [Invoice Date Key]
        BETWEEN DATEADD(YEAR, DATEDIFF(YEAR,
        '19000101',  GETDATE()), '19000101')
          AND GETDATE()
       ) AS CurrentYTD
    , ( SELECT SUM([Total Excluding Tax])
        FROM [Fact].[Sale]
       WHERE [Invoice Date Key]
        BETWEEN DATEADD(YEAR, DATEDIFF(YEAR,
         '19000101', GETDATE()) - 1 , '19000101')
          AND DATEADD(year, -1, GETDATE())
      ) AS PreviousYTD
```

How it works…

The Mobile Report Publisher can use either Excel files or datasets as sources. In this recipe we focused on the use of datasets as the source, as they will be the most common. Note that in order for the Mobile Report Publisher to use a dataset, it must be stored on the Report Portal.

In order for the Mobile Report Publisher to work across multiple platforms, SSRS must render the report and push the finished result to the end client. Using datasets makes this task much easier. SSRS understands how to read and populate the datasets. In addition, SSRS will allow you to cache datasets, saving them in memory and thus reducing the runtime for frequently used reports.

For our report to work, we will need three datasets to supply the data. The first dataset could be considered a general use dataset and be used in this report as well as many others.

The second dataset creates a summation of sales by state for each year. While the Mobile Report Publisher can do some summations, it currently lacks the ability to sum on a calendar year, hence the need to create a special query to do it for us.

The final query will be used on the radial gauge. The gauge requires a main value and a value to compare to. In this case we used two sub-queries, one for the current year and one for the previous year, and combined them into a single result set.

There are obviously many ways to retrieve this type of data. Ideally it would come from an SQL Server Analysis Services cube, where calculations such as year to date and previous year to date are much easier. To keep the complexity of examples in the book as low as possible, we stuck to a single database and a straightforward query.

How to use datasets for mobile reporting

Now that the datasets are created, it's time to incorporate them into your report. In this recipe, we'll add the datasets and then tie them to the report elements we've placed on the designer.

Getting ready

This recipe builds on the previous ones, so make sure you have completed them before proceeding. Have the Mobile Report Publisher open and set up to look like the end of the previous recipe.

How to do it…

1. Click on the **Data tab**.
2. In the upper right, click on the **Add data** button:



Figure 6.13

3. The **Add data** window appears. Select **Report** server as the source:

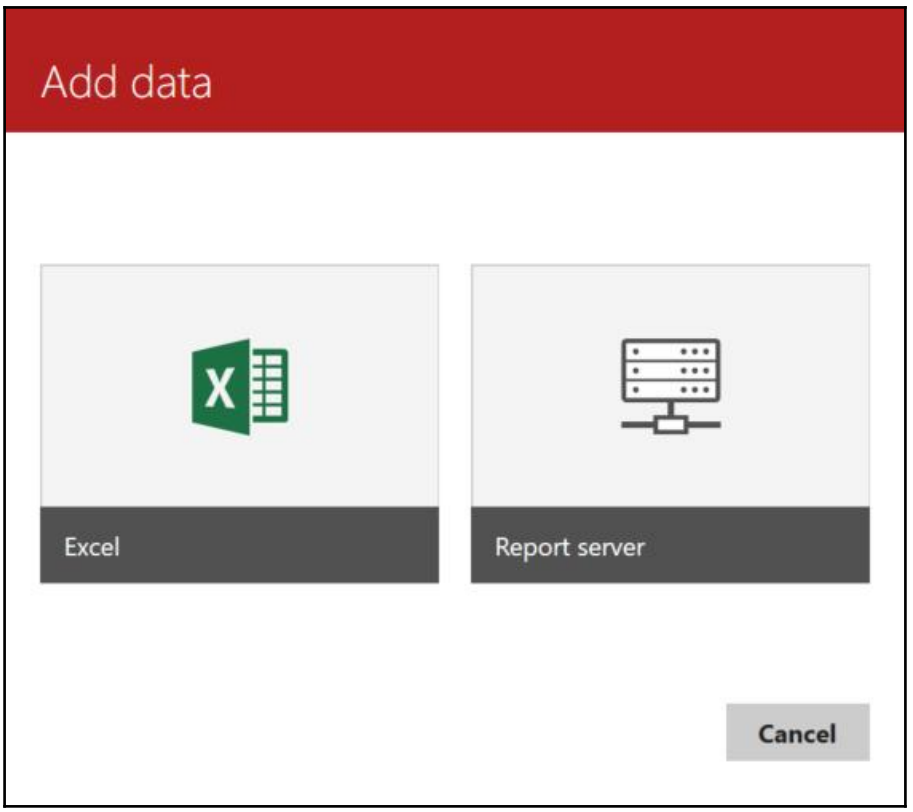


Figure 6.14

4. Select the server from the list of servers you've used before. If the server you want doesn't appear, you can cancel and use the **Servers** button in the upper toolbar (next to the **Save As** button) to add another server reference to the report.
5. Click on the **Datasets** folder to open it.

6. In the Datasets folder, select the **DashboardData** dataset you created earlier in the chapter:

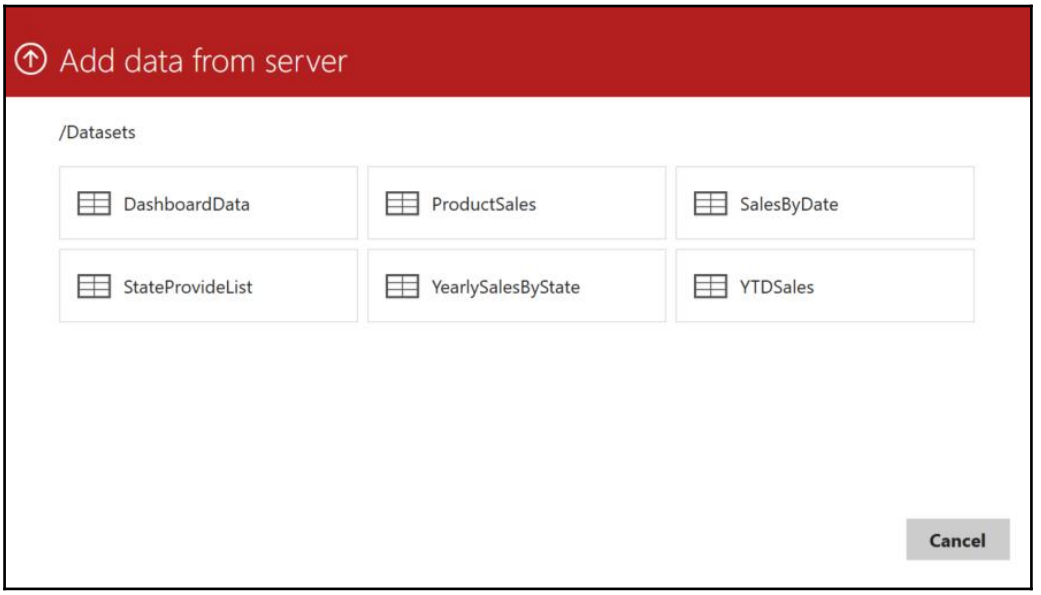


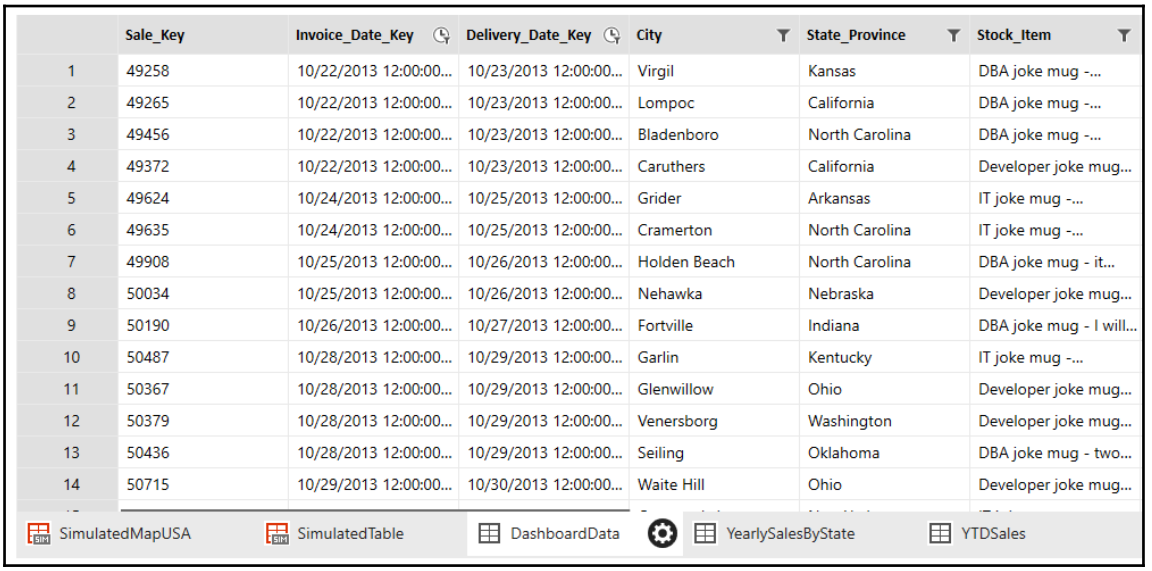
Figure 6.15



Note that you can navigate back up in the folder listing using the arrow button to the left of the **Add data from server** title.

7. Repeat the preceding steps, adding the **YearlySalesByState** and **YTDSales** datasets.

8. You should now see tabs for each of the datasets added. Click on the **DashboardData** tab to see the real data from the dataset:



	Sale_Key	Invoice_Date_Key	Delivery_Date_Key	City	State_Province	Stock_Item
1	49258	10/22/2013 12:00:00...	10/23/2013 12:00:00...	Virgil	Kansas	DBA joke mug -...
2	49265	10/22/2013 12:00:00...	10/23/2013 12:00:00...	Lompoc	California	DBA joke mug -...
3	49456	10/22/2013 12:00:00...	10/23/2013 12:00:00...	Bladenboro	North Carolina	DBA joke mug -...
4	49372	10/22/2013 12:00:00...	10/23/2013 12:00:00...	Caruthers	California	Developer joke mug...
5	49624	10/24/2013 12:00:00...	10/25/2013 12:00:00...	Grider	Arkansas	IT joke mug -...
6	49635	10/24/2013 12:00:00...	10/25/2013 12:00:00...	Cramerton	North Carolina	IT joke mug -...
7	49908	10/25/2013 12:00:00...	10/26/2013 12:00:00...	Holden Beach	North Carolina	DBA joke mug - it...
8	50034	10/25/2013 12:00:00...	10/26/2013 12:00:00...	Nehawka	Nebraska	Developer joke mug...
9	50190	10/26/2013 12:00:00...	10/27/2013 12:00:00...	Fortville	Indiana	DBA joke mug - I will...
10	50487	10/28/2013 12:00:00...	10/29/2013 12:00:00...	Garlin	Kentucky	IT joke mug -...
11	50367	10/28/2013 12:00:00...	10/29/2013 12:00:00...	Glenwillow	Ohio	Developer joke mug...
12	50379	10/28/2013 12:00:00...	10/29/2013 12:00:00...	Venersborg	Washington	Developer joke mug...
13	50436	10/28/2013 12:00:00...	10/29/2013 12:00:00...	Seiling	Oklahoma	DBA joke mug - two...
14	50715	10/29/2013 12:00:00...	10/30/2013 12:00:00...	Waite Hill	Ohio	Developer joke mug...

Figure 6.16

9. To indicate which of the two date columns is the primary one, click on the gear icon to the right of **DashboardData**.
10. It will now display a circular menu. Click on **Dates**.

11. On the **Set date/time settings**screen, change the **Date/time filter column** to **Invoice_Date_Key**. Leave the **Aggregation time unit** set to **Auto**, and click on **Apply**:

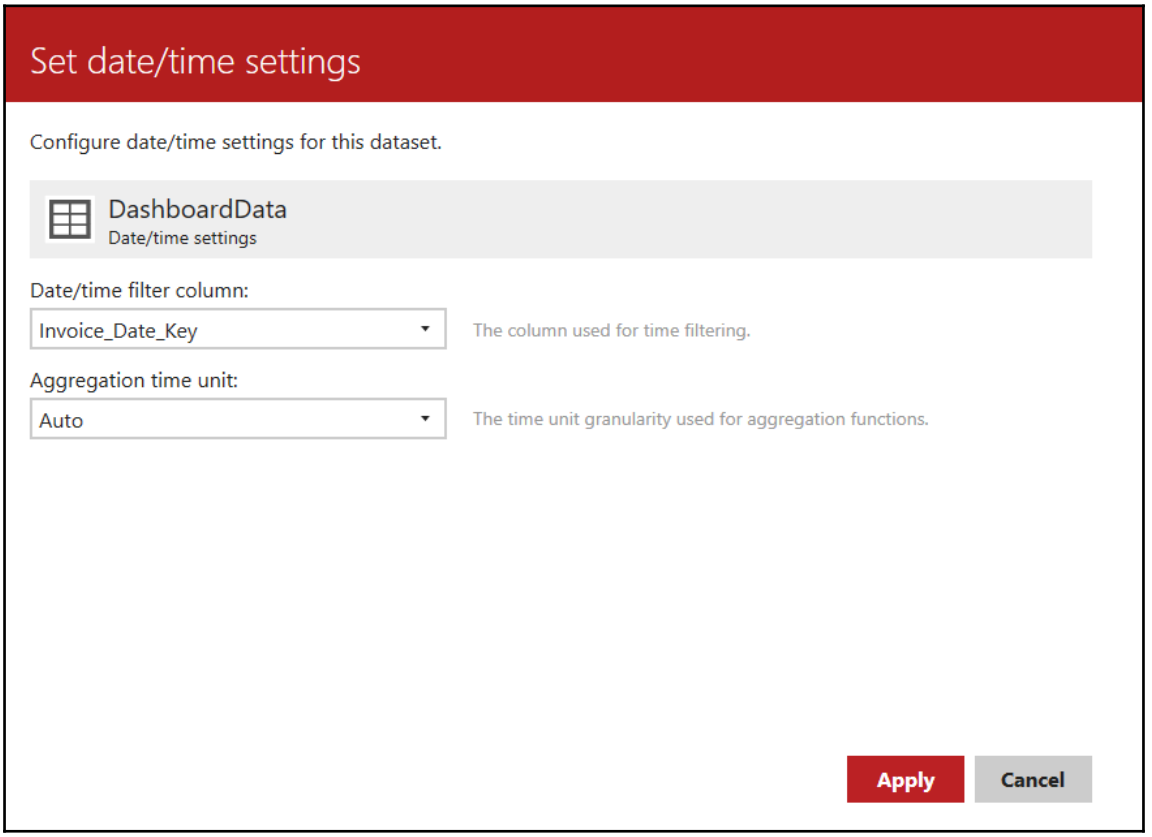


Figure 6.17

12. Next we will tie each dataset to the report elements. In the **Report elements** tab on the left, click on **Time chart 1**. Underneath the datasets, you will now see **Time chart 1** appear (if it wasn't there already).
13. Change the **Main series** to **DashboardData**.

14. In the dropdown to the right of it, uncheck all columns and place a check by **Total_Excluding_Tax**:

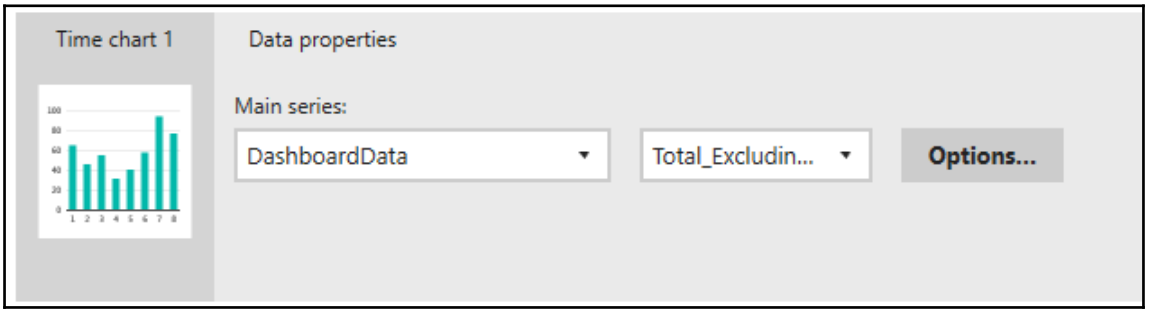


Figure 6.18

15. In the **Report elements** tab, click on **Radial gauge 1**.
16. Change the **Main value** to **YTDSales** and the metric to the right to **CurrentYTD**.
17. Change the **Comparison Value** to **YTDSales** and the metric to the right to **PreviousYTD**:

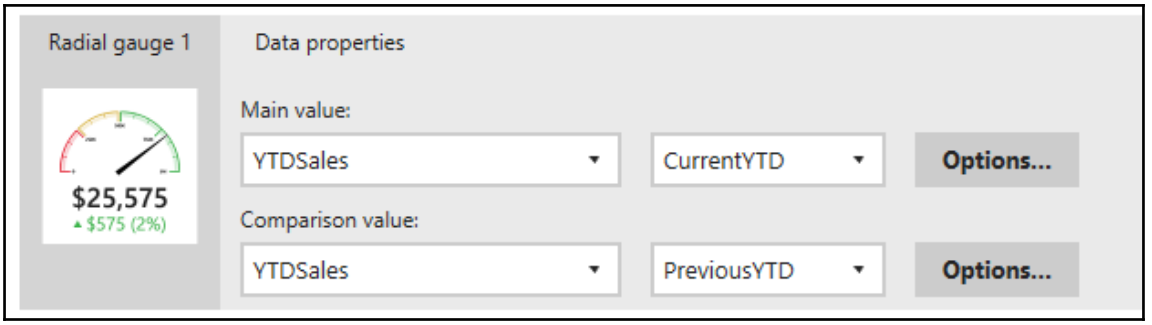


Figure 6.19

18. Click on **Simple data grid 1** in the Report elements area.
19. In the **Data properties** area under the dataset tabs, change **Data for the grid view** to `YearlySalesByState`.

20. Note that the columns from the dataset will appear to the right.



Figure 6.20

21. Beside **InvoiceYear** is a button labeled **Options…**. Click on it.
22. Change the string format to **None**.
23. Beside **SumTotalExcludingTax**, click on the **Options…** button.
24. Use the dropdown for the string format to change it to **Currency**.
25. Click on the **Gradient heat map 1** from the Report elements area.
26. Change the **Keys** to **YearlySalesByState**.
27. To the right of the Keys, ensure that **State_Province** is selected.
28. In the **Values** area, select **SumTotalExcludingTax**:

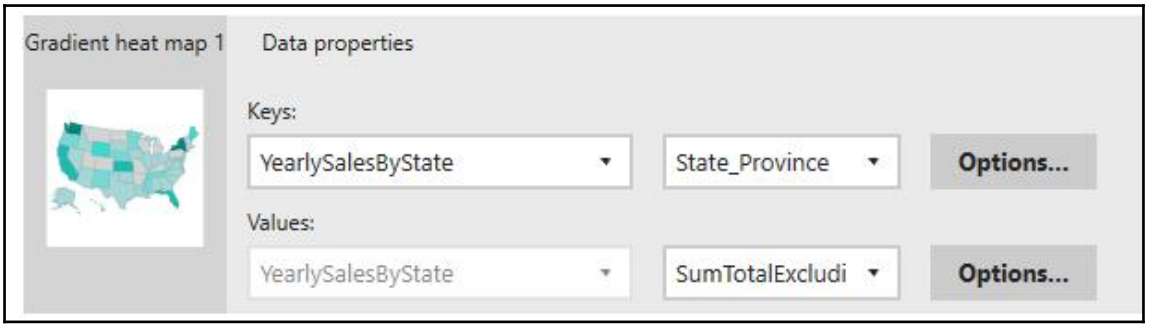


Figure 6.21

29. Click on the **Preview** tab. You should now see the report populate with real data:

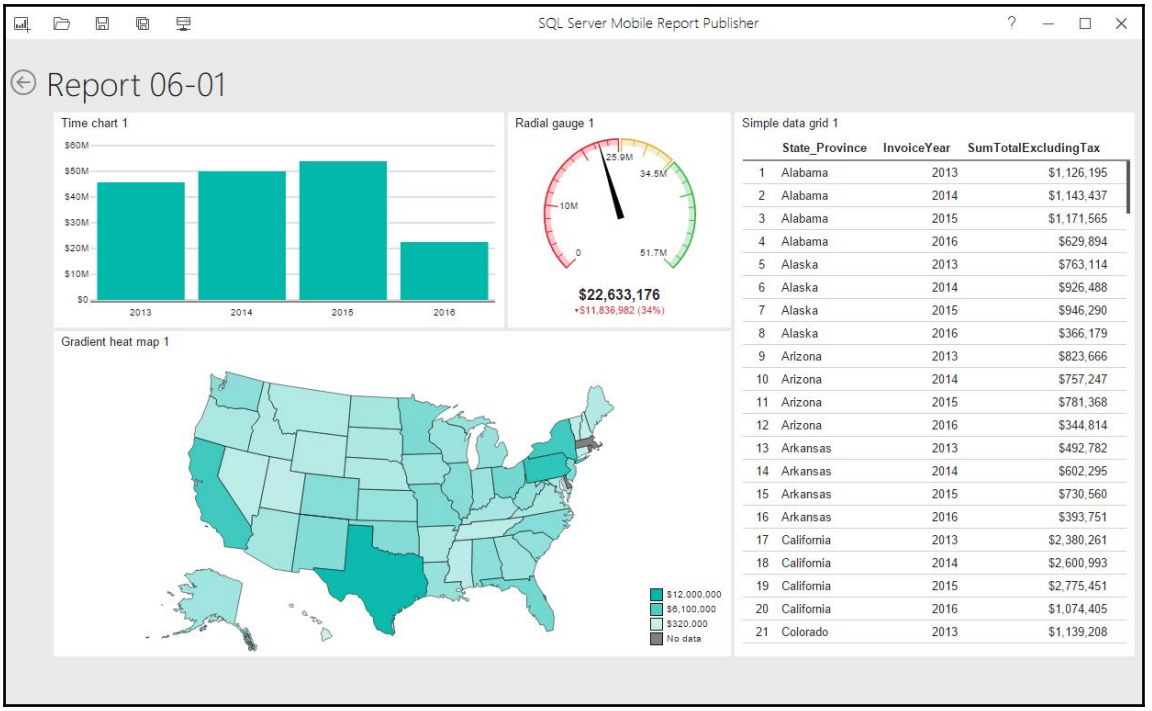


Figure 6.22

30. As a final step, we can remove the simulated data. Return to the Data tab.
31. Click on the **SimulatedMapUSA** tab.
32. Then click on the gear icon beside the dataset name.
33. Click on **Remove**.



Note how the removal of one simulated dataset removes all simulated datasets.

34. In the toolbar at the upper left, click on the **Save** icon.
35. The **Save mobile report** window appears. Click on **Save to server**.

36. Give the report a name; here we're using `Report 06-01`.
37. Enter the server to save the reports to.
38. Browse to the `Chapter06` folder you created on the Report Portal (if you haven't created it, jump over to the Report Portal in your web browser and create it):

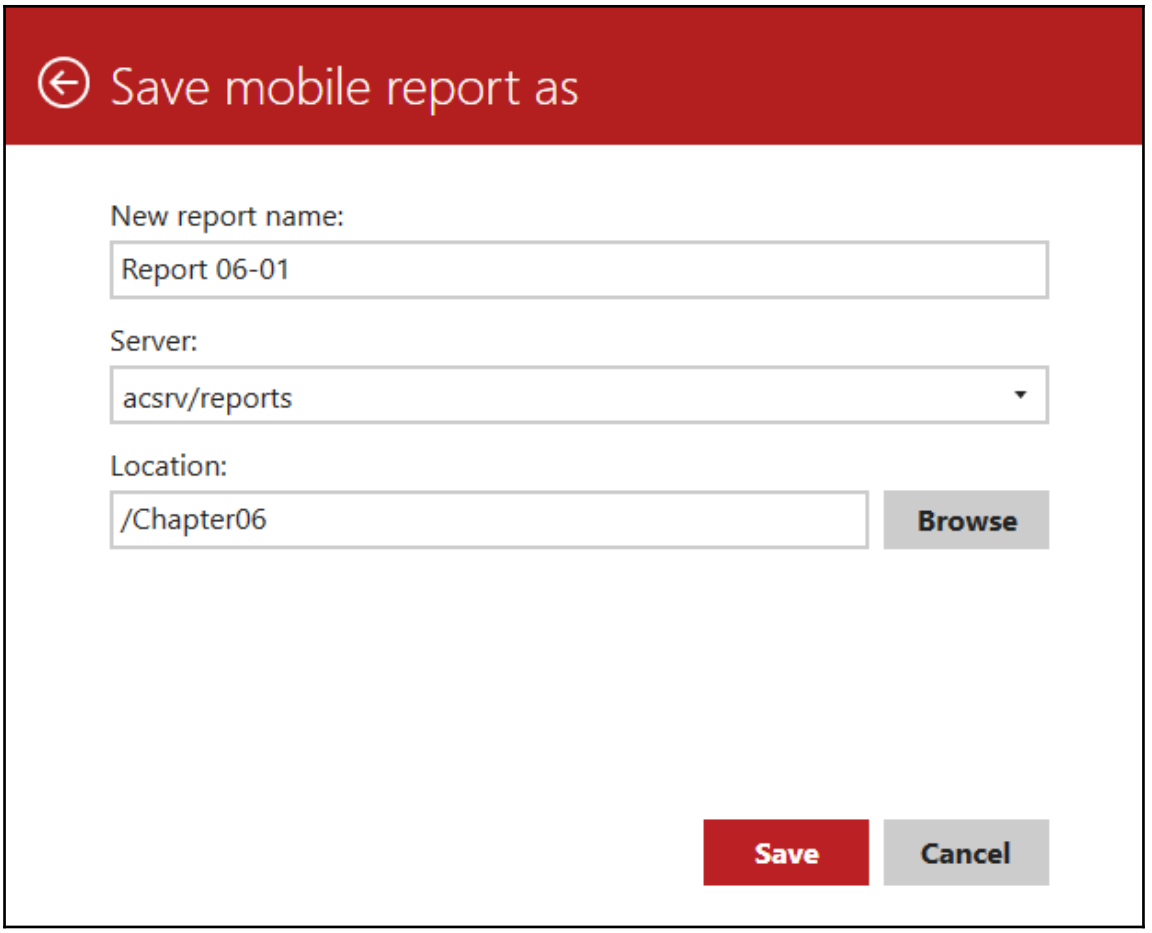


Figure 6.23

39. Click on **Save**.

How it works…

Each report element has a set of data properties associated with it. After importing a dataset into the report, you use the data properties to bind a report element to an imported dataset.

There are some things that can be customized. For example, on the Simple data grid, we used the **Options** button to format numeric values. Changing the format of the total excluding tax to currency seemed obvious, but you may have wondered about the setting of **Year to None**.

By default, the Mobile Report Publisher would attempt to format a numeric value using a comma; hence 2016 would have been listed as 2,016. Since this was a year, that is obviously incorrect. By changing it to None, we are essentially telling the Mobile Report Publisher to apply no formatting and let the number come through normally.

In the map, for the keys we needed to use something the map would understand as a location. Our map was of the USA, so we needed to use the name of a state as the key. An abbreviation (such as AL for Alabama) would have worked as well.

Had this been a world map, something like country names would have been needed to be used for the keys.

Understanding mobile report graphical elements

Our dashboard has come a long way. We've assembled it and tied it to real data on the back end. But we can still do a bit more to make the report look nicer.

Each report has a set of graphical elements properties that can be set to alter the appearance of each element even more. In this recipe, we'll see how to update the properties for each of our report elements to make it look even nicer.

Getting ready

As before, this recipe picks up where the previous one left off. If you don't have `Report 06-01` ready, open it or create it so that you can follow along.

How to do it…

1. Go to the Layout tab.
2. In the designer area, click on **Time chart 1** in the upper left.
3. At the bottom, under the designer area, the **Visual properties** for **Time chart 1** should appear.
4. Change the **Title** property to **Sales by Year**:

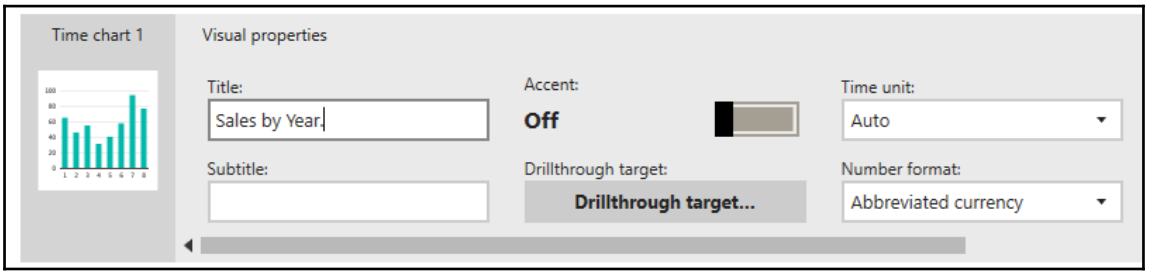


Figure 6.24

5. Note the scroll bar under the **Visual properties**. Scroll to see all the properties associated with a time chart, but there is no need to change them.
6. Click on the **Radial gauge 1** element in the designer area.
7. In the **Visual properties** at the bottom, enter `Current YTD` for the title.
8. In the Subtitle area, enter `vs Previous YTD`:



Figure 6.25

9. Scroll to see some of the other properties of the **Radial gauge**.
10. Click on the **Gradient heat map**.
11. Enter `Sales by State` for the title, `and Excluding Tax` for the subtitle.
12. Change the **Number format** to **Abbreviated**:

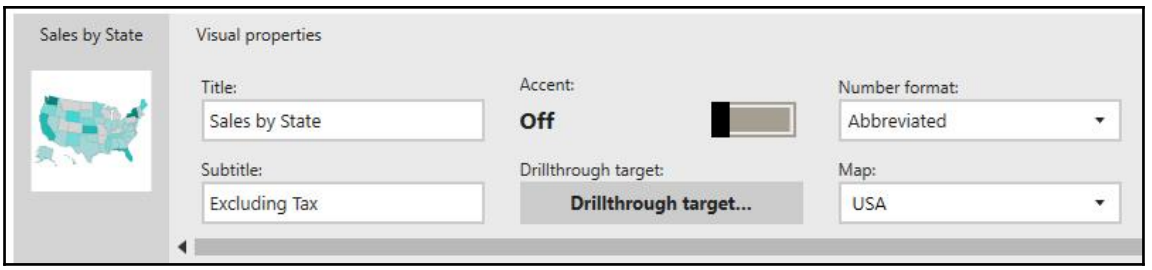


Figure 6.26

13. To fix up the Simple data grid, we actually have to go to two places. First, click on it while still on the Layout tab.
14. Enter `Sales by State/Year` for the title.
15. In the final version of the report in the previous recipe, you may have noticed the data grid had row numbers. Turn those off by looking to the very right and changing the **Row numbers** proper to **Hide**:

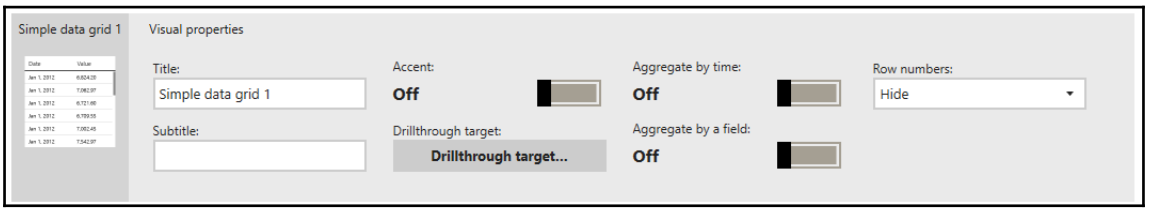


Figure 6.27

16. To clean up the column titles, we need to click on the Data tab.
17. Note how the Report elements have been renamed to match their titles. Click on the **Sales by State/Year simple** data grid.
18. To the right of the Data properties are the **Data grid columns**.

19. Beside each column name is a textbox. It defaults to the name of the column from the dataset, but we can click on each one and enter what we wish to have displayed. Replace what is there with these names: `State`, `Invoice Year`, and `Total Excl. Tax`:

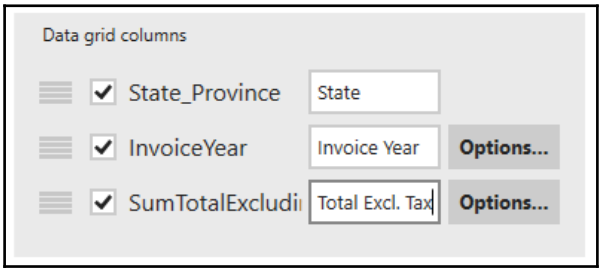


Figure 6.28

20. Click on **Preview** to see your cleaned-up report. After doing so, be sure to save your work:

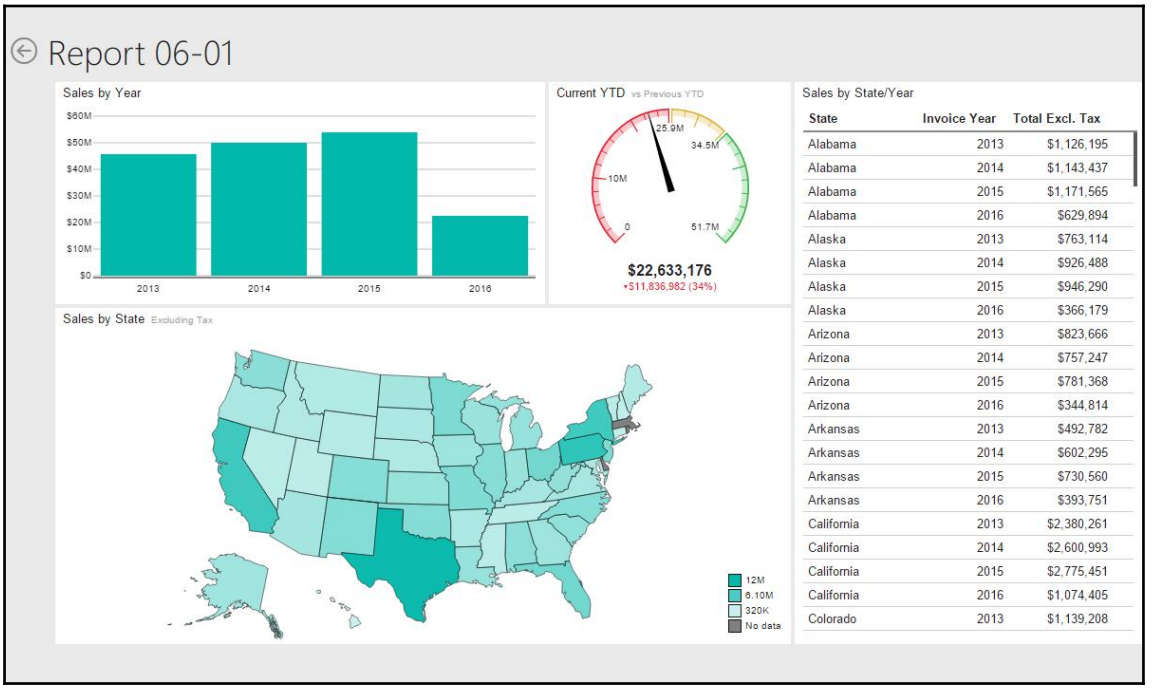


Figure 6.29

How it works…

Each report element has a set of visual properties that can be set. These visual properties can control the look of the report, allowing you to set titles and other visual items.

There is one more thing we didn't do, and that was altering the colors. Return to the layout tab. In the upper right corner is a color bar dropdown. Clicking on it brings up a variety of color palettes you can pick from:

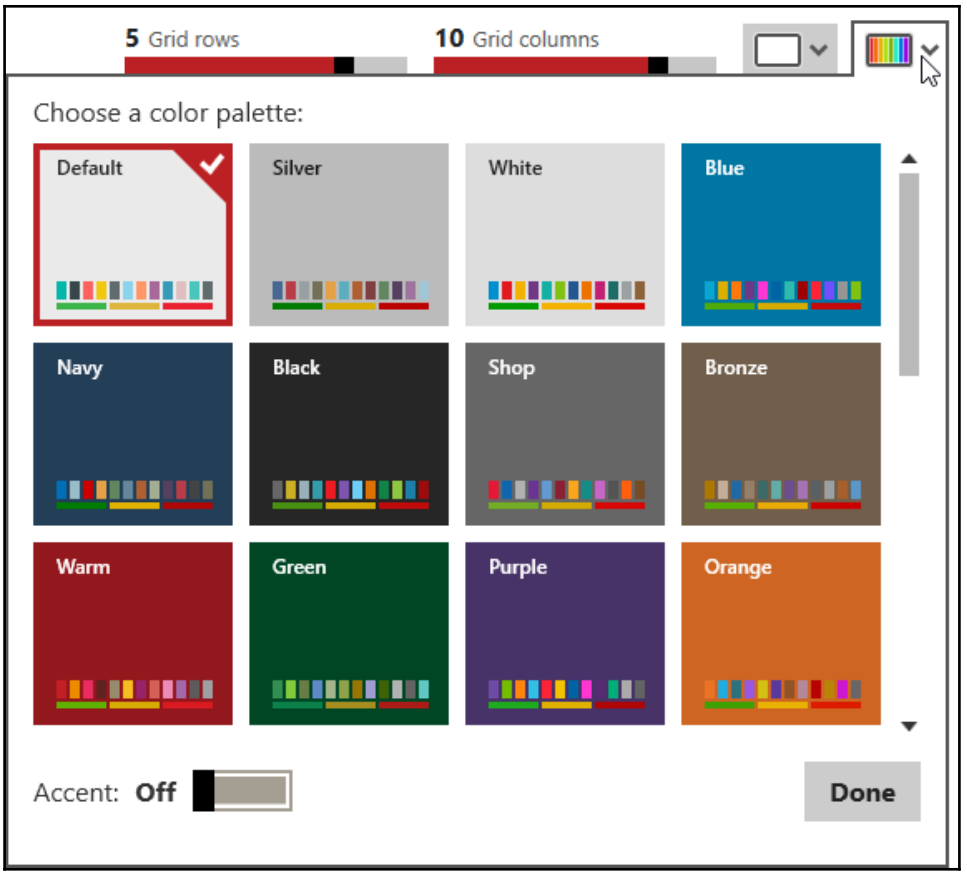


Figure 6.30

Note that these colors will probably not render well in print format, which is why they weren't included in the recipe. You should take the opportunity to check out the various color palettes and see what works for you.

Understanding the mobile report layout

So far our layout has been what Microsoft calls the *Master* layout. This is the format used when someone looks at your report using a web browser.

The *Mobile* in the Mobile Report Publisher implies you can create reports specially formatted for tablets and phones. In this recipe, we'll see just how easy it is to do so.

Getting ready

This recipe continues where the previous one left off, so if you don't have `Report 06-01` ready, open it now.

How to do it…

1. In the upper right corner is a white box with a drop-down arrow. Click on it:

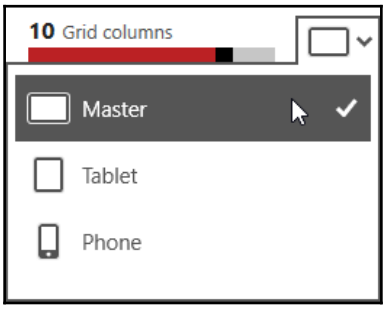


Figure 6.31

2. You will now see the three layout options for working with reports. Click on **Tablet** to change to tablet mode.

3. The four available report elements appear to the left. We'll drag each one onto the designer in a layout that will look good on a tablet. Start with the **Sales by Year** chart. Place it in the very upper left corner and then drag it so that it is two rows high by six wide.
4. Next, place the **Sales by State** map under it, taking up four wide by three high.
5. Place the **Current YTD** gauge to the right of the map, and size it to two wide by three high.
6. Finally, place the **Sales by State/Year** grid below the map, and size it to take the remaining space. Save the report, and then preview it to see what it looks like:



Figure 6.32

7. Now to do the layout for the phone, return to the layout tab, and in the layout dropdown, change from **Tablet** to **Phone**.
8. Place the chart in the upper left so that it takes up an area two high by three wide.
9. Place the gauge beside it, one wide by two high.
10. Under it, place the map so that it takes up an area two high by four wide.
11. Finally, place the **Sales by State/Year** grid at the bottom, taking up the remaining space two high by four wide.
12. Save and then preview the phone layout:



Figure 6.33

13. If you have a phone or tablet and wish to view the report, you'll need to install the Power BI application from Microsoft, which should be available in your device's App Store.
14. After installing, you'll first need to connect to your SQL Server Reporting Services server. Here is a screen capture from the author's' iPhone:

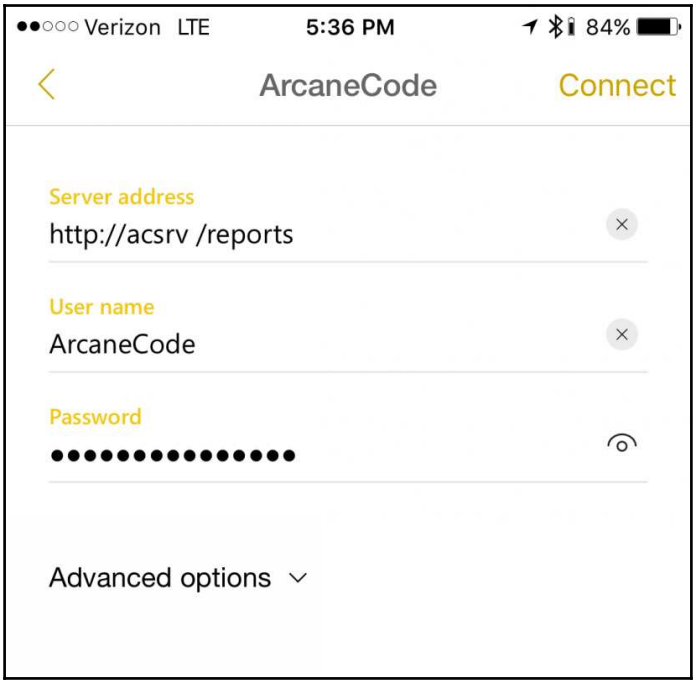


Figure 6.34

15. Once connected, it will display a list of folders, from which you can drill down to the folder with your reports. From there, you will see thumbnails of your reports. Click on the one for **Report 06-01** to open it. Here is another screenshot from the author's' iPhone:

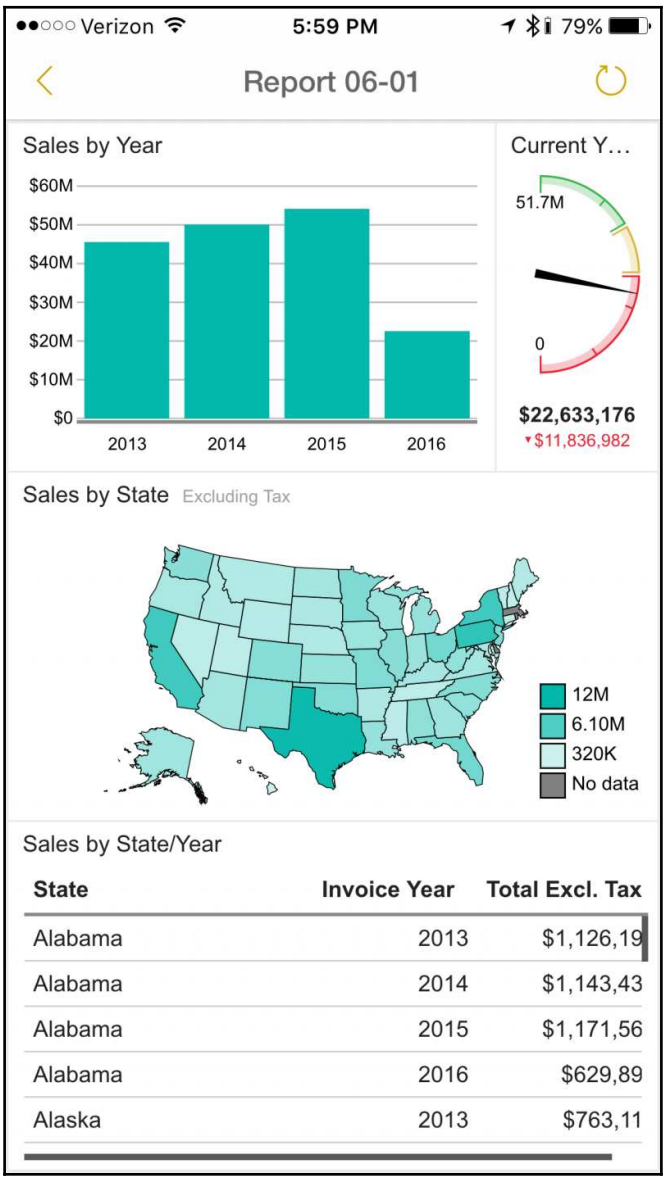


Figure 6.35

How it works…

The Power BI app communicates to Reporting Services what type of device it is running on, a tablet or phone. With this information, SSRS selects the layout you created for that device as the one to display.

The layout makes it very easy to arrange your report elements in a manner that fits well into the available area of the device.

Note that if you do not specify a layout, SSRS will take a best guess at what you may want and display something. However, the results are usually lacking. It's best to take a few moments to specify your own layouts based on the input from your users.

Configuring navigators

In the Mobile Report Publisher, navigators act as filters to other elements on the report. They may actually display information, as in the case of the Time navigator, or they may be just a selection box for picking an item.

These navigators are then tied to the datasets used in the report. When a navigator is sourced from a dataset, all other report elements that use the same dataset are automatically filtered. When a navigator is sourced from its own dataset, it must be tied to other datasets to filter them.

In this recipe, we'll see examples of both and create a brand new report.

Getting ready

Many of the features we'll use in this new report we've already used, so we'll be as brief as possible when covering those steps. If you need a refresher, refer back to previous sections in this chapter, such as *How to create a mobile report*, *Setting up data sources*, and *How to use datasets for mobile reporting* recipes.

We'll also need one new dataset. Use Report Builder to generate a new dataset on your server. Name it `State_Province`; the data source will be the same `WideWorldImportersDW` we've been using, and it should use the following SQL query:

```
SELECT DISTINCT [State Province]
  FROM [Dimension].[City]
 ORDER BY [State Province]
```

How to do it…

1. Open the **Mobile Report Publisher** and create a new report.
2. On the Layout tab, at the very top are the **Navigators** that are used to control what is displayed in the report. Drag and drop a **Time navigator** to the upper left block in the design area. Resize so that it is two blocks high and eight blocks wide.
3. Return to the Navigators area and drag a **Selection list** beside the **Time navigator**; make it two blocks by two blocks in size.
4. Place a **Simple data grid** under the **Time navigator**. Make it three blocks high by seven wide.
5. Place a **Gradient heat map** beside the simple data grid, making it three by three in size. Your result should resemble this screenshot:

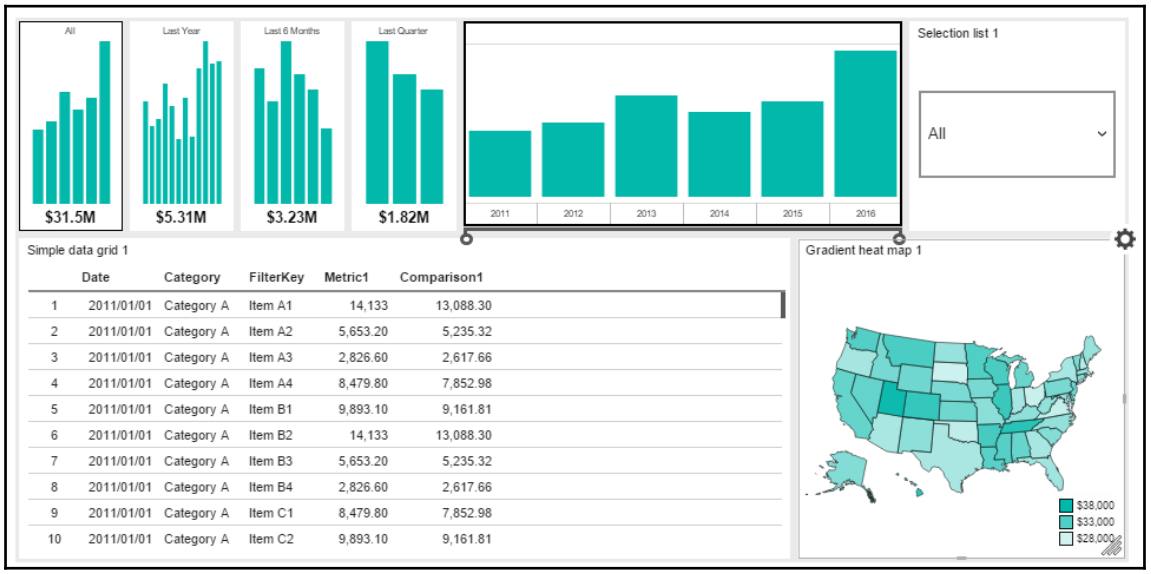


Figure 6.36

6. Now go to the Data tab.
7. Add the **DashboardData** dataset and then add the new **State_Province** dataset.
8. In the Report elements, click on **Time navigator 1**.
9. In the **Series for background** chart, select **DashboardData**. For the measure area, pick **Total_Excluding_Tax**.

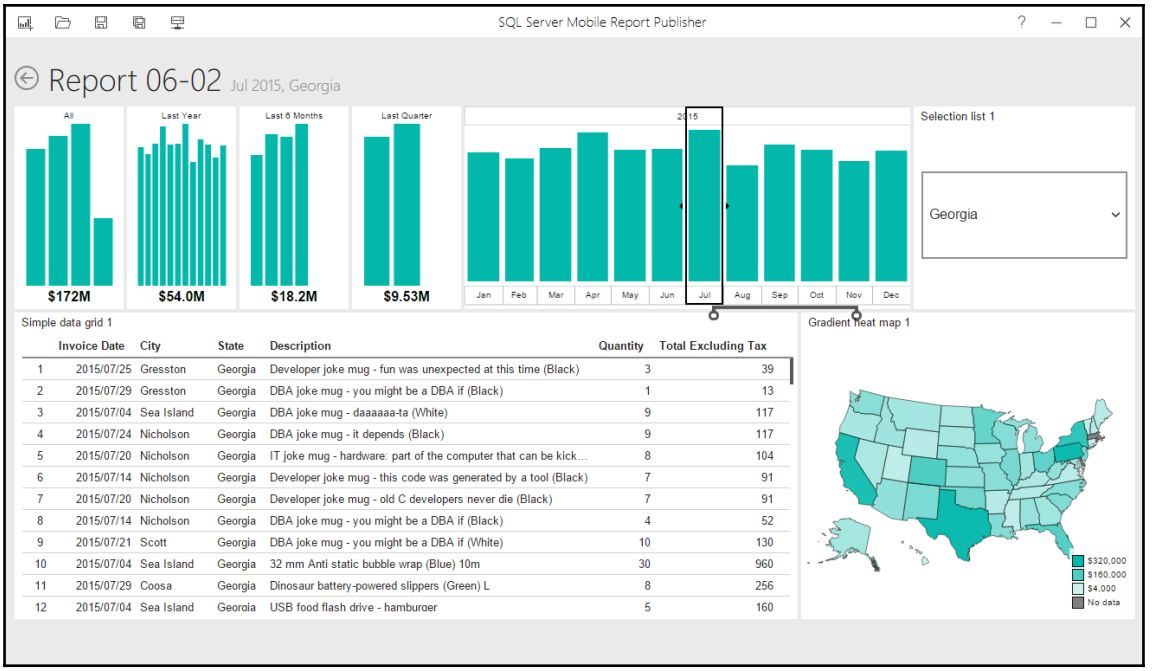
10. Click on **Selection list 1**. At the bottom, change the **Keys** to the **State_Province** dataset. Make sure both the key and the label are set to the **State_Province f**ield.
11. To the right is a section, **Filter these datasets when a selection is made**. This is where we tie the results of the selection to other datasets. Uncheck **SimulatedTable** and place a check beside **DashboardData**. Then in the dropdown beside it, select the **State_Province** field from the list:



Figure 6.37

12. Next, click on the **Simple data grid 1** from the Report elements area.
13. Change the data for the grid view to **DashboardData**.
14. In the Data grid columns, make sure that only the following fields are checked: **Invoice_Date**, **City**, **State_Province**, **Description**, **Quantity**, and **Total_Excluding_Tax**.
15. While you are here, edit the names for each column to something that looks nicer on a report, as shown in Figure 6.20 from *How to use datasets for mobile reporting*.
16. Click on **Gradient heat map 1** from the Report elements.
17. Change the keys to DashboardData and use `State_Province`. In the Values area under it, use `Total_Excluding_Tax` for the value field.
18. We no longer need our simulated data, so click on the **SimulatedTable** tab. Then click on the **Gear** icon to bring up its menu, and click on **Remove**.
19. Save the report in your `Chapter06` folder as `Report 06-02`.
20. Preview the report.
21. Click on various times in the **Time navigator** at the top. Note how the values for the simple data grid and the map both update when a new selection is made.

22. Now in the **Selection list 1** control, pick a state. Here we picked **Georgia**. Note how the **Simple data grid** again updates to show only **Georgia**. The map though did not change, as it is showing all states; filtering for only one state would be pointless:



	Invoice Date	City	State	Description	Quantity	Total Excluding Tax
1	2015/07/25	Gresston	Georgia	Developer joke mug - fun was unexpected at this time (Black)	3	39
2	2015/07/29	Gresston	Georgia	DBA joke mug - you might be a DBA if (Black)	1	13
3	2015/07/04	Sea Island	Georgia	DBA joke mug - daaaaaa-ta (White)	9	117
4	2015/07/24	Nicholson	Georgia	DBA joke mug - it depends (Black)	9	117
5	2015/07/20	Nicholson	Georgia	IT joke mug - hardware: part of the computer that can be kick...	8	104
6	2015/07/14	Nicholson	Georgia	Developer joke mug - this code was generated by a tool (Black)	7	91
7	2015/07/20	Nicholson	Georgia	Developer joke mug - old C developers never die (Black)	7	91
8	2015/07/14	Nicholson	Georgia	DBA joke mug - you might be a DBA if (Black)	4	52
9	2015/07/21	Scott	Georgia	DBA joke mug - you might be a DBA if (White)	10	130
10	2015/07/04	Sea Island	Georgia	32 mm Anti static bubble wrap (Blue) 10m	30	960
11	2015/07/29	Coosa	Georgia	Dinosaur battery-powered slippers (Green) L	8	256
12	2015/07/04	Sea Island	Georgia	USB food flash drive - hamburger	5	160

Figure 6.38

23. Finally, take a look at the report title. Directly to the right of **Report 06-02**, it is displaying the current set of filters used on the report, in this example **July 2015**, **Georgia**.
24. Be sure to save your report. You are free to refine the report using techniques shown in previous recipes, especially the *Understanding mobile report graphical elements* and *Understanding the mobile report layout* recipes.

How it works…

Using navigators, users can make selections that limit the amount of data on the report they view. In some cases, such as the Time navigator, these also display information in addition to their use as filters.

The key to navigators is to understand that they are used to limit the underlying datasets. Back in step 11, you saw the linking of the **State_Province** selecting navigator to the **DashboardData** dataset.

It should be noted that for each report element, you can pick and choose which navigators should apply. Go to the data tab, and click on the **Simple data grid**. Then click on the **Options** button to the right of **DashboardData**:

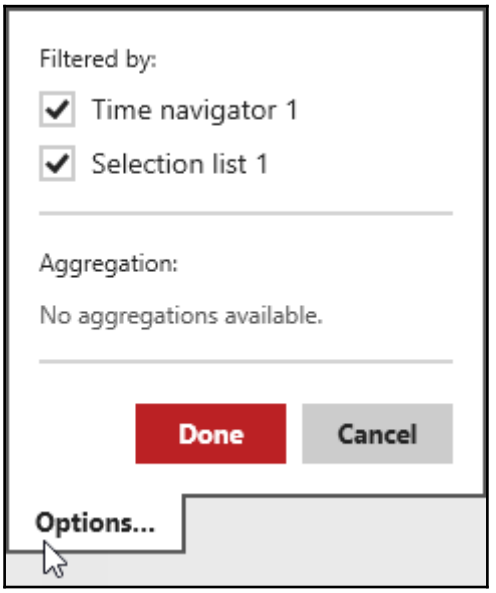


Figure 6.39

Here you can see this dataset is filtered by both the navigators.

But now go to the **Gradient heat map**, and look at the options beside the keys area:

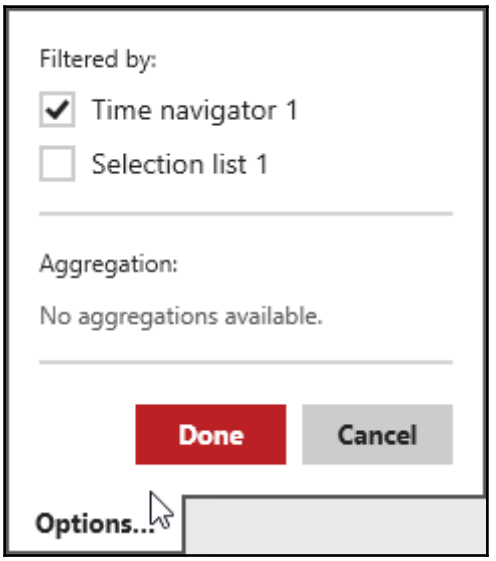


Figure 6.40

Note how only the **Time** Filter is checked. The Mobile Report Publisher was smart enough to figure out that the filter was a state, but for a map, it just doesn't make sense to limit it to a single state. It does though make sense to filter on time, so it left that checked on.

You are free to check and uncheck filters as you need them for the datasets in your reports.

Configuring parameters for drill-through reports

Just as with regular SSRS reports, it is possible to have one mobile report execute a second report, passing parameter values from the source to the target. However, the mechanism for passing parameters is radically different.

In traditional SSRS, you can select a specific value in the report body, and launch the target report with that value. This can be done with Mobile Reports, with the caveat that it should be mapped to a navigator in the target report. Typically it is easiest to map the navigators in the source to the navigators in the target, which is what we'll do with this recipe.

Getting ready

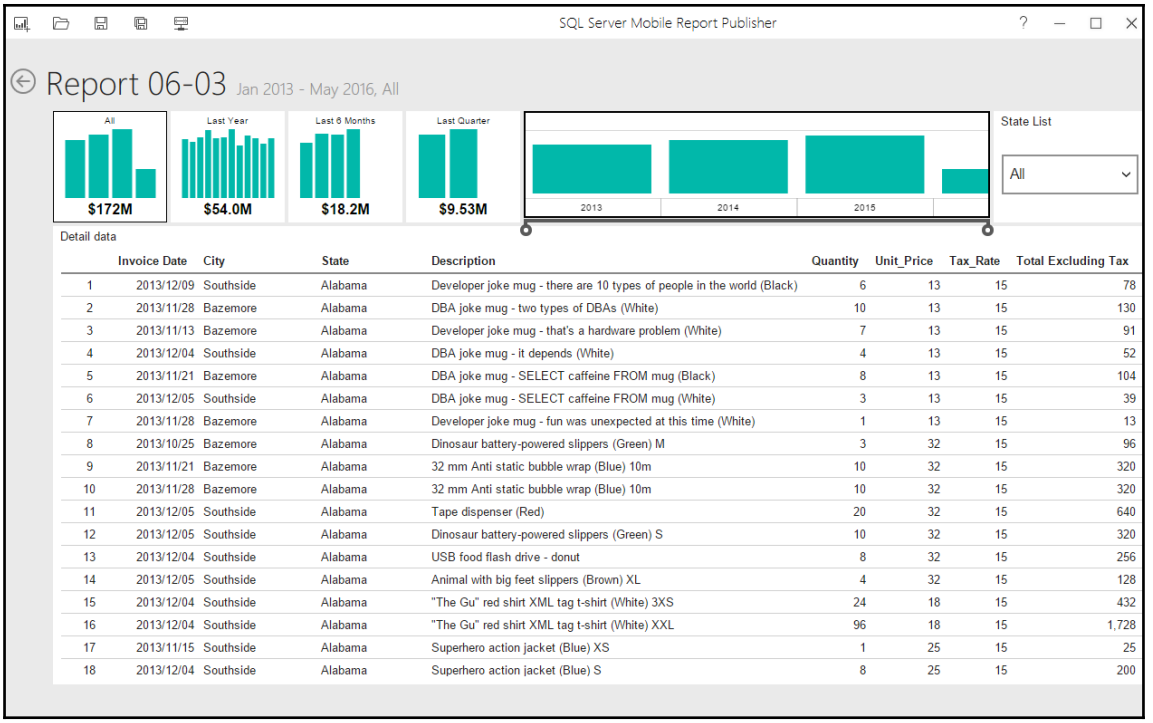
For this recipe, we'll create a new report for the target but will be using Report 06-02, as created in the previous recipe, as our source. Be sure you have it completed.

As in the previous recipe, we'll be brief for the items you've seen before. In addition, we'll leave a lot of the graphical clean-up such as titles up to you.

How to do it…

1. Create a new report, named `Report 06-03`.
2. Drag and drop a **Time navigator** onto the designer in the upper right corner. Make it one block high by eight blocks wide.
3. Place a selection list next to the **Time navigator**, making it two wide by one high. In the selection list, rename the title to State List.
4. Place a simple data grid under the **Time navigator**, and resize to take up the remaining area.
5. Go to the **Data** tab.
6. Add the existing datasets of **DashboardData** and **State_Province**.
7. While still on the **Data** tab, click on the **Time navigator 1**.
8. Set the series to **DashboardData** and the value (to the right) to **Total_Excluding_Tax**.
9. Click on the **State List**.
10. Set the keys to come from the **State_Province** dataset.
11. Ensure that the values are both set to the **State_Province** field.
12. In the filter area to the right, uncheck **SimulatedTable**. Then check on **DashboardData**, and change the dropdown to **State_Province**.
13. Click on the simple data grid and change the data for the grid to the **DashboardData** dataset.
14. In the data grid columns area, only have the following columns checked: **Invoice_Date_Key**, **City**, **State_Province**, **Description**, **Quantity**, **Unit_Price**, **Tax_Rate**, and **Total_Excluding_Tax**.
15. Remove the simulated data.

16. Preview the report, to ensure that it is working:



	Invoice Date	City	State	Description	Quantity	Unit_Price	Tax_Rate	Total Excluding Tax
1	2013/12/09	Southside	Alabama	Developer joke mug - there are 10 types of people in the world (Black)	6	13	15	78
2	2013/11/28	Bazemore	Alabama	DBA joke mug - two types of DBAs (White)	10	13	15	130
3	2013/11/13	Bazemore	Alabama	Developer joke mug - that's a hardware problem (White)	7	13	15	91
4	2013/12/04	Southside	Alabama	DBA joke mug - it depends (White)	4	13	15	52
5	2013/11/21	Bazemore	Alabama	DBA joke mug - SELECT caffeine FROM mug (Black)	8	13	15	104
6	2013/12/05	Southside	Alabama	DBA joke mug - SELECT caffeine FROM mug (White)	3	13	15	39
7	2013/11/28	Bazemore	Alabama	Developer joke mug - fun was unexpected at this time (White)	1	13	15	13
8	2013/10/25	Bazemore	Alabama	Dinosaur battery-powered slippers (Green) M	3	32	15	96
9	2013/11/21	Bazemore	Alabama	32 mm Anti static bubble wrap (Blue) 10m	10	32	15	320
10	2013/11/28	Bazemore	Alabama	32 mm Anti static bubble wrap (Blue) 10m	10	32	15	320
11	2013/12/05	Southside	Alabama	Tape dispenser (Red)	20	32	15	640
12	2013/12/05	Southside	Alabama	Dinosaur battery-powered slippers (Green) S	10	32	15	320
13	2013/12/04	Southside	Alabama	USB food flash drive - donut	8	32	15	256
14	2013/12/05	Southside	Alabama	Animal with big feet slippers (Brown) XL	4	32	15	128
15	2013/12/04	Southside	Alabama	"The Gu" red shirt XML tag t-shirt (White) 3XS	24	18	15	432
16	2013/12/04	Southside	Alabama	"The Gu" red shirt XML tag t-shirt (White) XXL	96	18	15	1,728
17	2013/11/15	Southside	Alabama	Superhero action jacket (Blue) XS	1	25	15	25
18	2013/12/04	Southside	Alabama	Superhero action jacket (Blue) S	8	25	15	200

Figure 6.41

17. Save the report to the server as `Report 06-03`.
18. Close this report and then open the previous report, `Report 06-02`.
19. On the layout, click on the **Simple data grid 1** located in the design area.

20. In the properties area at the bottom, click on **Drillthrough target**… and pick **Mobile report…:**

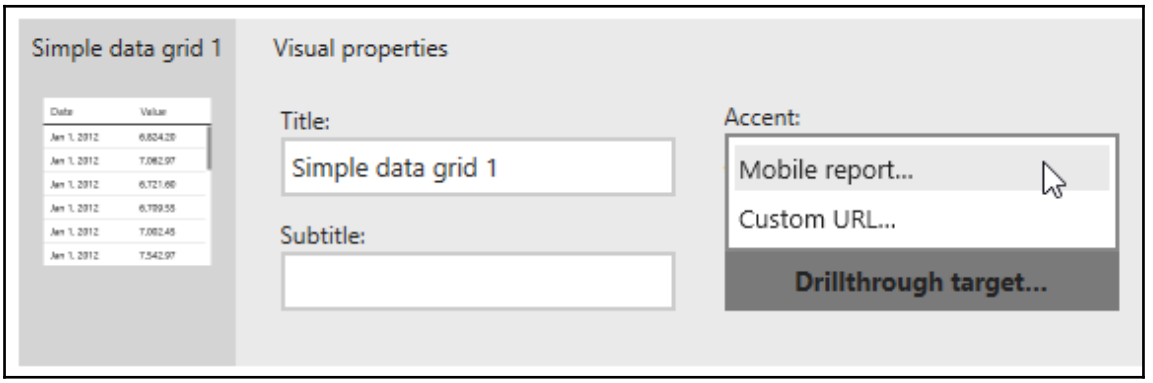


Figure 6.42

21. The Open from server window now appears. If not already in the `Chapter06` folder, navigate to it and pick **Report 06-03**:

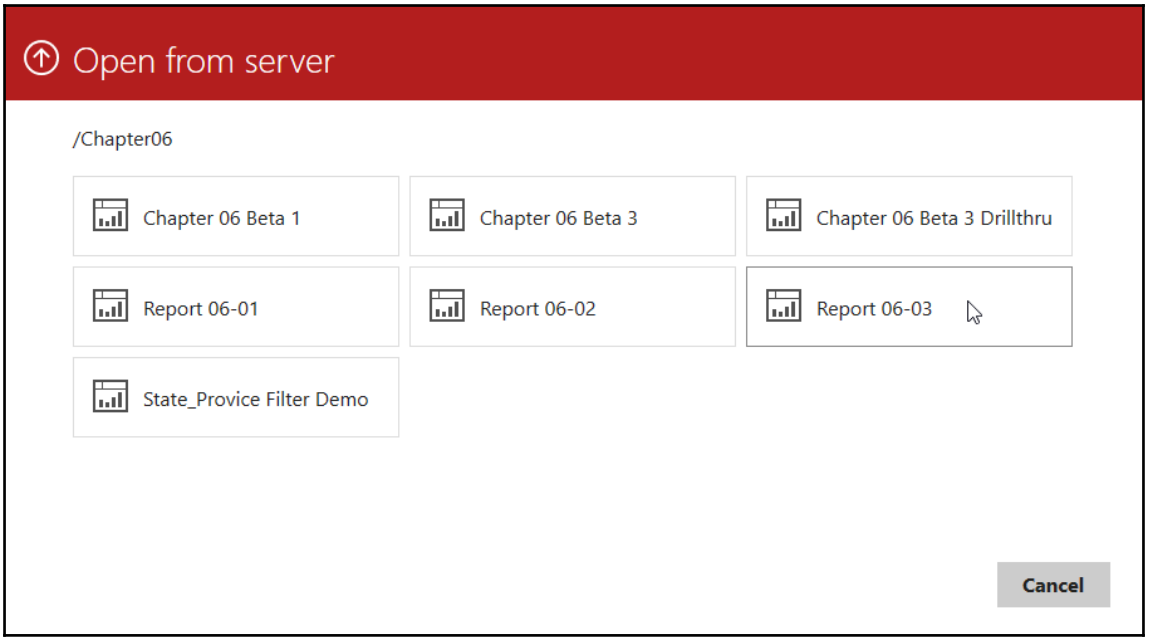


Figure 6.43

22. Now you need to map from the source report (`Report 06-02`) to the target (`Report 06-03`). When you initially see the parameters, they will all be default; you'll need to update them with real values.
23. Click on the dropdown by **SelectedStartTime** and pick **SelectedStartTime** from the **Time navigator** in the source report:

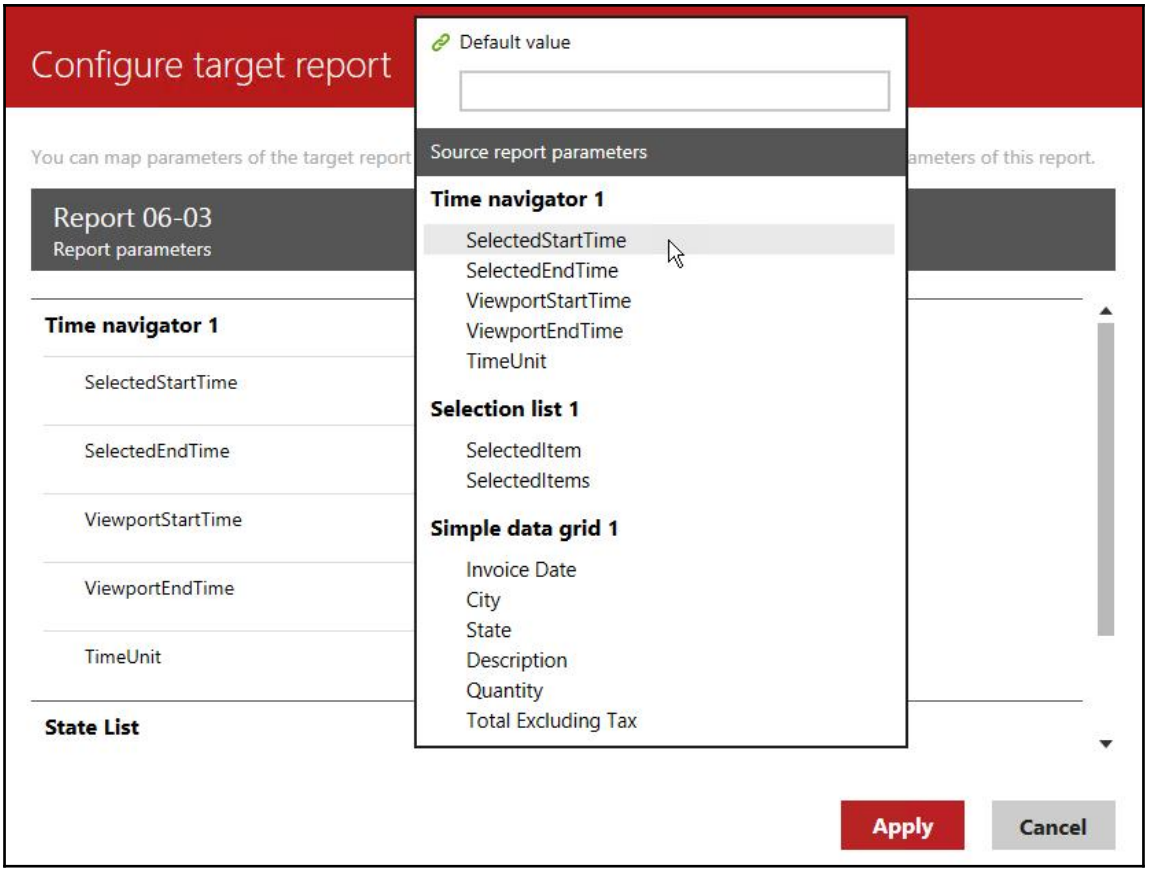


Figure 6.44

24. Repeat for each one.

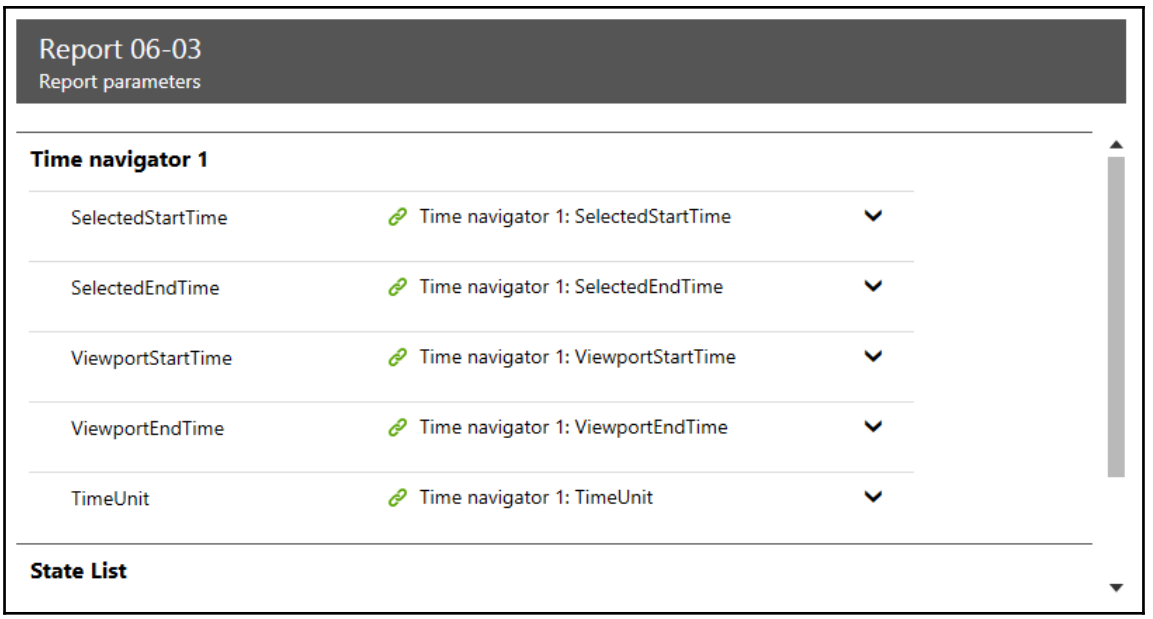


Figure 6.45

25. Now scroll down to the State List. Map the State List items to those from your selection list:

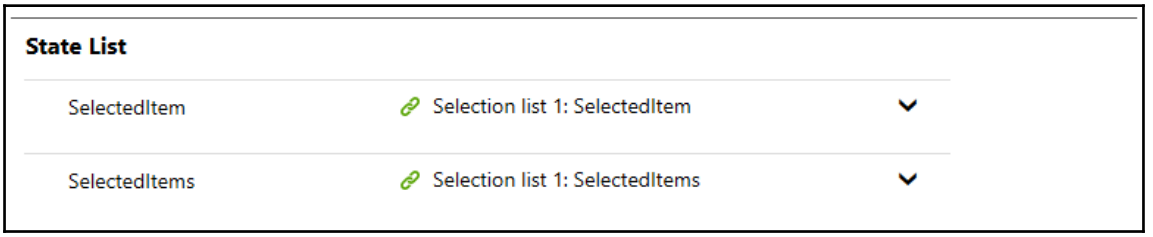


Figure 6.46

26. Click on **Apply**.
27. Save the report back to the server.
28. As the drill-through does not work in Preview mode, you'll have to open `Report Portal` in your web browser. Do so now.
29. Navigate to and open `Report 06-02`.

30. Make some selections on the report to limit the amount of data.
31. Click on the **simple data grid**.
32. You should now see `Report 06-03` open with the selections limited to the same that were used in the source report, `Report 06-02`.

How it works…

To pass parameters from one report to another, you first need to set up navigators in the target to receive the values. You then map data from the source to the navigators in the target as shown in Figure 6.44. When the target report renders, its navigators will have been set and the datasets will then be filtered from them.

When the mapping page appeared, the navigators from the target were all listed as **Default value**. If you leave this set to default, then no values are passed from the source to the target. This would be useful in situations where you have a navigator in your target, but nothing to map to it from your source report.

7

Consuming Reports – Report Access Enhancement

It is time to view published reports! You have just read six chapters for understanding Reporting Services, how to author a basic report, and how to make a complex report with various, modern, attractive visuals. You know that there was something called **Report Manager** which we have used with all previous versions of Reporting Services for viewing published reports and managing them. What if I say that it is no longer available with Reporting Services? Yes, you heard me, it is not available now.

Worried? Do not be; we have been given a nice web portal for viewing and managing our reports, not only for that, but with some more additional functionality. It is called **Reporting Services Web Portal**.

We will be exploring the new Web Portal in this chapter with new features and functionalities added.

Let's have a look at what we can do with this new Web Portal. In this chapter, we will cover the following recipes:

- Exploring the new Web Portal
- Managing reports with the Web Portal
- Viewing reports on any web browser
- Caching the report
- Refreshing the cache
- Creating a KPI with predefined values

- Creating a KPI with shared datasets
- Customizing the look-and-feel of the Web Portal
- Printing reports with PDF rendering extension
- Embedding reports in other applications

Introduction

The Reporting Services Web Portal is a web application that replaces our old friend, Report Manager. This portal is not only for paginated reports, it can be used for viewing and managing mobile reports as well.

Since the new Web Portal is designed with modern web standards, it supports all modern browsers such as Edge, Internet Explorer, Chrome, FireFox, and Safari, and we do not need to worry about browser compatibility and consistency with our reporting solutions. The design of this is user-friendly for both administrators and users, and well organized new features and enhancements can be easily used for performing various activities by both parties.

Generally, a business user uses the portal for viewing reports. This usage has been extended, giving more functionality such as KPI creation, report embedding, and printing reports. The application has been designed in such a way that a business user feels it is purely designed for business users, not for technical users such as administrators and engineers.

Administrators typically manage the environment by performing operations such as managing data sources, authentication, authorizations, and reports. The Web Portal makes these tasks less complex and simplifies their management.

This chapter discusses all the new featured introduced with Reporting Services 2016 related to the Web Portal and major enhancements on consumptions of reports using mentioned recipes.

Exploring the new Web Portal

The new Web Portal is only for **Reporting Services Native Mode**. It does not support on SharePoint integrated mode. The new portal looks organized and that makes it easy to navigate various items and explore different sections. The old look, with a few buttons and a top bar, has been replaced with multiple tabs and a set of menus that provides easy access to items which can be managed efficiently. Have a look on Figure 7.01 that shows the old Report Manager:

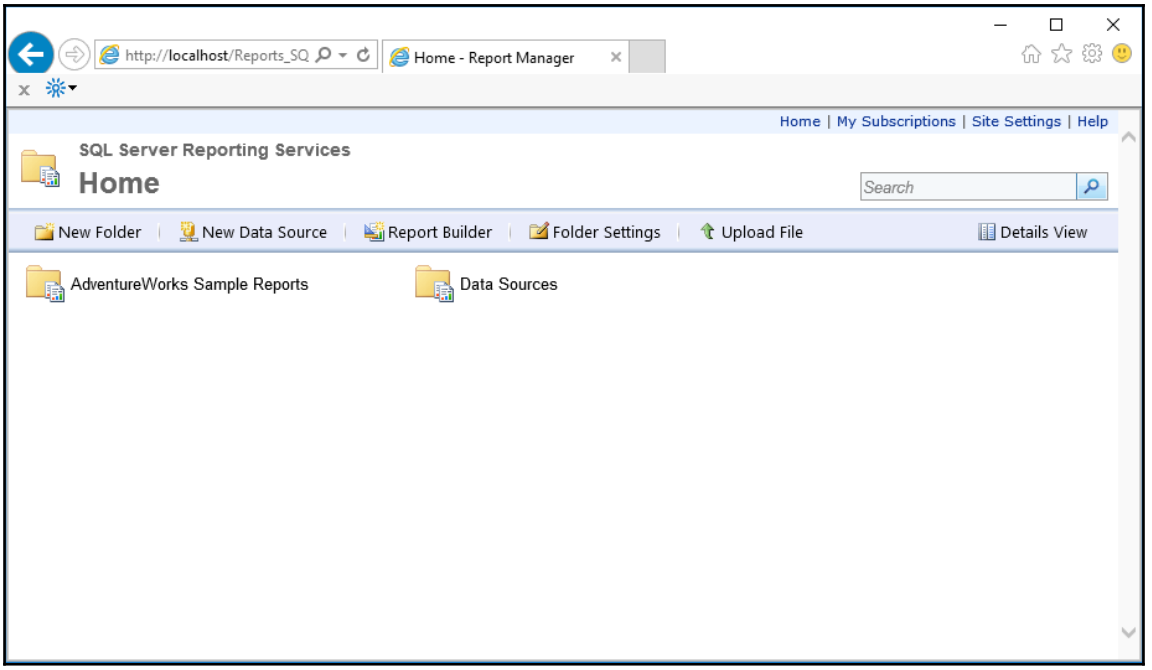


Figure 7.01

Now see how the appearance and the layout of the portal has changed with the new Web Portal. It is shown in the Figure 7.02:

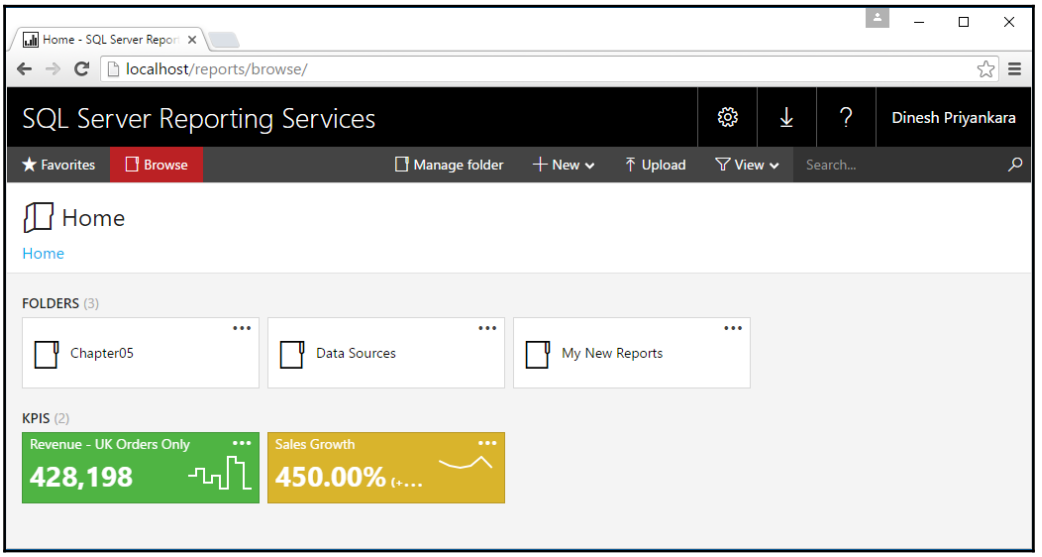


Figure 7.02

Getting ready

Since the Web Portal is a web-based application, it has to be opened using a browser. The URL configured for the Web Portal is based on the instance type of SQL Server. If the instance is the default instance, the URL is configured as follows:

`http://<servername>:80/reports`.

The port 80 is automatically added but it can be configured to a different port as well. If the instance is a named instance, the URL is as follows:

`http://<servername>:80/reports_<instancename>`.

When you open the Web Portal on the local machine that the Reporting Services is running, you might get an exception, indicating that you have no permission to view the Web Portal. The reason for this is **Universal Access Control** (**UAC**) and you are not running the browser as an administrator. If you experience this, open the browser as an administrator and then open the Web Portal.

The server we have configured for demos are based on the default instance. Let's start navigating the portal:

1. Open you web browser and type web portal URL.
2. Navigate through the items you see. You may not see all items because pages, links, and options are visible for you based on the permissions you have on the Reporting Services instance.

How to do it…

There are multiple sections to explore in the portal. Let's go through the most important items and understand them:

1. Check the header of the Home page (Figure 7.03). You will see icons for **Settings**, **Download**, and **Help**, along with the title **SQL Server Reporting Services**:



Figure 7.03

2. Click on the **Settings** icon. It opens menu items for **My settings**, **My subscriptions**, and **Site settings** that allows you to pin Reporting Services report items to Power BI dashboards, add subscriptions for reports, and manage branding, schedules and security respectively:

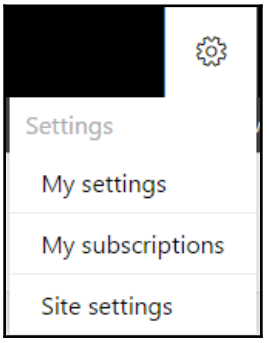


Figure 7.04



Note that you may not see the **My Settings** menu item if the Report Server is not registered with Power BI. The registration can be done with the Power BI Registration page in Reporting Services Configuration Manager.

3. Click on Download icon. This shows the menu items **Mobile Report Publisher**, **Report Builder**, **Power BI Desktop**, and **Power BI Mobile** for downloading the **Mobile Report Publisher**, **Report Builder**, **Power BI Desktop** and **Power BI Mobile** applications:

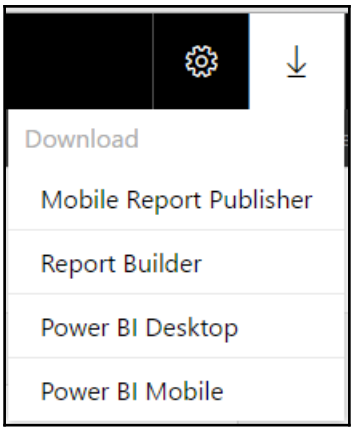


Figure 7.05

4. Check the **Help** icon. You will see links to the MSDN forum where you can find out how others have troubleshot issues experienced and assists you in getting your questions answered.

5. The next bar just below the header bar has three sections:
    - The left most section has two tabs that allow you to navigate all report items using **Browse** and see your most liked items using Favorites.
    - The middle section is formed using a set of menu items for managing folders, adding report items, uploading items, and setting the visibility of items marked as hidden.
    - The last section is for searching items in your Reporting Services instance. This search allows you to search all items in your instance and the result lists items out in an organized manner:



Figure 7.06

6. Let's search **Sales** and see. The following image shows the result:

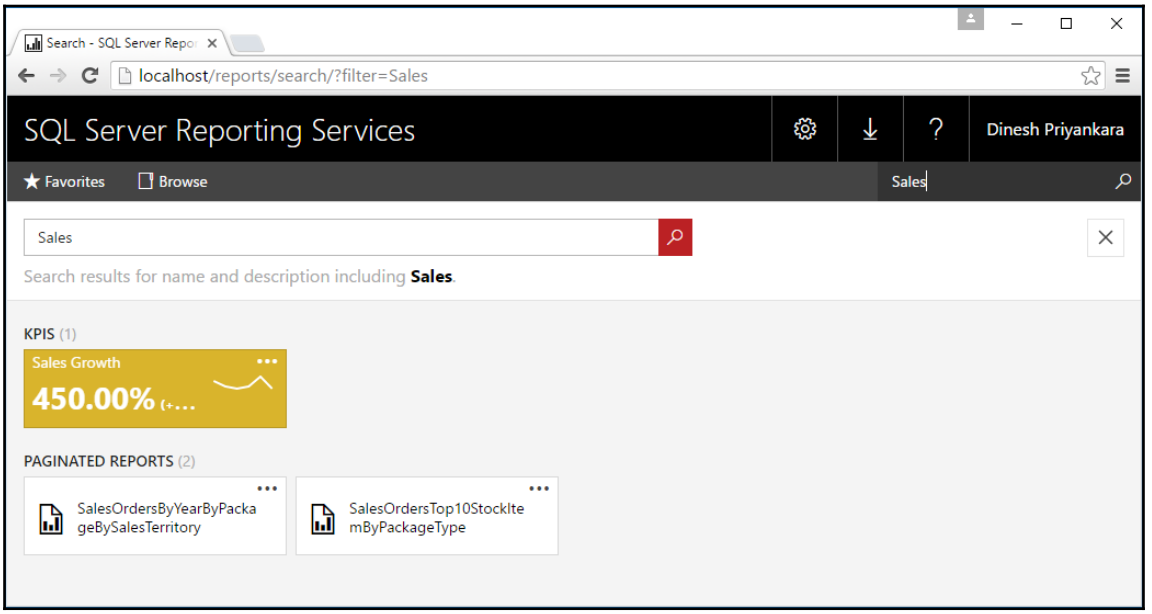


Figure 7.07

7. You can navigate back to **Home** anytime by clicking the `Title`: **SQL Server Reporting Services**. Click on it to bring you back to **Home** page.

8. To upload an Excel workbook, click on the **Upload** icon. You will see the standard **Browsing Dialog box** for selecting a file. Select an Excel workbook you have and upload it.
9. Have a look on the **Home** page now. You will see a new section for Excel named **EXCEL WORKBOOKS**. If you upload a Power BI report, you will see a section for that too:

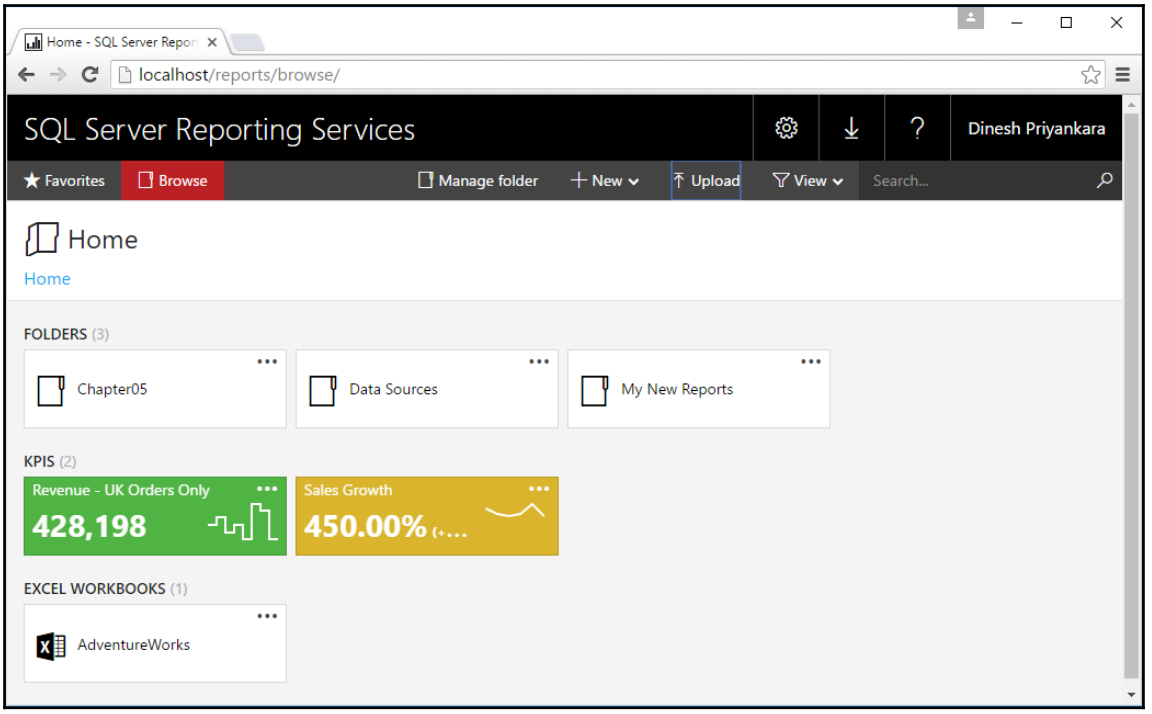


Figure 7.08

10. Set one of the reports as a Favorite and see how it appears. If you have deployed reports related to previous chapters, open one of folders.

11. You will see that reports are shown as tiles and every report has an ellipsis button in top right corner. Click on it. It opens a window that shows information such as created date and modified date. In addition to that, two links appear; one for adding the report as a favorite report and the other for managing the report:

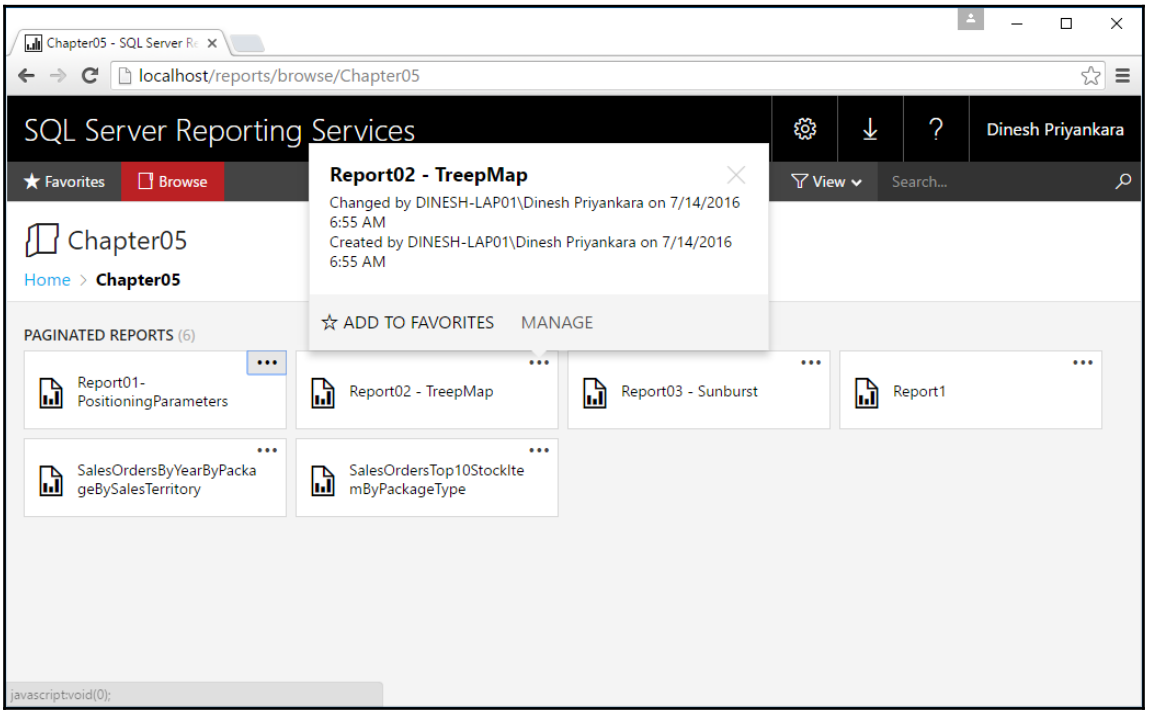


Figure 7.09

12. Click on **ADD TO FAVORITE** and browse the **Favorites** tab. You will see the tagged report in the **Favorites** tab:

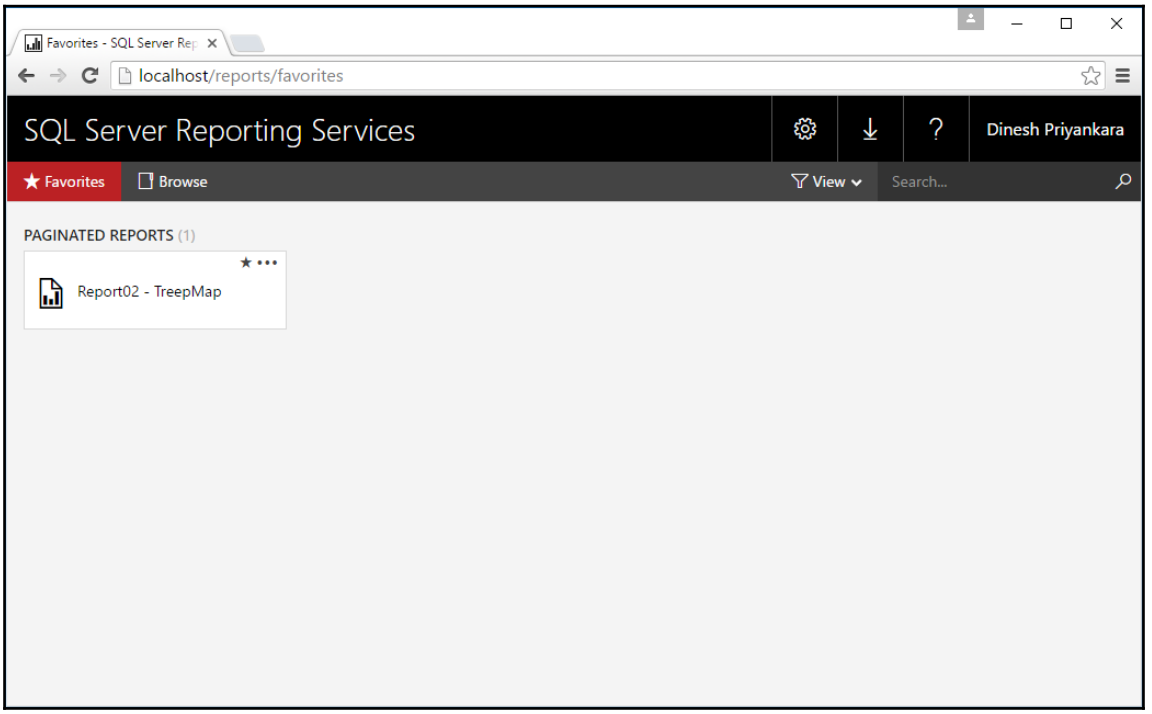


Figure 7.10

How it works…

As you see, the modern Web Portal is richer than what we had with Report Manager in terms of features and functionalities. You will be able to explore and work with features and functionalities based on the permission you have been granted on each item and area.

There's more…

There are lot more items to explore and discuss and that is what we are going to do with the next set of recipes. We will explore the most important items and understand the way of using and implementing them.

Managing reports with the Web Portal

Once a report is uploaded to Reporting Services, it may need some modifications on configurations in order to make the report available for end users. It can be related to data sources, security or even parameter values. One reason for these modifications could be, the environment used for development. During development, you may use different databases in different servers as data sources for the report. You may set testing values as default values for parameters. But they need to be changed once it is deployed to a different environment, generally to production environment. This is why we need to manage reports once they are uploaded.

Getting ready

We can use the Reporting Services Web Portal for managing the reports. It is easier to manage reports using the portal rather than changing configurations either manually or using another editor. Let's see how a report can be managed using the portal by reviewing and changing few configurations.



Note that you can either use the given solutions with the book for testing this or use your own Reporting Services project. Whichever project you use, you need to make sure that project is deployed and you can access it.

We will use the solution `Chapter07.sln` given with the book in the *How to do it…* section. Make sure the project is deployed to the Reporting Services instance.

How to do it…

Let's follow the steps for managing the report:

1. Open the Web Portal and click on the `Chapter07` folder for opening the folder.
2. Now you see reports related to the `Chapter07` project. Let's see how we can manage the `Report01-Sales` report. Click on the **ellipsis** button in the top-right corner of the `Report01-Sales` report:

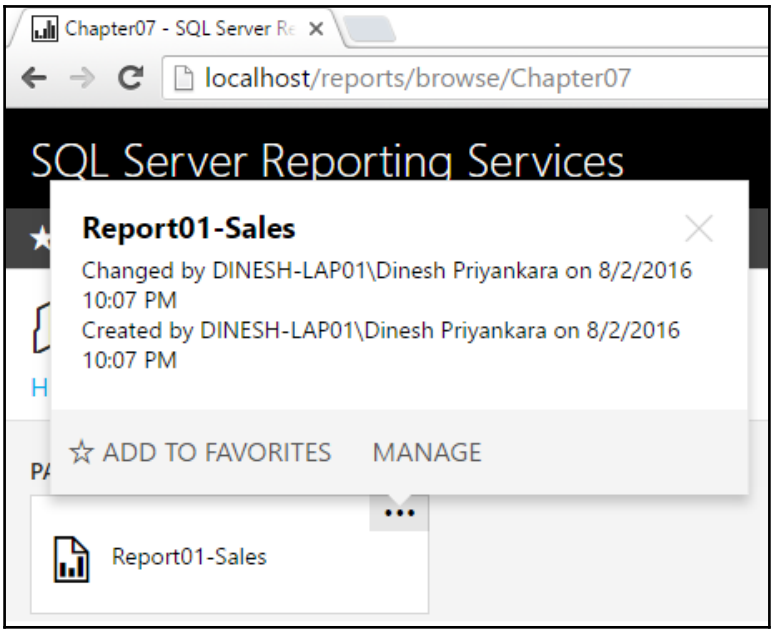


Figure 7.11

3. Click on **MANAGE** (see Figure 7.11) for opening the window that shows various configurations related to the report. This is what you should see when it is clicked:

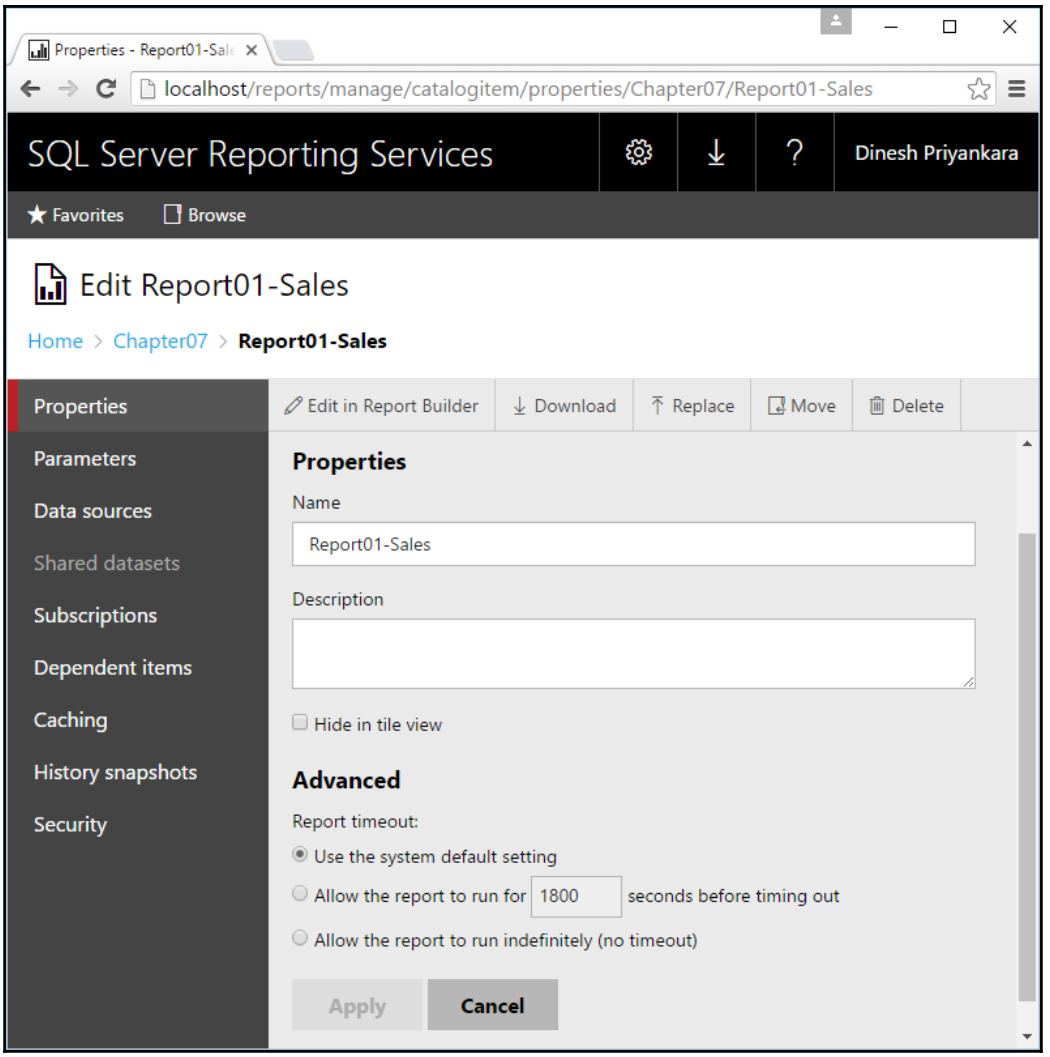


Figure 7.12

4. You can view the window that shows multiple pages in the left pane used for navigating various sections related to the report. Open the **Properties** page to show general properties of the report. Let's make some modifications. Add the **Description** as `This shows sales by Year` (see Figure 7.13).

5. Under **Advanced**, select **Allow the report to run indefinitely (no timeout)**. This makes sure that the user does not get any timeout related issues because of the time it takes for retrieving data from the source (see Figure 7.13).
6. Click on **Apply** to save the changes:

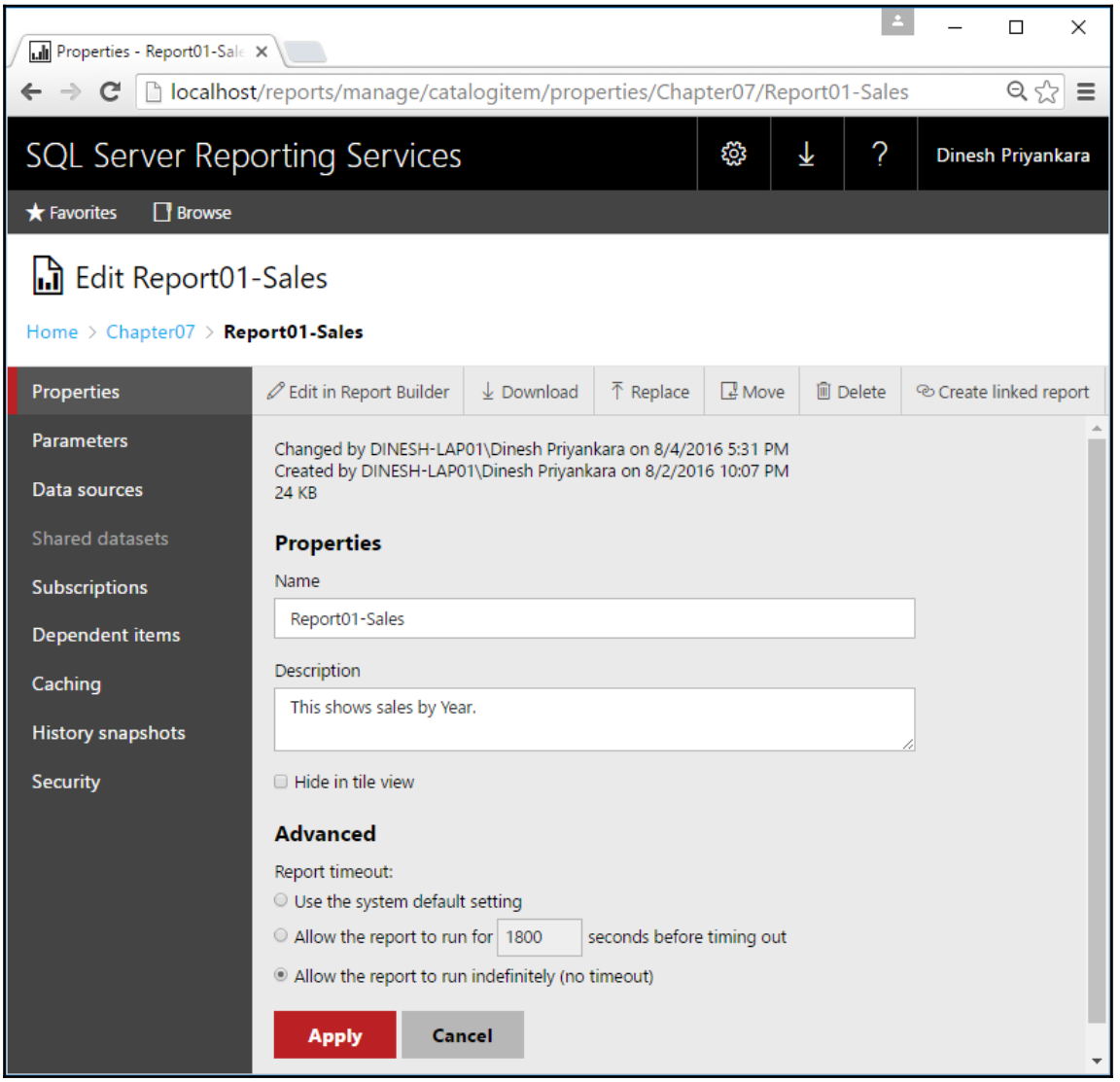


Figure 7.13

7. Click on **Parameters** on the left pane to see the parameters related to this report. This shows all the parameters added and it allows us to make some changes like changing the **Visibility**, the **Prompt**, and setting the **Default value**. Select the checkbox related to the **Use default** column as select 2016 from the **Default value** dropdown:

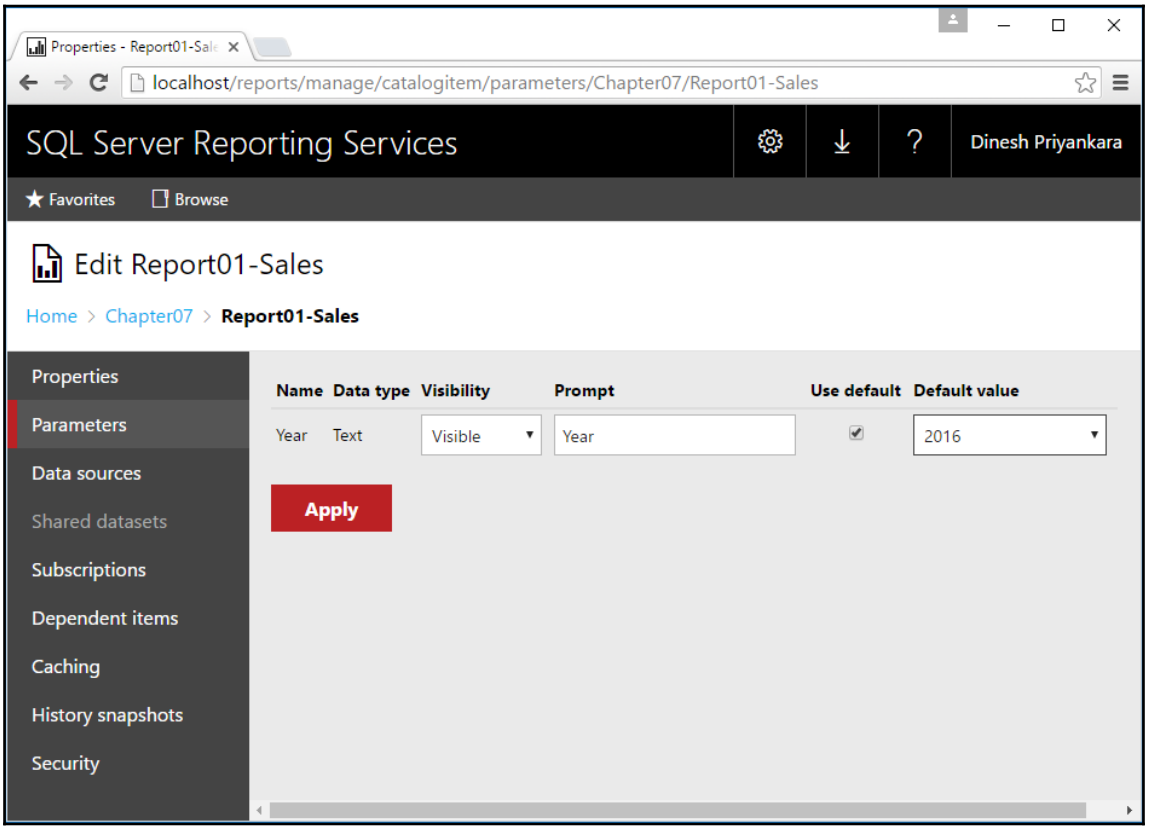


Figure 7.14

8. Click **Apply** to save the changes.
9. Click on `Chapter07` in the bread-crump-trailer. It opens `Chaper07` folder and then click on the `Report01-Sales` for opening the report. Since there is a default value to the Year parameter now, the report will be loaded immediately for the year 2016.

How it works…

When you open the window for managing the report, in addition to the pages in the left pane, a header appear with a set of buttons as shown here:

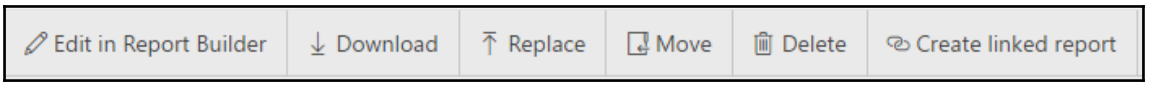


Figure 7.15

If you need to make more modifications that are not supported with this page, you can click on **Edit in Report Builder** and get the report opened in design-view with Report Builder (see `Chapter 4`, *Authoring Reports with Report Builder* for more information on Report Builder).

The **Download** link helps you to download the RDL file in to your machine. This is useful if you need to modify the report using SQL Server Data Tools. You can upload the same report, probably with some modifications, and overwrite the old one using the **Replace** button. The other three buttons; **Move**, **Delete**, and **Create linked report** allow you to perform the operation as the name says.

When you make changes with each page, you need to make sure that the **Apply** button is clicked before you navigate to another page. If not, you will lose the changes you made. Remember, the changes you made are applied only to the report in the portal. The changes will not be reflected in the report in your solution (or project). However, when you deploy the report using the project again, you will not lose the changes you made with the report using the Web Portal.

There's more…

There are some more pages such as **Subscriptions** and **Caching**, which we did not discuss with this recipe. We will be discussing some of these in `Chapter 10`, *Administering and Managing Reporting Services*.

Viewing reports on any web browser

An expected question; someone could ask can the Web Portal be opened with any non-Microsoft web browser? Obviously the answer is yes. Once the features and functionalities are explored using a non-Microsoft browser, another question might be thrown: *will all functionalities work with any browser?* You might be bit reluctant to answer if you do not know the enhancements done in report rendering with Reporting Services 2016 but you can simply answer the question without any hesitation. The answer should be; *yes, it is working with almost all modern browsers*. Interesting? Let' see.

Getting ready

The new **HTML Renderer** added to the Reporting Services 2016 renders to report accurately and it makes sure that reports are consistent across all modern web browsers. The following list shows browsers supported for the Web Portal:

- Microsoft Internet Explorer 10 or 11
- Microsoft Edge; latest version
- Google Chrome; latest version
- Mozilla FireFox; latest version

In addition to these it supports Apple Safari on Apple OS X (version 10.9 or later) and iOS.

In order to test the browser support, we need to have a different browser installed other than Microsoft Internet Explorer. Therefore, download Mozilla FireFox and install it for testing it as we use Mozilla FireFox in the *How to do it…* section.

You can see more details on browser support at: `https://msdn.microsoft.com/en-us/library/ms156511.aspx`.

How to do it…

It is worth testing the portal with a non-Microsoft browser so let's open the portal using Mozilla FireFox:

1. Open **Mozilla FireFox** browser and type the `Web Portal URL`.

2. You may get a dialog box for authentication. Enter the username and password that has permission for accessing Reporting Services:

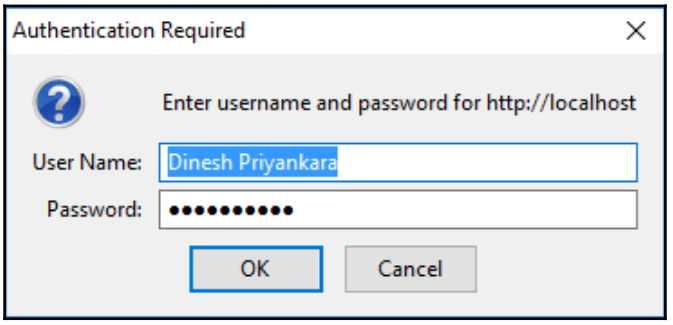


Figure 7.16

3. Now you should see the portal without any issue:

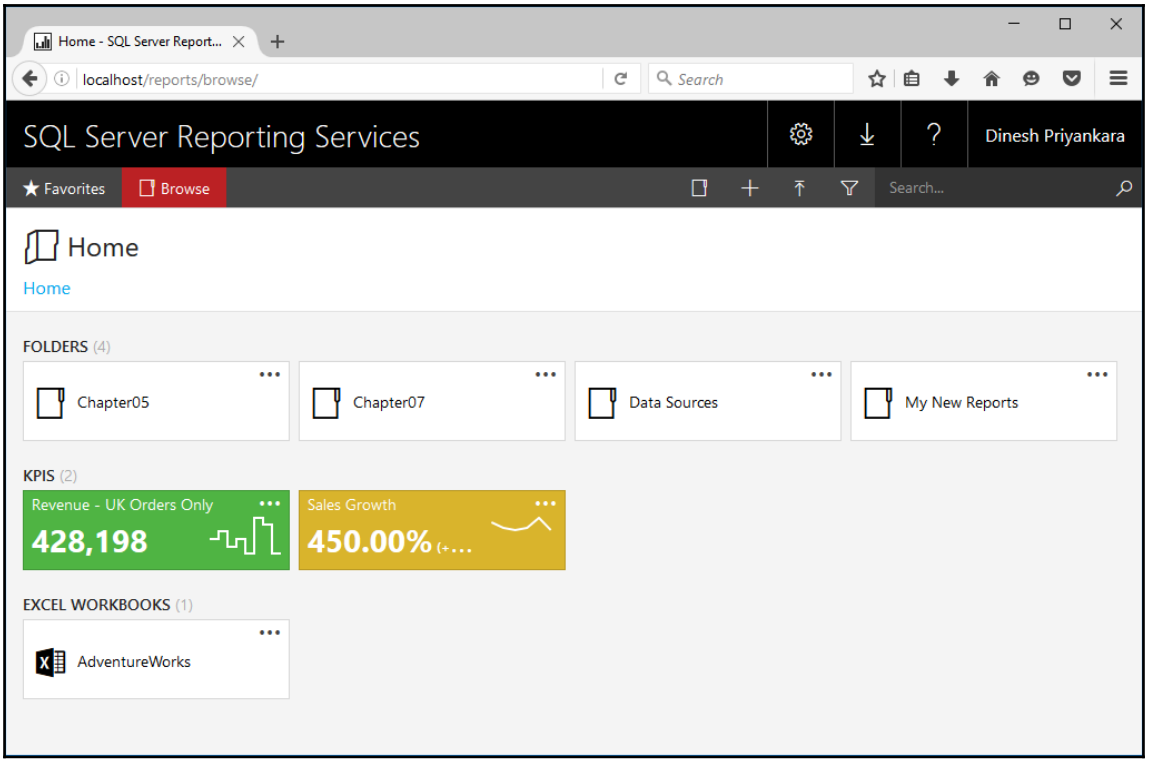


Figure 7.17

4. Open a report. You will see that all the functionalities available with the report work fine and the appearance is intact. Figure 7.17 shows how FireFox renders the report `Report03 – Sunburst` in the `Chapter05` folder:

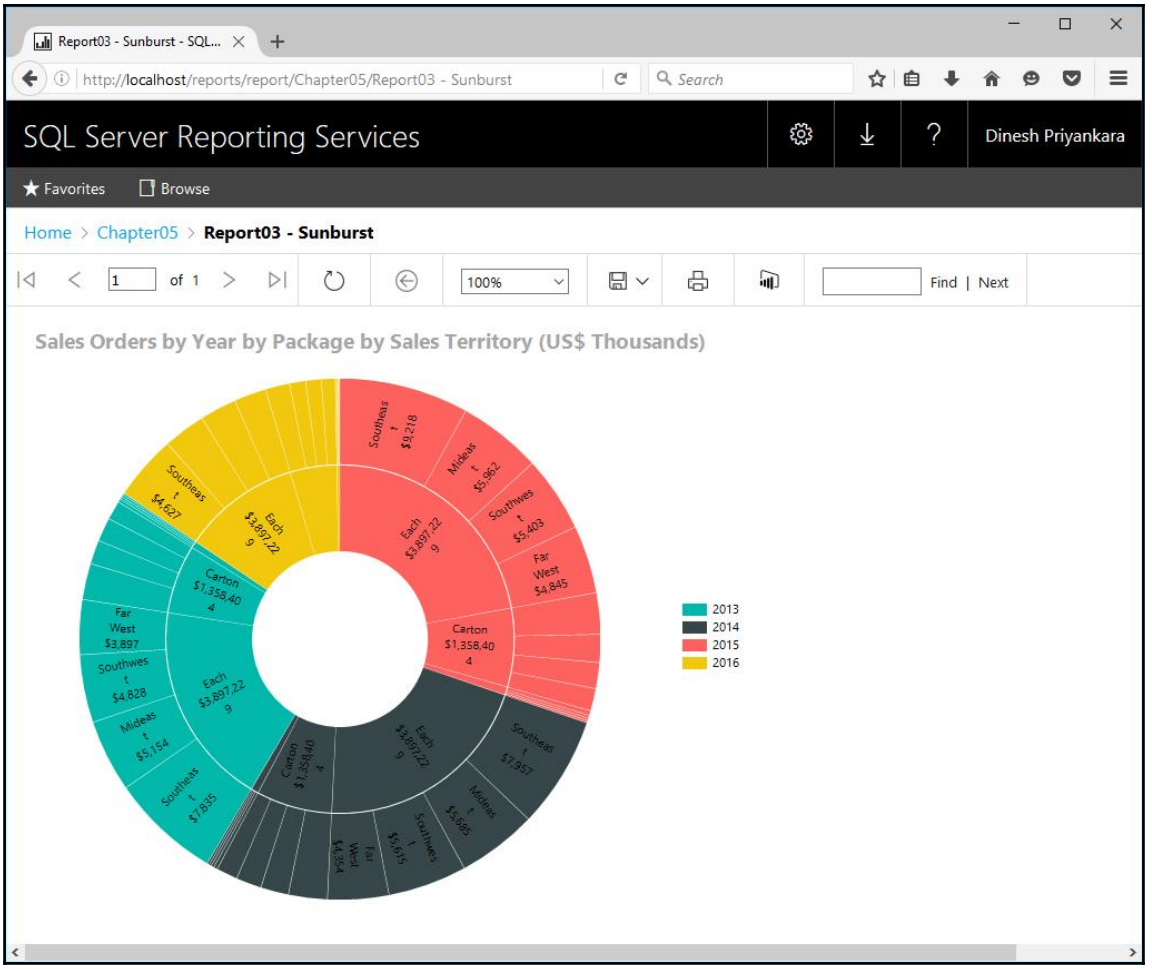


Figure 7.18

How it works…

With the inception of Reporting Services, though there were different types of browsers, we focused more on Internet Explorer and Microsoft has done the same when designing Reporting Services. Because of that, many features and functionalities were limited to Internet Explorer.

In the modern world, web browsing experience has been expanded and standards such as HTML5 and CSS3 have been introduced for optimizing the browsers for modern implementations. Not only that, we cannot expect that everyone uses Microsoft Internet Explorer and we cannot force anyone only for Microsoft Internet Explorer for Reporting Services. Considering all of these, the new renderer is added to Reporting Services is equipped with modern techniques and it generates HTML5 outputs. This is the reason for the consistency of Web Portal across all modern browsers.

Figure 7.19 shows a report rendered with four different browsers:

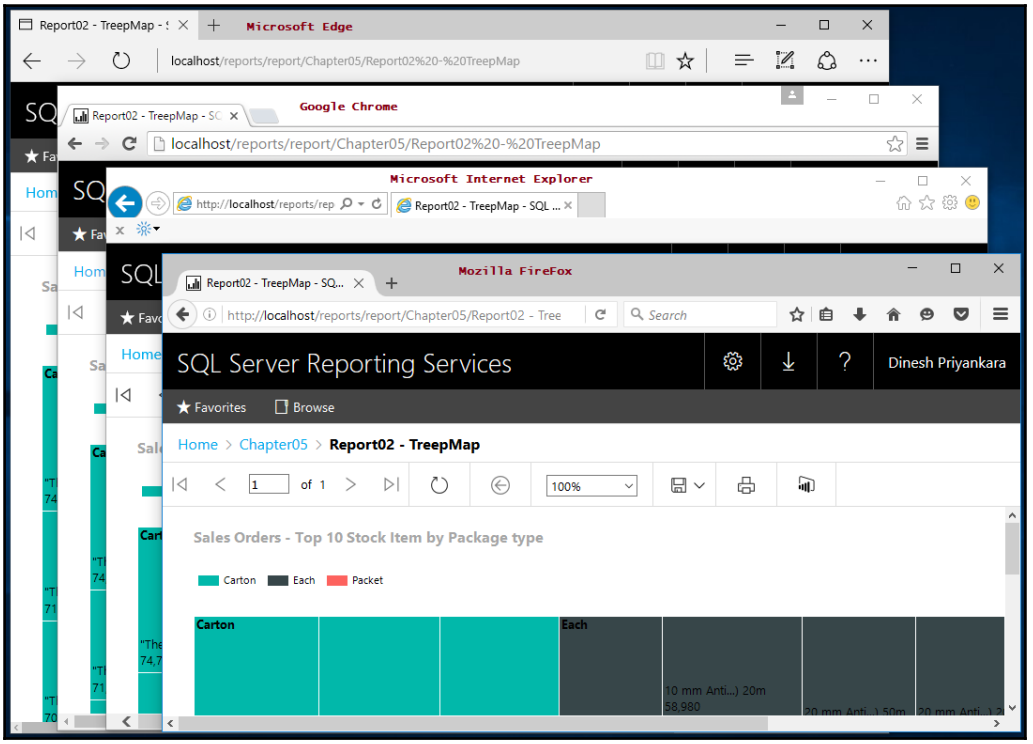


Figure 7.19

There's more…

Remember, though Microsoft Edge supports the new Web Portal, you will experience two known issues based on the version of Edge you have and the data source configuration. If you use an older version of Microsoft Edge and your portal URL refers localhost, you might get an exception. The solution for this is to enable the **Allow localhost loopback** option related to **about:flags** of Microsoft EDGE.

The other issue is integrated authentication. At the time of writing, the Microsoft Edge does not support integrated authentication method. Therefore, if the data source is set with Windows authentication, it will not work. This can be solved by using different authentication. For example, you may use SQL logins for SQL Server related data sources instead of Windows authentication.

Caching the report

When a request is sent by a user for a report to Reporting Services, with default configurations, it needs to extract data from the related source, process and then render. Depending on the volume of data, the complexity of the process and the time it takes for forming the output, it might take few seconds or it might take hours for rendering. If it takes a longer time than expected, it is considered as a performance issue.

Caching improves the performance. Reporting Services can cache the report with initial execution based on the instructions given by us and use the cached copy for stratifying next requests on the same report with the same parameter values if it is a parameterized report. Caching does not help with the first request. For an example, if it takes 5 minutes for opening the report, the next request for the same will be treated from the cache, hence request will be satisfied within a few seconds.

Getting ready

If you are going to enable caching for a report, it is always better to have a label in the report, either in the header or footer that indicates the generated time. This informs the user whether the report output is current or has been previously generated. The report `Report01-Sales` given with `Chapter07.sln` has a label set that shows the generated date and time. Figure 7.20 shows how `Report01-Sales` is displayed for **Year 2016:**

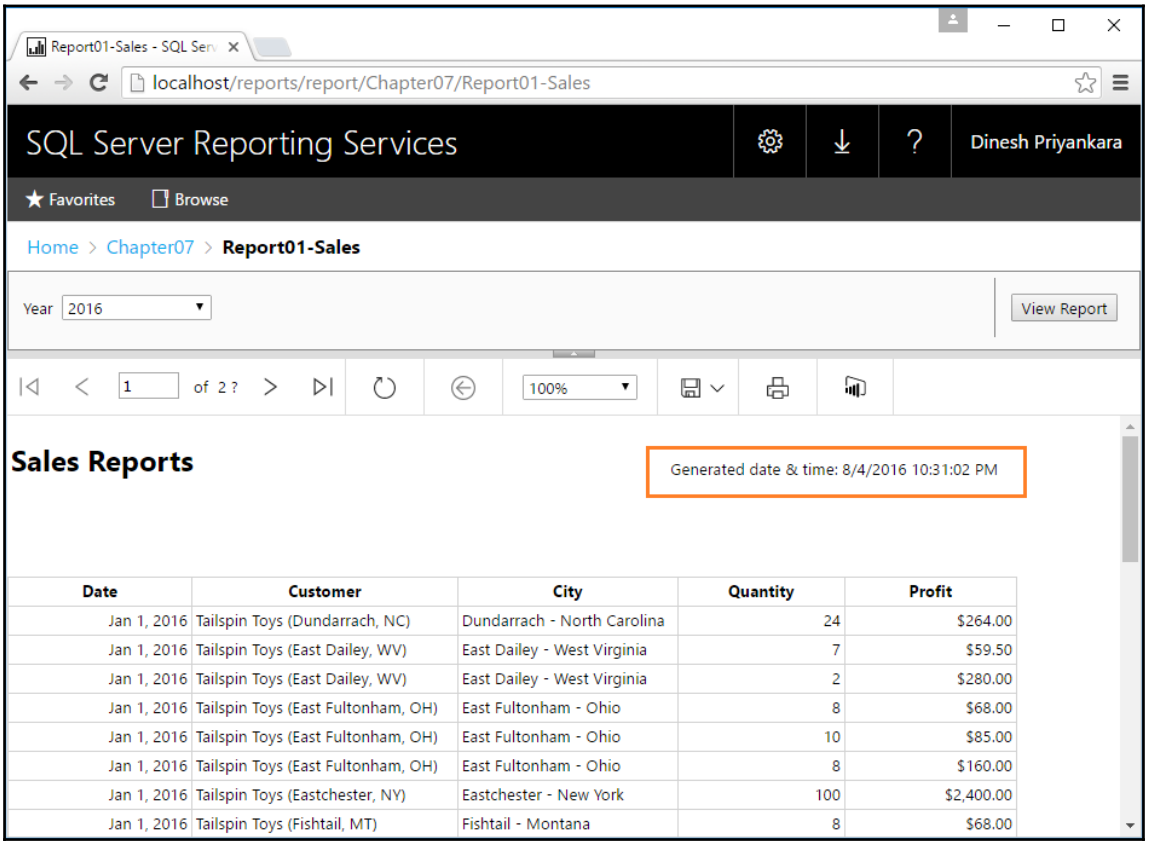


Figure 7.20

In order to enable caching on a report, we can use the new Web Portal that gives us the simple interface for handling the cache. We had this option with the previous release of Reporting Services previous release as well. However, it looked bit cluttered and busy when compared with the new page given. Figure 7.21 shows the older interface. You can compare it with Figure 7.22 and see how cleaner it is and how nicely it is arranged, addressing modern requirements:

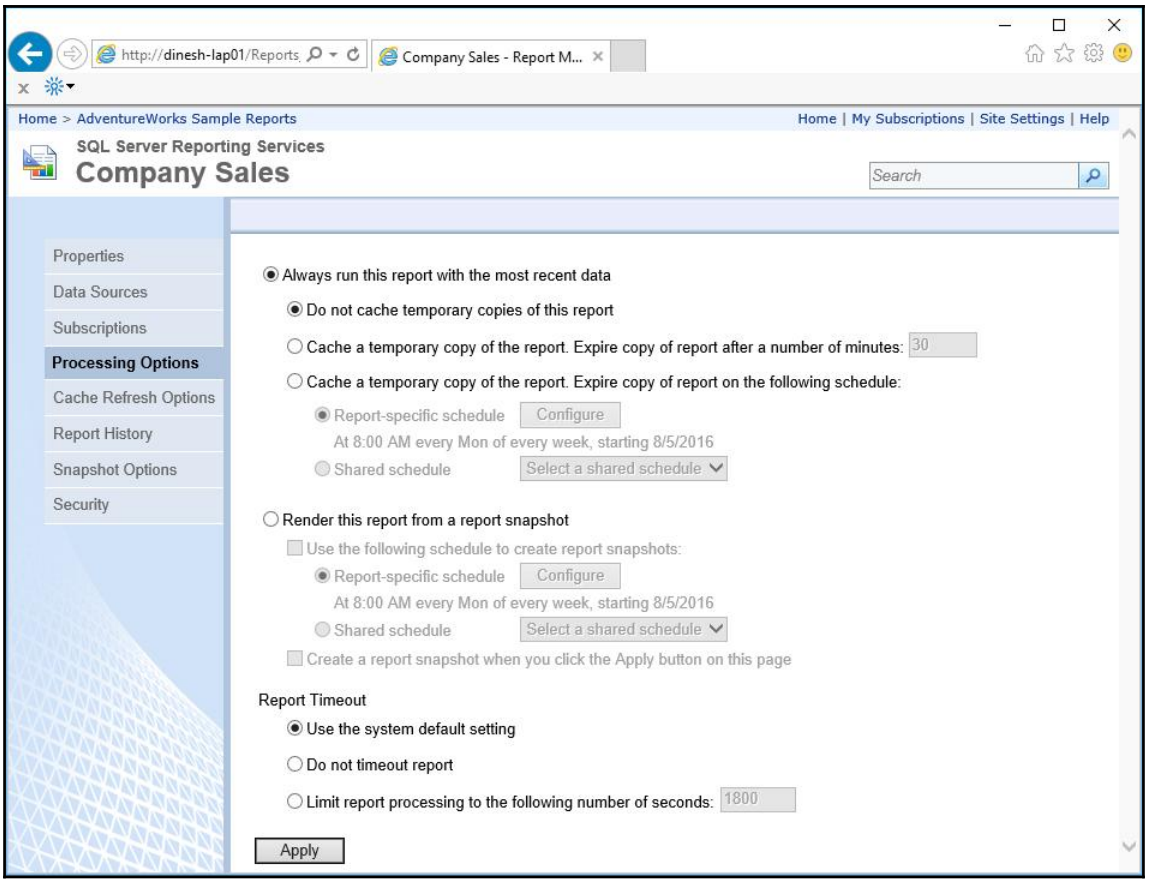


Figure 7.21

Let's see how we can enable caching for this.

How to do it…

Follow these steps for enabling caching for `Report01-Sales`:

1. Open the `Chapter07` folder in the Web Portal.
2. Click on the right-top corner ellipsis button of the `Report01-Sales` report tile and click on **MANAGE**.
3. Click the **Caching** page in the left pane for opening the page related to caching.



The first option is selected by default that makes the Reporting Services engine always retrieve data from the source, process, and renderer.

4. For creating cached copies, select the second option, **Cache copies of this report and use them when available**.
5. Once the second option is selected, a new section called **Cache expiration** appears. Select the first option, **Cache expires after 30 minutes,** and click on **Apply**. The default value for this is 30 minutes but, based on your requirement, you can either set it as 1 minute or a larger number:

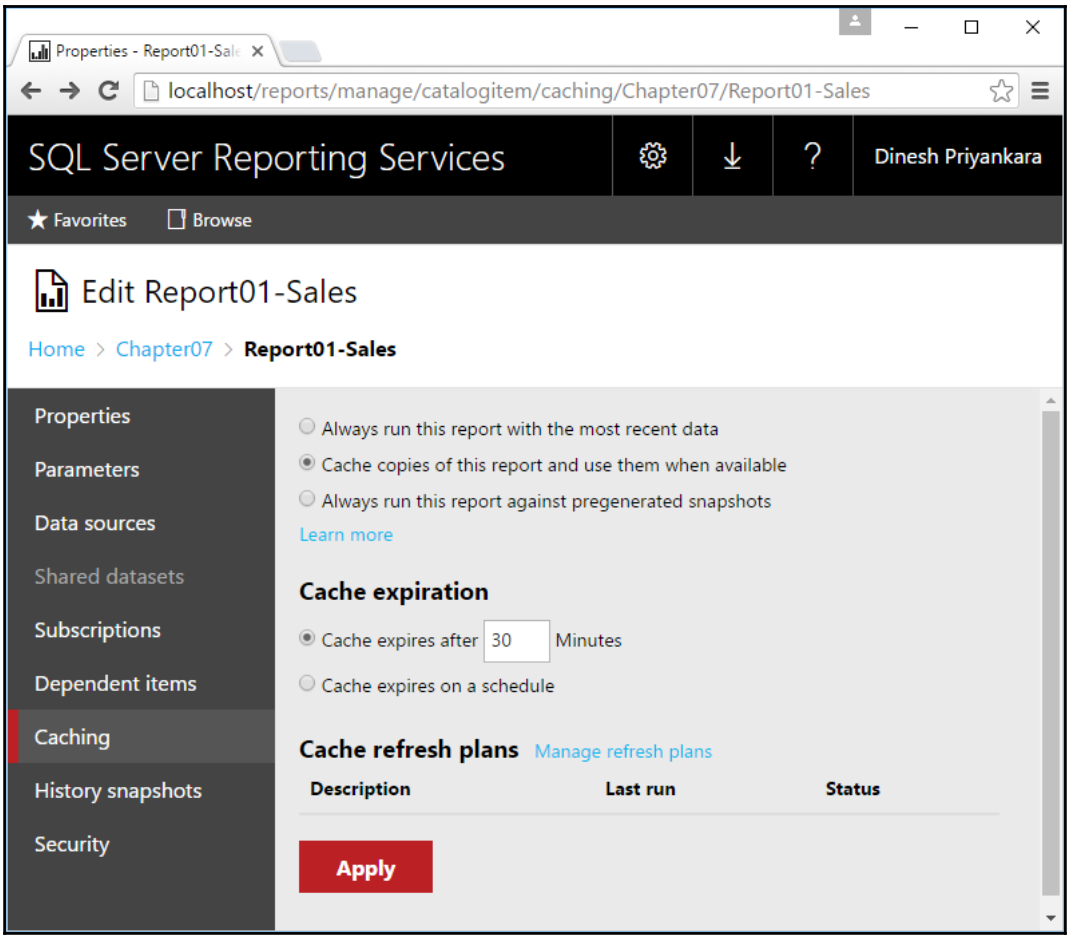


Figure 7.22

6. Open the report and note the date and time generated. As per Figure 7.23, it is **8/4/2016 10:57:28 PM**. With this execution, Reporting Services caches the report for the next request:

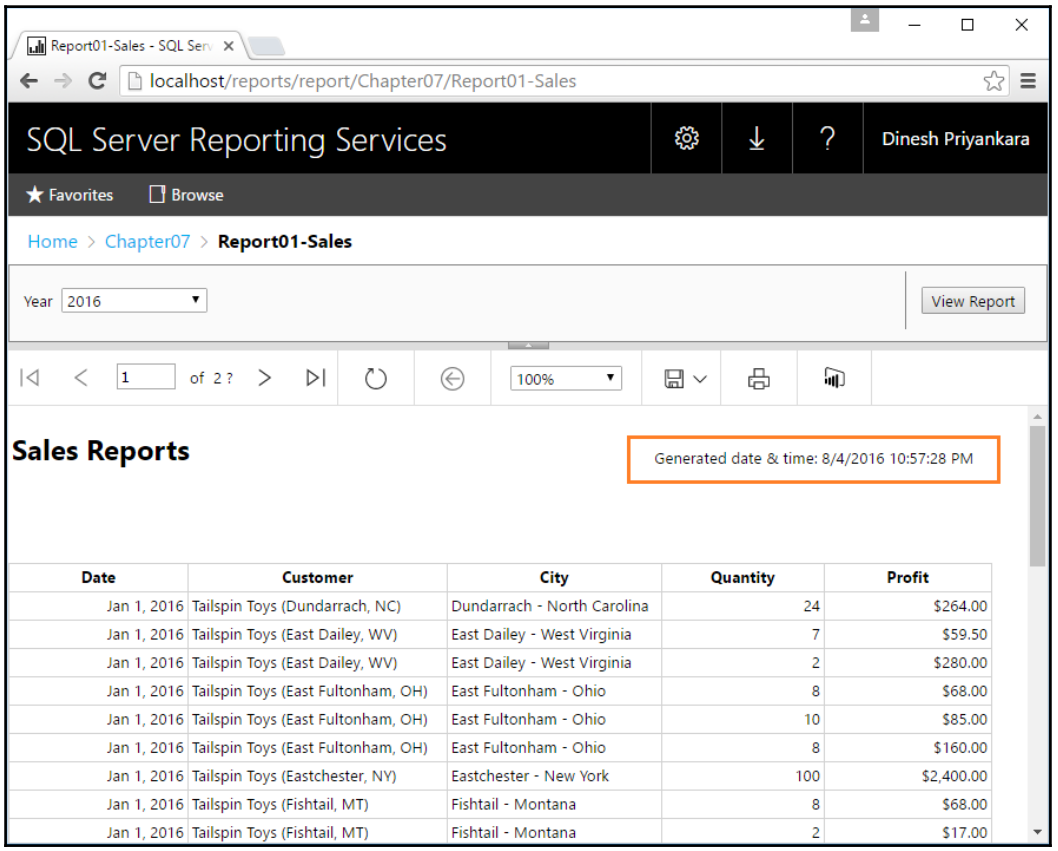


Figure 7.23

7. Generate the report again. You will see that it is quick, and the **Generated date & time** does not match with the current time and it is the same as the date and time shown with the previous output. This indicates that the report is generated from the cache without retrieving records again from the source.
8. Change the parameter value to 2015 and click on **View Report** again. You should see that the **Generated date & time** matches with the current date and time. This indicates that the report is a fresh copy and not based on a cached copy.

How it works…

Caching is not applicable for all reports you have. If you have a report designed for users which shows real-time data, you should not enable caching on a report like this because the user always needs to see new data, not a cached report.

In addition to that, you cannot enable caching on a report that:

- Uses Windows authentication on the sources
- Has user-dependant data
- Prompts for user credentials

This is what you get if you try to enable caching for a report that uses a data source configured for Windows authentication:

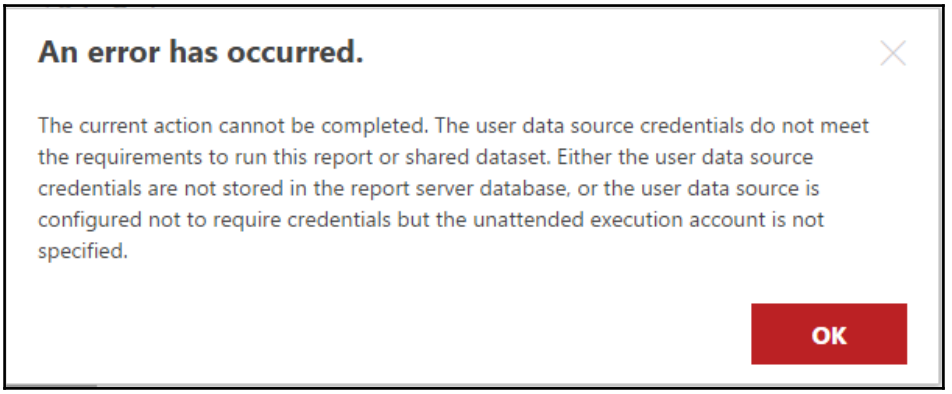


Figure 7.24

As you can see, it caches the report based on the value selected for the parameter. However, if you run the report again with the different parameter value, it will generate the cache for that as well.

Once the caching is enabled, at the first report generation, the formed intermediate format is cached in the `ReportServerTempDB` database. If you query the table called `ExecutionCache`, you will see a record for our report. Generally, it holds one instance of the report but it can keep multiple versions for different parameter values if the report is a parametrized report.

The cached instance expires after it reaches the configured number of minutes. Reporting Services creates another cache with the next request and replaces the old one. There is no direct method given for deleting the cache. However, cached instances will be automatically removed if you perform the following activities:

- Modify the report definition
- Modify the report parameters
- Change the credentials of the data source
- Change the report execution options

There's more…

When you enable caching for the report and select the second option (just like the way we did), there is another option called **Cache expires on a schedule** under **Cache expiration**. Instead of setting **Cache expires after 30 minutes**, you can set the expiration based on a schedule. If you have already created a shared schedule, it can be selected; otherwise you can create a schedule specifically for the report:

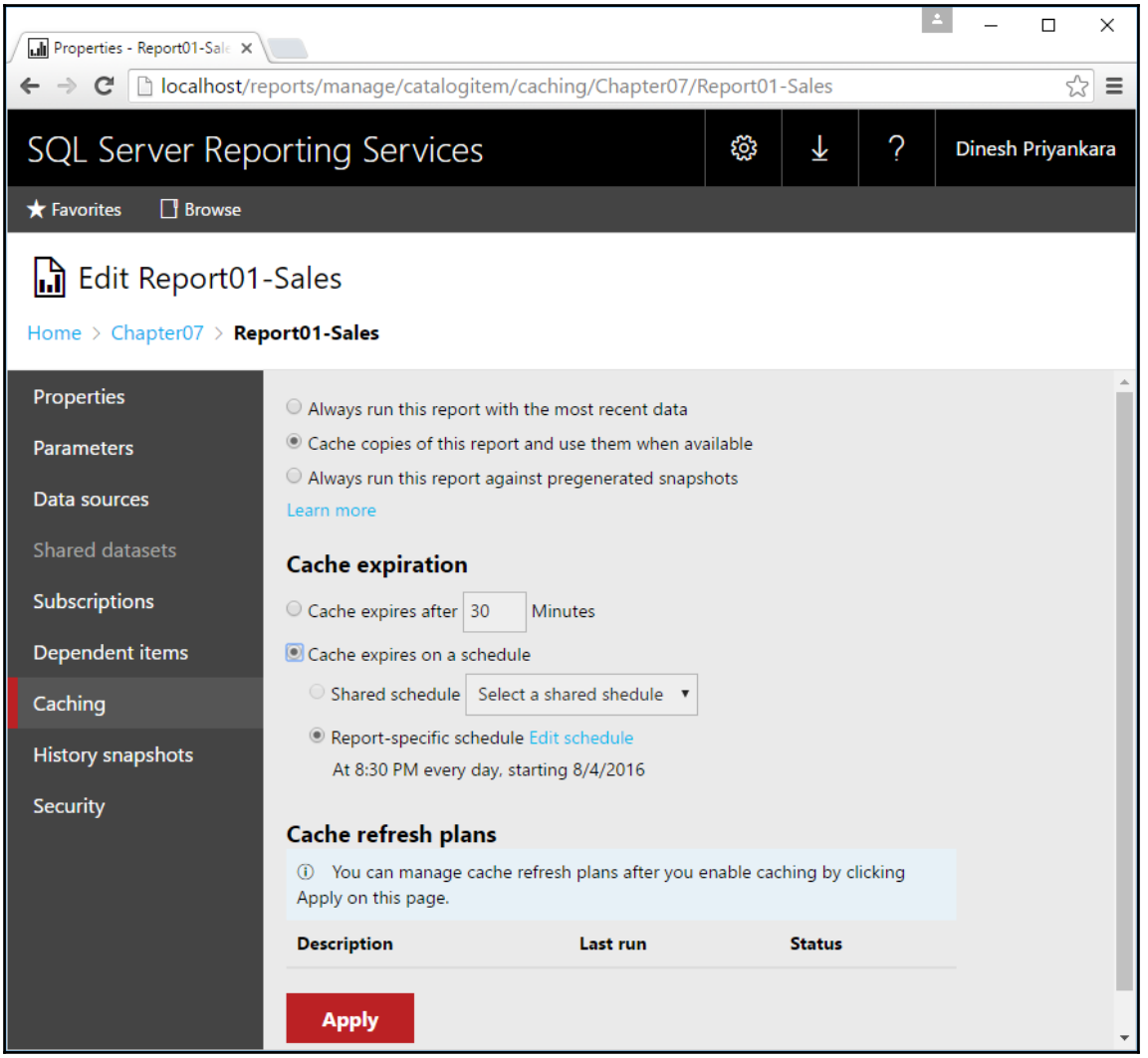


Figure 7.25



Note that if you select a shared schedule and it is paused for some reason, Reporting Services will continuously use the cache instance without expiring it. Not only that, Reporting Services will not create a cache if the scheduled is expired or not available. This makes the report always live and the cache will not be created until the schedule is resumed.

This is good if you expect less data changes in shorter time period. For an example, if the data is getting updated at the end of the day in the data source, you can have a schedule configured as a daily schedule to clear the cache just after the data update.

Refreshing the cache

Did you notice an issue with the previous recipe related to caching? I am sure that you have. The first issue is that it uses the cache continuously if the shared schedules set of expiring the cache is paused. Since the cache does not get expired, users will not get a fresh copy even after the scheduled time.

The second issue is, if the schedule set for expiring the cache is expired or not available, Reporting Services will not create a cache instance for the report again. Since Reporting Services does not to create the cache, performance of the report rendering becomes poor as it always needs to get the required data from the source.

We need to make sure that the caching we have enabled is continuously functioning even with the above mentioned issues. But we need to have an alternative way to get the cache reloaded. The good thing is, we have been given a way, and it is **Cache Refresh Plan**.

Getting ready

The **Cache Refresh Plan** can be configured with the same **Caching page**. You need to decide the plan for refreshing the cache. Assume that you used a shared schedule for expiring the cache as given here:

Starting date: 05 August 2016

Frequency: At 11:00pm every day

Ending date: No ending date

If for some reason, the schedule is paused or not available, when the the user opens the report next day, they will see the cached instance that was created on the previous day. You can reload the cache with the Cache Refresh Plan, configuring the time as 2:00am, making sure that the cache is generated for the day even though the old cache is not expired because of the unavailability of the shared schedule.

How to do it…

Let's create a **Cache Refresh Plan:**

1. Open the `Chapter07` solution and open **Caching** for the report `Report01-Sales`.
2. Make sure **Cache copies of this report and use them when available** is selected (see Figure 7.26).
3. Select **Cache expires on a schedule** and a **Shared schedule**. If you do not have a shared schedule, use the **Site Setting** menu in the **Settings** which is in the header of the Web Portal for creating a shared schedule (see Figure 7.26).

4. In order to create a refresh plan, settings the need to be saved first. Click on **Apply** for saving the settings:

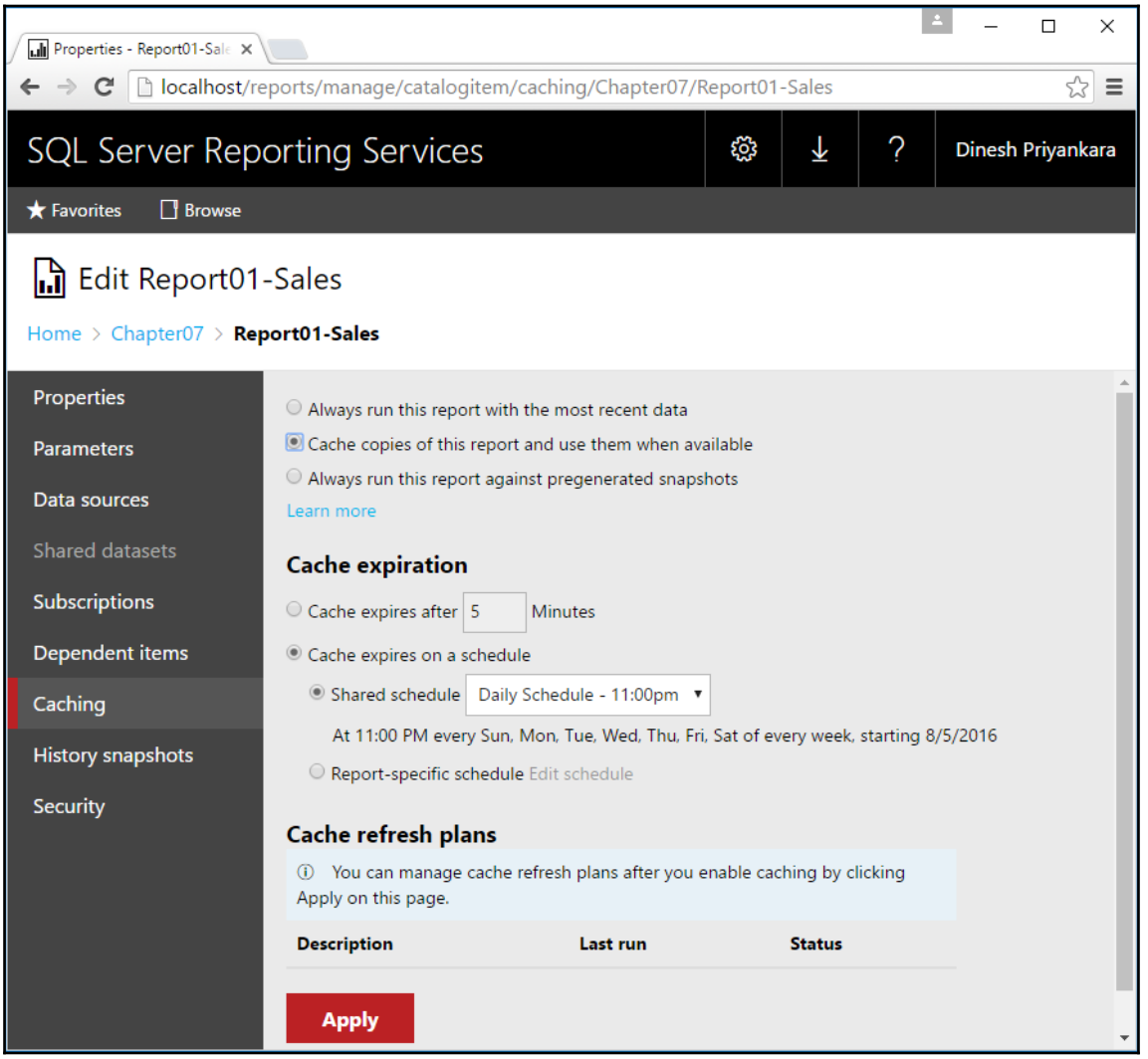


Figure 7.26

5. Once it is saved, you will see an additional link in the same page called **Manage refresh plans**. Click on it:

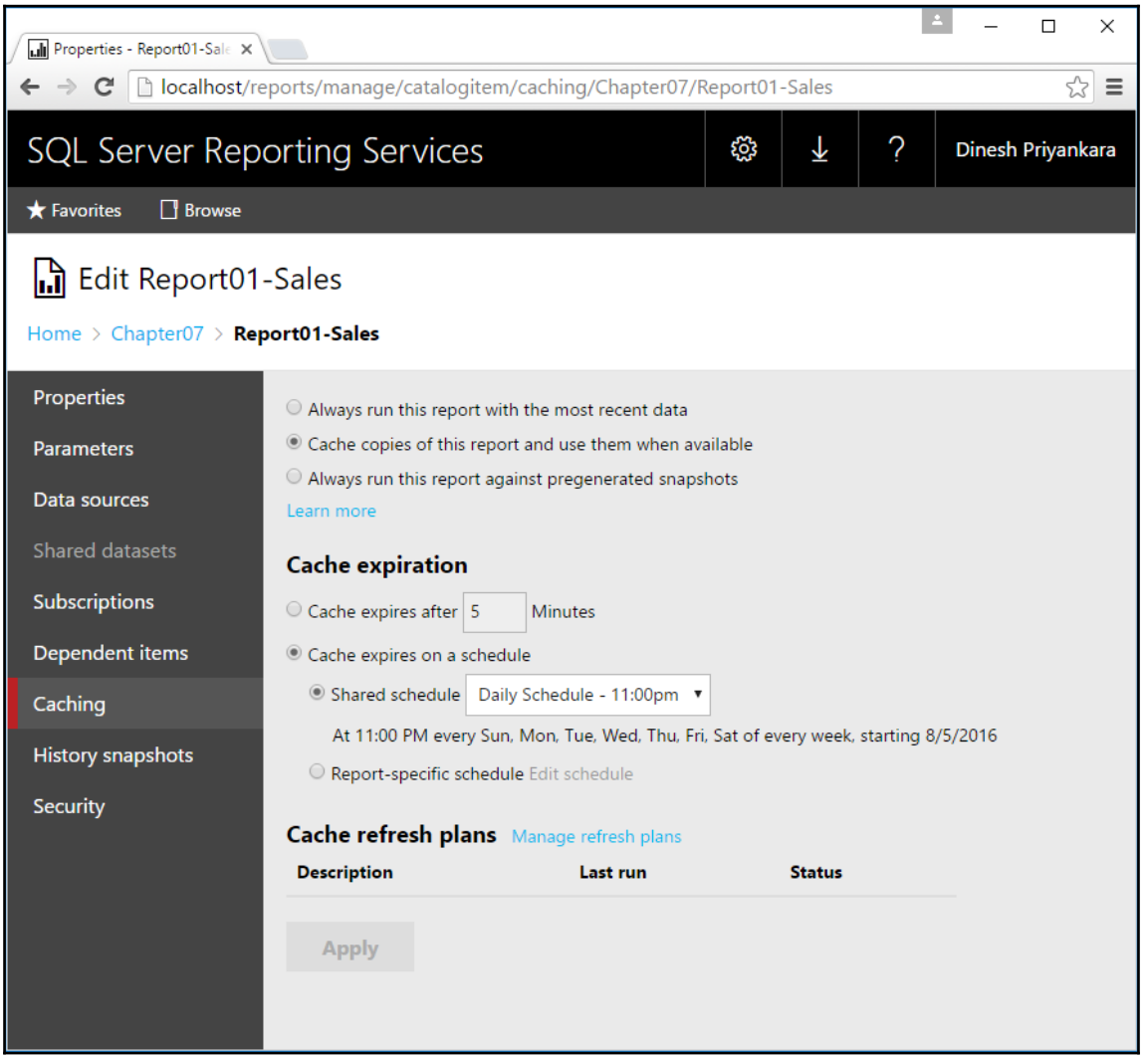


Figure 7.27

6. This link takes you to a new page which can be used for creating refresh plans. Click on **+ New cache refresh plan**:

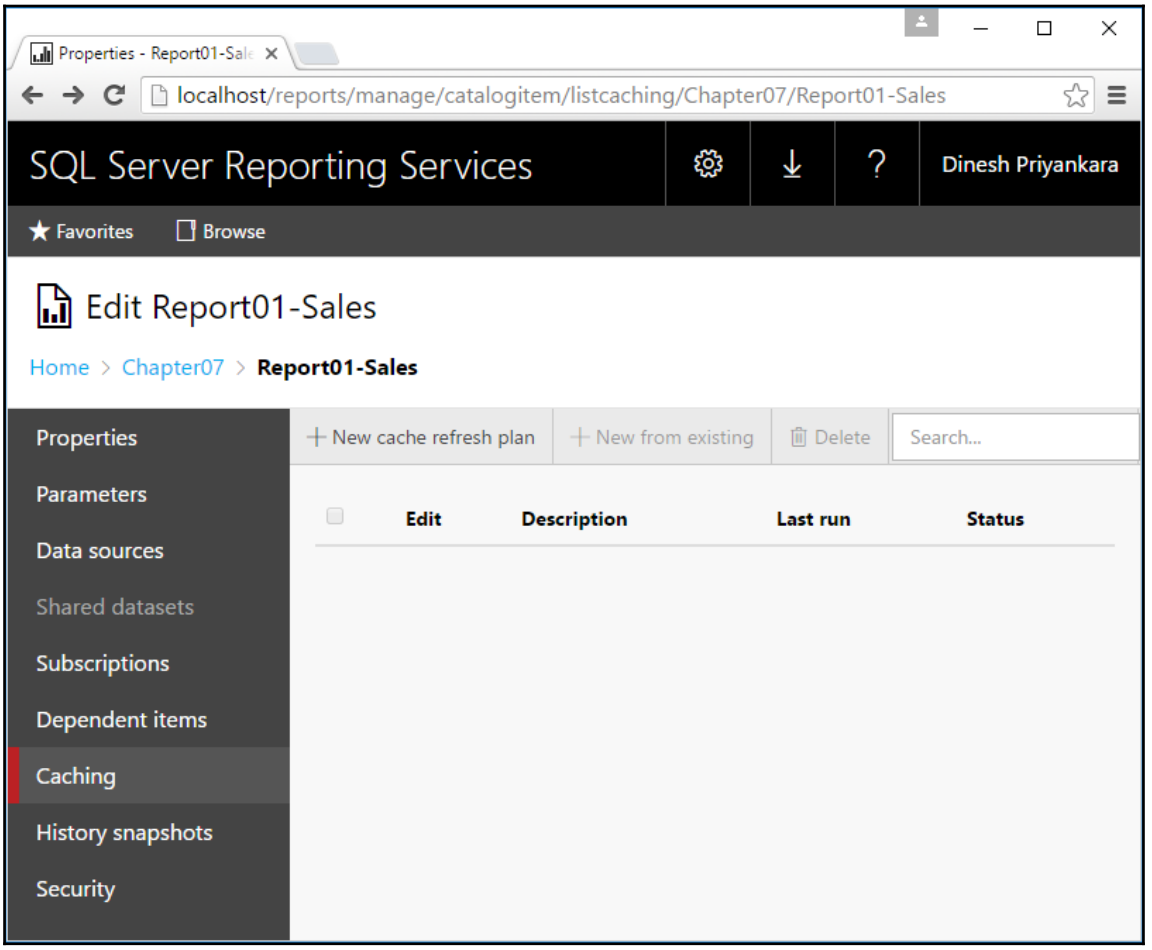


Figure 7.28

7. Create a schedule and click on **Create cache refresh plan**:

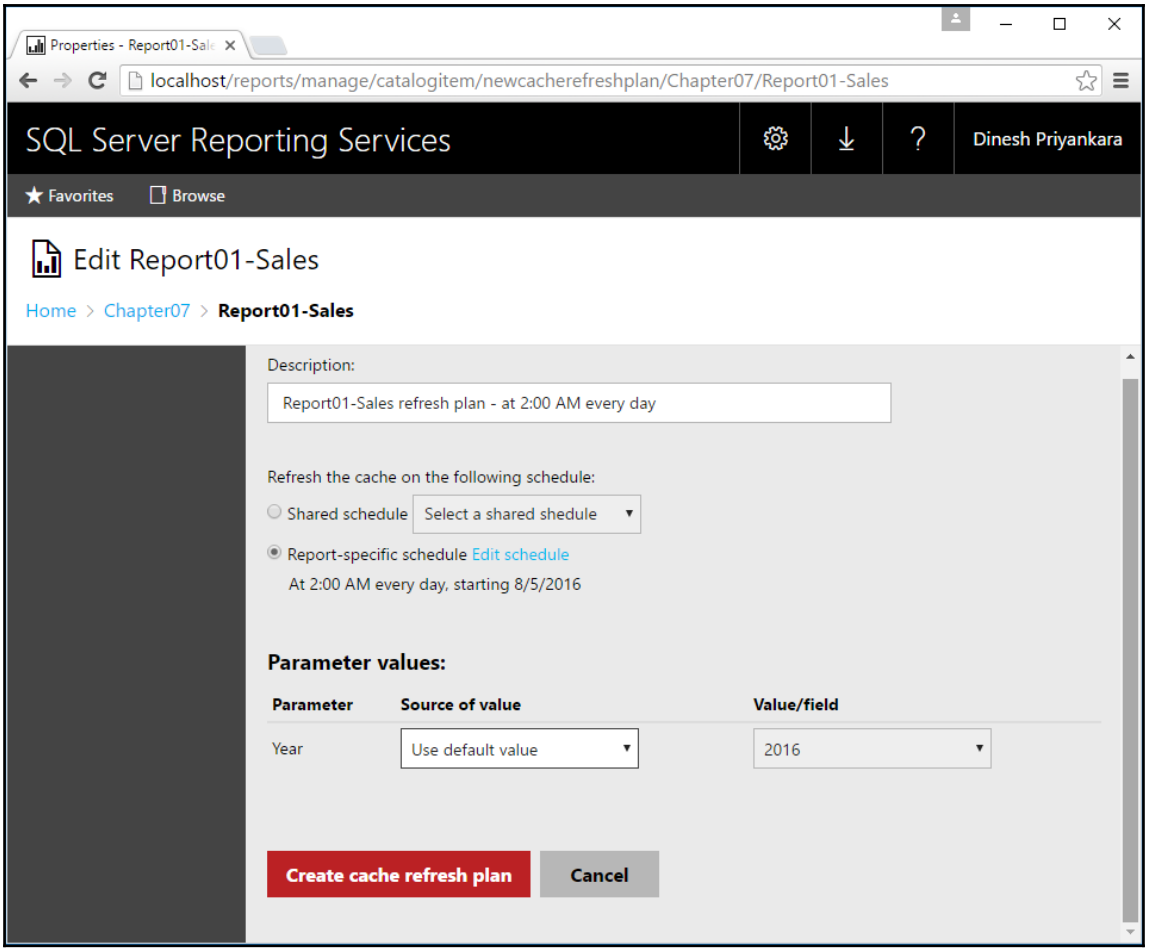


Figure 7.29

8. Go back to the **Caching**. You should see the caching setting as shown here:

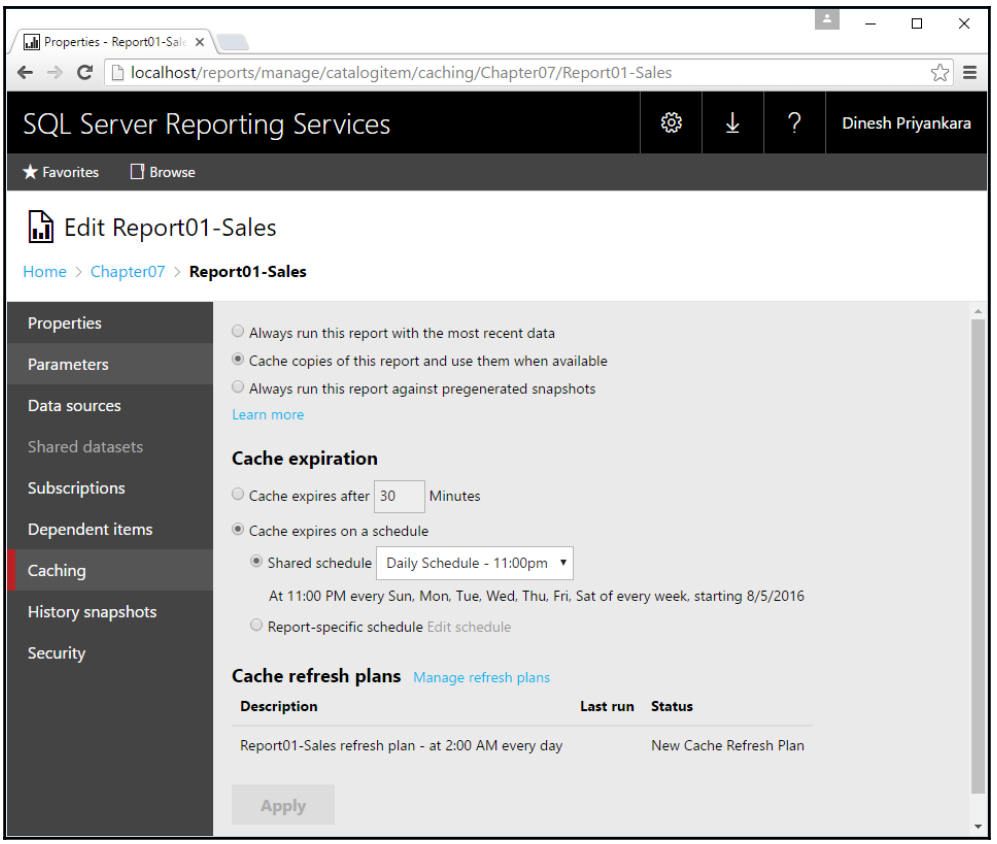


Figure 7.30

How it works…

Now the refresh plan is ready. If for some reason, the shared schedule becomes unavailable or it is expired, we do not need to worry; the configured refresh plan will reload the cache at 2:00am every day.

This gives us an additional advantage too. When the first user opens the report in the morning, he gets the report immediately because Reporting Services does not need to generate the report, it takes it from the cache.

There's more…

There is another option which we did not explore. It is the third option in **Caching** which is **Always run this report against pregenerated snapshots**. This option allows us to create a schedule for generating the cache without waiting untill the report has been opened by the first user. This makes sure that even the first user is treated with the cache and he gets the same speed for opening the report others get:

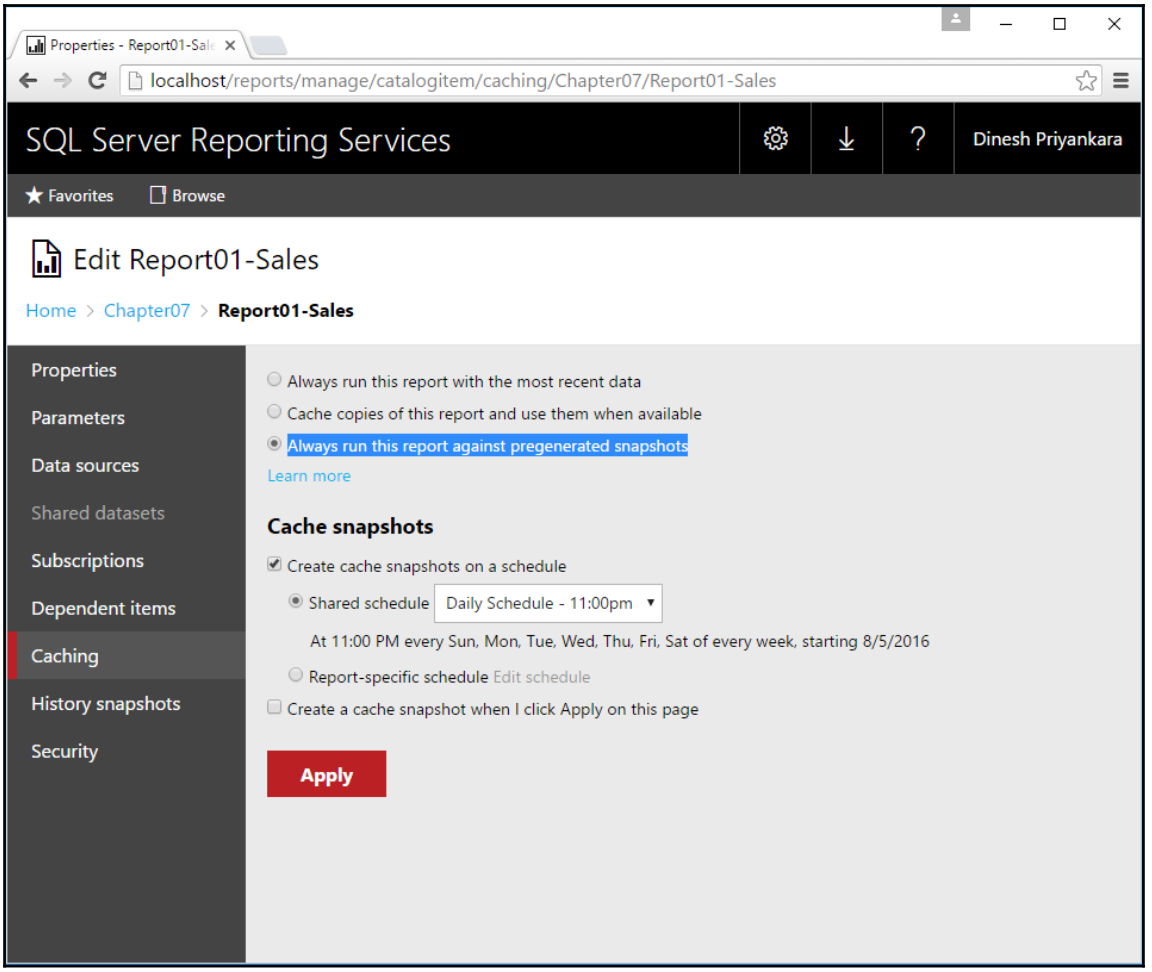


Figure 7.31

Creating a KPI with predefined values

A **Key Performance Indicator** is a measurable value that indicates the progress of the business and whether the target set is achieved or not. Many business users prefer to see the performance of business objectives established using KPIs rather than reading details of many transactional records because KPIs are always shown as visual indicators and it is easy to understand their status.

They are created by business users, not by developers though it is possible with some implementations. There was no direct support specifically designed for KPIs with previous versions of Reporting Services. Generally, the given data regions such as Gauge were used for adding and showing KPIs and they were always shown inside another data region such as Table or Matrix. Microsoft Reporting Services 2016 facilitates us creating KPIs easily and adding them to reports directly without embedding them to another data region.

Getting ready

Remember, you cannot use SQL Data Tools for creating KPIs. The facility has been given with Web Portal and it allows you to manually set values for actual and target KPIs or by using a shared dataset for creating a KPI.

It is always better to create the KPI in one of the folders rather adding it to the root. Therefore, you need to make sure that the selected folder is opened before adding the KPI.

Once the folder is decided, you need to decide the following items for creating the KPI:

- **KPI name**: Name of the KPI. Example, Sales Revenue, Profit Margin
- **Description**: You can add details of the KPI
- **Value format**: This decides how value should be displayed. There is little number of options given such as **Currency**, **Percent** but **General** option works with many cases
- **Value**: This is the **Actual** value of the KPI. It can be either set manually or by using a shared dataset
- **Goal**: This is the **Target** value of the KPI. It can be either set manually or by using a shared dataset
- **Status**: This is the status of the KPI. It can be either set manually or by using a shared dataset. The value should be between 0 and 1

- **Trend set**: This is the set of value for showing the trend. It can be either set manually, separating with semicolons, or by using a shared dataset
- **Visualization**: This is for visualizing the trend. If the trend is required and set, one of the given visualizations can be selected
- **Related content**: This is from enabling drilling through. It allows setting a URL for a report in the Web Portal or a mobile report

How to do it…

As mentioned previously, we need to use the Web Portal for creating KPIs. Follow these steps for creating a KPI, manually setting values:

1. Open the Web Portal and go into the `Chapter07` folder. If you have not deployed the project, you need to do it before hand in order to create this KPI.
2. Click on the **+New** menu in the header section of the Web Portal and select **KPI**:

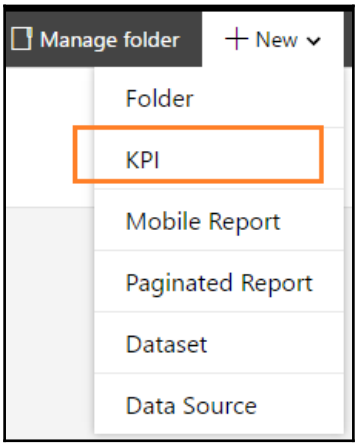


Figure 7.32

3. It opens the New KPI page. Set the name as **Customer Acquisition Rate** (see Figure 7.33).
4. Set the **Value Format** as **Percent**.
5. Select **Set manually** for the **Value** and enter `.12` as the value.

6. Select **Set manually** for the **Goal** and enter `15` as the value.
7. Select **Set manually** for the **Status** and select **0 (neutral)** for the **Enter goal**.
8. Select **Set** manually for the **Trend set** and enter `5;10;11;8;11` as the value.
9. Select the bar chart for **Visualization**.
10. Set **None** for **Related Content**:

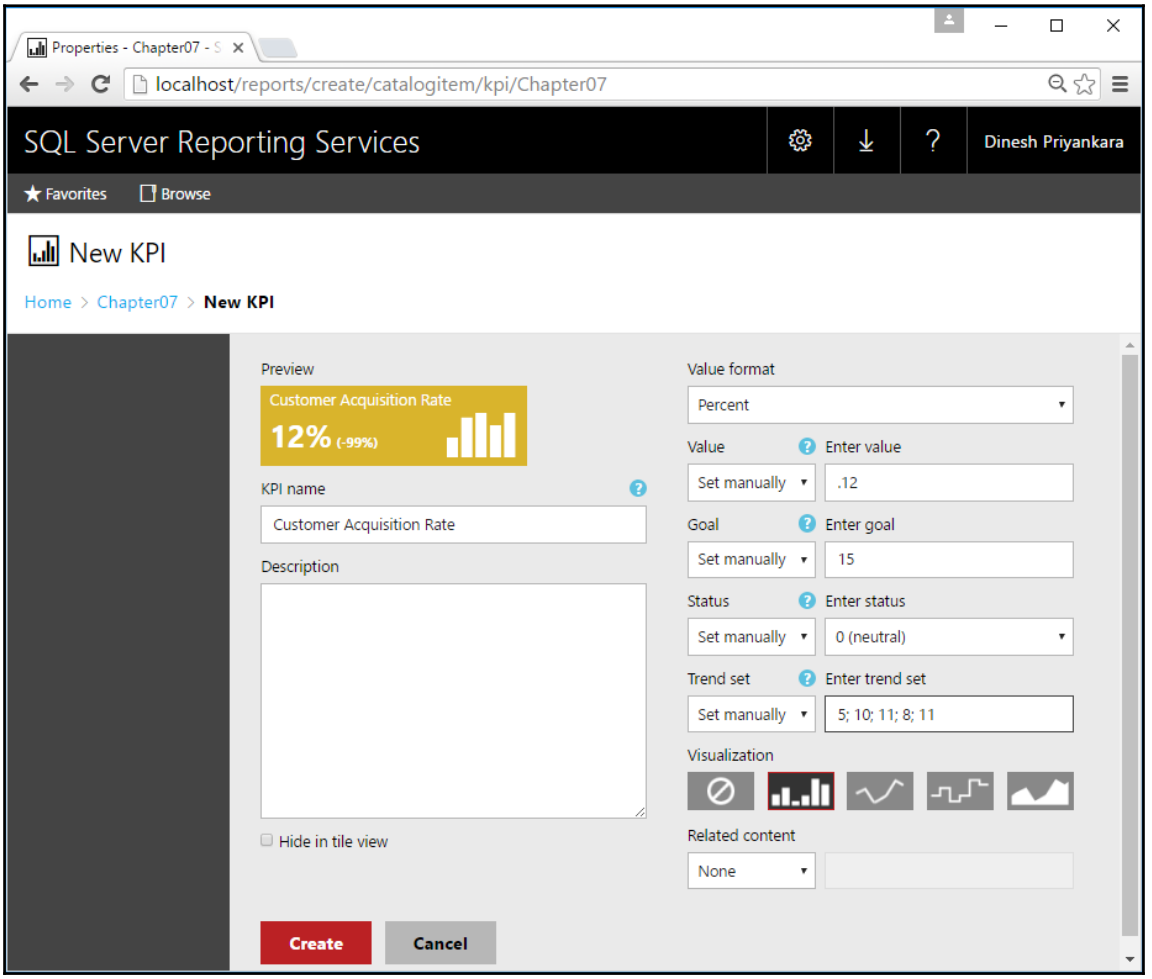


Figure 7.33

11. Click on **Create** for creating the KPI.
12. You should see the KPI created in the `Chapter07` folder. Click on the KPI. You should see it as Figure 7.34:

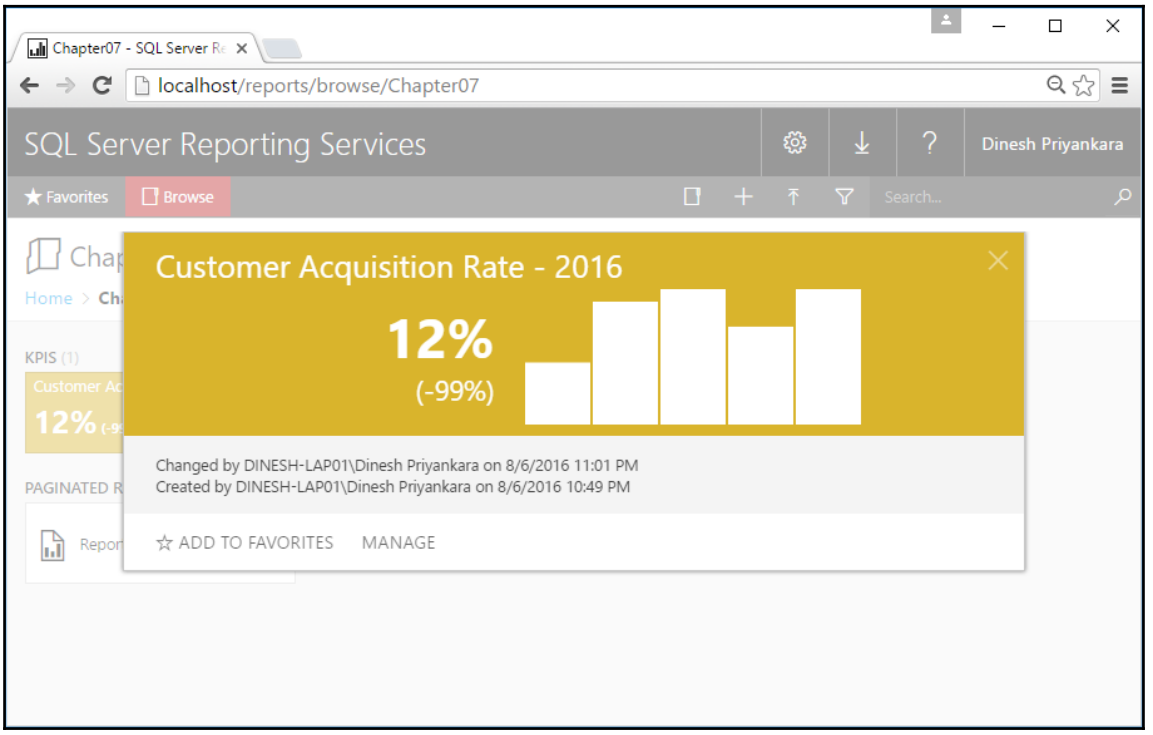


Figure 7.34

How it works…

Since we have set everything manually rather taking them from a data source, values related to the KPI will remain intact until we change them. In order to change, or even to delete, click the right-top ellipsis button and select **MANAGE**. This opens the same interface we used for creating KPIs. If you need to modify the KPI, you can modify and click on **Apply** for saving.

If you need to delete the KPI, simply click on the **Delete** button for deleting the KPI.

There's more…

Although it is possible to create a KPI with hard coded values, in many cases, values need to be continuously extracted from a source rather setting them as hard coded values. This makes sure that, once it is created, it shows the current value whenever it is displayed. Let's see how we can create a KPI with values loading from a source.

Creating a KPI with shared datasets

We have seen the way of creating a KPI with hard coded values. With this recipe, let's see how we can set values from a source rather hard coding them.

Getting ready

The `Chapter07` solution has a shared dataset called `DataSetSales`. We will use this dataset for creating our KPI. This dataset is a parameterized dataset on the **Year** column and **Month Number** column, and it returns the following dataset for the year 2015:

Results | Messages

	Calendar Year	Calendar Month Number	SalesAmount	SalesTarget	Status
1	2015	1	4401699.25	0.00	1
2	2015	2	4195319.25	4621784.21	0
3	2015	3	4528131.65	4405085.21	1
4	2015	4	5073264.75	4754538.23	1
5	2015	5	4480730.55	5326927.99	-1
6	2015	6	4515840.45	4704767.08	0
7	2015	7	5155672.00	4741632.47	1
8	2015	8	3938163.40	5413455.60	-1
9	2015	9	4662600.00	4135071.57	1
10	2015	10	4492049.40	4895730.00	0
11	2015	11	4089208.50	4716651.87	-1
12	2015	12	4458811.25	4293668.93	1

Figure 7.35



Note that values returned from this dataset may vary based on the loaded records into the `WideWorldImportersDW` database.

Since you need this dataset for creating the KPI, make sure that you have deployed this solution to your Web Portal.

How to do it…

Here are the steps for creating the second KPI:

1. Open the Web Portal and go into the **Chapter07** folder.
2. Click on the **+New** menu in the header section of Web Portal and select **KPI**.
3. Set the name as `Sales Revenue` (see Figure 7.41).
4. Set the **Value Format** as **Currency…**. Select **USD** as the **Currency** (see Figure 7.41).
5. Select the **Database field** for the **Value** and click on the ellipsis button for connecting with a shared dataset:

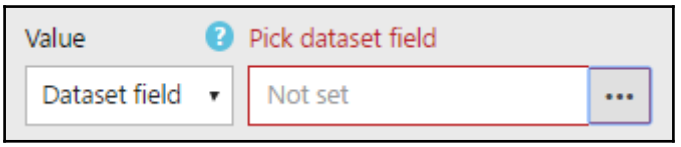


Figure 7.36

6. Click on **Datasets** in the **Choose a dataset** window and open the folder. You should see the **DataSetSales** dataset. Click on it for opening the dataset. Set **2016** as the **@Year** parameter and **7** as the **@Month** parameter values. Click on **Next** to continue:

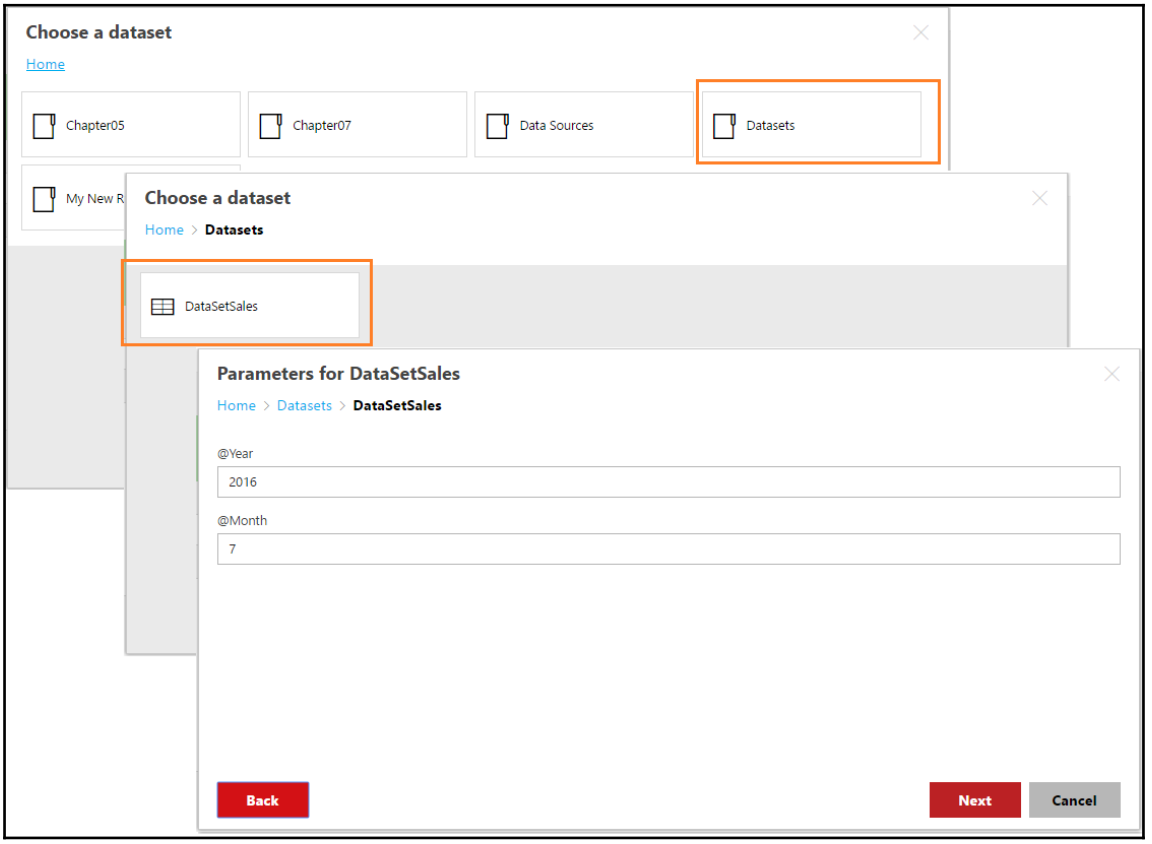


Figure 7.37

7. Select **Sum** for the **Aggregation** and select the **SalesAmount** column. Click **OK:**

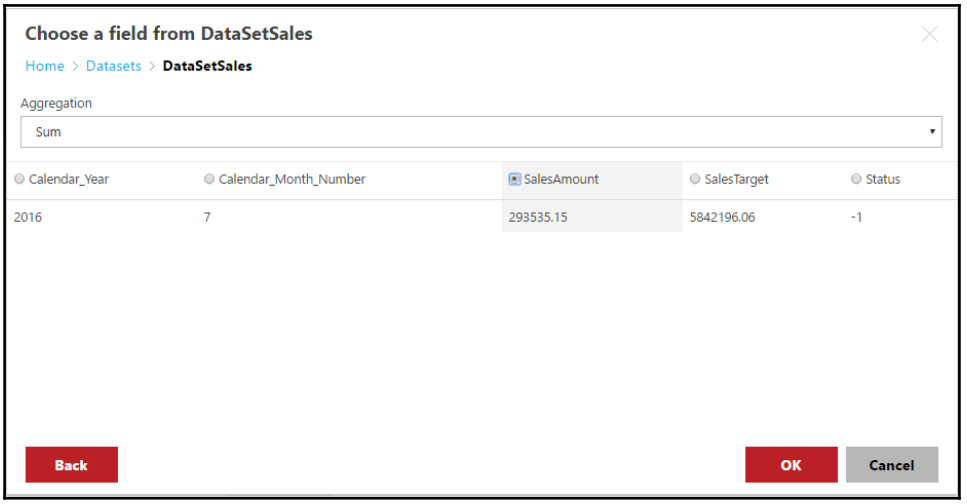


Figure 7.38

8. Select the **Dataset field** for the **Goal** (see Figure 7.41). Click on the ellipsis button for selecting the **DataSetSales** data (as described in steps 5 and 6). Select the **SalesTarget** column and click **OK**:



Figure 7.39

9. Select the **Dataset field** for the **Status**. Click on the ellipsis button for selecting the **DataSetSales** data (as described in steps 5 and 6). Select the **Status** column and click **OK**.
10. Select the **Dataset** trend for the **Trend** set. Click on the ellipsis button for selecting the **DataSetSales** data (as described in step 6 but select 0 as the month). Select the **SalesAmount** column and click **OK**:

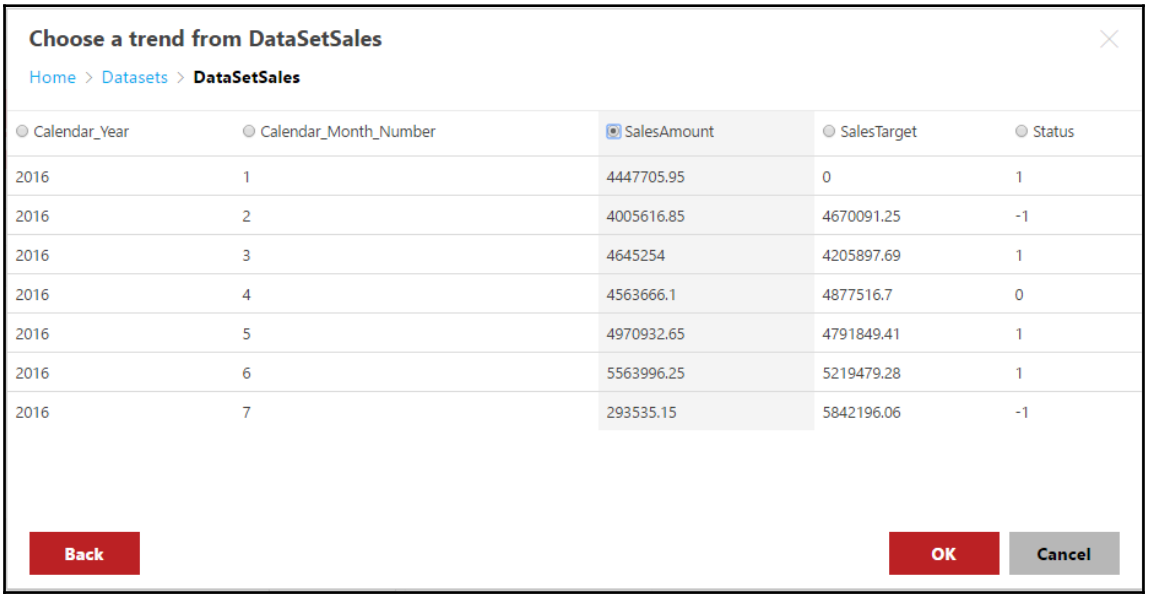


Figure 7.40

11. Select the bar chart for **Visualization** (see Figure 7.41).
12. Set **None** for **Related Content** (see Figure 7.41).

13. Click on **Create** to create the KPI:

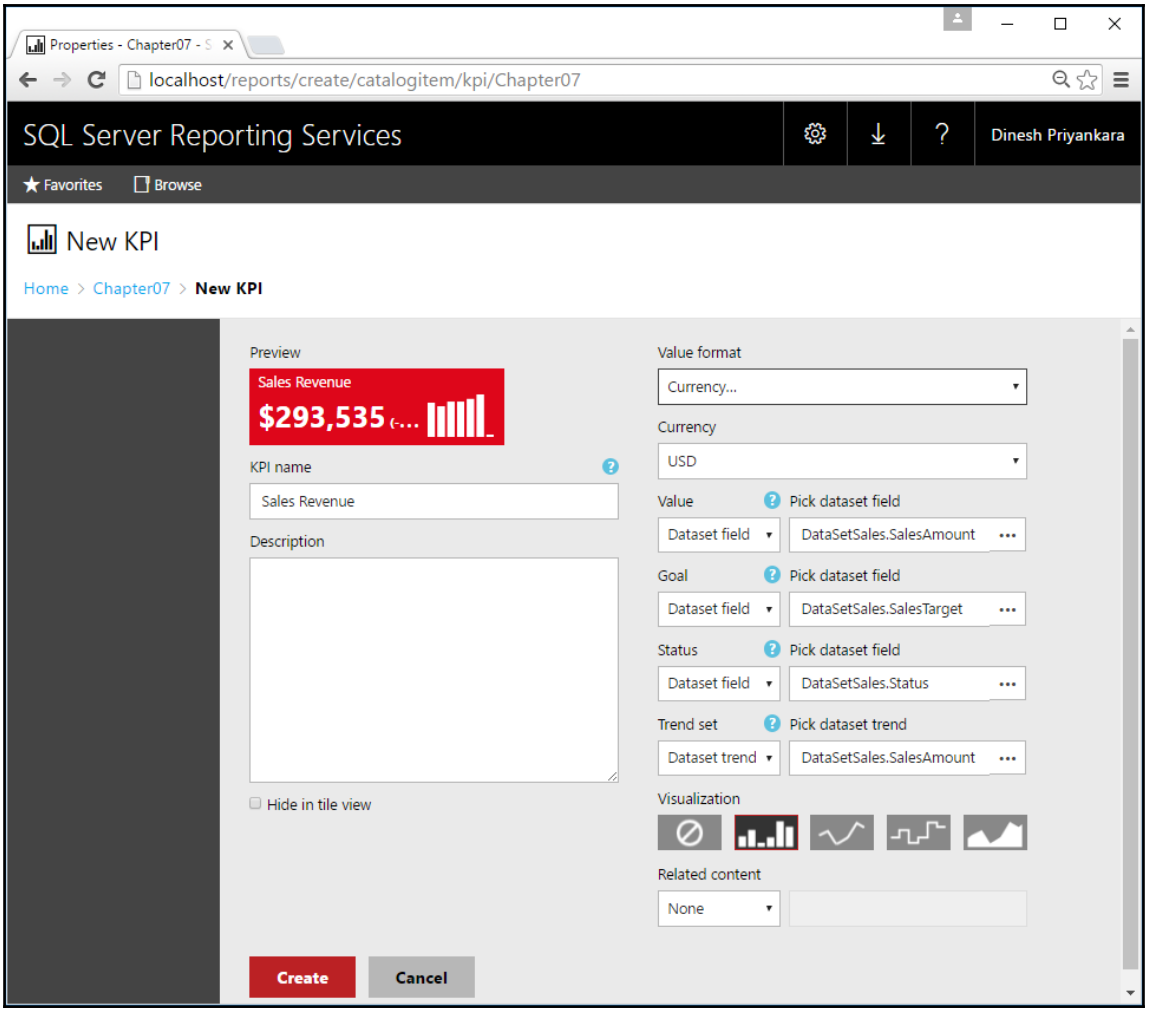


Figure 7.41

14. You should see the KPI in your portal now:

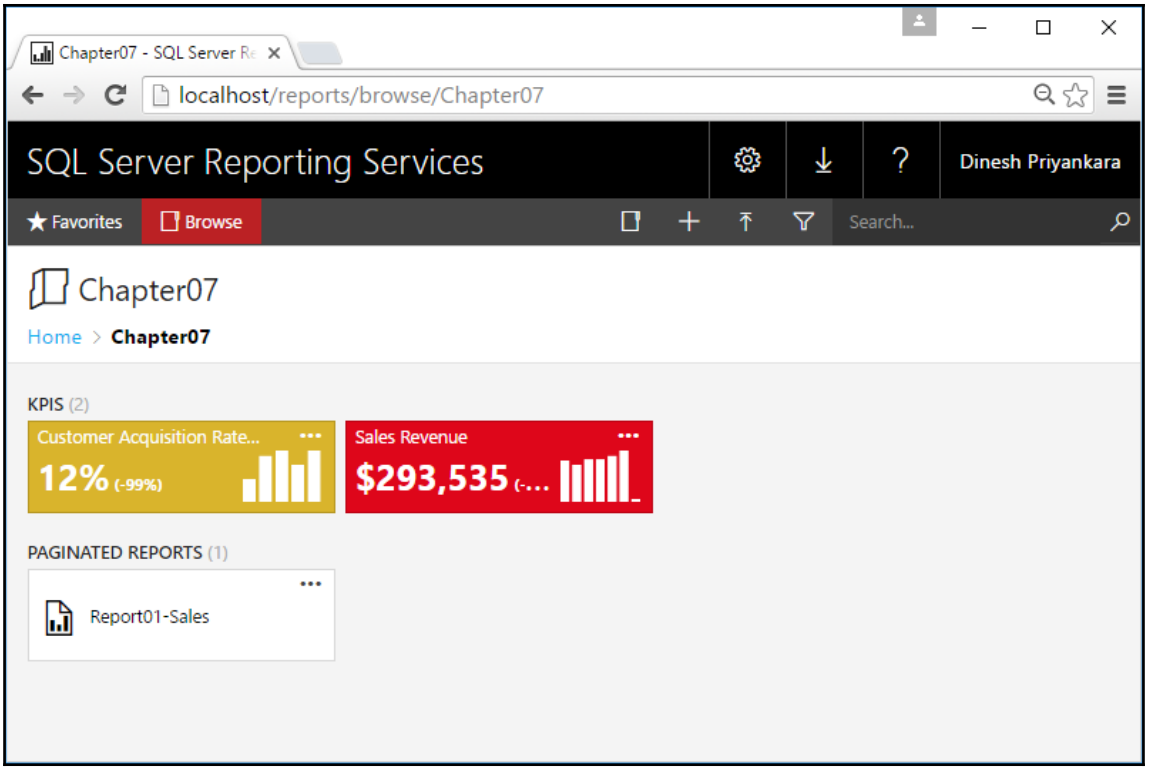


Figure 7.42

How it works…

The **Sales Revenue** KPI created is completely based on the data source. When it is displayed, it queries the database and retrieves required values as per the parameter values set. Even though we have hard coded parameter values, it can be automated by setting year and month based on the current date if this needs to be shown for the current month.

If it takes a considerable time for extracting the data, we can enable caching for the **DataSetSales** dataset. We can have a schedule for expiring the cache and have a **Cache Refresh Plan** for refreshing the dataset.

There's more…

Note that we did not set **Related Content** for both KPIs created. This can be used for setting **drill-through** functionality for showing additional related data to the KPI.

We can use the Report02-SalesWithTargets report for setting this property for the **Sales Revenue** KPI as shown in the following image:



Figure 7.43

Once it is set, the link is displayed when the KPI is open:

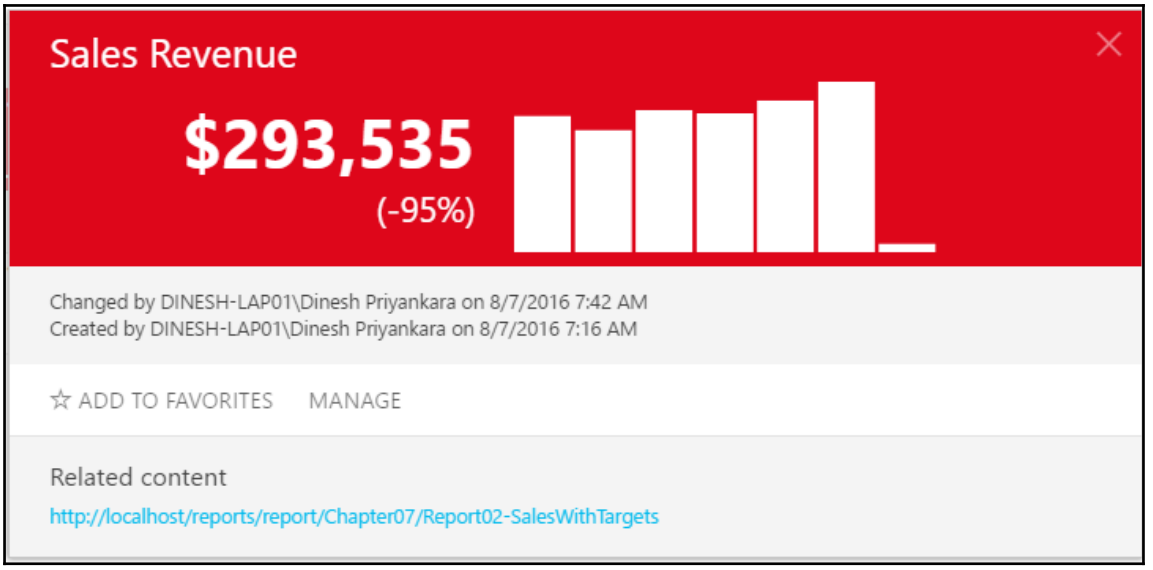


Figure 7.44

Users can see more details by clicking the link set:

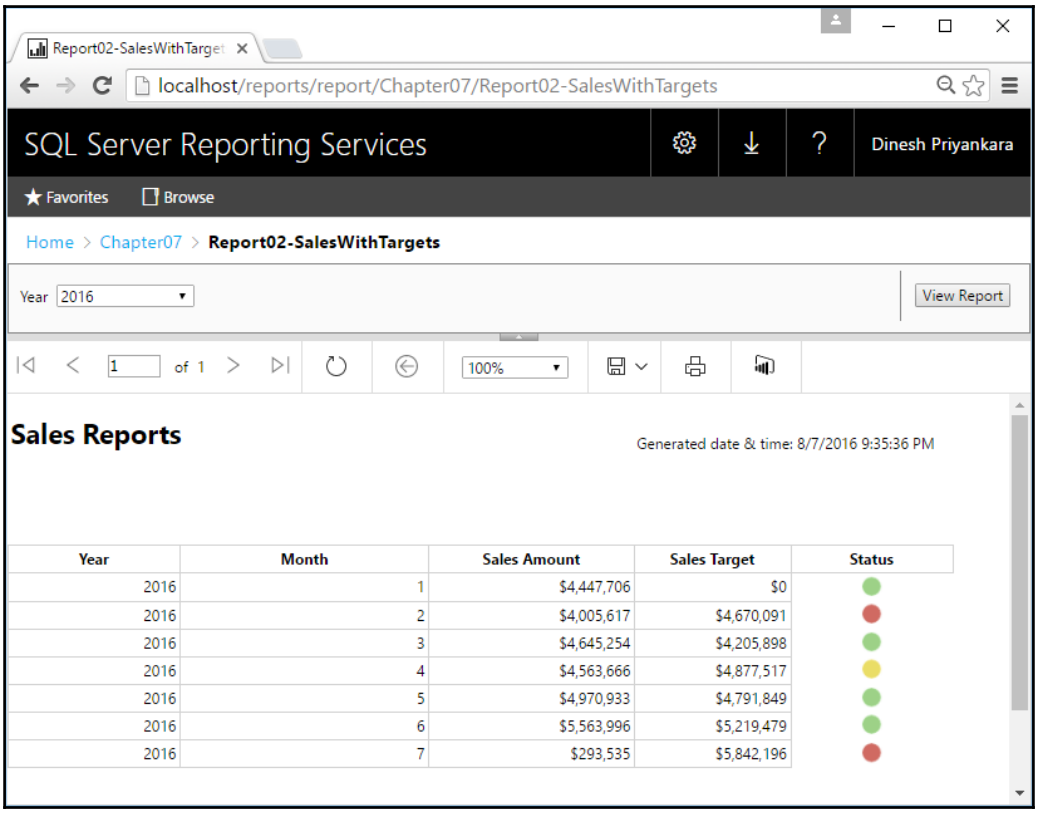


Figure 7.45

Customizing the look-and-feel of the Web Portal

One of the major issues we had with the Report Manager that was available with the previous Reporting Services versions is customizing the appearance of the Report Manager aligning with organization's brand (or theme). Since the portal, whether it is the Report Manager or the Web Portal, is supposed to be opened for business users for viewing reports and performing business user level actions, the custom branding is highly demanding and important. Everyone requested it, everyone insisted on it, and it has been answered by Microsoft with Reporting Services 2016.

With the latest version of Reporting Services, we can customize the portal with our own name, logo, and colors. Let's see how easily it can be done with this recipe.

Getting ready

Generally, theming is done with complex CSSs that are not much easier to implement. We do not need to worry about CSSs because the custom branding of Reporting Services is done by using **packages**.

The package represents the theme and the form of it is a ZIP file. It contains three files:

- `colors.json`: Contains color codes for the interface and theme
- `metadata.xml`: Contains the name of the package, path for the `color.json`, and the path for the logo file
- `logo.png`: The logo needs to be in top-left corner

You can name the ZIP file about you want but the above mentioned files must be exactly named as they are mentioned.

We will use a standard package available in Microsoft for seeing this feature. You can either download the package from: `https://msdnshared.blob.core.windows.net/media/2016/03/ssrs-logo.zip`or take the `ssrs-logo.zip` file available in the `Chapter07` solution folder.

How to do it…

Let's make a small modification to a file in ssrs-logo.zip and upload it as a new package:

1. Uncompress the `ssrs-logo.zip` file and open the `metadata.xml` using a text editor.
2. Change the value of the `Name` attribute in the `SystemResourcePackage` node as `My new package`:

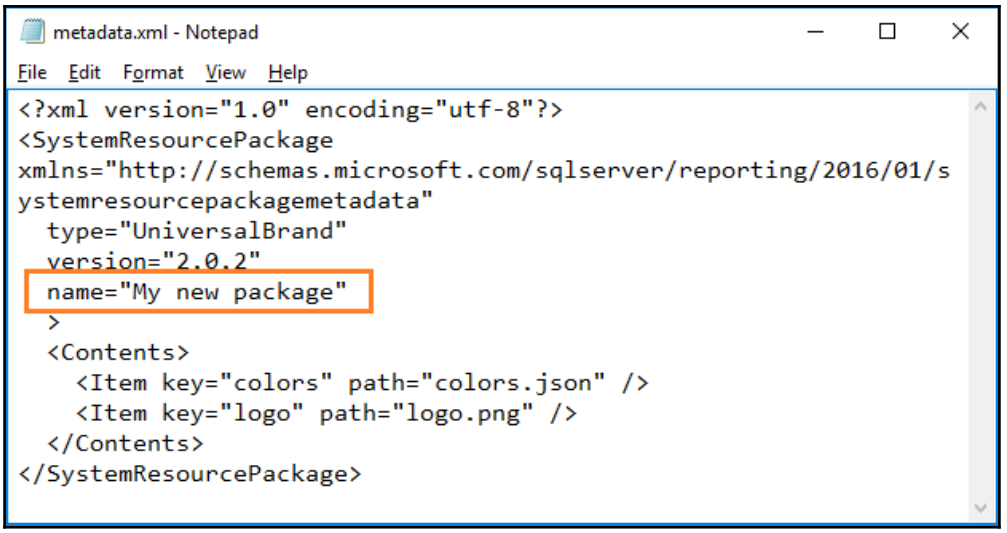


Figure 7.46

3. Open `colors.json` and replace `"primary":"#bb2124"` with `"primary":"#000000"`. This makes the primary color of the button black:

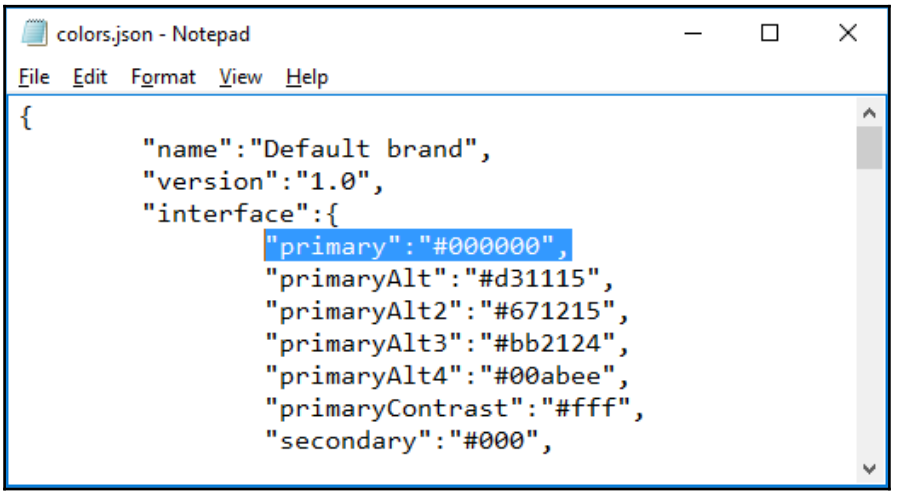


Figure 7.47

4. Replace `"kpiNeutral":"#d9b42c"` with "`kpiNeutral":"#ffa500"`. This makes the KPI neutral color orange. Save and close the file:



Figure 7.48

5. Make a ZIP file using the modified `metadata.xml`, modified `colors.json` and `logo.png` as `MyNewPackage.zip`.
6. Open the Web Portal and have a look on `Chapter07` folder:

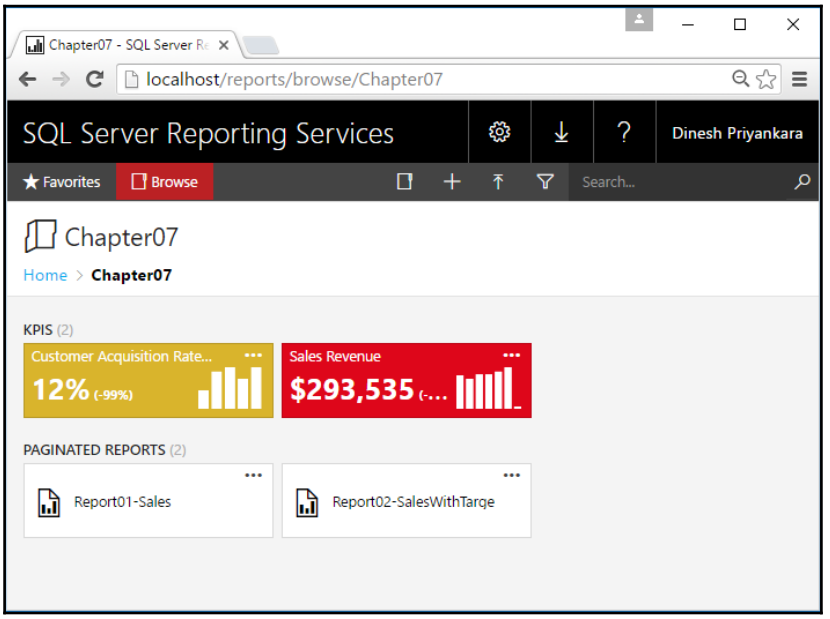


Figure 7.49

7. Click on the **Settings** icon and select **Site settings** for opening the `Site settings` page:

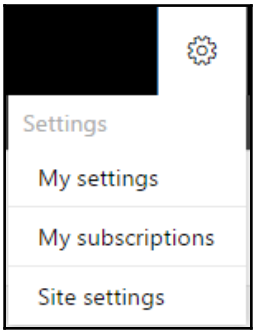


Figure 7.50

8. Click on Branding for opening the page that allows you to upload packages:

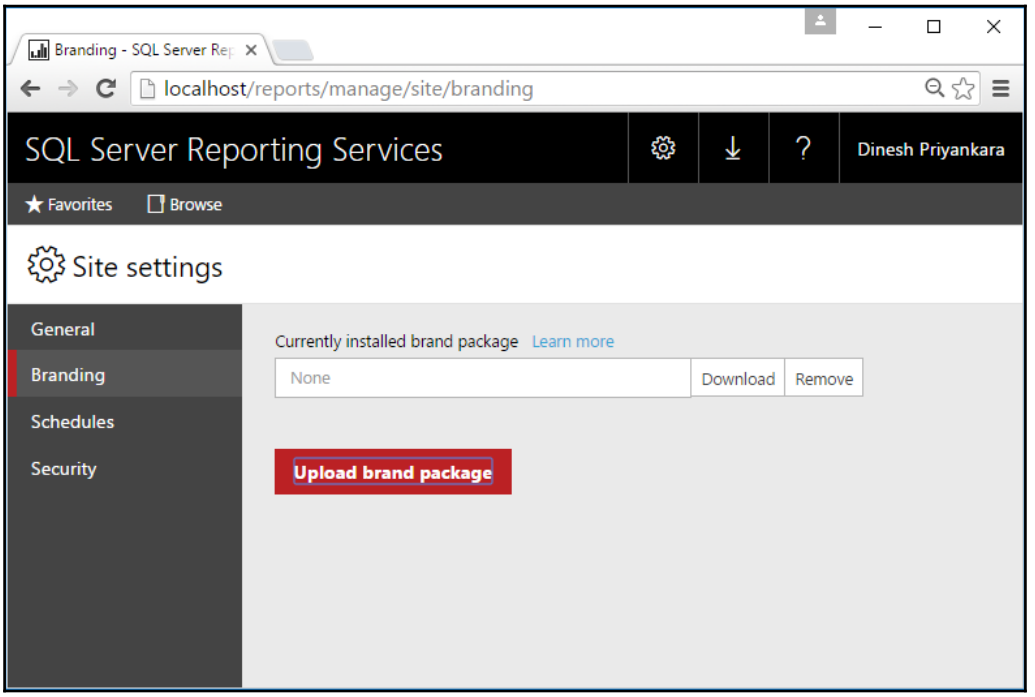


Figure 7.51

9. Click on **Upload brand package** button, locate the newly created ZIP file, `MyNewPackage.zip`, and select it. Once it is done, you should see the new appearance. Note the button color, logo, and KPI neutral color:

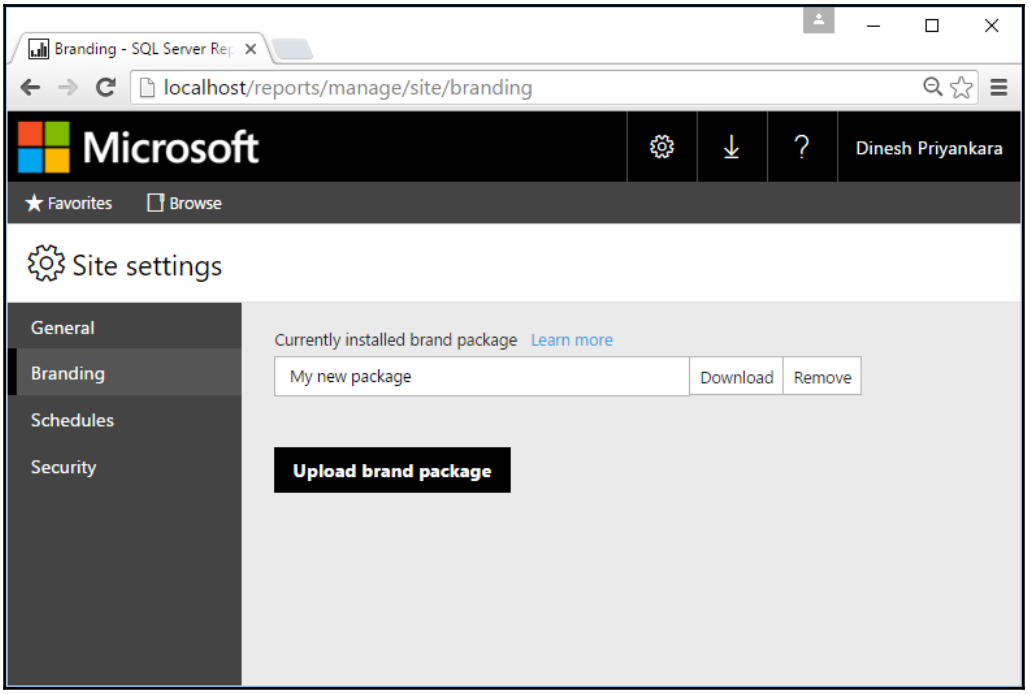


Figure 7.52

How it works…

When a new package is uploaded, it takes the relevant name/value pairs from the `colors.json` and merges with the master LESS style sheet. CSS styles are generated once the LESS files are processed and that is what is used in the client side. This happens as soon as the package is uploaded.

Note that the logo is optional. You can upload a package with or without a logo. If you need it, make sure that it is a PNG file and scales around 290px x 60px.

The `colors.json` includes two sections: interface and theme. The interface section represents styles related to the Web Portal and the theme section represents styles related to mobile reports.

The interface section is further categorized into seven sections:

- **Primary**: Describes the colors of buttons and hover colors
- **Secondary**: Describes the colors of the title bar, search bar, and the text color of them
- **Neutral Primary**: Describes the colors of home and report area backgrounds
- **Neutral Secondary**: Describes the colors of test box, folder options backgrounds, and the settings menu
- **Neutral Tertiary**: Describes the colors of site settings background
- **Danger/Warning/Success messages**: Describes the colors of messages
- **KPI**: Describes the colors of the KPI status

There's more…

You can bring in the default anytime by removing the uploaded one. All you have to do is, click on the **Remove** button in the **Branding** page.

If you need a backup of the current package, use the **Download** button in the same page.

Printing reports with the PDF rendering extension

Not all reports generated are required to print but many, especially paginated reports with less graphical data regions, are meant to be viewed as paper-based reports. This fact makes the functionality related to printing a must have and is crucial for any reporting applications. This capability must be provided regardless of the container used for viewing the report, whether it is shown in the browser or framed within a windows application.

Reporting Services has provided the printing facility since its inception but it was based on an ActiveX control. This works fine with reports embedded with Windows applications but some experienced issues when printing reports with Report Manager as it is required to download the ActiveX control. For making sure that there is no need to download ActiceX control and printing can be done with any browser, Reporting Services 2016 offers a simple solution with **Portable Document Format** (**PDF**).

Reporting Services 2016 includes a PDF rendering extension to render reports in PDF format. The **Print** button in the **Report Viewer** toolbar uses it for generating the PDF which can be used for printing.

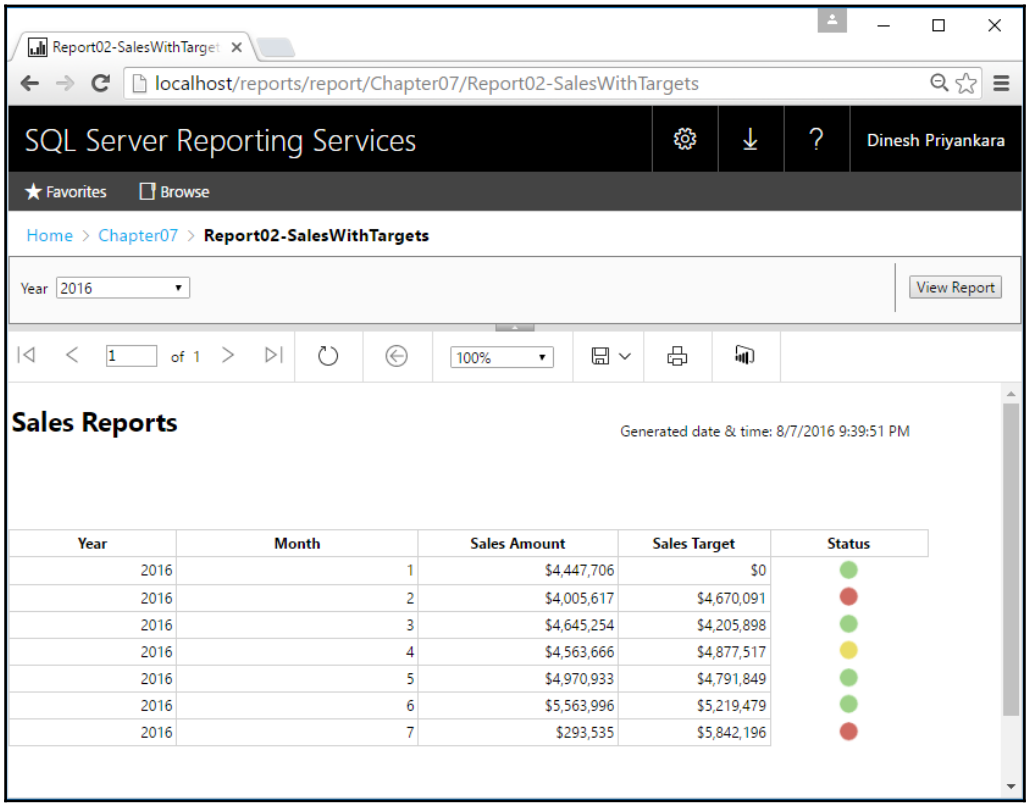
Getting ready

The solution `Chapter07` has a report called `Report02-SalesWithTargets`. Let's use this report for seeing the new report printing experience. You need to make sure that the project is deployed.

How to do it…

Let's start it with following steps:

1. Open the `Chapter07` folder and click on `Report02-SalesWithTargets` report for opening it. You will see the **Print** button in the toolbar. Click on the **Print** button:



Year	Month	Sales Amount	Sales Target	Status
2016	1	$4,447,706	$0	
2016	2	$4,005,617	$4,670,091	
2016	3	$4,645,254	$4,205,898	
2016	4	$4,563,666	$4,877,517	
2016	5	$4,970,933	$4,791,849	
2016	6	$5,563,996	$5,219,479	
2016	7	$293,535	$5,842,196	

Figure 7.53

2. You should see the first related dialog for printing. Set the **Page size** as **A4 (210mm x 297mm)** and the **Page orientation** as **Landscape**. Click on Print to continue:

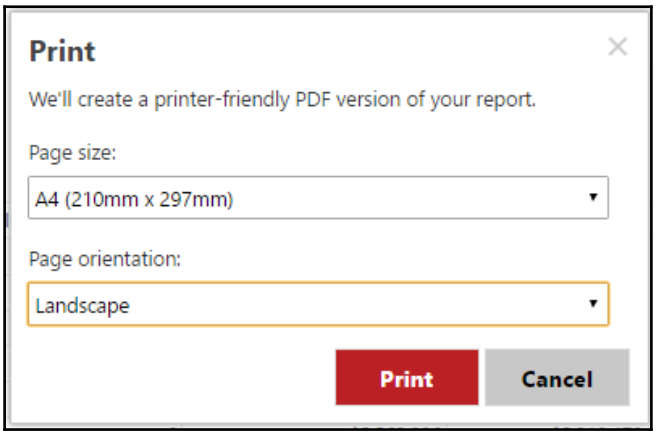


Figure 7.54

3. The next screen differs from browser to browser. If you have opened the Web Portal in **Chrome**, you should see a dialog for **Converting to PDF…** and **Done printing:**



Figure 7.55

4. If the Web Portal is opened using **Internet Explorer**, then you will see the following screen:

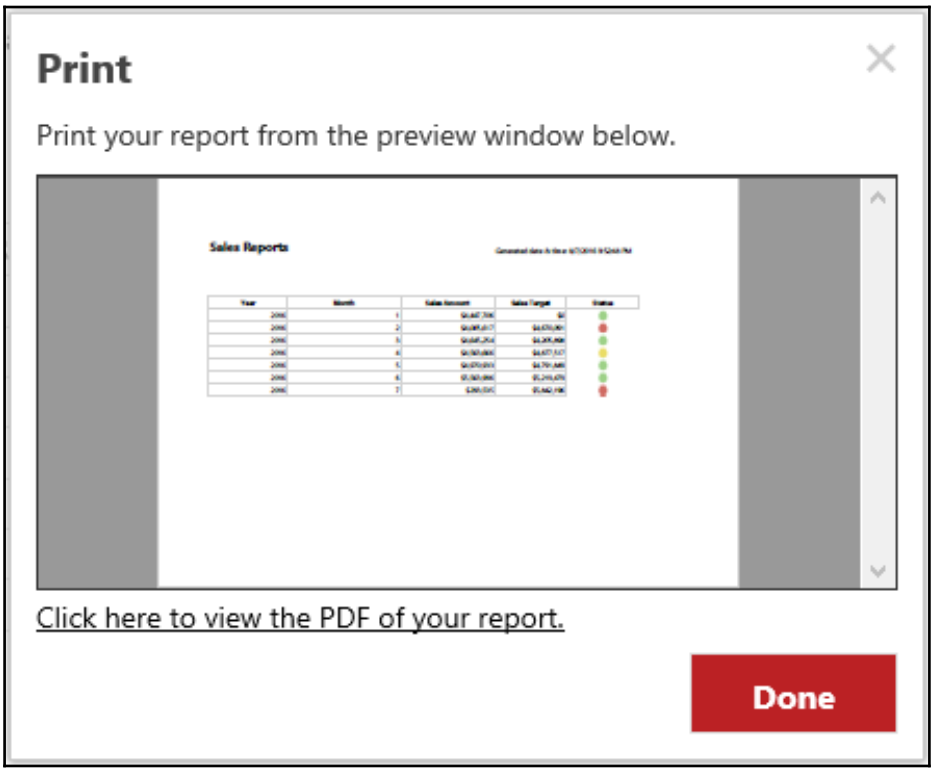


Figure 7.56

5. Following with previous screens, it immediately opens the printing dialog. The following image shows how the **Chrome** browser opens it. Click on **Print** for printing the report:

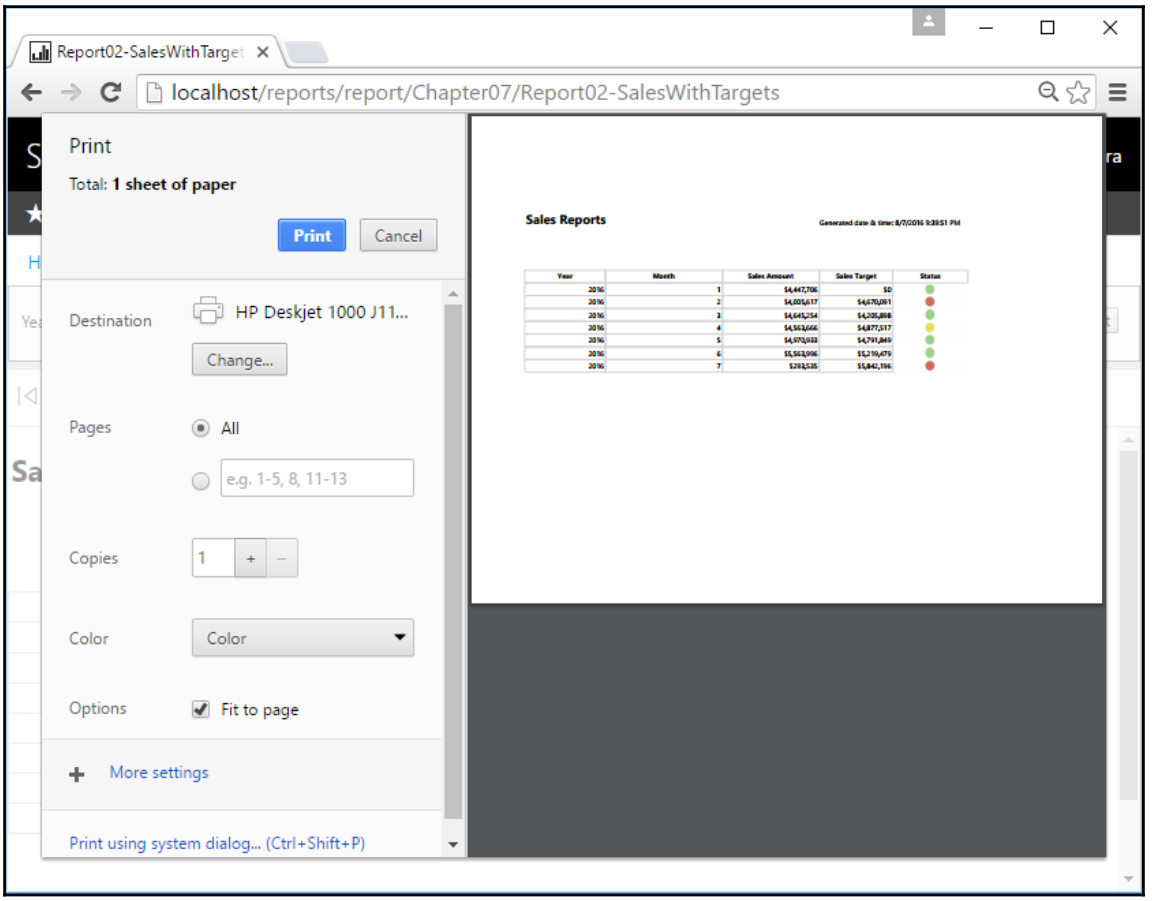


Figure 7.57

6. The second image shows how it is opened with **Internet Explorer**. Click on **Print** for printing the report:

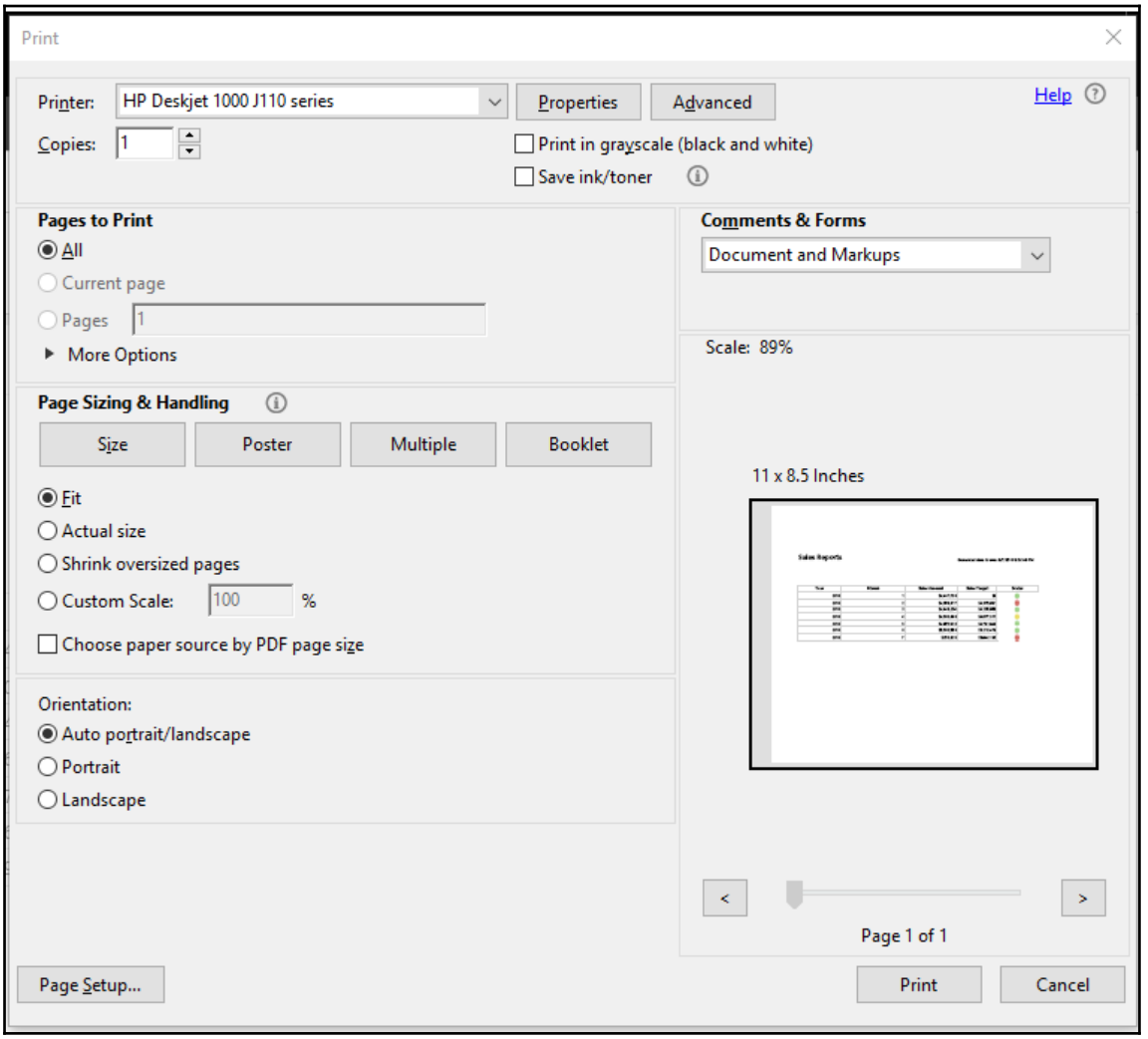


Figure 7.58

How it works…

When the print button is clicked, the behavior of it is different if the client machine has no PDF reader application installed. If it is installed, just like the way it was shown in the *How to do it…* section, the print dialog is opened for changing settings and printing the report.

If the PDF reader has not been installed in the client environment, the browser will automatically download the generated PDF file to the client machine.

There's more…

There are some instances where you may need to disallow users from printing the report. This can be done by disabling the facility using the Management Studio. If it is disabled, you will not see the printing icon in the toolbar but you will be able to download the report in PDF format.

Embedding reports in other applications

What if you need to embed a report in your organization's website? How do we do? We have seen a common answer for this requirement; many save the output of the report as an image and add it to the website because it is the easiest way of embedding an output of a report to another application.

Reporting Services supports **report embedding**. It allows us to make the report as an embedded report without much effort. It only needs an additional tag to the URL and it works for both mobile and paginated reports. Once we have the URL, it can be used with another web page or applications using an iframe along with its parameters for showing the report with the same functionalities.

Reporting Services supports embedding reports into Power BI dashboards. This is called as **Pinning to Power BI**. This is really useful and it eliminates the necessity of creating the same outputs in Power BI dashboards, saving time and resources.

Getting ready

With the *How to do it…* section, let's have a look at both embedding reports to other applications and pinning report items to Power BI dashboards. Let's use the report named **Report03-Sales2016** available in the solution given with this chapter.

This report generates an output of a line chart that shows sales for this year along with months. It shows last-year-same-month sales as well for allowing users to compare the sales between this year and last year:

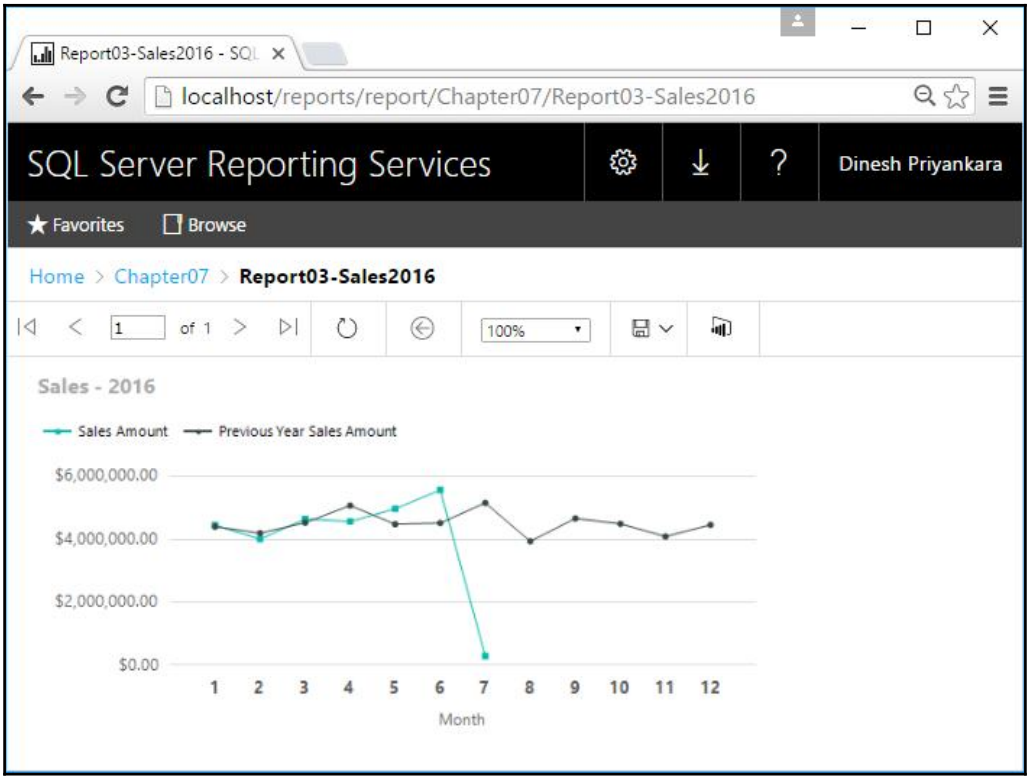


Figure 7.59

Make sure that this report is published and it can be viewed using the Web Portal.

How to do it…

Let's see how we can get the URL for embedding this report to another web application:

1. Take a copy of the URL of the report. It should be similar to this: `http://<yourservername>/reports/report/Chapter07/Report03-Sales 2016`.
2. Open the **Report03-Sales2016** report using the Web Portal.
3. Append `rs:Embed=true` to the URL: `http://<yourservername>/reports/report/Chapter07/Report03-Sales 2016?rs:Embed=true`.
4. This URL can be used with any application for embedding. Open a new browser tab and paste the URL. You will see the same without Reporting Services headers:

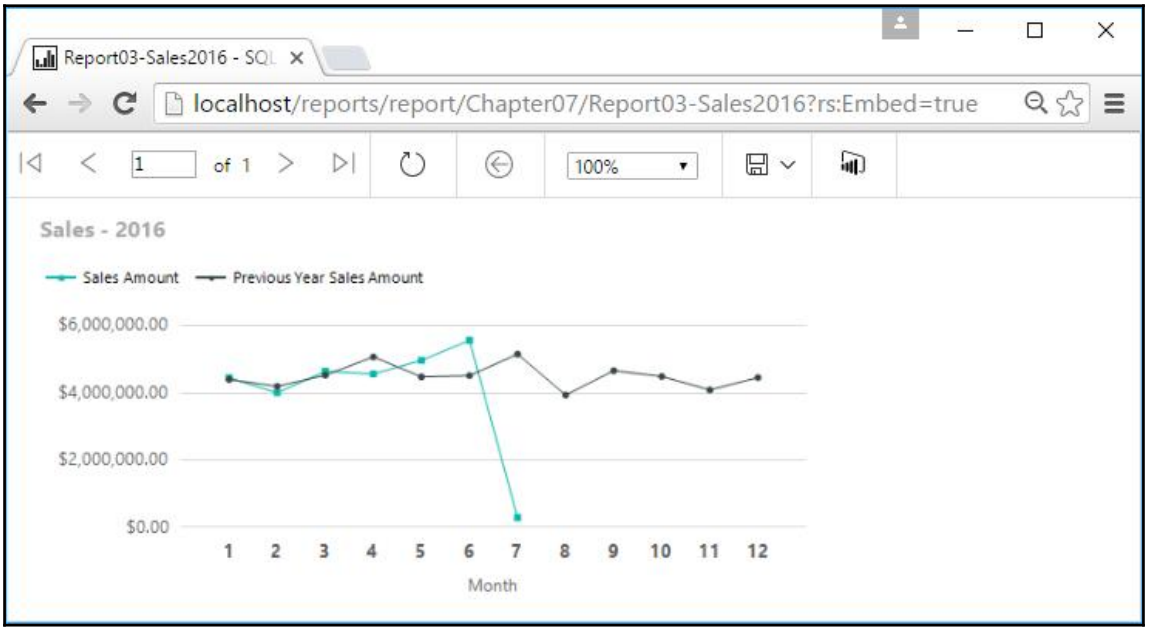


Figure 7.60

5. The integration to Power BI requires registering with a Power BI account. In order to register, you need to make sure that you have following in the Power BI portal:
    - You have a Power BI Pro account created
    - You have a group in addition to the default **My Workspace** group, created with your Power BI account. Figure 7.61 shows an additional group created called **Main Group**:

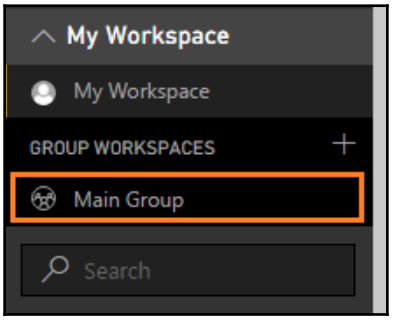


Figure 7.61

- You have a dashboard created in the newly created group
- You have permission to manage the added dashboard added

6. The next step is registering Power BI to Reporting Services. Follow these steps for registering a Power BI account to Reporting Services:

    1. Open the Web Portal and click on the **My settings** menu in **Settings**:

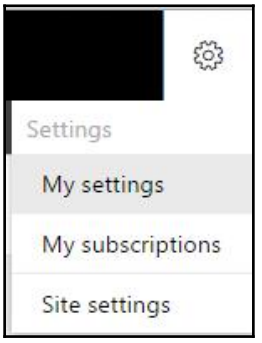


Figure 7.62

2. Click on **Sign in** and log in to the Power BI account:

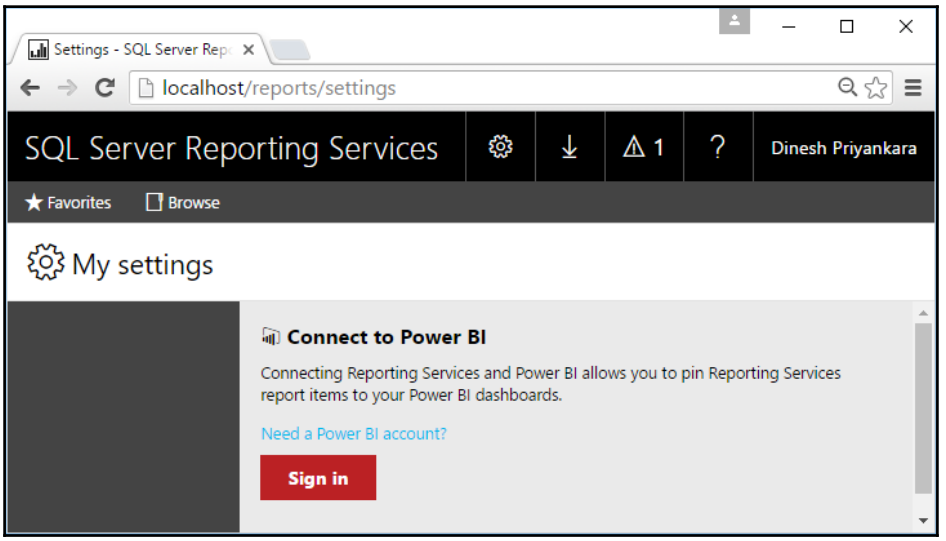


Figure 7.63

3. Once logged in and it is successful, you should see the registration with your account as shown here:

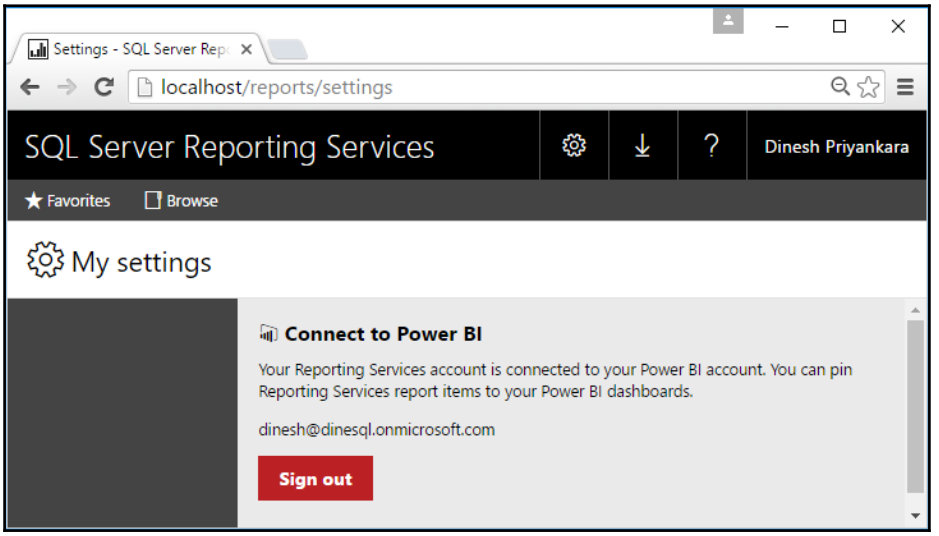


Figure 7.64

7. For pinning the report into the dashboard, follow these steps:

    1. Open the **Report03-Sales2016** report and look forthe Power BI icon in the toolbar. Click on it:



Figure 7.65

    2. Click on the chart to pin and find the following output:

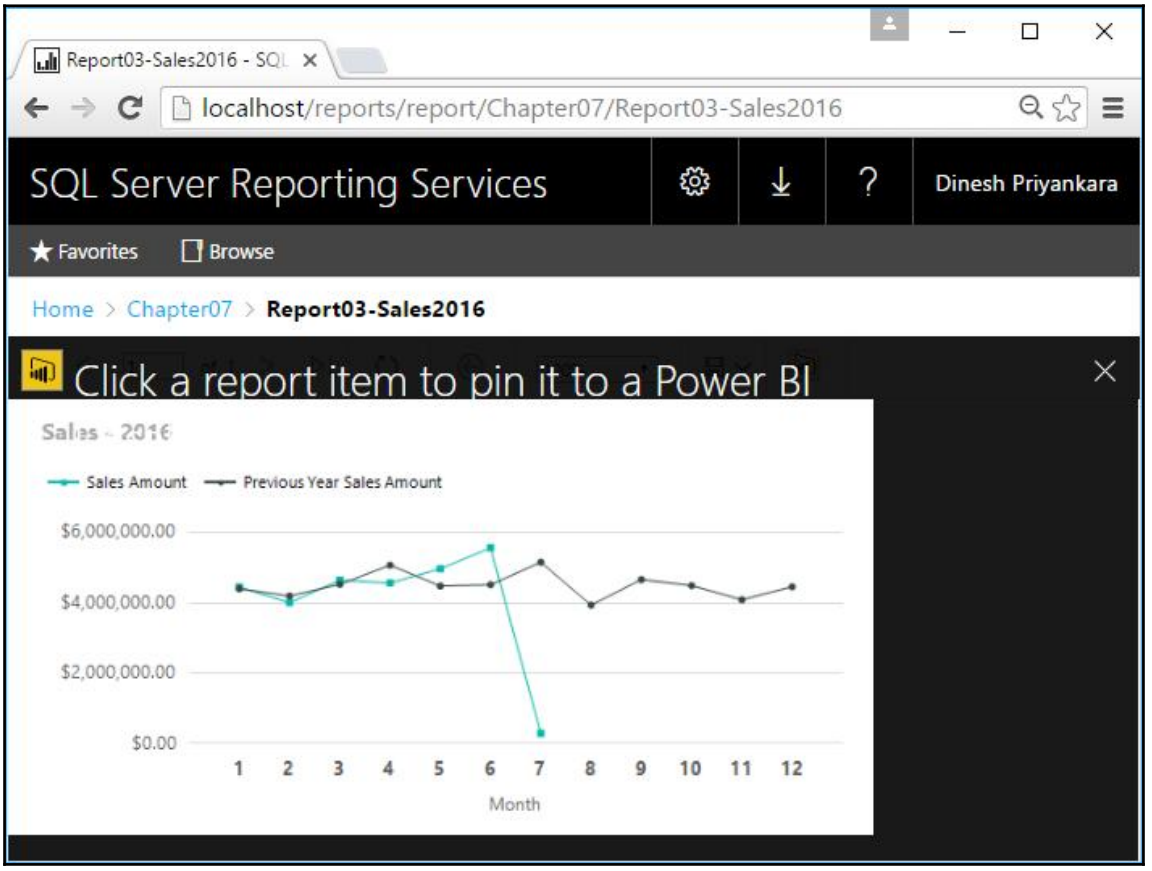


Figure 7.66

3. With the **Pin to Power BI Dashboard** window, select the newly created group, dashboard and select **Daily** for the **Frequency of Updates**. Click on **Pin** button:

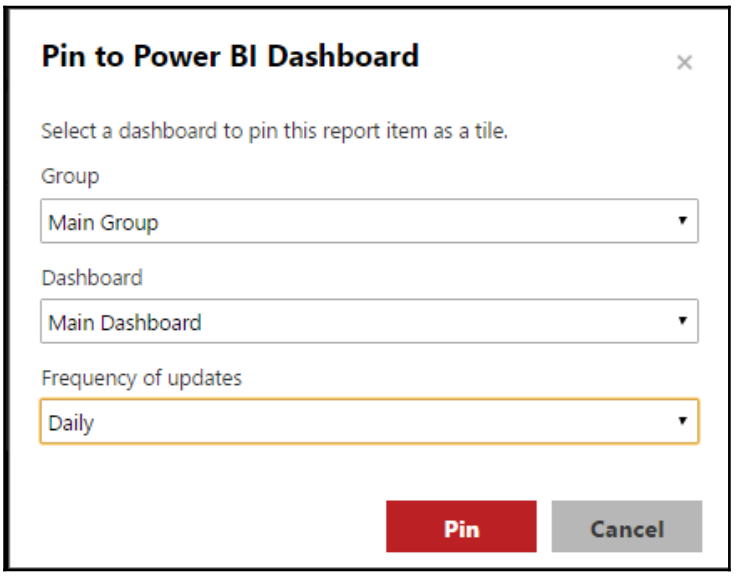


Figure 7.67

4. You will see the **Pin Successful** dialog. Click the **See in Power BI here** link for seeing how it has been pinned:

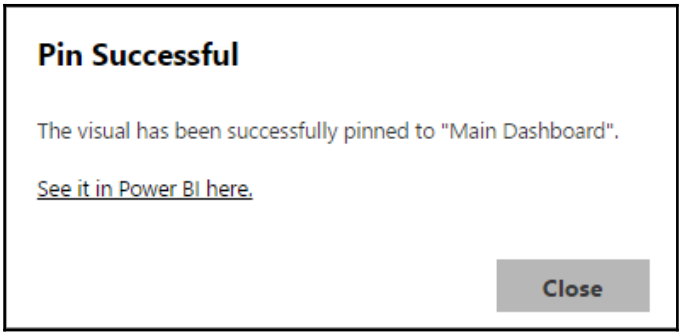


Figure 7.68

5. You can view the following report:

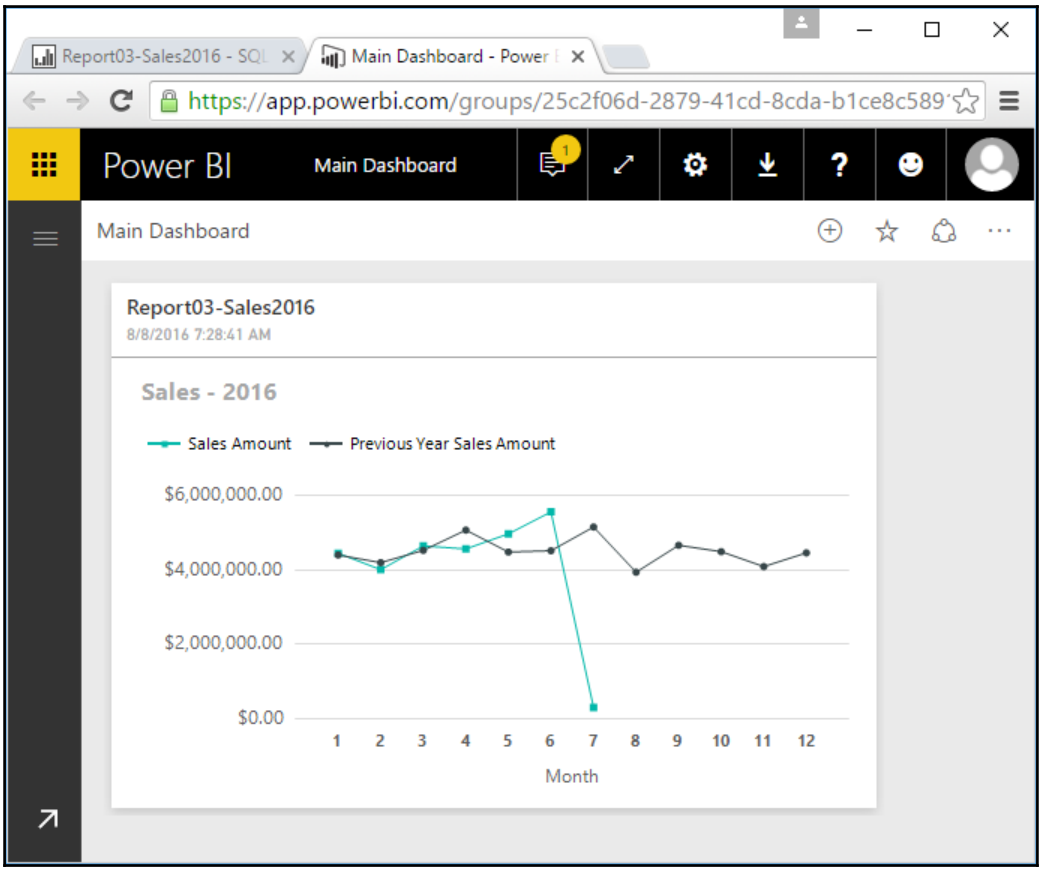


Figure 7.69

How it works…

When adding the additional tag `rs:Embed=true` for the URL for embedding purposes, it creates an optimized view by eliminating standard headers and menus. This makes the embedding simple and we can add it to any application without complications. Even though we do not see the Web Portal headers and menus, we still get the standard report toolbar which can be used for navigating, exporting, printing, and so on.

As you can see, pinning report items to Power BI is not that complex. However, not all report items can be pinned to Power BI and not all items in all sections of the report can be pinned to Power BI. Items should be in the report body and should not be inside another data region. The following items can be pinned to Power BI:

- Charts
- Gauges
- Maps
- Images

When the report item is added to the dashboard, you need to set the frequency as **Daily, Hourly,** or **Weekly**. This makes sure that the Power BI dashboard shows the content as per the business requirement. Based on the frequency you set, it refreshes the content in the dashboard.

This refresh is done using **Report Services Subscriptions**. The subscription is automatically created when the item is pinned and the schedule is created for the subscription based on the frequency you select. You can modify the schedule of the subscription any time as you prefer.

You can see your subscriptions using the **My subscriptions** menu in the **Setting** menu:

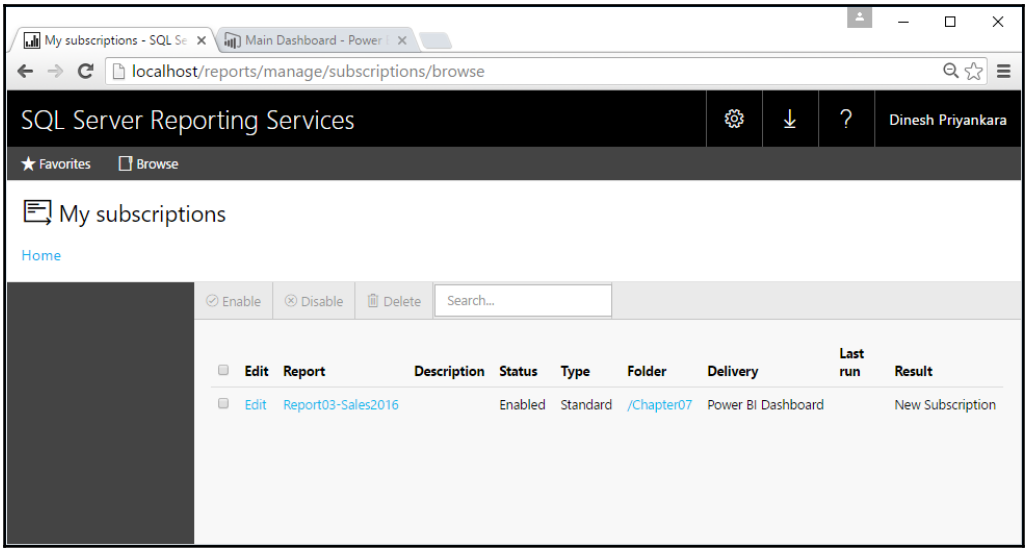


Figure 7.70

If you click on **Edit**, you can see the details of it and change the schedule if needed as shown here:

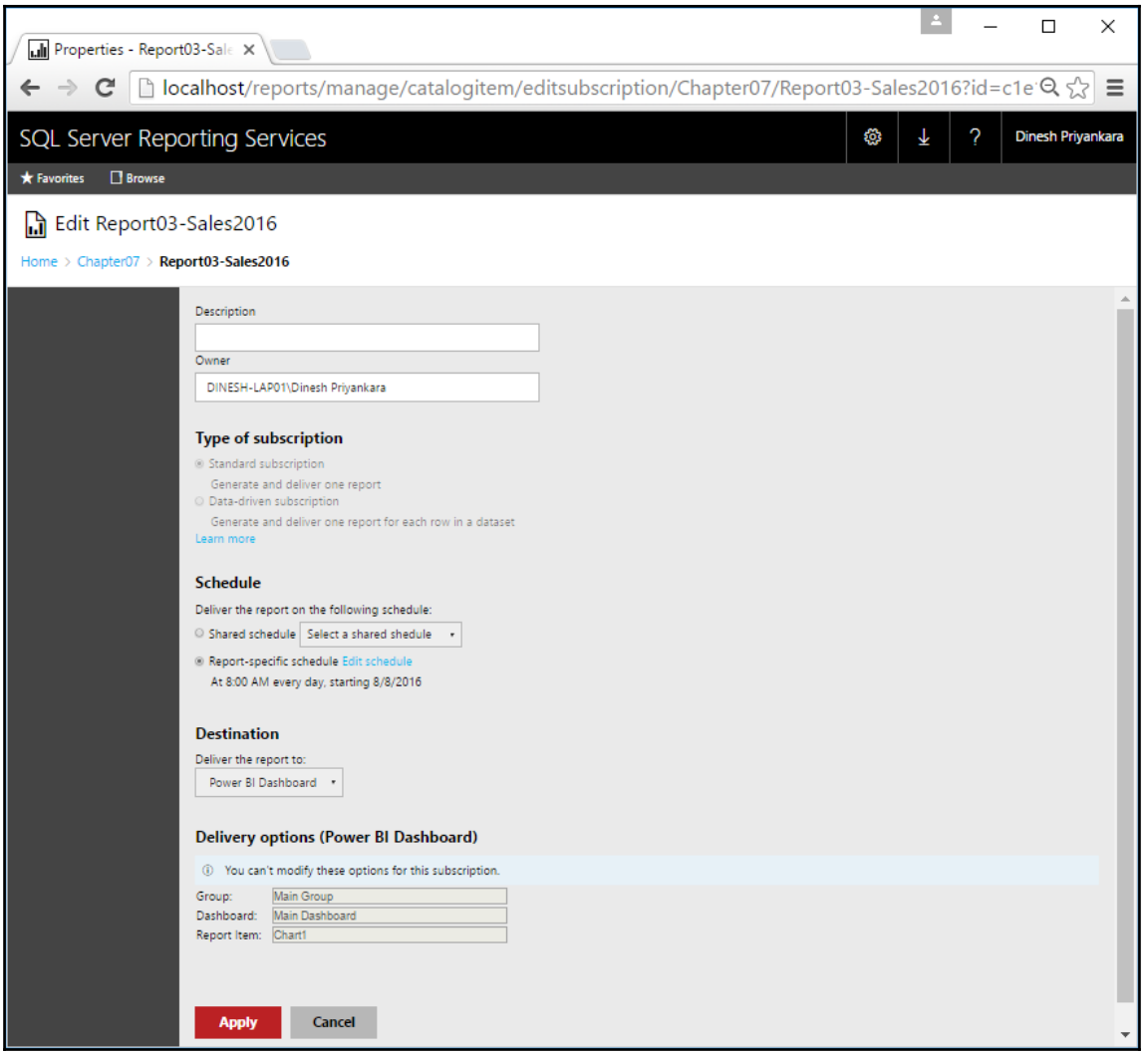


Figure 7.71

There's more…

Note that you can register Power BI using the **Reporting Services Configuration Manager** as well. It allows you to register, update, and unregister with Power BI:

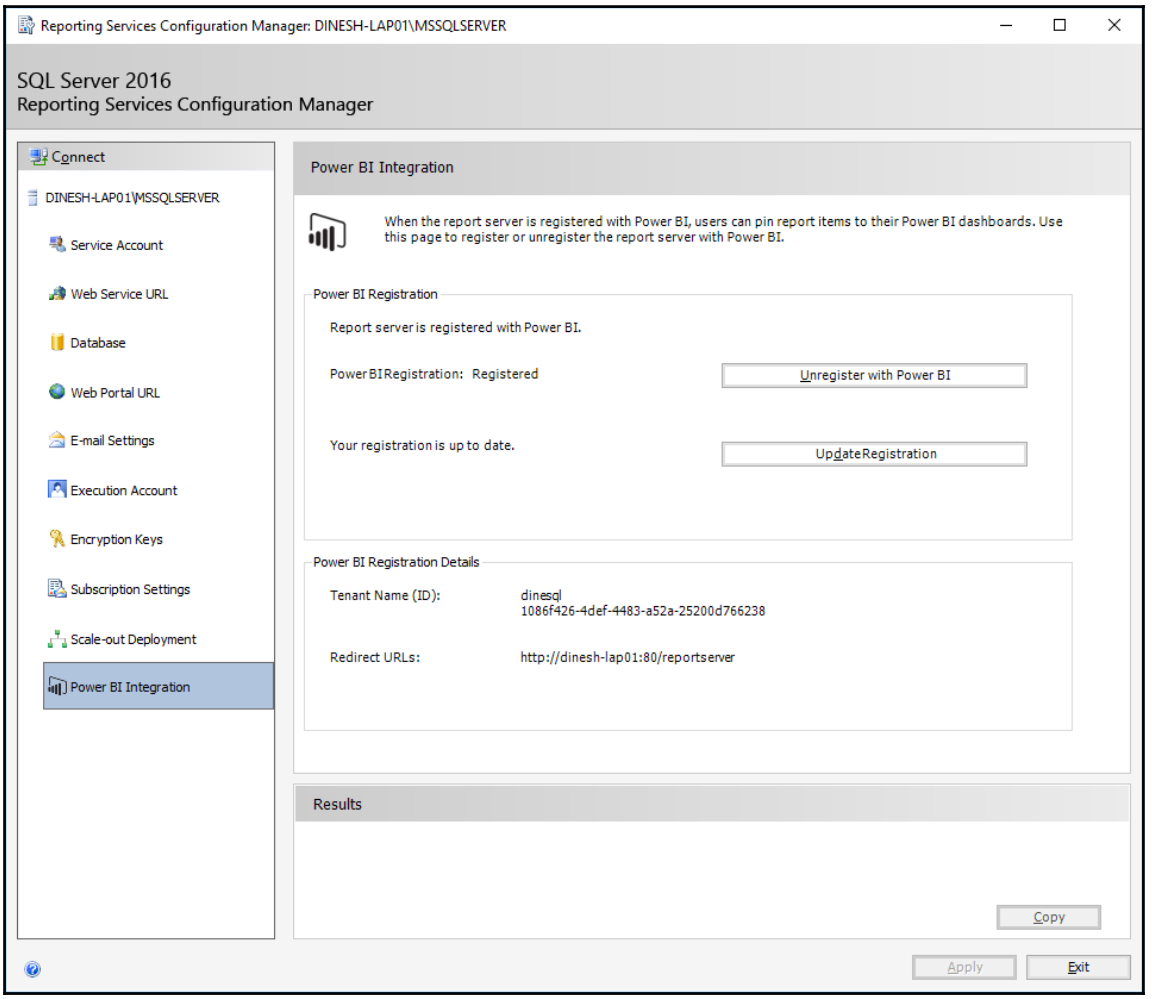


Figure 7.72

See also

You can get more details on how the process related to Power BI integration works with the following URL:

`https://msdn.microsoft.com/en-us/library/mt598750.aspx`.

8
Reporting Solutions for BI – Integration

Although most reporting solutions are designed for handling OLTP solutions, reports play a major role in **Business Intelligence** (**BI**) solutions as well. Generally, **Production Reports** or **Formal Reports** (sometimes called **Paginated Reports**) are the ones we create for OLTP solutions, whereas **Analytical Reports**, **Dashboard Reports**, and **Mashup Reports** are created for Business Intelligence solutions.

Reporting Services 2016 is an enterprise reporting platform that is rich enough to address requirements related to both OLTP and BI solutions. Its capabilities allow us to make reports suitable for BI solutions. It also enhances the user experience with its features providing both ad-hoc reporting and self-service reporting.

In this chapter, you will learn how to integrate Reporting Services with BI solutions and its components, such as OLAP databases. This chapter covers the following recipes:

- Making Reporting Services as a part of BI solution
- Creating reports using Multidimensional models
- Getting formatting from the Multidimensional model
- Getting aggregated values from the Multidimensional model
- Creating reports using Tabular models
- Connecting to the HDInsight cluster and getting unstructured data

Introduction

Traditional Business Intelligence implementation, also known as Corporate Business Intelligence, is widely used, and analytical reports built against it satisfy most reporting requirements. The rise of modern concepts and technologies has moved traditional BI to self-service BI, but the requirements for reporting still exist.

Microsoft **SQL Server Reporting Services** (**SSRS**) offers many features and functionalities to create a BI reporting solution. Modern Business Intelligence solutions maintain data with various structures, such as Relational Data Warehouses with schema like Star and Snowflake, OLAP Structures, and In-Memory data tables combined with ColumnStore indexing and Hadoop. Business users want a single reporting platform for connecting with all these data sources and getting data into reports as per need. Reporting Services allows us to author reports against all these data sources either directly, or by using middle-tier components.

Transact SQL (**T-SQL**) is used for authoring reports against SQL Server databases. In order to get data from other database types, different data extraction techniques or languages must be used. A common example used on the Microsoft platform is MDX (Multidimensional Expressions), for getting data from multidimensional databases hosted by Analysis Services. If the database type is Tabular in Analysis Services, we can use DAX (Data Analysis Expressions) along with MDX. If the data structure is non-SQL-Server-related, for example, Hadoop, then a language such as Pig or Hive is used.

When it comes to BI, output, or the appearance of the report layout, is highly considered. The need may be for a tabular report, or it can be a report with a graphical element, such as Sunburst. The generation of the output may be the responsibility of the reporting tool, or the tool might be taking the output generated from the database itself and just showing. You have already seen an example of this in `Chapter 5`, *Improving User Experience – New Designing and Visualization Enhancements*.

It is time to explore all these scenarios. Let us explore them one by one with the following set of recipes.

Making Reporting Services as a part of BI

The selection and specifications of servers and hardware infrastructure is something you need to consider in BI. Since Reporting Services is a part of BI solutions, whether or not you need a separate server and whether or not it should be scaled out or needs high availability implemented must be considered based on the size and complexity of the BI solution.

Getting ready

It's recommended to implement **Distributed Architecture** for BI solutions. However, it is not unusual to have all the required components related to BI in a **Single Server Architecture**, but the decision should be based on workloads involved with all components. Generally, the impact of all workloads related to components of BI such as data warehouse, ETL, reporting, comparatively low, in which case, Single Server Architecture will be enough for the BI solution. Since this book focuses only on Reporting Services, let's understand what sort of workloads we have to consider in Reporting Services for BI solutions:

- **Client requests**: You may get frequent requests from users for reports, and the report server must listen to and handle all of them. This is something that may introduce contention.
- **Data extractions from sources**: Reports are based on data sources. Handling more data requests with complex queries makes the servers busy, thus increasing the latency. Generally, in BI solutions, relational data warehouses and data models hosted in Analysis Services are the common data sources, but Reporting Services may be involved with processing data as BI solutions are not limited to relational data warehouses and models.
- **Report caching**: Reporting Services caches data and reports in a temporary database based on your caching requirements. If there is a lot of caching, the temporary database will be overloaded, degrading the performance.
- **Report rendering**: Report rendering can be more or less complex based on the rendering format, size, and layout of the report. This consumes server resources and a server will not be able to handle report rendering efficiently if it gets many requests.

How to do it…

There are two main components of Reporting Services to consider with Distributed Architecture:

- Reporting Services databases
- Reporting Services service

Assume that you have two different servers, one for the relational data warehouse and another for data models with Analysis Services. In a typical implementation, the data warehouse server becomes busy when ETL operations are performed, unless some reporting requirements are satisfied with the same server. However, in most cases, ETL is executed during off-peak hours and the report server makes data requests from Analysis Services. Therefore, to increase the scalability and performance, you can do the following:

- Install Reporting Services databases in the data warehouse server.
- Install Reporting Services in another dedicated server.

The previously mentioned architecture is good for a medium-sized BI solution. However, with extremely large BI solutions that expect heavy reporting requests, you can set up the architecture as shown here:

1. Install the Reporting Services database in a separate database server.
2. Install multiple Reporting Services report servers on multiple servers.
3. Connect all the report servers to the same Reporting Services database.

How it works…

Once the Distributed Architecture is configured, users can access a report using multiple servers, which reduces the workloads on the Reporting Services servers. This makes sure that processing and rendering are separated from the database workloads, increasing the data and report-retrieval performance of the reporting solution.

There's more…

You may consider high availability for your BI solution and configure Reporting Services as required. Generally, clustering is used to implement high availability; you can use either the **Always On Availability Group** or **Always On Failover Cluster** for Reporting Services. The distribution of all the requests can be efficiently handled with **Network Load Balancing** (**NLB**).

Read more about *Always On Availability Groups* at `https://msdn.microsoft.com/en-us/library/ff877884.aspx`.

See also

You can find more details on how to configure a Report Server Scale-Out deployment at the following URL:

`https://technet.microsoft.com/en-us/library/ms159114(v=sql.105).aspx`

Creating a report using Multidimensional models

As you know, SQL Server Analysis Services provides three database model types for implementing a database for BI: Multidimensional, Tabular, and PowerPivot. Multidimensional is the oldest and richest platform that has all the required functionalities, whereas Tabular is less complex but not as rich as Multidimensional. PowerPivot is simpler than both Multidimensional and Tabular, but provides sufficient functionalities to business users to create their own models. Let's start with the Multidimensional model and look at how Reporting Services works with it.

In this recipe, we will discuss how to create a standard report using the Multidimensional model. We will discuss some of the considerations you have to make with reports generated with Multidimensional models in the following set of recipes.

Getting ready

We have mainly been using the `WideWorldImporters` database to make demos and create our sample reports, but for this section, we will use our old friend, the `AdventureWorks2014` database. The solution `Chapter08.sln` is created with the source connection for the Multidimensional model which comes with `AdventureWorks2014`, hence you need to make sure it is available in your SQL Server 2016 Analysis Services instance. You can either use the solution given with the book or create a new solution. If you use the same solution, make sure that you change the connection string with credentials before using it.



Note that to connect with the Multidimensional model, you cannot use TSQL; you should use MDX. However, you do not need to explicitly write MDX code as designer constructs the statement based on your selection.

How to do it…

Follow these steps to create the report:

1. Open the `Chapter08` solution and open the **DataSourceAdventureWorksOLAP2014** data source under **Shared Data Sources** (see Figure 8.01).
2. Click on **Edit**, configure it to your instance, and make sure it is connected to the `AdventureWorks2014` Multidimensional data model (see Figure 8.01).
3. Click on **Credentials** and make sure **Use Windows authentication (integrated security)** is selected:

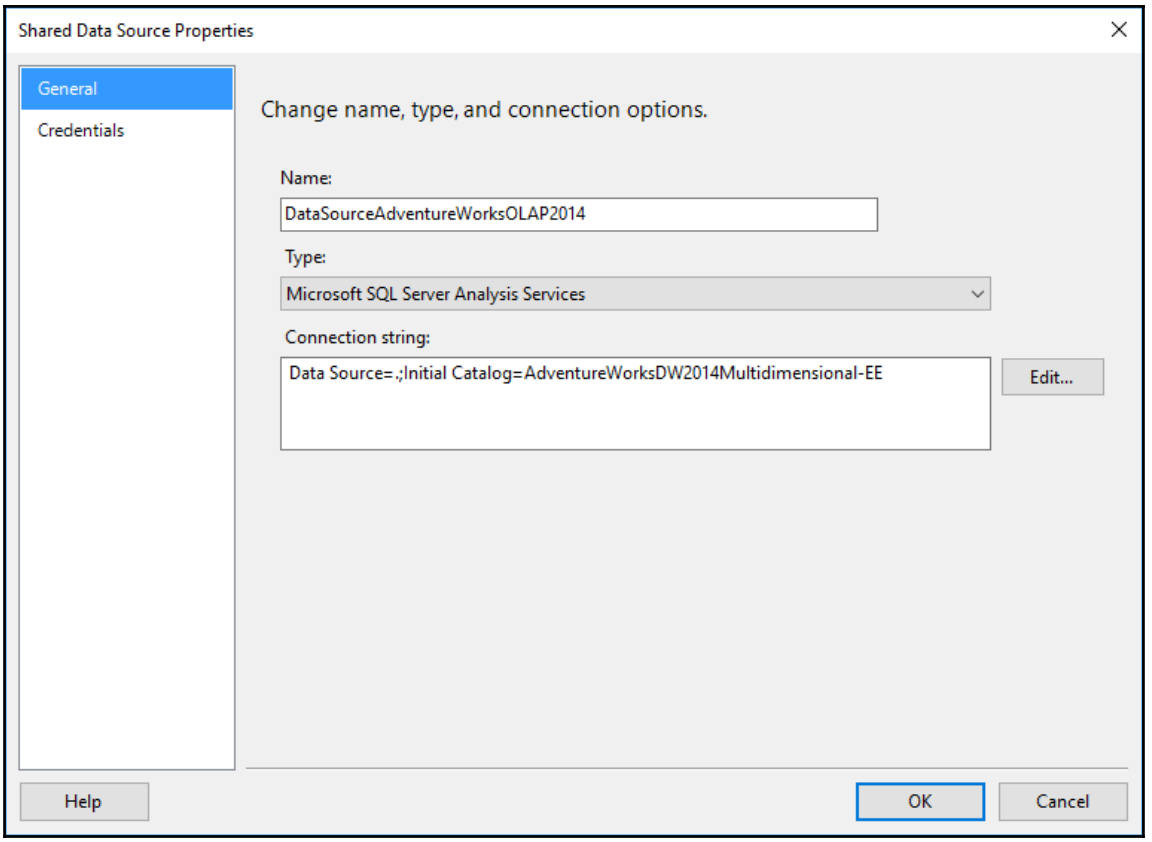


Figure 8.01

4. Click **OK** to save the settings.
5. Add a new report to the solution and name it `Sales2013`.
6. Create a new dataset called `DataSetSales2013`. Select **Use a dataset embedded in my report** (see Figure 8.02).
7. Click on **New…** to connect with an existing data source and add the **DataSourceAdventureWorksOLAP2014** data source (see Figure 8.02).

8. Click on **QueryDesigner…** to connect with the AdventureWorks OLAP database:

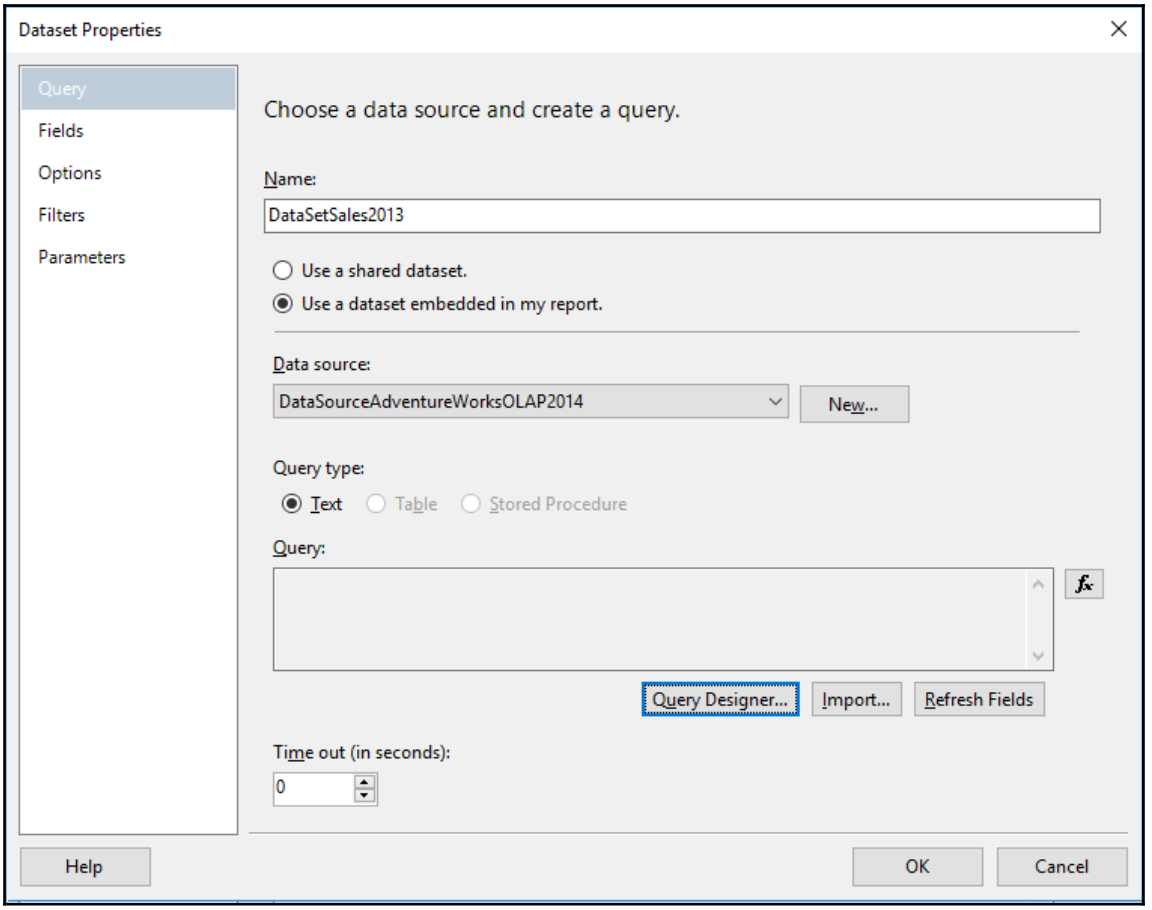


Figure 8.02

9. This opens the **Query Designer**. The left-hand pane in the designer shows KPIs, Measures, and Dimensions, which can be used to form the query. We can simply drag items and drop them into the right-hand pane to make the query. Expand **Measures** to see all **Measure Groups** (see Figure 8.03).
10. Expand the **Sales Summary** measure group to see all measures (see Figure 8.03).
11. Drag and drop **Order Quantity** onto the right-hand pane (see Figure 8.03).

12. Drag and drop **Sales Amount** onto the right-hand pane (see Figure 8.03):

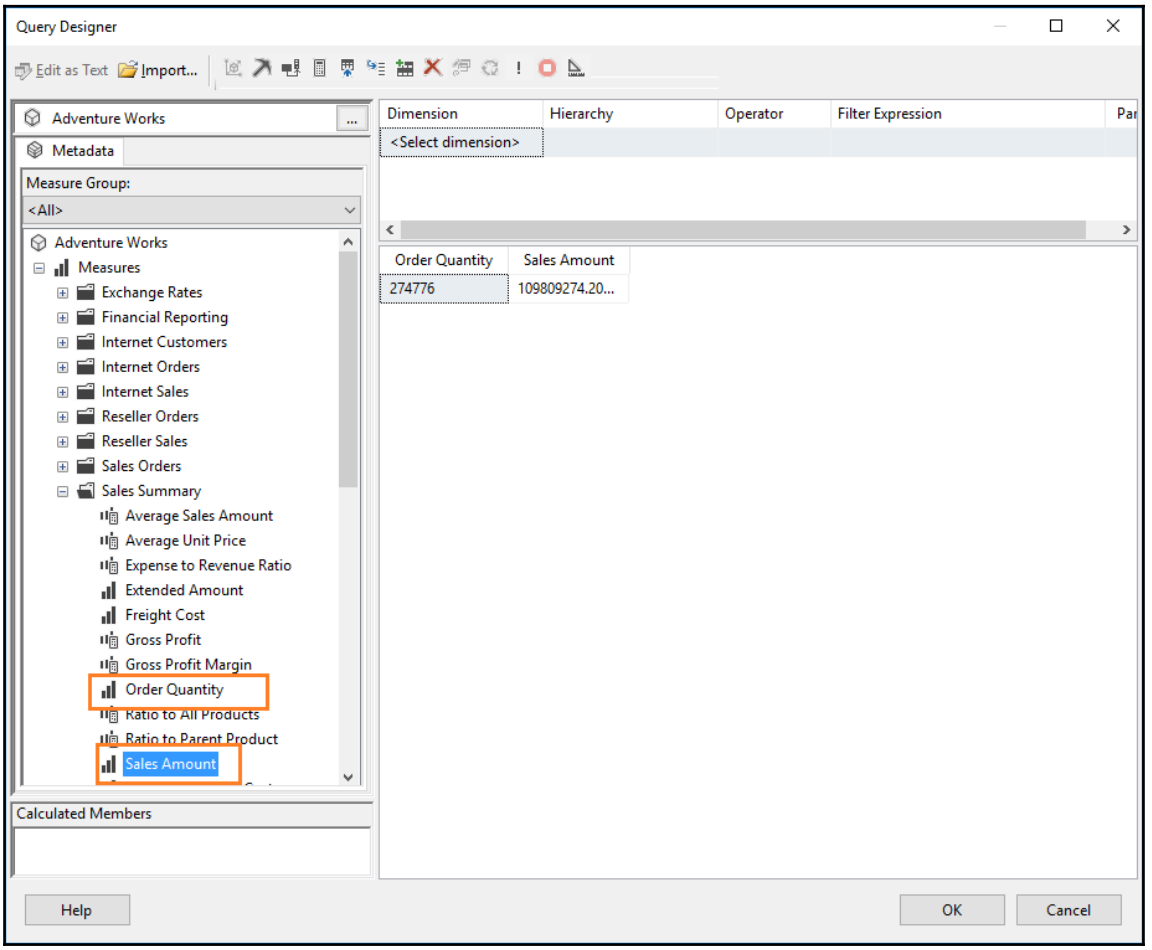


Figure 8.03

13. Expand the **Date** dimension and the **Calendar** folder. Expand the **Date.Calendar** hierarchy (see Figure 8.04).
14. Drag **Calendar Year**, **Calendar Quarter**, and **Month** from the hierarchy and drop in the left-hand corner of the right-hand pane. Now the grid should show the columns **Calendar Year**, **Calendar Quarter**, **Month**, **Order Quantity**, and **Sales Amount** in order, as shown in Figure 8.04.

15. Drag and drop **Calendar Year** to the top-right-hand grid to filter the year. Select **CY 2013** as the selected year (see Figure 8.04):

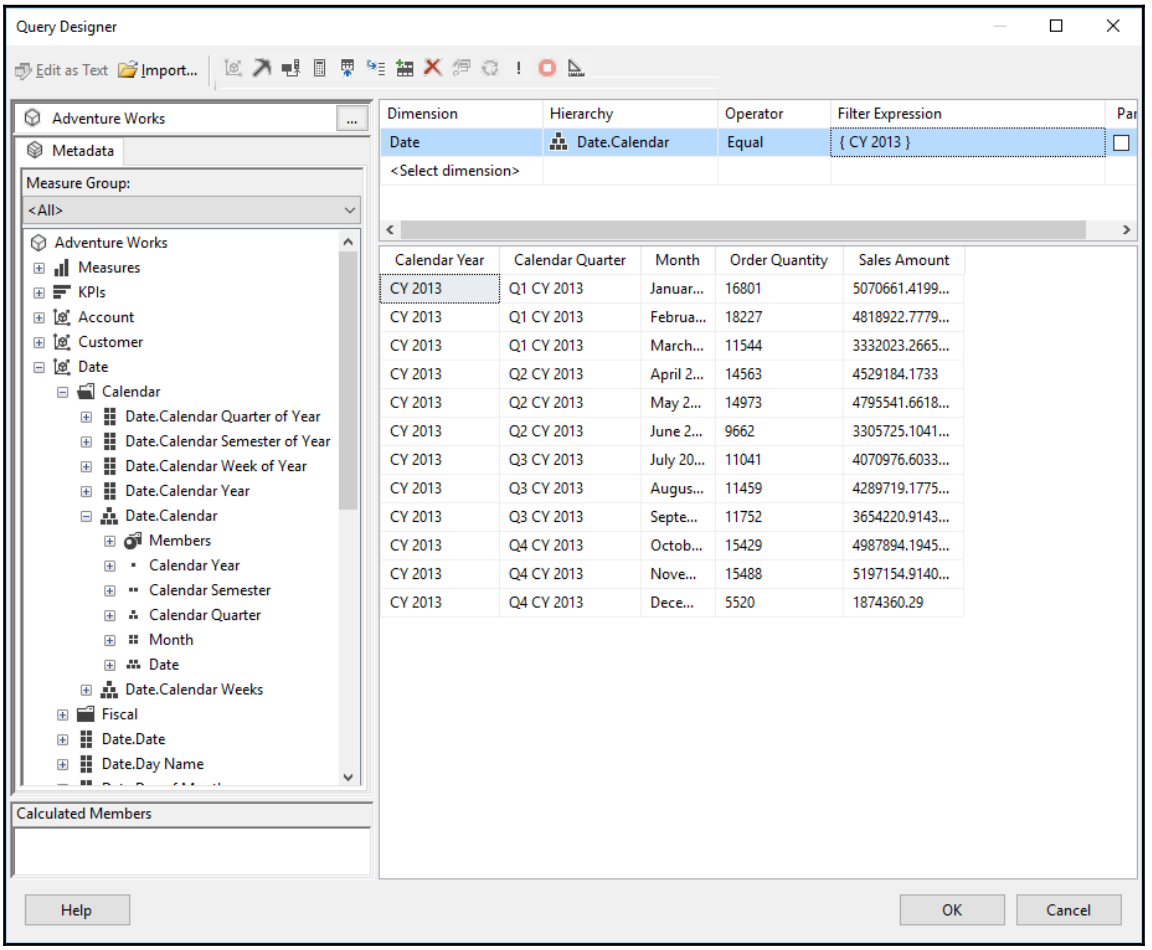
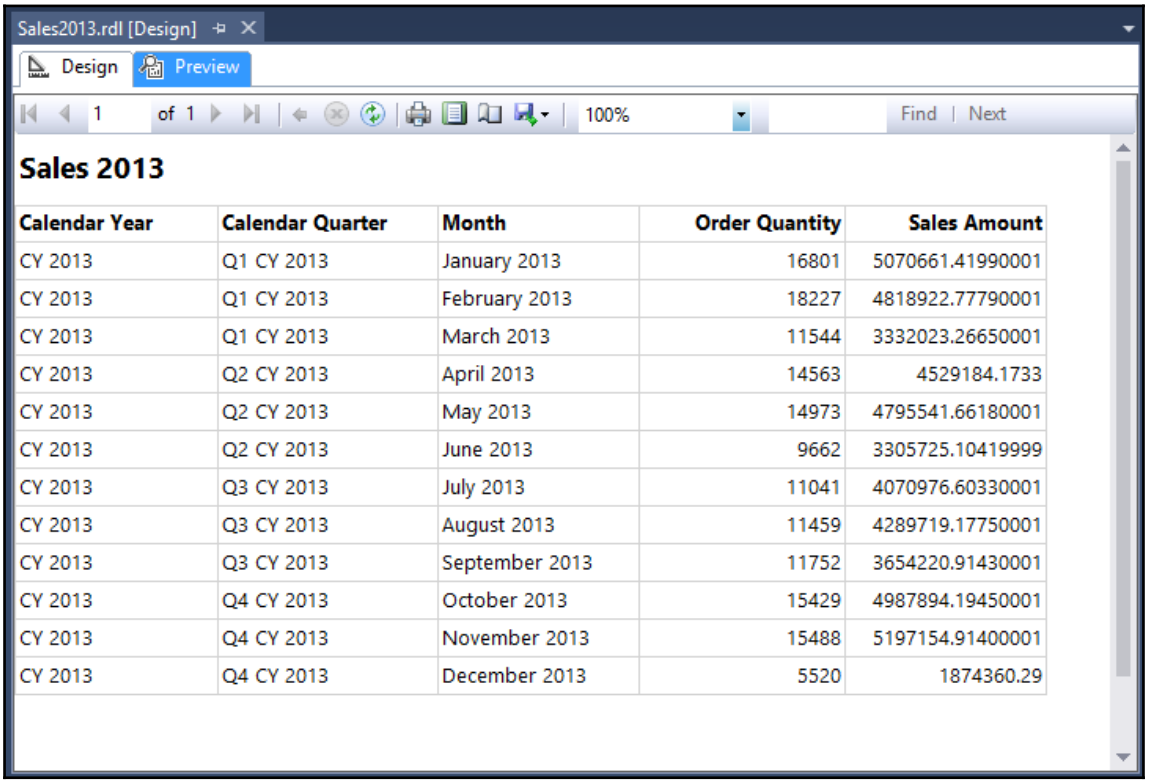


Figure 8.04

16. Click **OK** to save the settings. You will see how the MDX query has been formed based on the items selected.
17. Go to the **Fields** page and see all the selected columns in the grid. Click **OK** to save the settings and close the **Dataset properties** window.

18. Now we have our dataset ready. Let's add a table and look at how it displays data from an OLAP database.
19. Add a **Table data region** and add all columns in the dataset.
20. Adjust the width of the columns so that they have enough space to show values. Make **Order Quantity** and **Sales Amount** right-aligned.
21. Select the header and make the font bold.
22. Add a textbox on top of the table and set the text as `Sales 2013`.
23. Click on **Preview** to see the report. You should see an output such as that shown in the following screenshot:



Calendar Year	Calendar Quarter	Month	Order Quantity	Sales Amount
CY 2013	Q1 CY 2013	January 2013	16801	5070661.41990001
CY 2013	Q1 CY 2013	February 2013	18227	4818922.77790001
CY 2013	Q1 CY 2013	March 2013	11544	3332023.26650001
CY 2013	Q2 CY 2013	April 2013	14563	4529184.1733
CY 2013	Q2 CY 2013	May 2013	14973	4795541.66180001
CY 2013	Q2 CY 2013	June 2013	9662	3305725.10419999
CY 2013	Q3 CY 2013	July 2013	11041	4070976.60330001
CY 2013	Q3 CY 2013	August 2013	11459	4289719.17750001
CY 2013	Q3 CY 2013	September 2013	11752	3654220.91430001
CY 2013	Q4 CY 2013	October 2013	15429	4987894.19450001
CY 2013	Q4 CY 2013	November 2013	15488	5197154.91400001
CY 2013	Q4 CY 2013	December 2013	5520	1874360.29

Figure 8.05

How it works…

Similar to retrieving data from a relational database, Reporting Services makes a request to the Analysis Services database using the MDX query automatically created with the dataset. Let's have a look on the query formed. You can see the query by opening the dataset created for the report:

```
SELECT
    NON EMPTY { [Measures].[Order Quantity], [Measures].[Sales Amount] } ON COLUMNS
    , NON EMPTY { ([Date].[Calendar].[Month].ALLMEMBERS ) }
 DIMENSION PROPERTIES MEMBER_CAPTION, MEMBER_UNIQUE_NAME ON ROWS
 FROM ( SELECT ( { [Date].[Calendar].[Calendar Year].&[2013] } ) ON COLUMNS
        FROM [Adventure Works])
CELL PROPERTIES VALUE, BACK_COLOR, FORE_COLOR, FORMATTED_VALUE, FORMAT_STRING
    , FONT_NAME, FONT_SIZE, FONT_FLAGS
```

Figure 8.06

As you see, unlike with TSQL, in addition to the columns selected, additional keywords such as **NON EMPTY**, **DIMENSION PROPERTIES**, and **CELL PROPERTIES** have been added to the query to bring extra elements to the dataset. These extra elements are useful, and you will discover their uses in future recipes.

There's more…

Did you notice that the **Order Quantity** and **Sales Amount** columns are not formatted? Yes, we can format them using existing Reporting Services functionalities, but let's format them using a different technique instead. Read the following *Getting formatting from the Multidimensional model* recipe to learn more.

Getting formatting from the Multidimensional model

The OLAP structure is well known as an optimized data structure for data retrieval, which is the main reason to use it as a data source for reporting. However, there are also other advantages Reporting Services can obtain when the data source is Analysis Services. Analysis Services allows us to configure format-strings for measures and we can get the same applied when they are shown in Reporting Services reports. It is one advantage which we discuss in the recipe.

Getting ready

As a practice, we define the formatting for measures in our Multidimensional model when designing them. This ensures that all BI clients use the same formatting when displaying the values of measures and KPIs. For some of the measures, we also add additional properties such as font, font size, foreground color, and background color to give an extra indication to the value. For example, when a calculated measure is created, we can set the color of the value as green if it is higher than a predefined value or red if it is lower than the threshold. The advantage of this is that when you create the report to show the calculated measure, you do not need to make the same effort for adding the logic and coloring the value because it can be retrieved from the model.

The following screenshot shows how we set formatting for a measure:

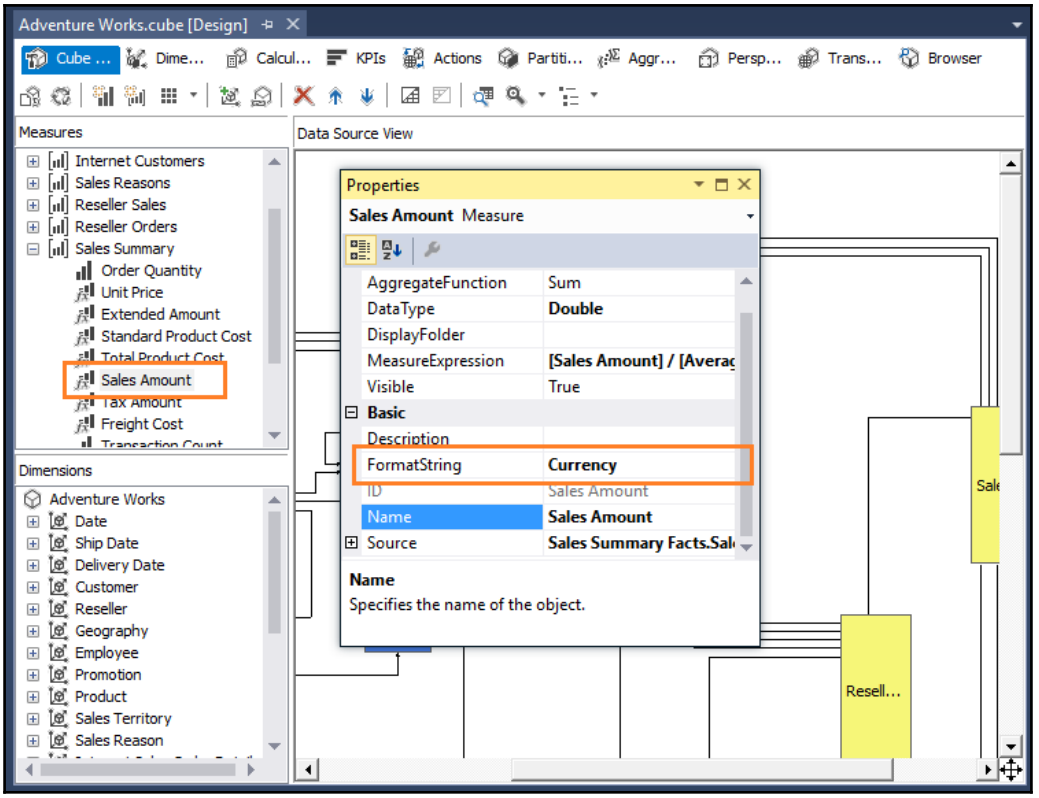


Figure 8.07

We created a `Sales2013` report with the previous recipe but we did not format the values.

How to do it…

Let's format a set for Order Quantity and Sales Amount from the Multidimensional model:

1. Open the **Design** of the `Sales2013` report created in the *Creating a report using a Multidimensional model* recipe.
2. Right-click on the **Order Quantity** cell and select **Expression** from the menu:

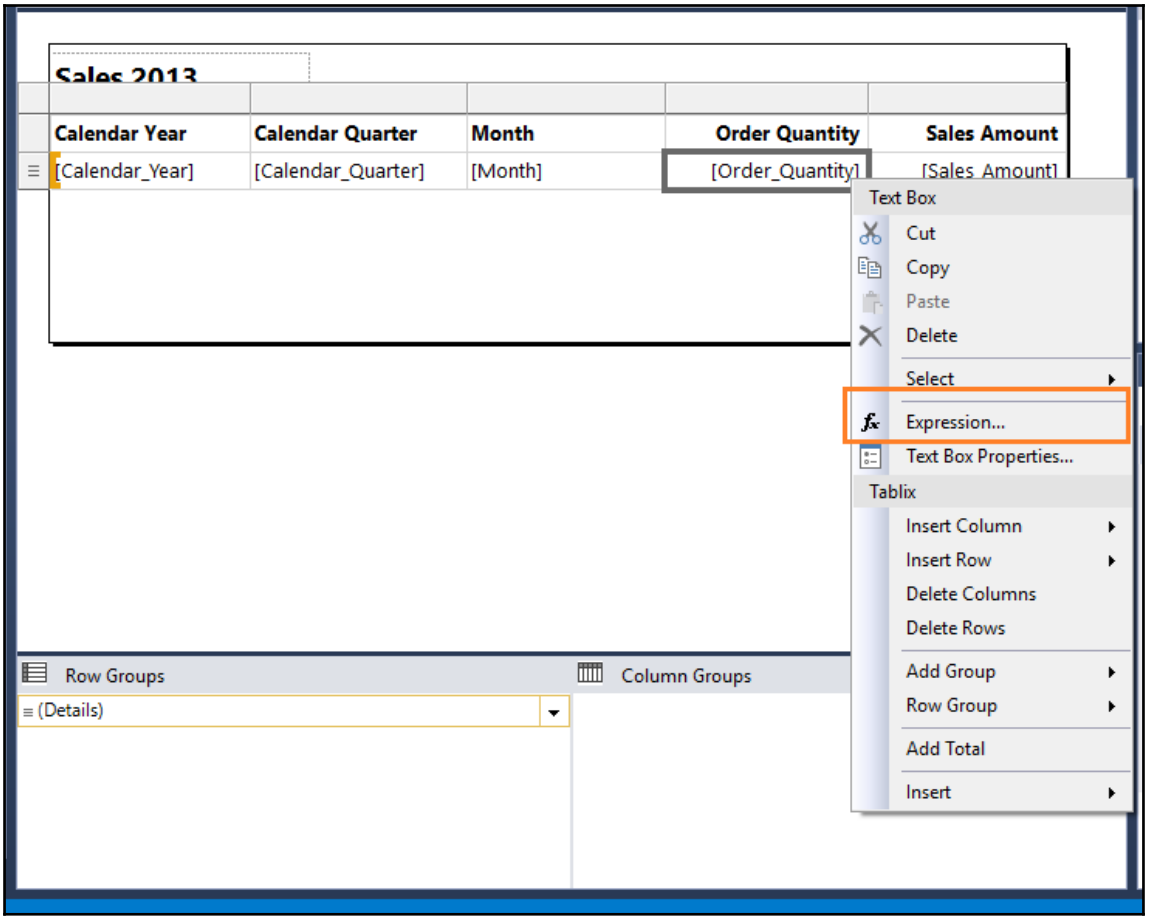


Figure 8.08

3. You will see that the **Value** property **Fields!Order_Quantity** has been selected as the expression. Set **FormattedValue** instead of **Value**:

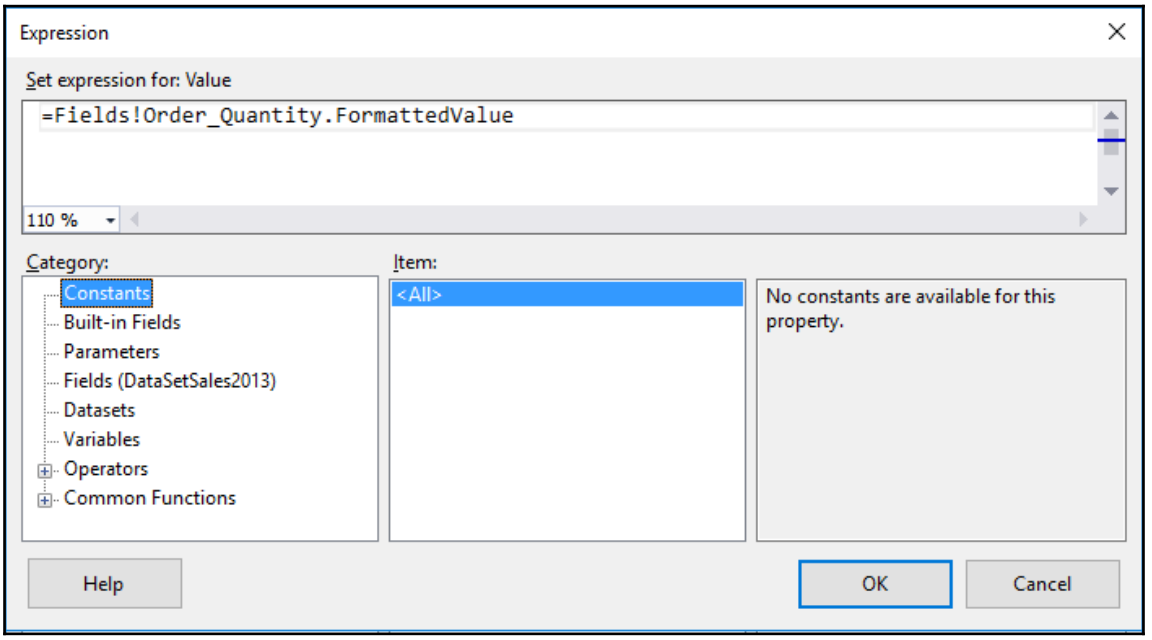


Figure 8.09

4. Click **OK** to save the settings.
5. Get the expression related to **Sales Amount** and repeat steps 3 and 4.
6. Preview the report. You will see that formatting is now derived from the data model (see Figure 8.10).



We generally add totals for reports such as this. Reporting Services allows us to add additional rows to the table data region, which we can use to show headers and footers. Let's show totals and see whether we can apply the same formatting to totals as well.

7. Go back to the design mode of the report and add a new row to the table to show totals. Make sure that you select the **Outside Group – Below** option to add the row:

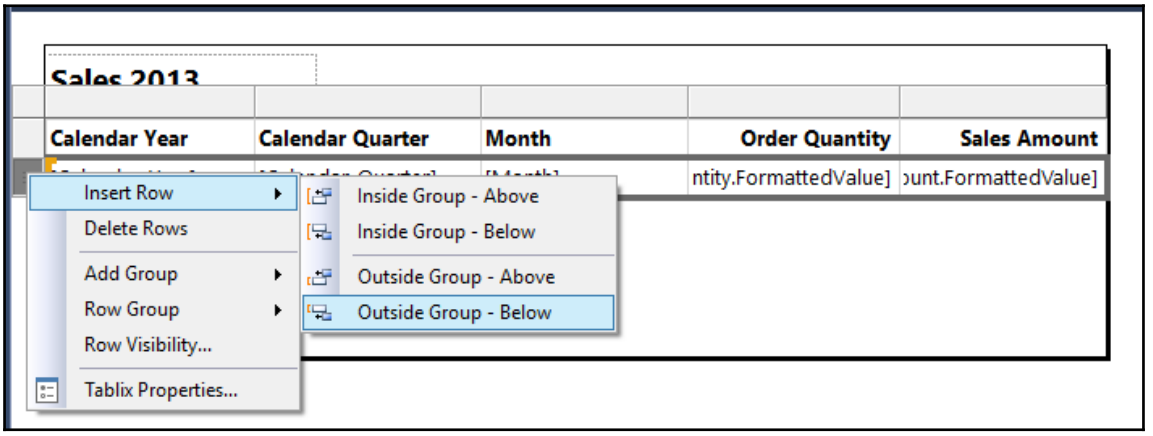


Figure 8.10

8. Set **Order Quantity** for the fourth column and **Sales Amount** for the fifth column. You will see that the **SUM** aggregate function is automatically added:

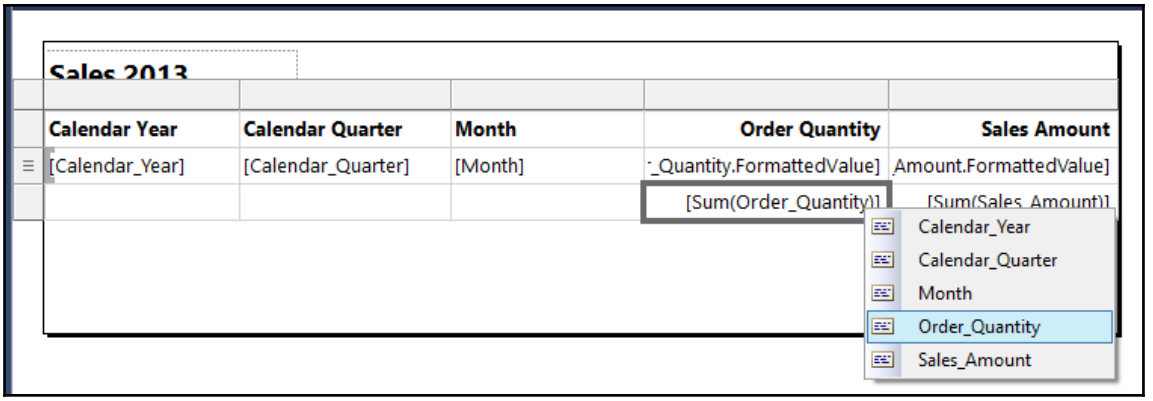


Figure 8.11

9. Preview the report. You will see that totals have appeared but formatting is not applied.

10. The `FormattedValue` function cannot be used with **Totals** as you used with individual cells. However, formatting applied to individual cells can be extracted using the `FORMAT_STRING` property and applied to **Totals**.
11. Following is the method for getting the `FORMAT_STRING` from the **Order Quantity** field:

```
Fields!Order_Quantity("FORMAT_STRING")
```

12. Reporting Services does not recognize all formatting strings. Therefore, some formatting strings must be changed as shown here:

```
Iif(Fields!Order_Quantity("FORMAT_STRING") = "Currency", "c"
  , Iif(Fields! Order_Quantity ("FORMAT_STRING") = "Percent", "p"
  , Iif(Fields! Order_Quantity ("FORMAT_STRING") = "Standard",
   "0"
  , Fields! Order_Quantity ("FORMAT_STRING") )))
```

13. Select the cell that is set for the total of the **Order Quantity** and open the **Properties** window. Find the **Format** property and select **Expression**:

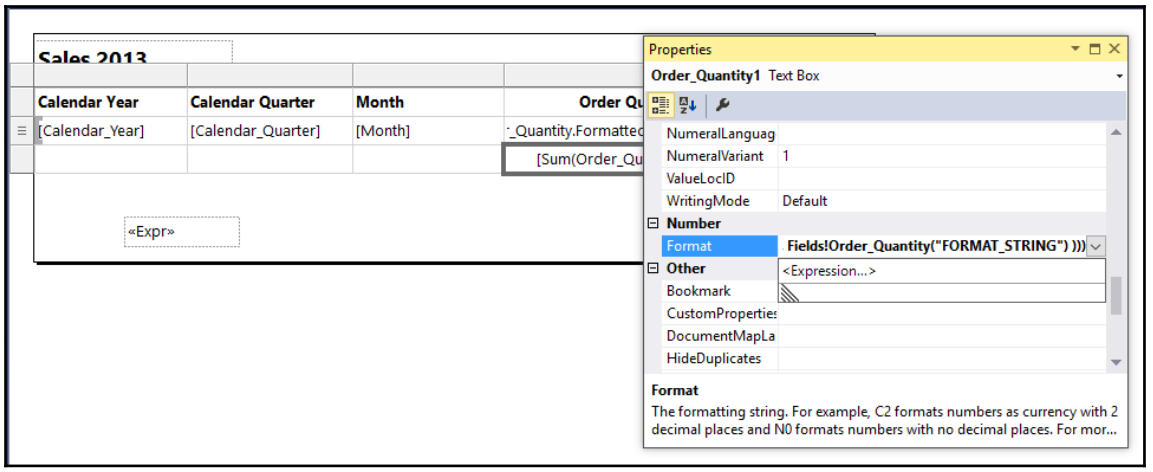
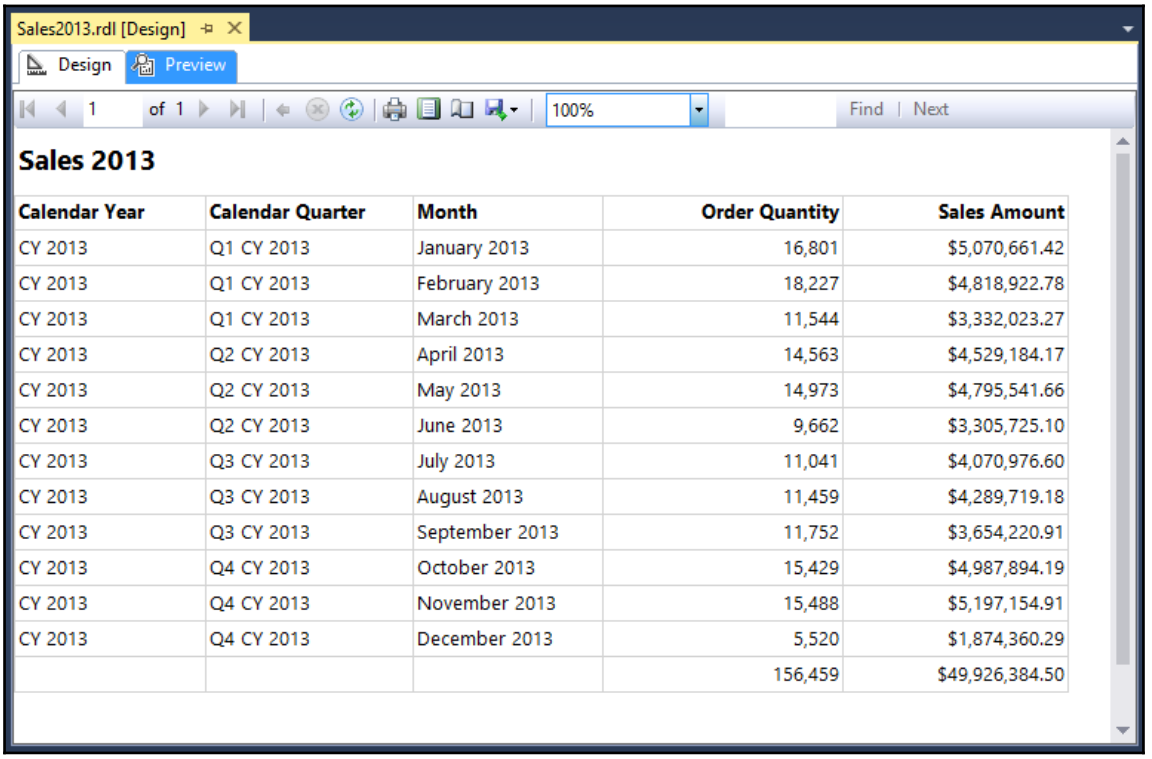


Figure 8.12

14. Write the expression as mentioned previously and save the settings.
15. Select the cell related to total **Sales Amount** and implement the same.

16. Preview the report. You will see that the same formatting is now applied to totals as well:



Calendar Year	Calendar Quarter	Month	Order Quantity	Sales Amount
CY 2013	Q1 CY 2013	January 2013	16,801	$5,070,661.42
CY 2013	Q1 CY 2013	February 2013	18,227	$4,818,922.78
CY 2013	Q1 CY 2013	March 2013	11,544	$3,332,023.27
CY 2013	Q2 CY 2013	April 2013	14,563	$4,529,184.17
CY 2013	Q2 CY 2013	May 2013	14,973	$4,795,541.66
CY 2013	Q2 CY 2013	June 2013	9,662	$3,305,725.10
CY 2013	Q3 CY 2013	July 2013	11,041	$4,070,976.60
CY 2013	Q3 CY 2013	August 2013	11,459	$4,289,719.18
CY 2013	Q3 CY 2013	September 2013	11,752	$3,654,220.91
CY 2013	Q4 CY 2013	October 2013	15,429	$4,987,894.19
CY 2013	Q4 CY 2013	November 2013	15,488	$5,197,154.91
CY 2013	Q4 CY 2013	December 2013	5,520	$1,874,360.29
			156,459	$49,926,384.50

Figure 8.13

How it works…

Reporting Services gets this formatting into the dataset along with the data because of extra keywords added to the MDX query by Query Designer when the dataset is designed. If you recall the MDX statement used for this report, it has `CELL PROPERTIES`, which includes `FORMAT_STRING`, which in turn brings formatting to Reporting Services.

Although we use a lengthy expression to handle this, you can have the logic implemented in a function and use the function to get the correct formatting. This makes the expression simple (for example, `Code.GetFormatting(Fields!Field("FORMAT_STRING"))`) and reusable. We will discuss this in more detail in `Chapter 12`, *Custom Programming and Integration to .NET Applications*.

There's more…

You might have already noticed that `CELL PROPERTIES` brings some more properties by default. Among them, properties such as `FORE_COLOR` and `BACK_COLOR` are useful when **Calculated Measures** are defined and set with colors based on conditions, and you need it in Reporting Services. Note that some dimension properties are added by default and can be used when required.

Getting aggregated values from the Multidimensional model

There are many important things to be discussed related to Reporting Services and Analysis Services. Since we have no way of discussing everything, let's take one more item that Reporting Services supports natively on an Analysis Services capability.

Analysis Services supports three different types of measures: **additive**, **semi-additive**, and **non-additive**. Generally, most of the measures are additive and easy to work. However, some of the measures cannot be summed up along with the dimensions used as they require a specific calculation. For example, stock needs to be maintained daily, and monthly stock is not just the total of all stock values for all dates for the particular month, it is the stock value of the last day of the month. This type of measure is called semi-additive.

Let's look at how Reporting Services supports the display of semi-additive measures.

Getting ready

Let's create a new report using the following steps:

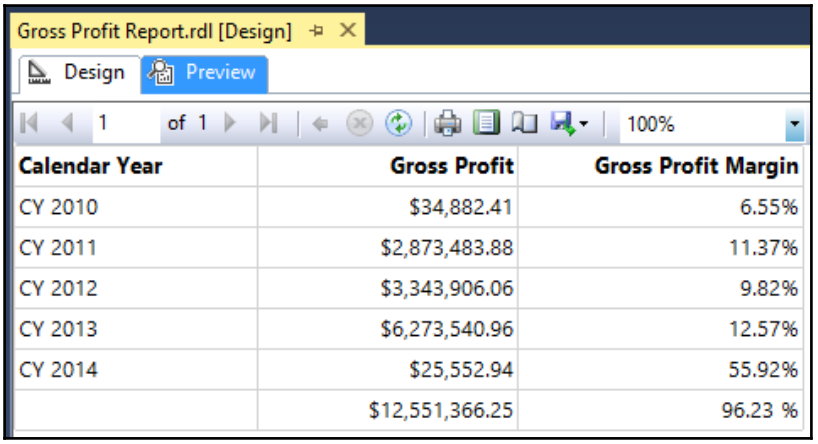
1. Add a new report to the solution and name it `Gross Profit Report`.

2. Add a dataset named **DataSetGrossProfit** using the **DataSourceAdventureWorksOLAP2014** data source. Add the **Calendar Year** attribute hierarchy from the **Date** dimension and **Gross Profit** and **Gross Profit Margin** measures from the **Sales Summary** measure group:

Calendar Year	Gross Profit	Gross Profit Margin
(null)	12551366.2483406	0.114301513596512
CY 2010	34882.4084	0.0654761775725588
CY 2011	2873483.87559989	0.113718794628328
CY 2012	3343906.06229999	0.098245816028063
CY 2013	6273540.96440021	0.125655823620716
CY 2014	25552.9376	0.559209851816577

Figure 8.14

3. Add a table-data region to the report. Add all three fields from the newly created dataset. Make sure **Gross Profit** and **Gross Profit Margin** columns are right-aligned.
4. Add a new row to the table and set totals for **Gross Profit** and **Gross Profit Margin**. Get formatting from **Gross Profit** and **Gross Profit Margin** and apply it to totals (follow the same steps described in the previous recipe).
5. Preview the report and note the totals:



Calendar Year	Gross Profit	Gross Profit Margin
CY 2010	$34,882.41	6.55%
CY 2011	$2,873,483.88	11.37%
CY 2012	$3,343,906.06	9.82%
CY 2013	$6,273,540.96	12.57%
CY 2014	$25,552.94	55.92%
	$12,551,366.25	96.23 %

Figure 8.15

As you can see, although the total of **Gross Profit** is correct, the total of **Gross Profit Margin** is wrong. If you query the **AdventureWorks** model for the **Gross Profit Margin** without the **Calendar Year** attribute hierarchy, you will see the correct value, which is **11.43%**. Let's see how we can correct this:

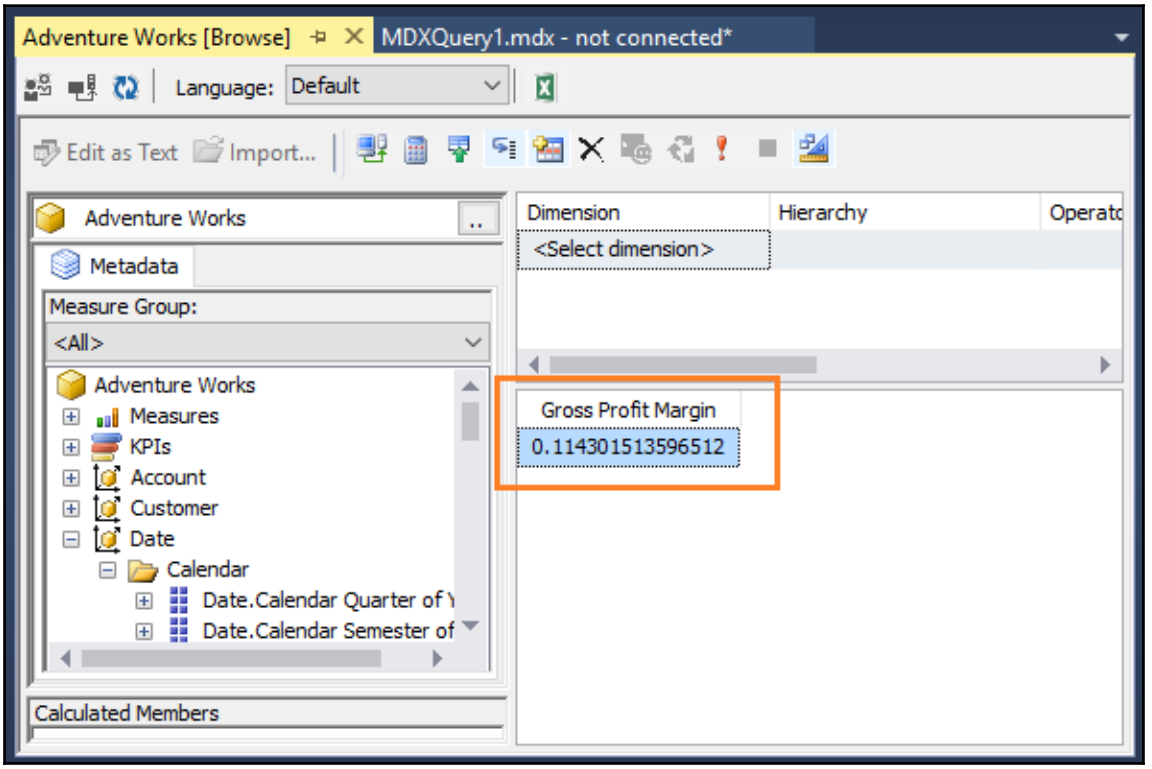


Figure 8.16

How to do it…

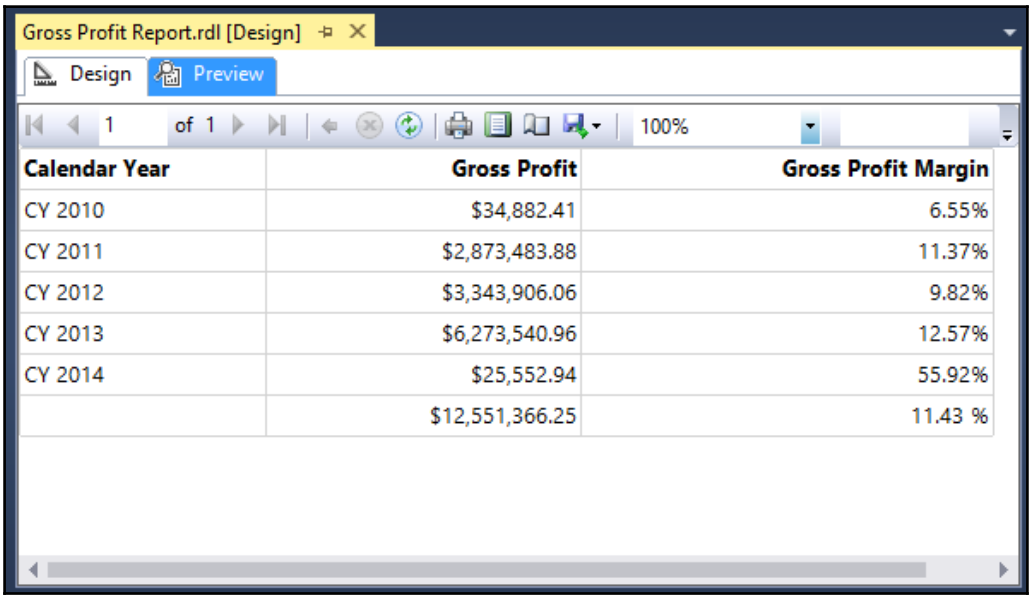
In order to get the correct value, we need to stop Reporting Services to calculate it, but we can take it from the model. This can be achieved using the **Aggregate** function:

1. Open the design of the report and replace the **SUM** function with the **Aggregate** function in the total cell related to **Gross Profit Margin**:

Calendar Year	Gross Profit	Gross Profit Margin
[Calendar_Year]	[Gross_Profit.FormattedValue]	[Gross_Profit_Margin.FormattedValue]
	[Sum(Gross_Profit)]	[Aggregate(Gross_Profit_Margin)]

Figure 8.17

2. Preview the report. You should now see the correct percentage for **Gross Profit Margin**:



Calendar Year	Gross Profit	Gross Profit Margin
CY 2010	$34,882.41	6.55%
CY 2011	$2,873,483.88	11.37%
CY 2012	$3,343,906.06	9.82%
CY 2013	$6,273,540.96	12.57%
CY 2014	$25,552.94	55.92%
	$12,551,366.25	11.43 %

Figure 8.18

How it works…

Analysis Services knows how to calculate the measure value at each level of the dimension, based on the instructions set with its expression. An expression can be written to calculate using certain factors and conditions at one level of the dimension and calculate using different factors and conditions at another level. This cannot be understood by Reporting Services, and we should not try to implement the same logic in Reporting Services either. Since Analysis Services knows to calculate and, in most cases, it calculates the aggregate values and keeps improving the performance, we should let Reporting Services use Analysis Services to get calculated values instead of calculating them itself. The Aggregate function instructs Reporting Services not to calculate but to obtain a value from Analysis Services instead.

There's more…

Although we only applied the Aggregate function to **Gross Profit Margin**, it is always better to use the functions for all types of measures. This ensures that Reporting Services does not worry about values and passes the responsibility to Analysis Services.

You will have noticed that the query contains the `NON EMPTY` keyword by default. This eliminates empty records being brought into the dataset, but in some cases you may need them as well. If required, remove the `NON EMPTY` keyword either by manually editing the MDX or by using the **Show Empty Cells** toggle button in the designer.

There is another toggle button, called **Show Aggregates**, which brings aggregated values into the result. However, by default, these values are not shown as part of the record set but will be used for the Aggregate function, if it is used. If you need them in the table, change the option **Interpret subtotals as detail rows** to true in the dataset properties.

Creating a report using Tabular models

As mentioned at the beginning of this chapter, a Tabular model is one of the model types that Analysis Services supports. There are many differences between Multidimensional and Tabular models. Holding data in the memory, simple tabular data structures, simpler design, and a comparatively shorter amount of time to create a model are some of the key differences. Some organizations still prefer the Multidimensional model because it is the richest platform for creating a BI solution, but you can try using a Tabular model because it is less complex and, most importantly, it can be implemented within a shorter time period.

In this recipe, we will look at how to create a simple report connecting with a Tabular model. Remember, with a simple report that loads many records, the Tabular model might offer greater performance than the Multidimensional model since the data is in the cache, but there may be certain scenarios where you see slower performance, particularly with multiple aggregations. However, as per implementations done in both, no significant performance difference has been recorded yet.

Getting ready

The expressions of the Tabular model are implemented using an expression language called DAX, which is used to query the database. However, you can use both DAX and MDX to query the database. When connecting with Reporting Services, you will see the same designer for selecting fields and for creating the dataset. Once the dataset is created, it shows the query formed as an MDX statement, not as a DAX expression.

Let's create a report using the Tabular model and see.

Note that you need to restore the AdventureWorks Tabular Model database in order to create this report. Make sure it is restored and can be browsed.

How to do it…

Open the Reporting Services solution you are using with this chapter and work through the following steps to create the report:

1. Create a new report in the `Chapter08` solution and name it `Internet Sales`.
2. Modify the shared data source `DataSourceAWInternetSalesTabularModel2014`, connecting with the `AW Internet Sales Tabular Model 2014` database:

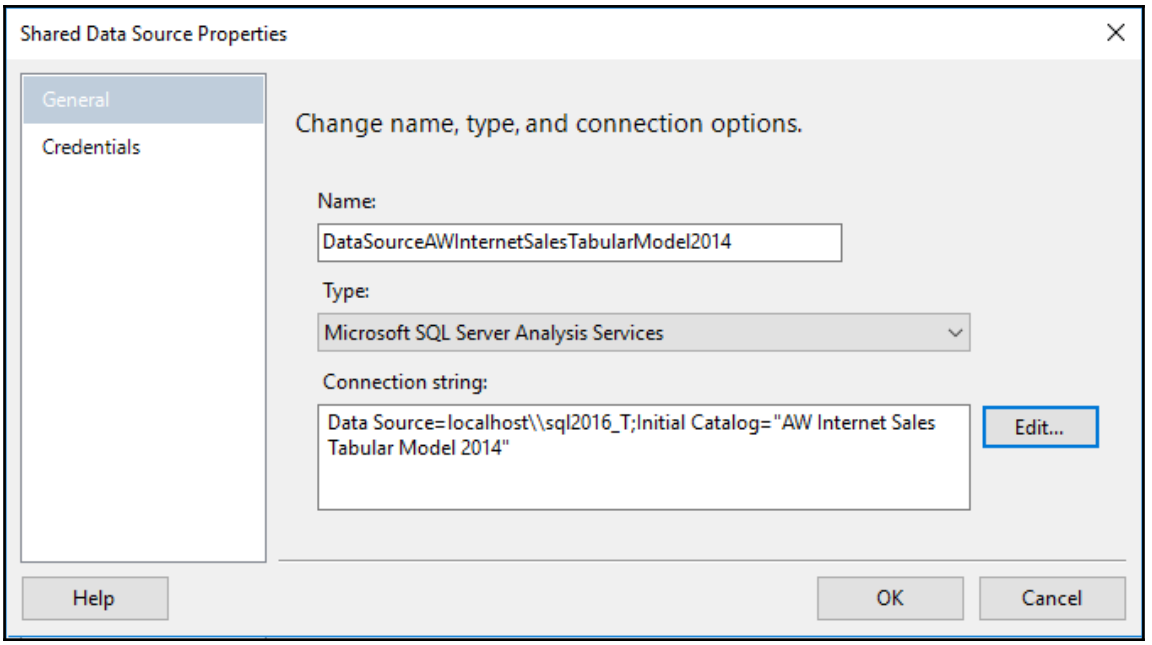


Figure 8.19



Note that if you use a new Reporting Services solution instead of `Chapter08`, then you need to create the shared data source.

3. Create a new dataset named `DataSetInternetSales` and use the data source `DataSourceAWInternetSalesTabularModel2014` for connecting with the Tabular model database.
4. Click on the **Query Designer** button to open Query Designer. You will see the same designer that you used with the Multidimensional model.
5. Expand **Measures** in the left-hand pane. Then expand **Internet Sales** to see all available measures (see Figure 8.20).
6. Drag and drop **Internet Total Sales** and **Internet Total Tax Amt** onto the right-hand pane (see Figure 8.20).
7. Expand the **Date** table and drag and drop **Calendar Year** to the same data grid (see Figure 8.20).

8. Let's add a parameter for filtering. Expand **Product Category**. Drag **Product Category Name** into the filter pane (top-right-hand pane) and filter it for **Accessories**. Make it as a parameter. Query Designer should now look like the following:

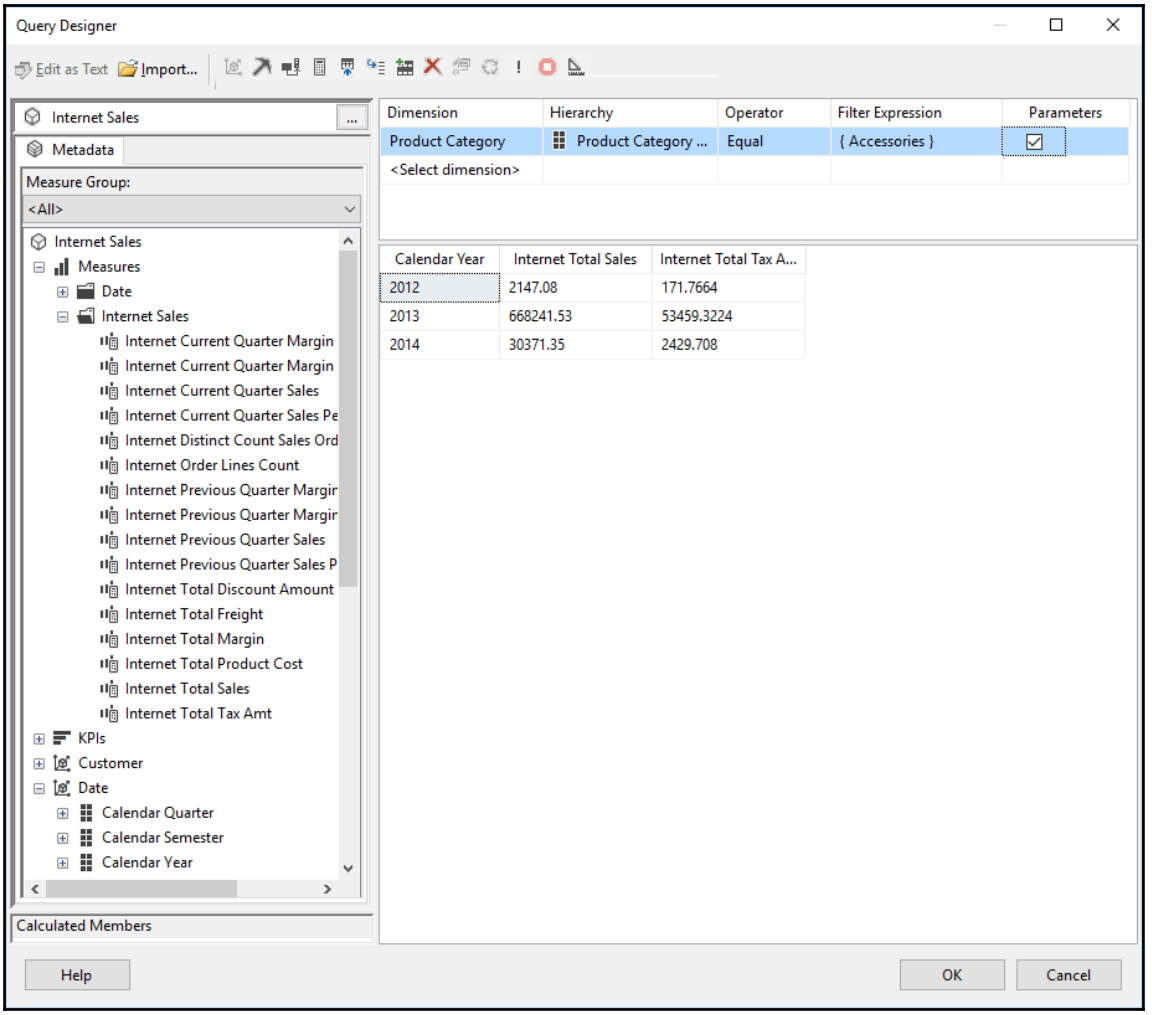


Figure 8.20

9. Click on **OK** to save the settings. You will see an MDX statement instead of a DAX expression:

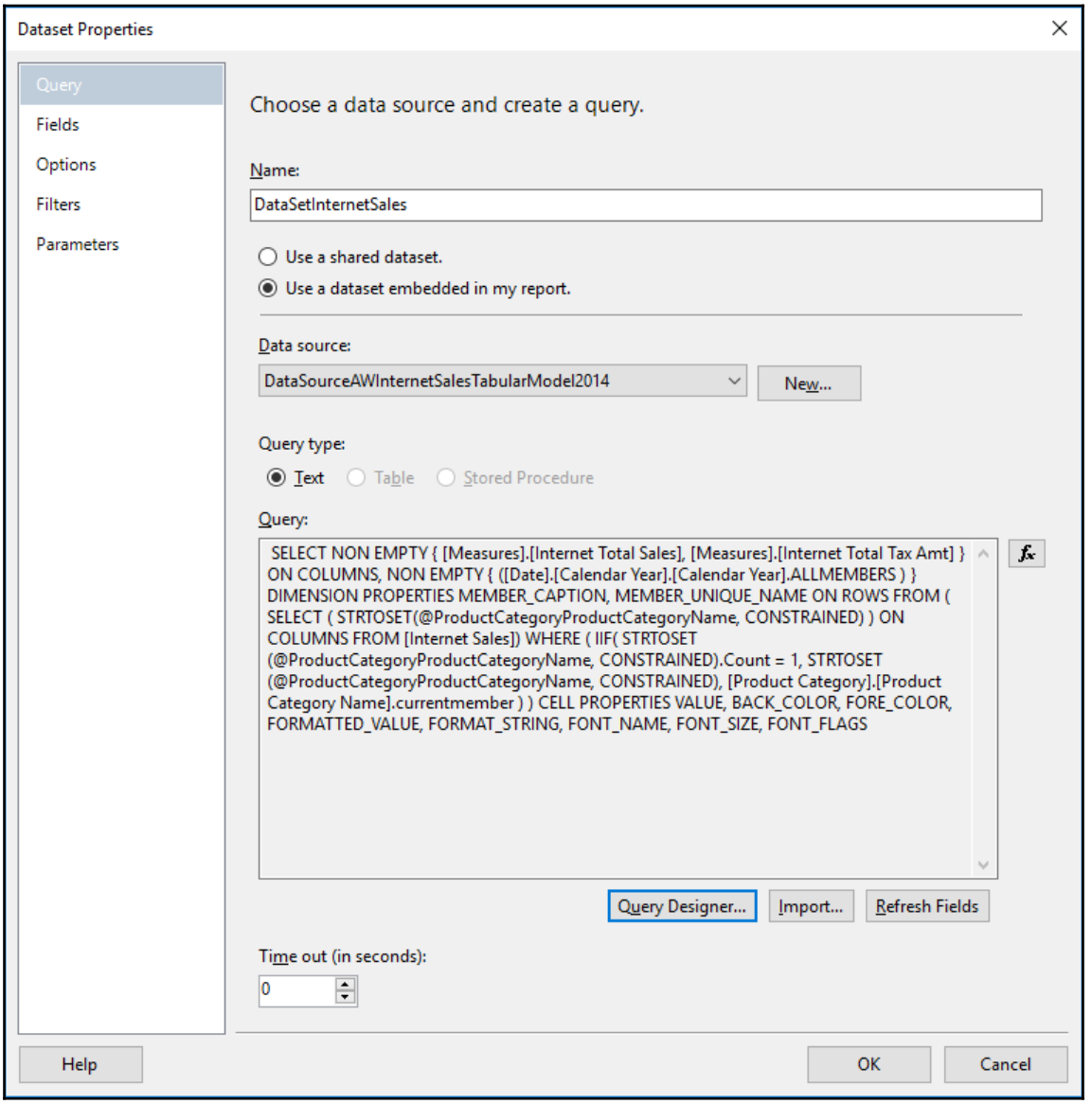


Figure 8.21

10. Click **OK** to create the dataset.
11. Now add a **Table data region** and add all columns to the table.
12. Change the expressions of **Internet Total Sales** and **Internet Total Tax Amt** cells as they use the **FormattedValue** property instead of the **Value** property, (Refer to the *Getting formatting from the Multidimensional model* recipe for details about setting formatting.)
13. Just like the report created with the Multidimensional model, you should now be able to run the report and see the preview.

How it works…

There is not much difference between a report created using a Multidimensional model and one created using a Tabular model in terms of design. However, when Reporting Services makes a request from a Tabular model, the request is satisfied by the Vertipaq engine using the data in the memory rather than in the disk. As mentioned previously, the query will be stratified immediately as it is a known fact that the fastest storage is memory, but a model with many tables and a query with many joins and aggregations might not give you the best performance as a Multidimensional model.

Connecting to the HDInsight cluster and getting unstructured data

Everyone talks about big data and everyone wants big data in their BI solution, hence previously-ignored data that is semi-structured or unstructured has now become part of BI solutions. Generally, the **ETL** solution transforms semi-structured or unstructured data into a structured format using a platform such as **Hadoop** and loads the transformed data back to the data warehouse. However, there are instances where we hold unstructured data in an environment such as Hadoop and query it directly when required for analysis and reporting using Hadoop-supported projects such as **Hive** or **Pig**.

Microsoft provides the Hadoop environment as a cloud service and it is known as **HDInsight Cluster**. It not only supports Hadoop, but also **HBase**, **Spark**, and **Storm.** You can create your Hadoop cluster in **Azure** as an HDInsight cluster to hold semi-structured and unstructured data, and query it using either Pig or Hive.

In this recipe, let's see how we can query unstructured data stored in HDInsight via Reporting Services and show it with a report.

Getting ready

In order to query data from HDInsight using Reporting Services, we need to make sure of the following:

- HDInsight Hadoop Cluster is created in the organization's Azure
- The Microsoft Hive ODBC driver is installed in the development machine and Reporting Services server.

First you need an Azure account to create an HDInsight Hadoop Cluster. The cluster requires a storage account; you can either create one when the cluster is created or use pre-existing storage.

Let's use a text file that contains sample **Caller Data Record** (**CDR**) records. The following screenshot shows the structure of the file:

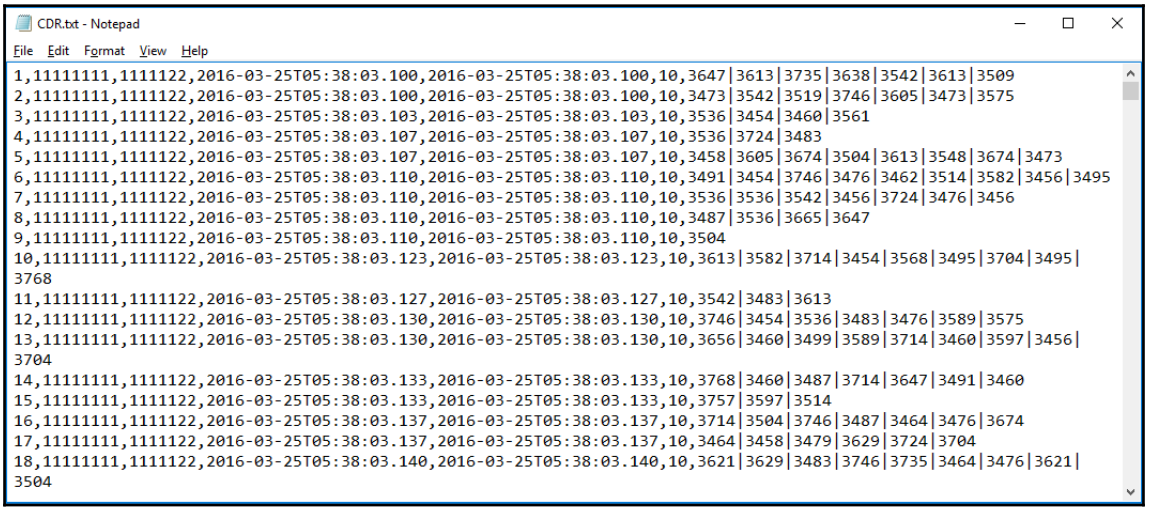


Figure 8.22

This file contains sample records, which are related to calls made by customers. The usual CDR file is a bit different from this and contains many more fields, but let's use this for testing purposes. A record in this file describes the record ID, ANumber, BNumber, StartDateTime, EndDateTime, Call Duration, and cell numbers in towers used for handling the call. As you can see, the number of cells related to each call is different.

Let's see how we can create an HDInsight Cluster to store similar CDR files:

1. Open a browser and go to `http://portal.azure.com`to open the **Azure portal**.
2. Log in to the portal using your credentials. If you do not have an **Azure account**, you need to create one before continuing.
3. If you do not have **HDInsight Clusters** marked as a favorite, you will not see it in the left-hand pane. Click on **More Services**, search for **HDInsight Clusters**, and open it. Click on **Add** to create an HDInsight cluster:

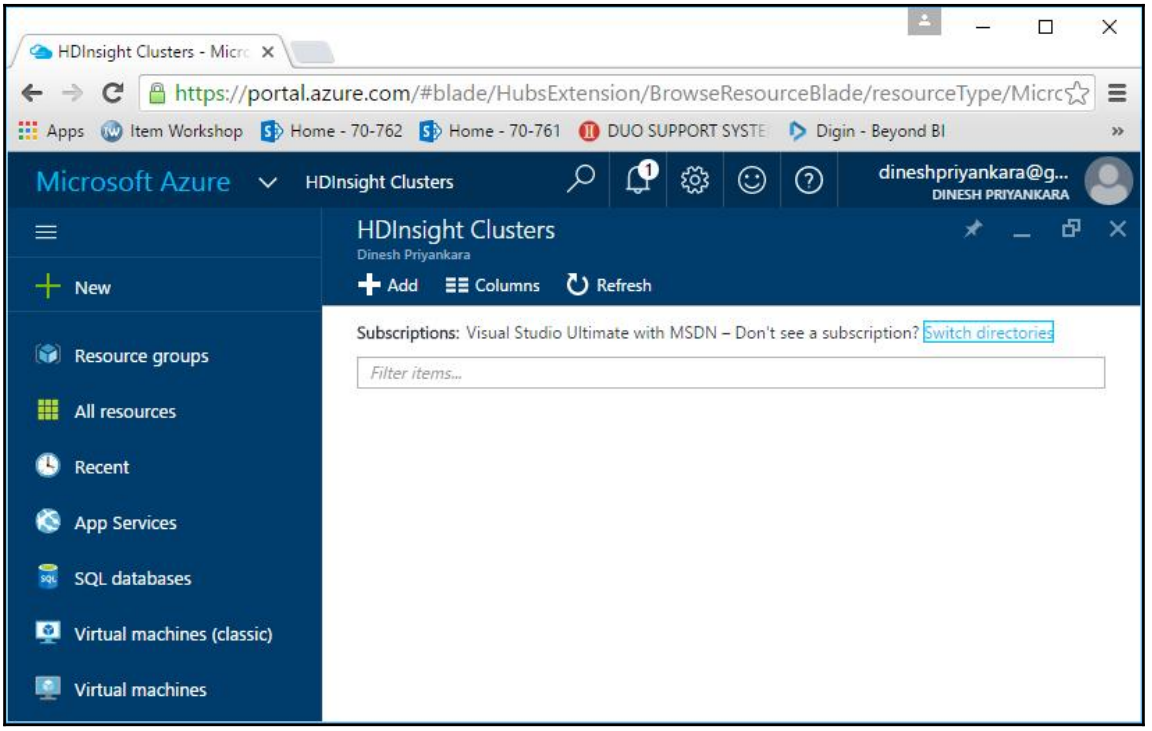


Figure 8.23

4. This opens a **New HDInsight Cluster** blade. Enter the name of the cluster as `ssrs2016democluster`, if available. If not, enter a different name.



Note that the name of the cluster must be globally unique. Once it is created, it is referred to as `<name>.azurehdinsight.net`.

5. Select the **Subscription** related to your Azure account (see Figure 8.24).
6. Configure the **Cluster Type**. Click on **Select cluster type** and it will open a **Cluster Type Configuration** blade. Select **Hadoop** as the type and select the **STANDARD** cluster tier:

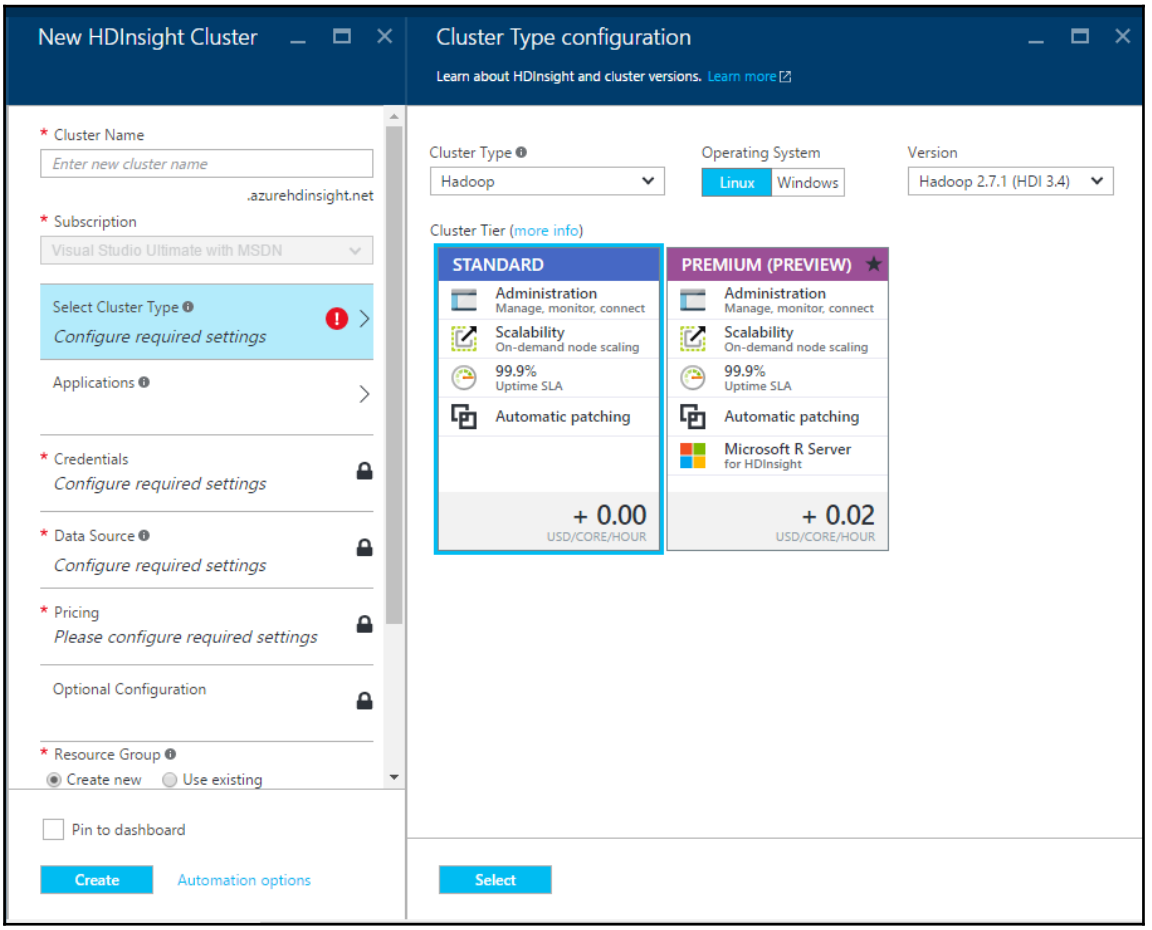


Figure 8.24

7. Click on **Credentials** to open the **Cluster Credentials** blade. Configure a password for the admin account and set a username and password for **SSH Username**:

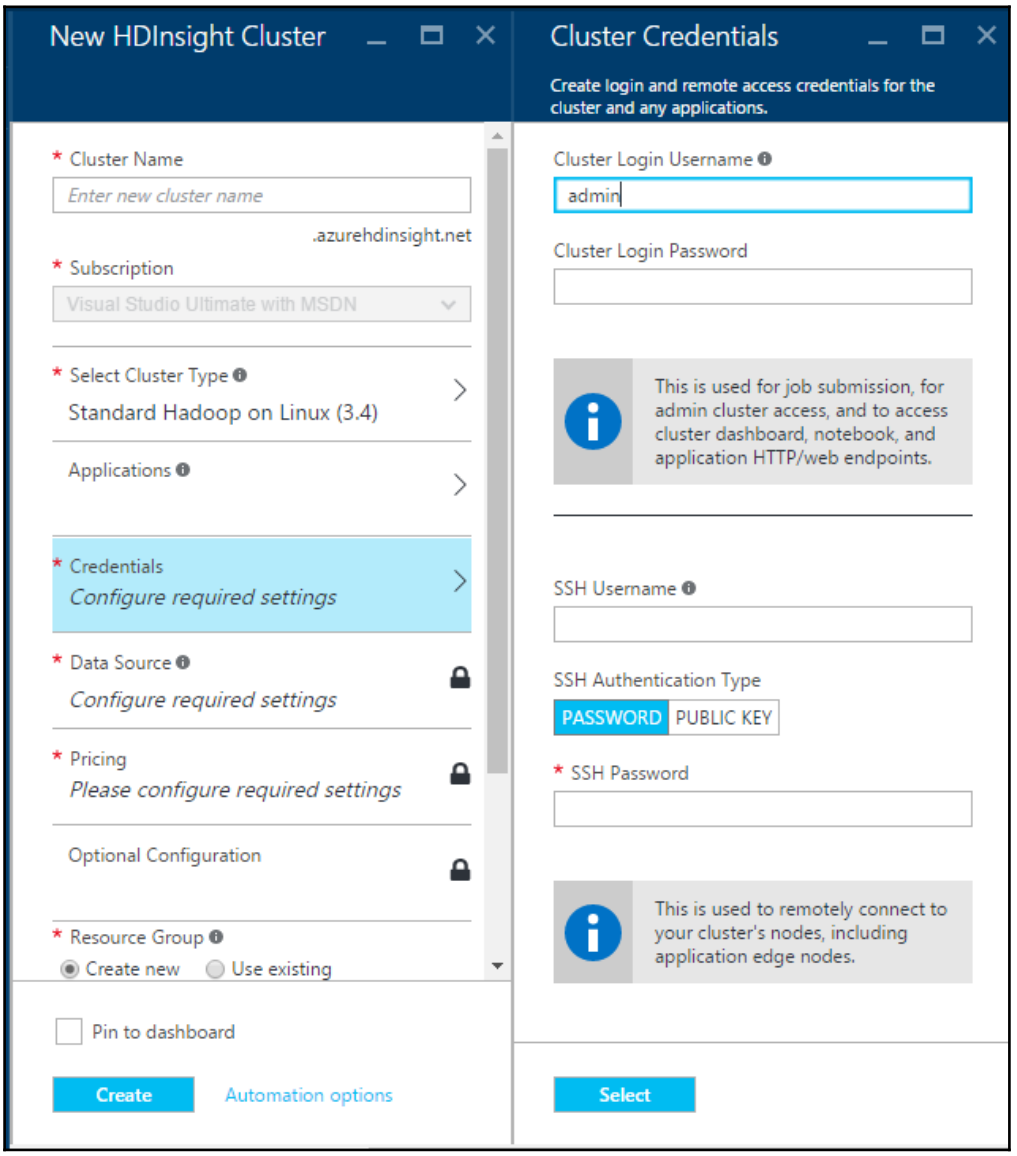


Figure 8.25

8. Click on **Data Source** to open a **Data Source** blade. Enter a name for your new storage account and container. It is better to use the same name used for the cluster for both the storage account and container (see Figure 8.26).
9. Once entered, select the **Location** appropriate for your region:

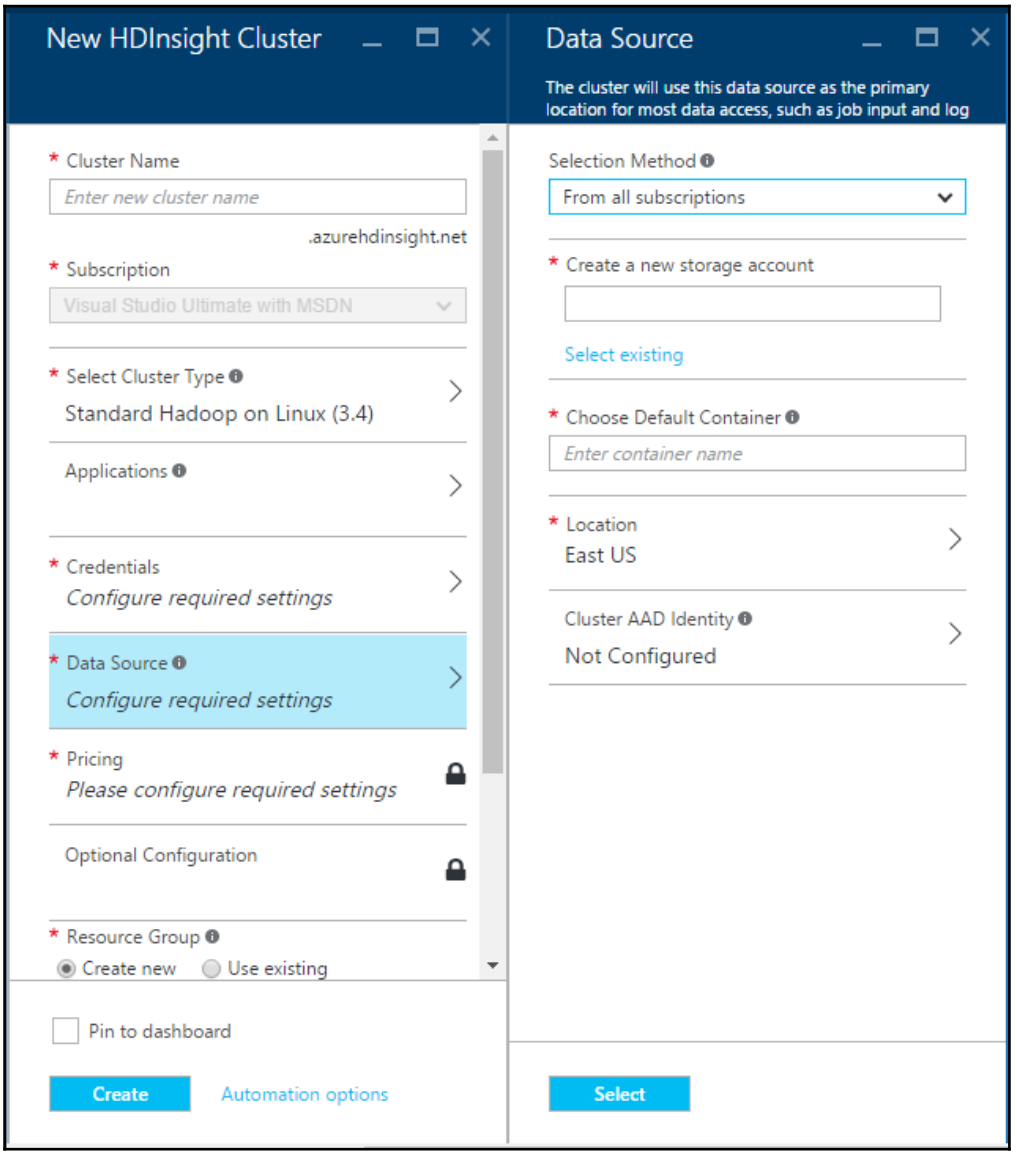


Figure 8.26

10. Click on **Pricing** to set the number of nodes you need in your cluster. The default is sufficient for this test, but you can view all options available for **Worker node size** and **Head node size** and select the best options for your organization:

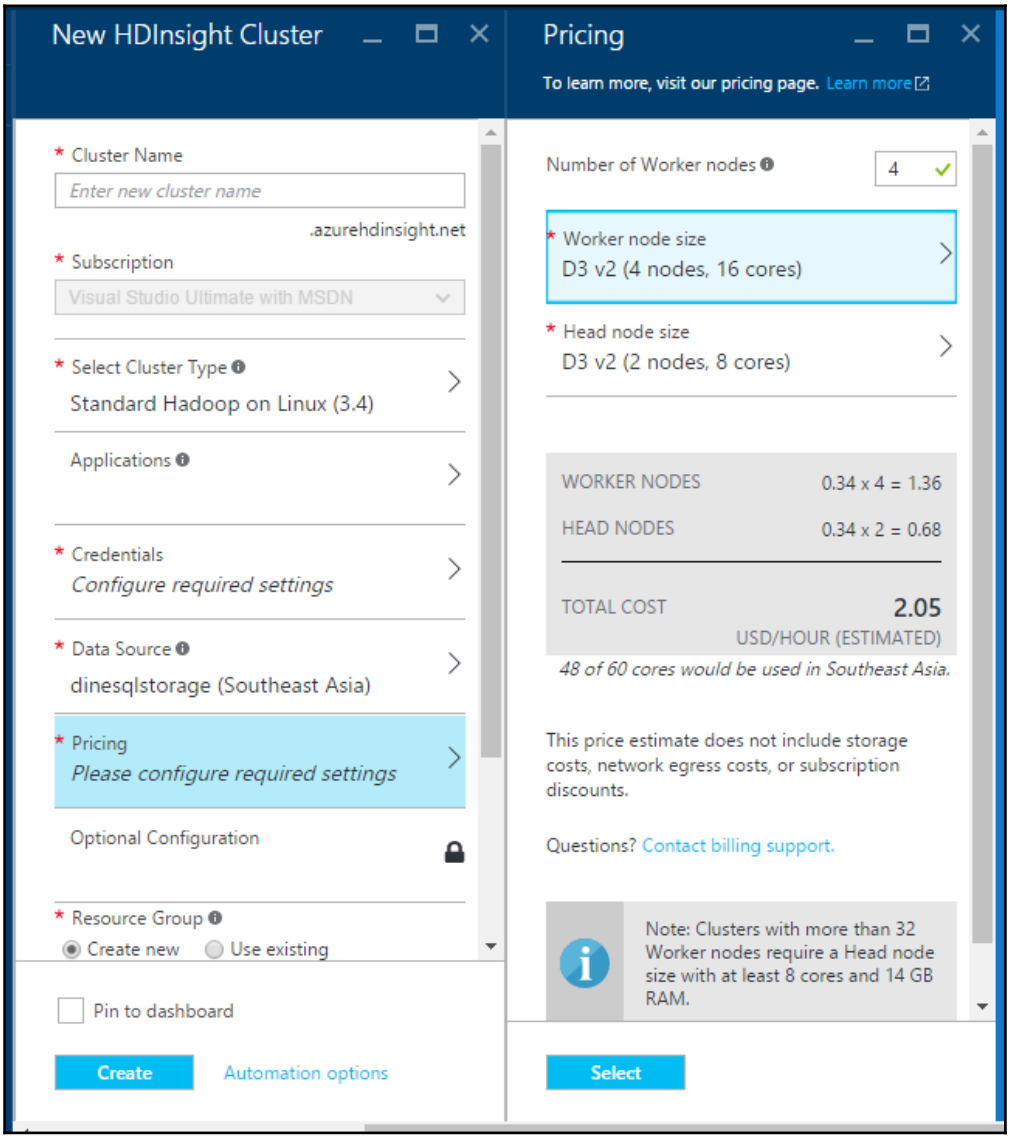


Figure 8.27

11. You can create a new **Resource Group** if you have not created one before. If you have one, select it and click on **Create** to create the cluster.

Generally, it takes about 10-15 minutes to create the cluster and storage. Once it is created, it is always better to check the container using a tool that can access cloud storage.

Let's use Microsoft Azure Storage Explorer to check storage. If you do not have it installed, download it from `http://storageexplorer.com`). You can use your **Storage Account Name** and **Primary Key** to connect with it (and you can find the keys of your storage by navigating to **Cluster** | **Azure Storage Keys** | **Storage** | **Access keys**). Once connected, create a folder called `CDRFiles` inside the container and upload the `CDR.txt` file given with `Chapter08 solution`:

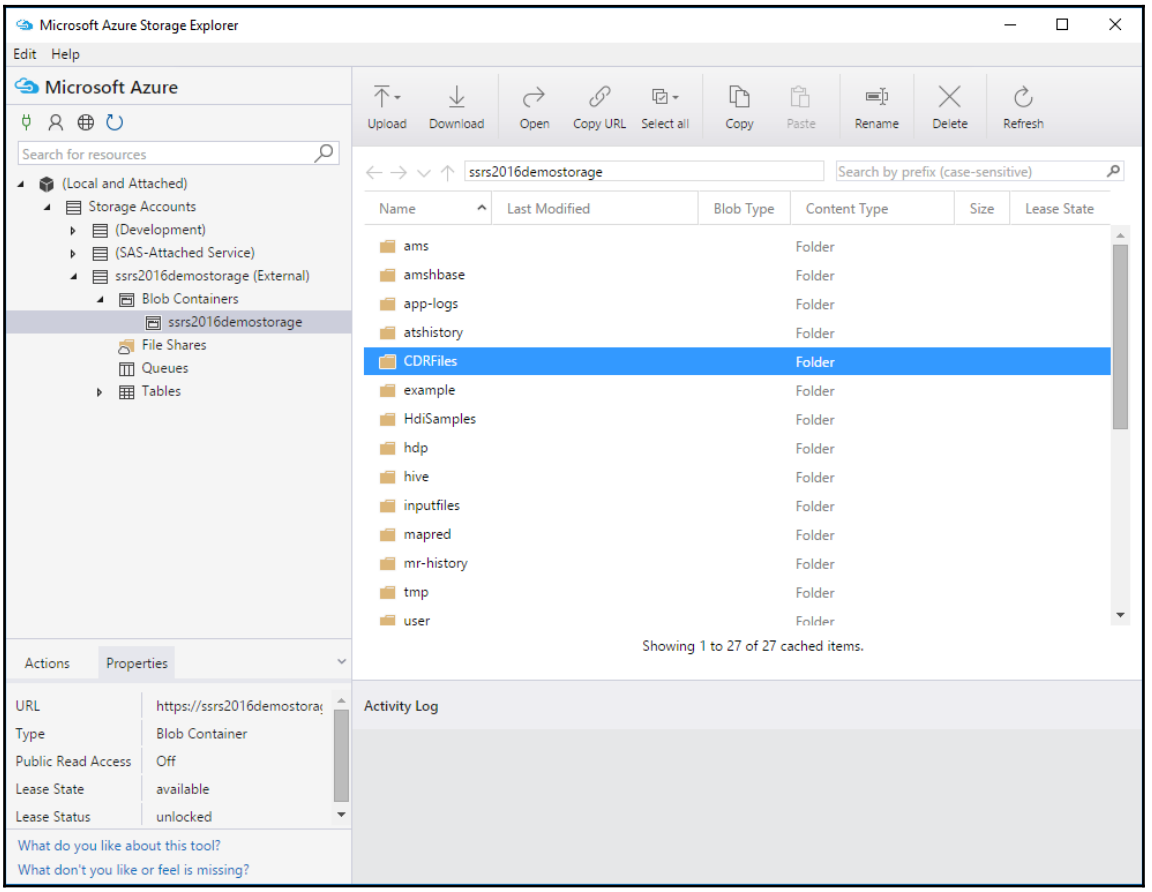


Figure 8.28

Now everything is ready for creating the report. Let's create the report using the `CDRfile` uploaded.

How to do it…

As discussed previously, we need to install Microsoft Hive drivers to connect with HDInsight and execute HiveQL codes. HiveQL is one of the languages used to connect with Hadoop and query data in it. It is similar to SQL, being rich with many useful functionalities.

We will use a HiveQL query to query the CDR file and get data into the report. Let's download, install, and configure the Hive drivers:

1. Go to `https://www.microsoft.com/en-us/download/details.aspx?id=40886`. Download both the 32-bit and 64-bit drivers and install.
2. Open **ODBC Data Sources Administrator (32-bit)** to configure **System DSN** for your HDInsight cluster. Click on **Add** to create a new data source. Select **MicrosoftHive ODBC Driver** and then click **Finish**:

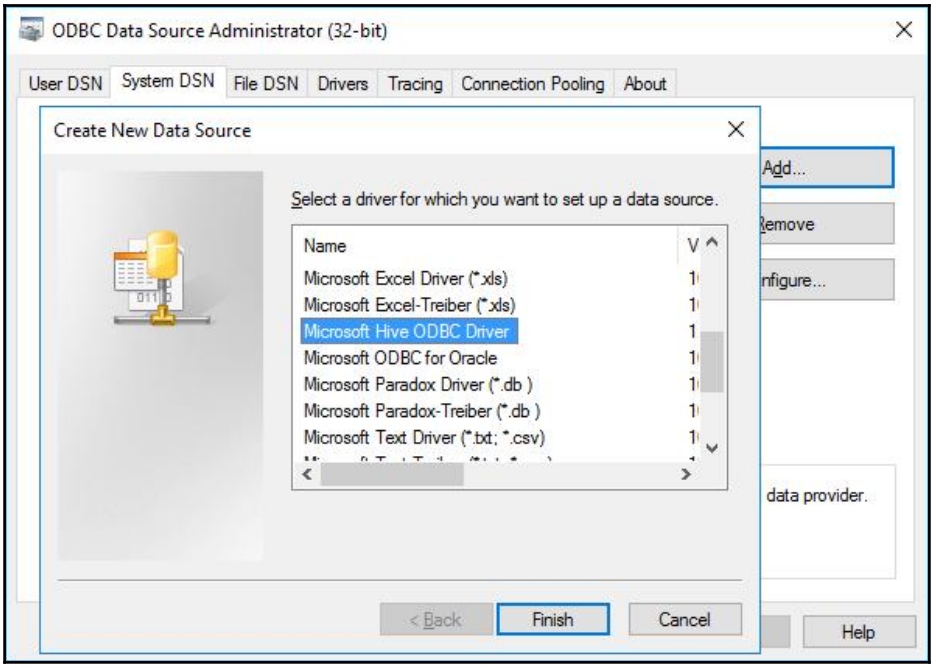


Figure 8.29

3. The **Microsoft Hive ODBC Driver DSN Setup** window will open. Enter the following information:
    - **Data Source Name**: `ssrs2016democluster`
    - **Host**: `<yourclustername>.azurehdinsight.net`
    - **Port**: `443`
    - **Database**: `default`
    - **Hive Server Type**: **Hive Server 2**
    - **Authentication Mechanism**: **Windows Azure HDInsight Service**
    - **User Name**: `admin` (or name you entered if it is not admin)
    - **Password**: the password you set for the account:

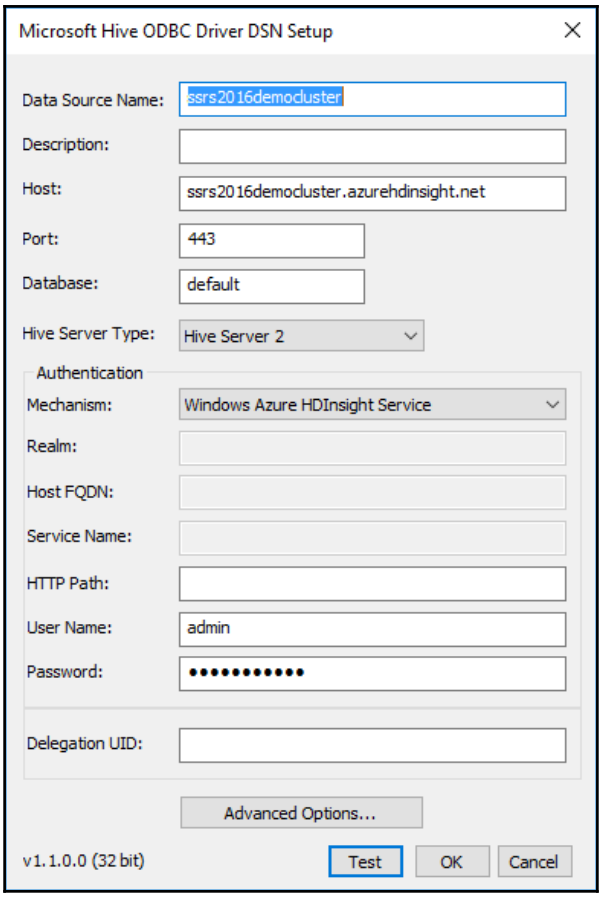


Figure 8.30

4. Click on **Advanced Options…** to open the **Advanced Options** window.
5. Set the default string column length as 4000 and click **OK**.
6. Click on **Test** and see whether it can connect with your HDInsight cluster. If everything is correctly fed, you should see something similar to the following screenshot:

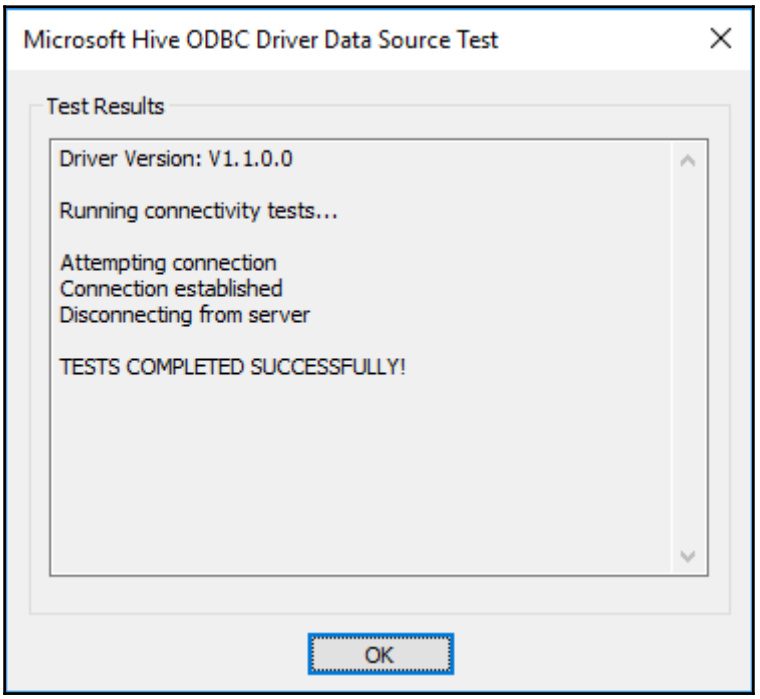


Figure 8.31

7. Click on **OK** to save the System DSN created.
8. Close **ODBC Data Sources Administrator (32-bit)**.
9. Open **ODBC Data Sources Administrator (64-bit)** and repeat steps 2-8.

Now the drivers are ready. Note that you need to have these drivers installed and ready on both the developer machine and Reporting Services server.

Files (or data) in Hadoop can be easily accessed, as if they are tables. For that, we do not need to convert uploaded files into tables, but we can read data in the files using Schema-on-read concepts. This allows you to set a schema on the data as you want to read and query the data using the schema set.

We can create the schema required using HiveQL. This allows us to create an external table linked to the folder and get data by querying the table. Remember, it does not create a physical table; it is just a schema referred to the data in the folder.

There are many ways to create the external table for our file. Let's use the Azure portal:

1. Open the Azure portal and your HDInsight cluster.
2. When you open your cluster blade, you should see a button called **Dashboard**. Click on it to open the dashboard related to your cluster. You must provide your admin account credentials to open it.
3. Click on the button just before the **Admin** button. Click **Hive View**:

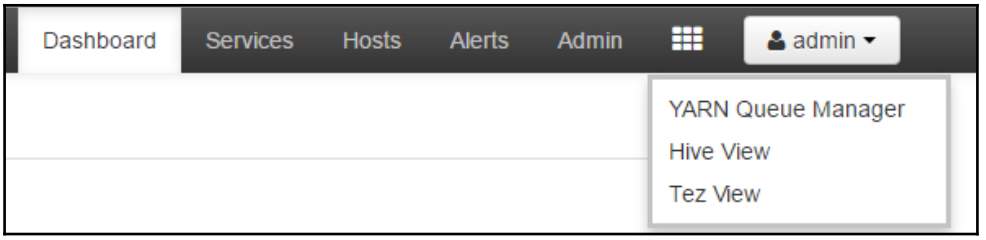


Figure 8.32

4. This opens a page you can use to work with Hive. Use the following HiveQL query to create an external table:

```
DROP TABLE IF EXISTS CDR;
CREATE EXTERNAL TABLE CDR
(
   Id int
 , ANumber string
 , BNumber string
 , StartTime string
 , EndTime string
 , Duration string
 , Cell array<string>
) row format delimited
fields terminated by ','
collection items terminated by '|'
lines terminated by '\n'
stored as textfile
location '/CDRFiles/';
```

5. Look at the structure we place on the file. We create a table with seven columns, and the seventh column is an array. The seventh column is to get all cells related to the call into a single column as an array (see Figure 8.33).
6. Place the code in **Query Editor** and click on **Execute**:

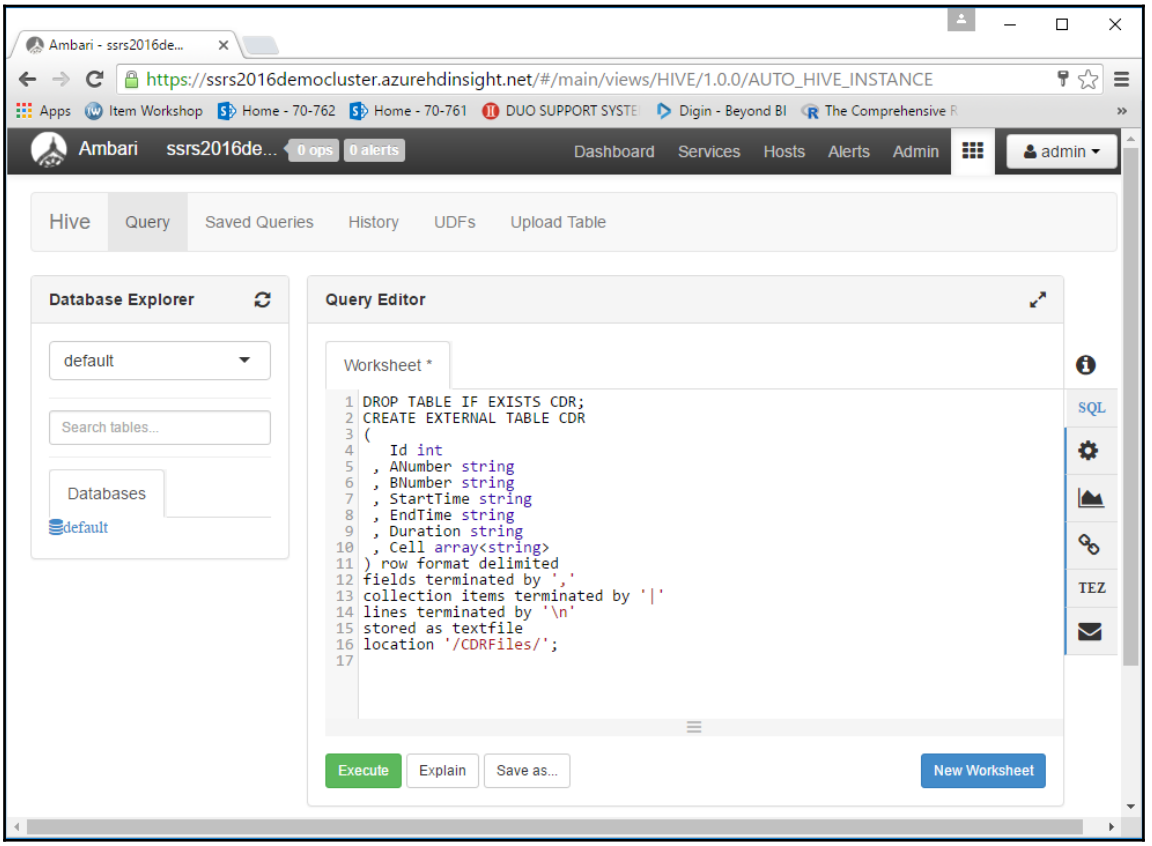


Figure 8.33

We have defined the schema for reading data now and we can create the report showing getting data using the schema created. Follow these steps to create the report:

1. Open your reporting solution and create a new report called `CDRReport`.
2. Add a new data source to the report and name it `DataSourceSSRS2016DemoCluster`. Select the **Type** as **ODBC**. Click on **Edit** to configure:

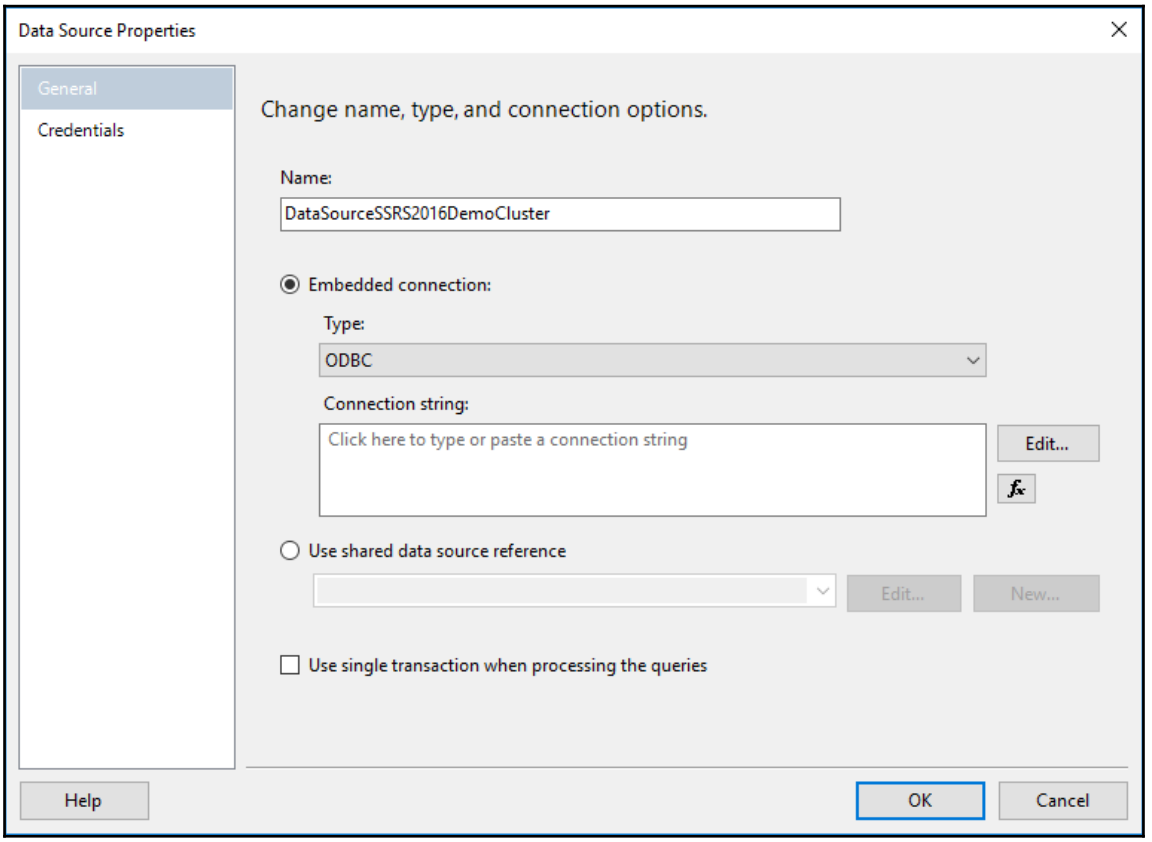


Figure 8.34

3. In the **Connection Properties** window, select **ssrs2016democluster** for **Use user or system data source name** (see Figure 8.35).

4. Enter **User name** as `admin` (this is your cluster username) and the password for your account (see Figure 8.35).
5. Click **Test** Connection to test it:

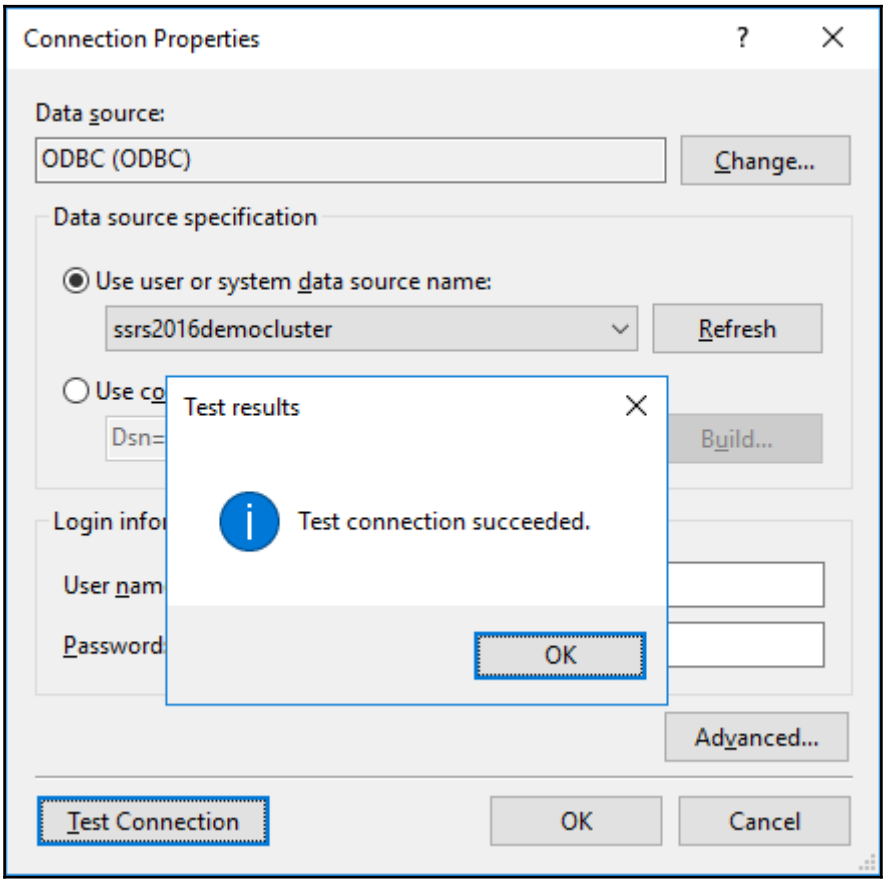


Figure 8.35

6. Click **OK** to save your settings in **Connection Properties**.
7. Click **OK** to save the data source.

Now the data source is ready, let's create the dataset:

1. Create a new dataset and name it `DataSetCDR` (see Figure 8.36).
2. Select the **Use a dataset embedded in my report** option (see Figure 8.36).

3. Select the data source created previously and add the following HiveQL query to the query window (see Figure 8.36):

```
select Id, ANumber, duration, size(Cell) as nofcells, Cell[0] as
startingcell, Cell[size(Cell) - 1] as endingcell from CDR  limit
10;
```

4. Note that the query retrieves ID, Caller number, duration of the call, number of cells used, the first cell used, and the last cell used.
5. Click on **OK** to save the dataset:

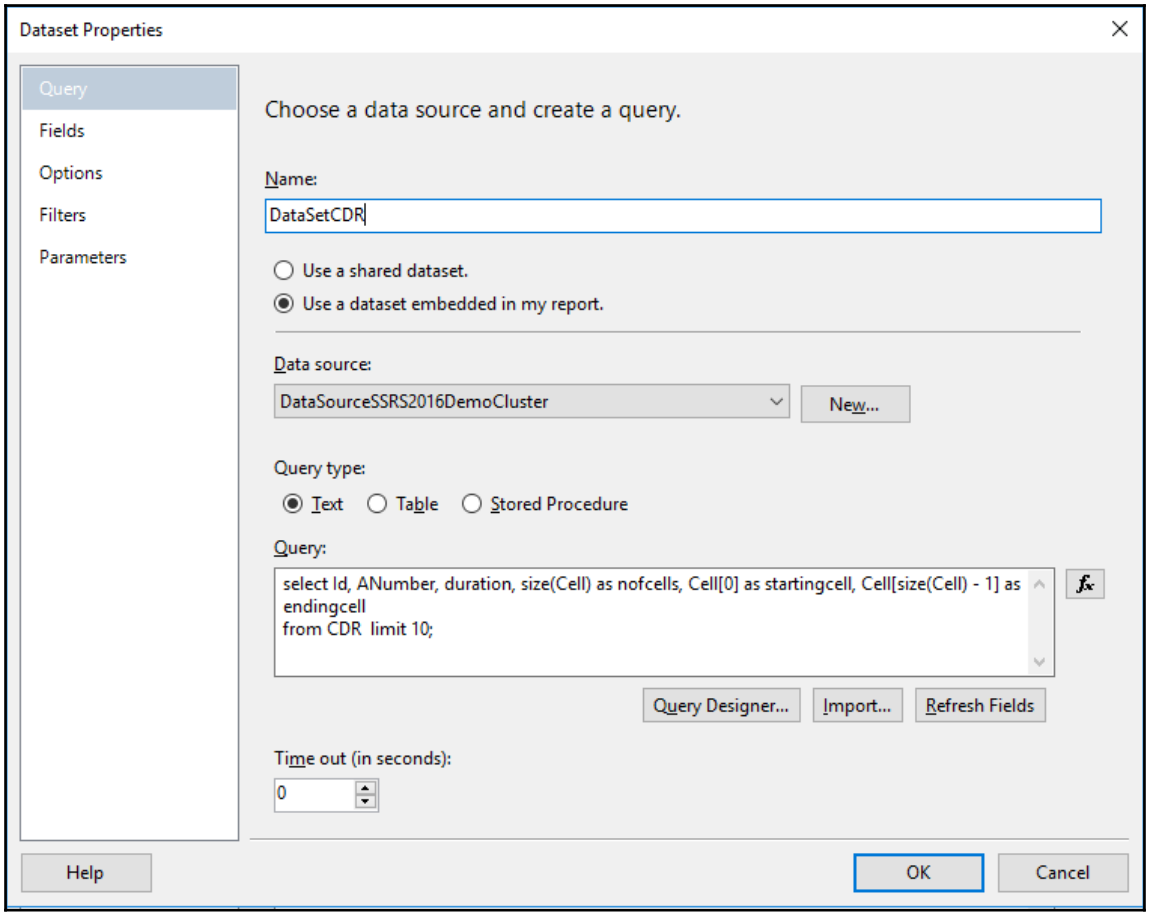
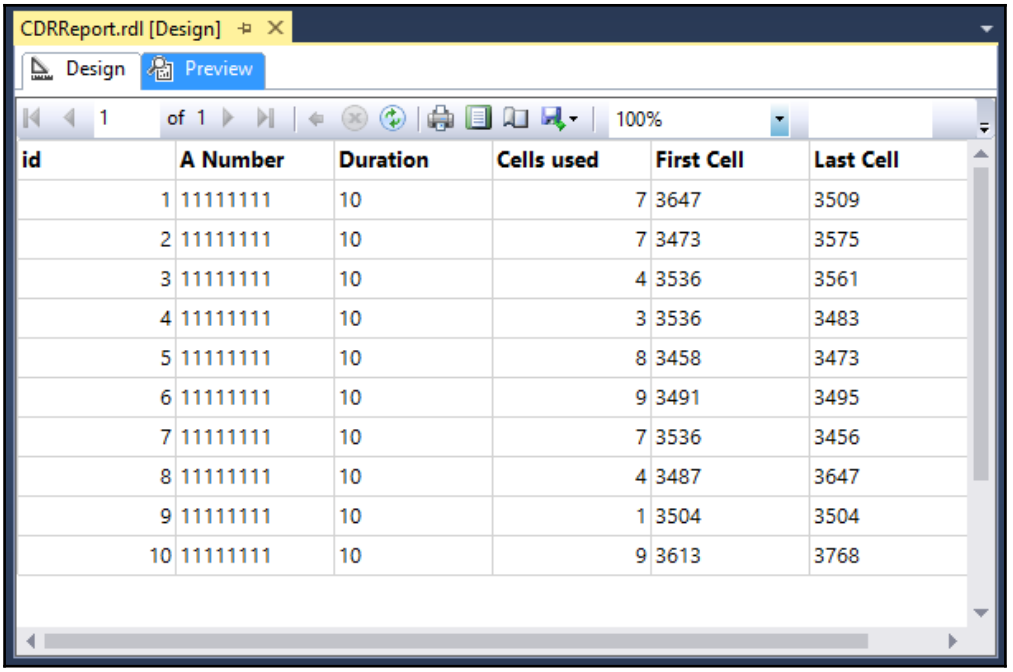


Figure 8.36

6. Add a table data region to the report and add all the fields from the dataset. Format them as required.
7. Run it. You should see a result similar to that shown in the following screenshot:



id	A Number	Duration	Cells used	First Cell	Last Cell
1	11111111	10	7	3647	3509
2	11111111	10	7	3473	3575
3	11111111	10	4	3536	3561
4	11111111	10	3	3536	3483
5	11111111	10	8	3458	3473
6	11111111	10	9	3491	3495
7	11111111	10	7	3536	3456
8	11111111	10	4	3487	3647
9	11111111	10	1	3504	3504
10	11111111	10	9	3613	3768

Figure 8.37

How it works…

Reporting Services retrieves data from the cluster using the installed Hive driver. Depending on the data volume and complexity of the query, it may take a considerable time to retrieve data and render the report, but in most cases, this is acceptable.

Note that we have used a very simple HiveQL query, but you can write a complex query for doing various analyses. Since Hive was created to work with unstructured data, it can efficiently handle the request and get you the results you need.

We installed both 32-bit and 64-bit Hive drivers. The reason for this is that in most cases, the 64-bit driver is used by applications running in 64-bit mode, but a tool like SSDT needs the 32-bit driver as it is a 32-bit application. Therefore, it is always better to install both drivers and create DSNs with the same name.

There's more…

If you do not use a Hadoop cluster but maintain blob storage in Azure for semi-structured data, you can create similar reports using SQL Server Polybase, which was introduced with SQL Server 2016.

See also

You can get more details on Hive, HDInsight, and related information using the following URLs:

- `http://dinesql.blogspot.com/2015/07/how-to-create-hive-table-and-execute.html`
- `http://dinesql.blogspot.com/2015/07/what-is-hive-what-is-hive-database-what.html`
- `http://dinesql.blogspot.com/2016/05/creating-hdinsight-hadoop-cluster-using-ssis-azure-feature-pack.html`
- `http://dinesql.blogspot.com/2016/05/creating-hdinsight-hadoop-cluster-using-ssis-azure-feature-packii.html`

9
SharePoint Integration

In this chapter we will cover the following topics:

- Configuring SharePoint for Reporting Services
- Handling security with SharePoint
- Creating dashboards using Reporting Services
- Configuring e-mail with SharePoint
- Configuring subscriptions
- Configuring data alerts

Introduction

In this chapter, we'll explore the use of SharePoint as a host for your Reporting Services reports. More specifically, we'll see how to set up and configure SharePoint to host and execute these reports.

Be aware that this chapter doesn't guide you through a full SharePoint installation. We assume you already have a SharePoint instance set up and working. Attempting to cover the installation of SharePoint is beyond the scope of this chapter, and indeed could require an entire book of its own.

Configuring SharePoint for Reporting Services

In this opening recipe, we'll see how to configure SharePoint 2016 to host reports. You'll see how to create a Reporting Services SharePoint application, and set it up so when reports are uploaded to SharePoint they can be viewed right within it.

Getting ready

For this recipe, all you'll need is a SharePoint instance you can work with. Be aware you will need administrative access to the SharePoint site. If you have an Azure account, free or paid, you could set up a test instance of SharePoint and use it to follow the instructions in this chapter. Note the setup of such an Azure instance is outside the scope of this chapter. In this chapter, we assume you are using an on premises SharePoint installation.

How to do it…

1. Open the SharePoint 2016 Central Administration web page.

2. Click on **Manage service applications** under the **Application Management** area:

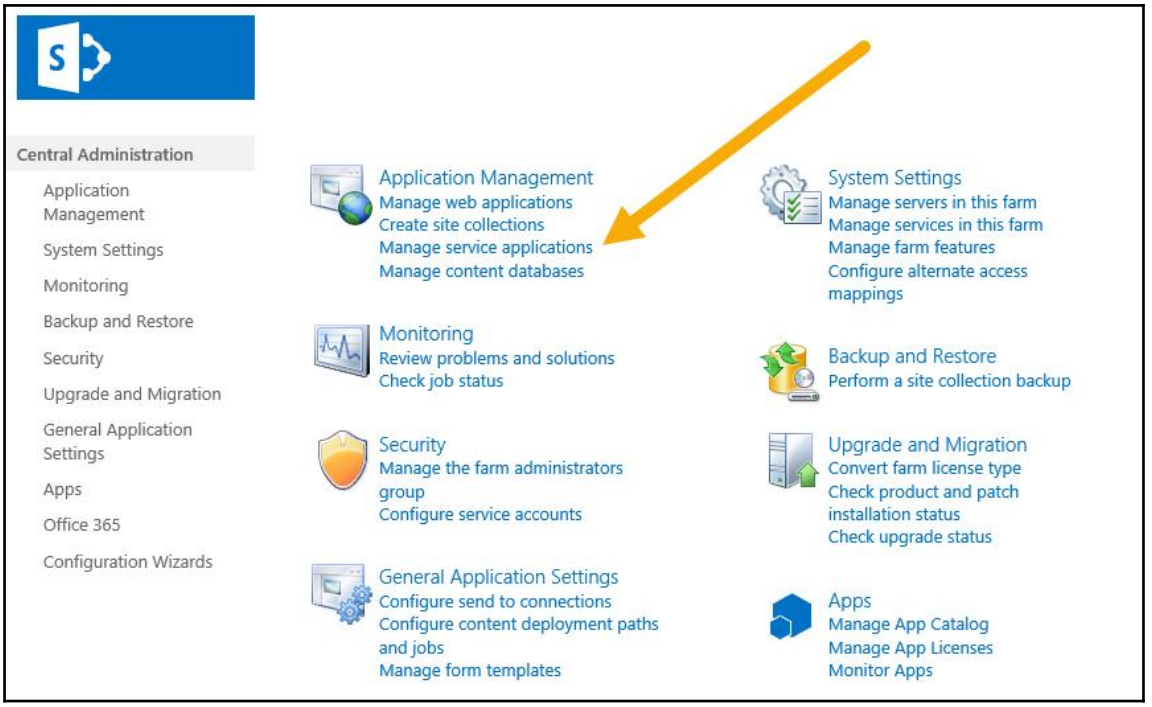


Figure 9.01

3. The **Service Applications** tab now appears at the top of the page. Click on the **New** menu:

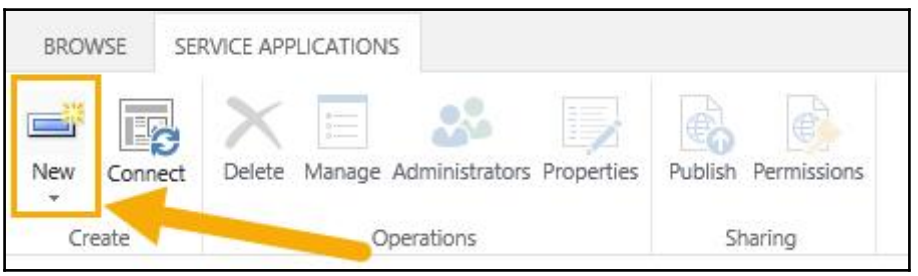


Figure 9.02

4. In the menu, find and click on the option for **SQL Server Reporting Services Service Application**:

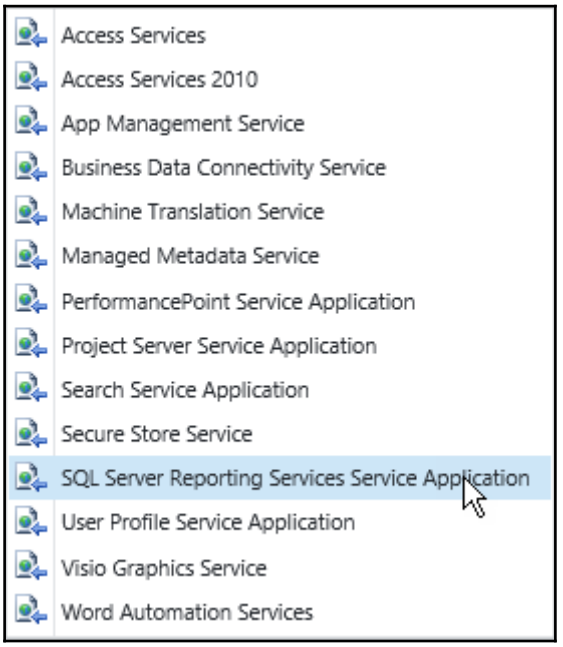


Figure 9.03

5. You'll now need to fill out the information for the service application. Start at the top by giving it a good name, here we are using `SSRS_SharePoint`.
6. Presumably this is a new install, so you'll have to take the **Create new application pool** option. Give it an appropriate name; in this example, we used `SSRS_SharePoint_Pool`.
7. Select a security account to run under. Here we selected an account set up by our Active Directory administrator, which has permissions to SQL Server where SSRS is installed.
8. Enter the name of the server which has SQL Server 2016 Reporting Services installed. In this example, our machine is `ACSrv`.

9. By default, SharePoint will create a name for the database that includes a GUID (a long string of letters and numbers). You should absolutely rename this to eliminate the GUID, but ensure the database name will be unique. In this example, we used `ReportingService_SharePoint`.
10. Review the information so that it resembles the following figure, but don't hit **OK** quite yet as there are few more pieces of information to fill out. Scroll down in the dialog to continue:

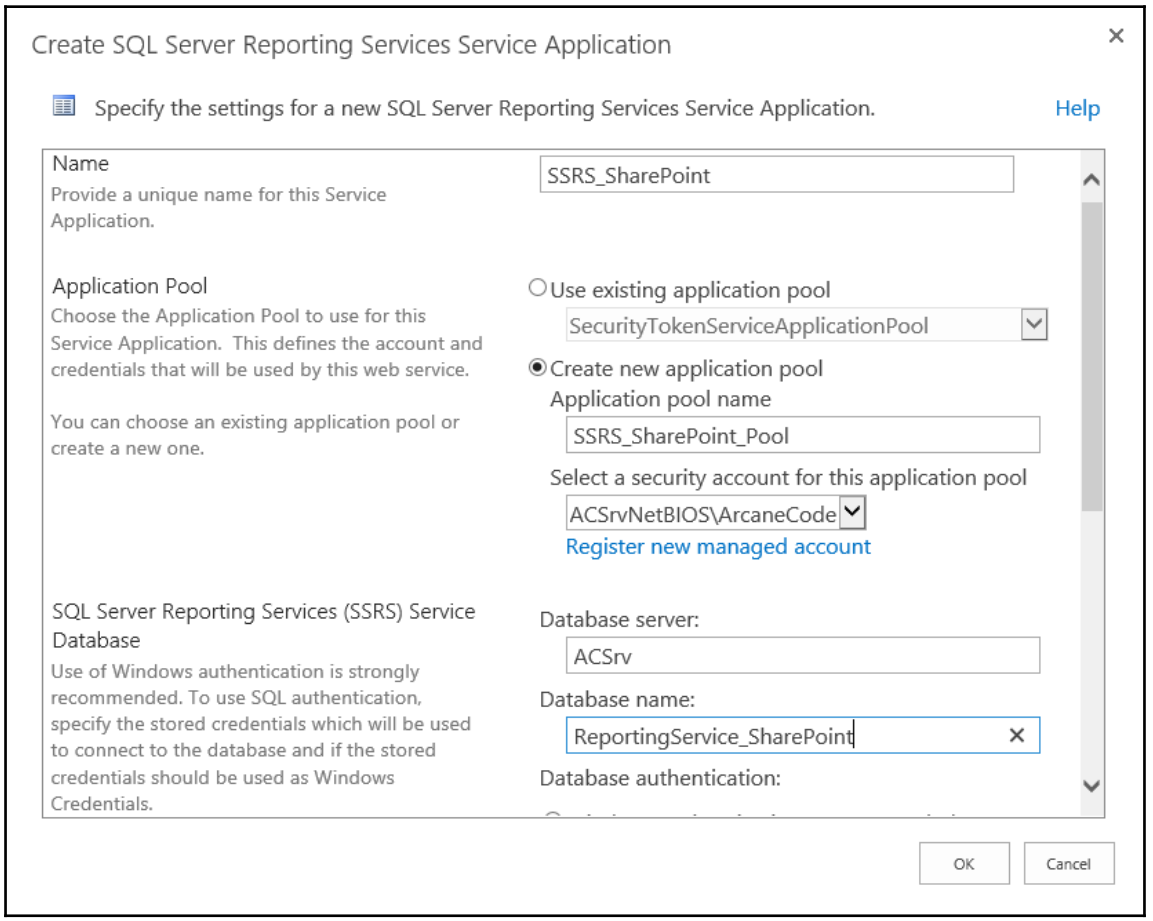


Figure 9.04

11. After the database name, you'll need to indicate the authentication method. Assuming the credentials you entered for the security account came from your Active Directory administrator, you can take the default of Windows authentication.
12. Place a check mark beside the instance of SharePoint to associate this SSRS application with. Here there is only one, SharePoint – 80.
13. Click **OK** to continue. Assuming all goes well, you should see the following confirmation dialog. If so, click **OK** to proceed:

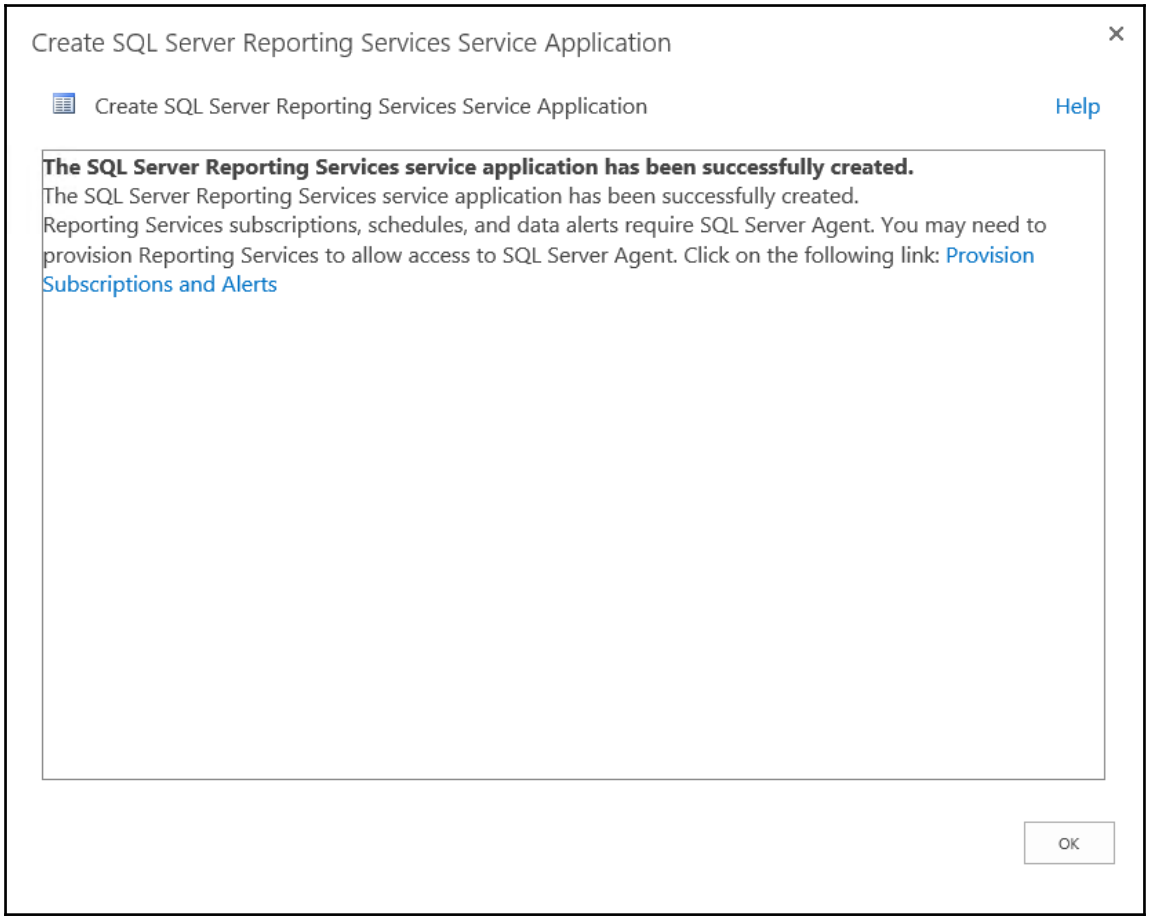


Figure 9.05

14. Now that SharePoint is configured, you'll now need to provide additional information to SQL Server. That is the purpose of this final screen, **Provision Subscriptions and Alerts**. Select the **Download Script** button, and save the generated SQL file:

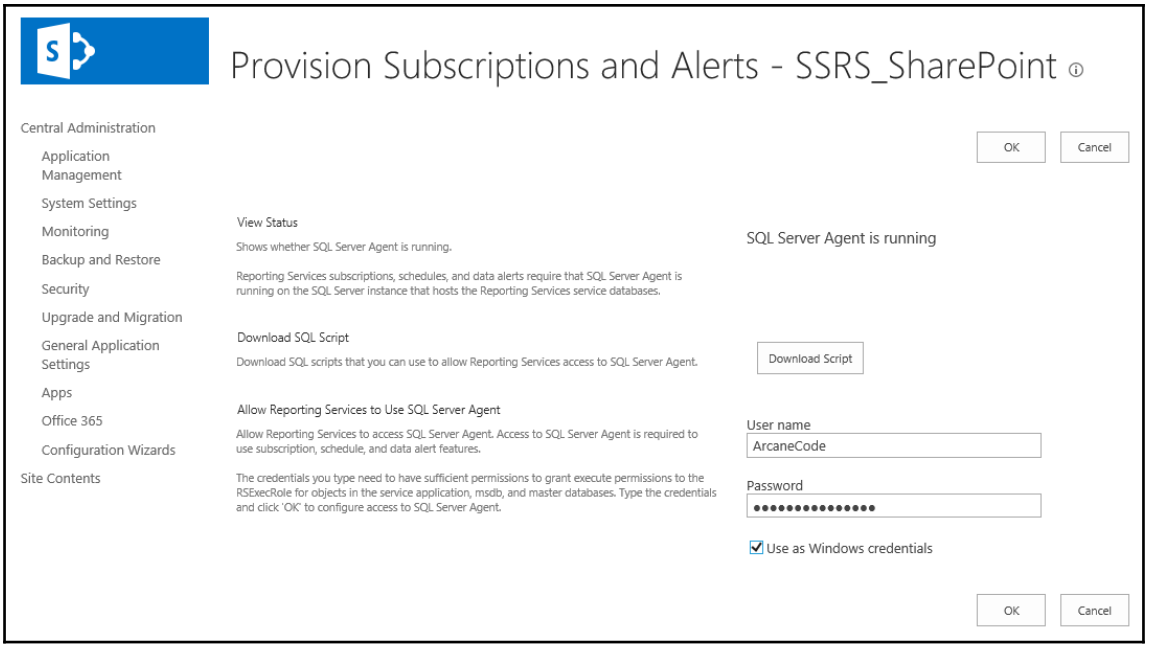


Figure 9.06

15. Pass the SQL file to a database administrator to execute, or open it in SSMS and execute it yourself, assuming you have administrative rights on the SQL Server.

How it works…

SharePoint uses the concept of Service Applications to manage items which run under the hood of SharePoint. SQL Server Reporting Services is one such service application. By integrating it as a service application, end users can upload, modify, and view SSRS reports right within SharePoint.

We began by generating a new Service Application, and picking Reporting Services from the list. We then needed to let SharePoint know where the SQL Server would be used to host both the database, as well as have a copy of Reporting Services for SharePoint installed. In addition, we also needed to provide security credentials for SharePoint to use to communicate with SQL Server.

As the final step, we needed to configure SQL Server to now work with SharePoint. This was the purpose of the **Provision Subscriptions and Alerts** screen. Note there is an option to fill out a user name and credential; clicking **OK** would then have immediately executed scripts against the target SQL Server.

In most mid-to large-size corporations, however, there will be controls in place to prevent this type of thing. Most companies will require a DBA to review scripts, or at the very least you'll want to keep a copy of the script in your source control system to be able to track what changes were made to a SQL Server.

Hence, we suggest taking the action laid out in this recipe, namely downloading the script and executing it manually in the SQL Server Management Studio.

To test your setup, we suggest creating a new report with embedded data sources and datasets. Upload that report to the server, and attempt to execute; it should display correctly if your install went well.

Handling security with SharePoint

Security with SharePoint can occur at multiple levels. There is controlling access to the entire site itself, to individual folders within the site, or to individual report items. At all three levels, security is performed similarly, you'll select the Share option, then indicate the Active Directory user or group to share with.

Most of the time, you'll be sharing with Active Directory groups, but for simplicity in this chapter's examples we'll use a user name. To keep things simple just remember when we say user in this section, we mean user or group. The technique is identical for both.

With security there are multiple actions to perform, such as adding, altering, and removing permissions for a user. Hence we've broken this up into mini-recipes for each action to make it easier to find what you need to do.

Getting ready

It is assumed you have administrative permissions on the SharePoint server, and are using Active Directory authentication for access to SharePoint. In addition, the user must also have AD permissions to access the database the report is being generated from. Finally, you should also have a delegation method such as Kerberos configured to allow credentials to be passed from SharePoint to the SQL Server hosting your source database.

How to do it…

There are multiple steps, in multiple locations, required to add a user to your SharePoint site. To make it easy to follow we've broken these into mini-recipes, each one detailing the steps involved for a particular task.

Adding a user to your SharePoint site

In this first mini-recipe, we'll select a user from the Active Directory and give them permissions to use our new SharePoint site:

1. From your site's home page, click **Share your site**:

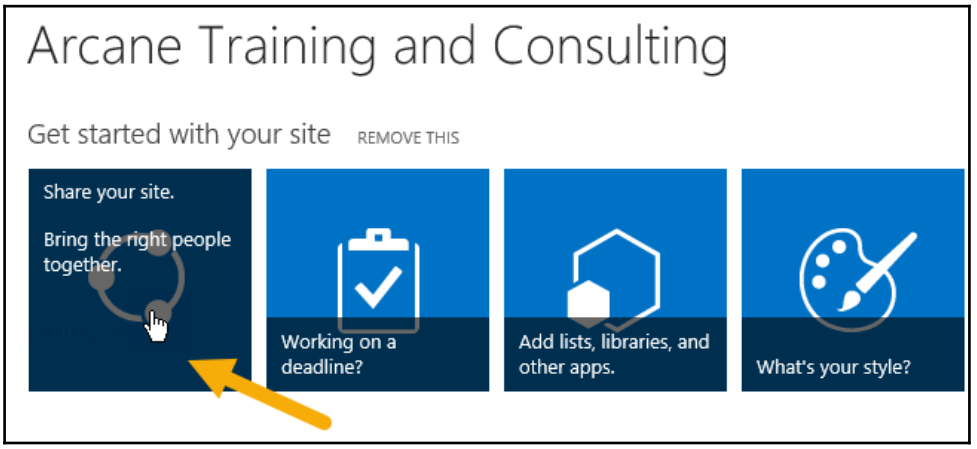


Figure 9.07

2. In the **Invite people** area, key in the Active Directory user ID for the person to add. As you type, a list will appear with the full names that match that ID. Select the name of the person to add.

3. Click **Show Options**.
4. Under **Select a group or permission level**, select the one that has your site name with **Visitors [Read]** at the end.
5. Click **Share**:

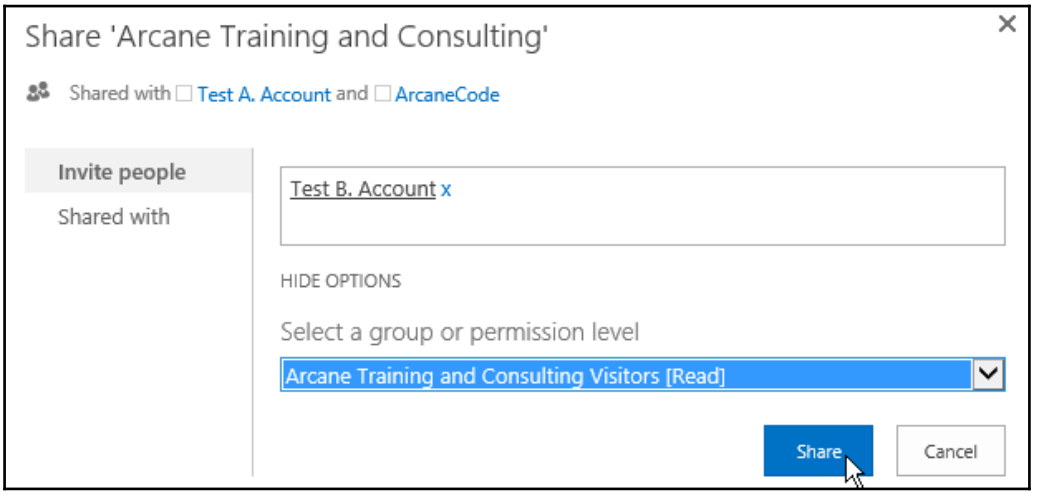


Figure 9.08

Altering permissions to a document library

Now that we have a new user, we need to grant that user permission to see one of our document libraries. We'll do that in this section:

1. Navigate to the SharePoint library you want to update permissions for.
2. In the tabs at the very top, click **Library**.
3. On the right, in the **Settings** area, click **Shared With**:



Figure 9.09

4. Click on **Advanced**:

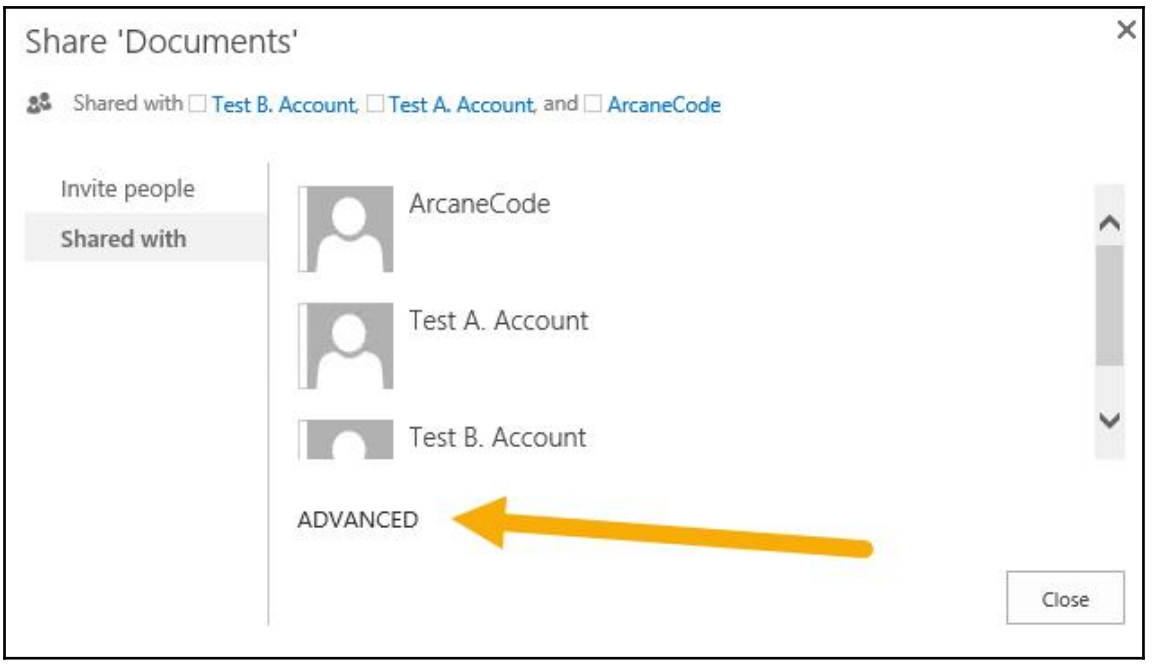


Figure 9.10

5. Click the **Permissions** tab at the top.
6. On the Permissions tab, select the **Stop Inheriting Permissions** button.
7. A window will appear asking you to confirm; just click on **OK** to accept the change.
8. In the updated Permissions tab at the top, click **Grant Permissions**.
9. In the **Invite people** area, begin by keying in the Active Directory ID of the person or group to update. When the name appears in the dropdown click on it.

10. Click on **Show Options**. In the options, select **Edit**, then click **Share**, as shown in the following screen.

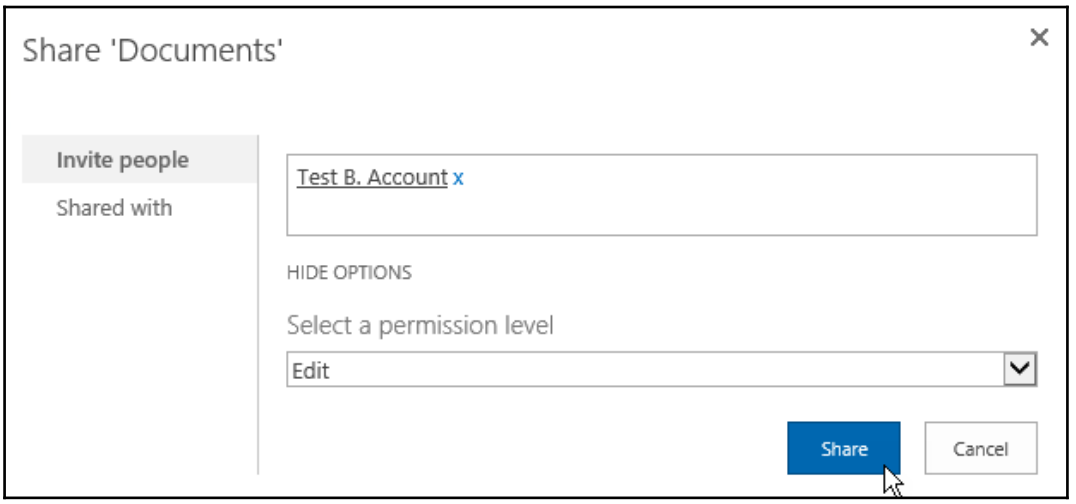


Figure 9.11

Altering permissions to a specific report item

Sometimes you will want to grant a user one level of access to a document library, but may want to grant different permissions for a specific item. For example, you may want a user to have read only access to the folder, but for one item want the user to have full access. In this mini-recipe we'll see how to accomplish this:

1. Navigate to the folder containing the item you wish to update permissions on.
2. Click on the ellipses (**...**) to the right of the item:

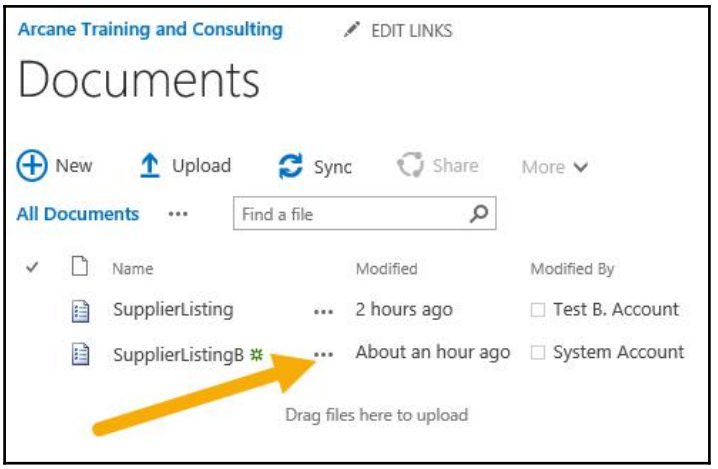


Figure 9.12

3. In the popup window, click on **Share**.
4. Click on **Shared with**, then click on **Advanced**.
5. In the **Permissions** tab, click on **Grant Permissions**.
6. In the **Invite people** area, begin entering the user ID of the individual or group to alter permissions on. Select the name from the dropdown when it appears.
7. Click on **Show Options**.
8. Under **Select a permission level**, select **Full Control**:

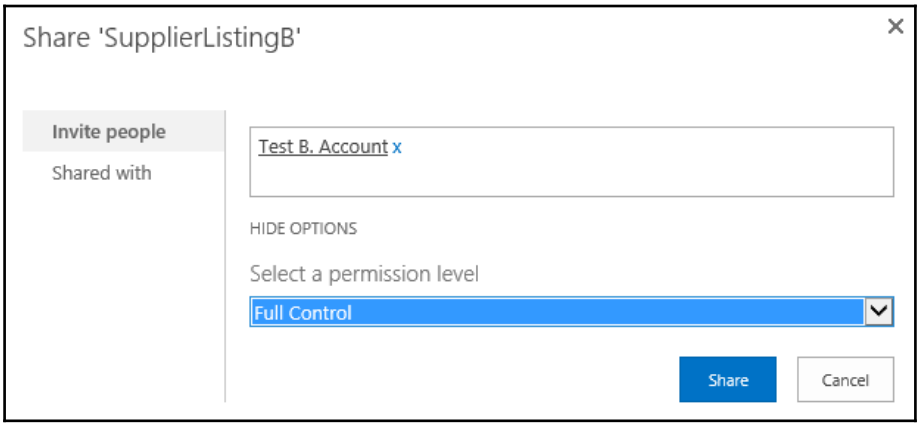


Figure 9.13

9. Click **Share**.

Removing a user's permissions from a report

In the previous mini-recipe we saw how to grant extra permissions to an item. In this one, we'll see how to remove a users permissions all together for an item in the document library:

1. Navigate to the folder containing the item you wish to update permissions on.
2. Click on the ellipses (**...**) to the right of the item.
3. In the popup window, click on **Share**.
4. Click on **Shared with**, then click on **Advanced**.
5. To remove permissions for an entire group or specific user:
    1. Place a check mark in the box beside their name.
    2. In the **Permissions** toolbar, click **Remove User Permissions**.
6. To remove permissions for a user that is in a group:
    1. Click on the name of the group.
    2. Place a check mark in the box beside their name.
    3. In the **Actions** menu above the list of names, select **Remove User from Group**:

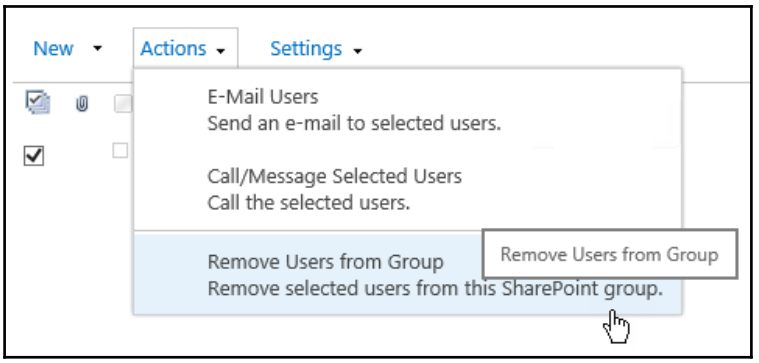


Figure 9.14

How it works…

In this section, we covered four basic tasks. To begin with, the user (or group) had to be added to the SharePoint site. That was the purpose of the first set of actions, *Adding a user to your SharePoint site*.

In that step, we selected the lowest level of permissions, View. This gives a user permission to see what is on the site, and execute reports, but not the ability to edit them, remove them, or upload new ones.

If we had used Members, the user or group would have been given the ability to not only view reports but edit them, add new ones, or delete them.

Owners gives permission to the entire site, up to and including the ability to delete the entire site. Obviously, this should be used sparingly and only granted to system administrators.

What if we want a user to have more access to a folder, let them have the rights to not only run, but create, update, and delete reports? That's where the second set of tasks, *Altering permissions to a document library*, came in. Through this set of tasks, we can upgrade a user's permissions to a specific document library, without giving them these permissions to other libraries.

You may have wondered about the step where we clicked stop inheriting permissions. By default, whatever permissions occur at the top level permeate downward to all folders under it. Thus, when we added the user to the view group in the previous set of tasks they are given view rights to the folder we are working with.

In order to give a user a different set of permissions, we had to tell SharePoint we are going to set up a new set of permissions don't automatically use the ones from the parent site. When you click stop inheriting permissions, SharePoint copies the parent permissions into those of the folder you are working with. From there on, it will use the set of permissions for the folder instead of those of the parent.

When we were assigning the new permissions, there was a section for options. Under the options we had several choices. Of these, three are really relevant to Reporting Services. Full Control gives the ability to create, update, and delete not just any report within the library, but the entire library itself. Edit grants the ability to create, update, and delete items within the library, such as reports and shared data sources, but the user cannot alter the library itself. Finally, Read allows a user to execute a report, but will not let them alter it, delete it, nor add new ones.

In the third set of actions, *Altering permissions to a specific report item*, we looked at how to alter the permissions of just a single report within the library. This would be useful in situations where you wish to upgrade a user's permissions to update just a single report within the existing library, but not allow them to affect the other reports.

Finally, there may be times when you do not wish a user to even know a report exists. That was the goal of our final task, *Removing a user's permissions from a report*. By performing these actions, not only would a user not be able to do any work with a report, they wouldn't even know it exists. SharePoint would hide it from the user's view when they navigate to this library, while leaving it available for other users.

There's more…

Be judicious with your use of permissions within SharePoint. By default, it is generally best to give the majority of your users View permissions to the entire site. Then create a document library for each set of users, for example Accounting, Human Resources, and so on. Grant Edit permissions to the corresponding active directory group for that library.

Following this pattern will greatly reduce your maintenance overhead.

Creating dashboards using Reporting Services

Dashboards are becoming commonplace in both large and small business. It's possible to create dashboards in SharePoint so that we can use Reporting Services reports for its information.

It does so through the use of *web parts*. Web parts are self-contained pieces of functionality that you can place onto a SharePoint page. After configuring Reporting Services into SharePoint a new web part will become available, one for SQL Server Reporting Services.

Using these web parts we'll create a new page and add three reports to it in order to form a dashboard.

Getting ready

There's quite a bit of setup work to do for this recipe. We'll need to create three reports that will be contained on our dashboard. For two of the reports we will recycle reports we created earlier in the book.

For the first, open Report Builder and follow the basic instructions found in `Chapter 2`, *Authoring Reports with SQL Server Data Tools*. In this version, simply embed the data source. Use the properties window to make the chart 4 inches wide by 2.5 inches tall. Omit the footer, and resize the report body to the same size as the chart. Save the report to SharePoint. For this example, we named the report `Report 09 - Sales by Invoice Date Line Chart`.

The second report will be a clone of the map report found in `Chapter 3`, *Advanced Report Authoring with SQL Server Data Tools*. As with the previous step, simply embed the data source. Make the map 6 inches wide by 4 inches tall, and resize the body to the same size. Omit the footer, as well as any titles. In this recipe the report was saved to SharePoint as `Report 09 - Sales by State Map`.

The final report is a new one. Again using Report Builder, use the following query to obtain the data and place it into a pie chart. For the size, use 3.5 inches wide by 2.5 inches tall. Omit any headers or footers, and as in the previous sections resize the report body to match that of the chart:

```
SELECT Year([Invoice Date Key]) AS InvoiceDateYear
     , SUM([Total Excluding Tax]) AS GrandTotalExcludingTax
  FROM [WideWorldImportersDW].[Fact].[Sale]
 GROUP BY Year([Invoice Date Key])
```

We saved this report to our SharePoint folder as `Report 09 - Sales by Year Pie Chart`. If you get stuck, the report is included in the downloadable content for this book. If you need help with Report Builder, we would refer you back to `Chapter 4`, *Authoring Reports with Report Builder*.

How to do it…

1. From the home page of your SharePoint site, click on the gear icon in the upper right corner of the page.
2. In the menu, select **Add a page**.
3. When prompted, enter `Sales Dashboard` for the new page name.

4. Under the **Format Text** toolbar, select the **Text Layout** menu, then click on **Two columns with header and footer**:

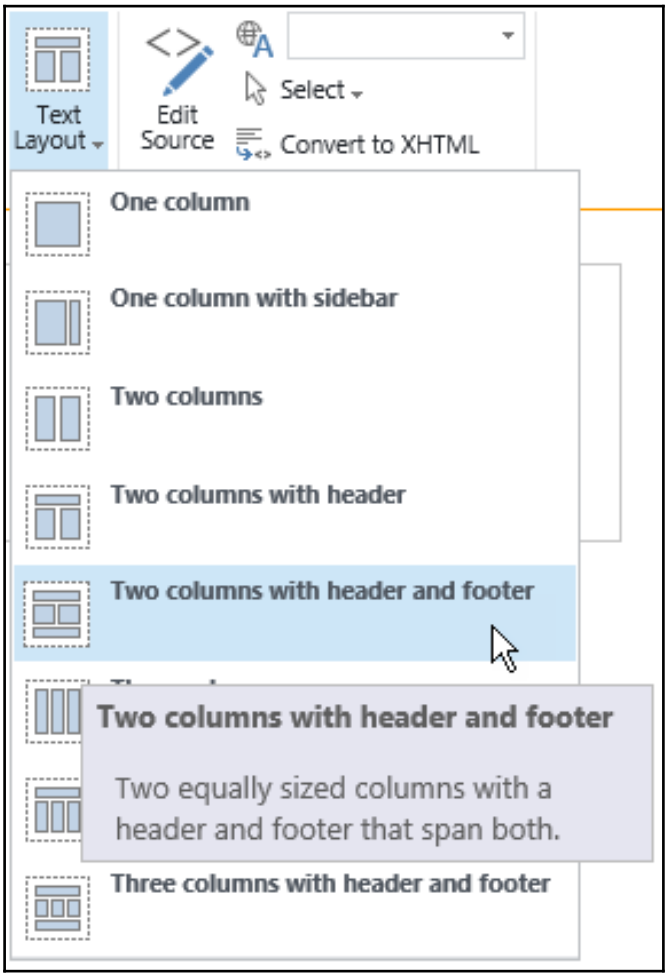


Figure 9.15

5. Click in the block at the top, click on **Heading 1** under styles, and then enter `Current Sales Data`.
6. In the middle are two blocks; click inside the left one.
7. Click on the **Insert** tab at the top and then on **Web Part**:

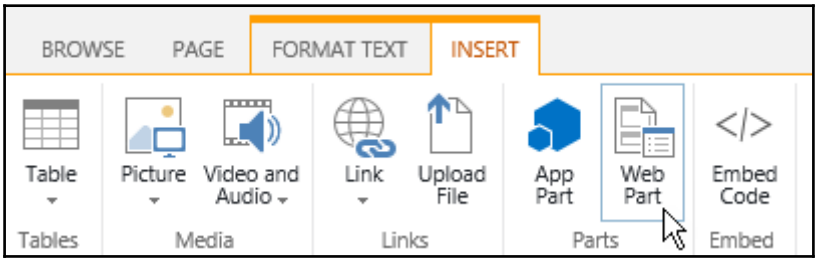


Figure 9.16

8. When the Web Part selection area appears, scroll down on the left to **SQL Server Reporting** and click on it.
9. Click **Add**:



Figure 9.17

10. The report part is now embedded in the middle left pane. A link appears, **Click here to open the tool pane**. Click it:

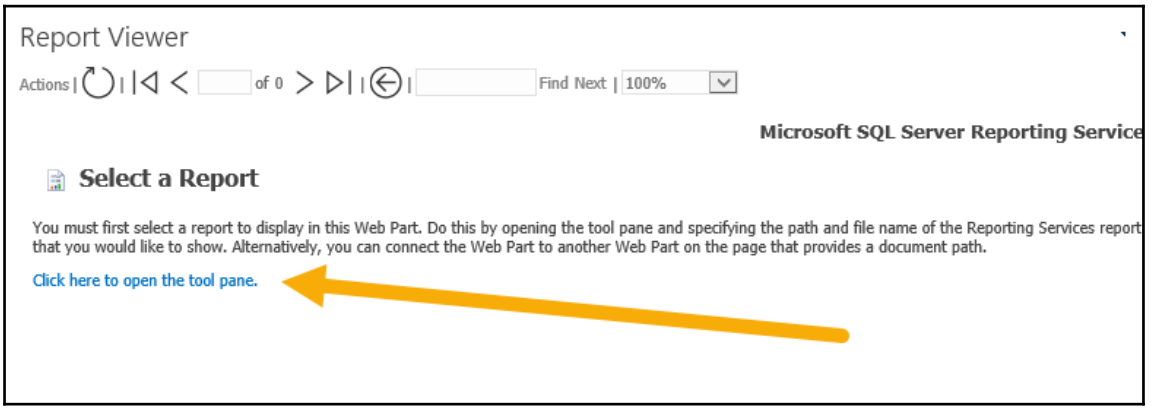


Figure 9.18

11. When the tool pane appears on the right, first select the report using the ellipse button to the right of the Report box. Navigate to the SharePoint folder with your reports and select the `Report 09 - Sales by Invoice Date report` you created in the *Getting ready* section of this recipe:

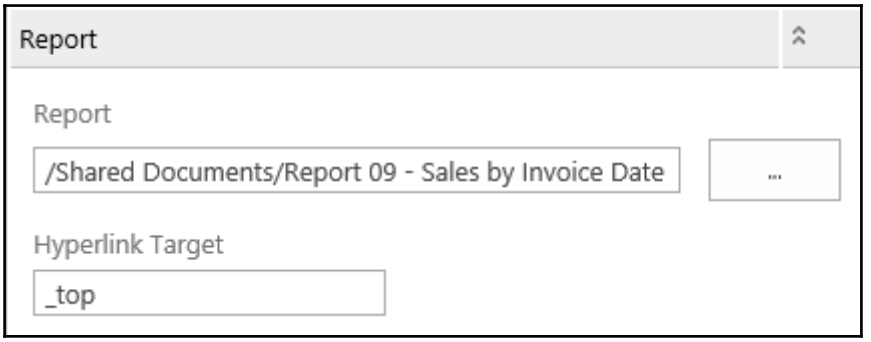


Figure 9.19

12. Expand the **View** area by clicking the double down arrows to the right of the name.
13. Uncheck the two **Auto-Generate** options at the top.
14. Uncheck all of the items under **ToolBar Items Visibility**.

15. Change the **Prompt Area** to **Hidden**.
16. Change the **Document Map** to **Hidden**:

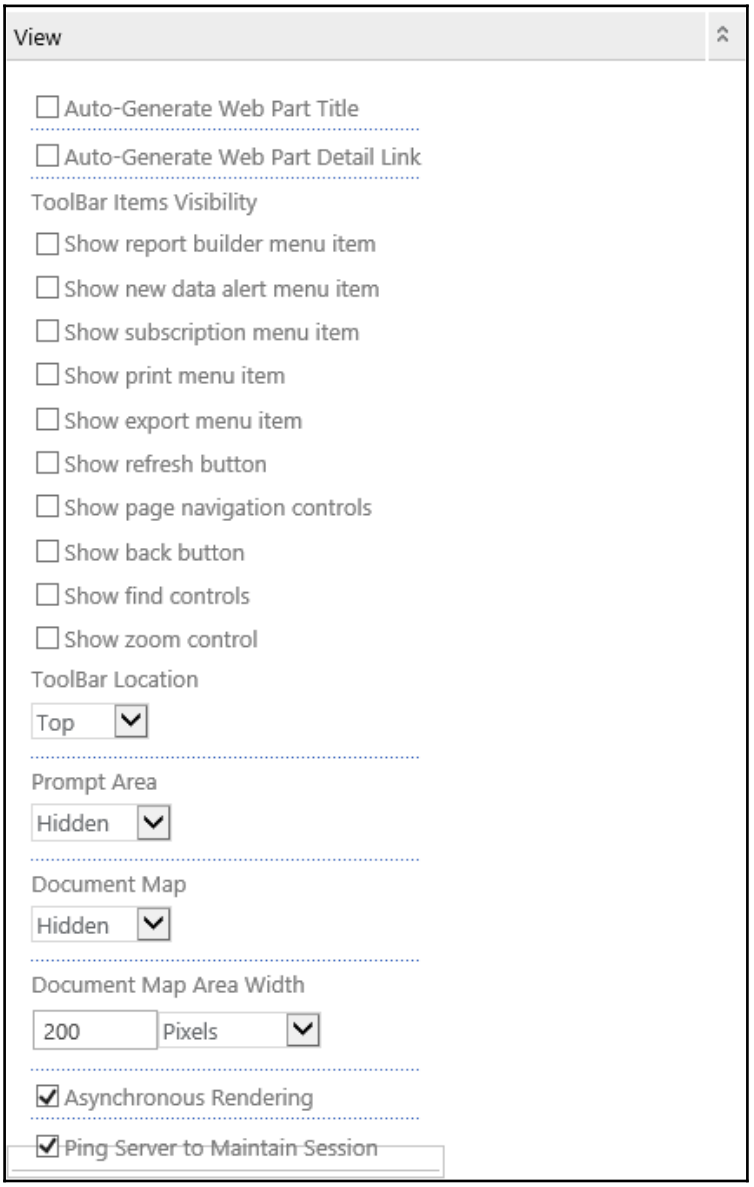


Figure 9.20

17. Expand the **Appearance** area.
18. For the title, enter `Yearly Sales Trend`.
19. Under **Height**, change the value to `250`:

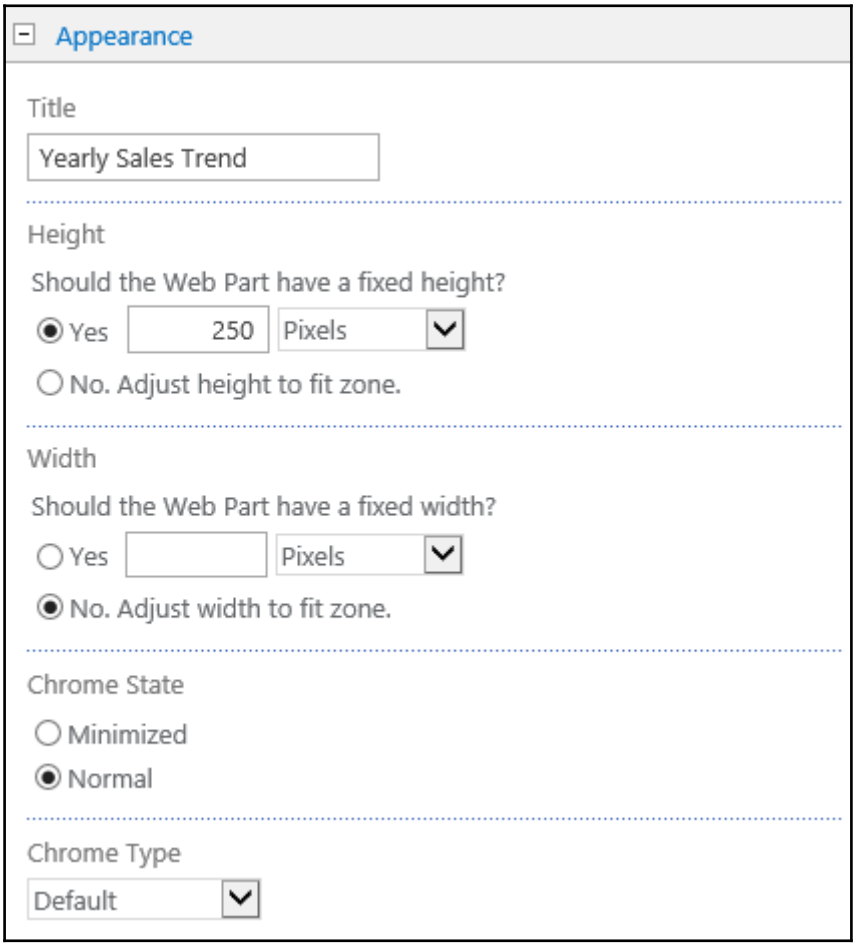


Figure 9.21

20. Click **OK** at the bottom of the tool pane to finish adding this web part.
21. Click in the box on the middle right.
22. Click on the **Insert** tab at the top.

23. Click on **Web Part**:
24. When the Web Part selection area appears, scroll down on the left to **SQL Server Reporting** and click on it.
25. Click **Add**.
26. Click on the **Click here to open the tool pane** link.
27. For the report, use the ellipse, to open the report selection window. Navigate to and select the `Report 09 - Sales by Year Pie Chart`.
28. Expand the **View** area by clicking the double down arrows to the right of the name.
29. Uncheck the two **Auto-Generate** options at the top.
30. Uncheck all of the items under **ToolBar Items Visibility**.
31. Change the **Prompt Area** to **Hidden**.
32. Change the **Document Map** to **Hidden**.
33. Expand the **Appearance** area.
34. For the title, enter `Yearly Sales Comparison`.
35. Under **Height**, change the value to `250`.
36. Click **OK** at the bottom of the tool pane to complete this web part.
37. Click in the box at the bottom of the page.
38. Click on the **Insert** tab at the top.
39. Click on **Web Part**.
40. When the Web Part selection area appears, scroll down on the left to **SQL Server Reporting** and click on it.
41. Click **Add**.
42. Click on the **Click here to open the tool pane** link.
43. For the report, use the ellipse to open the report selection window. Navigate to and select the **Report 09 – Sales by State Map**.
44. Expand the **View** area by clicking the double down arrows to the right of the name.
45. Uncheck the two **Auto-Generate** options at the top.
46. Uncheck all of the items under **ToolBar Items Visibility**.
47. Change the **Prompt Area** to **Hidden**.
48. Change the **Document Map** to **Hidden**.
49. Expand the **Appearance** area.
50. For the title, enter `US Sales`.

51. Under **Height**, change the value to `600`.
52. Click **OK** at the bottom of the tool pane.
53. In the upper left corner of the page editor, click **Save** in the **Format Text** toolbar.
54. You should now see your saved page. As a final step, add it to the quick links at the left to make it easy to find. Click on the **Site Contents** link on the very left:

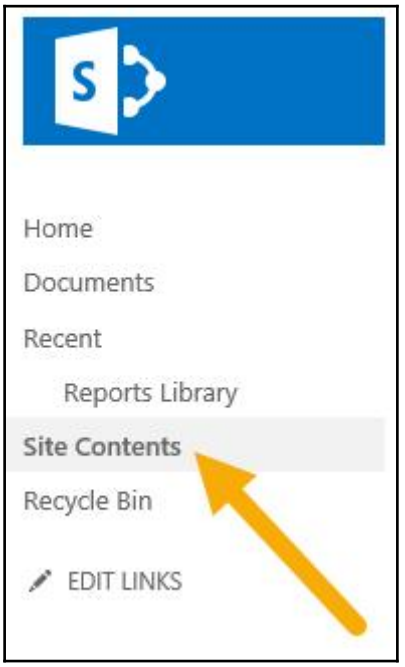


Figure 9.22

55. Click on the **Site Pages**.
56. At the bottom of the menu, as seen in the preceding figure, is **EDIT LINKS**. Click on it.
57. Drag and drop the **Sales Dashboard** page into the links area.
58. Click **Save**.

59. Click on the new **Sales Dashboard** link to see your finished dashboard:

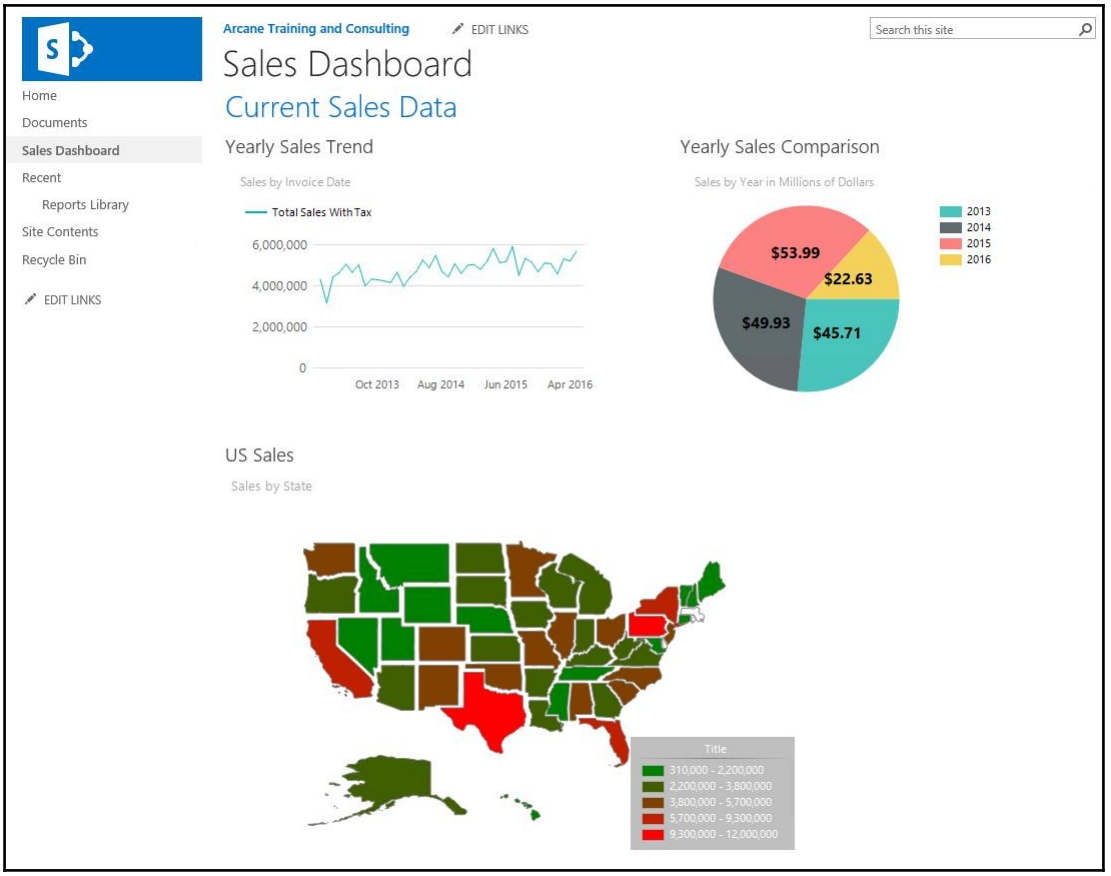


Figure 9.23

How it works…

SharePoint uses the concept of web parts to host other applications within itself. Reporting Services is one such application. Using these SSRS web parts, it is possible to embed one or more reports within a SharePoint page.

For this recipe,we created a new SharePoint page, and selected a layout that would contain our three reports plus a section at the top for our dashboard title. Note that there are many combinations included in SharePoint, in addition you can customize your own page layouts.

Each section of the page is displayed as a block, into which we can add our SSRS web parts. In each block, we inserted a new reporting services part, then brought up a tool pane to customize it.

In the tool pane, we of course had to indicate which report to display. We then went into the view area and hid and unchecked most of the options. You probably wondered why.

By default, when SharePoint displays a report it includes the native SSRS toolbar at the top. Much of the time this would be acceptable, as if we were running a paginated report and needed to be able to scroll through data.

In this example, however,we are creating a dashboard. Dashboards should be a quick and easy way to disseminate data at a glance. Thus, including the toolbars for each web part would be a lot of visual clutter that does nothing to aid your users. It's great then that SharePoint gives us the ability to suppress these, as we did in this recipe.

As a final step, we added the page as a quick link to the links area on the left. Of course this step is entirely optional, but it is very common within organizations to include dashboards in the quick links area.

Configuring e-mail with SharePoint

Reporting Services allows users to subscribe to reports, and have those reports e-mailed on a schedule. With SharePoint you can also create subscription based reports and have those e-mailed out. It does take a bit of set up however.

First, we'll need to configure the e-mail account for the Reporting Services application within SharePoint. Next, we'll have to provision SQL Server to allow SharePoint to manage these e-mail requests. This will put us in good stead for the next recipe, in which we will use this email address to manage subscriptions.

Getting ready

I'm sure it come as no surprise that you will need to have administrative rights to both the SharePoint and SQL Servers you are using. In addition, SQL Agent must be running on the SQL Server that is hosting Reporting Services.

How to do it…

1. In the SharePoint Central Administration page, click on **Application Management**.
2. Under the Service Applications section, click **Manage service applications**.
3. Scroll down to find the name of the service application for **SQLServer Reporting Services Service Application**. In figure 9.25, it is the top option:



Figure 9.24

4. On the page that appears, click **E-mail Settings**.
5. Place a check mark beside **Use SMTP server**.
6. Enter the name of your SMTP server.
7. Enter the e-mail address to use:

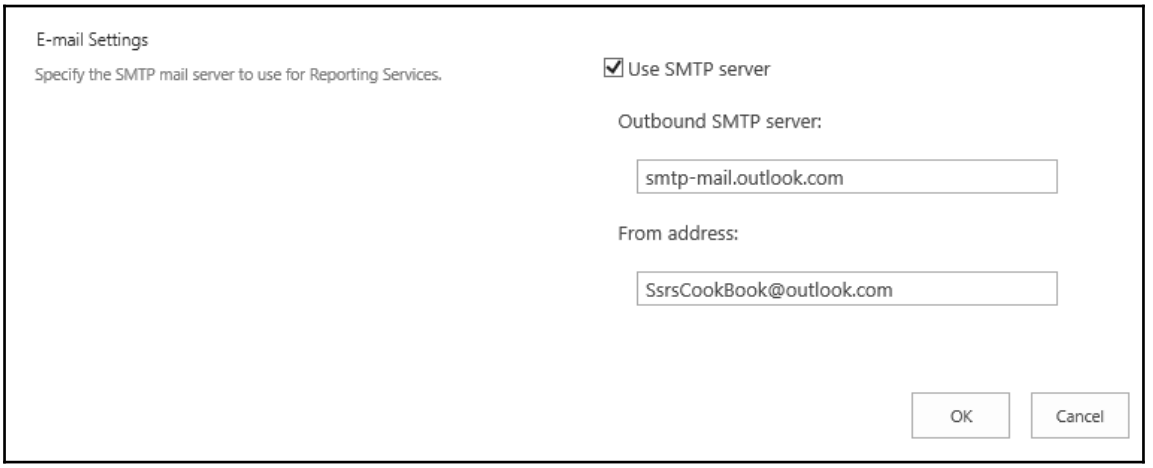


Figure 9.25

8. Click **OK**.

9. When the update process completes, you should be returned to the Manage Reporting Services Application page. Under the **E-mail Settings** is another link, **Provision Subscriptions and Alerts**. Click it.
10. First, at the top you should see a message that SQL Server Agent is running. If you don't see this, start SQL Server Agent and then refresh the page. Don't proceed until the agent is running.
11. Click the **Download Script** button.
12. Save the SQL file.
13. If necessary, pass it to your DBA, or if you have rights simply open it in SQL Server Management Studio and execute it.
14. You should see Command(s) completed successfully, which indicates your SharePoint and SQL Servers are now configured for e-mail.

How it works…

In order to have reports e-mailed, SharePoint first has to know the e-mail address and e-mail server it should use to send e-mail *from*. Once that is complete, it was then necessary to provision the SQL Server with the e-mail account information. As we did in an earlier chapter, we took the Download Script option in order to keep things simple and be able to preserve a copy of the script that was updating our servers.

There's more…

In this example, we used an SMTP based e-mail account. In many large corporate environments NTLM based authentication is used for e-mail. If that is the case, you will need to alter the settings on SharePoint via a PowerShell script. You can find complete instructions, plus a copy of the PowerShell script at the following URL.

`https://msdn.microsoft.com/en-us/library/hh231673.aspx`

Configuring subscriptions

Storing reports in SharePoint provides an easy way for users to come and run their reports. But many users would protest, *I ain't got time for that!* Instead they would like their reports delivered to them on a regular schedule.

Fortunately, SharePoint has the ability for users to setup their own schedules to execute and run reports.

Getting ready

Before a user can subscribe to a report, they will have to configure the data source of the report to execute based on stored credentials. Normally, most reports are setup to pass the credentials of the user running the report to the database. But from the standpoint of a subscription, there is no user, it's the SQL Server running the report. Thus the credentials will need to be stored in SharePoint. We'll do that in the first few steps.

How to do it…

1. Navigate to the folder containing the report you wish to subscribe to.
2. Click the ellipse (**…**) beside the report.
3. In the popup menu is another set of ellipses, click it.
4. Select **Manage Data Sources**.
5. A new page appears with the list of all the data sources used by this report. Click the first one (then repeat these steps as needed if there is more than one data source):

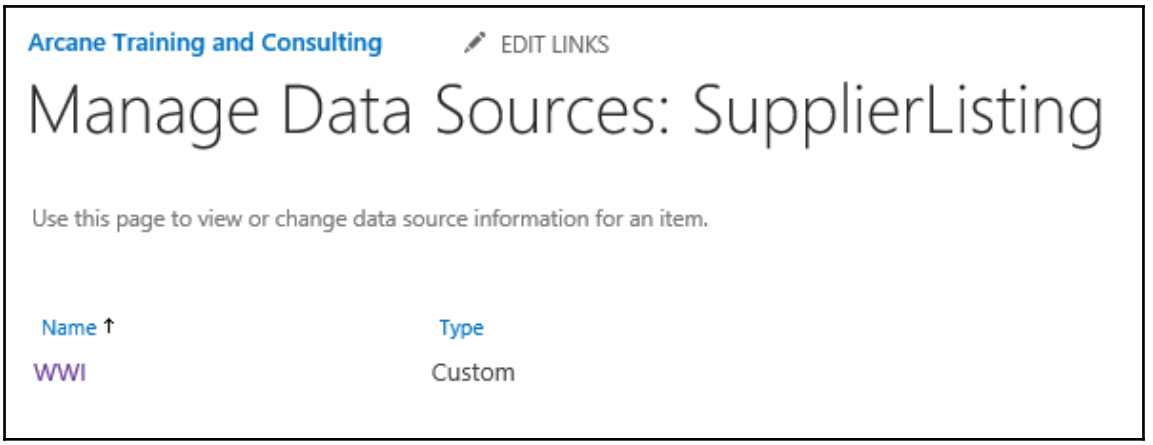


Figure 9.26

6. In the **Credentials** area, change the option to **Stored credentials**.
7. Enter a user name and password with rights to the data.
8. If these are Windows based credentials, check on the **Use as Windows credentials**. If these are SQL Server credentials leave it unchecked.
9. Click **Test Connection** to ensure everything is entered correctly. If so, click **OK** to save the changes:

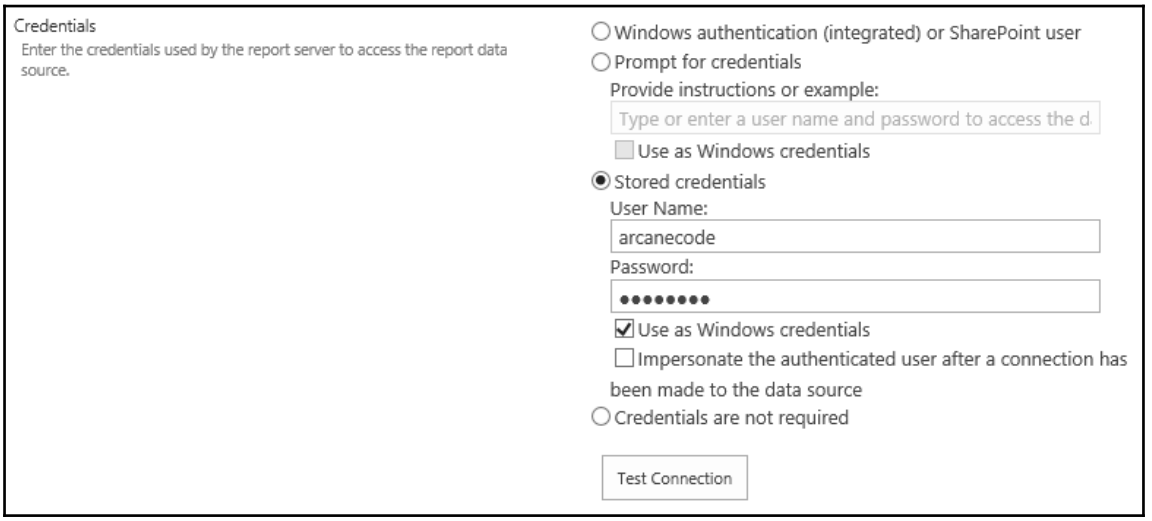


Figure 9.27



You only have to alter the credentials once, after that you can create and update subscriptions without the need to repeat these first steps.

10. Now that the credentials have been set up, we can now create a subscription to the report. Return to the folder containing the report.
11. Click the ellipse (**…**) beside the report.
12. In the popup menu, click its set of ellipses.
13. Click **Manage Subscriptions**.
14. On the **Manage Subscriptions** page, click on **Add Subscription**.

15. There are a variety of fields to fill out, but almost all of them are optional. There are two that are required, the first being the **To** email address. Fill it out with the e-mail for the person who wants to receive the report.
16. The second is the **Subject**, which is already filled in for you:

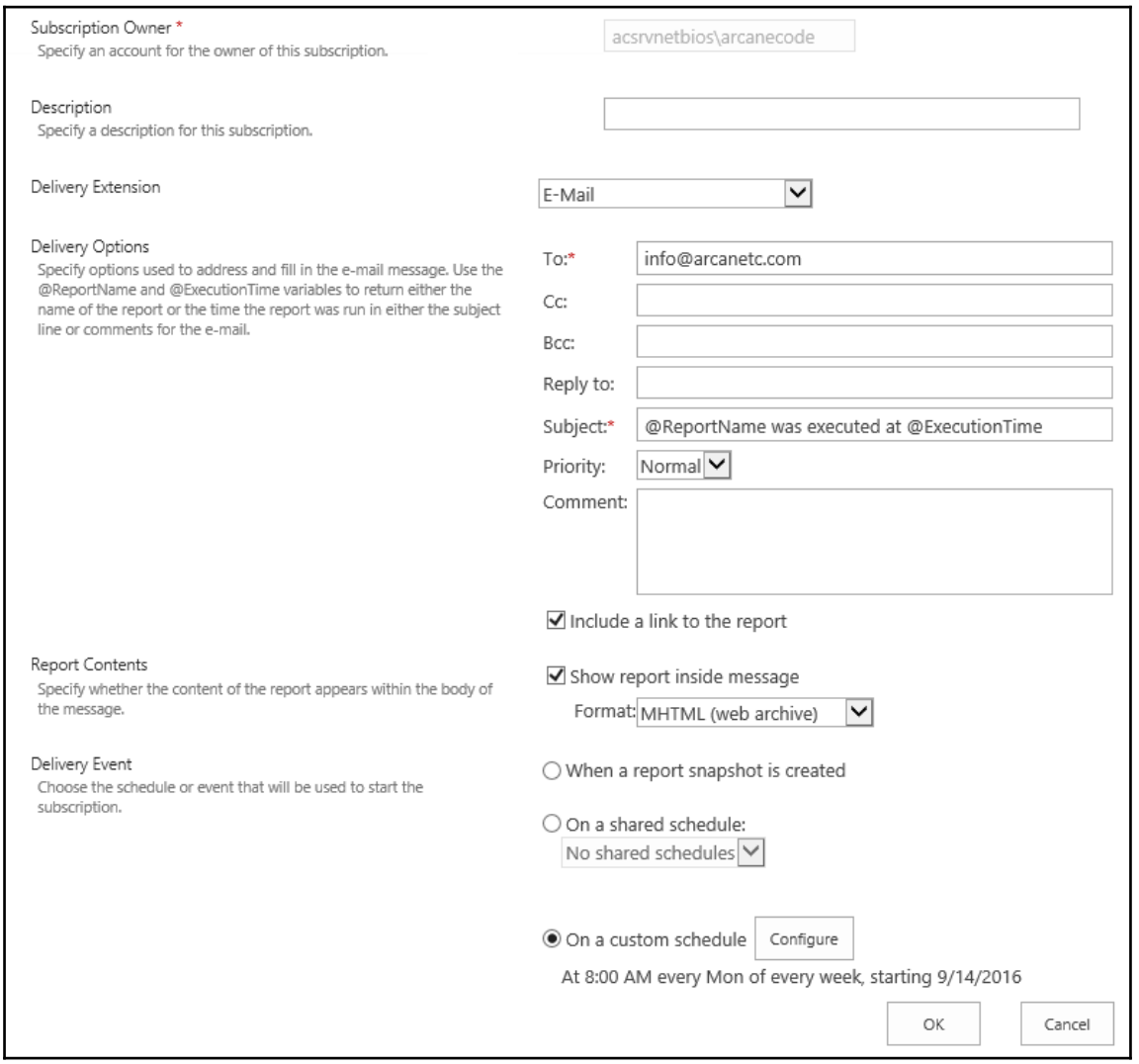


Figure 9.28

17. At the bottom of the page, click on the **Configure** button beside **On a custom schedule**.
18. Review the options to fit the needs of the user. The defaults, shown in the following figure, sets up the report to be e-mailed once a week, on Monday morning at 8 am. Change the schedule to one appropriate for your report. Click **OK** when you are done:

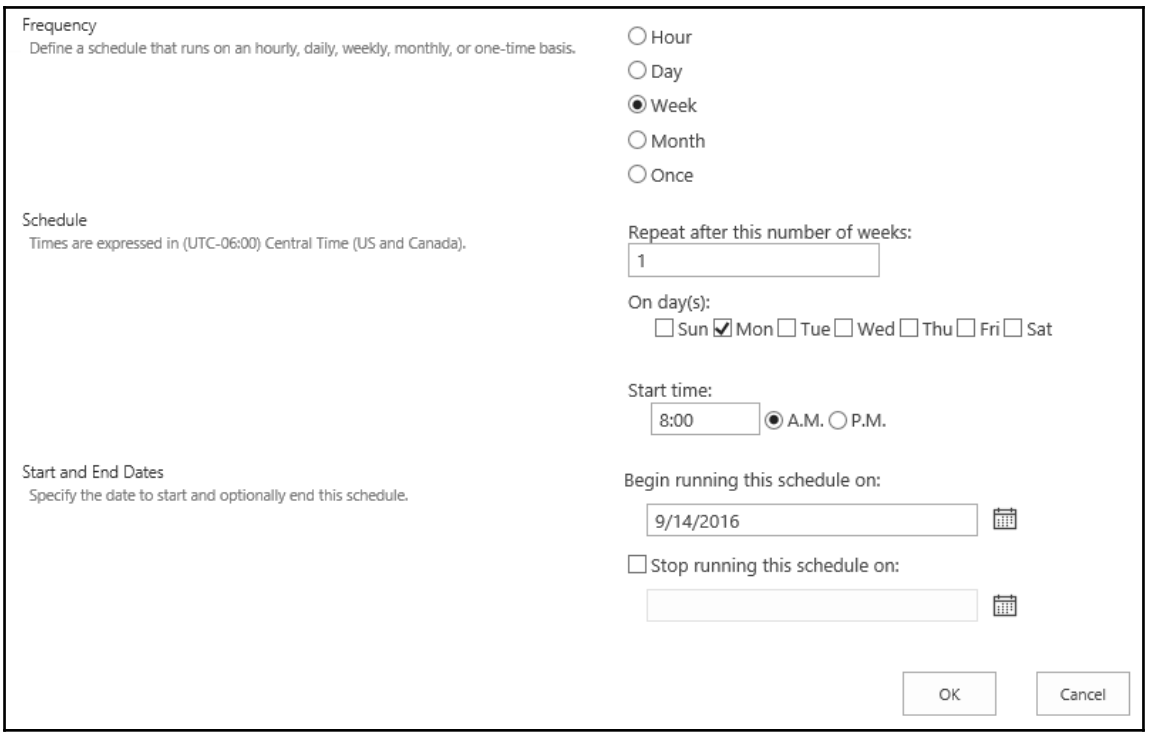


Figure 9.29

19. Click **OK** to save the new subscription.

How it works…

Setting up a subscription is straightforward. After indicating you want a subscription, you simply need to let SharePoint know who should be receiving the report, and what schedule you want it delivered on. There many optional elements you could add, such as a description, whether you wish a link to the report to be included, and more.

You may have noticed the subject line contained two `@` words in it. No, these aren't Twitter accounts, they are placeholders. Placeholders act like variables; in this case they will be replaced by the report name and the time the report is executed.

A report can have multiple subscriptions attached to it. This allows for situations where different groups of users need the same report but on different schedules.

Using Manage Subscriptions interface you can edit existing reports by clicking on the hyperlink in the Delivery Extension column. You can temporarily disable a subscription by placing a check in the box beside the subscription and picking Disable in the tool area. You can Enable or even Delete a subscription using the same technique.

Configuring data alerts

In addition to subscriptions, SharePoint can also alert you when specific conditions occur within the data of your report. For example, if monthly revenue drops a set value you wish to be alerted. Alerts allow you to know when something significant with your data has occurred.

Getting ready

There's not much to do, other than have a report in your document library, whose data source has been configured for unattended execution as you saw in the previous recipe.

How to do it…

1. Navigate to the folder containing the report.
2. Execute the report by clicking on its name.
3. In the upper left corner is a tiny menu, **Actions**. Click it:

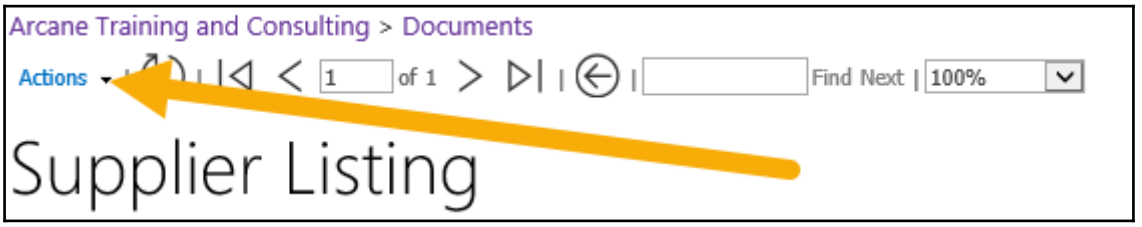


Figure 9.30

4. In the menu, select **New Data Alert**.
5. In the pane on the right, click **Add rule…**.
6. Select a field to set a rule on. In this example, we are going to use **Payment_Days**.
7. Click on the word **is**. Select **is greater than** from the popup menu.
8. Move to the box to the right of **is greater than**. Enter the value to trigger the alert on. In this example, we are entering `25`.
9. For this example, we will take the default schedule, but feel free to set up a custom schedule if you wish.
10. Under **Email settings**, enter a recipient.

11. Click **Save**:

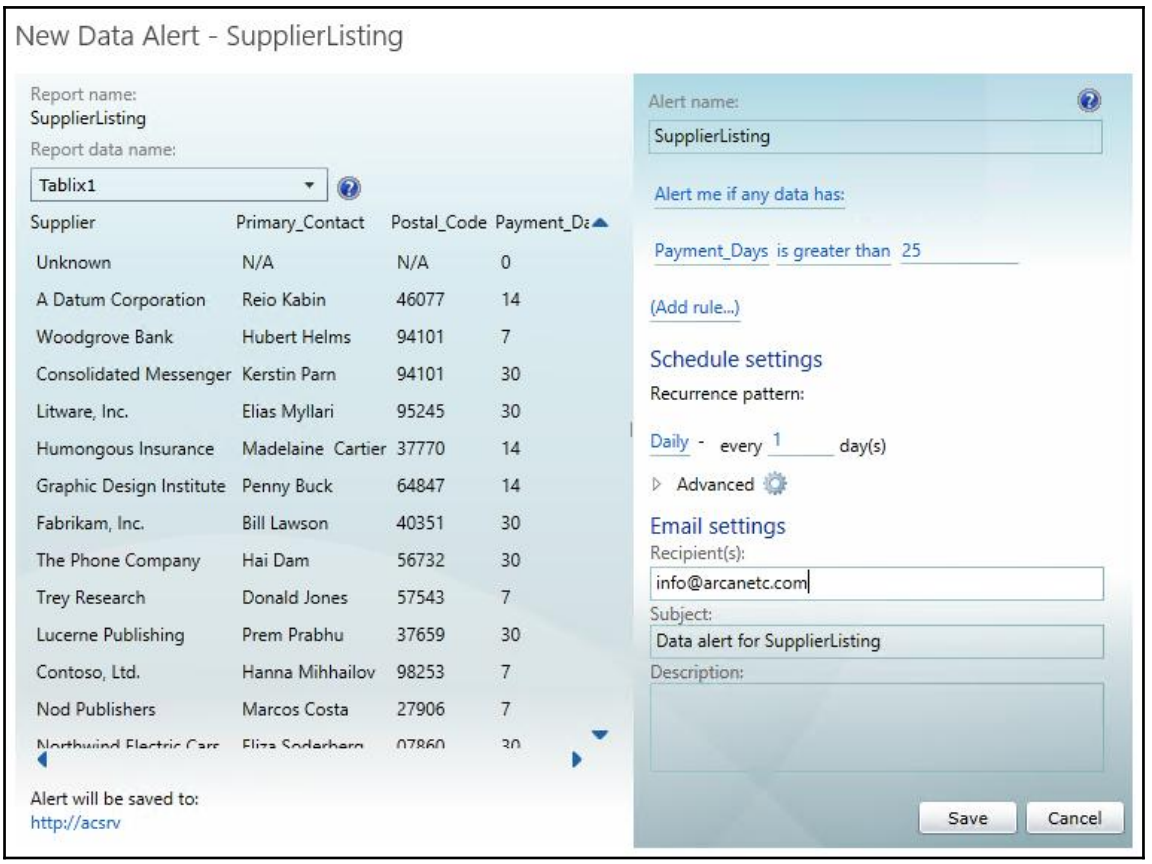


Figure 9.31

12. Your data alert is now set up and ready.
13. To review your data alert, return to the folder with the report in it.
14. Click on the ellipses (**...**) beside the report name.
15. In the menu, click on its ellipses.
16. Pick **Manage Data Alerts**.

17. Right click on the data alert. Through the popup menu, you can **Edit**, **Delete**, or **Run** the data alert:

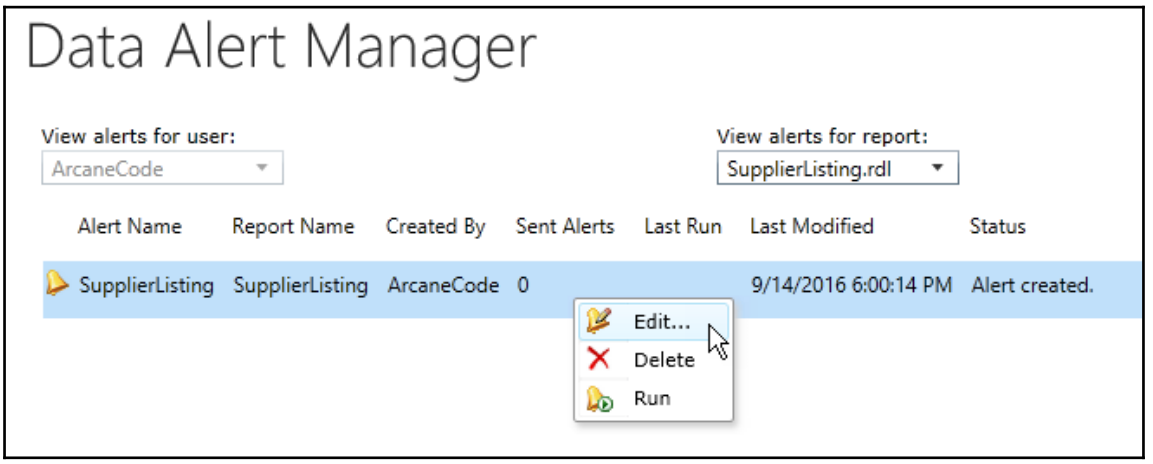


Figure 9.32

How it works…

Data alerts monitor the data behind the report and look for conditions that may occur. This can be valuable when some special event occurs that needs extra attention on the part of the user.

Oddly, the only way to set up a new data alert is to execute the report. Once the alert has been created however, you can edit it from the **Data Alert Manager** page, as well as delete and run it.

10
Administering and Managing Reporting Services

Every implementation requires administering and managing. Therefore, you cannot give up or forget once the implementation of your Reporting Services solution is completed. It needs frequent visits, it needs a regular health-check, it needs changes, it needs to be troubleshot, and of course it needs to be administered and managed.

Reporting Services provides a very rich portal for administering and managing Reporting Services environment. We have already discussed about the new portal and its capabilities with `Chapter 7`, *Consuming Reports – Report Access Enhancement*, *Exploring new Web Portal* recipe, but we did not discuss functionalities of administration and management.

In this chapter, you will explore the administration and management of Reporting Services environment along with its content. This chapter covers the following recipes:

- Managing data sources
- Assigning configured data sources to published reports
- Caching reports
- Creating snapshots of reports
- Backing up Reporting Services
- Monitoring Reporting Services

Introduction

The primary tool for administering and managing Reporting Services is Report Portal which is used for managing the published items. In addition to this, **Reporting Services Configuration Manager** and **SQL Server Configuration Manager** tools are given for handling various administrative tasks.

Before exploring functionalities and the way of managing the content, we need to understand the reasons for management.

Once a reporting solution is published to the Reporting Services, first thing we need to understand is how it works. In high level, when a request is made for a report by an end user, it reads the report definition, process it with data assigned and then finally it renders the report as per the format requested. It actually creates a standard output just after data and report processing called **Intermediate Format** used for rendering the final report to end user, caching and holding snapshots as well.

In most cases, since development environment is different with the production environment, attributes related to the data sources published need to change as per production data sources that is done through the portal. Not only that, there can be a requirement for caching certain set of reports for a specific time period, a requirement for delivering some reports automatically to users' inboxes or a requirement for pre-generating set of reports for speeding up the viewing reports. This is where we need management.

Apart of content management, reports have to troubleshoot, performance and security needs to be improved, managed and make sure that Reporting Services database are backed up. This is where we need administration.

Let us explore most important items out of all of discussed previously.

Managing data sources

Reports are published with connection details of **shared data sources** and they are stored in a common folder called data sources. However, during the publish, developer has a freedom to change the default folder for data sources for the solution, hence additional folders that hold data sources in the portal is not unusual.

Getting ready

There is no specific routine or pre-processes done for managing data sources. If all data sources are stored in the standard `Data Sources` folder, you will work with that folder, else you will be working with multiple folders. When you open the portal, you can see the folder as shown in the following screenshot:

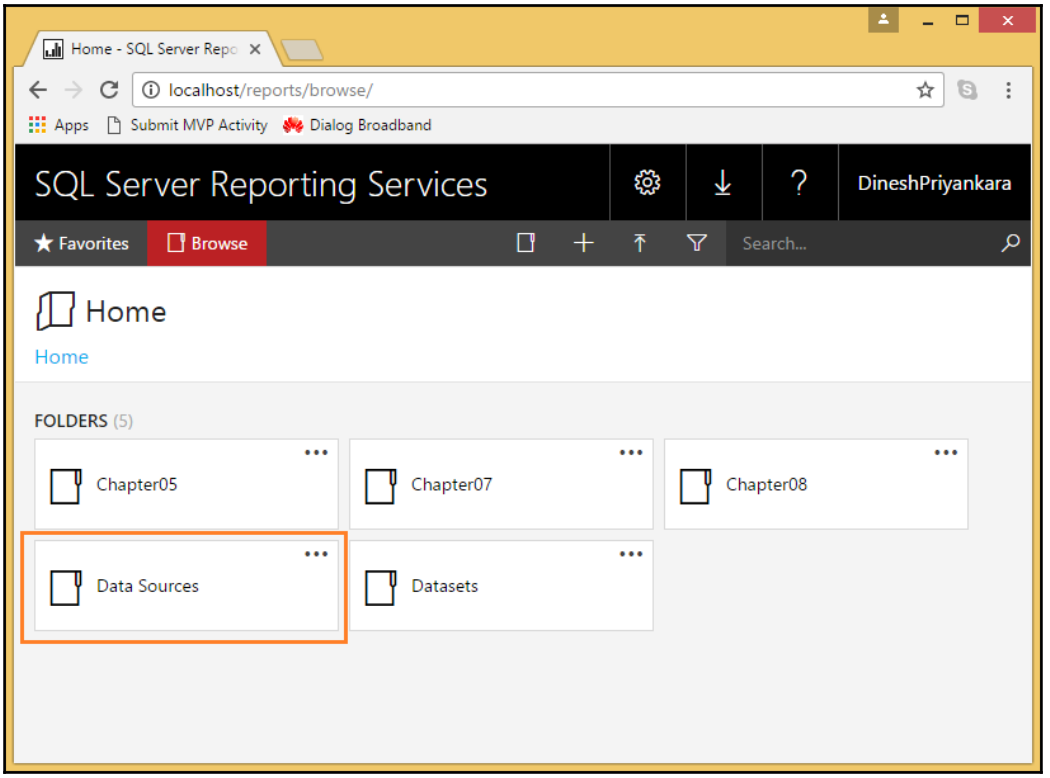


Figure 10.01

If you open the folder, you can see data sources added from all solutions.



Note that different solutions may maintain data sources with the same name and developers decide whether they should be overwritten. .Even though you can change the source from the portal, if a developer decides to overwrite, your changes are lost. Therefore, one way of making sure that your data source is not getting overwritten by different solutions, prefix the source name using either project code or solution code.

See the following image that shows all added data sources when you go into `Data Sources` folder:

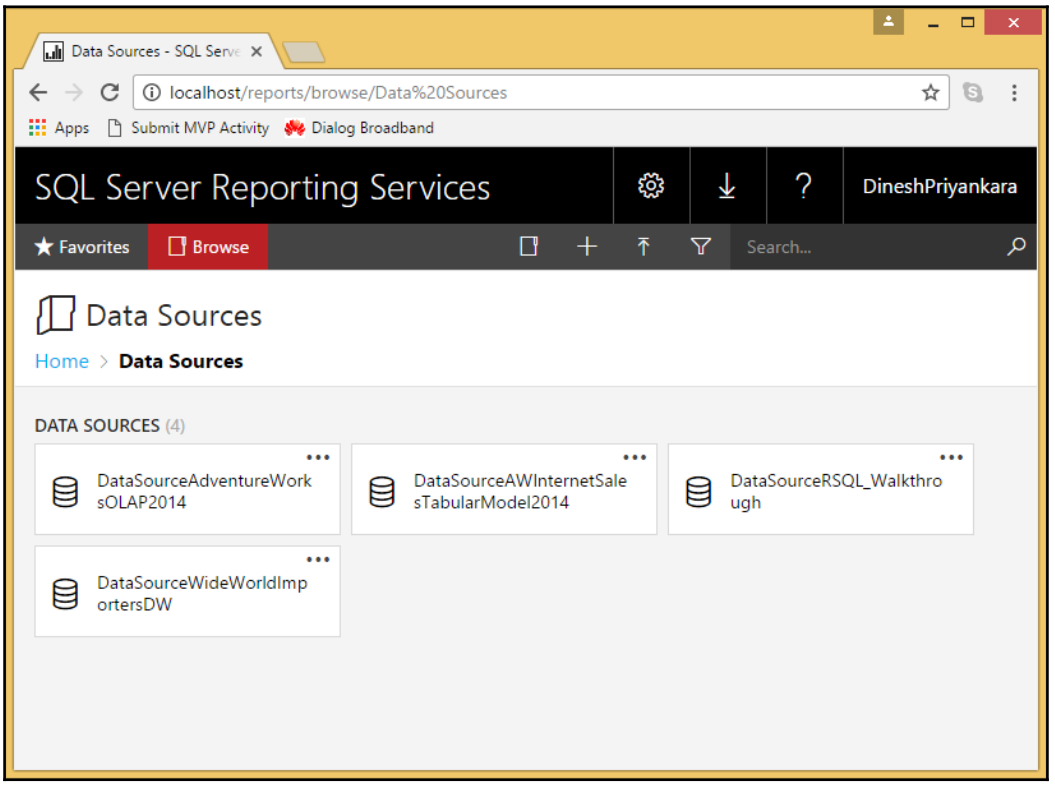


Figure 10.02

Let us see how we can manage a data source.

How to do it…

Following are the steps for managing a data source:

1. Open Reporting Services Web Portal and click on `Data Sources` folder.You should see many data sources when the folder is open (If you do not see, publish the solutions given with previous chapters for getting data sources into this folder).

2. Click on `DataSourceWideWorldImportersDW` data source (see Figure 10.02). You must see all properties of the data source. You can modify **Name**, but it is not recommended when the reports are linked to the data source:

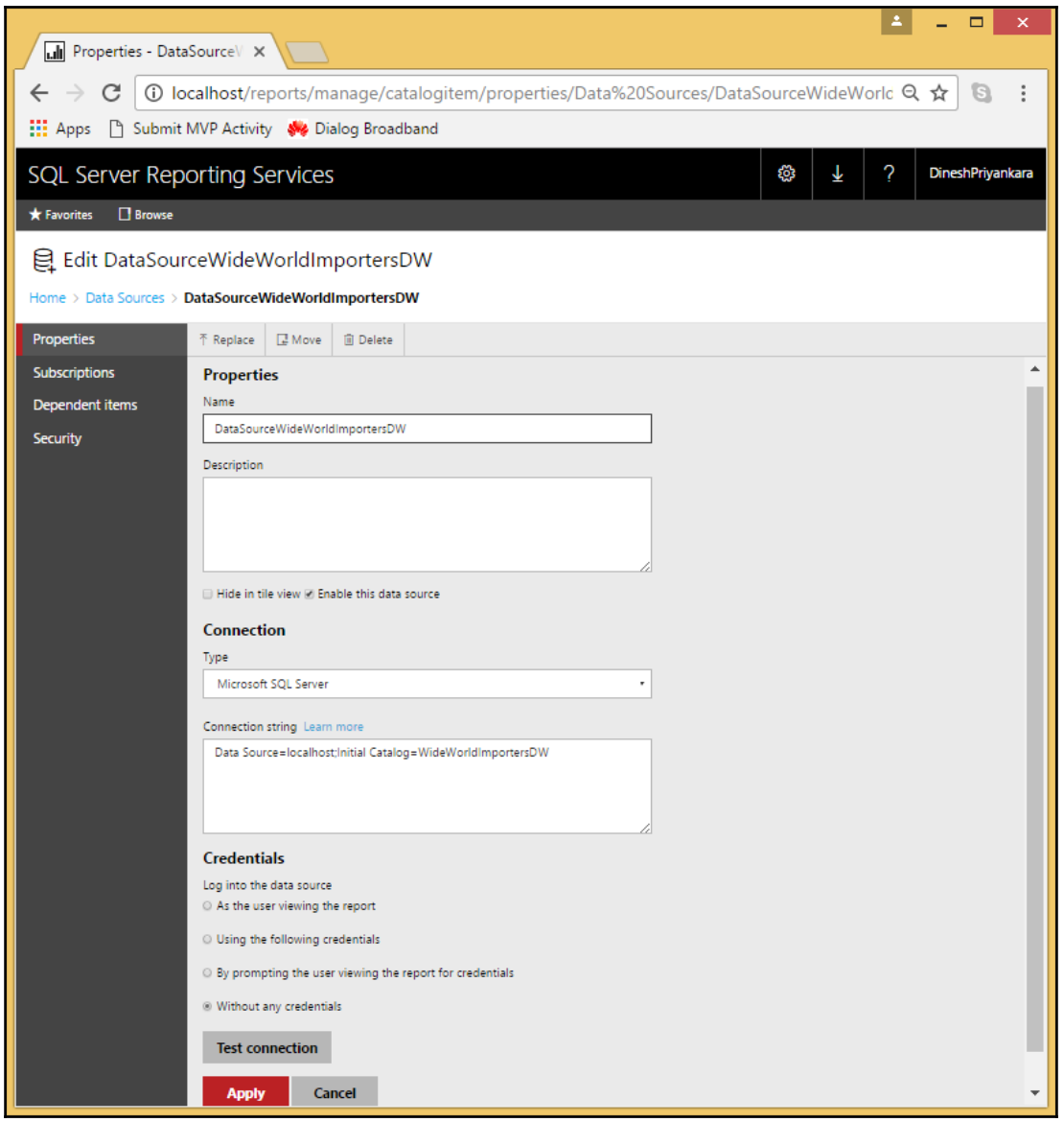


Figure 10.03

3. Expand **Type** drop-down and see. If you need, you can change the type of the data source.
4. See **Connection String**. It has the server name as **Data Source** and database name as **Initial Catalog**. If the database is different in production, you need to change the **Initial Catalog**.
5. Have a look at **Credentials** section. This explains how Reporting Services uses credentials for connecting to the data source. Let us understand each option given and select one:
    - **Option 1**: **As the user viewing the report**:This is the recommended settings as it uses end user's security token for accessing the report as well as accessing the data source. However, if the data source is on a different server, impersonation and delegation have to be enabled.
    - **Option 2**: **Using the following credentials**:You can either set up a SQL login or Windows login for accessing the data source. This option makes sure that credentials are stored with the Reporting Services in an encrypted format. This is specifically useful when report snapshots a regenerated without a user request and it is recommended to use for external data sources.
    - **Option 3**: **By prompting the user viewing the report for credentials**: These settings are recommended when the data is sensitive and should be opened only for specific users. If this option is enabled, Reporting Services prompts for user id and password for user and data source is accessed using credentials supplied with the request.
    - **Option 4**: **Without any credentials**: This option can be used if the source does not require authentication.

6. Let's configure the option. For testing purposes, select **Using the following credentials** option and set up a SQL Server login (if you do not have an account, you need to create a SQL Server login that can access `WideWorldImportersDW` database:

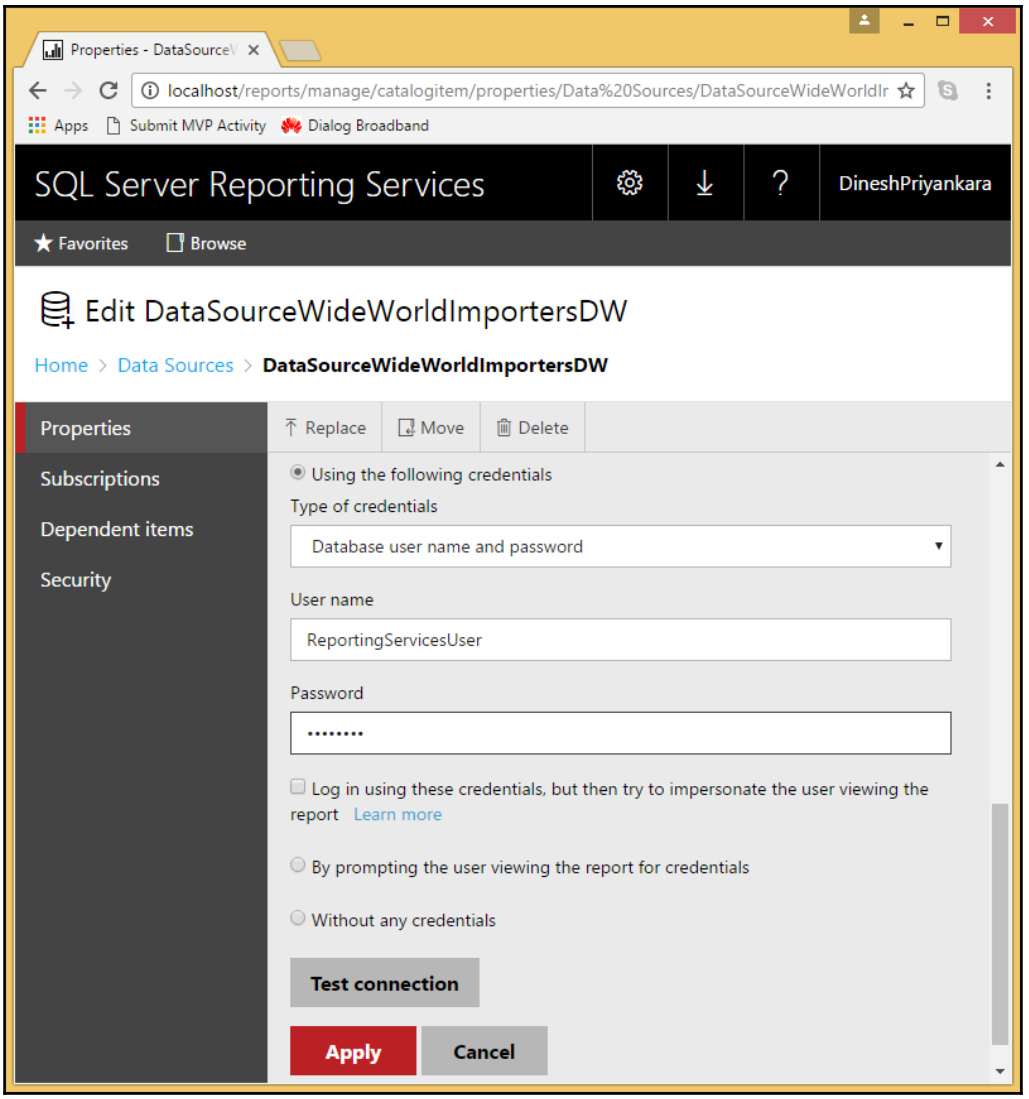


Figure 10.04

7. Click on **Test** connection and see whether the connection is established.
8. Click on **Apply** to save settings.

Now, lets test one of the reports that uses this data source and see whether it works:

1. Go back to **Home** and open `Chapter05` folder.
2. Click on `Report04 - TreeMap` report for opening it. If credentials you set are accurate, you must see the report.

How it works…

Reporting Services accesses the data source along with the report definition, for extracting data into the report. Regardless of the **Credentials** option selected, if the account is Windows account, it is authenticated from the relevant provider (such as Active Directory) or if the account is a SQL Server login, it is authenticated using the SQL Server itself. Data is extracted as per the permission granted to the account and then Reporting Services creates the Intermediate Format. The renderer uses the Intermediate format for rendering the report based on the format requested by the end user.

There's more…

When you upload a new report that uses the same data source, which is already configured in Reporting Services, you may need to connect the report with the data source that is already configured in the portal. Let us see how it can be done in *Assigning configured data source to a published report* recipe.

Assigning configured data sources to published reports

Even though the complete solution is published, we continuously change existing reports and add new reports. Sometime, we author the report separately completely outside the solution and, upload the report to the portal as a new report. This requires configuring data source to upload report.

Getting ready

Let us create new solution and a new report. Once the report is created, let's upload it manually rather using the SQL Server Data Tools.

Follow the steps for creating the report:

1. Open the SQL Server Data Tools and create a new project using Reporting Services template. Name the project as `Chapter10`.
2. Create a report called `DataSetCustomers`.
3. Create a dataset using `WideWorldImportersDW` database. Make sure that you create the data source as a Shared Data Source.
4. Use the following code for getting data:

```
SELECT Customer, Category, [Buying Group]
FROM Dimension.Customer
ORDER BY Customer;
```

5. Name data source as `WideWorldImportersDW` and dataset as `DataSetWideWorldImportersDW`:

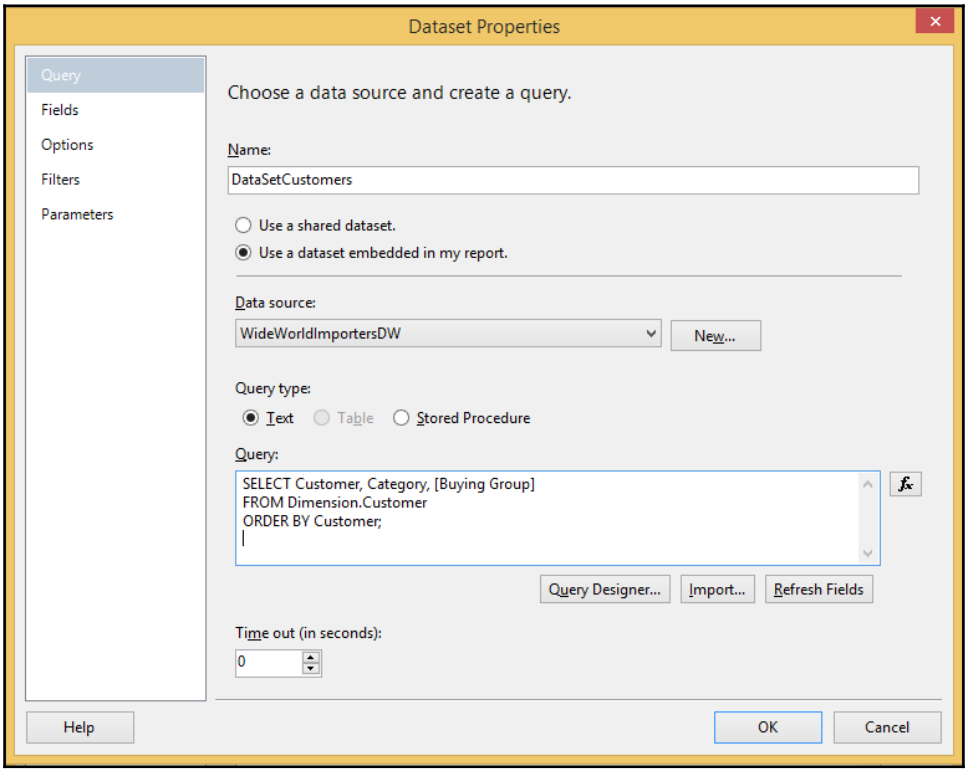
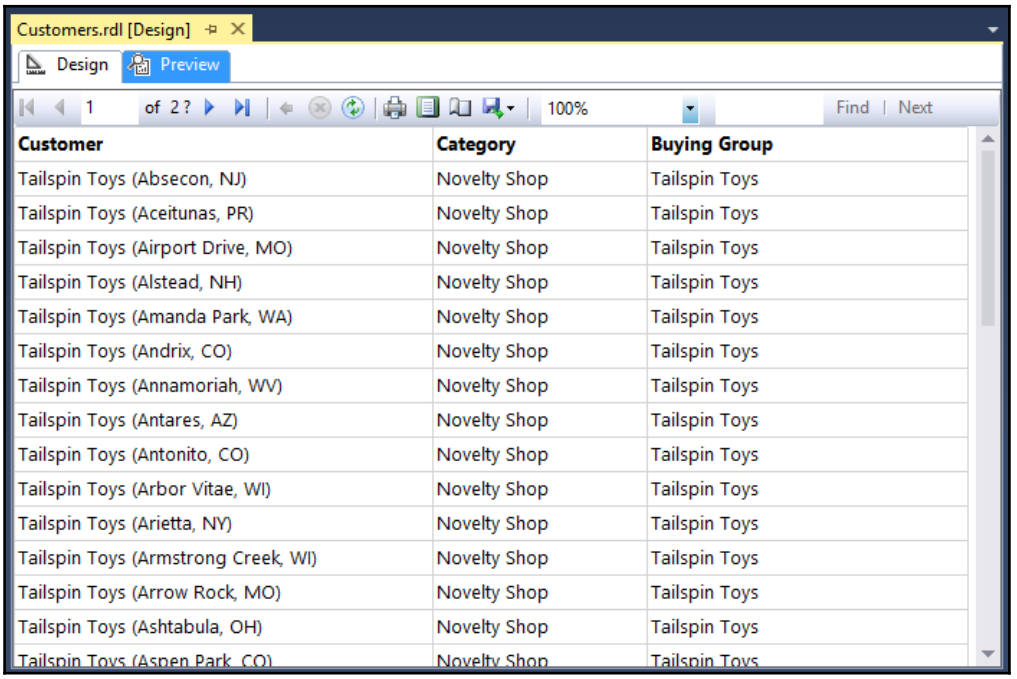


Figure 10.05

6. Click **OK** to save the dataset.
7. Add a **Table** data region to the report and set all columns to it.
8. Preview and see whether report shows data without any issue:



Customer	Category	Buying Group
Tailspin Toys (Absecon, NJ)	Novelty Shop	Tailspin Toys
Tailspin Toys (Aceitunas, PR)	Novelty Shop	Tailspin Toys
Tailspin Toys (Airport Drive, MO)	Novelty Shop	Tailspin Toys
Tailspin Toys (Alstead, NH)	Novelty Shop	Tailspin Toys
Tailspin Toys (Amanda Park, WA)	Novelty Shop	Tailspin Toys
Tailspin Toys (Andrix, CO)	Novelty Shop	Tailspin Toys
Tailspin Toys (Annamoriah, WV)	Novelty Shop	Tailspin Toys
Tailspin Toys (Antares, AZ)	Novelty Shop	Tailspin Toys
Tailspin Toys (Antonito, CO)	Novelty Shop	Tailspin Toys
Tailspin Toys (Arbor Vitae, WI)	Novelty Shop	Tailspin Toys
Tailspin Toys (Arietta, NY)	Novelty Shop	Tailspin Toys
Tailspin Toys (Armstrong Creek, WI)	Novelty Shop	Tailspin Toys
Tailspin Toys (Arrow Rock, MO)	Novelty Shop	Tailspin Toys
Tailspin Toys (Ashtabula, OH)	Novelty Shop	Tailspin Toys
Tailspin Toys (Aspen Park, CO)	Novelty Shop	Tailspin Toys

Figure 10.06

Report is ready now. Let us see how we can manually configure the data source once it is uploaded.

How to do it…

Follow the steps given here for uploading the report and configure the data source:

1. Open the Reporting Services Web Portal.
2. Click on **New** menu and click on **Folder**:

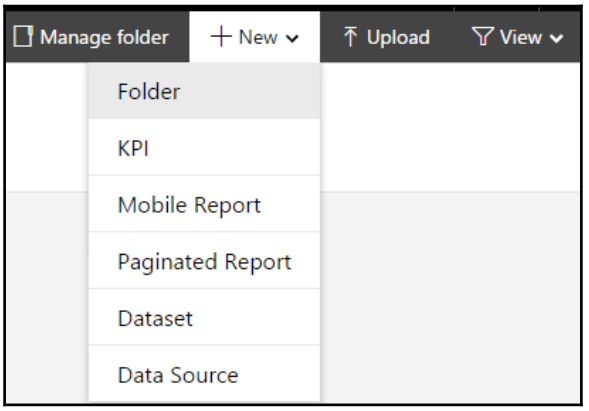


Figure 10.07

3. Type `Chapter10` as the **Name** and click on **Create**:



Figure 10.08

4. Go into `Chapter10` folder and click on **Upload** menu:

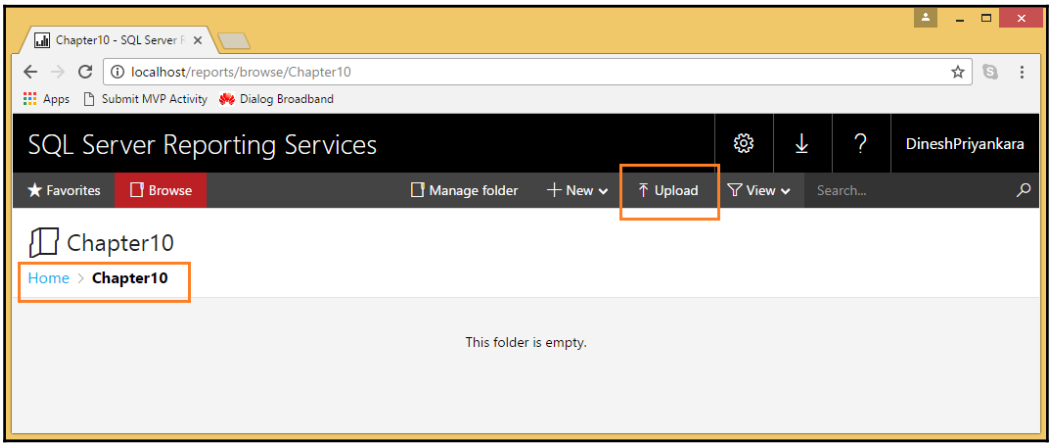


Figure 10.09

5. Select the `Customers.rdl` file created and click on **Open** to upload the report:

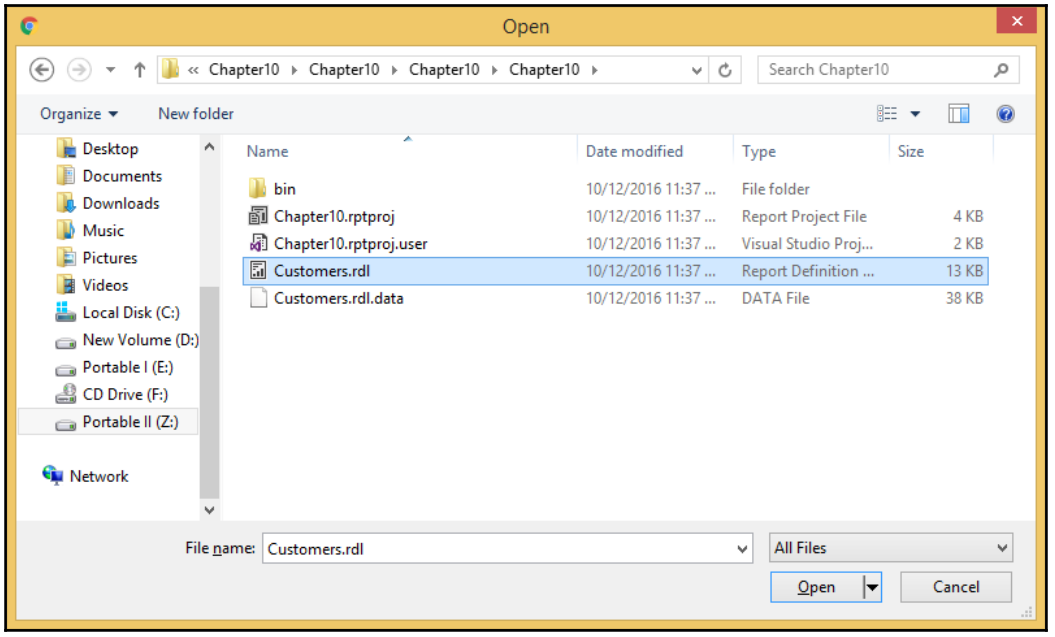


Figure 10.10

6. Now the report is uploaded. Click on the report and see whether you open it. The output is:

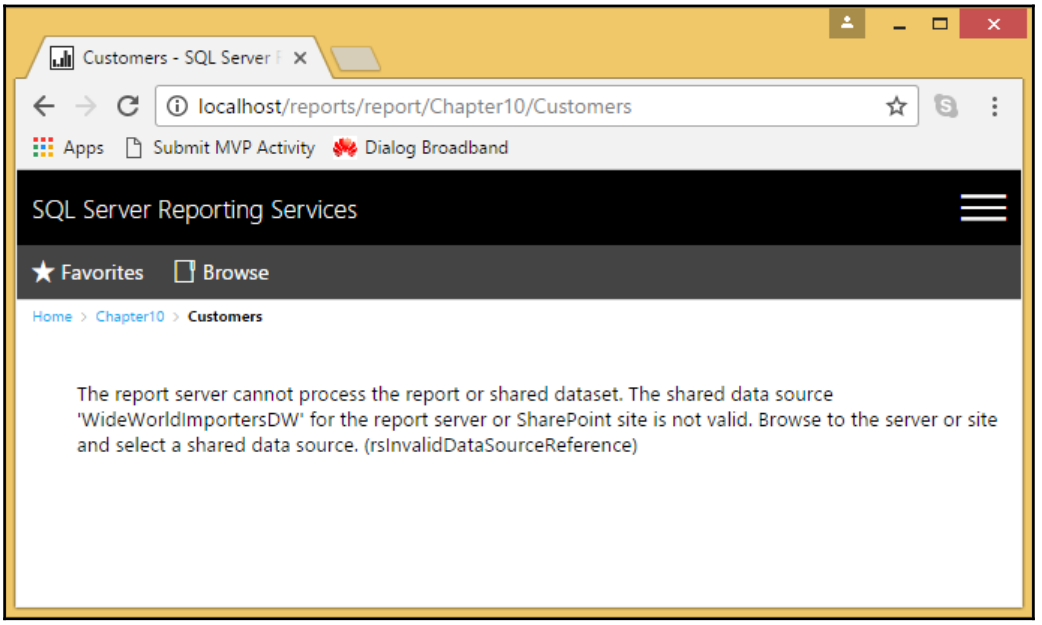


Figure 10.11

7. The reason for preceding message is the data source configuration. We need to match the data source recorded in the report with the data source exist in the portal.
8. Go back to the folder and click on the ellipsis button on the report. Click on **Manage** menu item:

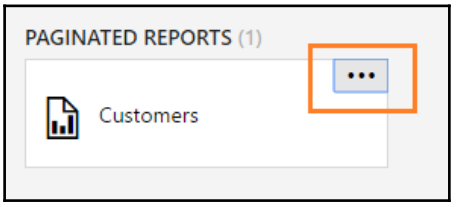


Figure 10.12

9. In **Edit Customers** page, select **Data** source and click on ellipsis button set with rectangle:

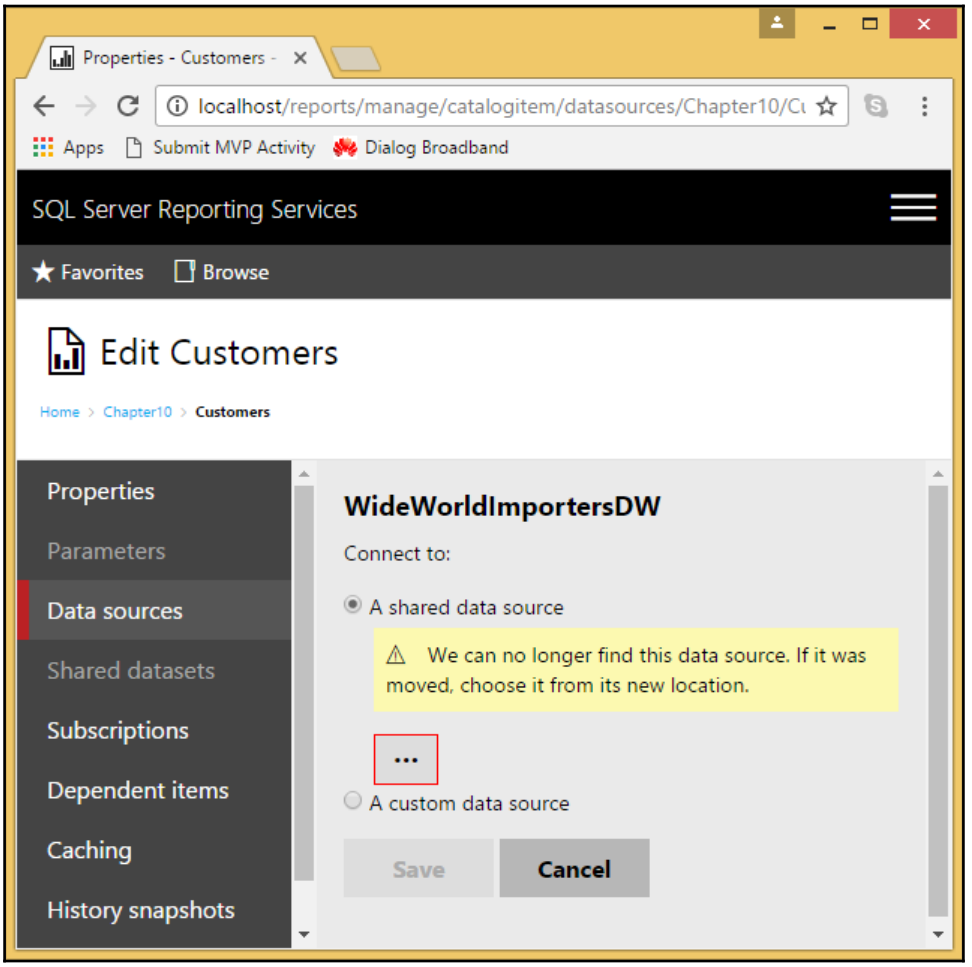


Figure 10.13

10. Once the **Data Source** window is opened, go into `Data Sources` folder and click on `DataSourceWideWorldImportersDW` data source:

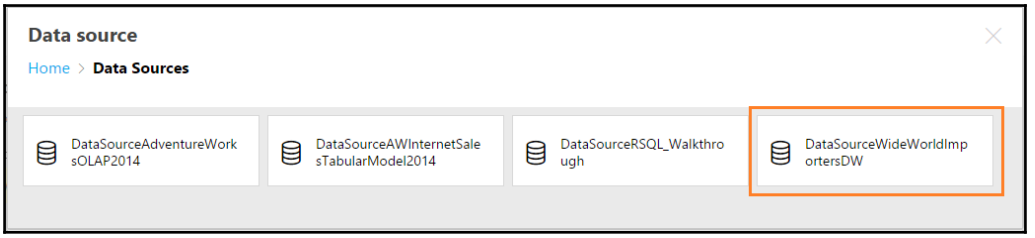


Figure 10.14

11. Click on **Save** and preview the report. You should be able to see the report now:

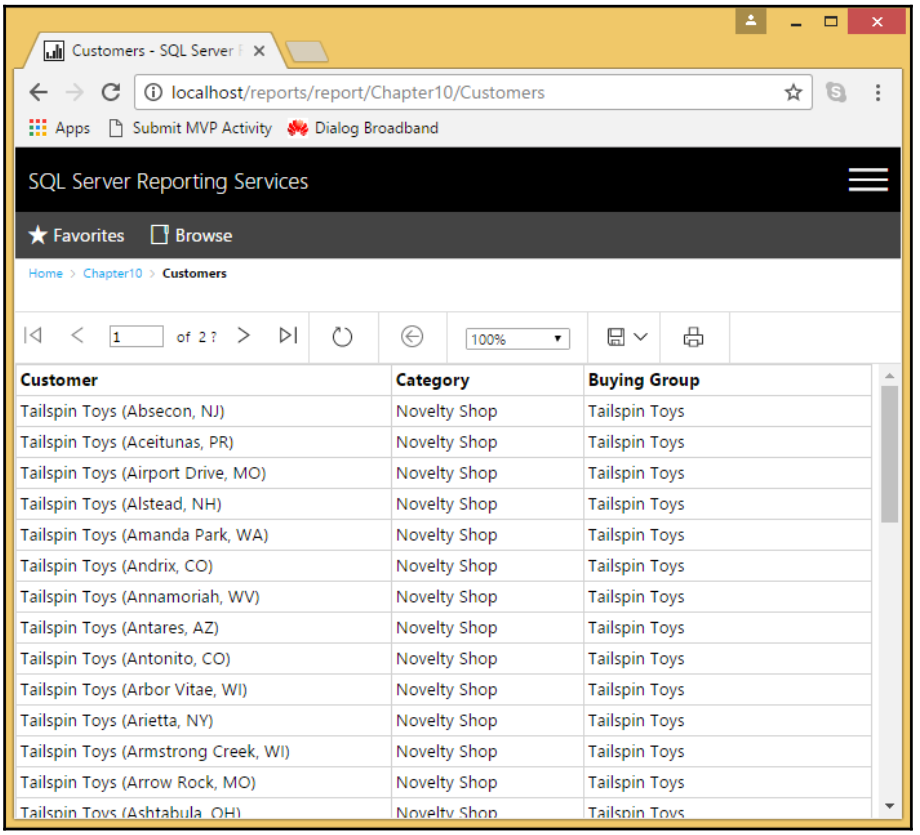


Figure 10.15

How it works…

As described with the previous recipe, Reporting Services looks for the data source at the request for processing data. Since the report we uploaded is configured with a shared data source and there is no data source available in the portal with the name set, it throws an exception during the data process.

Note that if the data source is created as an embedded connection during the development, report does not need a re-linking with the source once it is uploaded. The embedded connection added goes with the report when we upload, hence Reporting Services can use the same for connecting with the source. However, you may need to change it if the production environment needs to connect with a different database.

There's more…

The time it takes for rendering the report to the client is based on time it takes for processing data and creating the report. Some reports take only few seconds and some take hours for processing and rendering. Is there a way of speeding up report rendering that takes long time? There are few ways, let us check them with functionalities given, in the *Caching reports*, recipes.

Caching reports

Not all reports are required to be shown with real-time data, hence if the report can be internally saved with initial execution and can be used as a pre-loaded report for next request, it will surely shorten the time for satisfying user's request as it does not need to spend time on processing the report along with data extraction. This can be achieved by caching the report.

Getting ready

For improving the overall performance there are some important settings that you need to understand if you want to cache the report. Remember, if you enable caching, you need to make sure either it get's expired based on a pre-defined time-frame or it is linked with a *Cache Refresh Plan*. Otherwise, the report will deliver with the cached copy even though new data is available at the source.

Most common technique used for caching the report is, generate the cache with the initial request and set a time-frame for removing the cache. Once the cache is removed, the next execution processes the report again, spending the usual time it needs to complete the data and report processing, and deliver the report to the user while adding it to the cache. This makes sure that data is refreshed, report is generated for new data and it is cached for speeding up the next request.

Instead of setting a specific time-frame, you can create a schedule for expiring the cache. This schedule can be report specific or a shared schedule.

Not only for expiring, a schedule can be created for refreshing the plan.

Example, you can instruct to Reporting Services for removing the cache and generate a new cache as per the schedule set.

However, make sure you have an appropriate plan for maintaining caches because it might lead to a situation that users always see old data violating business requirements.

Note that when the report is delivered to the user, user does not know whether the report shows new data or cached data. Therefore, indicating the date and time of the report generation in the report must be done.

Let us make a report and add an indication:

1. Open the solution `Chapter10` created with the previous recipe.
2. Take a copy of **Customer** report and add it as a new report, naming `Customer_New`.
3. Open the design mode of `Customer_New` report and add a text box at the top of the data table.
4. Set the newly added text box with the following expression:

```
"Generated Date and Time: " & Globals!ExecutionTime
```

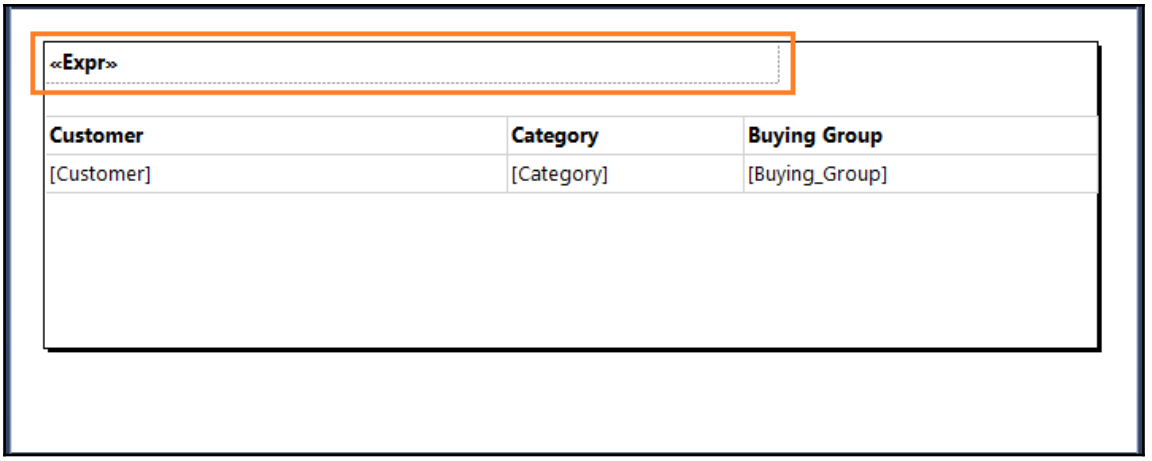


Figure 10.16

5. We need to make sure that report can be either generated by a schedule or on-demand. For that, Reporting Services needs credentials stored.

6. Open the **Data Source Properties** of the report and change it as an **Embedded connection**:

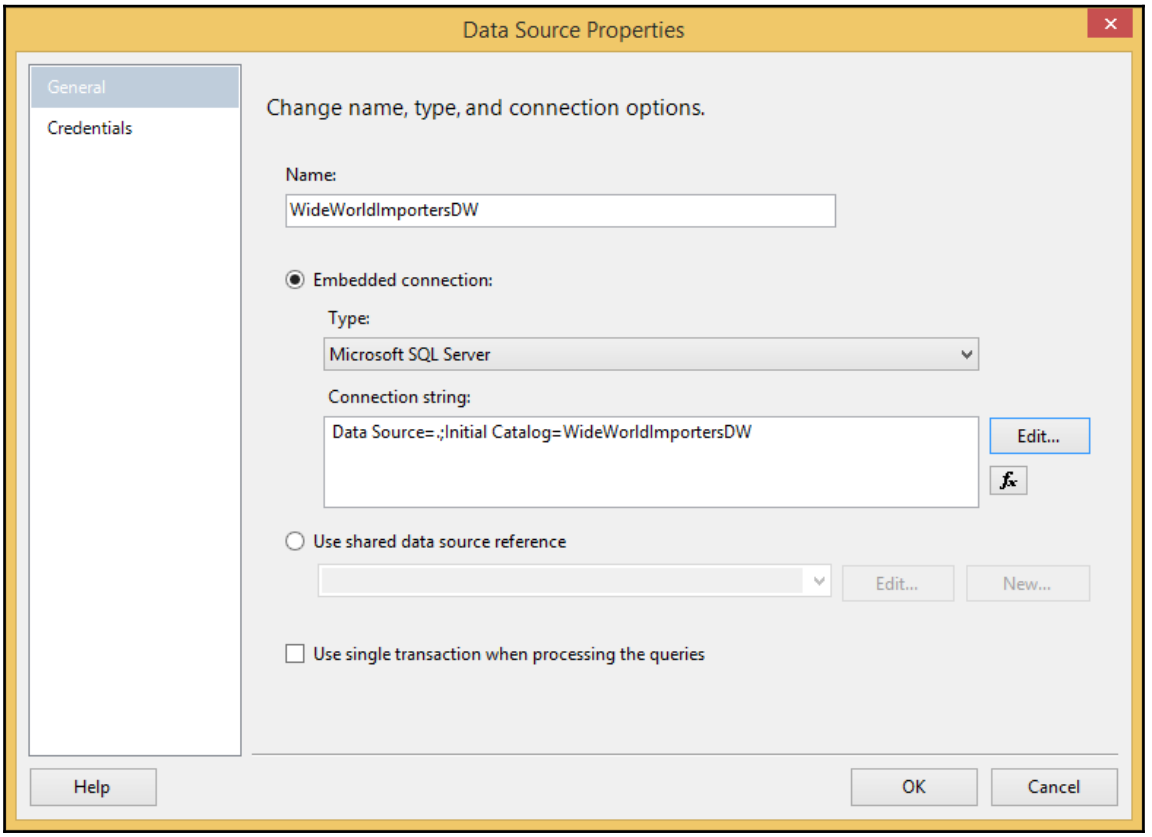


Figure 10.17

7. Make sure that **User name** and **Password** are set and **Save my password** is checked for saving the password:

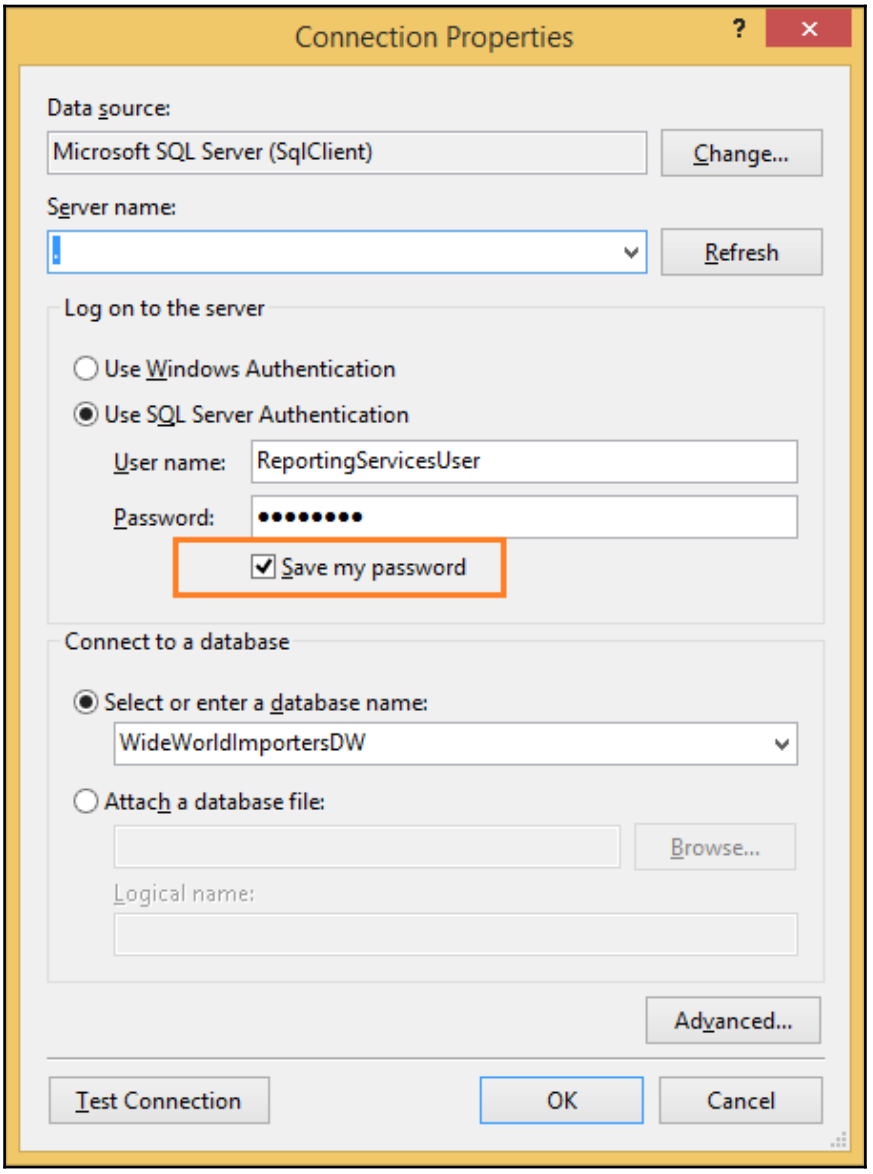


Figure 10.18

8. Save the data source and publish the report.
9. Once published, view the report and see. You should see current date and time print on the report along with the dataset:

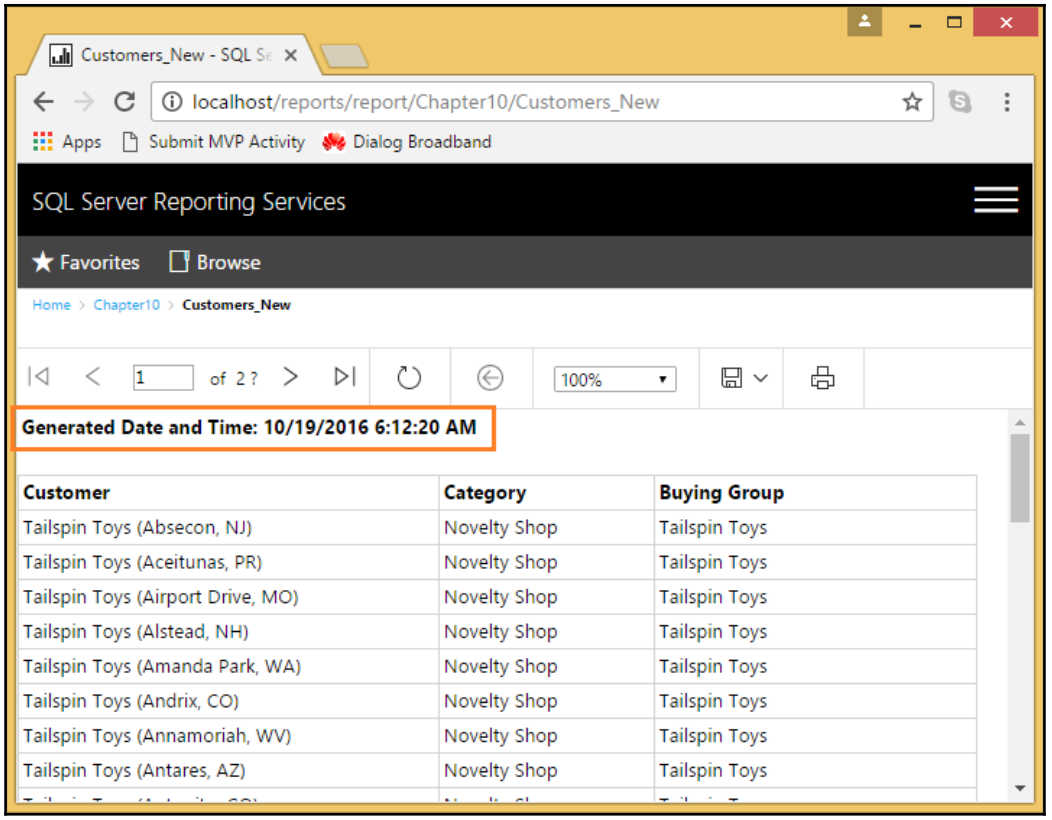


Figure 10.19

Now the report is ready for caching. Let us see how we can enable caching and manage it.

How to do it…

Follow the steps below for enabling caching on the `Customer_New` report:

1. Open the Web Portal and go into `Chapter10`.
2. Click on the ellipsis button on `Customer_New` report tile and click on **Manage** menu item for opening **Properties** of `Customer_New` report.

3. Click **Caching** in the left pane for opening settings related to caching (See Figure 10.20).
4. Note that **Always run this report with most recent data** radio button is selected. But, select **Cache copies of this report and use them when available** radio button (See Figure 10.20).
5. Set **Cache expiration** as **Cache expires after 15 minutes** (See Figure 10.20).
6. Click on **Apply** for saving settings (See Figure 10.20):

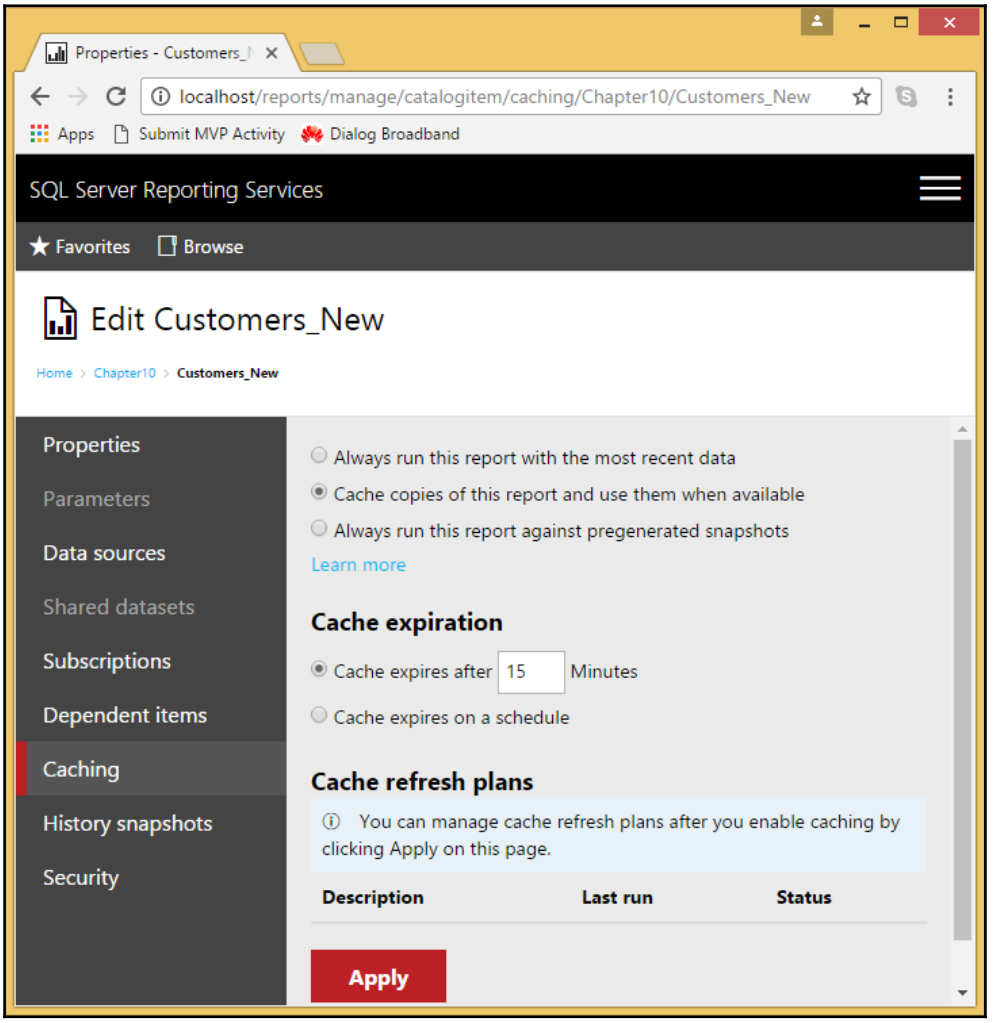


Figure 10.20

7. Now open the report and note the time:



Figure 10.21

8. Wait for few minutes and click on the refresh button to open the report again. You will notice that the date and time printed on the report is not current and it is same as the previous report.
9. Wait for more than 15 minutes and refresh the report again. Now you should see a current time printed on it. This indicates that the cache has been expired after 15 minutes and the request is satisfied by loading data and processing the report:

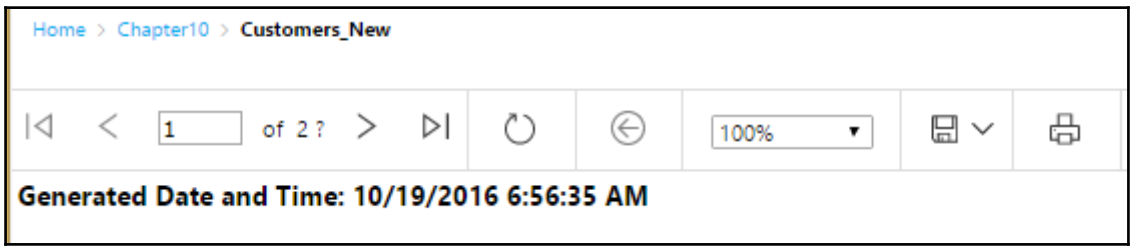


Figure 10.22

Now you know that how Reporting Services caches reports and expires cached reports. As you noticed with cache settings page, expiration can be scheduled without setting it with a specific time-frame. Let us see how it can be done:

1. Open the properties of `Customer_New` report and go into **Caching** section (See Figure 10.23).

2. Under **Cache expiration**, select **Cache expires on a schedule** and select **Report-specific** schedule radio button:

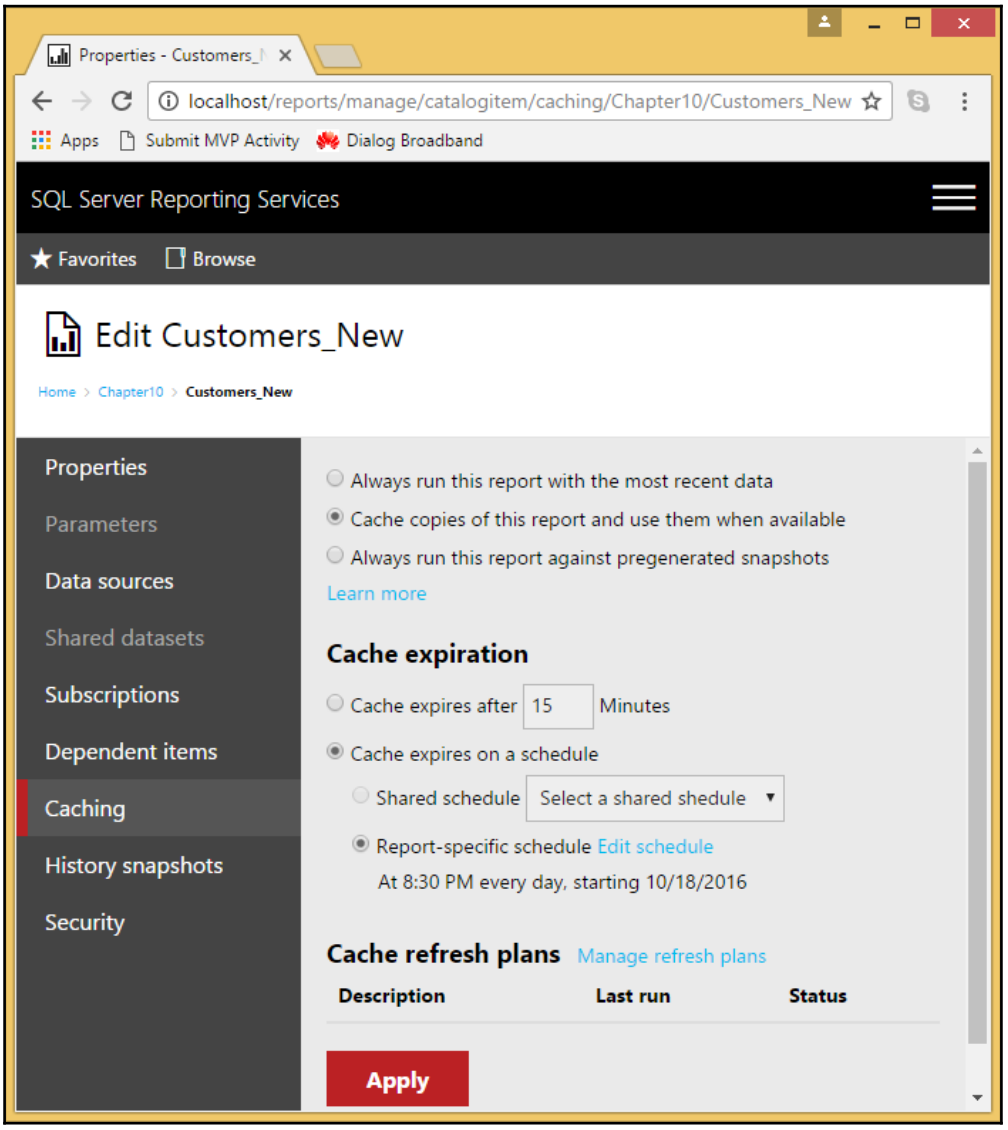


Figure 10.23

3. Click on **Edit schedule** for changing the default schedule (See Figure 10.23).
4. You will notice that the schedule can be set as an hourly, daily, weekly or monthly schedule and based on the selection additional configuration appears. Select Hour for creating an hourly schedule and set the **Hourly Schedule** as **Run the schedule every: 01 hours 00 minutes**:

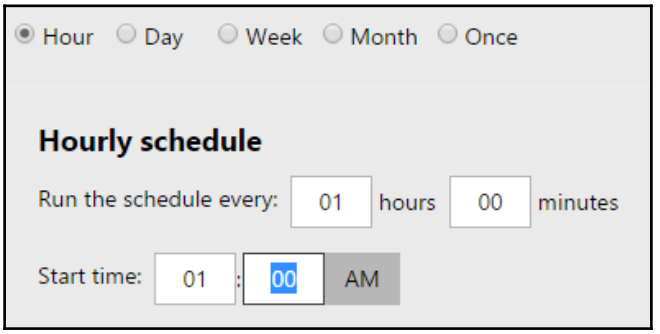


Figure 10.24

5. Set the **Start Time** as 01:00 AM. Keep the default value **Begin running this schedule as on** and click on **Apply** for creating the schedule.
6. Click on **Apply** again for saving cache settings of `Customer_New` report.
7. Now the cache will expire in every hour.

How it works…

Reporting Services stores the cached reports in an intermediate format. It saves the cached reports in its database and you will not lose it even if the server is rebooted.

Note that once the cache is expired, it does not generate the cache automatically. It caches the report only when user opens the report again. Reporting Services loads data from the source and processes the report, and once the processing is done, it caches it while rendering the report to user.

If the report is parameterized, Reporting Services caches reports are based on values selected for parameters.

If user1 connects and runs the report for Product1 using a parameter, the report cached only has data for Product1.

When the user2 opens the report for Product2, a new cache is created for it, resulting multiple cached intermediate reports in the database for same report.

Note that you cannot delete a report from the cache directly unless you use SOAP API. If a schedule is used for expiring the cache, make sure that it is running because if it is paused, cache will not be expired until it is active again.

There's more…

Did you notice that there is another setting called **Cache refresh plans** in settings of Caching? It is shown in the following screenshot:

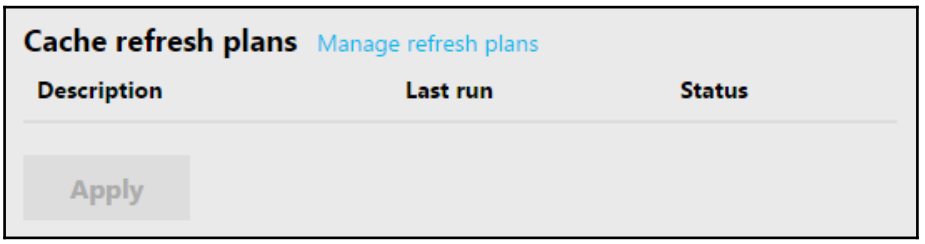


Figure 10.25

Yes, this is for getting the cache refreshed automatically. As you know now, once the cache is expired, it does not generate the cache again until the next user makes the request, making the next user wait until Reporting Services processes the report with the request. This can be avoided if you have a refresh plan for regenerating the cache.

It is the schedule that refreshes the cache. Once it is scheduled, it will create the new cache regardless of the cache expiry settings.

For an example, if the cache expiration is set as after 45 minutes, you can schedule the refresh plan to refresh cache in every 30 minutes. This makes sure that report is re-generated and cached before expiration, even though user does not make a request and is getting the output from the cache.

Creating snapshots of reports

You might have already noticed that caching does not hold multiple cached copies for the same report (for same parameter values if must be parameterized). This means that it does not save the history, or it does not create snapshots and keep them for viewing later. What if you need to view previous copies generated later?

This requirement often comes when business users need to do the comparison on a dataset before and after some data processing. Generally, the need is, make a report with existing data, process data applying business logics and then regenerate the report using new processed data.

For the comparison, either the old report has to be exported and kept or the old report must be printed and kept. Reporting Services provides a solution for this. Yes, it allows you to generate snapshots of the report and keep them as a separate report for later visit.

Getting ready

Before checking the functionalities of this, let's see how we need to maintain snapshots of reports.

Generation of snapshots vary from organization to organization, or user to user. One may need to generate a snapshot on-demand and another may need to generate multiple snapshots of a report based on a schedule.

Not only that, you need to decide retention period of snapshots generated. You may keep all generated snapshots until they are manually deleted or you may need to delete them automatically based on a condition.

Once above considerations are decided, you can use the portal for enabling snapshots. Let's see how we can work on it.

How to do it…

Let's work with the same report: `Customer_New` report:

1. Open the web portal and get the properties of the `Customer_New` report.
2. Click on **Caching** for getting caching properties opened.

3. Select **Always run this report against pregenerated snapshots** option (See Figure 10.26).
4. Under **Cache snapshots**, select **Create cache snapshots on a schedule** checkbox and select **Report-specific schedule** (See Figure 10.26).
5. Click on **Edit schedule** and add a schedule that gets executed in every 30 minutes (See Figure 10.26).
6. Select **Create a cache snapshot when I click Apply on this page** checkbox making a snapshot immediately (See Figure 10.26).
7. Click on **Apply** to save settings:

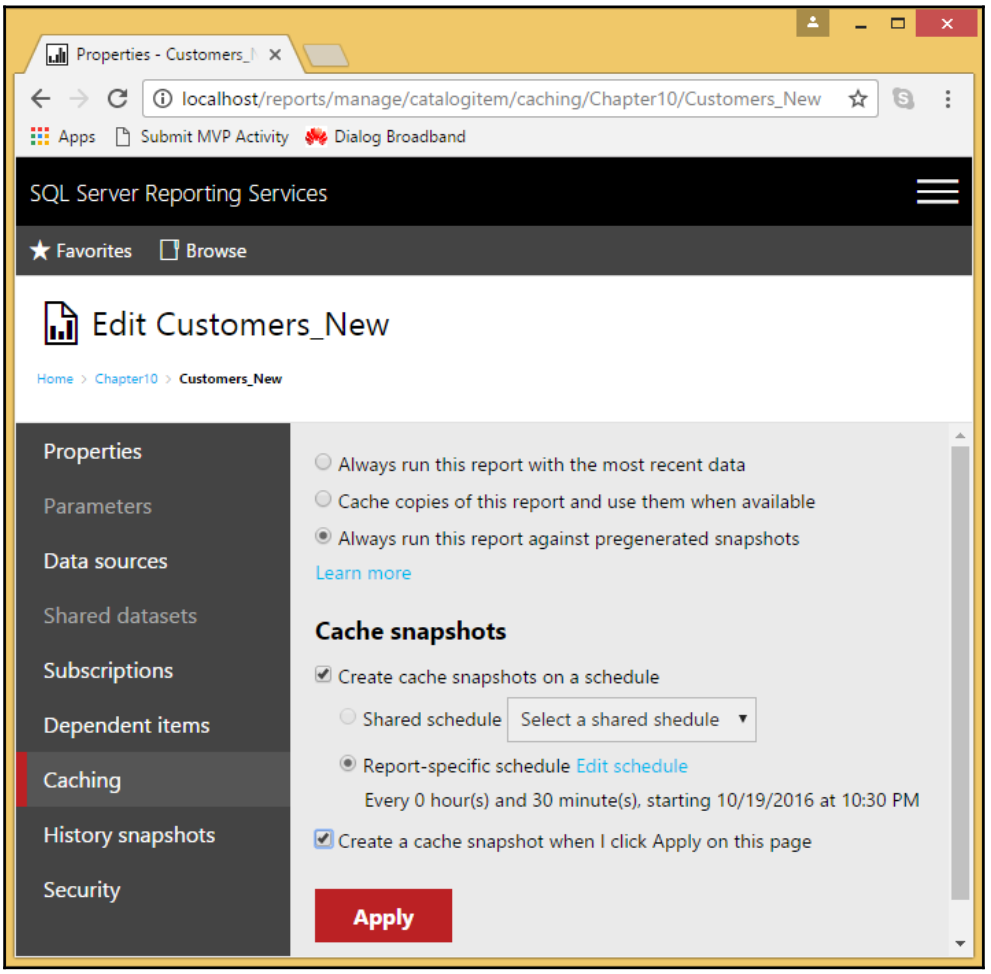


Figure 10.26

8. Click on **History snapshots** for seeing snapshots. You should see one link, which is for the snapshot created immediately. If you check after 30 minutes, you should see two links, one for the initial one and the second is for the snapshot created after 30 minutes:

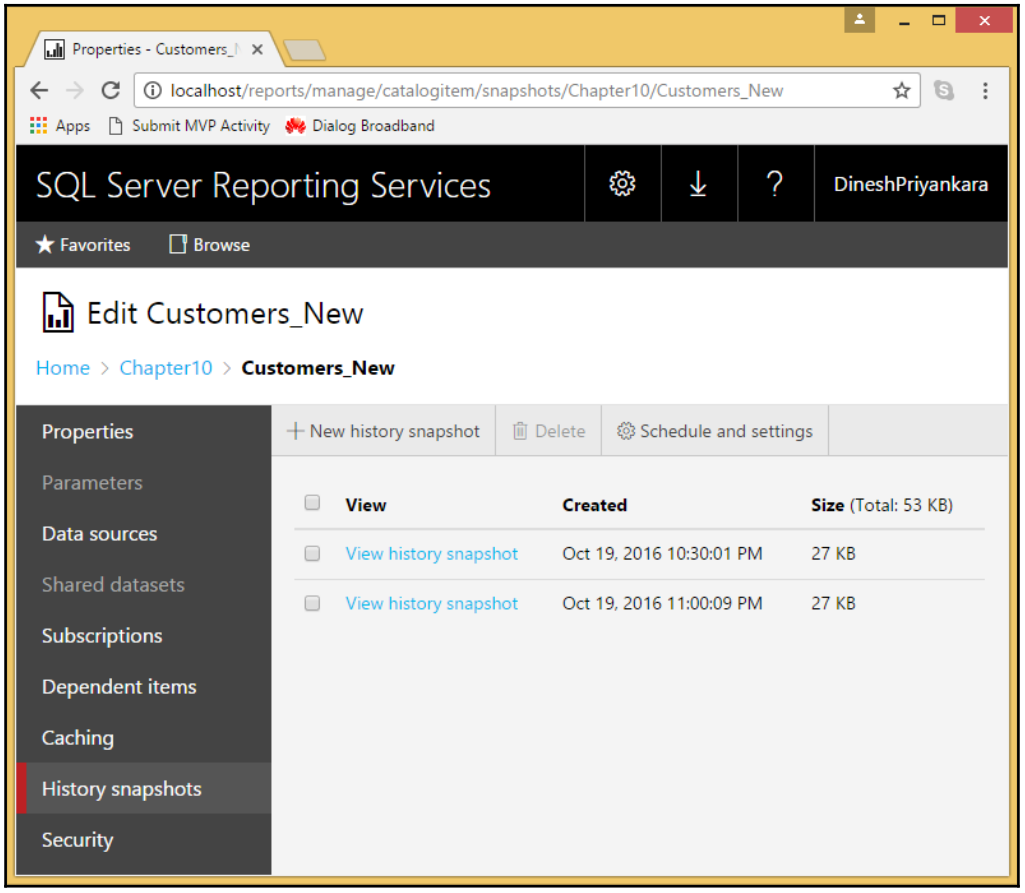


Figure 10.27

9. Right-click on one of the link and open the snapshot using a new tab. Note the date and time printed on the header. That indicates time that report has generated.
10. Return the first tab and note the created time of the last snapshot. Click on `Chapter10` link in the bread-crumb-trailer for opening the folder.

11. Click on `Customer_New` report for opening it. You will see that the date and time it shows is same as the date and time of the last snapshot. This proves that when the report is requested on-demand, it uses the last snapshot instead of regenerating the report.

You can create an additional history snapshot or create a history snapshot without using any option related to caching. If need, here is the steps for it:

1. Go to **History snapshots** and click on **+ New history snapshot** button (See Figure 10.28).
2. You should see an additional snapshot as shown here:

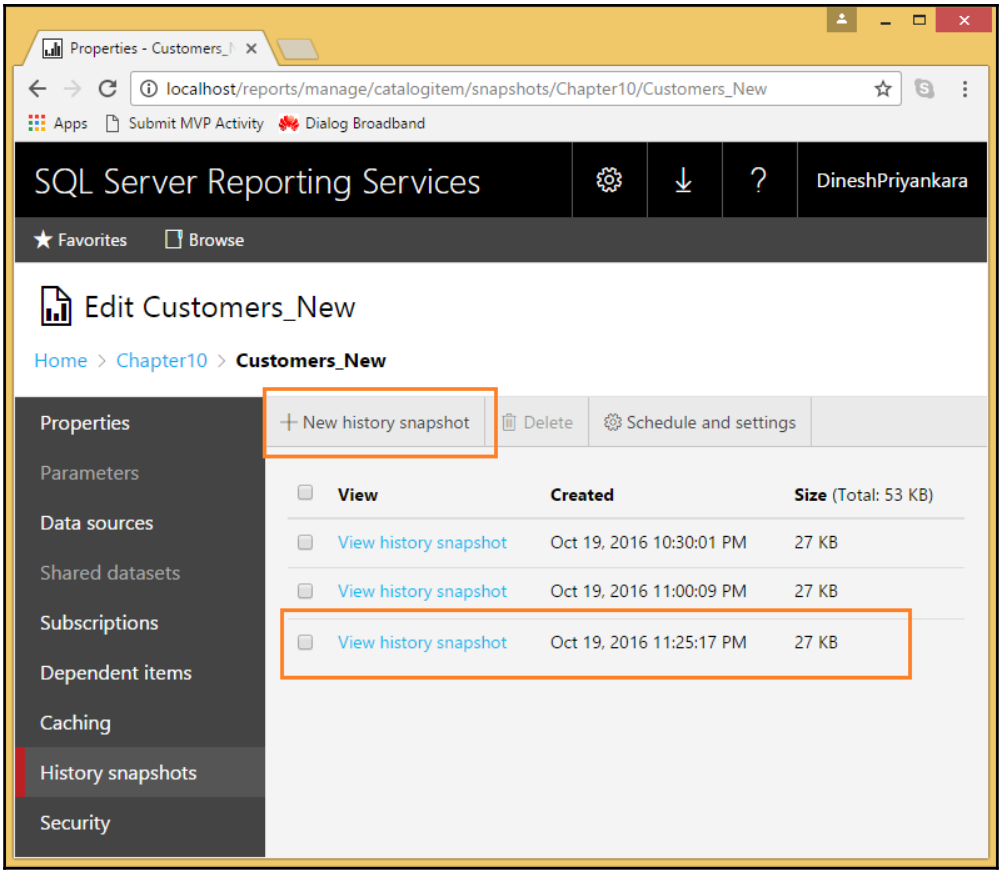


Figure 10.28

3. The retention period of snapshots created are based on the settings in **Schedule and settings** in the **History snapshots** (See Figure 10.28).
4. If you to check or change, click on **Schedule and settings** for opening properties. Note the default option selected that is **Use the system default settings**. Select the required option and click **Apply** button:

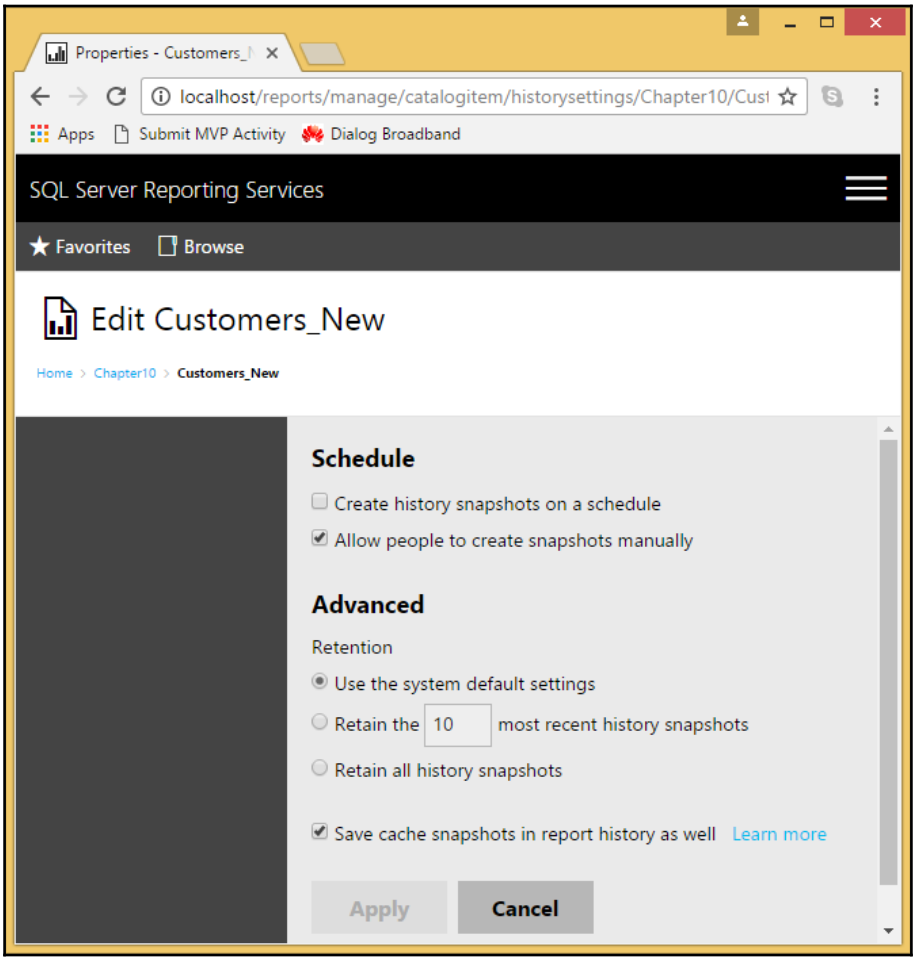


Figure 10.29

However, if you need to delete any snapshot created, you can simply select them and delete in History snapshots.

How it works…

The snapshots created are stored in the main Reporting Services database. Unlike cache copies, this has no expiry settings, hence they are remained until you delete or get deleted automatically based on retention settings.

You noticed that the first option of the retention section is **Use the system default settings**. If you need to see or change the default value of it, open **Site settings** of Reporting Services and change as you want:

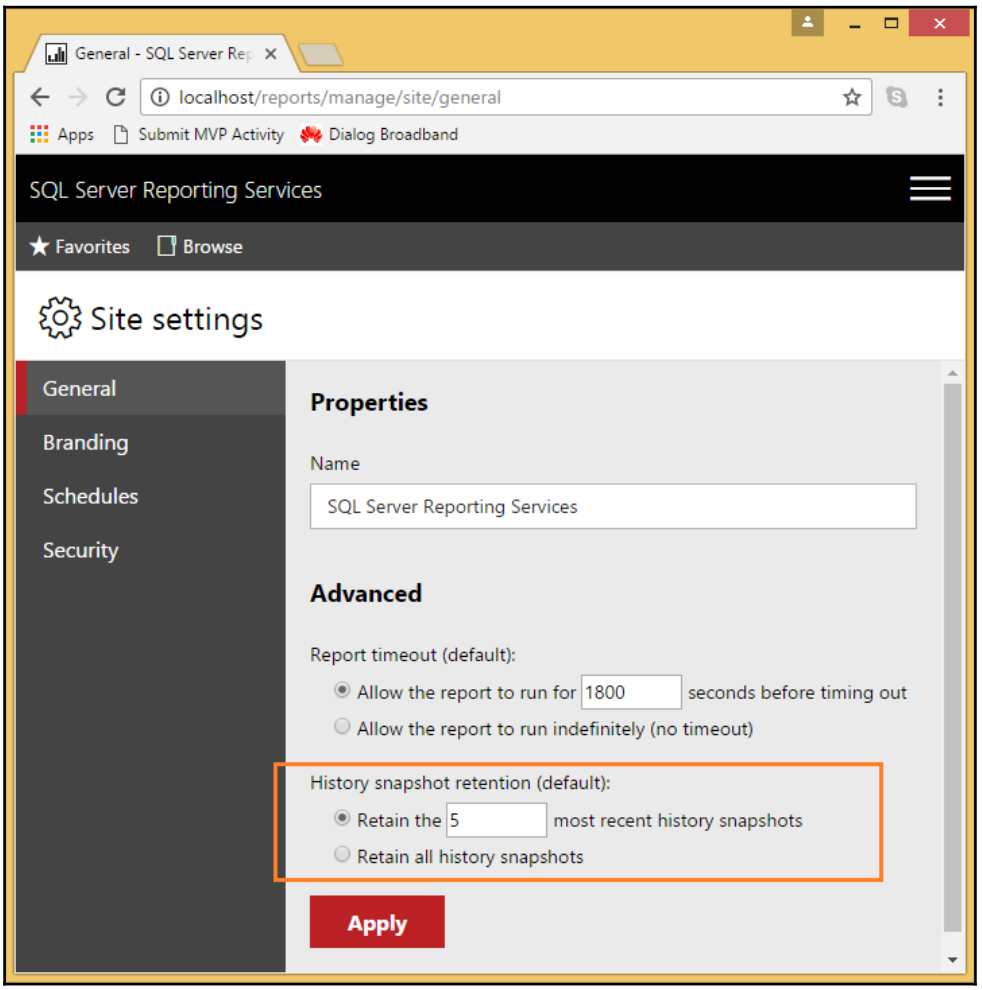


Figure 10.30

If you change the value as `5`, then Reporting Services maintain five snapshots for the report and if another is created, the oldest one is dropped for making sure that only five snapshots are maintained.

Backing up Reporting Services

Things can go wrong unnoticed. An unexpected hardware failure or software failure can damage Reporting Services environment, making it unavailable for users. Database Administrator or Reporting Services Administrator must work on it for fixing or recovering the environment and bring Reporting Services up and running as quickly as possible.

How do we recover if something has damaged the Reporting Services? In order to recover, you need to make sure that you have copies of all required components such as `.rdl` files, databases, modified configuration files, and encryption keys if used. Do we need a strategy for keeping these components?

Let us see what copies of components we need to keep and how they can easily use for recovering.

Getting ready

Deciding the recovery plan is based on two factors called RTO and RPO. **Recovery Time Objective** (**RTO**) refers how long can recover take and RPO; how much data is it acceptable to lost, must be discussed with business users and decided.

In simple terms, RTO says that environment should be up and running within the given time-frame. RPO says that I do not mind re-doing work done in last n number of hours. Based on these two values, we plan our strategy, generally achieve via backup and restore strategy.

What should be backed up in Reporting Services environment?

It is recommended to back up following components:

- `reportserver` database
- `reportservertempdb` database
- Encryption keys
- Configuration files
- Reporting Services data files such as `.rdl`, `.rds` files

There is a setting called **Recovery Model** in SQL Server database that decides how system should support on recovery and how operations should be maintained. There are three Recovery Models: Simple, Full and Bulk Logged. Practically, either Simple or Full is used for databases and Bulk Logged is used when performing bulk operations.

Simple Recovery Model makes the database simple, truncating transaction log of the database and Full Recovery Model maintains transactions for recovery purposes. It is recommended to set Simple Recovery Model for `reportservertempdb` database and Full Recovery Model for `reportserver` database.

Let us see how can we backup each and every component listed above.

How to do it…

Starting with database, you can use the standard technique used for backing up SQL Server databases for backing up Reporting Services databases as well. Use the following steps for performing the backup operation:

1. Open SQL Server Management Studio and connect with the instance related to Reporting Services.
2. Expand **Databases** node and find **Reportserver** database. Right-click on it, goto **Tasks** and select the **Back Up..** menu as shown in the following screenshot:

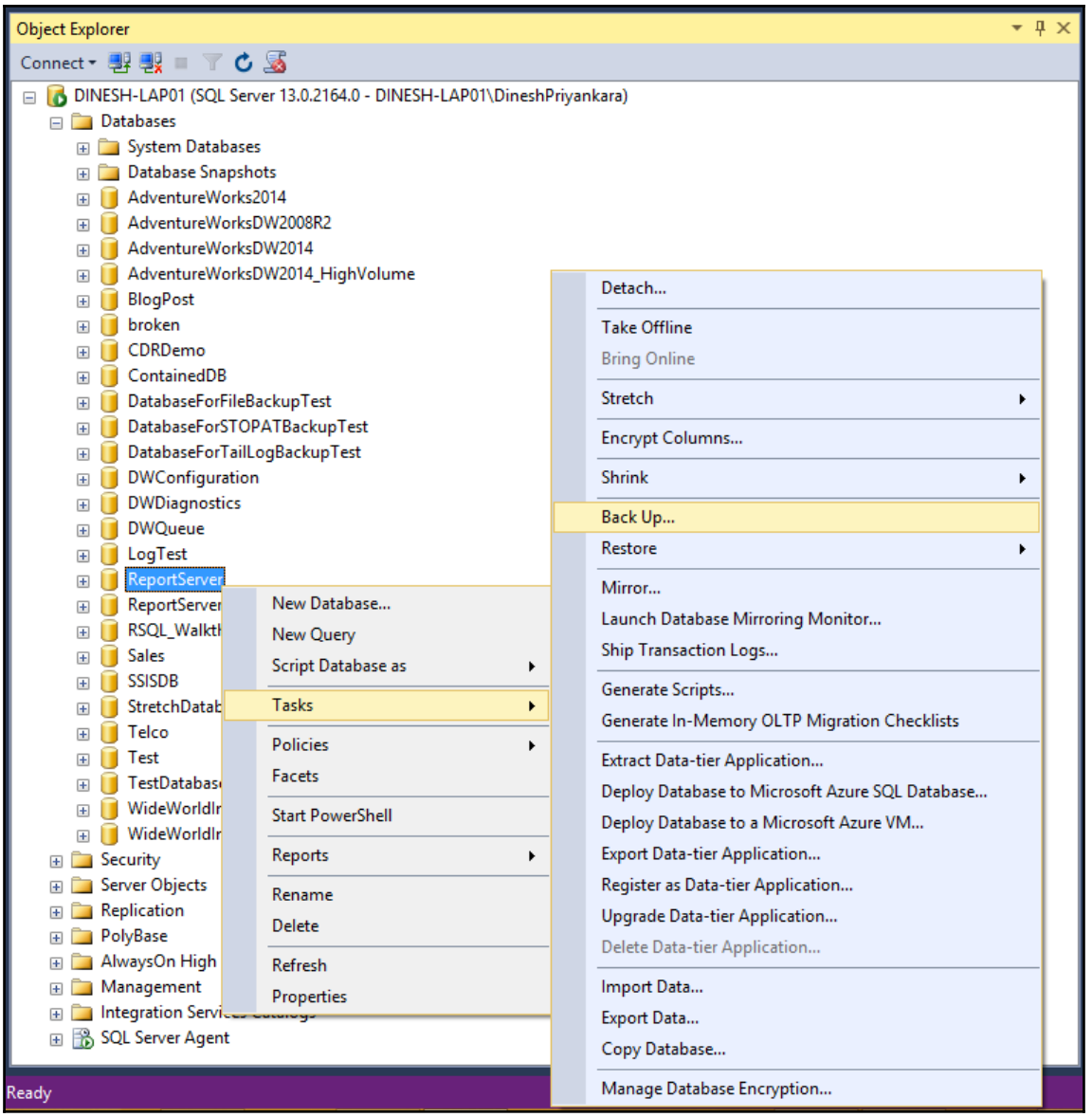

Object Explorer
Connect
DINESH-LAP01 (SQL Server 13.0.2164.0 - DINESH-LAP01\DineshPriyankara)
Databases
System Databases
Database Snapshots
AdventureWorks2014
AdventureWorksDW2008R2
AdventureWorksDW2014
AdventureWorksDW2014_HighVolume
BlogPost
broken
CDRDemo
ContainedDB
DatabaseForFileBackupTest
DatabaseForSTOPATBackupTest
DatabaseForTailLogBackupTest
DWConfiguration
DWDiagnostics
DWQueue
LogTest
ReportServer
ReportServer
RSQL_Walkth
Sales
SSISDB
StretchDatab
Telco
Test
TestDatabas
WideWorldIr
WideWorldIr
Security
Server Objects
Replication
PolyBase
AlwaysOn High
Management
Integration Servi
SQL Server Agent
New Database...
New Query
Script Database as
Tasks
Policies
Facets
Start PowerShell
Reports
Rename
Delete
Refresh
Properties
Detach...
Take Offline
Bring Online
Stretch
Encrypt Columns...
Shrink
Back Up...
Restore
Mirror...
Launch Database Mirroring Monitor...
Ship Transaction Logs...
Generate Scripts...
Generate In-Memory OLTP Migration Checklists
Extract Data-tier Application...
Deploy Database to Microsoft Azure SQL Database...
Deploy Database to a Microsoft Azure VM...
Export Data-tier Application...
Register as Data-tier Application...
Upgrade Data-tier Application...
Delete Data-tier Application...
Import Data...
Export Data...
Copy Database...
Manage Database Encryption...
Ready

Figure 10.31

3. Make sure you have changed the path of the backup device as you have planned. You can remove the device suggested automatically by clicking **Remove** button and add a new one by clicking **Add** button. Click **OK** to take the backup:

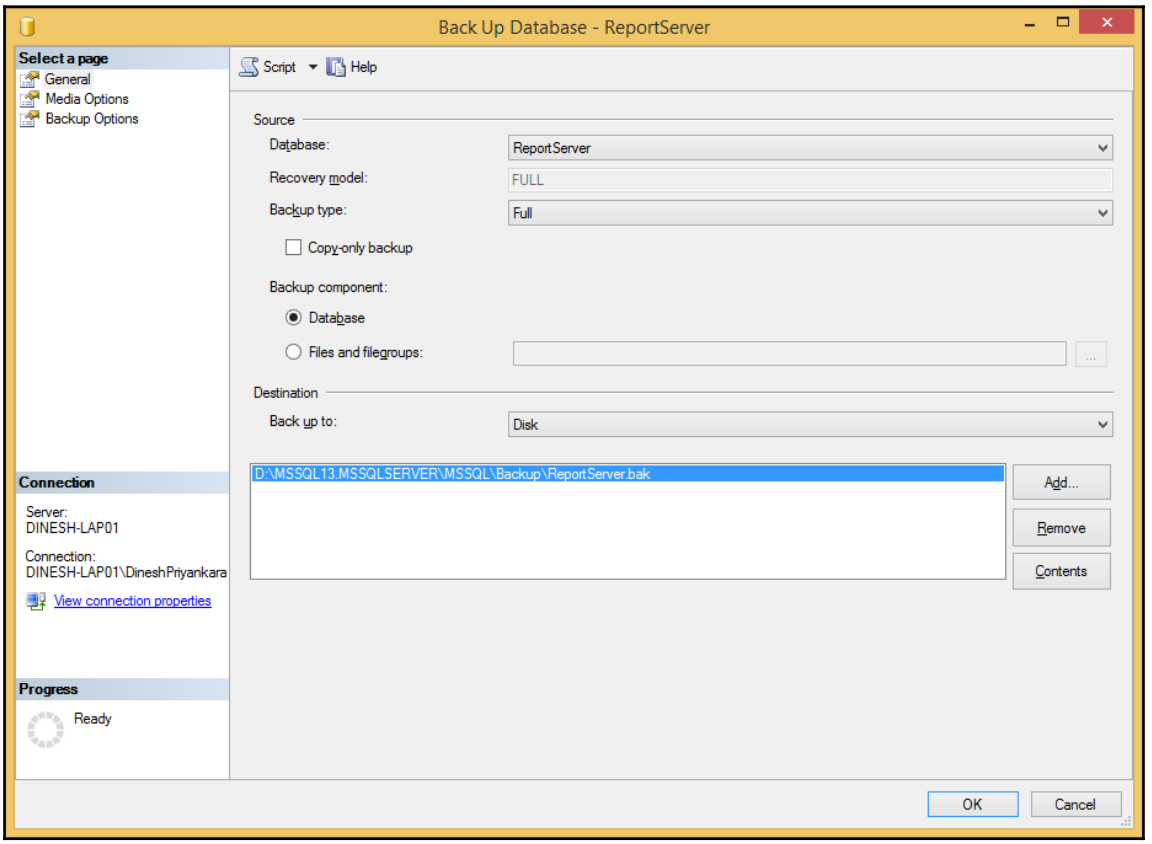


Figure 10.32

It is recommended to automate this operation and schedule as per the requirements. You can use **SQL Server Agent** for creating a job for backing up with a schedule.

You can perform the same step for backing up `reportservertempdb` database. However, you can get the `reportservertempdb` built anytime but the backup can be used for making it available.

For restoring both databases, you can use the standard restore operation that can perform with Management Studio.

Now let us see how we can get the encryption keys backed up:

1. Open SQL Server Reporting Services Configuration Manager and connect with your Reporting Services instance.
2. Click on **Encryption Keys** in the left pane for opening **Encryption Keys** section (See Figure 10.33).
3. Click on **Backup** button for opening **Backup Encryption Key** window. Set the **File Location** for the key and set a **Password** for protecting it:

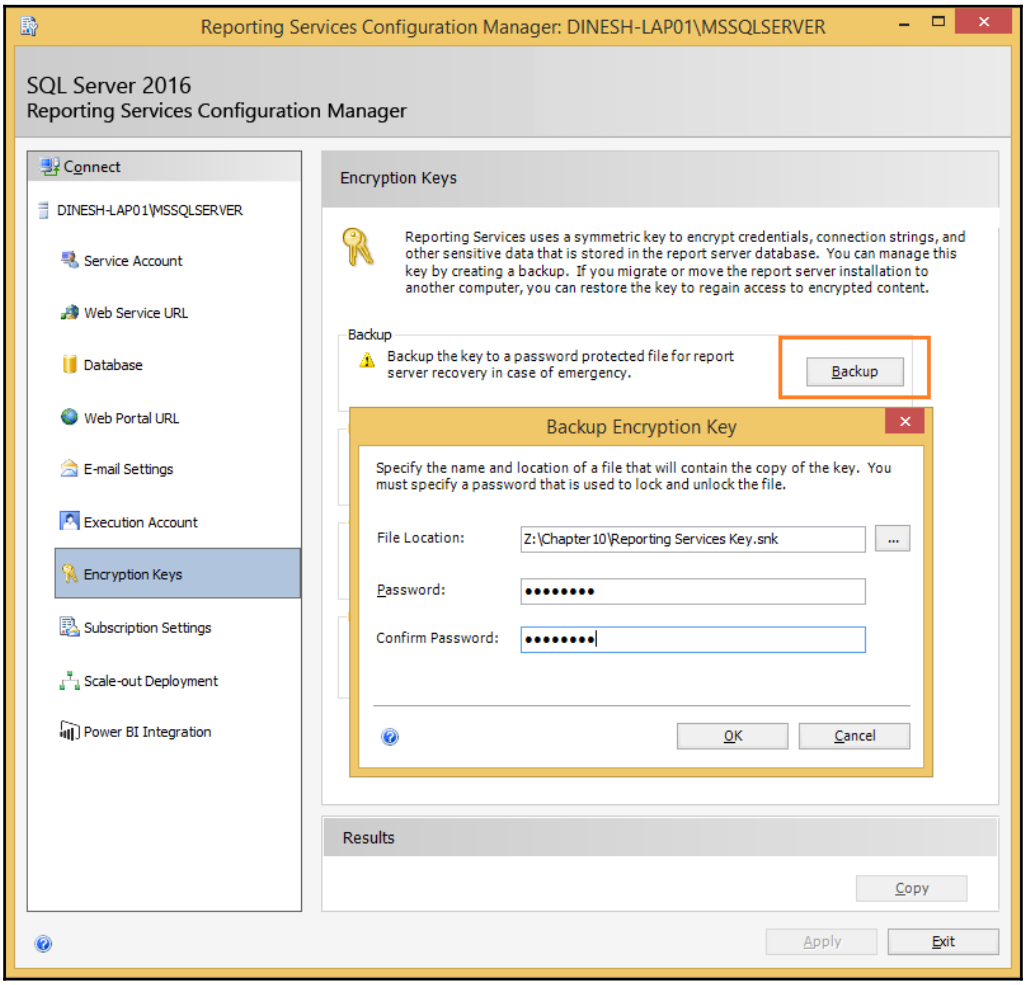


Figure 10.33

4. Click on to make a backup of the **Encryption Key**. Note that you can use the same window for restoring when required.

There are set of configuration files related Reporting Services. If you have changed them, for an example, you have enabled additional rendering types by changing the configuration files and then it is better to keep a copy of them as a backup. In case of a disaster, you can use the backup files for setting changes instead of changing new files. There are two configuration files to be backed up:

1. Open Reporting Services Report Server installation path folder. The default installation path is `C:\Program Files\Microsoft SQL Server\MSRS13.MSSQLSERVER\Reporting Services\ReportServer`
2. Take a copy of `rsreportserver.config` file:

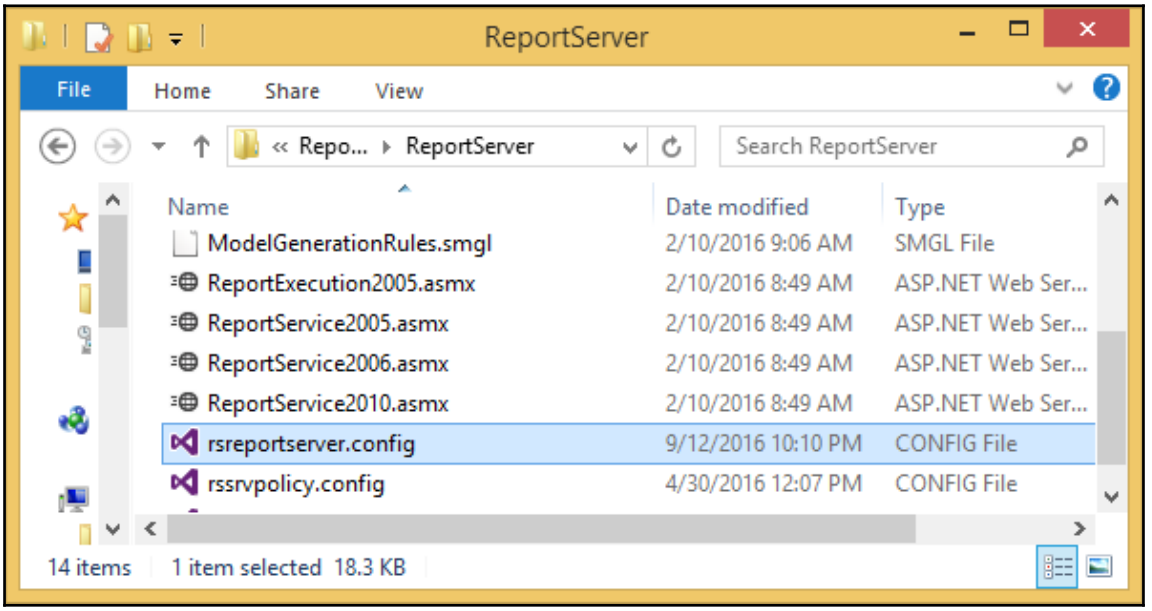


Figure 10.34

3. Open Reporting Services Report Server bin folder and take a copy of `ReportingServicesService.exe.config` file.

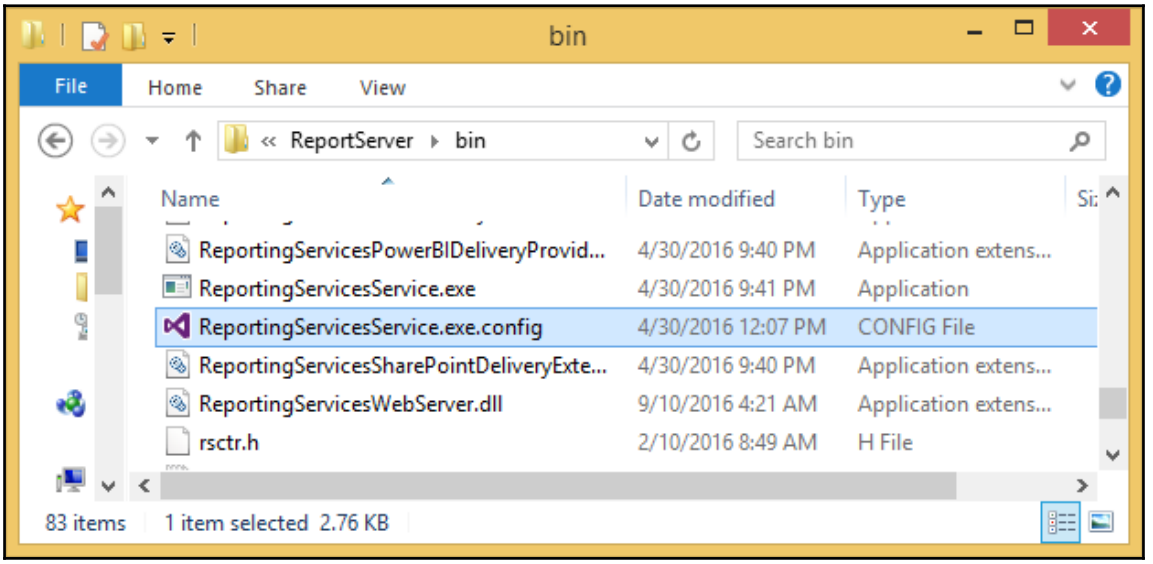


Figure 10.35

If you have taken a copy of `reportserver` database and you maintain a Reporting Services project for all reports using **SQL Server Data Tools**, you do not need to backup `.rdl` files separately. However, if you still need copies of rdl from the server, you can download them but Web Portal does not provide a way for downloading all files together unless you write your own code for that. For downloading individual files, follow the steps mentioned here:

1. Open Web Portal and go into a report folder that contains `.rdl` files need to download.
2. Click on the ellipsis button set with the report and click on **Manage** menu item for opening properties of the report.

3. Click on the **Download** button given with the header for downloading the report:

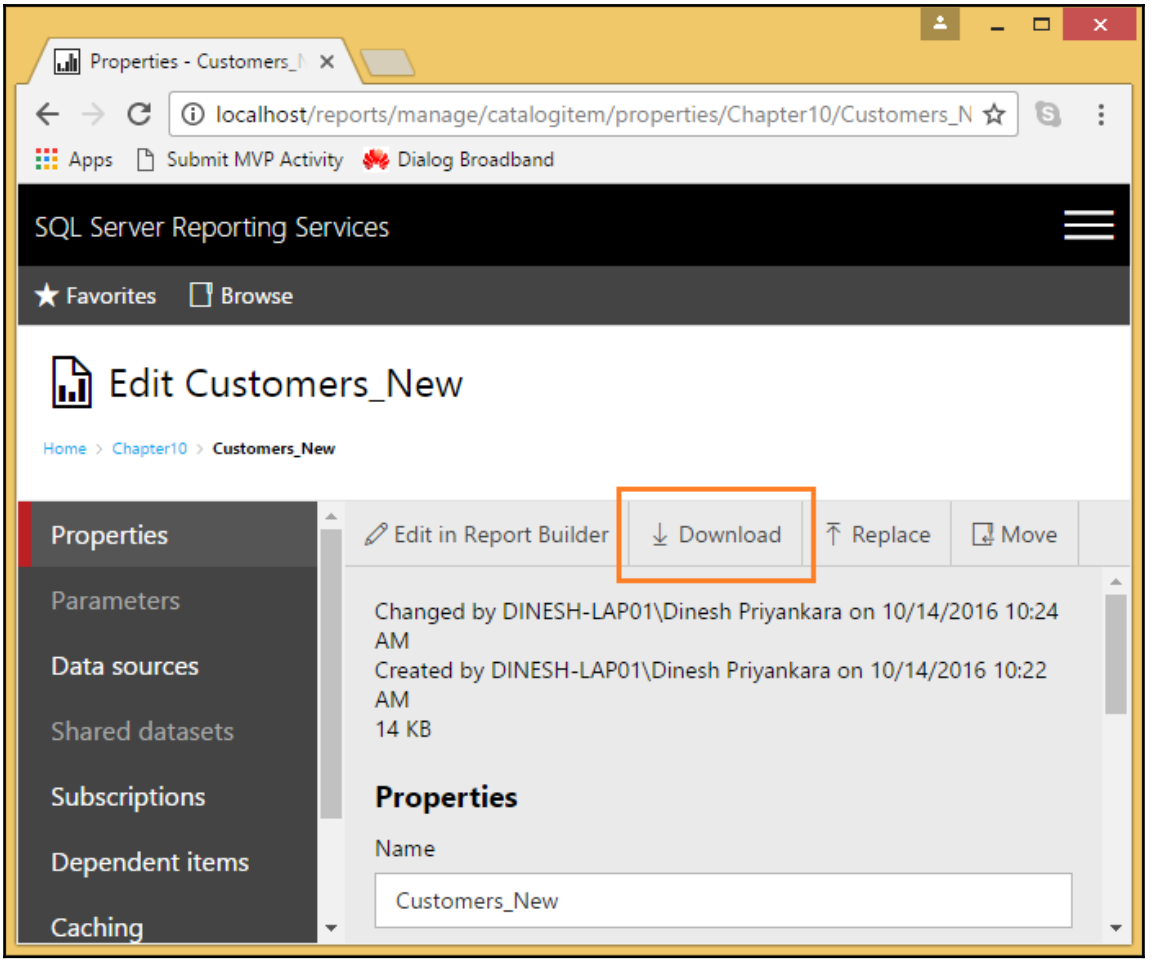


Figure 10.36

How it works…

Having backups of all Reporting Services components is recommended and you should do it. They should be placed in a secured location and tested regularly because you may encounter random issues when trying to restore at a disaster.

There's more…

There are couple of the other files you may backup if you need. They are `web.config` files of web applications such as Report Server and Web Portal. In addition to that, you may backup `machine.config` file for ASP.NET as well.

Monitoring Reporting Services

Although the deployed reporting solution works fine, we need to understand the user experience with deployed reports for improving the performance of the solution. Not only that, we need to make sure that the solution deployed is healthy and it has enough resources for processing requests received from end users.

For this, we need to keep on monitoring Reporting Services processes and activities performed by users. There can be a situation where user suddenly experiences slowness of a report or a crash with a report that has been working for long time without any issue. We need to make sure that information for all of these is captured and available for analysis when required.

Getting ready

There are no built-in tools or components for monitoring Reporting Services given with Reporting Services. We can use tools like Task Manager, Event Viewer or Performance Counter for monitoring and troubleshooting Reporting Services but remember, they are not specifically built for Reporting Services. You can use Task Manager to monitor the usage of server resources such as CPU and memory. You can use Event Viewer for checking errors for troubleshooting errors related to Reporting Services. You can use Performance Monitor to analyze the performance of Reporting Services using added Counters of objects such as MSRS 2011 Web Service, MSRS 2011 Windows Service and ReportServer:Service.

With this recipe, focus on something we have within Reporting Servers that is ExecutionLog view. The ExecutionLog view allows us to see information on published reports and execution details of them. This is very useful because this shows us how reports are used and that allows us to work on problematic or need-to-improve reports, prioritizing them.

You need to enable before using it. Here is the way of enabling it for Native Mode Report Server:

1. Open SQL Server Management Studio and connect to Reporting Services.

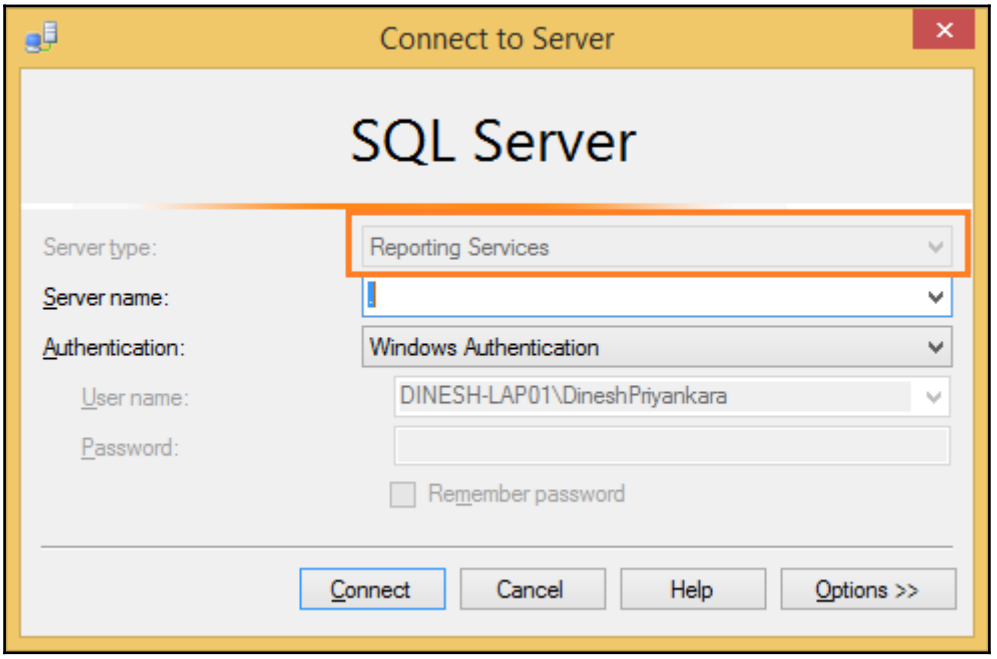


Figure 10.37

2. Right-click on the instance name and click on **Properties** in the context menu.
3. Click on **Logging** in the left pane for opening settings related to logging (See Figure 10.38).
4. Make sure **Enable report execution logging** is selected (See Figure 10.38).
5. Change the value of **Remove log entries older than this number of days** as `90`. This makes sure that information logged is kept for three months (See Figure 10.38).

6. Click on **OK** to save settings:

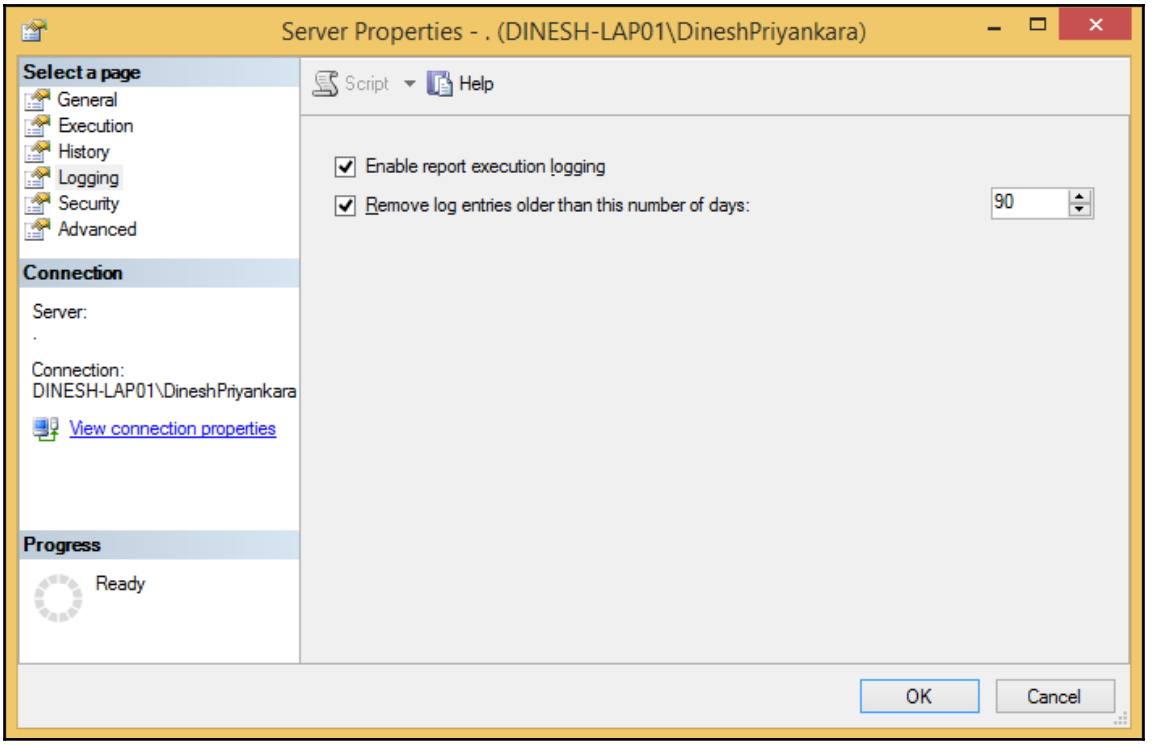


Figure 10.38

Once this is enabled, we can get information from a view named `LogExecution3`.

Note that there are two more views named `LogExecution` and `LogExecution2` that are older versions of `LogExecution3`. It is recommended to use `LogExecution3` because it has friendly names on columns and addition info as well.

Let us write some queries to get information from this view.

How to do it…

Start querying `LogExecution3` with the following steps:

1. Open SQL Server Management Studio and connect with the instance related to Reporting Services. Make sure you connect with Database engine, not Reporting Services.
2. Expand the **Object Explorer** and navigate via **Databases** node. Select **Reportserver** database and click on **New Query** for opening a new query window.
3. Type the following **SELECT** query:

```
SELECT * FROM dbo.ExecutionLog3
```

4. Click on **Execute** button to run the query:

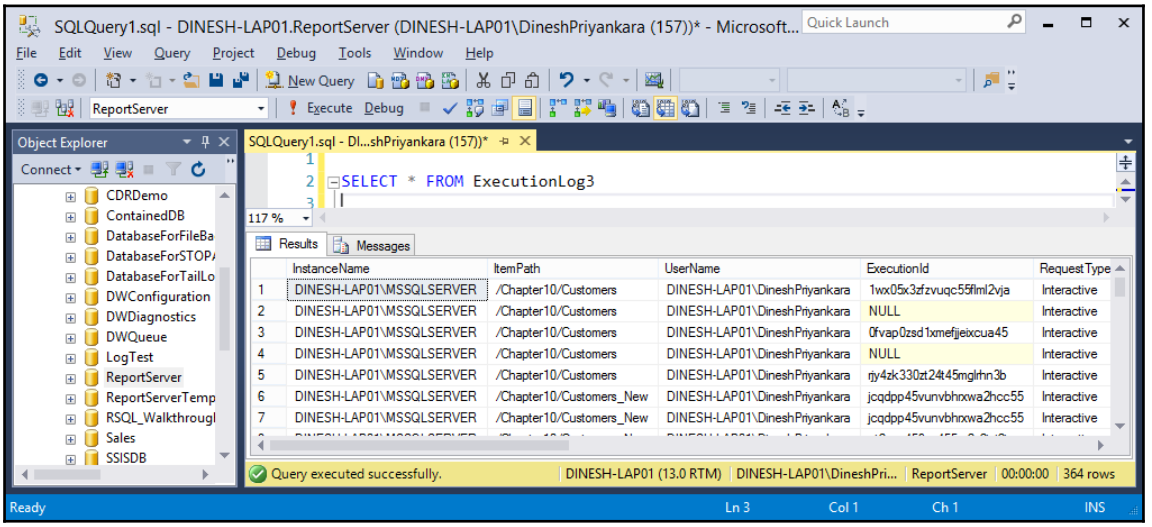


Figure 10.39

As you see, this view returns nineteen columns that give enough of details to understand user experience with Reporting Services. Let's understand the columns:

Column name	Description
`InstanceName`	Name of the Reporting Services instance
`ItemPath`	Report name including the folder path

`UserName`	User executed the report
`ExecutionId`	Unique execution id
`RequestType`	How the request is made, whether **Interactive** or **Subscription**
`Format`	Rendering type requested such as `RPL`, `PDF`, `PPTX`, `EXCELOPENXML`. Note that you will see `NULL` for this if request is made from a subscription of a schedule.
`Parameters`	Parameters along with values passed. Note that parameters are separated using ampersand.
`ItemAction`	The action performed for the request. Generally, it is **Render** but there are actions like Sort, Execute, `BookMarkNavigation`, `DocumentNavigation`, `GetDocumentMap`, `Findstring`, `RenderEdit`. For example, action is set as **Execute** when the cached report is getting refreshed.
`TimeStart`	Start time of the report processing
`TimeEnd`	End time of the report processing
`TimeDataRetrieval`	Time spent on data retrieval in milliseconds
`TimeProcessing`	Time spent on report processing in milliseconds
`TimeRendering`	Time spent on report rendering in milliseconds
`Source`	Source of the report content (or execution). When the report is loaded initially (from the data source), it indicates as Live. Other than that, possible values are Cache, Snapshot, History, AdHoc, Session, Rdce. For example, once the report is open and user requests on Excel import, then the source become **Session**.
`Status`	Whether the report request is success or not
`ByteCount`	Size of the rendered report in bytes.
`RowCount`	Number of records returned
`AdditionalInfo`	Additional info varies from execution to execution, which returns as an XML.

Let's write some queries to get some information using this view:

1. Write following TSQL code for getting most frequently accessed reports. This helps you to understand what reports should be prioritized and important:

```
SELECT TOP (10)
  ItemPath As ReportName
  , RequestType As RequestMode
  , Format As RenderingFormat
  , COUNT(*) NumberOfRequests
FROM dbo.ExecutionLog3
GROUP BY ItemPath, RequestType, Format
ORDER BY COUNT(*) DESC
```

2. Write the following code to see whether you have any failed reports for today:

```
SELECT ItemPath As ReportName
  , RequestType As RequestMode
  , Format As RenderingFormat
  , Parameters
  , UserName RequestedBy
FROM dbo.ExecutionLog3
WHERE Status != 'rsSuccess'
  AND Convert(date, TimeStart) = CONVERT(date, getDate())
```

3. Type following code for getting reports that have spent more time than yesterday:

```
SELECT today.ItemPath As ReportName
  , today.RequestType As RequestMode
  , today.TimeDataRetrieval + TimeProcessing + TimeRendering As
  TotalTime
FROM dbo.ExecutionLog3 today
  INNER JOIN (SELECT ItemPath As ReportName
        , RequestType As RequestMode
        , TimeDataRetrieval + TimeProcessing + TimeRendering As
  TotalTime
FROM dbo.ExecutionLog3
WHERE Convert(date, TimeStart) = CONVERT(date, DateAdd(d, -1,
   getDate()))
  ) yesterday
ON today.ItemPath = yesterday.ReportName
AND today.RequestType = yesterday.RequestMode
AND (today.TimeDataRetrieval + today.TimeProcessing +
       today.TimeRendering) > yesterday.TotalTime
WHERE Convert(date, today.TimeStart) = CONVERT(date, getDate())
```

As you see, you can find out many info related to report execution using `LogExecution3` and it can be used for troubleshooting as well.

How it works…

`LogExecution3` view holds information captured based on the retention period set. Since we set it as 90 days, it keeps information for 90 days and deletes information older than 90 days. The job for deletion gets executed at 2:00am every day.

Note that you cannot control the recording of entries by limiting with filters. It logs everything without filtering.

There's more…

You can create set of reports for showing these information, either with tabular data regions or graphical data regions using this view. It will be helpful for all users, mainly administrators to monitor and troubleshoot Reporting Services.

In addition to that, you can use other channels such as Performance Monitor and Windows Event viewer for getting more info and troubleshooting.

11
Securing Reports in Reporting Services

Security is a critical aspect in administrating your reporting server. In this chapter we'll learn about SSRS security through the following recipes:

- Authenticating users for Reporting Services
- Becoming a Report Portal System Administrator
- Authorizing users for accessing reports via SSRS roles
- Creating new roles in Reporting Services
- Editing existing roles in Reporting Services
- Filtering data for the current user
- Handling security on shared data sources
- Securing the channel using certificates

Introduction

In this chapter, we'll discuss the various layers of security built into Reporting Services. We'll begin with a look at how SSRS authenticates a user, followed by recipes for determining what a user can and cannot do once they are on the server. In addition, we'll cover security on data sources, as well as how to secure your SSRS channel with certificates.

Authenticating users for Reporting Services

Authentication is the mechanism by which SSRS will establish the identity of a user. It doesn't identify the user, but rather establishes the method by which a user will identify themselves to the Reporting Services server.

Getting ready

This recipe is more focused on determining what the authentication type is set to and how to understand what your choices are. You will need to have access to the server on which SSRS is running, with administrative rights.

How to do it…

1. Open **File Explorer** in Windows.
2. Navigate to the folder where your instance of Reporting Services is installed. This will typically be in the format of `Program Files\Microsoft SQL Server\instance name\Reporting Services\ReportServer`. (Note there is a space between Reporting Services, but no space with ReportServer.)
3. In a default installation of SQL Server 2016, this would be `C:\Program Files\Microsoft SQL Server\MSRS13.MSSQLSERVER\Reporting Services\ReportServer`.
4. Open the `rsreportserver.config` file in a text editor.
5. Locate the section with the `AuthenticationTypes` tag. This will hold the authentication type for your server. An example is shown here:

```
<AuthenticationTypes>
  <RSWindowsNegotiate/>
</AuthenticationTypes>
```

How it works…

There are several valid authentication types that may appear here. Each has its own set of rules that accompany it, which are as follows:

- **RSWindowsNegotiate**: With this mode, SSRS will first attempt to use Kerberos to authenticate the end user. If Active Directory cannot grant access via Kerberos, it then falls back and attempts to use NTLM authentication. NTLM is a form of Windows authentication used when Active Directory is not present or Kerberos has not been configured. In most corporate environments, this is likely to be the default.
- **RSWindowsNTLM**: This mode is for situations where Kerberos has not been configured. It relies strictly on Windows-based claims authentication. In some situations, a user may have to manually authenticate with their credentials, depending upon the configuration of their workstation. This mode will generally be used only in very small organizations.
- **RSWindowsKerberos**: With this authentication type, SSRS will only recognize Kerberos authentication. It does have some drawbacks, in that many browsers do not support it. Indeed, even Microsoft's own browsers, Microsoft Edge and Internet Explorer, can only connect via Kerberos when RSWindowsNegotiate is set as the authentication type. As such, this mode is rarely used.
- **RSWindowsBasic**: In basic mode, authentication is done by passing in credentials via the HTTP URL using SSL. This mode is rare, and is generally not in use today.
- **Custom**: With custom mode, all authentication requests are redirected to the ASP.NET server, which uses Forms authentication. This scenario is used when you wish to open up access to your SSRS server to individuals who may not be present in your Active Directory. An example may be making reports available to your vendors. Using custom authentication requires the development of custom ASP.NET code.

Of this list, the two most common choices are RSWindowsNegotiate and RSWindowsNTLM. RSWindowsKerberos is included as part of the RSWindowsNegotiate authentication mode. RSWindowsBasic is outdated, designed for earlier versions of SSRS. Finally, custom mode would only be employed if your environment uses a custom developed security mechanism.

If you have Kerberos installed and configured in your environment, then RSWindowsNegotiate would be your choice. Without it, you will need to use the NTLM mode of authentication.

Becoming a Report Portal System Administrator

In order to perform the remaining recipes, you will need to be a system administrator on the report portal. On a new installation, only the administrators of the computer upon which SSRS is running will have rights to the server. Thus, the first thing you'll want to do is add your Windows ID (or a Windows Active Directory Group you are part of) to the list of administrators.

Once you've done that, you'll then need to give yourself permissions to the Home folder. Let's see how.

Getting ready

To perform this recipe, you will need to be logged into the computer upon which SQL Server Reporting Services is running (remote desktop is fine), and your ID must be one of the administrators for that system.

How to do it…

1. Access the computer upon which SSRS is installed.
2. Right-click on your web browser (Internet Explorer is recommended) and select **Run as Administrator**.
3. Navigate to the report portal, typically `http://servername/reports` or `http://localhost/reports`.

4. Click on the **Gear** icon in the upper right-hand corner and select **Site Settings**.
5. Next, select the **Security** option on the left-hand side of the page.
6. Click **Add group or user**.
7. On the next page, enter your ID (or the Active Directory group you belong to) into the **Group or user** area.
8. Place a check mark beside **System Administrator**:

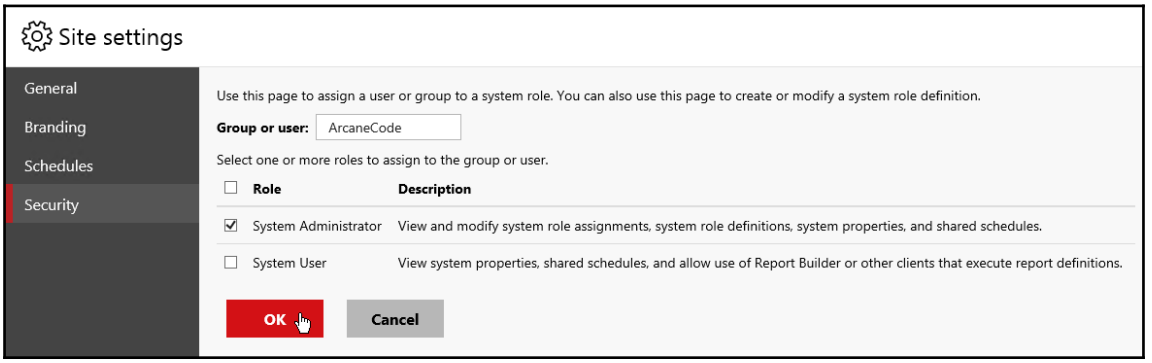


Figure 11.01

9. Click **OK**.
10. Return to the **Home** page of Report Portal.



You can get there by clicking on the title of your server (by default, **SQL Server Reporting Services** located in the top-left corner).

11. Click on **Manage folder** in the tool area at the top (see Figure 11.03).
12. Click on **Add group or user**.
13. Enter your user ID or group into the **Group or user** area.

14. Place a check mark beside **Content Manager:**

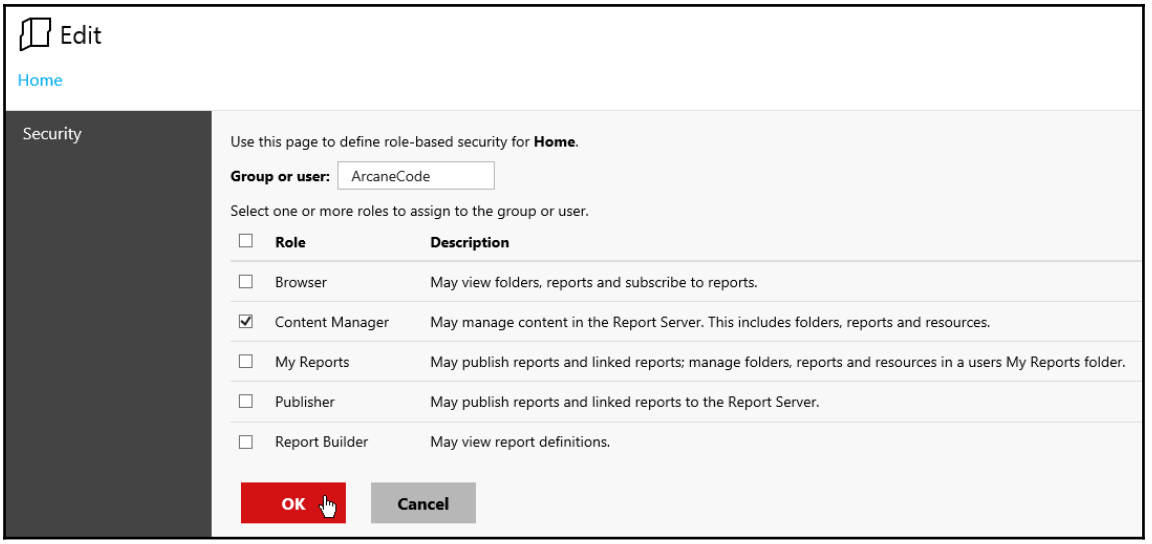


Figure 11.02

15. Click **OK**.
16. This will now grant you permissions to the server as an administrator. From here on, you should be able to access the Report Portal from your own workstation with administrative rights.

How it works…

In order to have full access to the Report Portal, your ID must have been given permissions in two areas. First, you'll need to have it placed into the System Administrators role of the server itself. Then, you'll need to have been granted Content Manager rights over the server. The way the Content Manager role works, along with the other types of roles, is explained in our next recipe.

Authorizing users for accessing reports via SSRS roles

SSRS grants users permissions to various reports and folders in the Report Portal through Roles. You can assign either an individual user, or more commonly, an Active Directory group, to a role. In this recipe, we'll explore the various roles built into SSRS and what rights are granted through those roles.

Getting ready

To grant a user permission, you will need to have administrative privileges for the Report Portal. If you've not done so, ensure you first follow the steps in the previous recipe, *Becoming a Report Portal System Administrator*, or else the steps in this recipe (as well as the remaining ones in the chapter) will not work.

How to do it…

1. Navigate to the **Home folder** in **Report Portal**.
2. Click the **Manage folder** link at the top:

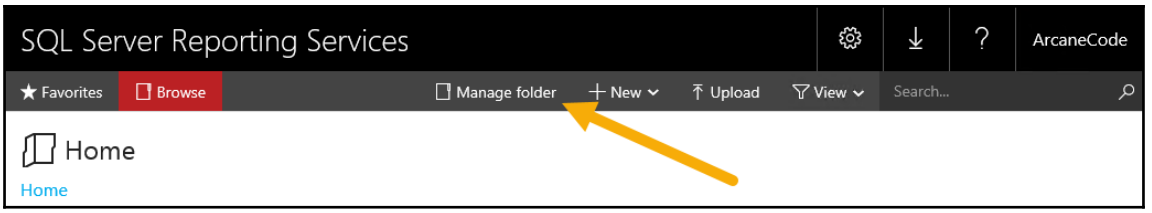


Figure 11.03

3. In the **Security** settings, click on **Add group or user**.
4. Enter the name of a user or Active Directory group at the top.

5. Place a check mark beside the role **Browser**:

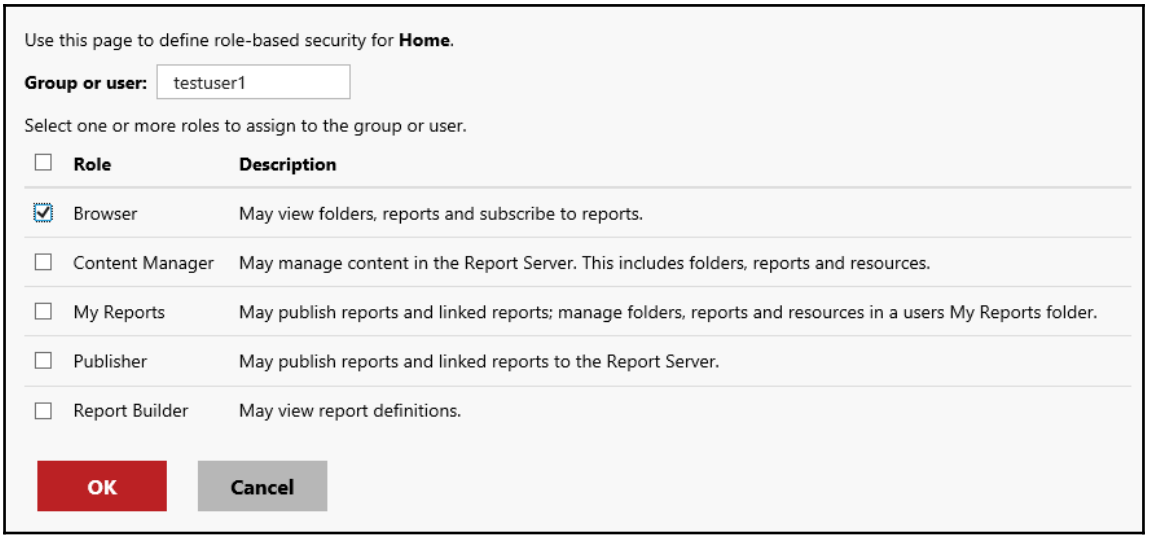


Figure 11.04

6. Click **OK** to save.
7. You should now see the user appearing in the list of users for the server:

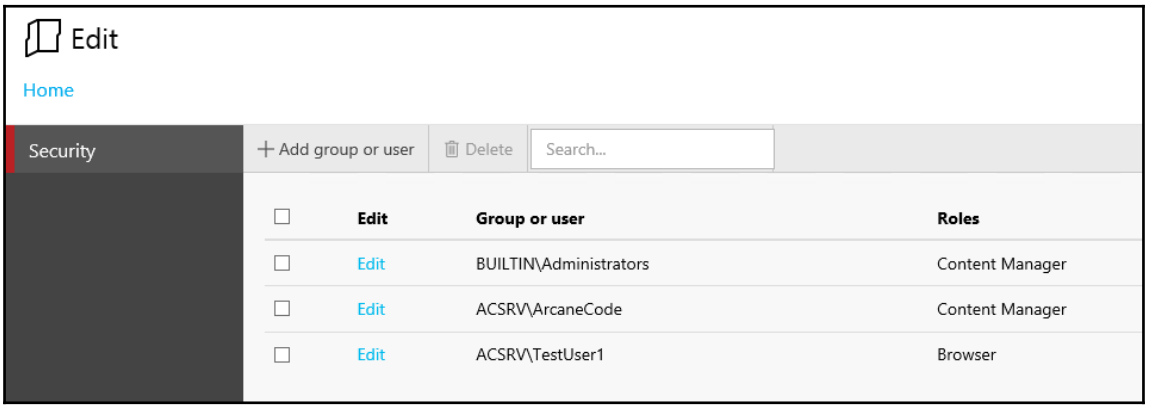


Figure 11.05

How it works…

In this recipe, we saw how to add a user to the SSRS server. Adding them was pretty simple, but there were a range of roles available. These roles control what a user may and may not do with reports. Please note that when we use the term reports, this includes report-related items such as data sources and shared datasets:

- The **Browser** role, which we used in this example, grants the user the ability to run reports, subscribe to them, and navigate through all the folders on the server. This is the most basic level of access.
- **Content Manager** is the opposite of the **Browser** role. Those who have the role of content manager have full permission over content on the server. In addition to being able to run reports, those in this role can also create new reports or modify existing ones. Furthermore, they may grant rights to other users, just as you did with this recipe.
- **My Reports** is a special role. SSRS has an option to generate a folder for each user of the Report Portal. This folder will appear with the name of **My Reports**. You, as the administrator, would need to grant the **My Reports** role to the user for them to use it. To activate the **My Reports** feature, use SQL Server Management Studio to connect to the Reporting Services server (note: not the Database Engine but the SSRS instance). Right-click on the server itself, and select **Server Properties**. There will be a checkbox to enable the **My Reports** folder for each user. Use this with caution. While you can turn it off, most users will not be happy having their own reports folder taken away from them.
- The **Publisher** role is for people who will need to create new reports, edit existing ones, and create folders to hold reports. This is an ideal role for those who will not only be authoring reports but organizing them within the Report Portal.
- The final role, **Report Builder**, is for people who will only be writing or updating reports. This gives the ability to work with reports, but unlike the Publisher role, they cannot create folders. This is the role you would generally grant to report writers in your organization.

There's more…

In this recipe, we granted a user rights on the **Home** level of the Report Portal. It is possible to grant different rights in individual folders. Indeed, this will be a common scenario in most organizations.

At the Home level, you would want all your users to be able to browse folders and run reports. However, you may want to grant some users the right to author their own reports for a folder devoted to their department.

To do this, you would first create a folder for their department; in this example, we'll use Accounting. Simply move to the **Accounting** folder, then click **Manage folder**. Once on the **Manage folder** page, select the **Security** link on the left.

By default, permissions from the Home level are carried to all folders. To override, click the **Customize security** link. Report Portal will warn you that you are about to override the permissions from the **Home** level; just click **OK**.

SSRS will now copy the permissions from the **Home folder** to this one. From here, you may grant new permissions as described in this recipe. There will also be an **Edit** link beside each user, with which you may modify an existing user or groups permissions.

Finally, if you don't wish a user to be able to see this folder at all, you can place a check mark beside their name and click the **Delete** link at the top. This will remove the user's ability to see this folder, but won't affect their ability to work with other folders in Report Builder.

Creating new roles in Reporting Services

As you saw in the previous recipe, SSRS comes with an array of predefined roles that should meet the needs of most of your users. You may, however, find situations in which the predefined roles are not quite right for your needs. SSRS allows you to create new roles, in which you may tune the permissions to suit your needs.

Getting ready

Adding new roles is not done through the Report Portal, as you might expect. Instead, it is handled within SQL Server Management Studio. You must have administrative rights to the reporting server to create new roles.

How to do it…

1. Open SQL Server Management Studio.
2. Connect to the instance of Reporting Services you wish to add new roles to.
3. Expand the Security branch.
4. Right-click on the **Roles** branch.
5. In the pop-up menu, select **New Role…**.
6. For the name, enter `Manage Subscriptions for All Users`.
7. In the description, enter `This role will allow the management of subscriptions for all users`.
8. In the list of tasks, find the one labeled **Manage all subscriptions** and place a check mark beside it:

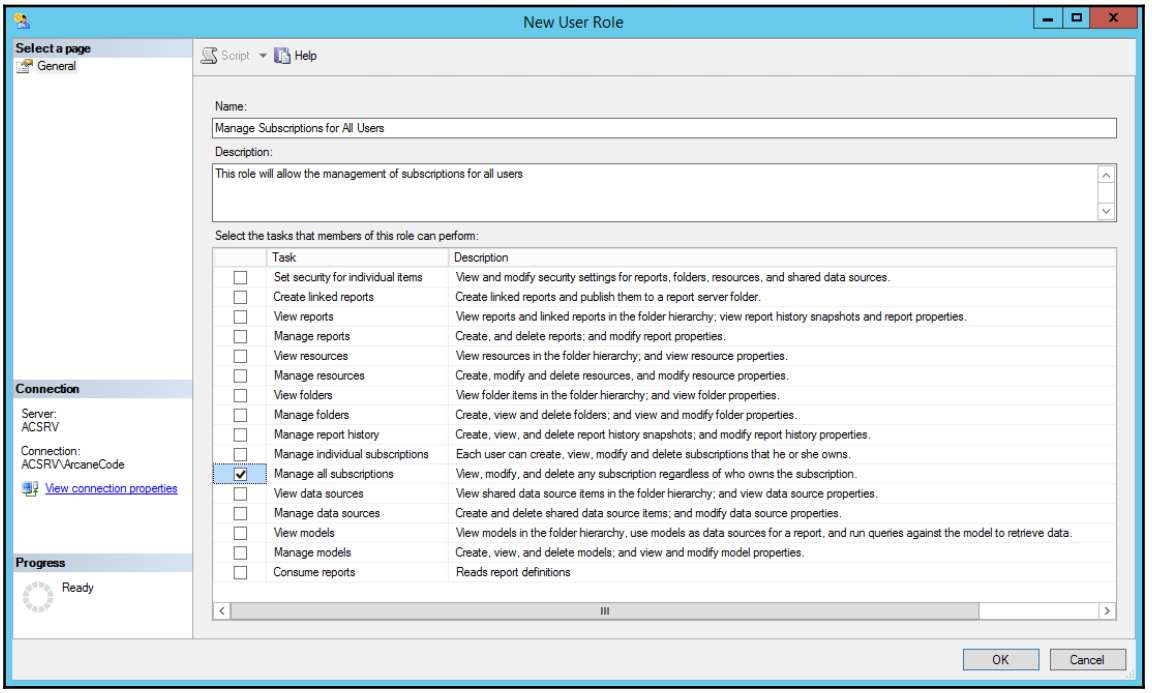


Figure 11.06

9. Click **OK** to add the new role.
10. Follow the steps in the previous recipe to add a user to this new role.

How it works…

Through the addition of new roles, you can grant very specific permissions to your users. In this example, only one task was selected, but you may select any combination of tasks for your end users.

This recipe was an example of a role that might be applicable to a help-desk employee. To manage subscriptions for all users of the server, you would normally need to place them in the predefined role of **Content Manager**. However, it is unlikely you would want someone in the Help Desk role to have that much power on your SSRS server.

By creating the customized role as we did in this recipe, we can now safely give the help-desk employee the ability to assist users with subscriptions, while at the same time not giving them too much power on the server.

Editing existing roles in Reporting Services

In addition to adding new roles, it is possible to modify existing roles. Doing so is handled in much the same way as adding new ones.

Getting ready

Just as with adding roles, modifying roles is handled within SQL Server Management Studio. You must have administrative rights to the reporting server to create new roles.

How to do it…

1. Open SQL Server Management Studio.
2. Connect to the instance of Reporting Services you wish to add new roles to.
3. Expand the **Security** branch.
4. Expand the **Roles** branch.
5. Right-click on the **Report Builder** predefined role.
6. Select **Properties** in the pop-up menu.

7. Place a check mark beside **View data sources**:

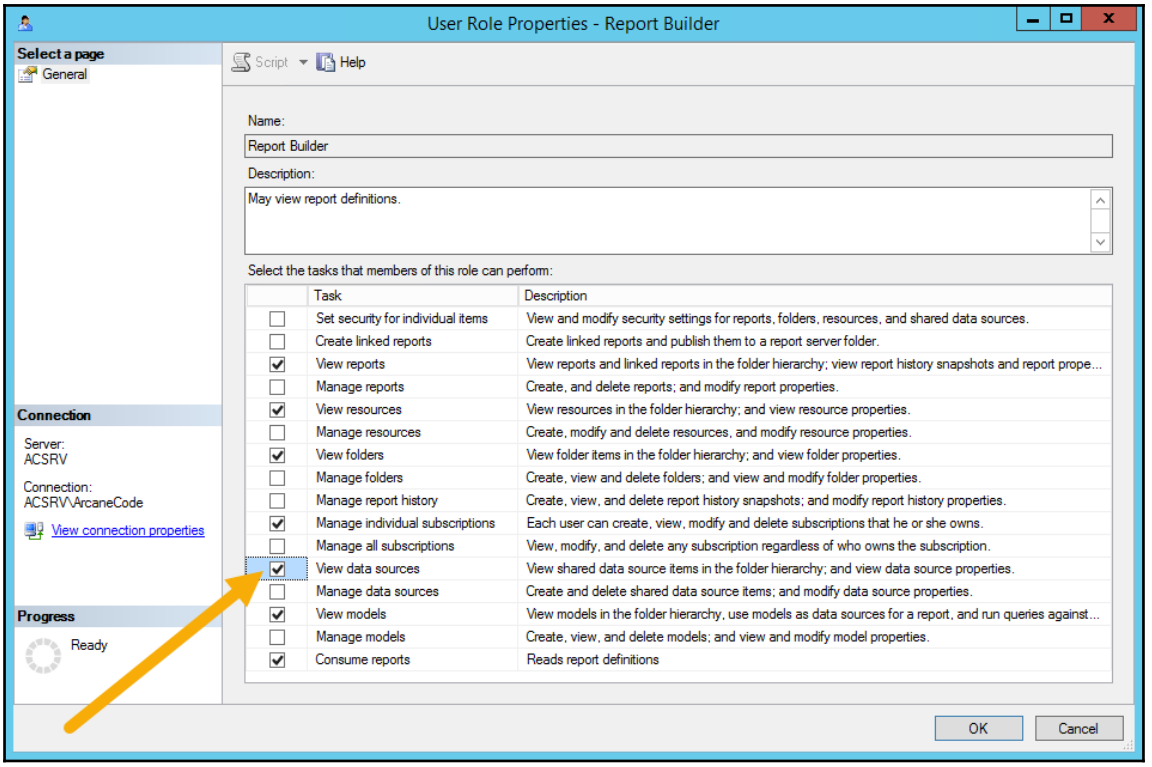


Figure 11.07

8. Click **OK** to save the changes.

How it works…

There will be times when you need to modify a role already in existence. Perhaps it lacked a required permission, or when it was created, a permission was given in error. Using the steps in this recipe, you can control the exact set of tasks granted. In addition to granting additional permissions, you can also remove permissions from existing roles.



Modify roles included with SSRS with caution. New administrators won't normally expect default roles to have changed. If possible, consider creating a copy of an existing role and making the changes to it.

It is also possible to delete roles. Through the same pop-up menu where you can add or modify roles, you can also delete them. Be aware that if the role is in use, you won't be able to delete it.

Filtering data for the current user

SSRS Reports has the ability to use the identity of the current user as a built-in variable within the report. This means the user ID can be used as a filter for SQL queries against a data source, assuming the data source stores that same ID.

Getting ready

In order for this recipe to work correctly, we will need to add IDs to the database. We will modify the `Dimension.Employee` table within the `WideWorldImportersDW` database to hold a fictitious manager ID field. First, open SSMS and execute the following query against the `WideWorldImportersDW` database:

```
ALTER TABLE [Dimension].[Employee]
ADD [ManagerID] NVARCHAR(50)
```

Next, we will need to insert data into this column. In this example, you should replace the IDs shown with ones that will be valid in your organization or computer; the ones used here are for testing purposes:

```
UPDATE [Dimension].[Employee]
   SET [ManagerId] = 'ACSRV\ArcaneCode'
 WHERE [Employee Key] < 101
UPDATE [Dimension].[Employee]
   SET [ManagerId] = 'ACSRV\TestUser1'
 WHERE [Employee Key] > 100
```

How to do it…

1. Create a new SSRS project within Visual Studio. Name the solution and project `Chapter11`. (Note, if you need a refresher on using Visual Studio to create reporting projects, please refer back to `Chapter 2`, *Authoring Reports with SQL Server Data Tools*.)
2. Right-click on the **Reports** branch, and select **Add New Item…** in the menu.
3. Select **Report** (be careful not to pick the default of Report Wizard by accident), use `Report 11-1` as the report name, and click **Add**.
4. In the **Report Data** tool pane, right-click on **Data Sources** and select **Add**.
5. Enter `WideWorldImportersDW as the name`.
6. Ensure the **Embedded connection** option is selected.
7. Click the **Edit** button to the right of the connection-string text box.
8. Select the values to point to the `WideWorldImportersDW` database. Click **OK** after successfully testing your connection.
9. Click **OK** to close the **Data Source Properties** window.
10. Right-click on the **Parameters** branch. Select **Add Parameter** from the pop-up menu.
11. For both the **Name** and the **Prompt**, enter `ManagerID`.
12. Set the **Select parameter visibility** to **Internal**.
13. Click on **Default Values**.
14. Select the **Specify values** option.
15. Click **Add**.
16. Click the **fx** button to the right of the value field.
17. In the **Expression** editor, under **Category**, click **Built-in Fields**.
18. In the **Item** box, double-click on **UserID**.

19. Click **OK** to close the **Expression** editor and return to the properties window:

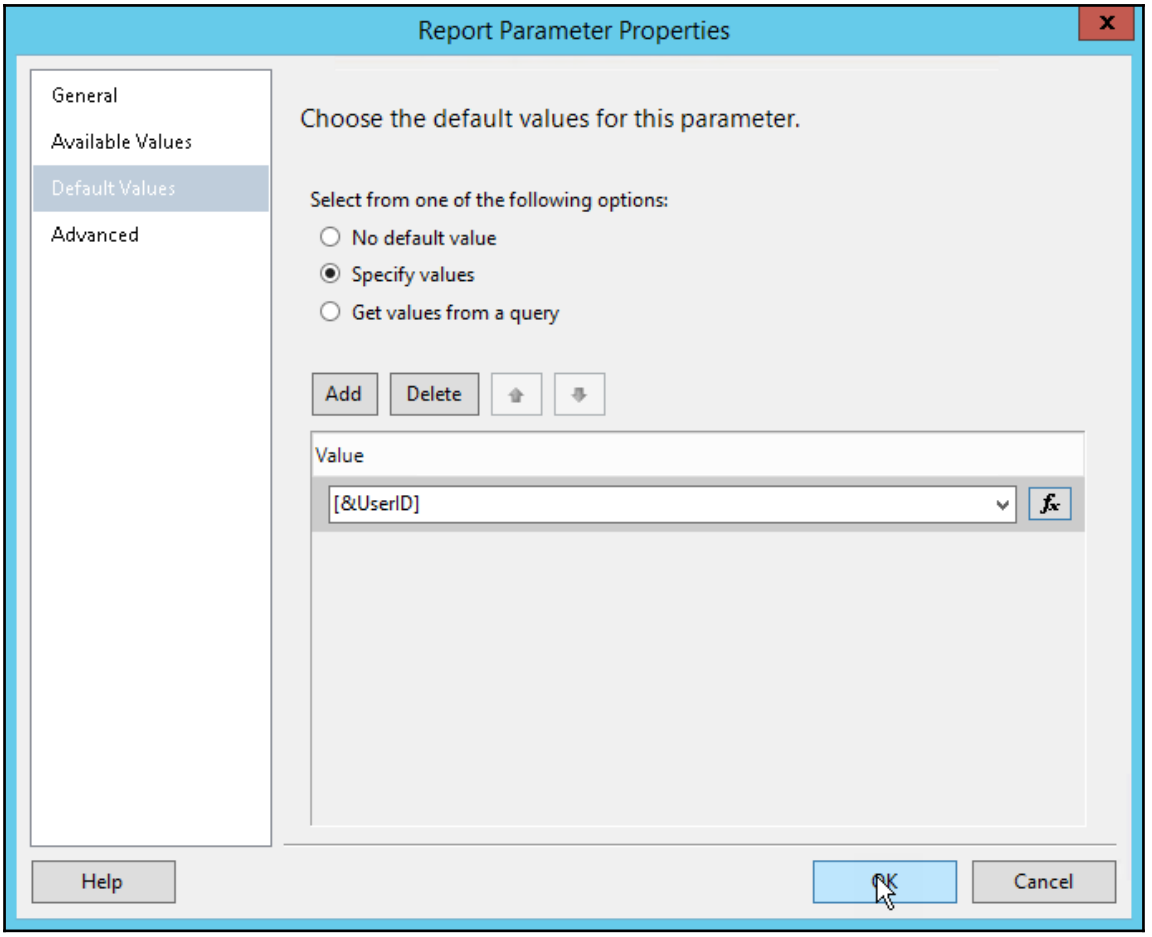


Figure 11.08

20. Click **OK** again to close the **Report Parameter Properties** window.
21. Return to the **Report Data** tool pane, right-click on **Datasets**, and select **Add Dataset…**.
22. In the **Name** area, enter `Employees`.
23. Select the **Use a dataset embedded in my report** option.
24. Select the **WideWorldImportersDW** connection as the **Data Source**.

25. Ensure the query type is defaulted to **Text**, then use the following query:

```
SELECT [Employee Key]
     , [Employee]
     , [Preferred Name]
     , CASE [Is Salesperson]
       WHEN 0 THEN 'Non-Sales Role'
       ELSE 'Salesperson'
       END
       AS [IsSalesperson]
     , [ManagerID]
  FROM [Dimension].[Employee]
 WHERE [ManagerID] = @ManagerID
```

26. Click on the **Parameters** tab on the left.
27. Under **Parameter Value**, use the drop-down menu, to select **[@ManagerID]**.
28. Click **OK** to close the **Dataset Properties** window.
29. Go to the **Toolbox** pane.
30. Place a table control on the report.
31. Return to the **Report Data** pane.
32. Drag and drop each column from the `Employees` dataset into the table control. Note that after you fill up the three columns placed by default, you can add additional columns to the end by dropping them on the right-hand side of the control.
33. Expand the column widths of the `Employee`, `Preferred Name`, and `Manager ID` columns to give them a bit more room.
34. Preview the report. You should only see employees whose records match your user ID in the Manager ID column.

How it works…

When SSRS executes a report, it can obtain the current user ID through the built-in `User!UserID` value. We then place this into a report parameter. Because this parameter will never be passed in, we set its visibility property to internal, meaning it will only be accessible within the report.

Placing the User ID into a parameter makes it easy to use the text property of our SQL query. We can just set the query to use the `@ManagerID` SQL parameter, and then map the SQL parameter to the report parameter.

At this point, the parameter behaves just like any other parameter, as we saw in *Managing and configuring parameters* recipe in `Chapter 2`, *Authoring Reports with SQL Server Data Tools*.

There's more…

There are two other methods we could choose to filter down the data. First, we could take our SQL Query and place the whole thing into an expression, concatenating it with the `User!UserID` value inside the query text property of the dataset. However, this approach makes it harder to keep the query up to date, and can cause debugging issues.

The second approach is to apply the `UserID` in the filters. Filters are similar to parameters, but with one important difference. When a parameter is used, the dataset is limited on the SQL Server; in other words, only the data we want is brought back to the machine.

With a filter, all the data is brought back and then the filter is applied. In our example, we would have fetched all 200+ rows from the employee table and placed them in memory. The filter is then applied, showing only the 100 rows associated with your user ID, and hiding the remaining rows. These rows are still taking up space in memory, just hidden from the user.

Using a parameter, the SQL Server only returned 100 rows from the database to the reporting server. With a small example like this, it doesn't make much difference, but bear this in mind when working with larger datasets.

As a final note, in this example we displayed the **Employee Key** column in the report. This was only done for demonstration purposes. In a real-world situation, we would *never* display the primary key from a data warehouse, since in a true data warehouse, primary keys are designed to have no meaning to the end user.

Handling security on shared data sources

In the *Authorizing users for accessing reports via SSRS roles* recipe, you saw how to manage access to folders via roles. Now take a moment to think all the way back to `Chapter 3`, *Advanced Report Authoring with SQL Server Data Tools*, specifically the *How to publish reports* recipe. In it, the reports you deployed were placed in a folder named `Chapter 3`, *Advanced Report Authoring with SQL Server Data Tools*. However, the data sources went into their own **Data Sources** folder. It is quite common in an organization for all data sources to go into a shared folder.

How, then, can you allow a user to access some of the data sources in a folder, but not all of them? Well, fortunately that is quite easy, as you'll see in this recipe.

Getting ready

To prepare for this recipe, you'll need a few things. First, you'll need a test account, which you will remove from certain roles (Obviously, you won't want to do this to yourself.) The account we set up is **TestUser1**. It's a good idea to have test accounts for testing security with any application, including SSRS.

You will also need to have administrative rights on the server, or at least be in the System Administrator role on the Report Portal. (See the *Becoming a Report Portal System Administrator* recipe in this chapter for more information.)

Second, you will need to have your data sources and reports deployed to the server. For this recipe, we'll reuse the reports and data sources from `Chapter 3`, *Advanced Report Authoring with SQL Server Data Tools*. If you haven't published those to your test machine, now is a good time to do so, although in fact, the techniques shown here should work with any chapter that employs the use of shared data sources.

How to do it…

1. Logged in as yourself, with system administrator rights, open the Report Portal and navigate to the **Data Sources** folder.
2. Click on the **WideWorldImportersDW** data source.

3. Click the **Security** link on the left.
4. Click **Customize** security at the top:

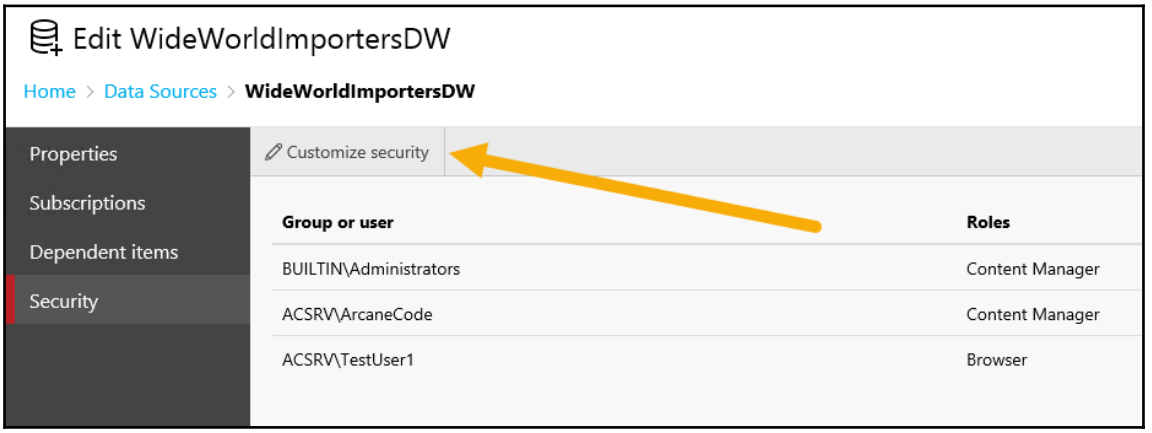


Figure 11.09

5. A confirmation dialog will appear asking if you want to apply security settings differently for this item than for the parent (that is, the Data Sources folder). Click **OK** to confirm you wish to do so.
6. Place a check in the box beside your test user and then click **Delete** at the top to remove all permissions to see the data source from this user:

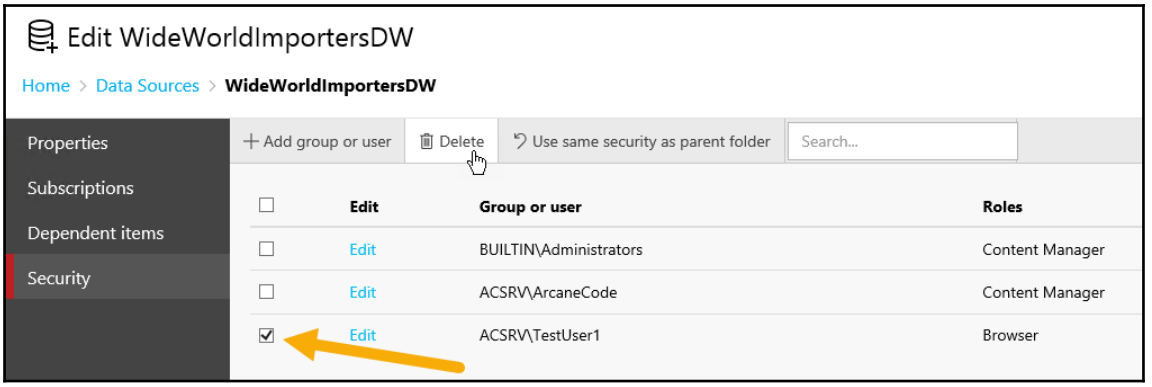


Figure 11.10

7. When the dialog appears, confirm that you do indeed wish to delete that user.
8. On a second computer, log in as the test user you created (or connect to the SSRS report portal using the credentials of the test user).
9. Navigate to the Data Sources folder. The `WideWorldImportersDW` should not be present.
10. Navigate to the folder with the `Chapter 3` reports.
11. Attempt to run one of the reports based on the shared data source.
12. The report should fail to run, and instead you should see an error similar to the following:

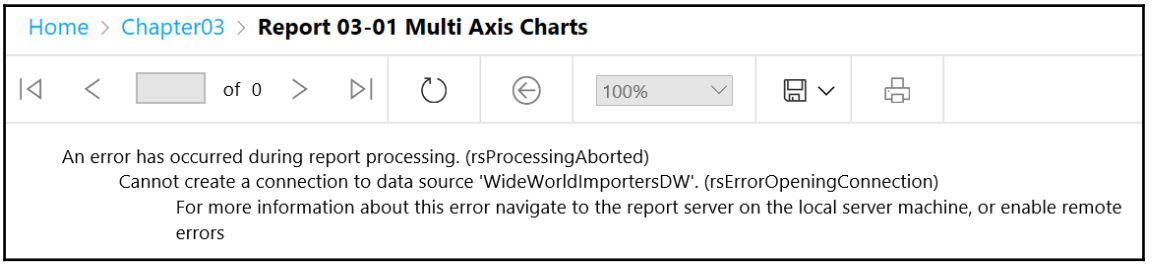


Figure 11.11

How it works…

In previous recipes, you saw how to grant or remove permissions at the folder level. It is also possible to do so at the individual item level, in this case, a data source. When we remove the test user's permissions completely from the data source, they can no longer see it.

In addition, when they attempt to run a report based on that data source, that report failed. They can still see the report, but can't run it.

This brings up an important point. Be sure the security for your data sources and reports based on them stay in sync. If a user is not allowed to see the data source, they should not be able to see any reports based on those data sources. By using groups to set the security on data sources as well as reports (or folders with reports), this shouldn't be too difficult a task.



We'd suggest you create a matrix of which user groups see which reports and data sources, and store it in a database on the same server where SSRS is installed. This will make the information readily available to all system administrators. It will also make it easy to query which users can see what reports.

There's more…

Interestingly, it is possible to hide the data sources from the end user but still allow reports based on those data sources to still execute. Simply go to the **Data Sources** folder, use the **Manage folder** option, and remove that user from the folder. The user should still have access to the data sources contained in it, and the existing reports should still run.

Be aware, though, that if new data sources are added, you will need to explicitly grant rights to those data sources. By default, new items added to a folder inherit the security roles of the parent, which you just eliminated the user from.

Securing the channel using certificates

In today's age of constant security breaches, it is important to keep the data in your reports safe and secure. One important way to do this is to enable SSL on your report server. SSL allows you to connect using the `https` protocol.

In this recipe, we'll see how to bind your instance of Reporting Services to an SSL certificate. There are actually two components we will be securing. The first is the Web Portal URL, which is the URL your users will use to access the Report Portal.

The second is the Web Service URL. This is what your developers will use to upload reports to from Visual Studio or any of the other reporting tools.

Getting ready

In order to enable SSL, you must have a certificate installed on the Windows Server running Reporting Services. Normally, certificates would be supplied through your administrator and obtained from a certificate authority, such as DigiCert.

If you don't have such a certificate but wish to try out this recipe, you can follow the steps given here to create one on this server. Be aware that this should only be done for testing, not for production situations:

1. Log in to the server running SSRS using an account with administrative rights.
2. Open **Internet Information Services** (**IIS**) Manager.
3. Click on the name of your server in the connections area to the left.
4. Double-click on the **Server Certificates** under the **IIS** area:

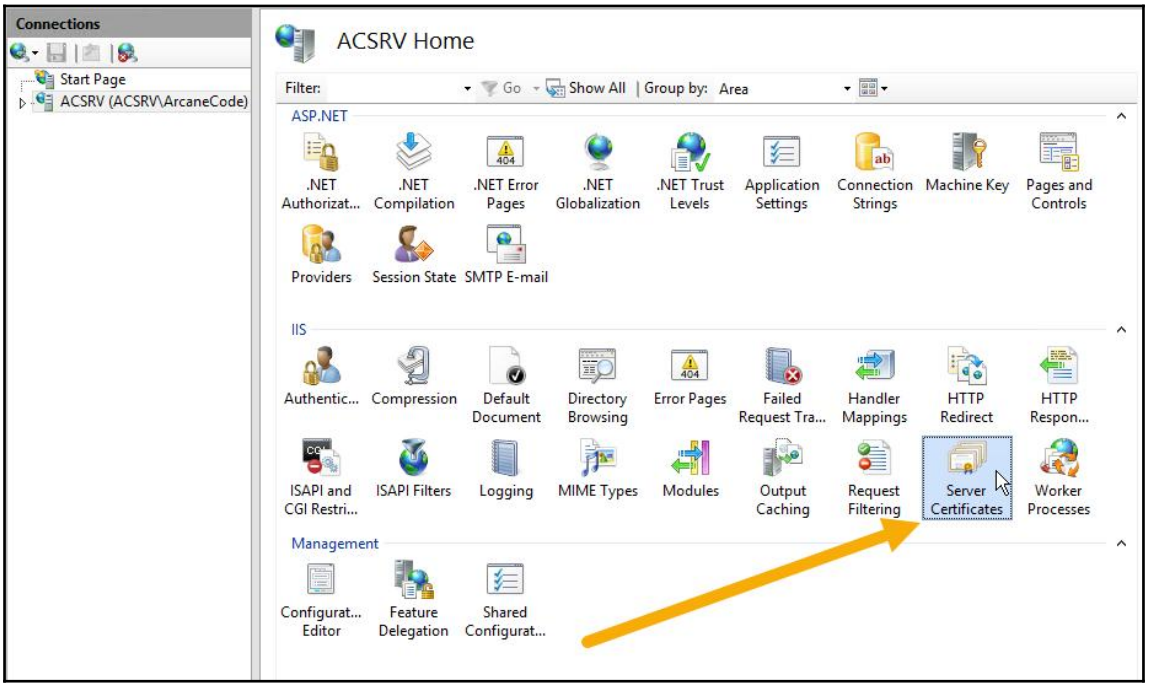


Figure 11.12

5. In the area under the certificates, right-click, then select **Create Self-Signed Certificate** in the pop-up menu:

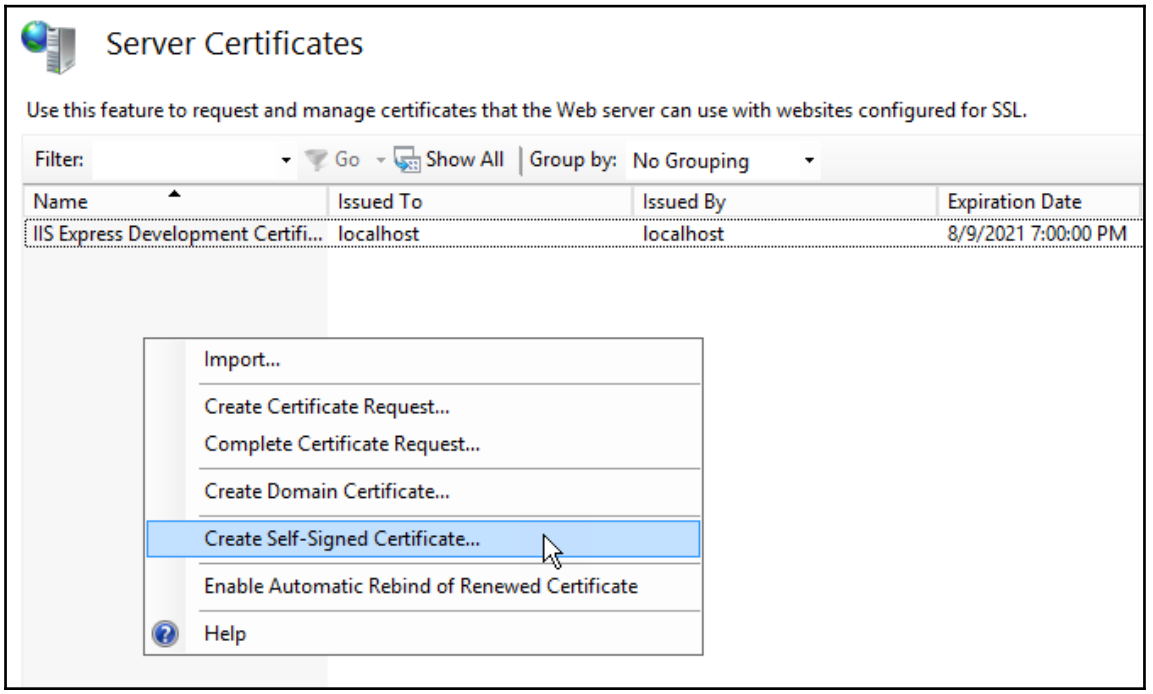


Figure 11.13

6. In the **Create Self-Signed Certificate** dialog, enter `SSRS` as the friendly name for the certificate and click **OK**:

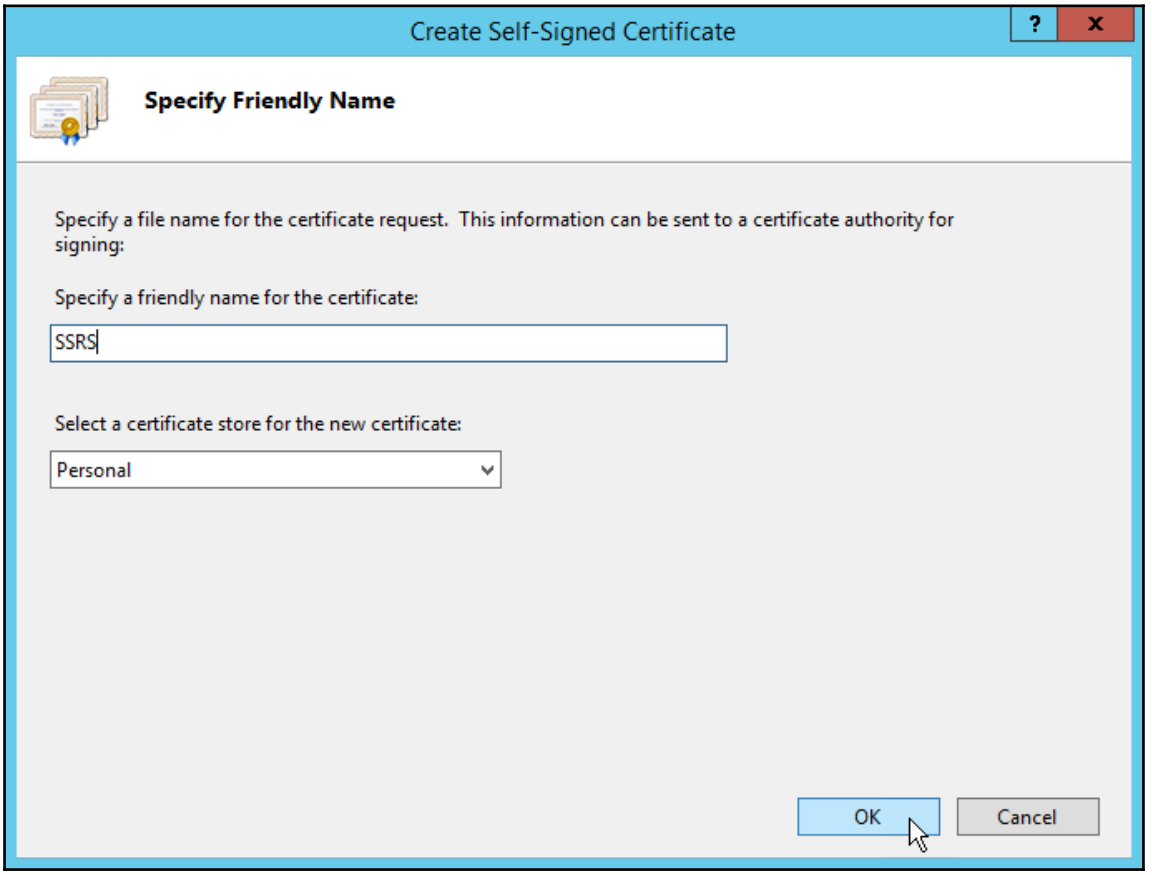


Figure 11.14

How to do it…

1. Log in to the Windows Server using an account with administrative credentials.
2. Open the **Reporting Services Configuration Manager**.
3. Connect to your server.
4. From the menu on the left, click **Web Portal URL**.
5. Click the **Advanced** button.
6. In the **Advanced Multiple Web Site Configuration** dialog, the bottom half deals with HTTPS. Click the **Add** button at the bottom.
7. The **Add a Report Server HTTPS Binding** dialog appears. In the **Certificate** area, use the drop-down menu to select your certificate:

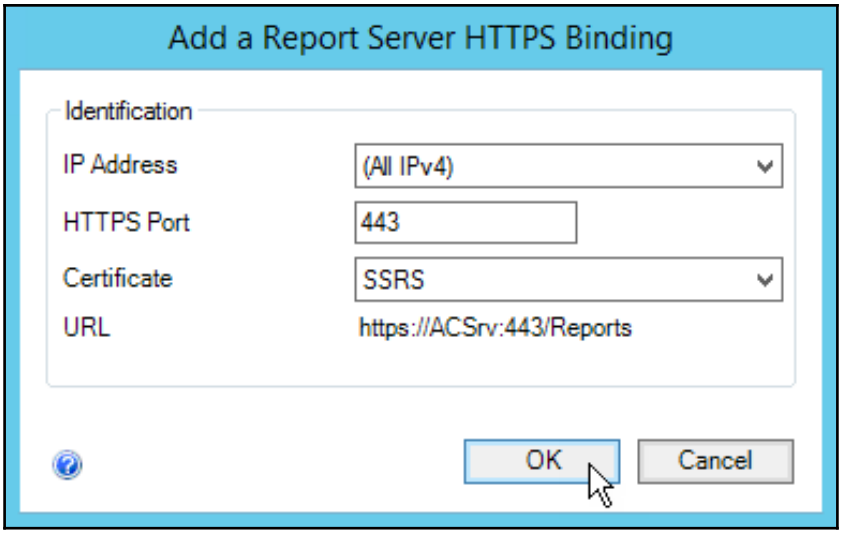


Figure 11.15

8. Click **OK**:

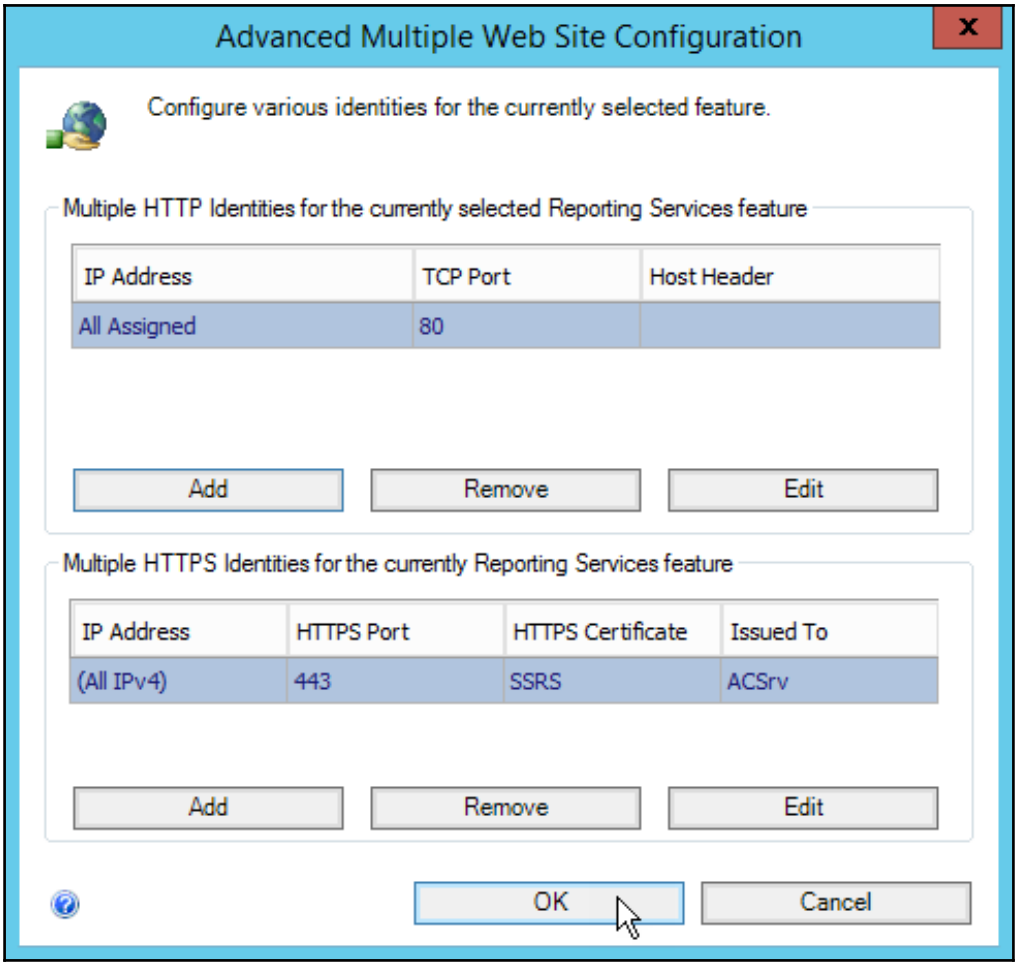


Figure 11.16

9. Click **OK** again to close the **Advanced Multiple Web Site Configuration** dialog.
10. A new link will appear under the URLs area. Click the link with HTTPS in it to test. Now to secure the web service.
11. From the menu on the left, click on **Web Service URL**.
12. In the **HTTPS Certificate** drop-down menu, select the certificate you wish to use.

13. Unless you need to do otherwise, use the default port of 443, which appears in the **HTTPS Port** field:

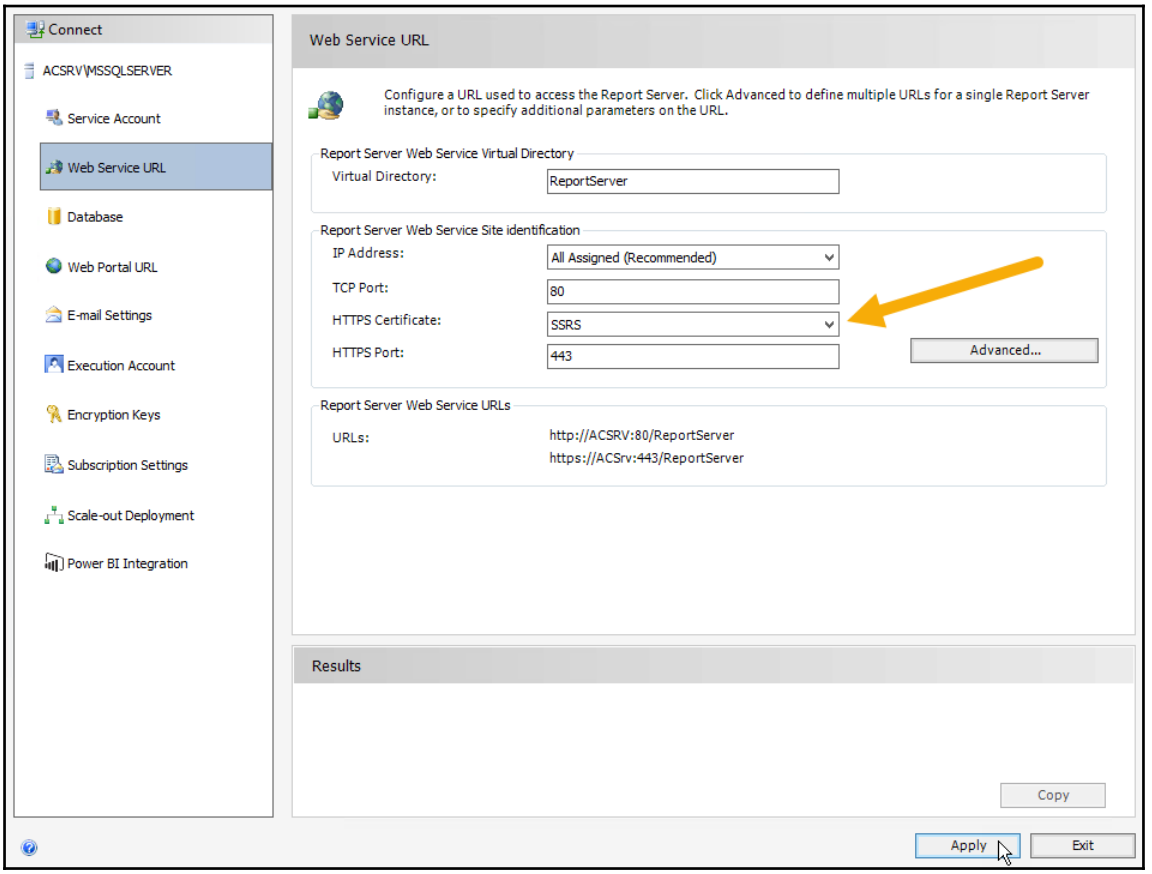


Figure 11.17

14. Click **Apply**. Be patient; it will take several minutes to apply the settings.
15. Click the URL with HTTPS in the name to test.



If you used a self-signed certificate and attempt to access the Report Portal or Web Service from a machine other than the server, you will receive an error that there was a problem with the certificate. You will be prompted to either abort or continue. You may safely continue for testing purposes. As stated in the getting started section, self-signed certificates should only be used for testing purposes. You'll want to obtain a domain certificate from either your Active Directory administrator or a certificate authority such as DigiCert.

How it works…

A certificate serves two purposes. First, it provides a way to verify the identity of the server. Second, it provides a way of encrypting information between the server and the client, so that only the two ends are able to see the information. Anyone in between is unable to view the information. For an overview of SSL, we suggest this article from SSL.com: `http://info.ssl.com/article.aspx?id=10241`

For an in-depth examination of certificates and SSL, we recommend the following article on Wikipedia: `https://en.wikipedia.org/wiki/Secure_Sockets_Layer`

Once the certificate has been installed on the server, it is easy to tell SSRS to use it. First, you will want to bind the Report Portal to the certificate. That was accomplished in steps 1 to 10.

In the remaining steps, we bound the web service URL so developers would have a secure channel through which they could upload and download their reports.

Note that these two items do not have to be done together. For example, you could choose to bind your Report Portal to SSL but leave the web service unbound. In addition, you could choose to bind them using two different certificates. This is a nice feature of the way SSRS implements SSL binding: it is flexible to your needs.

12
Custom Programming and Integration to .NET Applications

Reporting requirements can't be satisfied with given user-friendly built-in functionalities, though there are many options given for authoring reports. Some reports need to be extended with various business logic and you may find that they cannot be implemented easily with simple configurations or standard expressions. However, Reporting Services is rich enough for accepting custom code written for handling business logic and executing them on requests.

Not only extending reports, in most solutions, reports are viewed via either Windows applications or web applications. This requires integrating reports published with these applications and the integration is different from solution to solution.

Let's talk about both custom programming and integration, specifically with .NET applications in this chapter.

This chapter covers the following recipes:

- Adding embedded codes to reports
- Referencing external .NET assemblies
- Opening reports using URL access
- Using ReportViewer control for accessing reports

Introduction

Extending reports with custom coding is not a complex implementation with Reporting Services. However, it certainly accepts complex code and lets us use it with less burden. We can simply add custom codes to reports using .NET languages and use them with data regions via expressions. Reporting Services currently supports only Vb.NET codes for direct embedding. Functionalities required can be added to the coding section of Report Properties as functions and of course it allows us to add more than one function.

For an example, assume that we have an input box in the report for accepting dates in a certain format and use it as a parameter. We can use a custom code for validating the input value and act based on it. Another example is that we need to form the SQL code based on parameters and assign it to dataset. We can write a custom code to read all the parameters, form the query dynamically, and get the data loaded.

In addition to the embedded codes, for maintaining the consistency among all reports, we can add an external assembly to reports that has functions. Once added, we can access methods written, using expressions and extend the report with many advanced functionalities.

Not all users can access published reports using Reporting Services Web Portal. Generally, reports are exposed via other applications such as Windows or web application and they are open either using .NET controls given or URLs formed for reports. Even though it seems as an extra work, it provides many advantages because it allows us to control the user behavior as well as behavior of the report.

It is time to explore our last set of recipes.

Let's look at the method of extending reports with the integration of .NET applications.

Adding embedded codes to reports

With this recipe, let's see how to add a parameter in order to accept a value that can be converted to a date and use it for the dataset.

Getting ready

The solution related to this chapter is named as `Chapter12.sln`. We will be using this solution for working with recipes and the report we use for this recipe is DailySales. Now publish the solution. If you do not have the solution, you need to create a report with the given query that connects with the `WideWorldImportersDW` database. Following are the steps:

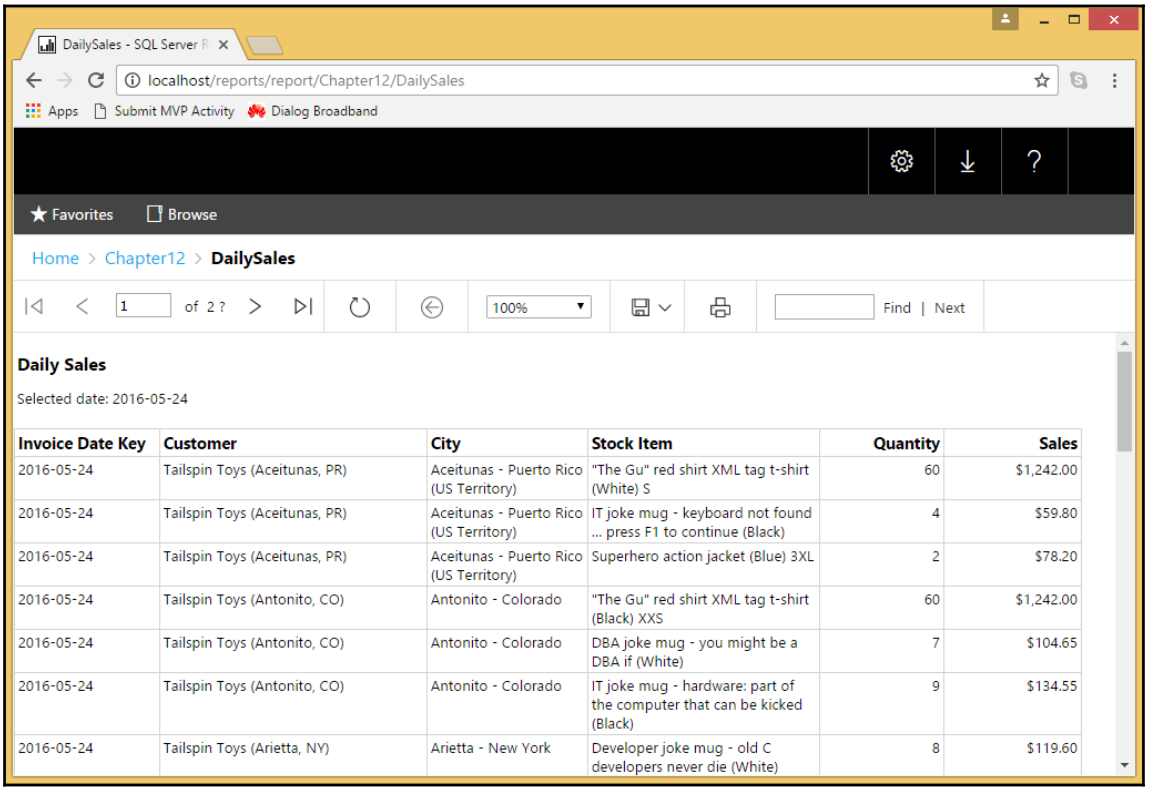
1. Open the `Chapter12.sln` solution and configure the data source `DataSourceWideWorldImportersDW` with settings related to your database server and configure the Reporting Services server settings.
2. If you are making your own report, make sure that the following query is used and the table data region is added as per the output shown in the Figure 12.01:

```
SELECT s.[Invoice Date Key], c.Customer, ct.City + ' - ' + ct.
State Province] City
  , e.[Preferred Name], si.[Stock Item]
  , Sum(s.Quantity) Quantity, Sum(s.[Total Including Tax]) Sales
FROM Fact.Sale s
  INNER JOIN Dimension.Customer c
    ON c.[Customer Key] = s.[Customer Key]
  INNER JOIN Dimension.City ct
    ON ct.[City Key] = s.[City Key]
  INNER JOIN Dimension.Employee e
    ON e.[Employee Key] = s.[Salesperson Key]
  INNER JOIN Dimension.[Stock Item] si
    ON si.[Stock Item Key] = s.[Stock Item Key]
WHERE s.[Invoice Date Key] = '2016-05-24'
GROUP BY s.[Invoice Date Key], c.Customer, ct.City + ' - ' + ct.
State Province]
  , e.[Preferred Name], si.[Stock Item]
```

3. In addition to that, have a text box added to show the dateset with the following expression:

```
="Selected date: " & format( First(Fields!Invoice_Date_Key.Value,
"DataSetSales"), "yyyy-MM-dd")
```

4. Publish the report and view. You should see a report as shown in the following screenshot:



Invoice Date Key	Customer	City	Stock Item	Quantity	Sales
2016-05-24	Tailspin Toys (Aceitunas, PR)	Aceitunas - Puerto Rico (US Territory)	"The Gu" red shirt XML tag t-shirt (White) S	60	$1,242.00
2016-05-24	Tailspin Toys (Aceitunas, PR)	Aceitunas - Puerto Rico (US Territory)	IT joke mug - keyboard not found ... press F1 to continue (Black)	4	$59.80
2016-05-24	Tailspin Toys (Aceitunas, PR)	Aceitunas - Puerto Rico (US Territory)	Superhero action jacket (Blue) 3XL	2	$78.20
2016-05-24	Tailspin Toys (Antonito, CO)	Antonito - Colorado	"The Gu" red shirt XML tag t-shirt (Black) XXS	60	$1,242.00
2016-05-24	Tailspin Toys (Antonito, CO)	Antonito - Colorado	DBA joke mug - you might be a DBA if (White)	7	$104.65
2016-05-24	Tailspin Toys (Antonito, CO)	Antonito - Colorado	IT joke mug - hardware: part of the computer that can be kicked (Black)	9	$134.55
2016-05-24	Tailspin Toys (Arietta, NY)	Arietta - New York	Developer joke mug - old C developers never die (White)	8	$119.60

Figure 12.01

As you see, the dataset is specifically set for **2016-05-24**. Rather hardcoding it, let's add a parameter and make sure that user can input following values:

- Today
- Yesterday
- Values between -1000 and 0 (0 for Today and negative values for previous days)
- Date formatted as YYYY/MM/DD

We just cannot accept the value without validating and formatting. We have to validate and then convert the value to a date before passing it to the dataset.

How to do it…

The following are the steps for adding a parameter, accepting a value, and adding an embedded code for validating and formatting the value:

1. Open the `Chapter12` solution and open the DailySales report.
2. Add a report parameter and name it as `ReportParameterSalesDate` (see Figure 12.02).
3. Set the **Prompt** as `Sales Date` and set the **Data type** as **Text**:

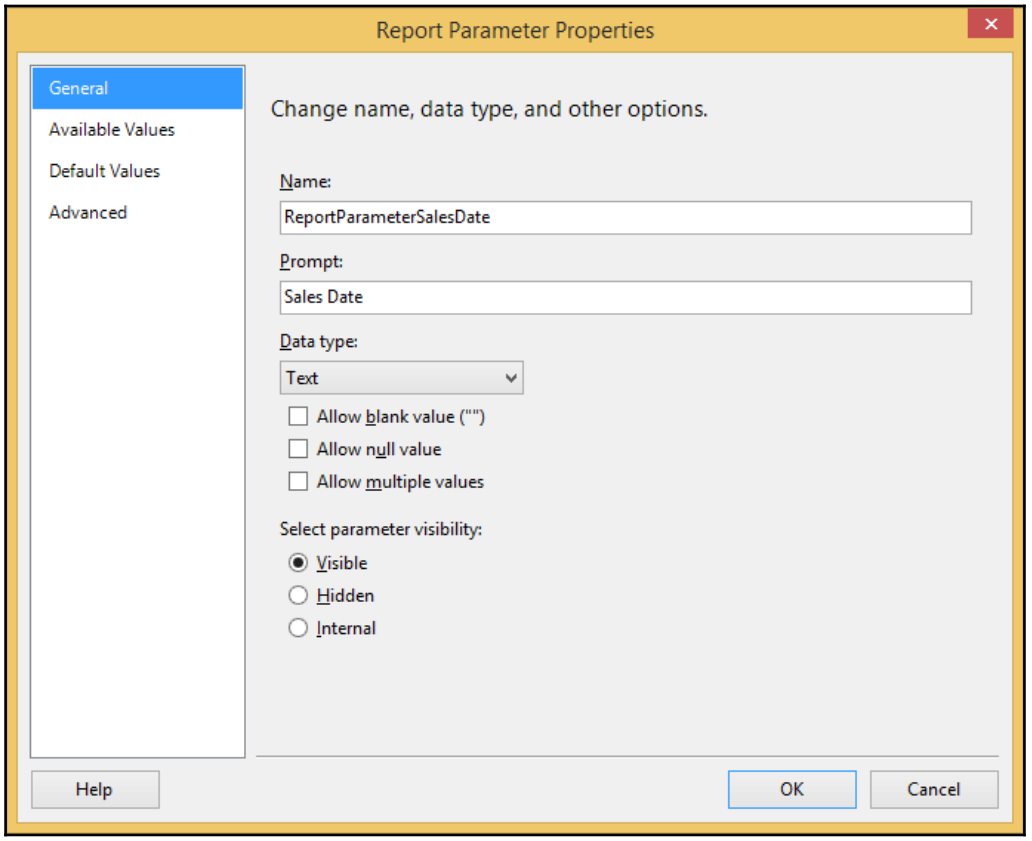


Figure 12.02

4. Open the **Default Values** page and set the default value as `Yesterday`. Click **OK** to save the parameter:

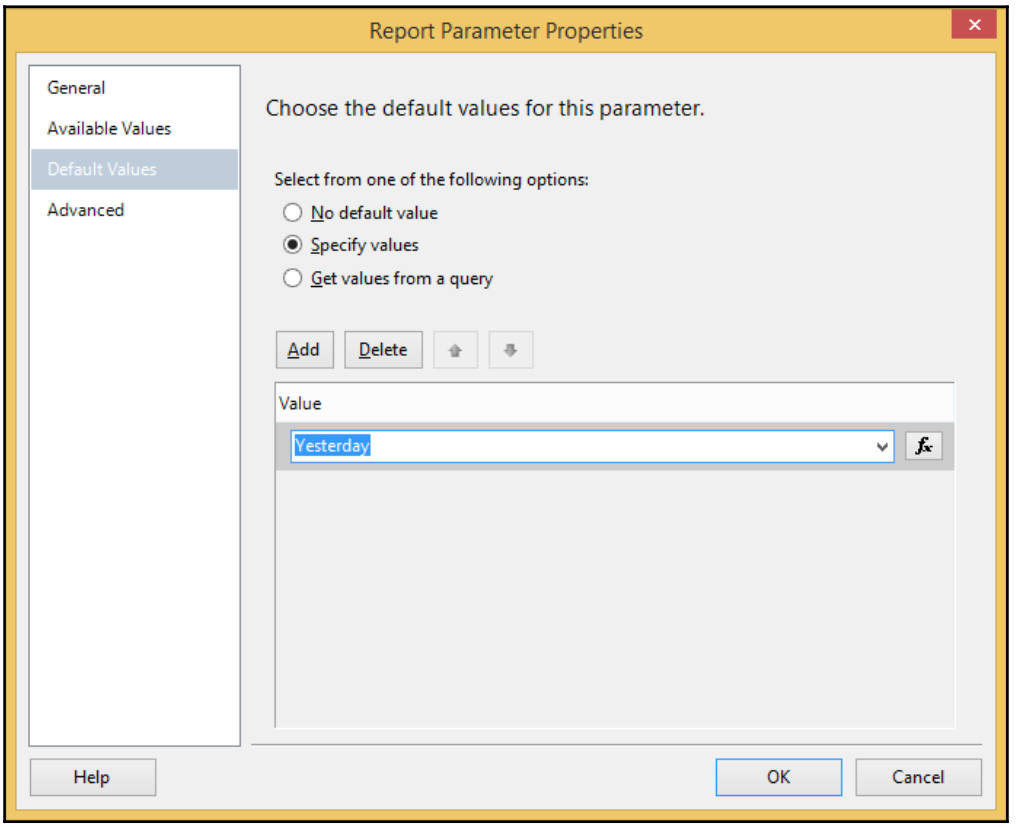


Figure 12.03

5. In order to add the code, right click on the reporting area (not the body) for getting the following context menu opened and select **Report Properties…**:

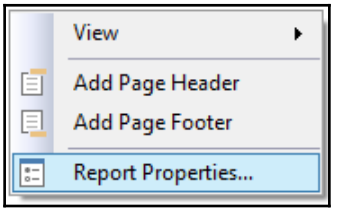


Figure 12.04



Note that **Report Properties…** can be opened using the Report main menu. If you click on the **Report** menu, the first item of it is **Report Properties…**.

6. Click the **Code** in the left pane to open the **Custom code** section and set the following code:

```
Function GetDate(ByVal dateString as String) As Date
  Dim dateEntered = Convert.ToDateTime("1900-01-01")
  Try
       If dateString.ToString.ToLower().Trim() = "today" Then
dateEntered = Now()
       ElseIf dateString.ToString.ToLower().Trim() = "yesterday"
  Then
            dateEntered = Now.AddDays(-1)
       ElseIf (CInt(dateString) > -1001 And CInt(dateString) < 1)
       Then
            dateEntered = Now.AddDays(CInt(dateString))
       Else
            If (dateString.Length = 8) Then
                 Dim year As Integer = CInt(Left(dateString, 4))
                 If year > Now.Year Or year < 2000 Then
                    Throw New Exception
                 End If
                 Dim month As Integer =
                 CInt(dateString.Substring(4, 2))
                 Dim day As Integer = CInt(Right(dateString, 2))
                 dateEntered = New Date(year, month, day)
            End If
       End If
  Catch ex As Exception
       dateEntered = Convert.ToDateTime("1900-01-01")
    End Try
    Return dateEntered
End Function
```

Look at the written code once. It initially checks values like `today` and `yesterday` and sets the relevant dates. Then it addresses numeric values entered and set it as a date. If user has entered as value with the YYYY/MM/DD format, then it extracts year, month and date separately, and format the date. If an exception occurs or an invalid value is found, it sets for the date as 1900-01-01.

7. Click **OK** to save the settings:

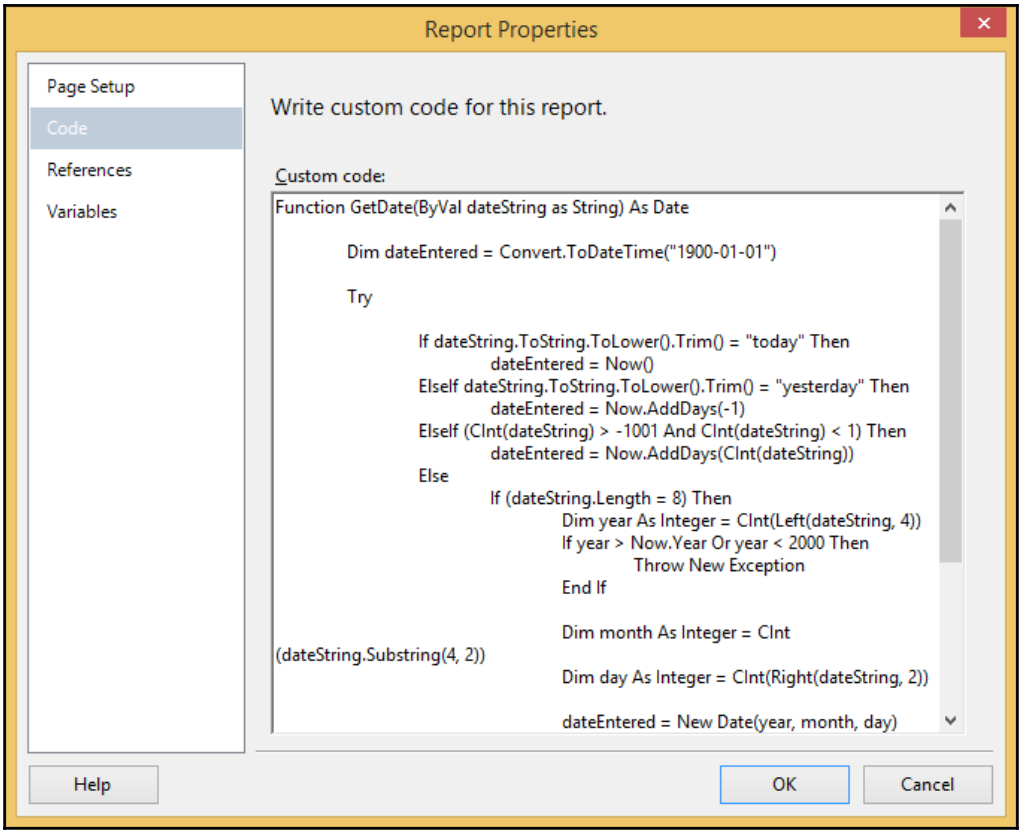


Figure 12.05

8. Next, we need to change the dataset query. Open the dataset and change the `WHERE` clause of the query as shown here:

```
WHERE (s.[Invoice Date Key] = Convert(date, @Date))
```

9. The preceding statement adds a parameter to the dataset. Open the **Parameters** page of **Dataset properties** and add the following expression for the `@Date parameter`:

```
=Code.GetDate(Parameters!ReportParameterSalesDate.Value)
```

10. Click **OK** to save the **Dataset properties**:

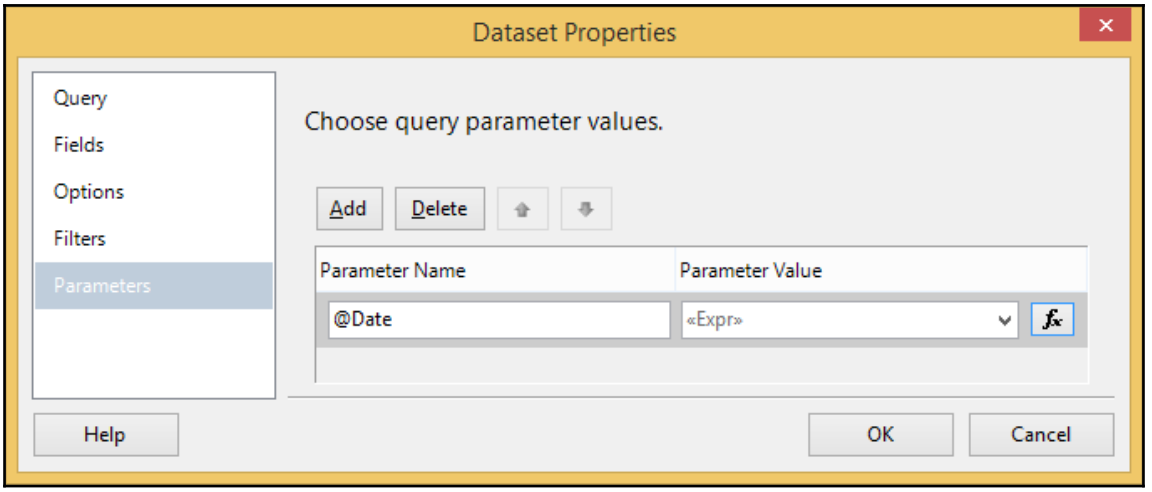


Figure 12.06

11. It is always better to show the date selected in the report. Not only that, it is better to notify the user that the date entered is invalid. That can be easily achieved by changing text box added that shows selected date as shown here:
12. Right-click on the text box to get the context menu and click on **Expression** menu.
13. Change the expression as shown in the following code:

```
="Selected date: " &
Iif(format(Code.GetDate(Parameters!ReportParameterSalesDate.Value)
,"yyyy-MM-dd") = "1900-01-01"
, "Invalid date entered",
format(Code.GetDate(Parameters!ReportParameterSalesDate.Value),
"yyyy-MM-dd"))
```

14. Click **OK** to save the expression.
15. Deploy the report and view it. You should see the report showing data for yesterday values (if exist only).

16. If you enter a value like -200, it sets the relevant date for the query and executes it:

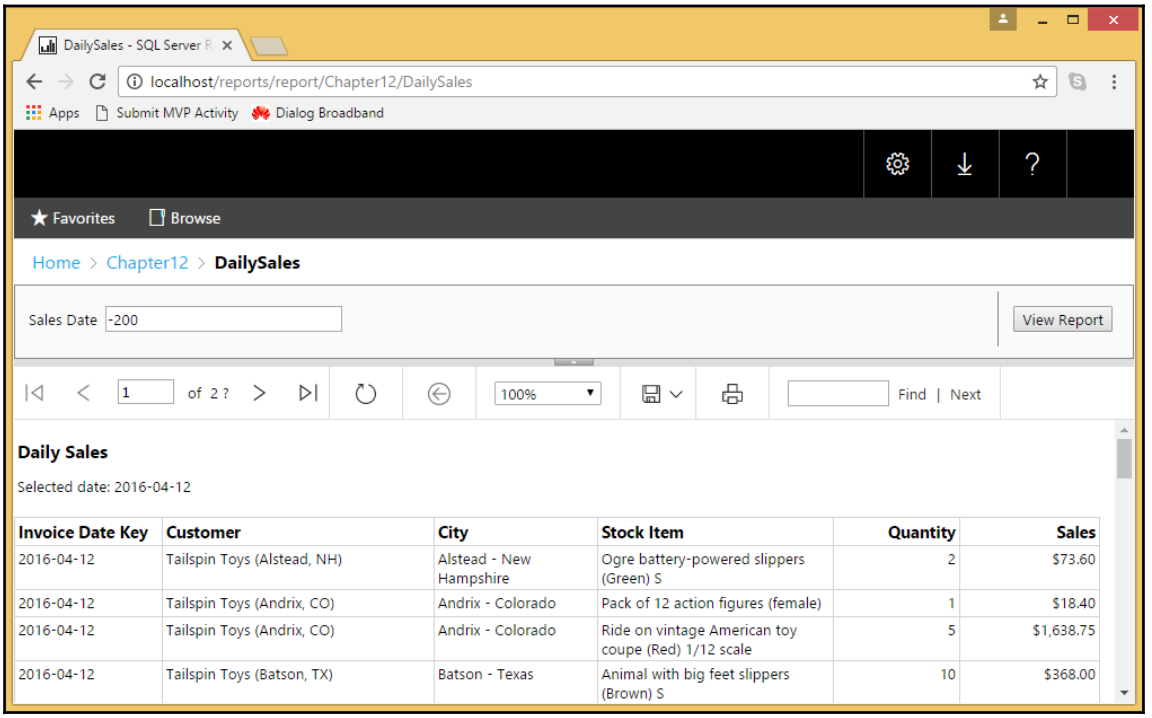


Figure 12.07

17. If you enter an invalid value, you should see a message as we configured with the expression:

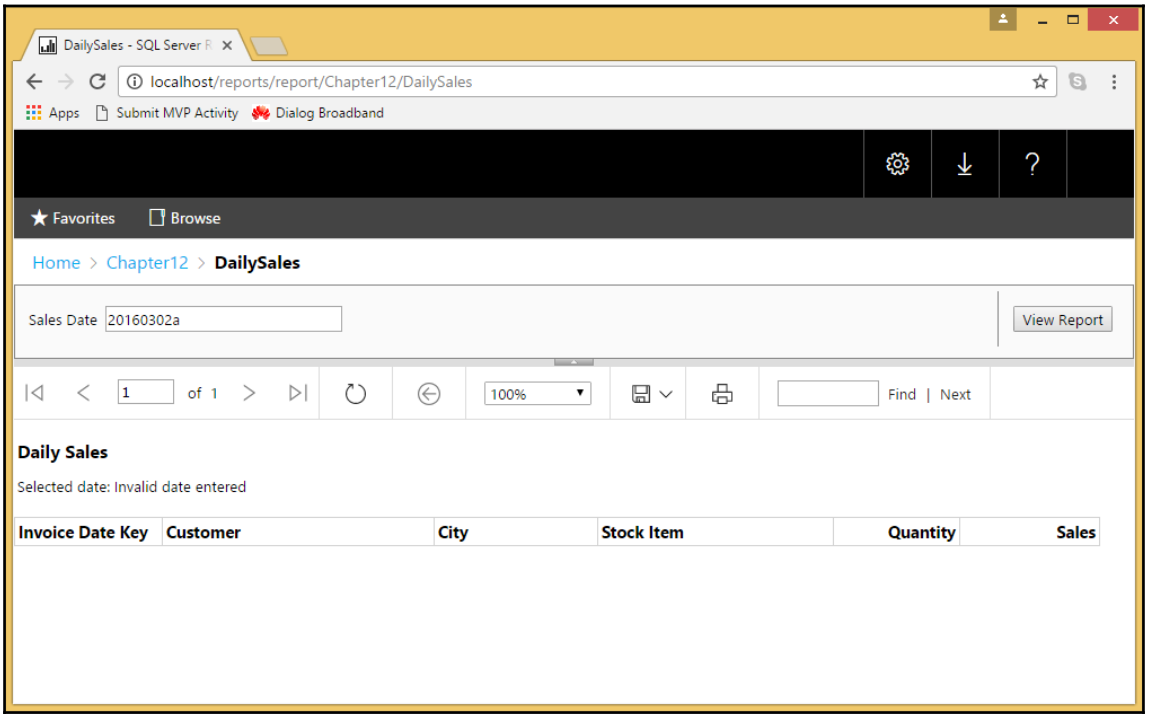


Figure 12.08

How it works…

Reporting Services allows us to use the functions available with .NET framework `System.Math` and `System.Convert` namespaces and Visual Basic run-time library functions. You do not need to add any references to the report or the project if you have not used other functions related to other namespaces. If you need to use other functions, you need to add references as full-qualified references such as `System.Text.StringBuilder`.

When the report is deployed, the code added to the report is deployed. It does not get compiled during the deployment and Reporting Services executes the code when it is requested by a user.

There's more…

Adding various functions to each report may not be the best solution if functions have to be used with many more reports. If you have a set of functions that are required by many reports, it is always better to have all the function in a single .NET class, compile it into a single assembly and use it with all reports. That is what we are going to talk with our next recipe that is *Referencing external .NET assemblies*.

Referencing external .NET assemblies

You have already seen the way of adding custom code to reports and you may have already experienced the issues with it such as:

- It supports only Vb.NET
- The same function has to be added to each and every report

With this recipe, we can overcome it by making an assembly that contains all required functions and use it with the report rather repeating it in every report.

Getting ready

In order to make an assembly with all required functions, you need to create a .NET class library using one of .NET languages. Yes, it is not limited to Vb.NET. Let's make one using C#.NET language.

Follow these steps for creating a class library:

1. Open the Visual Studio and create a C#.NET Class Library project. Name it as `StandardFunctions`.
2. Create a public class called `Functions` and add a public static method named `GetTextColor`:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace StandardFunctions
{
    public class Functions
    {
```

```
        public static string GetTextColor(string messageType)
        {
            string color = "black";
            switch (messageType)
            {
                case "Warning":
                    color = "Yellow";
                    break;
                case "Error":
                    color = "Red";
                    break;
                case "Success":
                    color = "Green";
                    break;
            }
             return color;
        }
    }
}
```

3. Open the `AssemblyInfo.cs` file in your project. Add `System.Security` namespace and make sure the `AllowPartiallyTrustedCallers` attribute is added:

```
using System.Reflection;
using System.Runtime.CompilerServices;
using System.Runtime.InteropServices;
using System.Security;
[assembly: AssemblyTitle("StandardFunctions")]
[assembly: AssemblyDescription("")]
[assembly: AssemblyConfiguration("")]
[assembly: AssemblyCompany("")]
[assembly: AssemblyProduct("StandardFunctions")]
[assembly: AssemblyCopyright("Copyright ©  2016")]
[assembly: AssemblyTrademark("")]
[assembly: AssemblyCulture("")]
[assembly: AllowPartiallyTrustedCallers()]
[assembly: ComVisible(false)]
[assembly: Guid("4d89a812-f315-48f3-af97-a7576cc4060b")]
[assembly: AssemblyVersion("1.0.0.0")]
[assembly: AssemblyFileVersion("1.0.0.0")]
```

4. Get the project properties and open Application page. Make sure that the Assembly name is set as `StandardFunctions`.
5. Open **Signing*** page of **Project Properties** and check **Sign the assembly** checkbox. This is for signing the assembly with a strong name as it is required when adding to GAC. Select **<New…>** from **Choose a strong name key file** drop down:

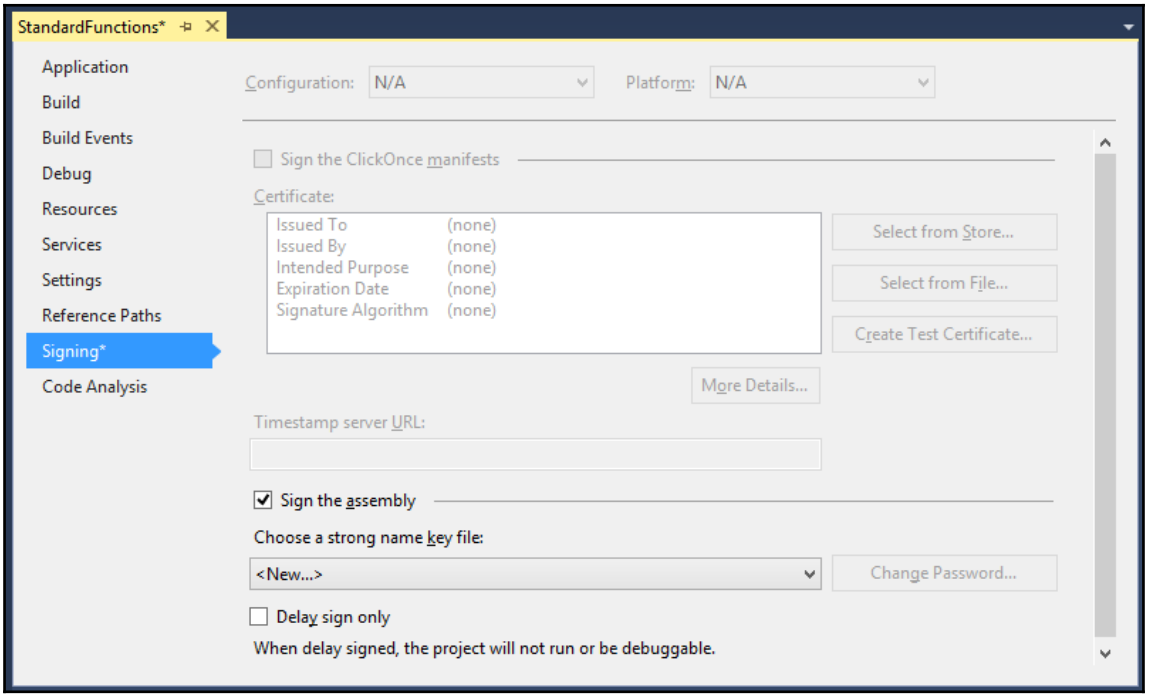


Figure 12.09

6. Set the **Key file name** as `StandardFunctionsKey` and uncheck **Protect my key file with password**. Click **OK** to create the file:

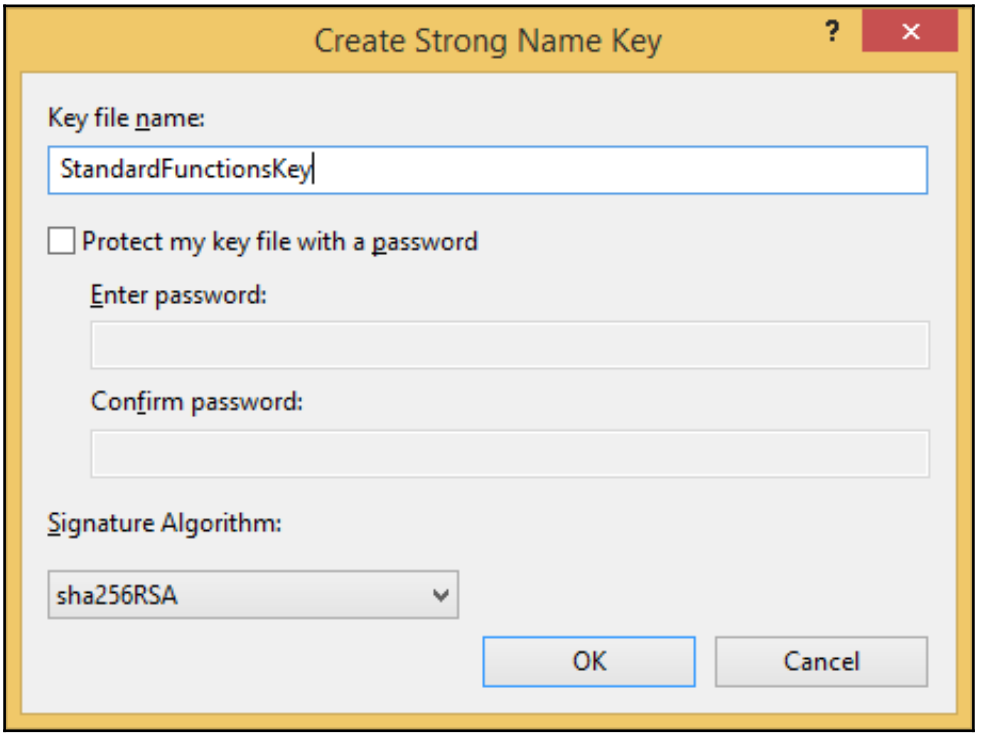


Figure 12.10

7. Save the project and build it. You should see the assembly created in the bin folder of the project:

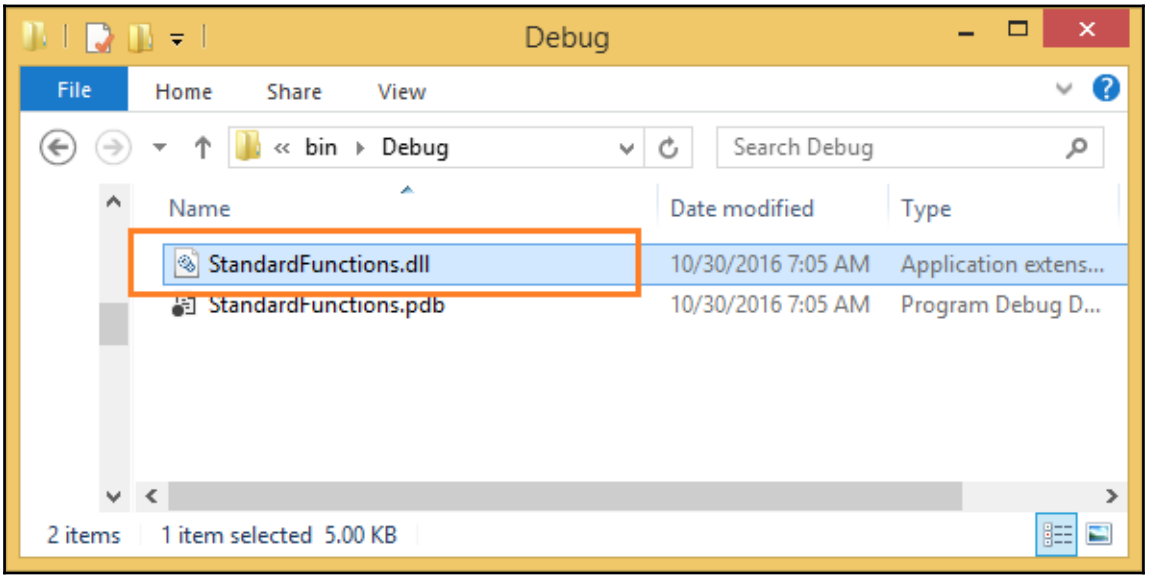


Figure 12.11

8. Now the assembly is ready. Let's see how it can be used with our reports.

How to do it…

The first thing we need to do is, make sure that the assembly can be accessed during development. This needs the assembly to be placed in the `Reporting Services bin` folder and add a reference adding it with **Report Properties**.

Follow these steps for accessing the assembly:

1. Take a copy of `StandardFunctions.dll` and place it in `bin` folder of Reporting Services. The default path is `C:\Program Files\Microsoft SQL Server\MSRS13.MSSQLSERVER\Reporting Services\ReportServer\bin`.
2. Open `Chapter12` solution and get the **Properties** of the DailySales report. Note that you need make sure the DailySales report is modified as per the first recipe.
3. Go to **References** page and click on **Add** to add a reference (see Figure 12.13). Browse to the placed `StandardFunction.dll` and select it. Click **OK** to add it:

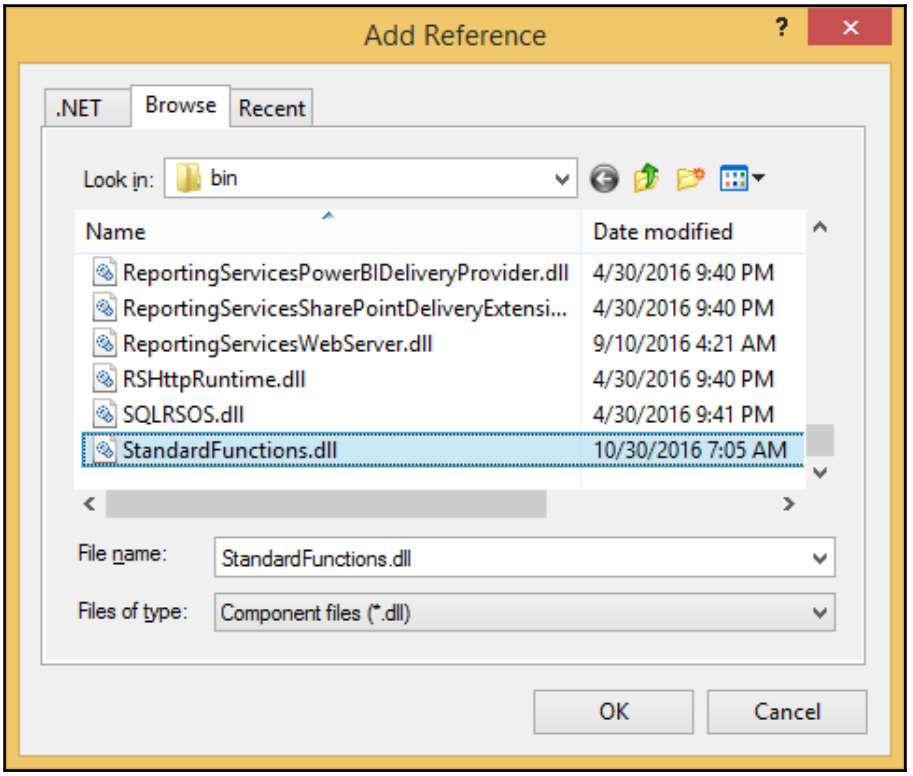


Figure 12.12

4. Once added, you should see the assembly in the grid. Click **OK** to save it:

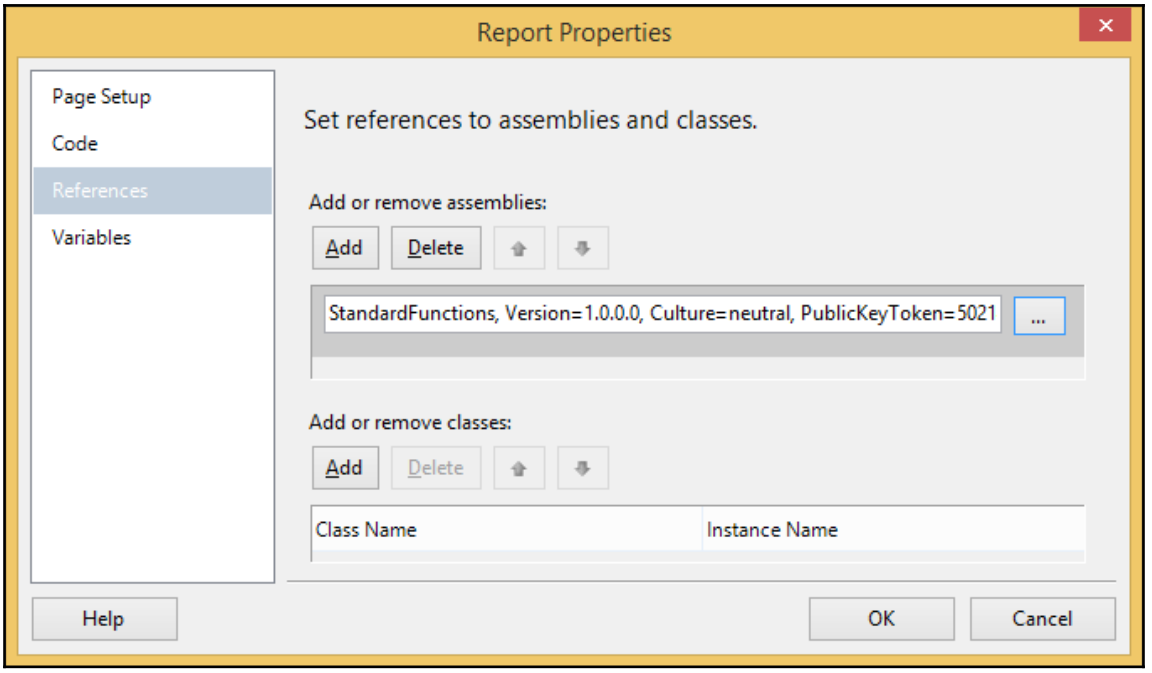


Figure 12.13

Let's use the function in the assembly with the text box we have to display the selected date:

1. Go to the **Text Box Properties** of the text box and open **Font** page. Click on expression icon to set the color through an expression:

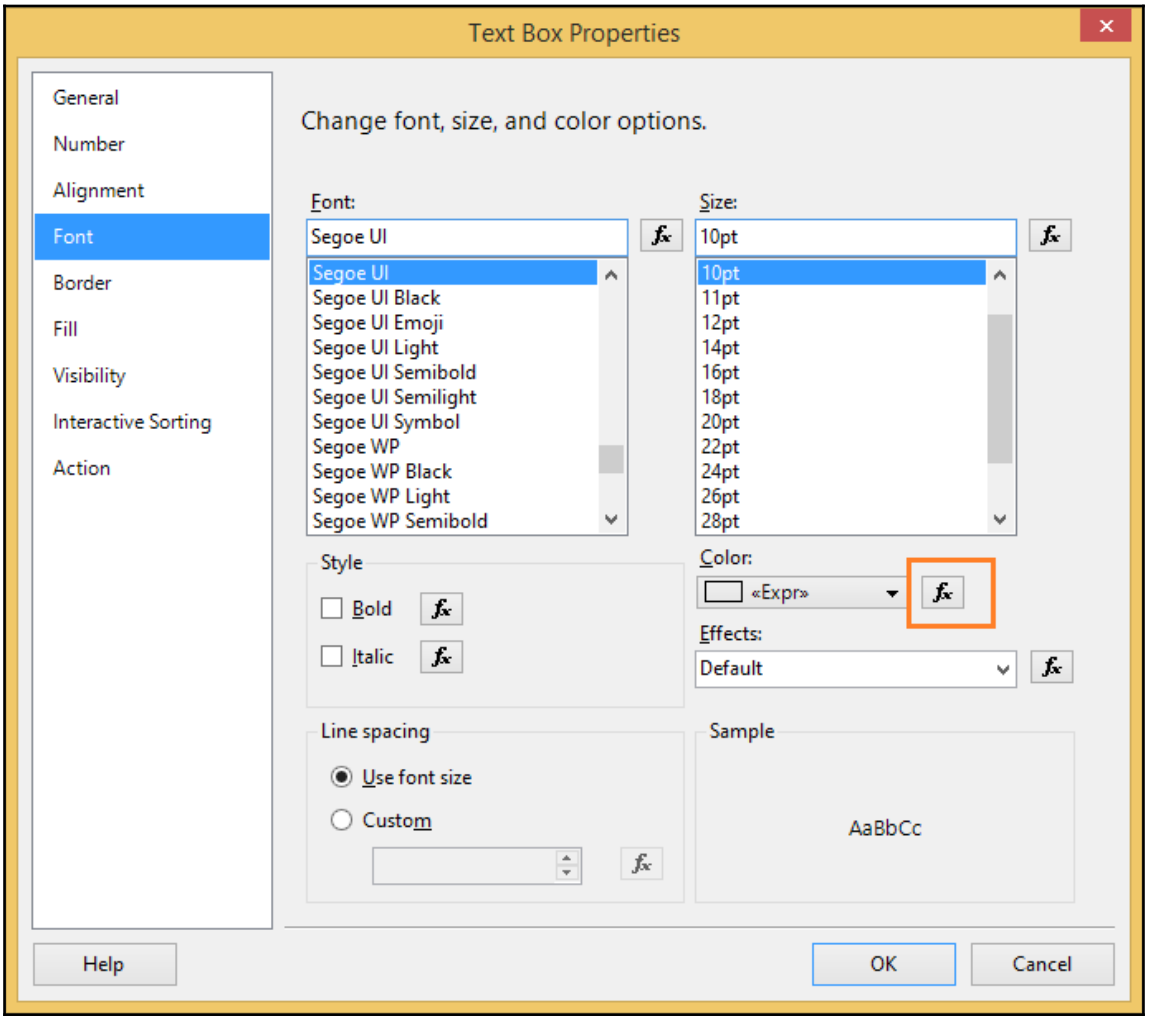


Figure 12.14

2. Set the following expression. Note the way of referencing the method. It is `AssemblyName.ClassName.MethodName`. If the namespace is different, then you need to mention it as well:

```
=Iif(format(Code.GetDate(Parameters!ReportParameterSalesDate
.Value), "yyyy-MM-dd") = "1900-01-01"
, StandardFunctions.Functions.GetTextColor("Error"),
StandardFunctions.Functions.GetTextColor(""))
```

3. Click on **OK** to save the expression and click on **OK** again to save the text box properties:



Remember, even though we could use the assembly during development, Reporting Services requires the assembly in the **Global Assembly Cache** (**GAC**) when running the report. Let's add the assembly to GAC.

4. Open Command Prompt with administrative privileges (as an Administrator) and change the path to following folder (your path may be different based on the version of Visual Studio installed):

```
C:\Program Files (x86)\Microsoft SDKs\Windows\v10.0A\bin\NETFX
4.6.1 Tools
```

   This folder has `gacutil.exe` and can be used for registering the assembly to GAC. Execute the following code to register it with GAC:

```
gacutil.exe /i "<your folder that contains the
dll>\StandardFunctions.dll"
```

5. You should see a similar output with a success message:

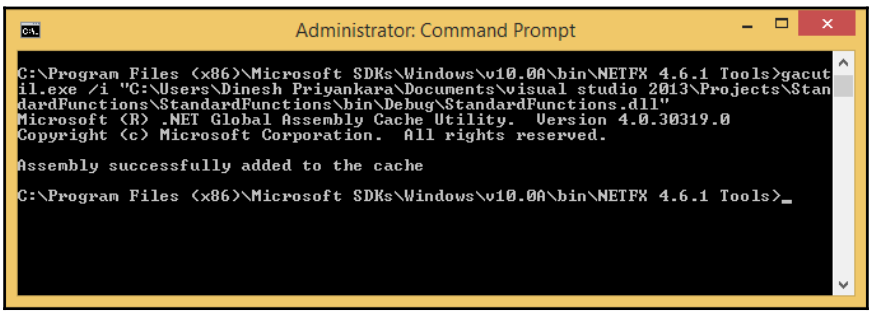


Figure 12.15

Everything required is done. Let's publish the report and see whether it works:

1. Go back to the project and deploy the report.
2. Open the report using Web Portal. If you input an invalid date, the message set with text box should be displayed in red color:

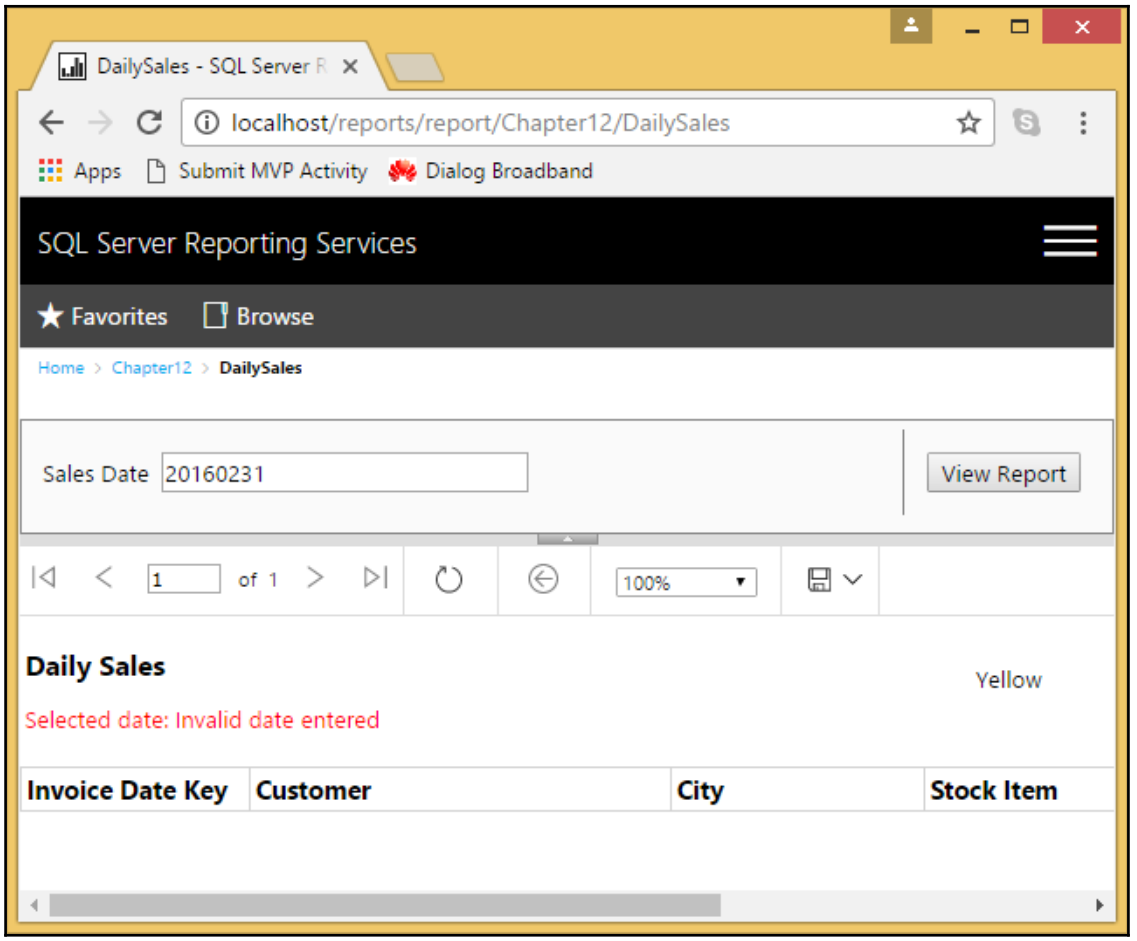


Figure 12.16

How it works…

Remember, if the Reporting Services is installed in a different server, all you need to do is, register the assembly in GAC. If it is not registered, Reporting Services cannot access the assembly as it uses the assembly in the GAC when running the report.

The reference to the assembly is recorded in the `.rdl` file. If you open the `.rdl`, you will see a node that holds details of the assembly. In addition to this node, you will see an additional node if you have added `Class Name` and `Instance Name` with References page in Report Properties. This is not required for the code, we have written the function as a static method:

```
<CodeModules>
    <CodeModule>StandardFunctions, Version=1.0.0.0, Culture=neutral,
PublicKeyToken=5021460a45a99ff0</CodeModule>
</CodeModules>
```

Opening reports using URL access

A user's access reports are published using many techniques. Even though the Web Portal given with Reporting Services is heavily used for viewing reports, some solutions use different techniques for accessing reports. One way of accessing reports in Reporting Services is, URL Access that enables accessing all reports via Reporting Services Web Service.

URL Access allows us to send necessary commands to the Report Server through the URL. There are various commands that can be used for passing instructions. These commands facilitate submitting name of the report, values for parameters, rendering format, and much more. Let us explore the URL Access along with some commands.

Getting ready

The first thing you need to do is, get the Web Service URL configured for the Reporting Services. Let's start it:

1. Open the Reporting Services Configuration Manager and connect with the instance of Reporting Services.

2. Click on **Web Service URL** for opening configurations of the web service. Note the URL under **Report Server Web Service URLs**. It is the URL, you have to use for accessing reports. Click on the URL:

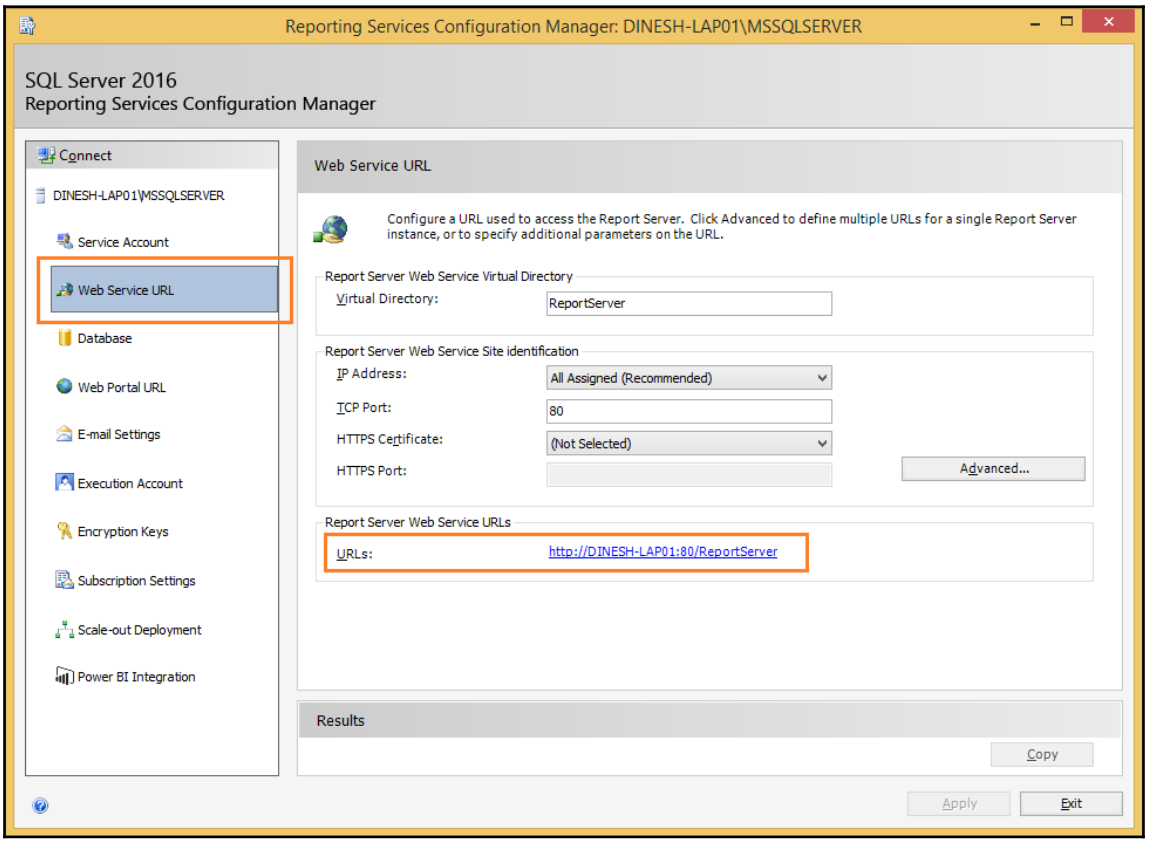


Figure 12.17



You may see multiple URLs if Reporting Services is configured with more than one URL.

3. When you click on the URL, it opens a web page that shows all high-level elements you have in Reporting Services. Click on the `Chapter12` link for seeing all the elements published under `Chapter12`:

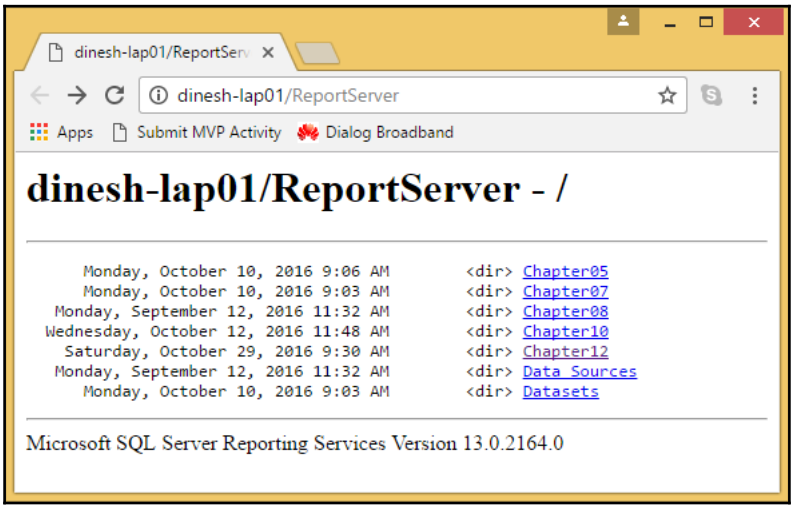


Figure 12.18

4. When the `Chapter12` folder is open, you should see `DailySales` report. Click on it. It opens the report as you have seen with Web Portal but notice the URL:

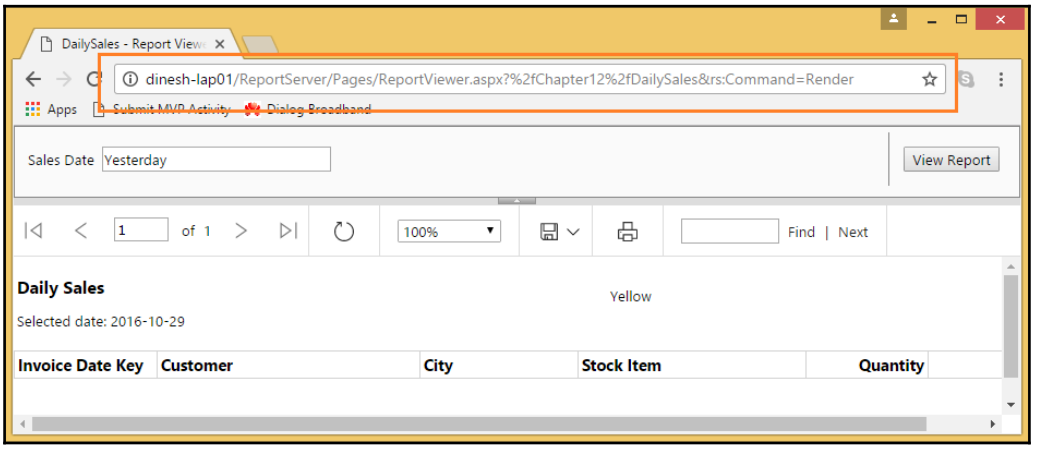


Figure 12.19

As you can see, the URL is different from the URL we have seen with Web Portal:

```
http://dinesh-lap01/ReportServer/Pages/ReportViewer.aspx?%2fChapter12%2fDailySales&rs:Command=Render
```

This is how the URL should be formed if you need to access report using URL Access. Let's see how we can access the DailySales report using different commands.

How to do it…

The syntax for URL Access is:

```
<Reporting Services Web Service URL>?<Report Path>[&prefix:param=value]..n]
```

As you can see, the web service URL, report path must be separated using a question mark (`?`). After the path, multiple commands can be added with the appropriate prefix such as `rc`, `rs` or `rv`.

Let's form a URL for DailySales report and view:

1. Get the Web Service URL and combine it with the DailySales report path. Your URL should be similar to;
   `http://dinesh-lap01/ReportServer?/Chapter12/DailySales`
2. Open a Web Browser and place the URL just formed. You should see the DailySales report opened:

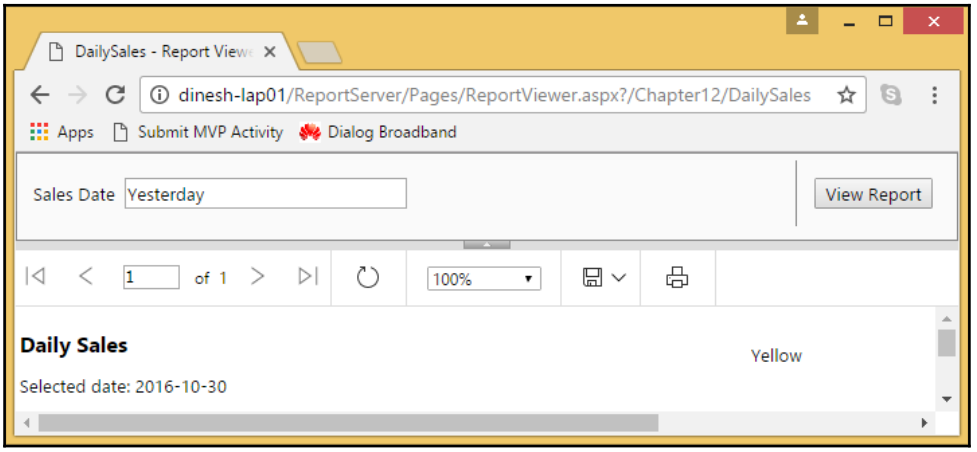


Figure 12.20

Notice the URL. You will see that the `/Pages/ReportViewer.aspx` part is automatically added. Let's check on the prefix: `rs` (Report Server Command). You have already noticed **Render** command. This is one of the Report Server Commands and it renders the report to the browser. Add it as `&rs:Command=Render` to the URL and view the report.

Let's see how we can request for a different rendering format such as PDF:

1. Add `&rs:Format=PDF` to the URL and view the report. The URL should be similar to:
   `http://dinesh-lap01/ReportServer?/Chapter12/DailySales&rs:Format=PDF`:

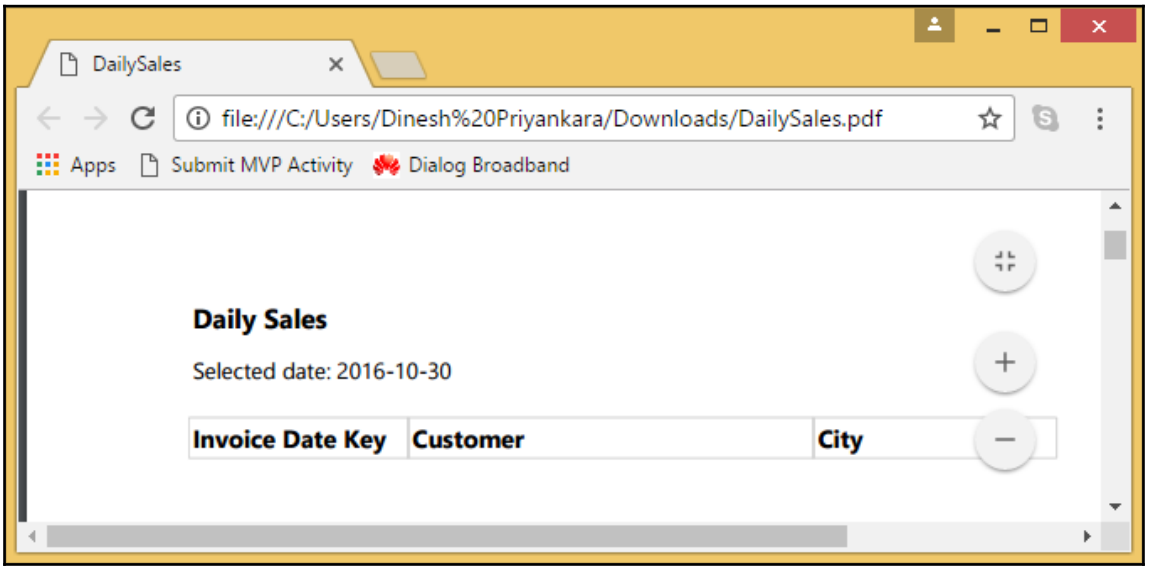


Figure 12.21

Let's check some `rc` (HTML Viewer commands). The Toolbar command allows us set the visibility of the toolbar:

1. Add `&rc:Toolbar=false` to the URL and run it in the browser. Your formed URL should be similar to
   `http://dinesh-lap01/ReportServer?/Chapter12/DailySales&rc:Toolbar=false`

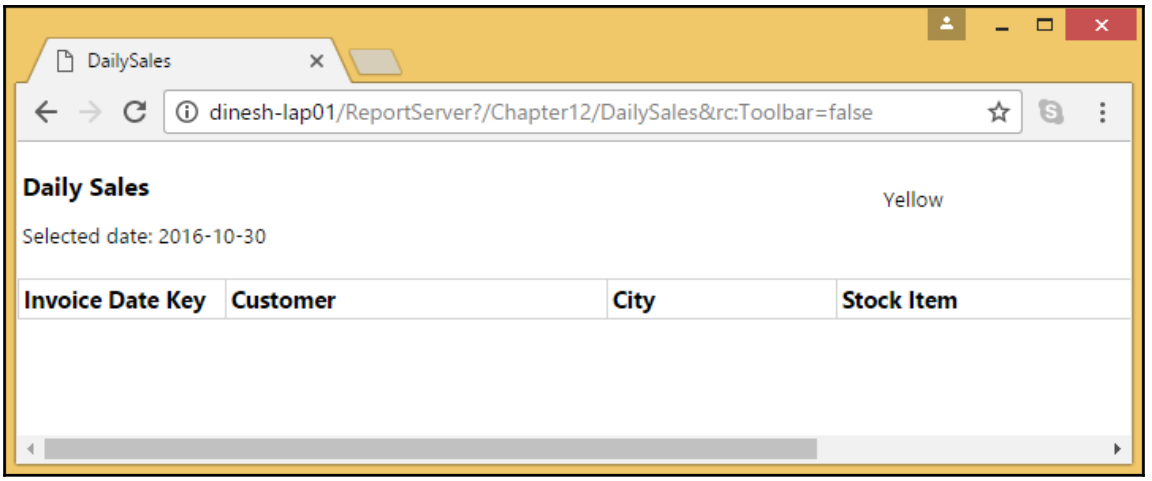


Figure 12.22

2. If you want to hide only the parameter section while showing other items in the toolbar, you can use the `&rc:Parameters` command that accepts `true`, `false`, and `Collapsed`. The `Collapsed` value does not completely hide the parameter pane. It allows users to get it opened when required by toggling the pane.
3. Add `&rc:Parameters=Collapsed` to the URL and run it in the browser. Your URL should be similar to `http://dinesh-lap01/ReportServer?/Chapter12/DailySales&rc:Parameters=Collapsed`.

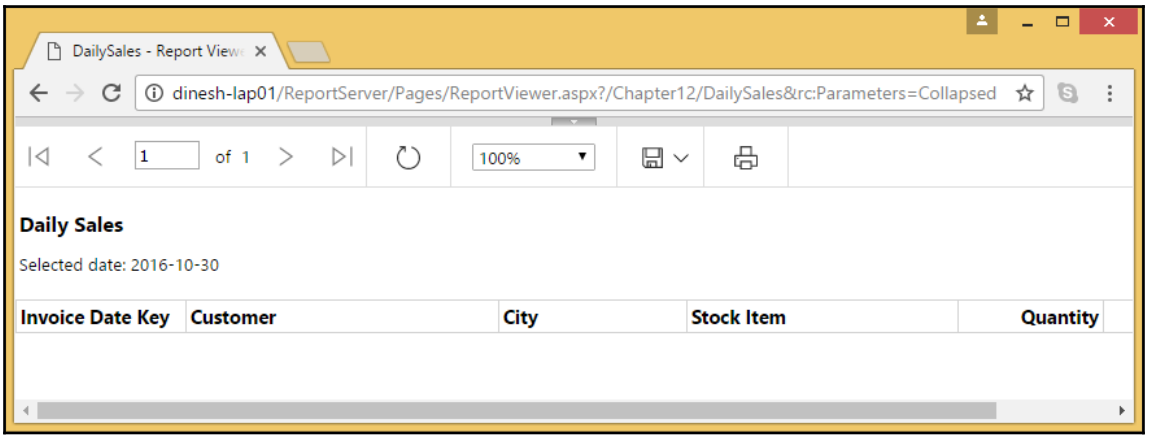


Figure 12.23

4. We did not check how to pass values for parameters. Multiple parameters can be submitted through the URL and the format of it is:

```
&<parametername>=value
```



Note that the parameters are case-sensitive. You need to make sure they have been mentioned in the URL as they have been defined. In addition to that, if the parameter contains space characters, they are replaced with %20. This is automatically done by the browser. Hence you do not need to worry on it. Let's see how we can pass a value to parameters in the DailySales report.

5. Add `&ReportParameterSalesDate=-200` to the URL and run it in the browser. Your formed URL should be similar to `http://dinesh-lap01/ReportServer?/Chapter12/DailySales&ReportParameterSalesDate=-200`:

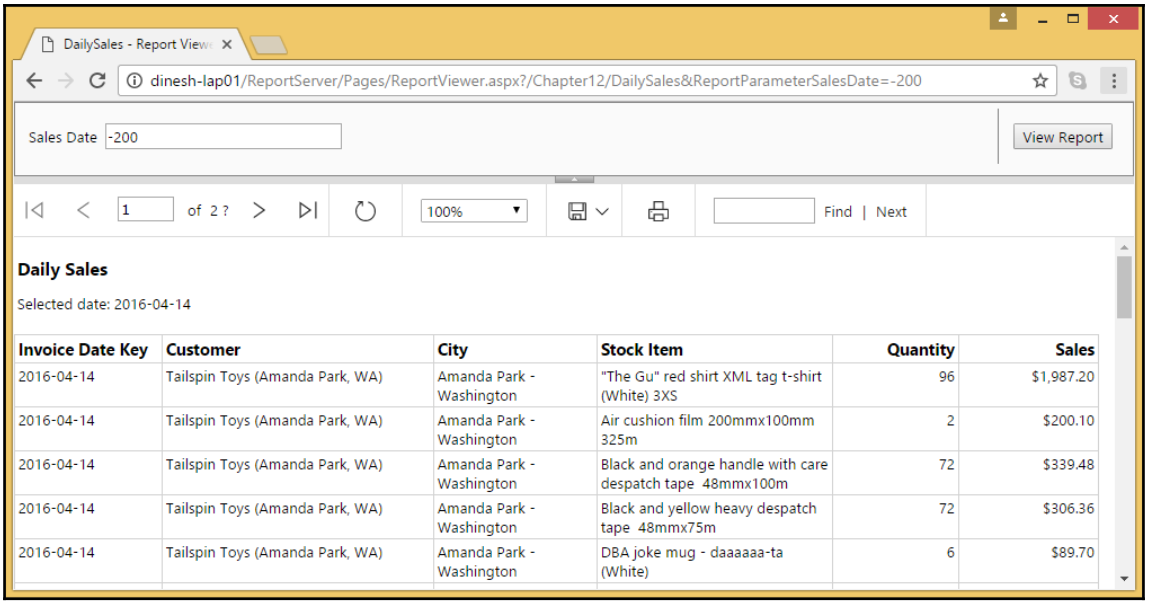


Figure 12.24

6. There are many more commands that can be included the URL. You can read more in the following URLs:
    - URL Access Parameter Reference: `https://msdn.microsoft.com/en-us/library/ms152835.aspx`
    - Pass a Report Parameter Within a URL: `https://msdn.microsoft.com/en-us/library/ms155391.aspx`

How it works…

This is completely based on the web service configured to Reporting Services. When the user sends a request using a URL, it accepts it and satisfies the request as per the command added to the URL.

Generally, URL Access is used with applications and when the application needs to open the report outside the application. For an example, if a .NET application needs to open a report in a browser, report can be launched using URL Access through a .NET class like `System.Diagnostics.Process`.

There's more…

If you need to make sure that the requests are satisfied via a secured channel, you can configure one or more HTTPS Certificates to the web service. This can be done using Reporting Services Configuration Manager.

As you see, this method is good for opening the report in a browser by other applications To open it within the application, refer to the *Using ReportViewer control for accessing reports* recipe.

Using ReportViewer control for accessing reports

It is always better to open functionalities related to reports within the solution rather directing users to different applications or tools. It makes the solution complete and user does not need to worry on other applications required for accessing reports. This recipe speaks about it, let's see how we can integrate Reporting Services reports with .NET applications.

Getting ready

Microsoft Visual Studio provides a control called ReportViewer that can be used for integrating Reporting Services reports with .NET applications. There are two controls: one for Windows applications and another for Web applications. Both work in same fashion and have same functionalities but design is specific to the application type.

The ReportViewer control has property called ProcessingMode that decides the place of processing the report. It has two options: Local and Remote.

Remote is the preferred option because it makes sure that processing is done at the Reporting Services server without burdening resources in application server. Not only that, if the Report Server is configured with multiple servers, requests can be distributed improving the overall performance of the solution.

However, if the Reporting Services is not available, Local mode is preferred as an alternative way though it provides less functionalities when compared to Reporting Services.

Let's create a Visual Studio project and see how we can integrate Reporting Services reports with it.

How to do it…

Here are the steps for integrating reports with the .NET Windows application:

1. Open Visual Studio 2013 or 2015 and create a new Visual C# Windows Forms Application:

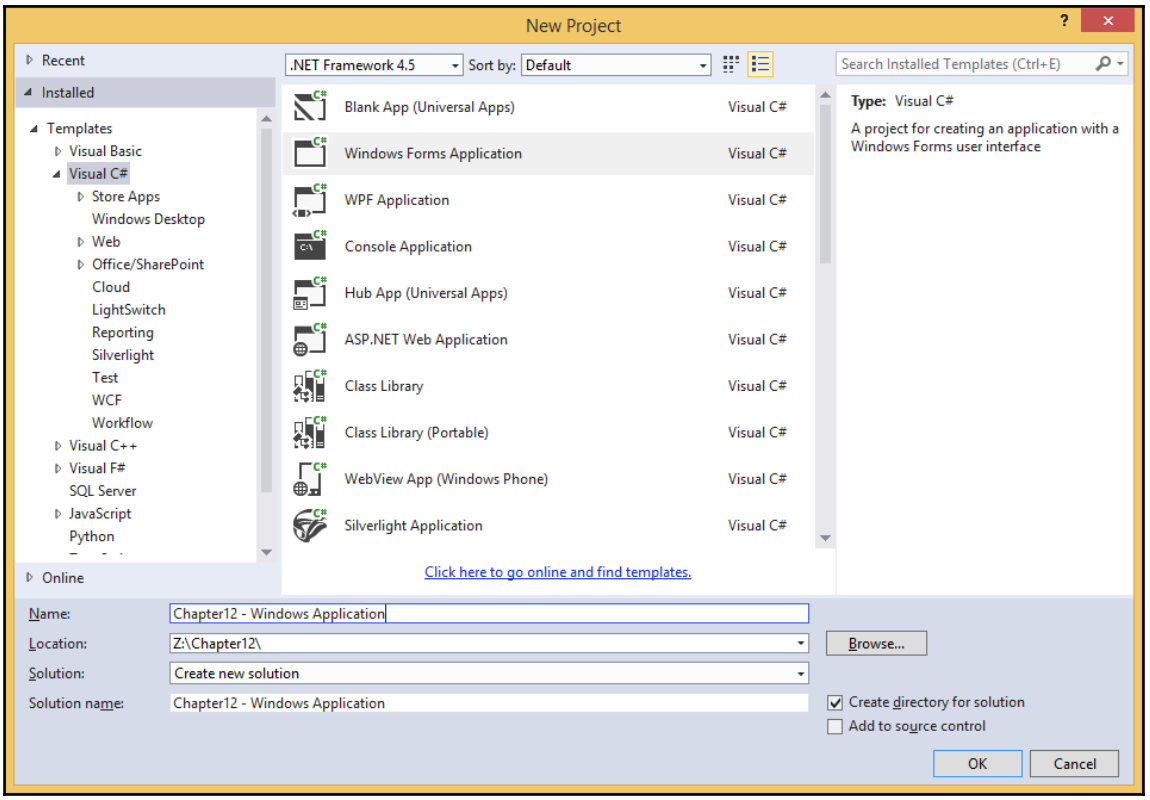


Figure 12.25

2. Visual Studio creates the default Windows Form. Rename it as `MainForm`.

3. Open the `MainForm`. Open the **Toolbox** and navigate to the **Reporting** section. Expand **Reporting** and double click on **ReportViewer:**

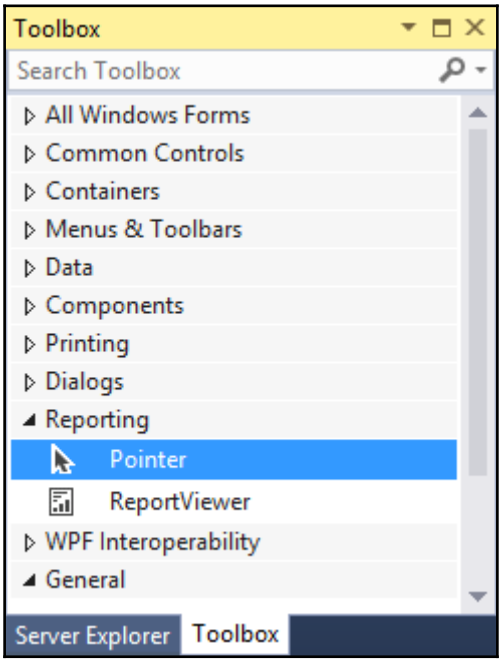


Figure 12.26

4. Check the control in your Windows form. Locate a small right-arrow-icon on the top-right corner of the control and click on it. You should see the **ReportViewer Tasks** pane:

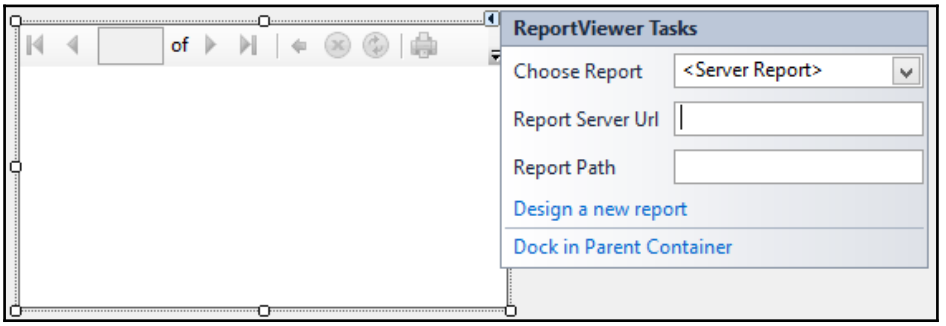


Figure 12.27

5. Enter the Report Server URL as `http://localhost/reportserver`. If the server name is different or it is a named instance, then replace the localhost with it.
6. Enter the report path as `/Chapter12/DailySales` for configuring the control to the DailySales report.
7. Open the Properties of ReportViewer and make sure **ProcessingMode** is set to **Remote**.
8. Run the application. You should see that report is opened within the application:

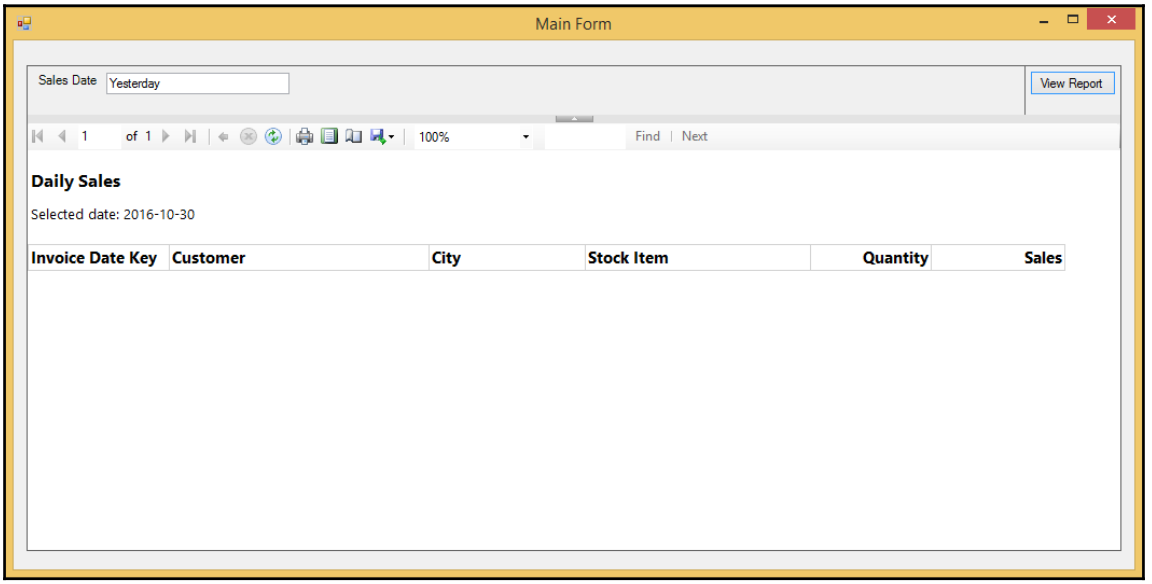


Figure 12.28

9. The advantage of this integration is, we can control some of the elements and processing programmatically rather giving the full responsibility to Reporting Services. For an example, we can change the way of displaying parameters and have additional validations by adding our own input boxes to accept inputs of users on parameters.

10. Let's make the default toolbar and parameter pane invisible. Click on the ReportViewer control and get the properties of it. Make both **ShowParameterPrompts** and **ShowToolbar** as **False**:

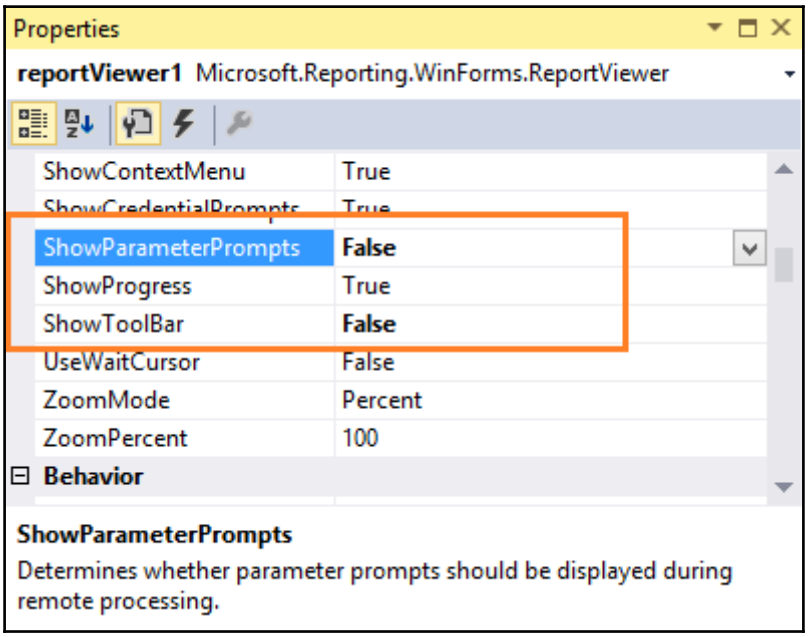


Figure 12.29

11. Next add a label, a text box, and a button to the Windows form. Name the text box as `textBoxSalesDate` the and button as `buttonViewReport`. Set the text property of the label as **Sales Date** and the text property of the button as **View Report**:

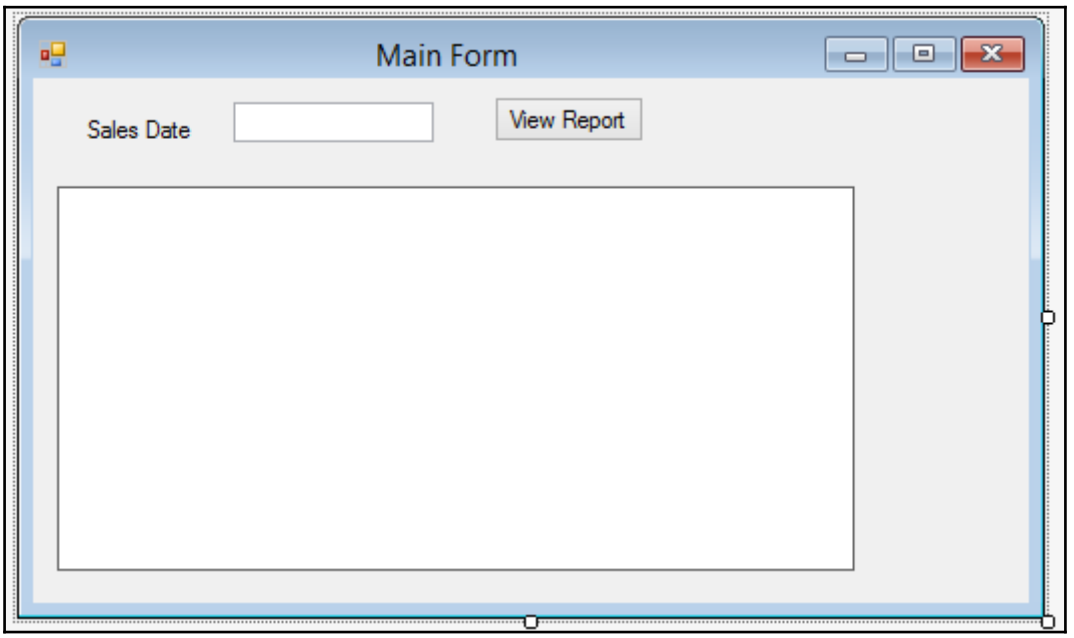


Figure 12.30

12. Double-click on the **View Report** button. This opens the code behind of the form and creates a method for handling the button-click event. Add the following code to button-click event and save it:

```
private void buttonViewReport_Click(object sender, EventArgs e)
{
 reportViewer1.ServerReport.SetParameters(new
 Microsoft.Reporting.WinForms.ReportParameter
("ReportParameterSalesDate", textBoxSalesDate.Text));
    reportViewer1.RefreshReport();
}
```

13. Run the application. Enter `-200` as the input value and hit the **View Report** button. You should see the output as shown in the following screenshot:

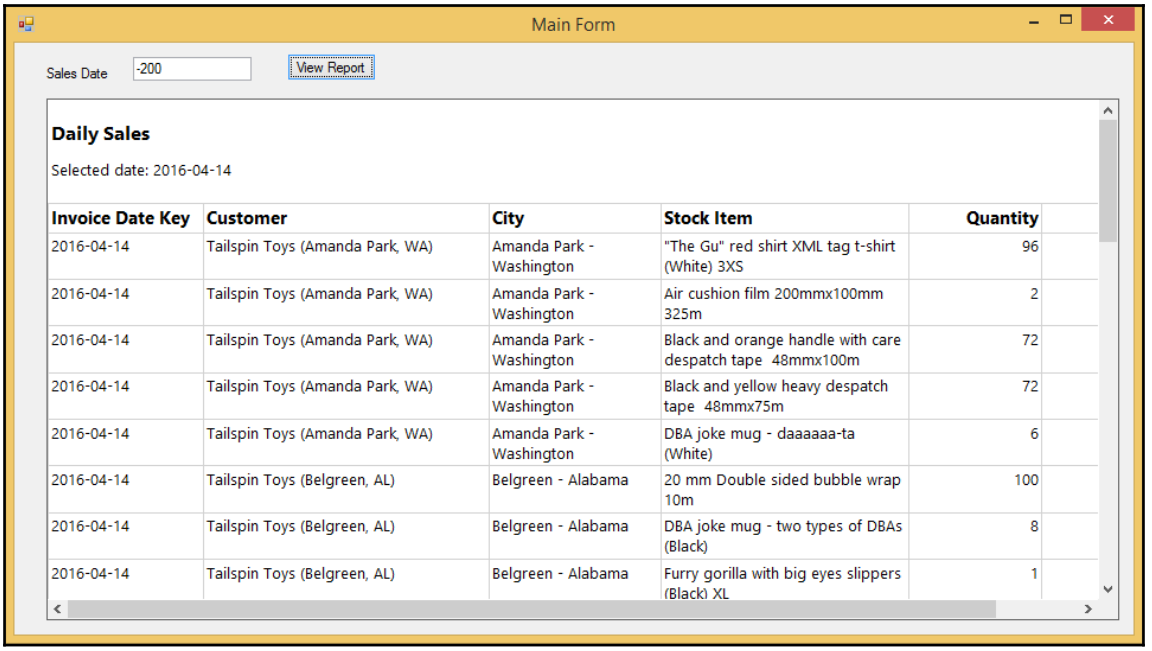


Figure 12.31

How it works…

If the Processing mode is set as Remote, the request is sent to Reporting Services and processing is done by the Reporting Services, including the rendering. The ReportViewer control is showing only the output of the processing.

Since you can do a lot with .NET coding, you may take the entire validation to the .NET application. In addition to that, you can use .NET controls for handling report parameters in a better way.

There's more…

Remember, this control is available in .NET Web solutions as well. The implementation is the same and reports can be viewed as a part of the web solution.

Index

.

.NET assemblies
  external .NET assemblies, referencing 542, 544, 546, 549, 550, 552

A

Always On Availability Group
  reference link 378
Always On Failover Cluster 378
Analysis Services
  about 385
  additive measure 392
  non-additive measure 392
  semi-additive measure 392
Analytical Reports 374
Analytics Platform System (APS) 11
authentication types
  Custom 504
  RSWindowsBasic 504
  RSWindowsKerberos 504
  RSWindowsNegotiate 504
  RSWindowsNTLM 504
Azure portal
  URL 403
Azure Storage Explorer
  URL 408

B

BI solution
  with Reporting Services 376, 377, 378
Business Intelligence 11
Business Intelligence Development Studio (BIDS) 20

C

cache
  enabling, for reports 322, 323, 324, 325, 326, 327, 328, 329, 330
  refreshing 330, 331, 333, 334, 335, 336, 337
Caller Data Record (CDR) 402
cascading parameters
  reports, creating with 162
certificates
  used, for securing channel 523, 524, 525, 526, 527, 528, 530
chart data region
  reports, creating with 99, 100, 101, 102, 103, 104, 106, 107, 108, 109, 111, 112, 113, 114, 115, 116, 117, 118
configured data sources
  assigning, to published reports 462, 463, 464, 465, 466, 467, 468, 469, 470

D

Dashboard Reports 374
dashboard
  about 257
  creating 434, 435, 437, 442, 444
Data Alerts
  configuring 451, 452, 454
data sources
  managing 456, 457, 458, 459, 460, 462
  security, handling on shared data sources 520, 522, 523
  setting up 267, 268, 269, 270, 271
data visualization
  with modern styles 222, 223, 224, 225
  with Sunburst data region 236, 237, 239, 240, 241, 242, 243, 244, 245
  with TreeMap data region 225, 226, 227, 228,

230, 231, 232, 233, 234, 235
Data Warehouse (DW) 197
datasets
using, for mobile report 271, 272, 273, 274, 275, 276, 277, 278, 279, 280
Distributed Architecture 376
drill-through functionality 349
drill-through reports
parameters, configuring 295, 296, 297, 298, 299, 300, 301
dynamic datasets
reports, creating with 164, 166, 167, 168, 169, 170

E

email
configuring, with SharePoint 444, 446
embedded codes
adding 532, 534, 536, 538, 539, 541
encryption keys
backing up 491

F

Formal Reports 374

H

Hadoop 401
HBase 401
HDInsight cluster
connecting to 401, 403, 404, 405, 406, 407, 408, 411, 412, 413, 414, 415, 416, 417, 418
unstructured data, obtaining 401, 403, 404, 405, 406, 407, 408, 409, 411, 412, 413, 414, 415, 416, 417, 418
High Availability and Disaster Recovery 45, 54
Hive 401
Hive drivers
URL 409
HTML Renderer 318

I

Integrated Development Environment (IDE) 62
Intermediate Format 456
Internet Information Services (IIS) 35, 524

K

Key Performance Indicator (KPI)
creating, with predefined values 338, 339, 340, 341, 342
creating, with shared datasets 342, 343, 344, 345, 346, 347, 348, 349, 350

M

map data region
reports, creating with 151, 152, 153, 154, 155, 157, 158, 159, 160, 161, 162
Mashup Reports 374
matrix data region
reports, creating with 94, 95, 97, 98, 99
Mobile Report Publisher
about 258
installing 258
launching 259, 260, 261
mobile report
creating 262, 263, 264
datasets, using 271, 272, 273, 274, 275, 276, 277, 278, 279, 280
graphical elements, updating 280, 281, 282, 283, 284, 285
layout, creating 285, 286, 287, 288, 289, 290
modern styles
for data visualization 222, 223, 224, 225
Multidimensional model
aggregated values, obtaining 392, 393, 395, 396
for formatting 386, 387, 388, 389, 390, 391, 392
used, for creating reports 378, 379, 381, 382, 383, 384, 385
multiple axis charts
reports, creating with 129, 131, 132, 133, 134, 135, 136
multiple data regions
reports, creating with 136, 137, 138, 139, 140, 141, 142, 143, 144, 145

N

navigators
configuring 290, 291, 292, 293, 294, 295

nested data regions
reports, creating with 145, 146, 147, 148, 150
Network Load Balancing (NLB) 378

P

Paginated Reports 374
Parallel Data Warehouse (PDW) 11
parameters
configuring 118, 119, 120, 121, 122, 123, 124, 125, 126, 127
managing 118, 119, 120, 121, 122, 123, 124, 125, 126, 127
positioning 218, 219, 220, 221, 222
PDF rendering extension
reports, printing 356, 358, 359, 360, 361, 362
Pig 401
Pinning to Power BI 362
Portable Document Format (PDF) 356
post-installation tasks
executing 53, 54
Processing Mode
Local mode 560
Remote mode 560
Production Reports 374
published data sources
creating 197, 198, 199, 200
published datasets
creating 201, 203, 204, 205
published reports
configured data sources, assigning 462, 463, 464, 465, 466, 467, 468, 469, 470
parts, creating 208, 210, 211, 212, 213

R

R Graphics Device
about 255
URL 255
Recovery Model 488
Recovery Time Objective (RTO) 487
Report Builder
about 183
connecting, to Reporting Services 187, 188
enhancements 185, 186
installing 183
using 183, 184
report embedding 362
Report Manager 302
Report Portal System Administrator
authorizing 505, 507
Report Server Scale-Out
reference link 378
Report Services Subscriptions 370
Reporting Services Configuration Manager
about 456
accessing 23, 24, 25, 26, 27
Reporting Services Native Mode 304
Reporting Services
backing up 487, 488, 490, 491, 492, 493, 494, 495
database, configuring 44, 45, 46, 47, 48, 49, 50, 51, 52, 53
edition, selecting 11, 12, 13
existing roles, editing 513, 514, 515
history 9, 10
installing, with default configurations 13, 14, 15, 16, 17, 18, 19
installing, with related tools 20, 21, 22
monitoring 495, 496, 497, 498, 500, 501
published reports, modifying 213, 215
R, integrating with 246, 247, 248, 249, 250, 251, 252, 253, 254, 255
reports, publishing 205, 206, 207
roles, creating 511, 513
service account, configuring 28, 30, 31, 32, 33, 34
URLs, configuring 34, 35, 36, 37, 38, 40, 41, 42, 43, 44
users, authenticating 503, 505
reports
caching 470, 471, 472, 473, 474, 475, 477, 478, 479, 480
creating, with cascading parameters 162
creating, with chart data region 99, 100, 101, 102, 103, 104, 106, 107, 108, 109, 110, 111, 112, 113, 114, 115, 116, 117, 118
creating, with dynamic datasets 164, 166, 167, 168, 169, 170
creating, with map data region 151, 152, 153, 154, 155, 156, 157, 158, 159, 160, 161, 162
creating, with matrix data region 94, 95, 96, 97,

98, 99
creating, with multiple axis charts 129, 131, 132, 133, 134, 135, 136
creating, with multiple data regions 136, 137, 138, 139, 140, 141, 142, 143, 144, 145
creating, with nested data regions 145, 146, 147, 148, 149, 150
creating, with table data region 64, 66, 67, 68, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 82, 83, 84, 85, 86, 87, 88, 89, 90, 92
creating, with wizard 188, 190, 191, 192, 193, 194
designing, manually 194, 195, 196, 197
embedding, in applications 362, 363, 364, 365, 366, 367, 368, 369, 370, 371, 372
publishing 170, 171, 172, 173, 174, 175, 177, 178
ReportServer database 44
ReportServerTempDB database 44
ReportViewer control
about 560
used, for accessing reports 559, 561, 564, 566

S

security
handling, on shared data sources 520, 521, 522, 523
Server Analysis Services 64
service account
configuring 28, 30, 31, 32, 33, 34
domain user account 30
local service account 30
local system account 30
local user account 30
managed service account 30
network service account 30
virtual service account 30
shared data sources 456
SharePoint
configuring 420, 424, 425, 426
email, configuring 444, 446
permissions, adding to document library 428, 429
permissions, altering to specific report item 430
security, handling 426, 433, 434
user's permissions, removing from report 432
user, adding 427
simulated data
using 264, 265, 266, 267
Single Server Architecture 376
snapshots
creating, of reports 481, 482, 484, 485, 486, 487
SQL Server 2016
edition, selecting 11
SQL Server Agent 490
SQL Server Configuration Manager 456
SQL Server Data Tools (SSDT)
about 20, 55, 493
exploring 56, 57
launching 57, 58, 59, 60, 61, 62
URL, for downloading 57
Visual Studio 2015 support 62, 63, 64
SQL Server Management Studio (SSMS) 20, 69
SQL Server Mobile Report Publisher 23
SQL Server Reporting Services (SSRS) 151, 375
SSDTBI 20
SSL
references 530
SSRS roles
users, authorizing for report access 508, 511
subscriptions
configuring 446, 448, 450, 451
Sunburst data region
for data visualization 236, 237, 238, 239, 240, 241, 242, 243, 244, 245
System Configuration Checker 13

T

table data region
reports, creating with 64, 66, 67, 68, 70, 71, 72, 73, 74, 75, 77, 78, 79, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 92
Tabular model
report, creating 396, 397, 399, 400, 401
Transact SQL (T-SQL) 375
TreeMap data region
for data visualization 225, 226, 227, 228, 229, 230, 231, 232, 233, 234, 235

U

Universal Access Control (UAC) 305
URL access
  references 559
  used, for opening reports 552, 554, 555, 556, 558, 559
users
  authenticating, for Reporting Services 503, 505
  authorizing, for report access via SSRS roles 508, 511
  data, filtering for current user 515, 516, 517, 518, 519

V

Visual Studio 2015
  support, for SSDT 62, 63, 64

W

web browsers
  reports, viewing 318, 319, 320, 321
Web Portal
  about 35, 302, 303
  customizing 350, 351, 353, 354, 355, 356
  exploring 304, 305, 306, 307, 308, 309, 310, 311, 312
  interface 356
  reports, managing 312, 313, 315, 316, 317
Web Service application 35
wizard
  reports, creating 188, 189, 190, 191, 192, 193, 194

X

XML code
  viewing, related to reports 178, 179, 180, 181